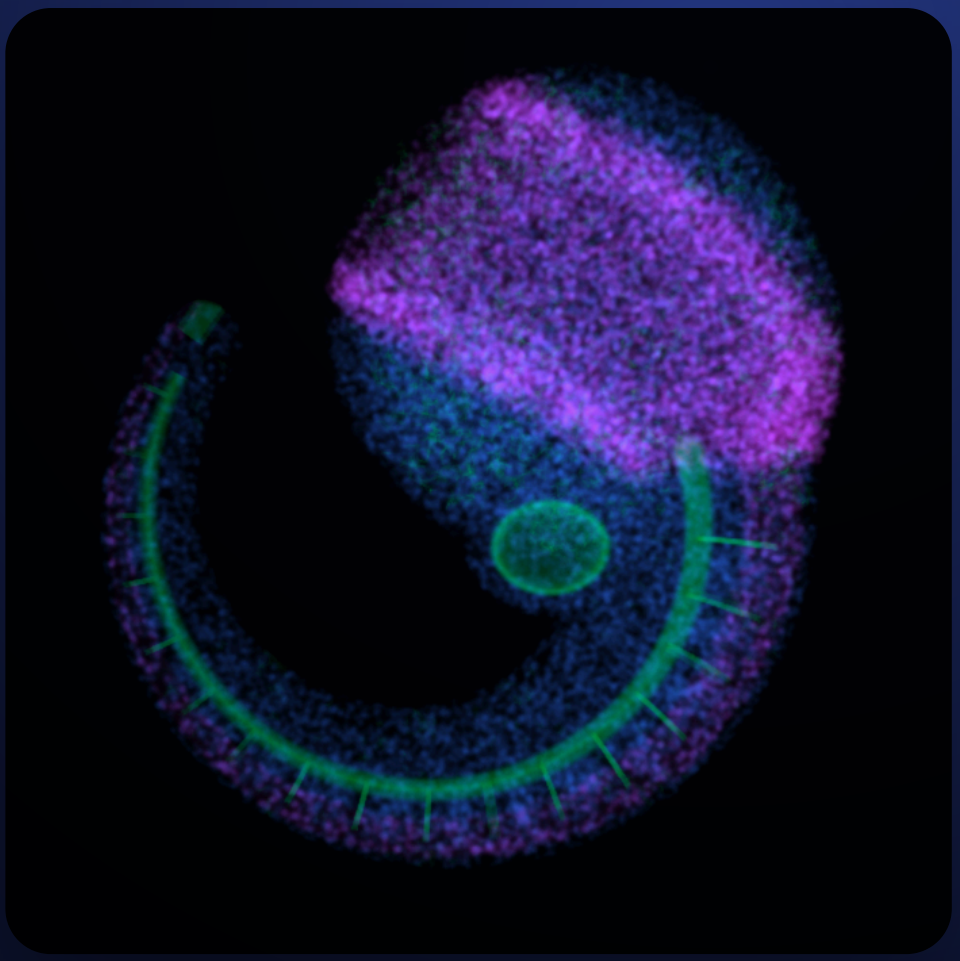


IRIBHM · ULB — LUMEN3D

# Lumen3D from A to Z

## The complete documentation of the platform

How an Imaris file of several gigabytes becomes an embryo you can spin around in a browser — and everything around that journey: the languages and pages, image cleaning, compression, bricks, 3D rendering, streaming, histograms, every tool and how it works inside, plugins, customisation and languages, data formats, import, hosting, updates and security. 21 chapters written for biologists, illustrated on every page.

Web platform 1.65.0 · Pipeline 0.22.0 · Data format 5

October 2026

# Contents


## How to read this document

- Four ways to read it
- The boxes
- The conventions

## 1. The platform at a glance

- 1.1 What is Lumen3D, and who is it for?
- 1.2 The problem: a file too big for a browser
- 1.3 The three dataset types
- 1.4 The journey of an acquisition
- 1.5 Key figures
- 1.6 Map of the documentation

## 2. The technologies: who does what

- 2.1 A web page is a human body
- 2.2 The client: what runs in your browser
- 2.3 GLSL: the tiny program run millions of times
- 2.4 The server: a counter, not a brain
- 2.5 Python: the laboratory workbench
- 2.6 File formats: containers
- 2.7 Why everything is “self-hosted”
- 2.8 What the browser needs
- 2.9 The project folder

## 3. A tour of the pages

- 3.1 The map of the pages
- 3.2 The Home page
- 3.3 The Data Explorer
- 3.4 The 3D viewer
- 3.5 The Live viewer: the timeline
- 3.6 The 2D page
- 3.7 Compare
- 3.8 About and Legal notice
- 3.9 The administration: an overview
- 3.10 The addresses worth knowing

## 4. From the microscope to the Imaris file

- 4.1 From pixel to voxel
- 4.2 A stack of slices, several channels
- 4.3 How many grey levels? 8 or 16 bits
- 4.4 An .ims is a “hard disk inside a file”
- 4.5 Imaris’s pyramid: ignored
- 4.6 Step 1: reading the metadata
- 4.7 Calibration: the voxel size
- 4.8 The clock of a time series

## 5. Cleaning the image: removing the background noise

- 5.1 Why clean?
- 5.2 The idea on one page
- 5.3 A complete example, voxel by voxel
- 5.4 Stage A: the floor ( `bg_floor` )
- 5.5 Stage B: the ceiling ( `sig_max` )
- 5.6 Stage C: the mask, or “where is the embryo?”
- 5.7 Stage D: the median, outside the mask only
- 5.8 Stage E: the window, from 16 to 8 bits
- 5.9 On real images
- 5.10 Minimum size of an object
- 5.11 Before and after
- 5.12 What was tried, then abandoned
- 5.13 Time series: a single window
- 5.14 Large volumes: in tiles, without changing the result
- 5.15 What this means for you

## 6. Shrinking the size without losing anything important

- 6.1 The size budget in numbers
- 6.2 From 16 bits to 8 bits: dividing by two
- 6.3 The resolution pyramid
- 6.4 Compressing without losing anything
- 6.5 Not storing emptiness
- 6.6 Final size: from the .ims to the published folder

## 7. Cutting into bricks

- 7.1 Why cut a volume?
- 7.2 A brick: 64 useful voxels and a border
- 7.3 From a cube to an image: the mosaic
- 7.4 The brick grid of each level
- 7.5 Packs: boxes of 64 bricks
- 7.6 The index and the manifest
- 7.7 The tree of a published dataset
- 7.8 `planes/` : an XY slice without reading 64 planes
- 7.9 `mips/` : the maximum of a layer of 64 planes
- 7.10 Data formats 1 to 5
- 7.11 Format 5: a resolution × compression matrix

## 8. The end of the pipeline: metadata, cell tracking, publication

- 8.1 The thumbnail
- 8.2 The histograms
- 8.3 `metadata.json` : the dataset’s identity card
- 8.4 Cell tracking (time series)
- 8.5 Publishing “all or nothing”
- 8.6 What the pipeline keeps when you reprocess a dataset
- 8.7 The `download/` folder
- 8.8 2D photographs, in half a page
- 8.9 Launching all this: `RUN.bat`

## 9. The 3D viewer: Three.js and ray marching

9.1 Two brains: the processor and the graphics card
9.2 WebGL2 and Three.js: who does what
9.3 Ray marching, step by step
9.4 Random grain against banding
9.5 Skipping the empty space
9.6 The three render modes
9.7 Smoothing between voxels
9.8 Smooth when you move, sharp when you stop
9.9 The safeguard against graphics-card crashes
9.10 True proportions: calibration and Z stretch

## 10. Displaying gigabytes: streaming

10.1 The loading sequence
10.2 The quality menu: 512, 1024, Native
10.3 The GPU atlas and the page table
10.4 The video memory budget
10.5 Downloading and decoding without blocking the page
10.6 Packs: an ordered download
10.7 Time series
10.8 The “Zoom detail” mode
10.9 When the graphics card crashes

## 11. Colours, contrast and histograms

11.1 The processing chain of a channel
11.2 The hidden step: the background floor
11.3 The min / max window and gamma
11.4 A channel’s panel
11.5 Histograms
11.6 The quick-setting buttons
11.7 Colours, opacity, exposure, solo
11.8 The Gaussian blur
11.9 A safeguard: what the grey levels mean

## 12. The tools, one by one

12.1 The toolbar at a glance
12.2 Measuring a distance
12.3 The oblique slice
12.4 The Z-stack browser
12.5 Orientation axes, the grid and the displays
12.6 Capture, export, presentation, download, channels, chunks
12.7 The Studio: annotating and exporting a figure
12.8 The Compare page
12.9 The timeline and cell tracking
12.10 The 2D page: photographs
12.11 Keyboard shortcuts and sharing a view

## 13. The tools under the bonnet

13.1 Cell tracking in the viewer
13.2 Stabilisation: moving the gaze
13.3 The Studio in depth
13.4 Compare under the bonnet
13.5 The 2D page in depth
13.6 The timeline and live series
13.7 What the platform remembers: link, workspace, browser
13.8 Measurement and pose algorithms
13.9 Orientation in practice, and the sample side
13.10 Gallery, linked datasets, thumbnail

## 14. The administration panel in brief

14.1 What the panel is for, and how to reach it
14.2 The 15 tabs, in 4 groups
14.3 Putting a dataset online
14.4 Editing a dataset
14.5 Data updates
14.6 Plugins and the signed catalogue
14.7 Updating the platform
14.8 Customising the site (white label)

## 15. Plugins, trust and the signed catalogue

15.1 What is a plugin?
15.2 The identity card: `plugin.json` , field by field
15.3 How the platform finds plugins
15.4 Loading: seven barriers
15.5 Who loads where? `dataTypes` and `contexts`
15.6 A plugin’s life cycle
15.7 What a plugin receives: the `ctx` object
15.8 The toolbar is built, not written
15.9 Plugins and the workspace, exports, embedded pages
15.10 The 28 plugins of the catalogue
15.11 Writing a plugin
15.12 Why be wary of a plugin
15.13 The fingerprint: a signature of the exact content
15.14 Operator approval, and the sandbox
15.15 Compatibility: `platformCompat`
15.16 Updating the platform: reversible erasure
15.17 The signed catalogue: the official store
15.18 Installing, updating, uninstalling
15.19 Publishing a plugin (publisher side)
15.20 The core modules that make all this work
15.21 A plugin does not show up: what to check?

## 16. Customising and translating

16.1 One platform, many homes
16.2 `instance.json` , field by field
16.3 How a value reaches the screen
16.4 The customisation screens
16.5 The theme: colours, font, corners
16.6 The page builder
16.7 The legal notice
16.8 Where does a channel's starting colour come from?
16.9 Translating: how the site speaks four languages
16.10 Accessibility, theme and mobile
16.11 Usage statistics
16.12 `PerfTelemetry` : the viewer's logbook
16.13 Summary

## 17. Data in depth: files, formats, migrations, import

17.1 One dataset, one folder: the three trees
17.2 Who writes what: three authors
17.3 `metadata.json` , field by field
17.4 Manifests and description files
17.5 The catalogue and the type vocabulary
17.6 `download/` and `gallery/`
17.7 Formats 1 to 5
17.8 Four migrations, one registry
17.9 Work units and the journal
17.10 Two executors
17.11 `finalize` : the swap and the version change
17.12 The network governor and slow hosts
17.13 The course of an import
17.14 The allowlist
17.15 Tiers: why a dataset is editable before the end
17.16 Blocks and the import journal
17.17 Import states and the purge
17.18 Validate, publish, replace

## 18. Hosting, updating and releasing a version

18.1 Three ways to serve Lumen3D
18.2 The path of a request
18.3 HTTP caching: never re-download for nothing
18.4 Compression and byte ranges
18.5 Closed folders and guard files
18.6 Behind a proxy, rate limits and the session lock
18.7 Running a server day to day
18.8 Updating the platform: the principle
18.9 The signature: proving a package is authentic
18.10 Updating on a Python server: eight steps
18.11 The switch-over and the way back
18.12 Updating on PHP hosting
18.13 What an update never touches
18.14 First installation: `install.php`
18.15 Releasing a version (for the publisher)
18.16 The test suite: what guards the door
18.17 Working without Internet
18.18 What to do if… ?

## 19. Security and reliability

19.1 The administrator password
19.2 The Content Security Policy (CSP)
19.3 Uploaded files are never served
19.4 Plugins: trust and cage
19.5 Signed versions and catalogue
19.6 Your data stays with you
19.7 Reliability
19.8 What happens if…?
19.9 On screen: bricks, workers and metadata
19.10 The graphics card: full memory and lost context
19.11 Imports, conversions and metadata
19.12 Server: updates, access and hosting limits
19.13 What to remember

## 20. Appendices

20.A Module map
20.B Browser memories and address parameters
20.C What Lumen3D does not do
20.D Key figures of the platform
20.E Versions and history

## 21. Glossary

# How to read this document

This document explains **everything** Lumen3D does, without assuming that you can program. Every page has an illustration; the text stays short.

## Four ways to read it

**🚀 In 20 minutes**
Read the green **"In 30 seconds"** boxes and look at the figures.

**🧬 "My dataset"**
Chapters **4 to 8**: what happens to your Imaris file, step by step.

**🖥️ "The screen"**
Chapters **9 to 12**: how the viewer draws and how the tools work.

**⚙️ "Behind the scenes"**
Chapters **13 to 20**: how the tools work inside, administration, plugins, customisation, formats, hosting, security.

## The boxes

**⏱ IN 30 SECONDS**
The summary of a chapter or section, in three points.

**💡 A PICTURE TO UNDERSTAND**
A comparison with everyday life.

**🔧 FOR THE CURIOUS**
Technical detail (formulas, file names). Skippable.

**🔢 WORKED EXAMPLE**
A hand calculation with real numbers.

**⚠️ WARNING**
A limit to know before interpreting images.

## The conventions

| YOU SEE... | IT MEANS... |
|---|---|
| **Save** | a button or a label exactly as it appears on screen |
| 3 | numbered marker 3 on the screenshot above |
| `metadata.json` | a file name or a technical value; "→ chapter 7" = explained in detail elsewhere |

*Images come from synthetic demonstration embryos, not laboratory data; numbers are read from the code (platform 1.65.0, pipeline 0.22.0). A short version of about 25 pages, “The Essentials”, is in the administration panel.*

# 1. The platform at a glance

- **Lumen3D** is a website for exploring confocal microscopy images that weigh several gigabytes, in an ordinary browser: nothing to install.
- A stack of ~10 GB fits neither in a browser's memory nor in a graphics card's: the whole platform is designed to load only what the screen shows.
- There are **three types** of dataset (3D, Live, 2D) and a single path from the microscope to the screen, which this chapter walks you through.

## 1.1 What is Lumen3D, and who is it for?

**Lumen3D** (*Light-based Unified Microscopy Exploration in 3D*) was born at the IRIBHM (ULB) to look at mouse embryos. It then became a "white-label" platform: name, colours, pages and vocabulary can be changed from the administration panel, without writing a line of code.

Four kinds of people use it, each with a different share of this document:

**The biologist**
Opens an embryo, adjusts colours, measures, slices, compares, builds a figure.
chapters 3, 11, 12

**The technician**
Runs the Python **pipeline** that turns the Imaris file into a web folder.
chapters 4 to 8

**The administrator**
Imports, publishes, customises the site, updates the platform.
chapters 14 to 19

**The public visitor**
Browses the published catalogue and looks, with no account or password.
chapter 3

ℹ️ GOOD TO KNOW

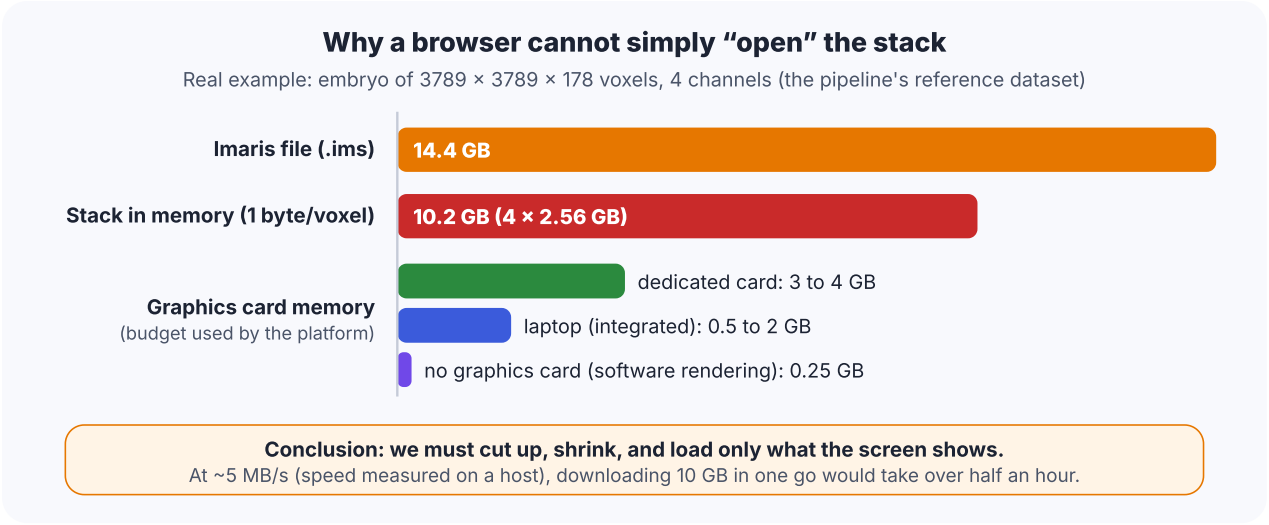
There are **no user accounts**. Only the administration is protected by a password; everything that is published is visible to anyone who has the site's address.

## 1.2 The problem: a file too big for a browser

Take the largest reference dataset of the pipeline: an embryo of **3789 × 3789 × 178 voxels** on **4 channels**. The original Imaris file weighs **14.4 GB**.

🔢 WORKED EXAMPLE

**The calculation.** 3789 × 3789 × 178 = 2,555,460,738 voxels per channel. At 1 byte per voxel: **≈ 2.56 GB per channel**, so **≈ 10.2 GB** for 4 channels. The graphics card, for its part, offers the platform only 0.25 to 4 GB depending on its class.




▲ Even when reduced to 1 byte per voxel, the stack exceeds the memory of most graphics cards.

Three obstacles add up:

- **the network**: at about 5 MB/s, 10 GB takes more than half an hour;
- **the browser’s memory**: it is not built to hold gigabytes of image;
- **the graphics card’s memory** (VRAM): it does the drawing, and it is small.

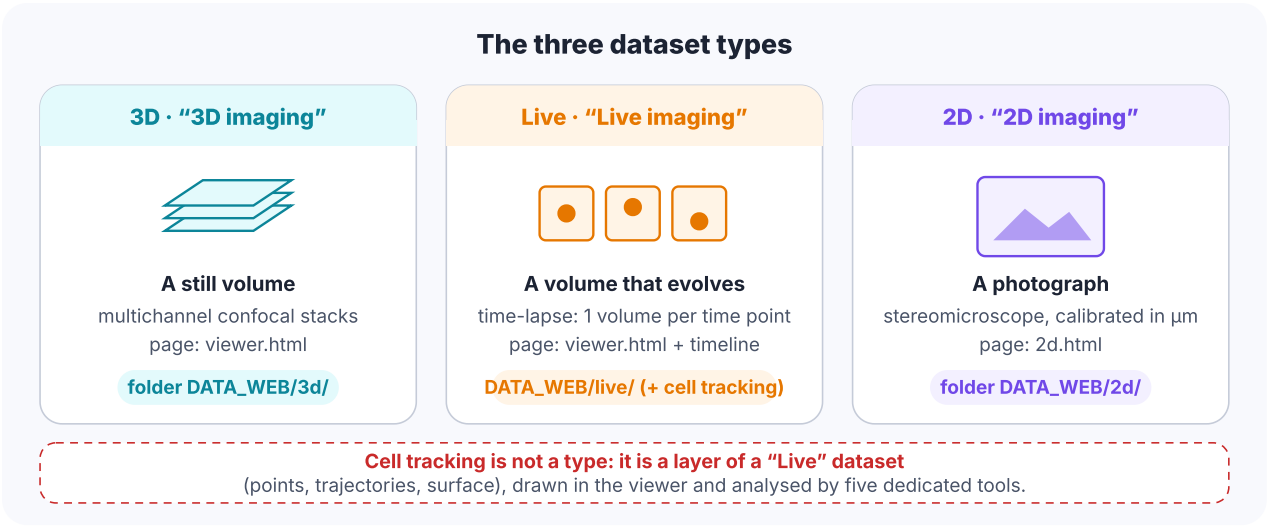
> 💡 **A PICTURE TO UNDERSTAND**
>
> **A 50-volume encyclopaedia.** Nobody lays all 50 volumes open on the desk to read one paragraph. You take the index, then open the right volume at the right page. Lumen3D does the same with the image: an index (the *manifest*) and small pieces (the *bricks*) fetched on demand.

The answer comes down to three ideas, detailed in chapters 5 to 10:

1. **Reduce**: remove the background noise, switch to 8 bits, make smaller versions of the image (the levels of detail).
2. **Cut up**: cube the image into bricks 64 voxels on a side and store them in a few large files.
3. **Stream**: the browser first downloads the coarse version (an instant picture), then refines only the area you are looking at.

## 1.3 The three dataset types

Every published dataset is of one of these three types. The same word is used as the folder name, as the start of the identifier (for example `3d/Embryo-E95-Em2-Pecam1-Sox2`) and as a filter in the Explorer.




▲ Three types, three display pages. The names in quotation marks are those the public reads by default.

> ℹ️ **GOOD TO KNOW**
>
> The displayed names (“3D”, “Live”, “2D”) belong to the administrator: they can be renamed in the **Data types** tab without touching the folders. In this document we use the default names.

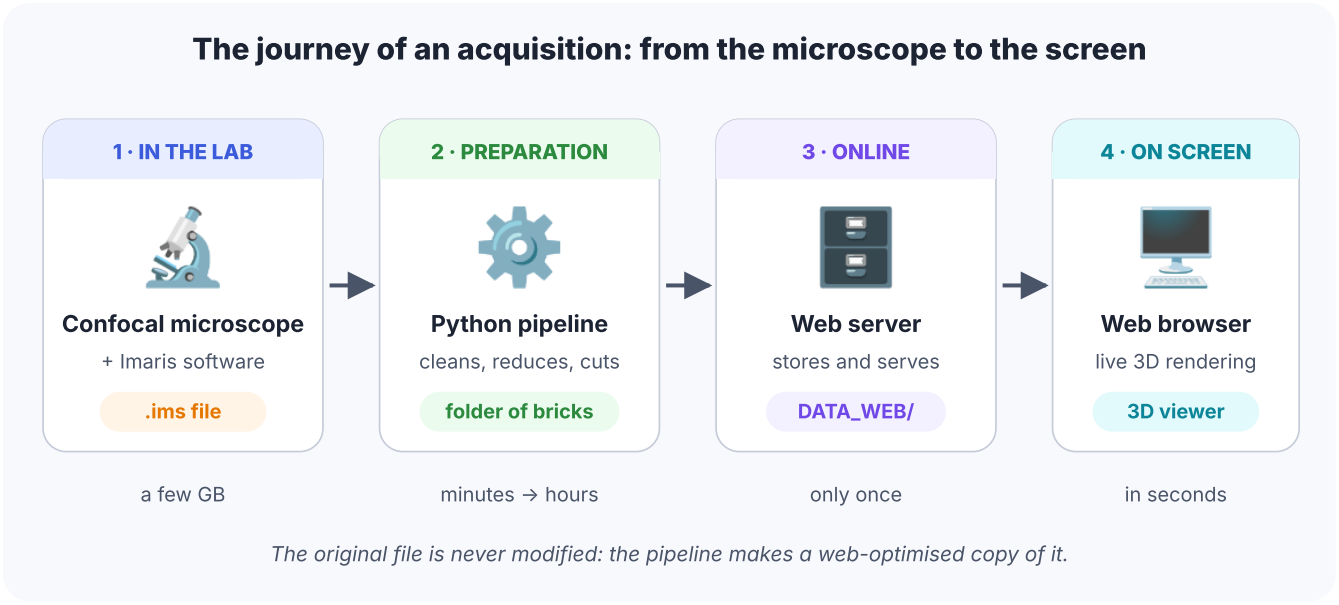
| TYPE | WHAT THE DATASET CONTAINS | EXAMPLE FROM THE DEMONSTRATION SET |
|---|---|---|
| 3D | one volume, 1 to 4 channels | `Embryo-E95-Em2-Pecam1-Sox2` (768 × 576 × 112, 3 channels) |
| Live | one volume per time point, with a timeline | `Demo-Lumen3D-E85-Em1-30min-2ch-4tp` (4 time points) |
| 2D | one photograph calibrated in µm | `DLL4xCD1-E95-x3.2-240913-1` (X-gal staining) |

> ❓ **WHY THIS CHOICE?**
>
> **Why isn’t cell tracking a type?** Because it describes a Live dataset; it does not replace it. The cell positions, their trajectories and the surface are **layers** drawn on top of the volume, and five dedicated tools analyse them (chapter 12). A Live dataset without tracking remains a Live dataset.

> ⚠️ **WARNING**
>
> The datasets in this document are **synthetic demonstration datasets**, made for the occasion: they are not real laboratory data.

## 1.4 The journey of an acquisition

From the specimen to the screen, the data pass through four places.




▲ From the microscope to the screen, in four steps.

| STEP | WHO | TYPICAL DURATION | CHAPTER |
|---|---|---|---|
| **1. Acquisition** | confocal microscope + Imaris software | depends on the specimen | 4 |
| **2. Preparation** | technician, Python pipeline on their computer | minutes to hours, only once | 5 to 8 |
| **3. Going online** | administrator: import by drag-and-drop or SFTP, then publication | depends on the bandwidth | 14 and 17 |
| **4. Display** | anyone, with a browser | a few seconds for the first image | 9 to 12 |

📌 **REMEMBER**

The heavy work (cleaning, cutting up) is done **once, offline**. After that, each visitor downloads only pieces: this is what makes exploration smooth.

## 1.5 Key figures

**3**
data types: 3D, Live, 2D

**$64^3$**
voxels per brick ($66^3$ with border, formats 4 and 5)

**4**
channels displayed at most

**4**
languages: English, French, Spanish, Dutch

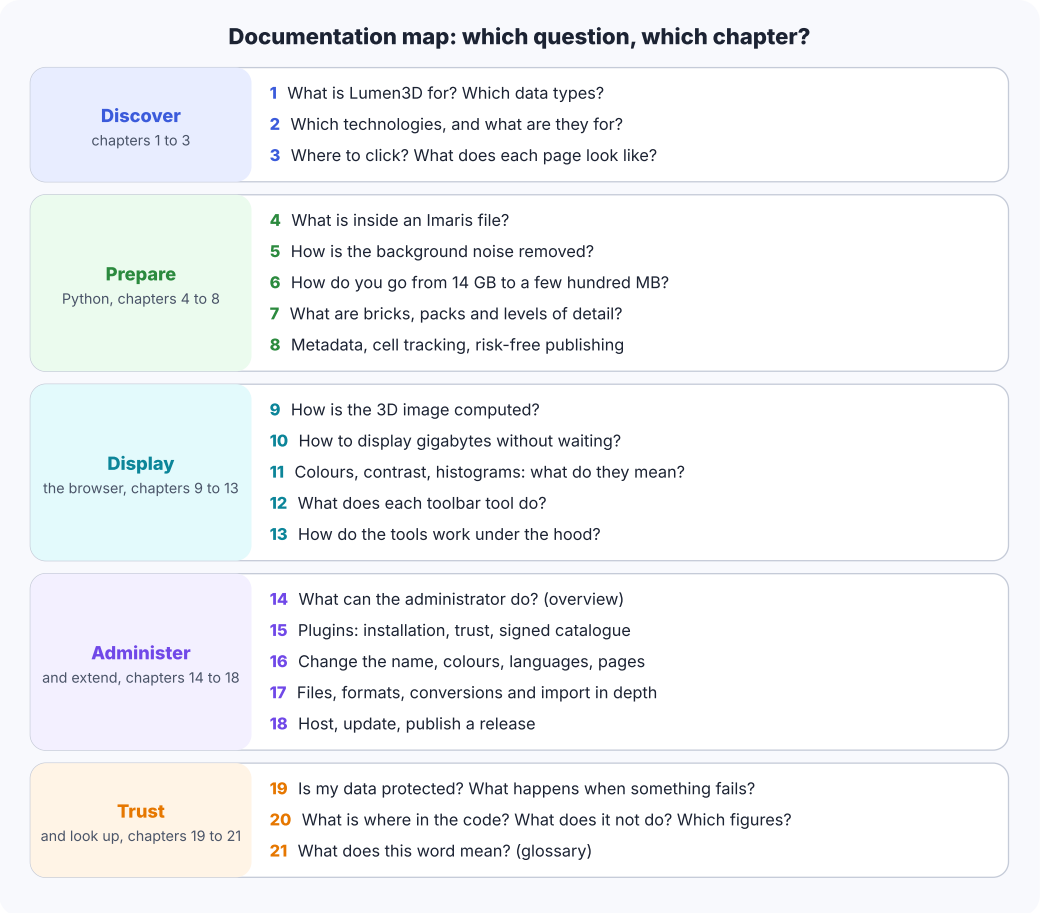
| 28 | ≈ 126,000 | 512 · 1024 · native | 201 |
|---|---|---|---|
| plugins in the signed catalogue | lines of code, excluding tests | the three rendering qualities | automatic test files |

A few version landmarks, valid at the date of this document:

- web platform **1.65.0**; preparation pipeline **0.22.0**;
- current data format: format 5 (formats 1 to 4 are converted in place, see chapter 17);
- every release is checked by the automatic tests before it is published.

## 1.6 Map of the documentation

This document has **21 chapters**, organised like the data's journey: from the microscope to the screen, then administration, reliability and the appendices. Find your question below.




▲ Which question, which chapter?

| CHAPTER | TITLE | FOR WHOM |
|---|---|---|
| 1 – 3 | The platform at a glance; the technologies; a tour of the pages | everyone |
| 4 – 8 | From Imaris to the web folder: background, reduction, bricks, metadata and tracking | technician |

| CHAPTER | TITLE | FOR WHOM |
|---|---|---|
| 9 – 11 | 3D rendering, streaming, channels and histograms | biologist, the curious |
| 12 – 13 | The tools one by one, then under the bonnet | biologist |
| 14 | The administration panel in brief | administrator |
| 15 | Plugins, trust and the signed catalogue | administrator |
| 16 | Customising and translating | administrator |
| 17 | Data in depth: files, formats, migrations, import | administrator, technician |
| 18 | Hosting, updating, publishing a release | system administrator |
| 19 | Security and reliability, with the “What happens if…?” table | everyone |
| 20 | Appendices: module map, browser memories, limits, figures, versions | reference |
| 21 | Glossary | everyone |

👉 **TIP**

In a hurry? Read the **In 30 seconds** boxes at the start of each chapter: they are enough to tell whether the rest concerns you.

# 2. The technologies: who does what

- Lumen3D mixes five languages and a few file formats; each has **one single job**, and this chapter tells you which.
- Everything the browser runs is **supplied by the site itself** (no outside service): the platform works offline and stays under control.
- The server never computes the 3D image: it **hands out files**. The drawing is done on your side, on your graphics card.

## 2.1 A web page is a human body

Before the details, one image to remember it all:

### HTML: the skeleton

Says what is on the page: a title, a button, a drawing area. One file per page, at the root of the project.

### CSS: the clothes

Says what it looks like: colours, sizes, light or dark theme. 15 style sheets (plus the theme generated by the administrator).

### JavaScript: the muscles

Makes things move: a click on a button, downloading bricks, computing measurements.

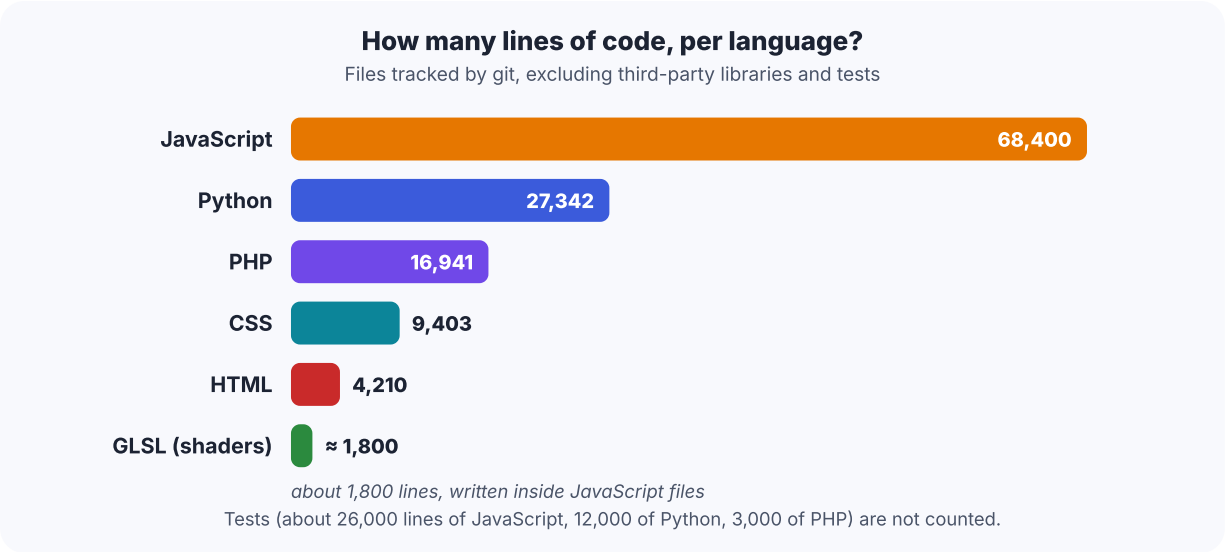
💡 **A PICTURE TO UNDERSTAND**

**A web page = a body.** Without a skeleton (HTML) nothing holds together; without clothes (CSS) it looks bare; without muscles (JavaScript) nothing moves. Lumen3D adds a fourth organ: a very fast **eye**, the shader (section 2.3).

## 2.2 The client: what runs in your browser

The “client” is your browser. It receives the HTML, CSS and JavaScript files, then does all the visible work.

- **“Vanilla” JavaScript**: plain JavaScript, organised in small modules, with no framework. About **68,400 lines**.
- **Web Workers**: the browser’s “behind-the-scenes employees”. They decode the WebP images and do the heavy calculations so that the screen never freezes.
- **Libraries**: Three.js (3D scene), Lucide (icons), Plotly (tracking charts). They are copied into the `js/vendor/` folder.




▲ The languages, by amount of code (the “shaders” bar is included in the JavaScript).

❓ WHY THIS CHOICE?

**Why JavaScript without a framework?** There is no build step: you edit a file, reload the page, and that is it. The code stays readable for someone who has not learned one more tool, and there is no dependency that could disappear or change from one month to the next. Only the administration panel uses modern modules ( `import` / `export` ).

## 2.3 GLSL: the tiny program run millions of times

The 3D image is not a photo: it is **computed for every frame displayed**. For each pixel of the screen, an imaginary ray is sent through the volume and the light it meets is accumulated (chapter 9). That makes millions of small, identical calculations.

The graphics card is built for this: it runs them in parallel. The small program it executes is written in **GLSL**, the language of **shaders**.

💡 A PICTURE TO UNDERSTAND

**A shader is one recipe handed to a million cooks.** Each cook prepares a single pixel, all at the same time, with the same recipe. An ordinary processor would be one very fast cook: it would lose the race.

- About **1,800 lines** of GLSL, written inside JavaScript files (volume rendering, the oblique slice, the recolouring of slices).
- The browser hands them to **WebGL2**, the standard interface for talking to the graphics card.

🔧 FOR THE CURIOUS

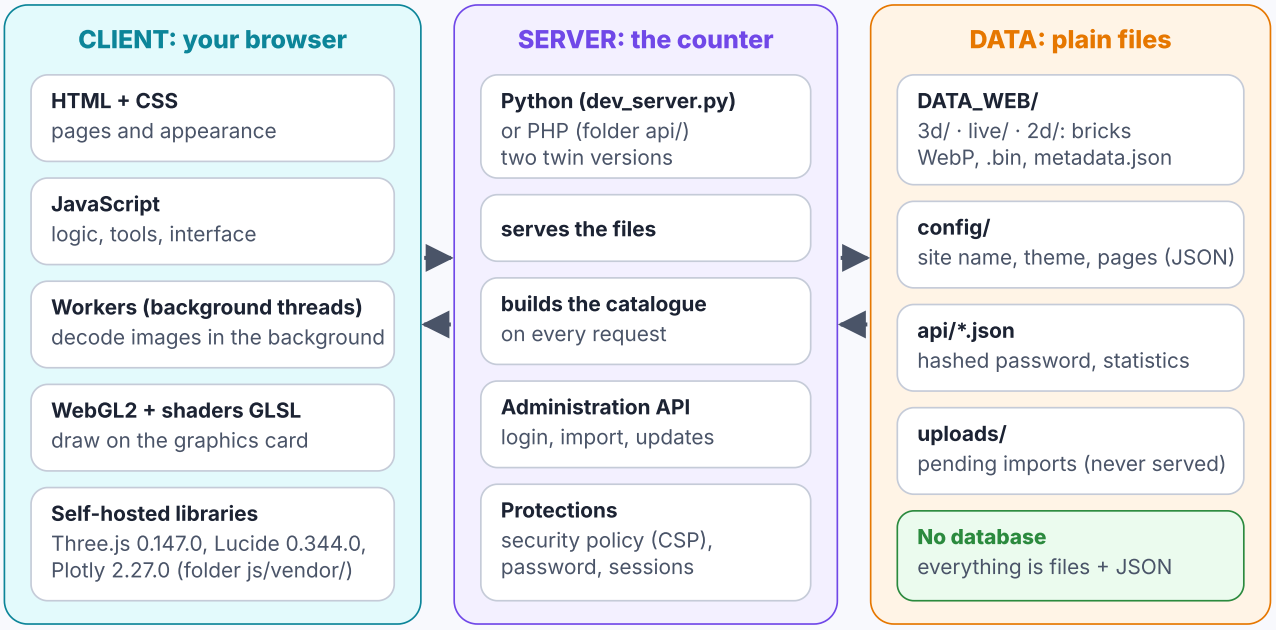
The volume is stored in the graphics card as **3D textures**, divided into slots of $64^3$ voxels ($66^3$ in formats 4 and 5). The shader reads these slots; empty ones are skipped. The details are in chapters 9 and 10.

## 2.4 The server: a counter, not a brain

The server is a program that answers the browser's requests. Here, it has **five jobs**:

| JOB | IN PLAIN WORDS |
|---|---|
| **Serving files** | pages, scripts, bricks; with compression and caching |
| **Building the catalogue** | the list of datasets is **recomputed on every request** by reading the `metadata.json` files |
| **Administration API** | login, editing of dataset records, statistics |
| **Importing** | receive a folder in pieces, check it, publish it |
| **Updating itself** | download a signed version, check it, switch over, roll back if needed |

**Who does what: client, server, data**




*The server never computes the 3D image: it hands out files, the browser does the rest.*
Arrows: HTTP requests from the browser (→) and files sent back (←).

▲ The three actors: the browser draws, the server hands out files, the data are plain files.

🔢 **WORKED EXAMPLE**

**The catalogue is not a file.** You copy a dataset folder into `DATA_WEB/3d/` : it appears in the Explorer straight away. Nothing to regenerate, because the server rereads the records on every request (it remembers the result as long as no file has changed).

### Two twin servers

The same server exists **in two languages**, with the same interface towards the browser:

**Python: `dev_server.py`**

- recommended, port 8080;
- standard library only: nothing to install;
- it updates itself in one block: a switch-over checked by a health probe, with **automatic rollback** if the new version does not start.

**PHP: the `api/` folder**

- for ordinary university or institute hosting, which offers PHP but not Python;
- same files, same formats, same password: an import started under one resumes under the other;
- PHP 8.1 or later;
- it can also update itself (in a single request, with SHA-256 and signature checks, no restart), but **without automatic rollback**: a failed update is simply run again (chapter 18).

🔧 **FOR THE CURIOUS**

A small server, `fast_server.py`, only hands out files (no administration): it is used for performance measurements. The two full servers are kept identical by automatic tests that compare their results.

## 2.5 Python: the laboratory workbench

The **preparation pipeline** is written in Python, because that is the toolbox of science: reading an HDF5 file (`h5py`), computing on large arrays (`numpy`), filtering (`scipy`), writing images (`Pillow`).

💡 **A PICTURE TO UNDERSTAND**

**The workbench.** The pipeline is the workshop where the piece is prepared, away from the shop window. The visitor never sees it: they only see the finished piece on the shelf (`DATA_WEB/`).

- It runs on the technician's computer, **once per dataset**.
- Pinned dependencies: `h5py` 3.16.0, `numpy` 2.5.1, `scipy` 1.16.0, `Pillow` 11.1.0, `tqdm` 4.67.1.
- Python 3.10 or later. The five steps are detailed in chapters 4 to 8.

## 2.6 File formats: containers

A format is a **container**; what matters is what you put in it.

| FORMAT | MENTAL IMAGE | ROLE IN LUMEN3D |
|---|---|---|
| **JSON** | labelled index cards | `metadata.json` (a dataset's record), `manifest.json` (the brick index), configuration, translations |
| **Lossless WebP** | an airtight box for an image | every brick; the pixels come back exactly |
| **PNG** | the same, an older format | the whole planes of format 2 (fast slices) |

| FORMAT | MENTAL IMAGE | ROLE IN LUMEN3D |
|---|---|---|
| `.bin` **(packs)** | a ring binder | hundreds of bricks placed end to end, for fewer files |
| **HDF5 (** `.ims` **)** | a **file system inside a file** | the original Imaris file: folders, arrays and metadata in a single block |
| **glTF / GLB** | a cardboard 3D model | `model.glb`, the surface exported with cell tracking |
| **TIFF** | the laboratory negative | only the optional downloads and the input of photographs |

ℹ️ **GOOD TO KNOW**

The photograph of a 2D dataset is saved as WebP **with** slight loss (quality 90) by default. The bricks of volumes, on the other hand, are always lossless.

## 2.7 Why everything is “self-hosted”

Three.js, Lucide and Plotly are not loaded from the Internet: their files are **in the site’s folder**. Here is why.

1. **Offline**: on a closed institute network, nothing breaks.
2. **Security**: the browser’s security policy (CSP) accepts only code coming from the site itself. A foreign script would be **blocked**.
3. **Integrity**: each library is checked by a fingerprint (SRI); if a single byte changes, it is refused.
4. **Privacy**: no third party knows who is looking at which dataset.

| LIBRARY | VERSION | ROLE |
|---|---|---|
| **Three.js** | 0.147.0 | 3D scene, camera, the starting cube of the rays |
| **Lucide** | 0.344.0 | button icons |
| **Plotly** | 2.27.0 | tracking charts (loaded only when a chart is opened) |

❓ **WHY THIS CHOICE?**

**Why such an old version of Three.js?** Version 0.147.0 is pinned on purpose: a single stable file that does not change under the platform’s feet. The volume calculation itself is custom-written (the “ray marcher” of chapter 9); Three.js only provides the framework.

ℹ️ GOOD TO KNOW

**The only exception**: the fonts (Google Fonts), loaded in the background. They only affect how the text looks.

## 2.8 What the browser needs

- a recent browser with **WebGL2** and **3D textures**; without them, the platform refuses to open a volume and shows a message rather than crashing;
- a graphics card: the more memory it has, the higher the quality that can be reached;
- the light/dark theme, the language and a colour-blind filter are in the header of every page.

The platform **measures** the available memory and adapts to it:

| GRAPHICS CARD CLASS | MEMORY BUDGET USED |
|---|---|
| **no card (software rendering)** | 256 MiB |
| **integrated (laptop)** | 0.5 to 2 GiB |
| **dedicated** | 3 to 4 GiB |
| **unknown** | 1 to 2 GiB |

📌 REMEMBER

If the requested quality exceeds the budget, the platform picks the smaller level and tells you: it does not crash (chapter 10).

## 2.9 The project folder

What are the folders you will see if you open the project for? A glance at the first two levels:

### The project folder from above (two levels)

**What the visitor sees**

| | |
|---|---|
| `*.html` | one page = one file (home, viewer...) |
| `css/` | 16 style sheets (the look) |
| `js/` | all the browser code: `core/ pages/ viewers/ components/ workers/ modules/ (tools) vendor/` |
| `lang/` | translations: en, fr, es, nl |
| `config/` | site name, theme, pages |
| `assets/ logos/` | static images |
| `DATA_WEB/` | the published datasets |

**What serves the files**

| | |
|---|---|
| `dev_server.py` | full Python server |
| `api/` | PHP twin of the server |
| `install.php` | one-file installer |
| `.htaccess, router.php` | Apache / PHP rules |
| `upload_staging.py` | import engine |
| `dataset_migrations.py` | data updates |
| `fast_server.py` | simple test server |

**What prepares the data**

| | |
|---|---|
| `preprocess/` | the Python pipeline (steps 1 to 5) |
| `SCRIPTS/` | lab analyses (cell tracking) |

The pipeline runs on the technician's computer, not on the web server.

**What guarantees quality**

| | |
|---|---|
| `tests/` | 201 automatic test files |
| `changelog/` | one note per version (147) |
| `tools/` | building the signed releases |
| `marketplace/` | signed plugin catalogue |

**Created on a running server (never public)**

| | | | |
|---|---|---|---|
| `uploads/` | imports in progress | `backups/` | backups before an update |
| `logs/` | server logs | `secrets/` | signing keys (local machine) |

The server refuses to serve them: a web address can never reach them.

▲ The project from above: visitor, server, preparation, quality.

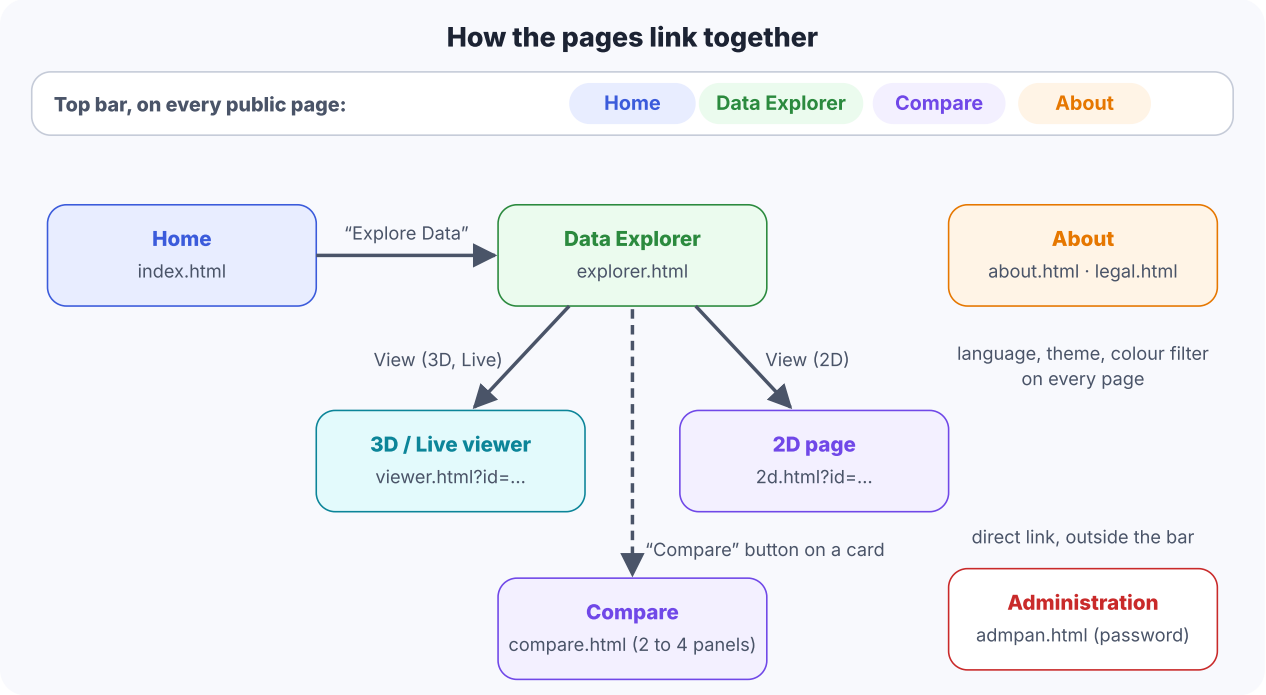
📖 **GOING FURTHER**

The `DATA_WEB/` folders (published data) and `uploads/` (pending imports) are described in chapter 7 and chapter 17. The rules that prevent them from being reached from a web address are in chapter 19.

# 3. A tour of the pages

- The public site has four main pages: **Home**, **Data Explorer**, **Viewer** (3D and Live) and **Compare**, plus the **2D** page and the **About** / **Legal** pages.
- You move from one to another with the top bar or with the buttons on the dataset cards; **the address (URL)** of a page remembers which dataset is open, and even your view.
- Each section below shows a real screenshot (demonstration dataset) with numbered markers.

## 3.1 The map of the pages




▲ The public pages and their links. Solid arrows are clicks; dotted ones are direct links.

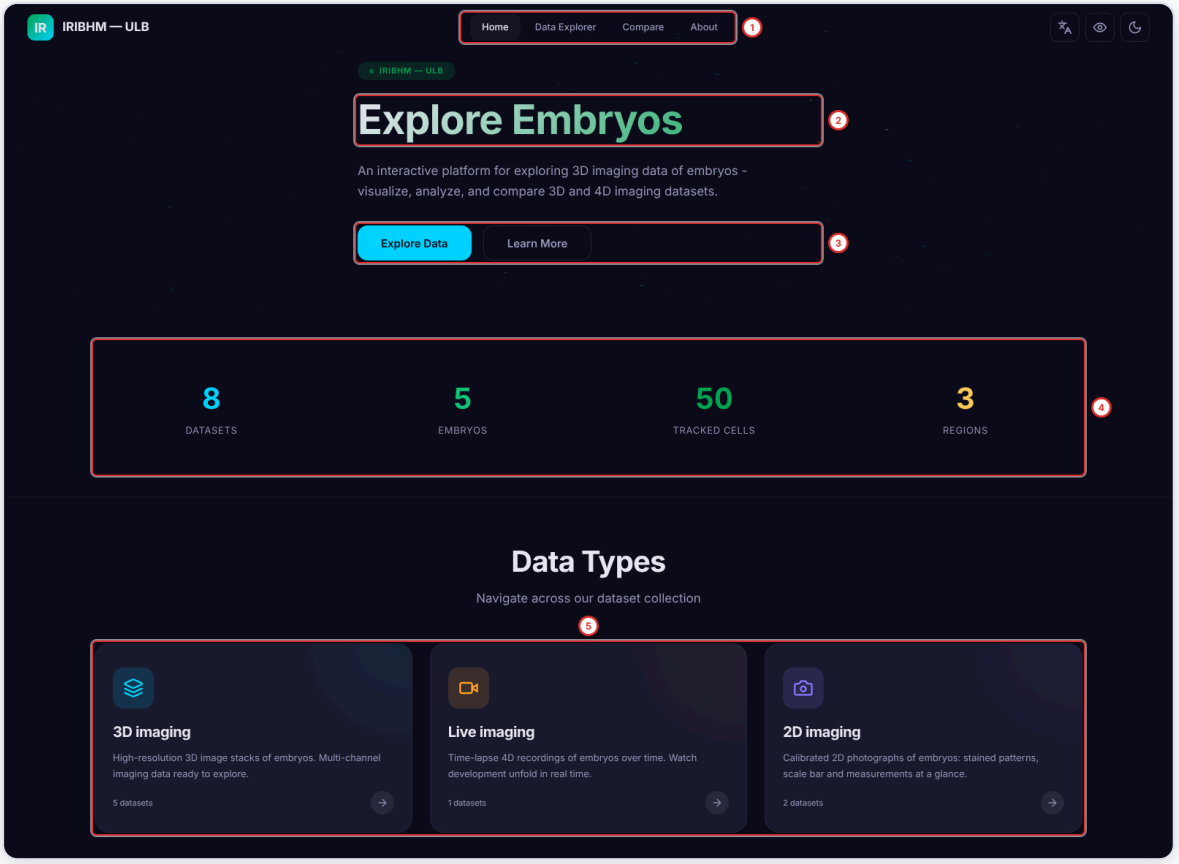
All public pages share the same top bar: the logo, the four links, then three buttons on the right: **language**, **colour-blind filter** and **light/dark theme**.

ℹ️ **GOOD TO KNOW**

The screenshots in this document are in English and in the dark theme. The site name ("IRIBHM — ULB"), the colours and even the texts of the home page are those of the instance that was photographed: another institution may have changed them (chapter 16).

## 3.2 The Home page

This is the shop window: a welcome message, a few figures and three cards, one per data type.



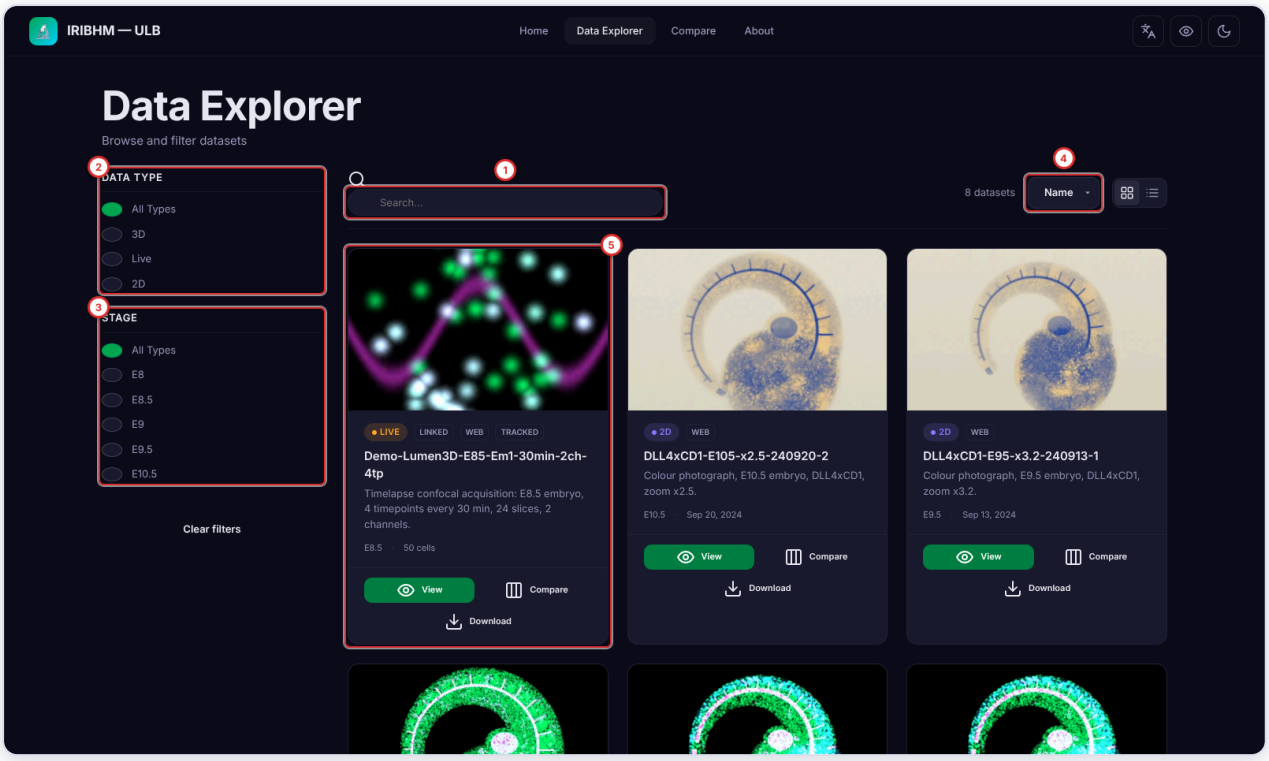

▲ The home page (index.html).

| N | WHAT IT IS |
|---|---|
| 1 | The navigation bar: Home, Data Explorer, Compare, About. |
| 2 | The title: “Explore Embryos”. The word “Embryos” comes from an instance setting. |
| 3 | **Explore Data** opens the Explorer; **Learn More** opens About. |
| 4 | The counters: datasets, embryos, tracked cells, regions. They are computed from the published catalogue. |
| 5 | The type cards: a click opens the Explorer already filtered ( `explorer.html?type=3d` ). |

Below, the **Featured Datasets** section highlights three datasets chosen automatically (the richest tracking, the deepest volume...). The administrator can replace the whole home page with a page of their own.

## 3.3 The Data Explorer

The Explorer lists the published datasets. **Hidden** datasets do not appear in it.




▲ The data explorer (explorer.html).

| N | WHAT IT IS |
|---|---|
| 1 | The search: it looks in the name, description, channel names, stage and embryo. |
| 2 | The **Data Type** filter: 3D, Live or 2D. |
| 3 | The **Stage** filter: one box per stage present (E8, E8.5, E9…). **Clear filters** resets everything. |
| 4 | The sort order (**Name (A-Z)**, **Name (Z-A)**, **Newest first**, **Stage (earliest)**) and the grid / list choice; next to it, the number of results. |
| 5 | A card: thumbnail, type, badges, name, description, stage and date. |

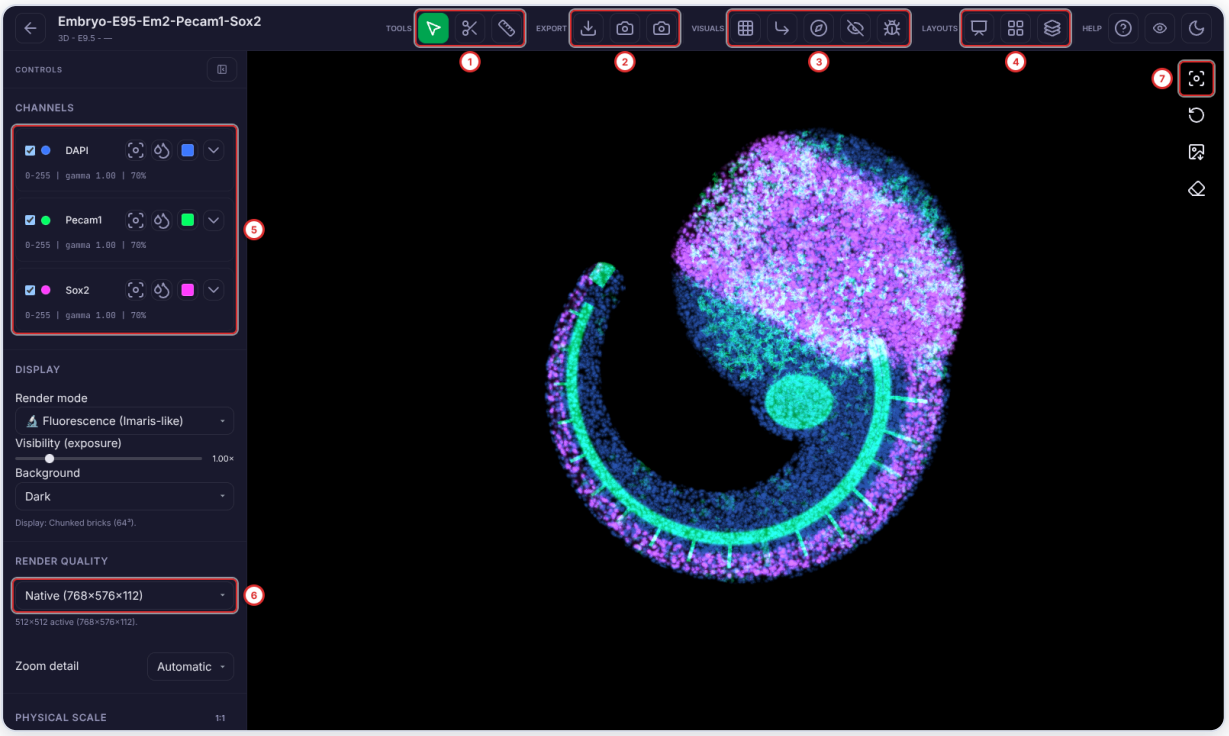
Each card has three actions:

- **View** opens the viewer (or the 2D page for a photograph);
- **Compare** adds the dataset to the Compare page;
- **Download** opens the dataset's download centre (originals and exports, when the administrator has attached them).

> 👉 **TIP**
>
> The badges on a card say a lot: **Web** (can be displayed), **Raw** (original file available for download), **Tracked** (cells tracked), **Linked** (other datasets are attached, for example a photograph of the same embryo).

## 3.4 The 3D viewer

This is the heart of the platform. Open a 3D dataset: the volume first appears blurry, then sharpens.




▲ The viewer on a 3D dataset (viewer.html?id=3d/...).

| N | WHAT IT IS |
|---|---|
| 1 | **Tools** group: **Navigate** (default), **Slice through volume**, **Measure distance**. |
| 2 | **Export** group: **Download Center**, **Screenshot**. |
| 3 | **Visuals** group: grid, axes, orientation axes, hide the volume, brick debugging. |
| 4 | **Layouts** group: **Presentation mode**, **Decompose by channel**, **Z-Stack Browser**. |
| 5 | The **Channels** panel: one row per channel (checkbox, colour, adjustable histogram). |
| 6 | **Render Quality**: 512, 1024 or native; on a format-5 dataset, a “resolution × compression” table (chapter 10). The text below gives the quality actually active. |
| 7 | Four buttons on the volume: centre the sample, reset the view, export the view as PNG, reset the workspace. |

**Moving around**: drag = rotate; Shift + drag (or right button) = pan; wheel = zoom; two fingers = pan and pinch.

Under the channels, the **Display** panel sets the **Render mode** (default “Fluorescence”), the **Visibility (exposure)** and the **Background**. Further down, **Zoom detail** loads finer bricks when you zoom in, and **Physical Scale** stretches the display along Z.

ℹ️ **GOOD TO KNOW**

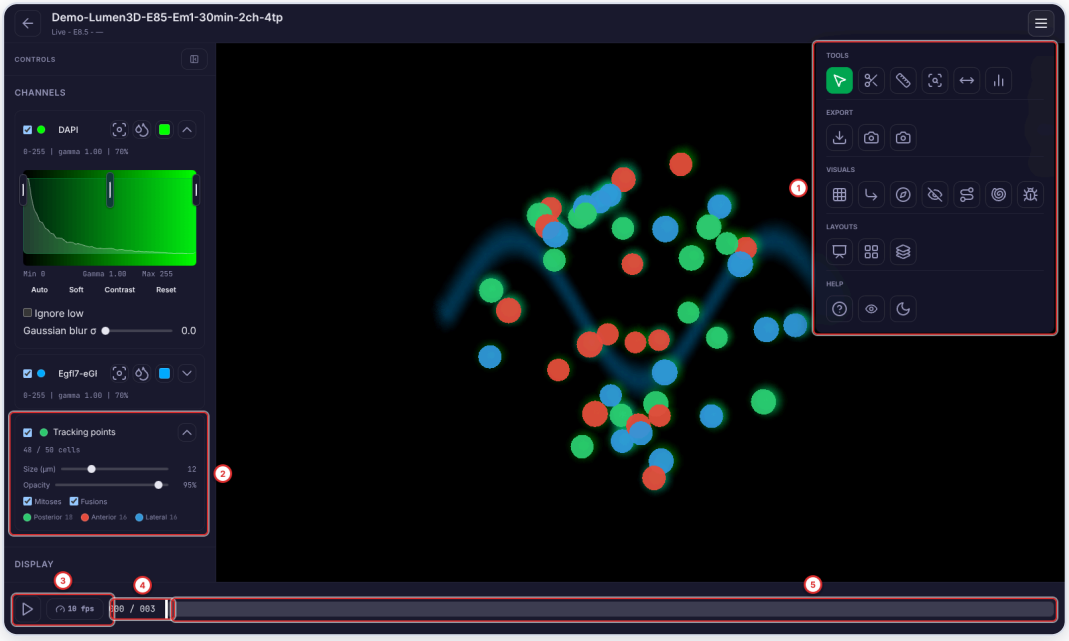
The buttons depend on the **plugins installed** on your instance. Chapter 12: each tool; chapter 11: colours and histograms.

👉 **TIP**

**Sharing your view.** The browser address soon ends in `#state=…` : it contains camera, channels, active tool and measurements. Copy it, and the person who opens it can use **Open the saved view**.

## 3.5 The Live viewer: the timeline

A Live dataset opens in the same page. Two things are added: the **timeline** at the bottom and, if it was tracked, the **Tracking points** layer in the channels panel. On a real time-lapse, playback **waits** until the requested time point is loaded: you see finished images, at the speed at which they arrive (chapter 10).

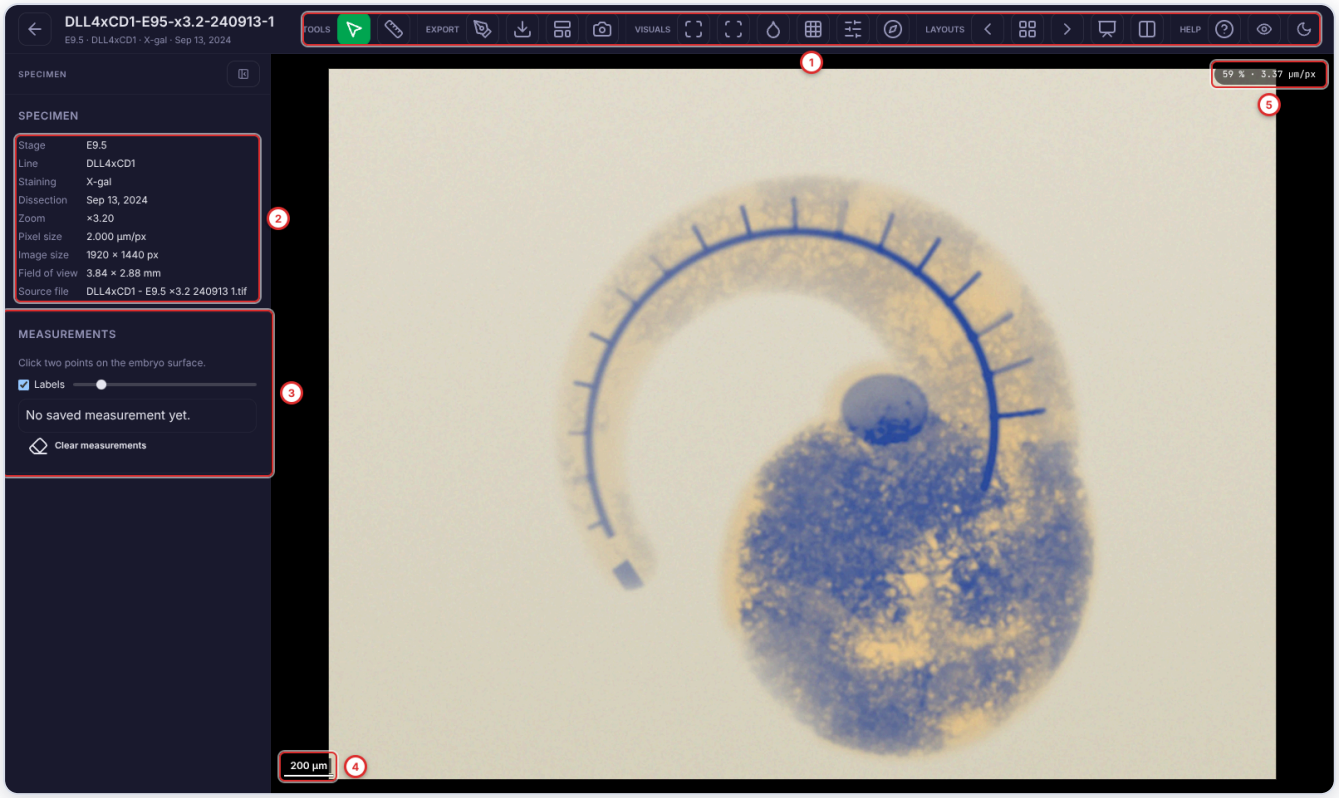



▲ The viewer on a Live dataset (demonstration dataset with 4 time points, 50 tracked cells). The tools menu is open.

| N | WHAT IT IS |
|---|---|
| 1 | The tools menu. When the title is long or the window narrow, the groups fold behind the ☰ button. It includes the tracking tools (inspect a cell, distance between cells, charts, trajectories, surface). |
| 2 | The **Tracking points** layer: size and opacity of the spheres, mitoses, fusions, colour by region. |
| 3 | Play / pause and speed (in frames per second; each click changes the speed). |
| 4 | The counter: current time point / last time point (numbered from 0). |
| 5 | The time slider; the light band shows the time points already in memory. |

## 3.6 The 2D page

A stereomicroscope photograph, calibrated in micrometres. No channels and no 3D: you pan, zoom and measure.




▲ The 2D page (2d.html?id=2d/...).

| N | WHAT IT IS |
|---|---|
| 1 | The toolbar: Navigate, Measure; Studio, download, contact sheet, screenshot; adjust, native 1:1 resolution, isolate the staining; navigation within the collection. |
| 2 | The **Specimen** sheet: stage, line, staining, dissection, zoom, pixel size, field of view. |
| 3 | The **Measurements** section: clicking two points gives a distance in µm. |
| 4 | The scale bar (here 200 µm), which follows the zoom. |
| 5 | The zoom shown, and the size of a screen pixel in µm. |

- Wheel = zoom, drag = pan, double-click = fit to the screen.
- **Browse the collection** opens a contact sheet of all the photographs; ← and → go to the next one.
- The page first shows a light preview, then the native resolution as soon as it is ready.

## 3.7 Compare

Up to **four panels** side by side. Each panel is a **real viewer** (or a real 2D page) embedded: everything you can do in the viewer, you can do here, and volumes and photographs can be mixed.

▲ Comparing three embryos (E8.5, E9.5, E10.5: demonstration dataset).

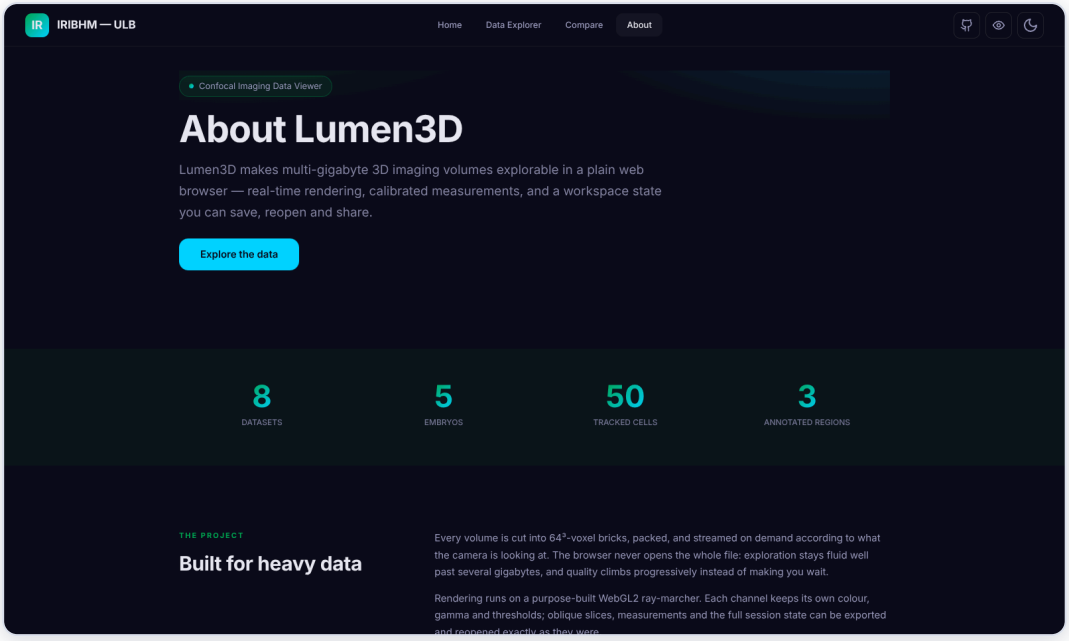
| N | WHAT IT IS |
|---|---|
| 1 | **Add Dataset**: opens a picker window with search and filters by type. |
| 2 | The shared tools (Navigate, Slice, Measure…): one click applies to all panels. |
| 3 | **Studio** (a figure composed from all the panels), **Export**, **Save** and **Restore** the workspace. |
| 4 | **Auto layout** (columns, rows, grid) and **Auto quality**. |
| 5 | **SYNC**: what is synchronised between panels, **Z-Stack**, **Time**, **Camera / view**, **Channels**. Everything is ticked by default. |
| 6 | The buttons specific to a panel (grid, axes, orientation, volume, Z-stack), its settings and its close button. |
| 7 | The name of the panel's dataset. |

- **Camera / view** ticked: rotating one embryo rotates the others, even if they are calibrated differently. For time-lapses of different lengths, the **fraction** of elapsed time is synchronised.
- **Auto quality**: panels open at 512, then go up one by one (up to 1024 for two volumes, 512 beyond) to fit in a shared graphics memory.
- A single volume with several channels? The **Decompose** button clones it into one panel per channel.

- The page address ( `#state=…` ) remembers the panels and their layout: copy it to share the comparison.

## 3.8 About and Legal notice

**About** presents the project. Its default content is a page the administrator can edit (page editor, chapter 16). **Legal** displays the legal text entered in the **Legal** tab; the footer link appears only if it is enabled.



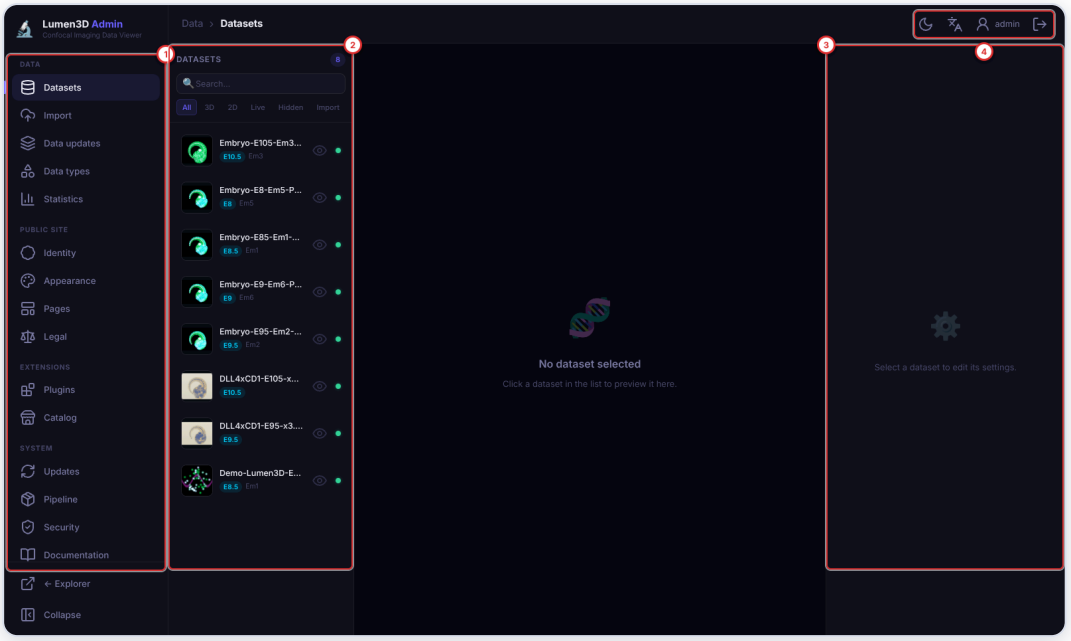

▲ The About page.

> ℹ️ **GOOD TO KNOW**
>
> On a fresh installation, the legal notice is a template to complete (organisation name, address, host): these are bracketed fields that the administrator fills in.

Other pages may exist: the administrator can create **custom pages**, opened by `page.html?slug=<name>` and added to the top bar.

## 3.9 The administration: an overview

The administration lives in `admpan.html` and asks for a password. It is not in the public bar. Here is only what it looks like; the detail is in chapter 14.




▲ The administration panel, Datasets tab.

| N | WHAT IT IS |
|---|---|
| 1 | The menu: four families, **Data**, **Public site**, **Extensions**, **System**. |
| 2 | The list of datasets (published, hidden, being imported) with search and filters. |
| 3 | The preview area of the chosen dataset: the viewer is shown there as visitors will see it. |
| 4 | The theme, the language and the logout. |

## 3.10 The addresses worth knowing

An address (URL) can say everything about a page. The most useful ones:

| ADDRESS | EFFECT |
|---|---|
| `viewer.html?id=3d/<folder>` | opens a 3D volume (same form for `live/…`) |
| `2d.html?id=2d/<folder>` | opens a photograph |
| `explorer.html?type=3d` | Explorer already filtered (`3d`, `live` or `2d`) |
| `compare.html?add=<id>&add=<id>` | Compare with these datasets already in place (4 at most) |
| `…#state=…` | the exact view (camera, channels, tools, measurements): copy it to share |
| `page.html?slug=<name>` | a custom page |

⚠️ **WARNING**

The `hideHeader` and `panelIndex` parameters are used by the Compare page to embed pages without their header; do not use them for normal use.

👉 **TIP**

**Keyboard shortcuts** (viewer): V or Esc Navigate; C slice; M measure; I inspect a cell; D distance between cells. 2D page: F fit, B contact sheet, ← → previous / next photograph.

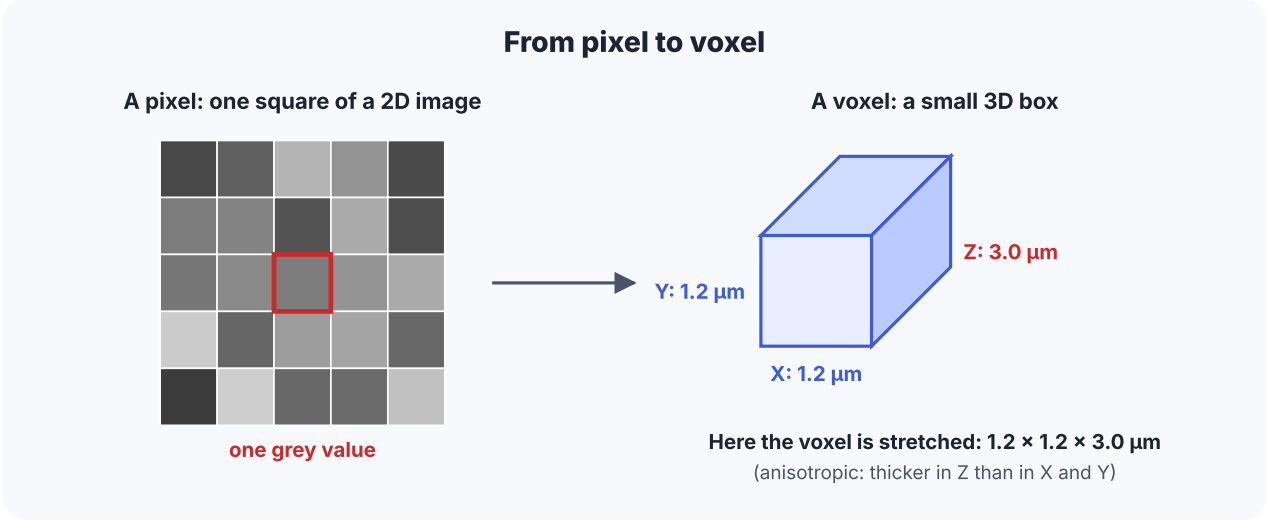
# 4. From the microscope to the Imaris file

- A 3D image is a **stack of slices**; every 3D cell is called a **voxel**, and its real size in micrometres matters.
- An Imaris **.ims** file is a "file system inside a file" (HDF5): the pipeline reads only the image size, the calibration, the channel names, the clock and the full-resolution voxels.
- **Step 1** of the pipeline writes a small `meta.json` file: it computes nothing but the voxel size, and never invents a calibration that is missing.

## 4.1 From pixel to voxel

A digital photo is a grid of **pixels**: each little square carries a grey value (or a colour). A confocal microscope does not take one photo, but a series of photos taken at different depths.

One point of that series is called a **voxel**: a pixel that also has a thickness. It is a small 3D box, and it is the basic unit of this whole document.




▲ From pixel to voxel. Here the voxel measures 1.2 × 1.2 µm in the plane and 3.0 µm in depth: this is the calibration of the demonstration dataset (synthetic embryo).

💡 **A PICTURE TO UNDERSTAND**

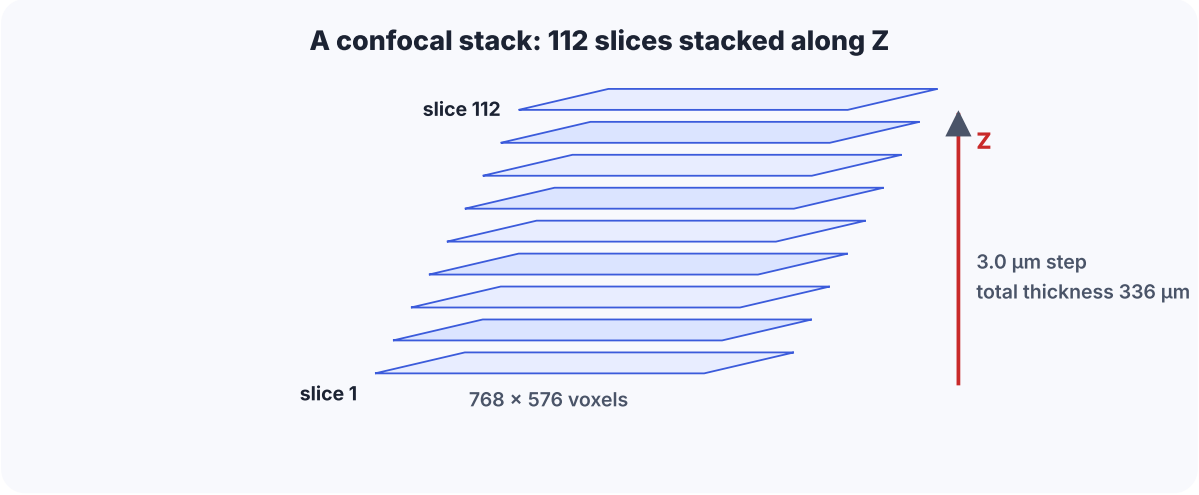
Think of a **box of Lego**: a flat plate of studs is a 2D image (pixels). Stack 112 plates and every stud becomes a 3D brick: a voxel. If your plates are thicker than the width of a stud, your bricks are "stretched" in height.

ℹ️ **GOOD TO KNOW**

An **anisotropic** voxel (thicker in Z than in X and Y) is the rule in confocal imaging: changing depth costs scanning time. The pipeline allows for it at every step (see chapter 6 for the pyramid, chapter 9 for display).

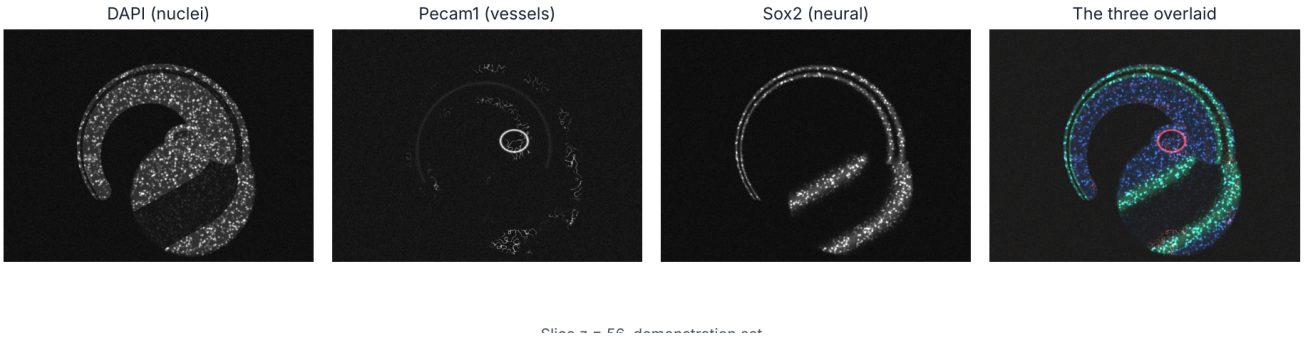
## 4.2 A stack of slices, several channels

The microscope scans the sample slice by slice, along the **Z** axis. The demonstration dataset has **112 slices** of 768 × 576 voxels, spaced 3.0 µm apart: the embryo is therefore 112 × 3.0 = **336 µm** thick.




▲ A confocal stack: 112 slices stacked along Z (diagram, figures from the demonstration dataset).

A **channel** is a complete stack recorded for a single fluorophore, that is to say a single emission colour. The demonstration dataset has **three channels**: DAPI (the nuclei), Pecam1 (the vessels) and Sox2 (the neural tissue).




▲ The same slice (z = 56) seen in each of the three channels, then overlaid. The images come from the demonstration dataset (synthetic embryo); each channel is stretched between 0 and its 99.9th percentile for display.

⏱ **IN 30 SECONDS**

- 1 channel = 1 stack of slices = 1 fluorophore.
- The channels are **not mixed** in the file: each has its own 3D array.
- Colour mixing happens later, in the browser (chapter 11).

## 4.3 How many grey levels? 8 or 16 bits

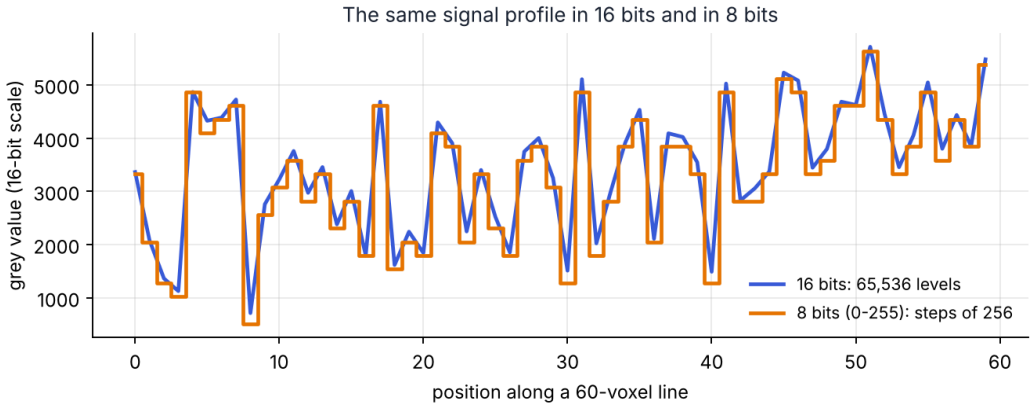
Every voxel carries an **integer**: the measured light intensity. The number of bits says how many values are possible.

| DEPTH | POSSIBLE VALUES | RANGE |
|---|---|---|
| **8 bits** | $2^8$ = **256** | 0 to 255 |
| **16 bits** | $2^{16}$ = **65,536** | 0 to 65,535 |

The lab's Imaris files are 8-bit or 16-bit depending on the acquisition; the pipeline reads both. The demonstration dataset is **16-bit** (type `uint16`).

🔢 **WORKED EXAMPLE**

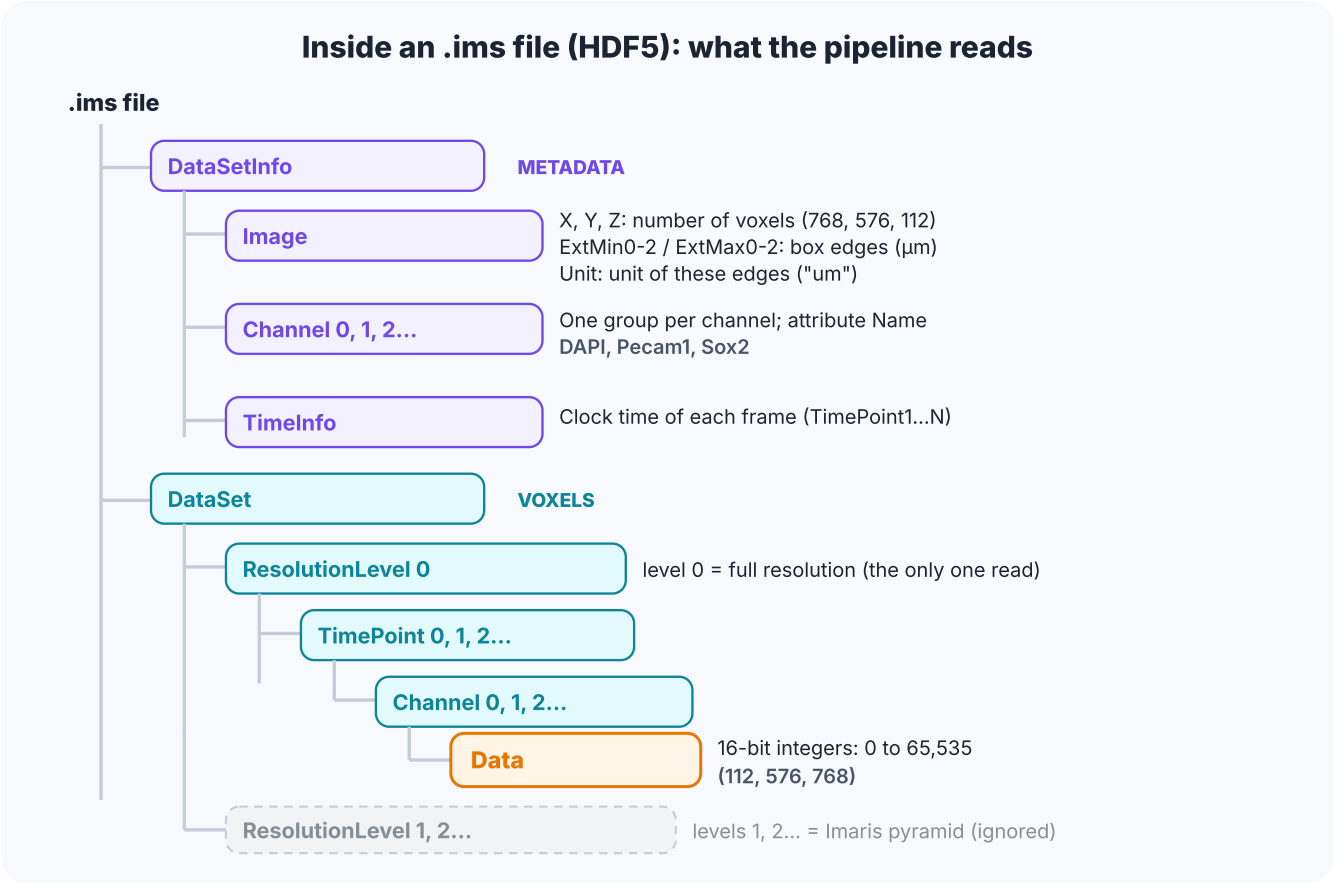
Two fluorescent spots measure **1,000** and **1,200** on a 16-bit scale: a gap of 200 levels, easily measurable. If we simply divided by 256 to get to 8 bits, they would become **3** and **4**: a single step apart. That is why the pipeline does not divide blindly: it first sets a suitable "window" (chapter 5).




▲ A real profile of 60 voxels (Pecam1 channel, demonstration dataset): the blue curve keeps all its nuances in 16 bits; in orange, the same values stored in steps of 256. Here the signal is strong and the steps stay small; on a weak signal, they flatten the details.

## 4.4 An .ims is a "hard disk inside a file"

An **.ims** file is a container in **HDF5** format. Picture a folder of folders: it holds groups (the folders), attributes (small text labels) and arrays (the voxels).




▲ The paths the pipeline actually reads in the .ims of the demonstration dataset. In violet: the metadata (step 1). In green: the voxels (step 2).

🔢 **WORKED EXAMPLE**

In the demonstration file, `DataSetInfo/Image` contains the attributes `X = 768`, `Y = 576`, `Z = 112`, `ExtMin0 = 0.000`, `ExtMax0 = 921.600` and `Unit = um`. Everything is written as **text**: the pipeline converts it to numbers.

🔧 **FOR THE CURIOUS**

HDF5 attributes are sometimes stored as arrays of ASCII bytes; `attr_str` (step 1) glues them back into text. The voxel arrays are compressed (gzip) in blocks of 16 × 64 × 64 voxels: the pipeline always reads sub-boxes, never more than the declared size.

## 4.5 Imaris's pyramid: ignored

Imaris often stores, next to level 0 (full resolution), reduced versions: `ResolutionLevel 1`, `2`... This is its own pyramid, designed for its own display window. The demonstration dataset has none: only level 0 exists. A real lab file has several.

The pipeline **uses only level 0**, and never the others, for the volume.

❓ **WHY THIS CHOICE?**

Imaris's pyramid is computed on the **raw** data, with its own block sizes: it matches neither Lumen3D's brick layout nor the cleaning of chapter 5. It is better to clean level 0 and then build **your own pyramid** from the result (chapter 6): every level is then consistent with the others.

## 4.6 Step 1: reading the metadata

Step 1 ( `1-ims_metadata.py` ) opens the .ims, reads a few attributes and writes `meta.json`. It touches no voxel.

| META.JSON FIELD | WHAT IT IS | VALUE (DEMO) |
|---|---|---|
| `width` , `height` , `depth` | Number of voxels in X, Y, Z | 768, 576, 112 |
| `n_channels` | Number of channels | 3 |
| `n_timepoints` | Number of images over time | 1 |
| `voxel_size` | Size of one voxel in µm (x, y, z) | 1.2; 1.2; 3.0 |
| `extent` | Image box in µm (min, max) | 0 → 921.6 / 691.2 / 336.0 |
| `timestamps` | Time of each image | 2026-03-11 10:00:00 |
| `time_interval_minutes` | Median interval between images | empty (single image) |
| `channel_names` | Channel names | DAPI, Pecam1, Sox2 |

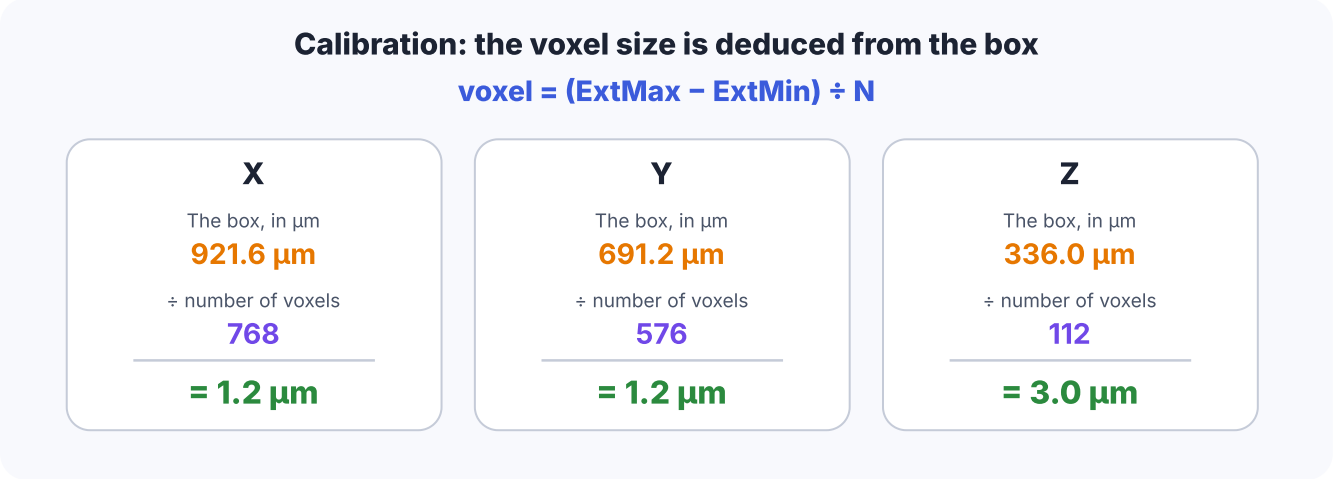
ℹ️ **GOOD TO KNOW**

The number of images determines the dataset **type**: a single image gives a `3d` dataset; several give a `live` one (time series). Step 1 makes that call.

Channel names are cleaned: everything after a null character is removed, and an empty name or one like "ch1" becomes "Channel 1", "Channel 2"... Colours do not come from the .ims: they are assigned later, from a fixed palette (chapter 8).

## 4.7 Calibration: the voxel size

The file does not give the voxel size directly. It gives the **box** (ExtMin / ExtMax) and the number of voxels. The pipeline divides.




▲ Voxel size = (ExtMax − ExtMin) ÷ N, axis by axis. Values from the demonstration file (synthetic embryo).

🔢 **WORKED EXAMPLE**

On X: (921.6 − 0) ÷ 768 = **1.2 µm**. On Z: (336.0 − 0) ÷ 112 = **3.0 µm**. We divide by N (not by N − 1), a deliberate choice in the code.

If the file's unit is not µm, it is converted first:

| UNIT IN THE FILE | FACTOR TO MM |
|---|---|
| `um` , `µm` , `micron` ... | × 1 |
| `nm` | × 0.001 |
| `mm` | × 1,000 |
| `m` | × 1,000,000 |

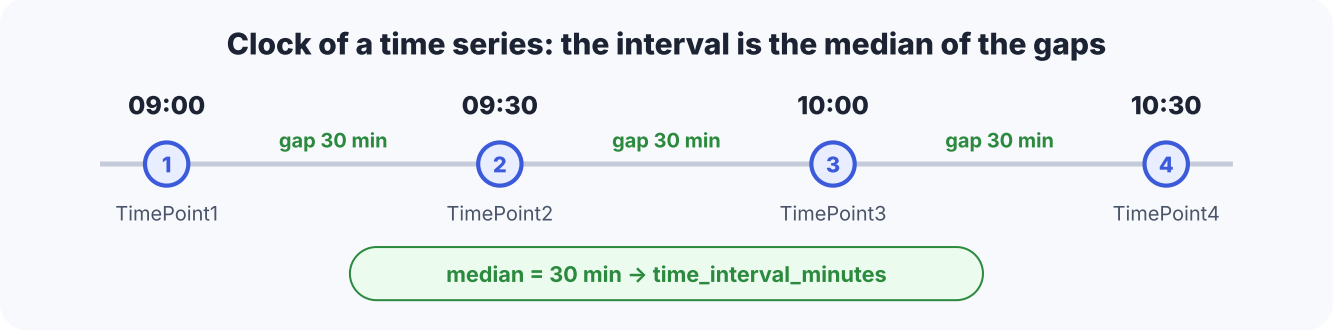
⚠️ **WARNING**

**The calibration is never invented.** If `ExtMin/ExtMax` are missing or the unit is unknown, the three voxel sizes are 0 and `extent` is empty; the platform records `calibrationStatus = metadata-missing` : no scale bar will be made up.

## 4.8 The clock of a time series

For a movie (several images), Imaris notes the time of each image: `TimePoint1` , `TimePoint2` ... (numbering starts at 1 here, whereas the voxel arrays start at 0).

Step 1 converts these texts into dates, then computes the **median interval** between successive images. The **median** (middle value) resists a one-off delay: an image that is a few minutes late does not distort the announced interval.




▲ Example on the time-series demonstration dataset (4 images, one every 30 minutes). The median interval, 30 minutes, becomes `time_interval_minutes`.

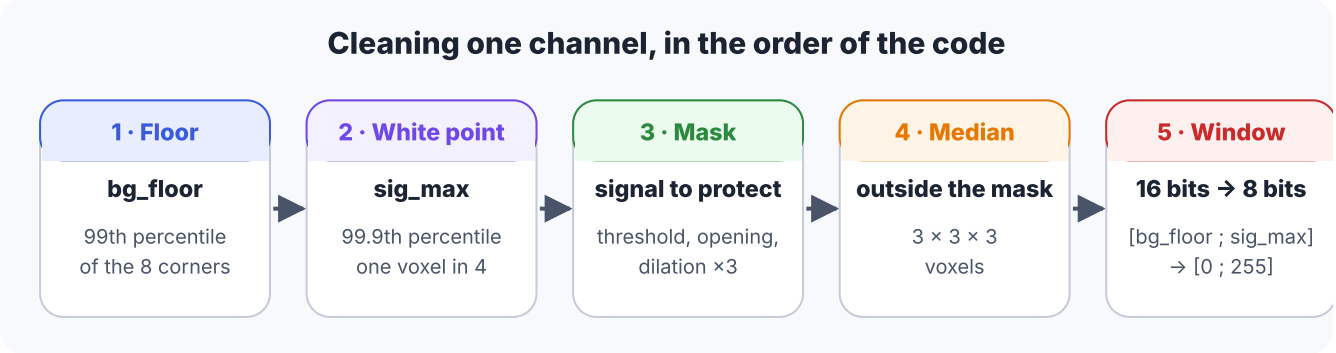
📌 **REMEMBER**

- The voxel = one 3D cell; its size comes from **(ExtMax − ExtMin) ÷ N**.
- Only level 0 of the .ims is read; Imaris's pyramid is ignored.
- Channel names, the clock and the calibration are read; **nothing is guessed**.

# 5. Cleaning the image: removing the background noise

- The pipeline processes **each channel separately** in five stages: measure the background, measure the white point, protect the signal, smooth the background, then convert from 16 to 8 bits.
- Everything is computed **from reference voxels** (the 8 corners, one voxel in 4): no value is chosen by hand, and nothing is “prettified” or invented.
- Result: a background of **exactly black (0)**, an intact signal, and 8-bit values that are **relative** to each channel. The original .ims remains the only quantitative source.

After an overview and a complete example (§ 5.2 and 5.3), this chapter follows the **exact order of the code** ( `2-image_processor.py` ), stages A to E. All the images come from the demonstration dataset (synthetic embryo), DAPI channel, slice z = 56; they are computed with the pipeline’s own functions.




▲ The five stages of cleaning a channel.

## 5.1 Why clean?

A microscope camera is never perfectly black: even with no sample, every voxel shows a small random number (the **camera noise**). In the demonstration dataset it hovers around 2,600 on a scale from 0 to 65,535.

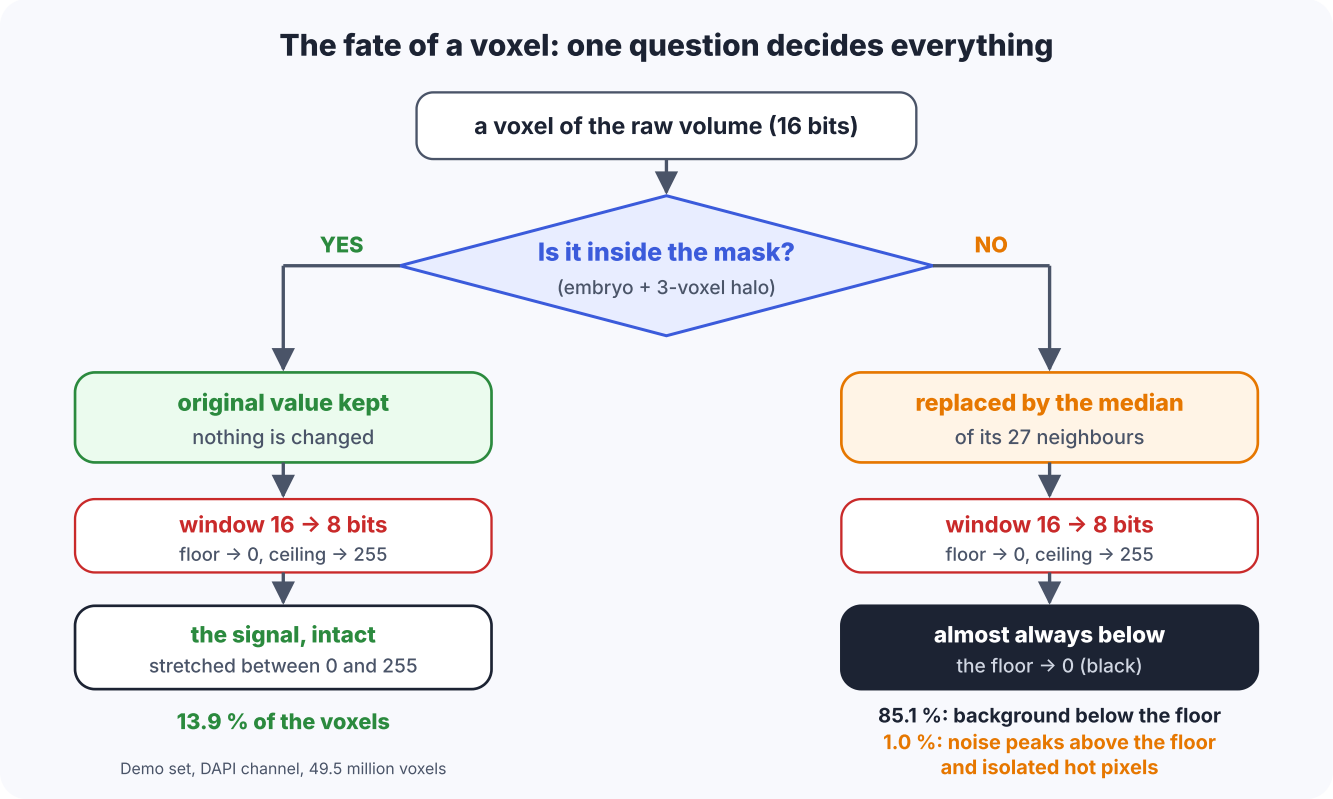
In the browser, rendering **accumulates** light along each ray. A background of 2,600 repeated over 112 slices ends up as a grey fog that hides the embryo.

💡 **A PICTURE TO UNDERSTAND**

It is a **cinema with a screen that is never quite black**: the emergency lights illuminate the room. Cleaning sets the screen’s “black” to the level of the room, so that only the film remains visible.

## 5.2 The idea on one page

All of the cleaning comes down to **a single question**, asked of every voxel: *is it part of the embryo?*




▲ The fate of a voxel. The percentages are measured over the whole volume of the demonstration dataset (DAPI channel).

**YES** (it is inside the "mask"):

- **nothing is touched**;
- its original value is simply converted to 8 bits.

**NO** (it is outside the mask):

- it is **replaced by the median** of its 27 neighbours;
- that value is almost always below the floor, so it becomes **0**.

**85.1 %** background already below the floor

**1.0 %** noise peaks above the floor

**13.9 %** inside the mask: kept as they are

To answer the question, the pipeline needs **two landmarks** and **one map**:

| QUESTION | THE PIPELINE'S ANSWER | NAME IN THE CODE |
|---|---|---|
| **At what level does the real signal start?** | 99 % of the noise of the 8 corners is below it | **floor** ( `bg_floor` ) |
| **At what level should white be set?** | 99.9 % of all the voxels are below it | **ceiling** ( `sig_max` ) |
| **Where is the embryo?** | the voxels clearly above the floor, without isolated points, plus a halo | **mask** |
| **What to do with the rest?** | replace each voxel by the median of its neighbours | **median** |

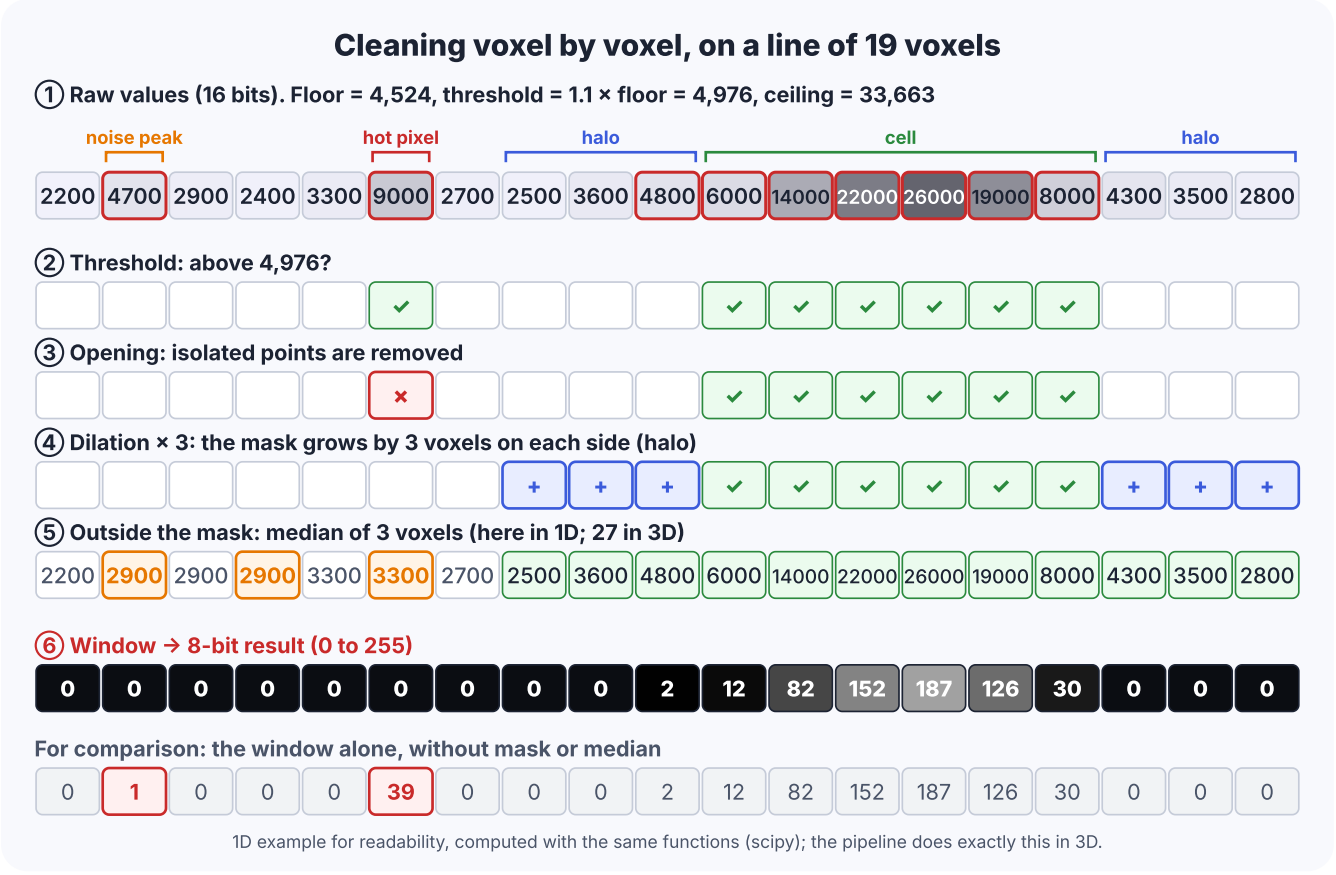
| QUESTION | THE PIPELINE'S ANSWER | NAME IN THE CODE |
|---|---|---|
| **How to go to 8 bits?** | floor → 0, ceiling → 255, a straight line in between | **window** |

💡 **A PICTURE TO UNDERSTAND**

Think of a **proofreader** who must remove ink spots from a page. The written lines (the mask) are left alone. An isolated spot is erased. To decide what a spot is, the proofreader looks around it: if everything around is white, it is a spot; if it sits in the middle of a word, it is writing.

## 5.3 A complete example, voxel by voxel

Take **19 voxels in a row**, with the real landmarks of the demonstration dataset (floor 4,524, ceiling 33,663). They hold a **noise peak** (4,700), a **hot pixel** (9,000) and a **cell** surrounded by its halo.




1D example for readability, computed with the same functions (scipy); the pipeline does exactly this in 3D.

▲ The six stages, row by row. A one-dimensional example so that every number can be read; the pipeline applies exactly the same rules in 3D.

Read the figure from top to bottom:

1. **Raw values.** The cells framed in red are above the floor. There are 8: the noise peak, the hot pixel, the cell and the edge of the halo (4,800).

2. **Threshold.** Only what is above **1.1 × floor = 4,976** counts as "certain signal". The noise peak (4,700) and the edge of the halo (4,800) do not pass; the hot pixel (9,000) does.

3. **Opening.** The hot pixel is **alone**: it is removed (×). The cell, wide enough, stays.

4. **Dilation × 3.** The mask grows by **3 voxels on each side** of the cell (+). The halo, including the 4,800, is now protected.

5. **Median.** Outside the mask, each voxel takes the middle value of its neighbours. The 4,700 noise peak becomes **2,900**; the 9,000 hot pixel becomes **3,300**. Inside the mask (green), nothing changes.

6. **Window.** Everything below 4,524 becomes **0**. The cell goes from 12 to 187; the halo at 4,800 gives **2** (it is kept, very dark).

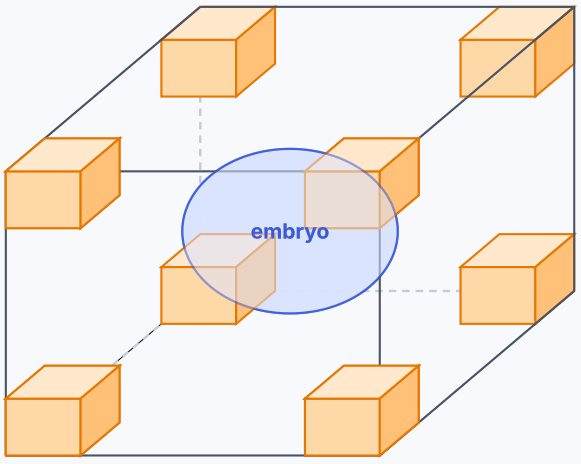
🔢 WORKED EXAMPLE

**And without mask or median?** (last row of the figure) The window alone would let the noise peak (**1**) and the hot pixel (**39**) through: two stray grey dots on a black background. That is the whole point of the mask and the median.

## 5.4 Stage A: the floor ( `bg_floor` )

Where can pure background, with no embryo, be found? **In the corners** of the volume: the embryo is in the centre and touches none of them. The pipeline takes **8 cubes**, one per corner, **at most 32 voxels** on a side.

**The 8 corners: pure camera background, no embryo**




**8 small cubes**

**up to 32 × 32 × 32 voxels each**

(here 28: a quarter of the Z thickness = 112)

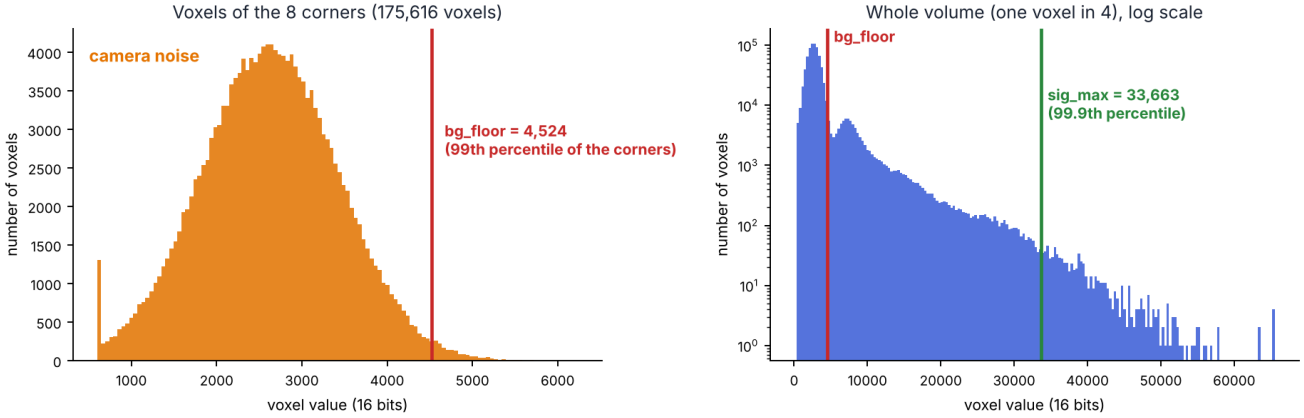
▲ The 8 corner cubes (diagram). In the demonstration dataset the stack has only 112 slices: the side is min(32, X÷4, Y÷4, Z÷4) = 28 voxels, i.e. $8 \times 28^3 = 175{,}616$ reference voxels.

Then: **floor = 99th percentile of these voxels**, i.e. the value below which 99 % of them lie.

🔢 **WORKED EXAMPLE**

Line up the 175,616 corner voxels from smallest to largest. The voxel at 99 % of the queue is **4,524**: that is the floor.

- the noise averages about 2,600;
- 99 % of the noise is **below 4,524**;
- 1 % of the noise **exceeds** it (the strongest corner voxel is 6,238).




▲ Left: the voxels of the 8 corners form a “bell” of noise; the red line is their 99th percentile. Right: the whole volume (one voxel in 4, log scale); the big peak on the left is the background, the long tail is the signal (demonstration dataset, DAPI channel).

❓ **WHY THIS CHOICE?**

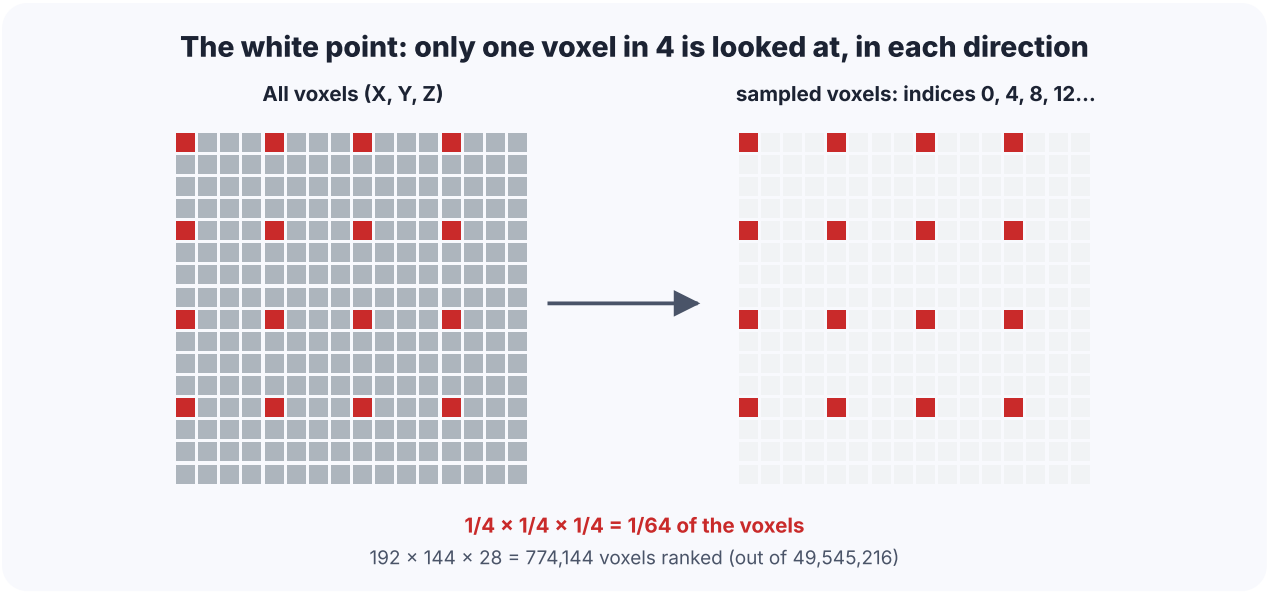
**Why the 99th and not the maximum?** The maximum depends on a single voxel, the most extreme one: it varies a lot from one acquisition to the next and would cut off faint signal. The 99th percentile is stable; the 1 % of noise above it is handled afterwards by the mask and the median. The code tried other rules (5th then 20th percentile, 10th percentile of the corners) before settling on this one.

⚠️ **WARNING**

If a corner touches the sample (tight crop, tile mosaic), the floor can be too high and cut off faint signal. The pipeline **writes it in its log** (“corner noise well above the volume median”) but changes nothing: it never corrects silently.

## 5.5 Stage B: the ceiling (`sig_max`)

For the top of the scale, the **whole volume** is looked at, but reading **only one voxel in 4** in each direction (indices 0, 4, 8…). That reads 1 voxel in 64 without changing the statistics.




▲ Sampling the white point: 1/64 of the voxels is enough.

**Ceiling = 99.9th percentile of these voxels**: the brightest 0.1 % will be shown white (“saturated”). In the demonstration dataset: **33,663**.

⏱ IN 30 SECONDS

- Floor = 4,524: whatever is below will become 0.
- Ceiling = 33,663: whatever is above will become 255.
- The useful range is therefore **[4,524; 33,663]**, not [0; 65,535].

## 5.6 Stage C: the mask, or “where is the embryo?”

The mask is a **yes / no map** the size of the volume: *yes* = this voxel belongs to the embryo (or to its halo) and will not be touched. It is built in three moves.

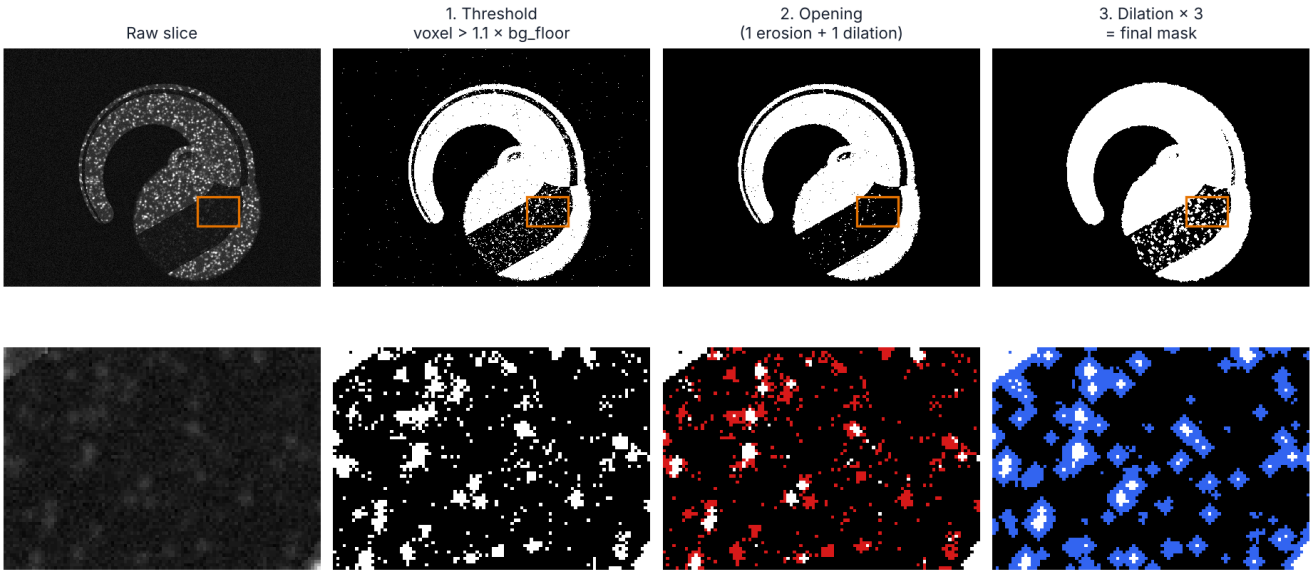
**1. Threshold: above 1.1 × floor.** Here 1.1 × 4,523.85 = **4,976**.

❓ WHY THIS CHOICE?

**Why 10 % above the floor?** By definition, 1 % of the noise exceeds the floor. With a 10 % margin, only **0.2 %** is left (measured in the corners of the demonstration dataset): five times less noise mistaken for signal.

**2. Opening: removing isolated points.** It is an **erosion** (“nibble” 1 voxel all around) followed by a **dilation** (“give back” 1 voxel). A thick enough object gets its shape back; an isolated point, entirely nibbled away at the first step, does not come back.

**3. Dilation × 3: adding a halo.** The mask is widened by **3 voxels** all around. Fluorescence does not stop sharply at a cell’s edge: it fades. The halo protects that fade (and faint signal right next to it) from the median.




▲ The three moves of the mask on slice z = 56. Bottom row: zoom. In red, the voxels removed by the opening; in blue, those added by the dilation.

**11.77 %** voxels above the threshold

**11.32 %** after the opening

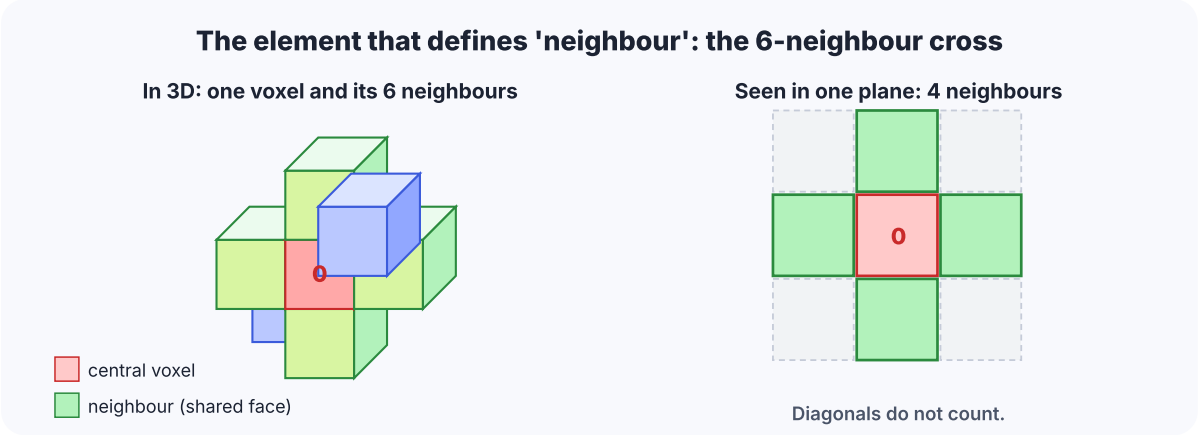
**13.92 %** final mask (with halo)

🔢 **WORKED EXAMPLE**

**The halo in micrometres.** With the voxels of the demonstration dataset (1.2 × 1.2 × 3.0 µm), 3 voxels are **3.6 µm** in X and Y and **9 µm** in Z. Because the dilation works through faces, the halo is a little narrower along diagonals (a diamond shape).

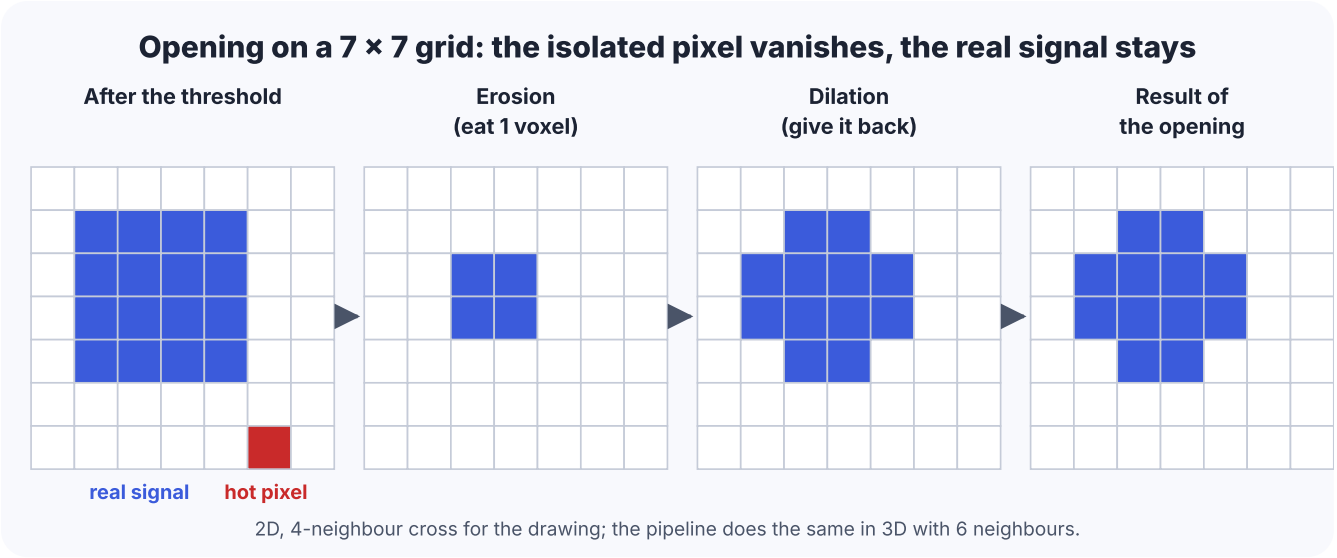
## What is a “neighbour”?

Erosion and dilation look at a voxel’s neighbours. The pipeline uses the **6-neighbour cross**: the two neighbours in X, in Y and in Z, which share a face. Diagonals do not count.




▲ The structuring element: the 6-neighbour 3D cross (scipy’s default setting, not changed by the pipeline).

### Opening in miniature




▲ A 7 × 7 mini-grid. The 4 × 4 block survives (its corners slightly rounded), the isolated hot pixel disappears. Computed with scipy.

🔢 **WORKED EXAMPLE**

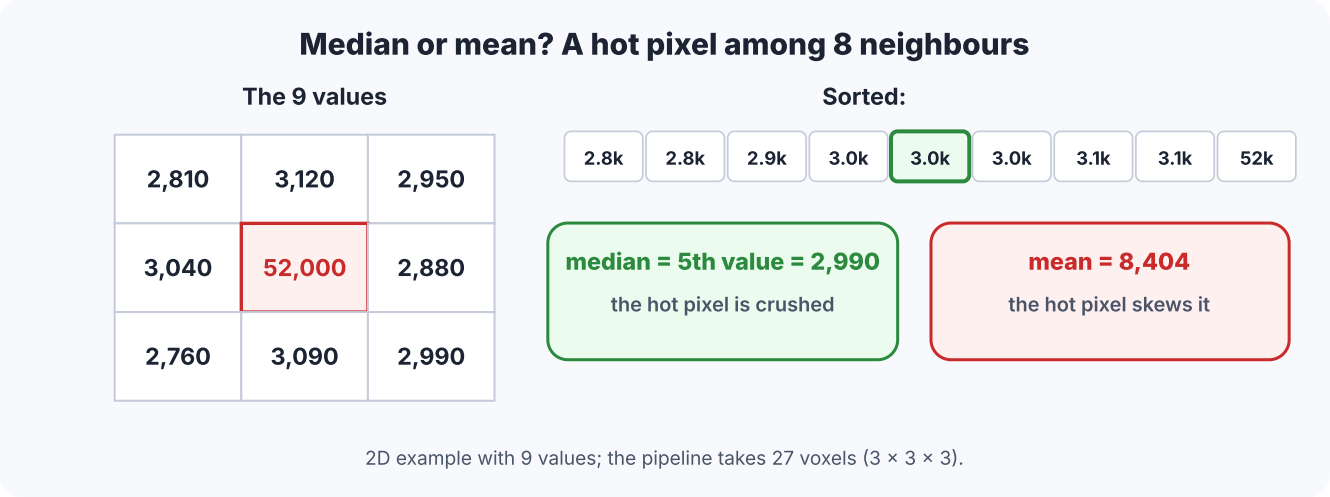
On the grid: 17 pixels lit at the start (16 of signal + 1 hot); after the opening **12** remain: the hot pixel and the 4 corners of the block are gone. The block's corners come back with the dilation × 3.

❓ **WHY THIS CHOICE?**

The opening is done **before** the dilation. In the reverse order, a hot pixel would be "protected" and even **enlarged** by the dilation. Here it stays outside the mask, so it will be crushed by the median.

## 5.7 Stage D: the median, outside the mask only

For each voxel **outside the mask**, the pipeline takes the **27 voxels** of the small 3 × 3 × 3 cube around it, sorts them, and keeps **the middle one** (the 14th). Inside the mask, the original value is kept.




▲ Median versus mean on 9 values (2D example): a 52,000 hot pixel is crushed by the median (2,990), but drives the mean up to 8,404.

💡 **A PICTURE TO UNDERSTAND**

The **median** is the "middle" salary of a street: a billionaire moving in does not change it. The **mean** explodes. That is why a median filter crushes a hot pixel without inventing any new value.

**What is it really for?** To deal with the **1 % of noise that exceeds the floor**. Without it, those voxels would become small grey dots on the black background.

**341,674** visible noise dots with the window alone

**10,069** with mask + median

**− 97 %** stray dots

ℹ️ **GOOD TO KNOW**

The 10,069 voxels left (0.02 % of the volume) have a median above the floor: they are **faint but extended areas**, where more than half of the 27 neighbours are above the floor. An isolated point never survives: its neighbours are background.

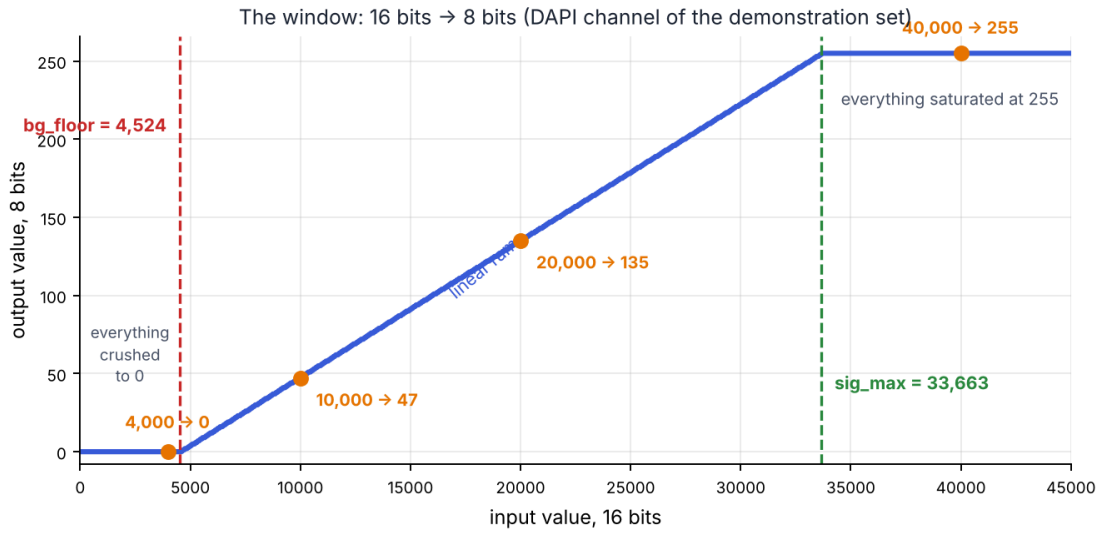
## 5.8 Stage E: the window, from 16 to 8 bits

The last operation, voxel by voxel, the same for the whole volume. The exact formula of the code:

🔢 **WORKED EXAMPLE**

**u8 = truncate( 255 × (clip(v, floor, ceiling) − floor) ÷ (ceiling − floor) )**

*v* is the voxel's value (original inside the mask, median outside); *clip* brings *v* back between the floor and the ceiling; "truncate" means the decimals are dropped (no rounding).




▲ The window: flat at 0 below the floor, flat at 255 above the ceiling, linear in between. The orange points are the micro-examples below.

🔢 **WORKED EXAMPLE**

With floor = 4,523.85 and ceiling = 33,662.57 (difference: 29,138.72):

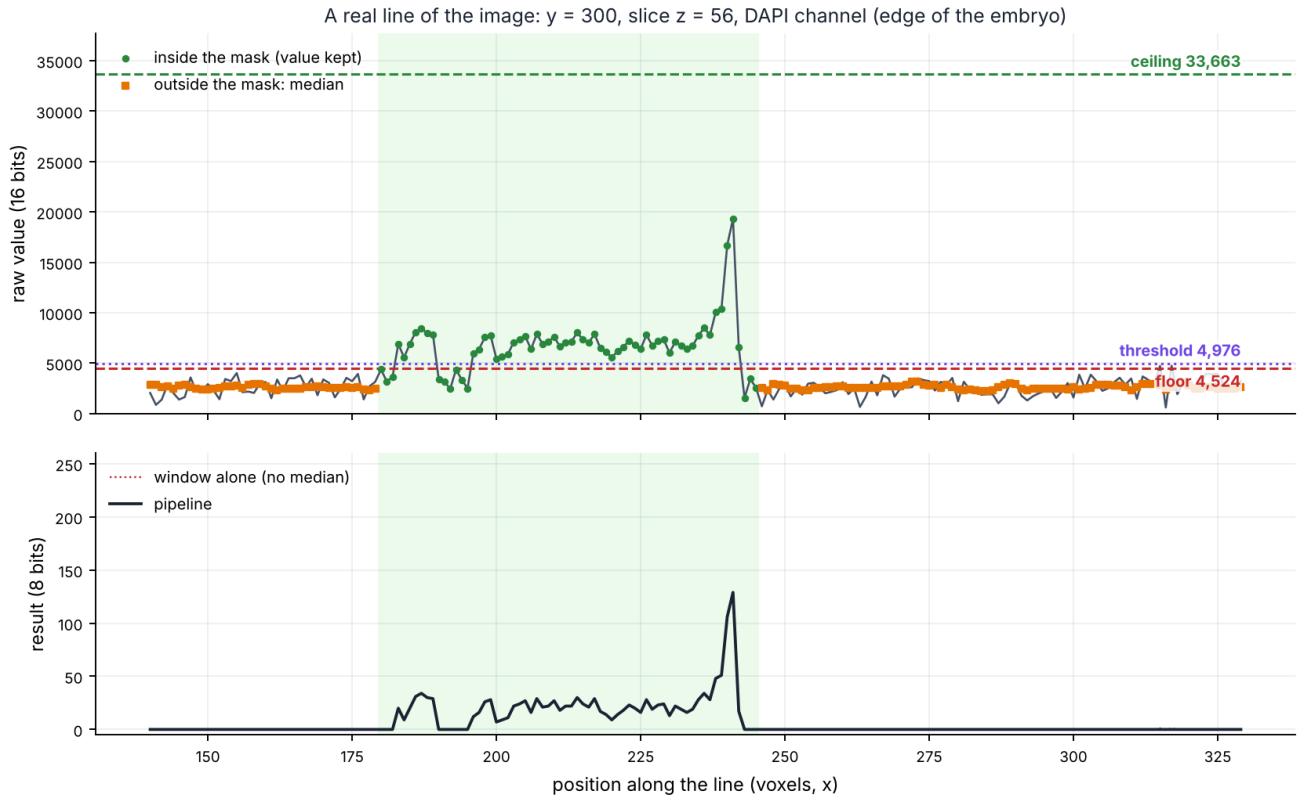
- v = 4,000 → below the floor → **0**;
- v = 10,000 → 255 × 5,476.15 ÷ 29,138.72 = 47.9 → **47**;
- v = 20,000 → 255 × 15,476.15 ÷ 29,138.72 = 135.4 → **135**;
- v = 40,000 → above the ceiling → **255**.

ℹ️ **GOOD TO KNOW**

The conversion is **linear**, with no gamma curve. Any voxel ≤ floor becomes **exactly 0**: the background is a perfect black, which makes it possible to store no empty brick at all (chapter 6). In the demonstration dataset, **88.2 %** of the DAPI channel's voxels are 0 after cleaning.

# 5.9 On real images

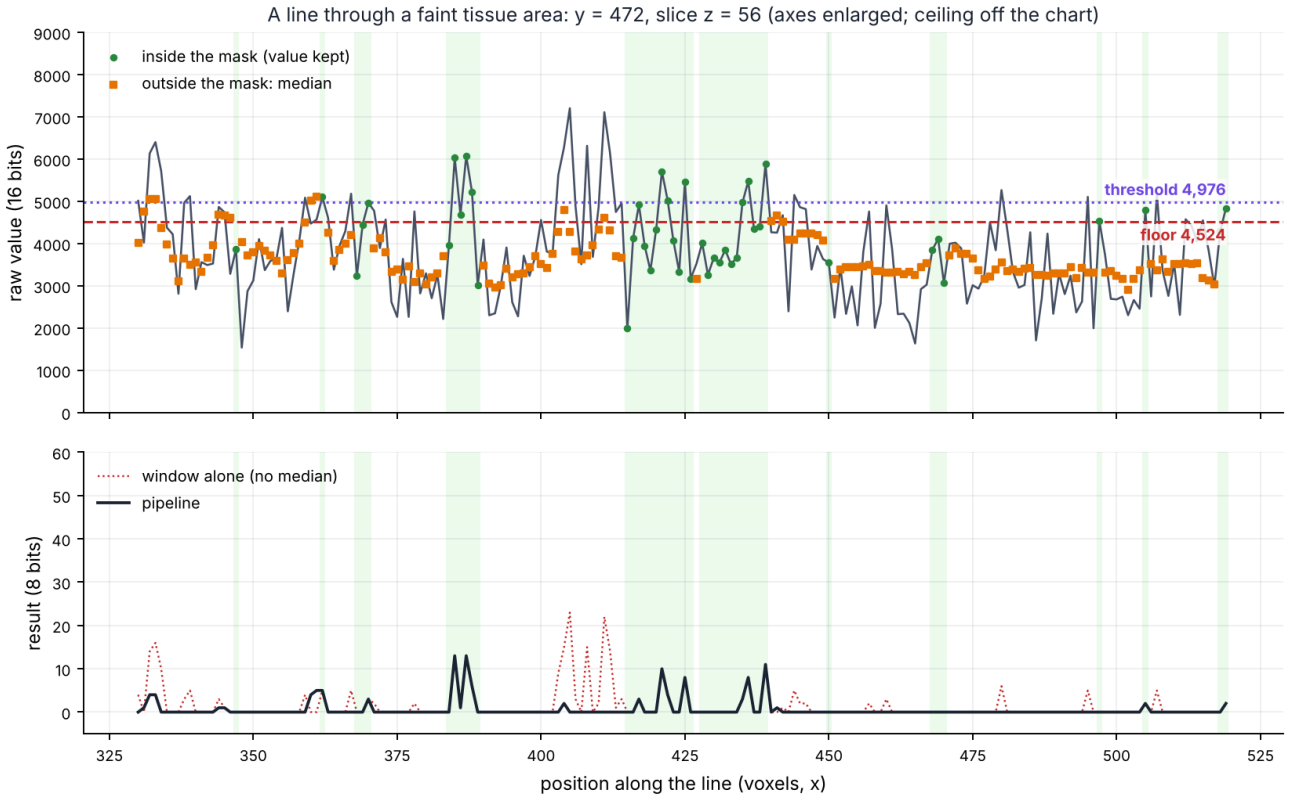
## A line at the edge of the embryo




▲ Top, the raw values along a line: green = inside the mask (value kept), orange = outside the mask (the median, which replaces the value). Green band = mask. Bottom, the 8-bit result.

- **Left and right** (background): the raw values swing between about 1,000 and 4,000; the median (orange) smooths them around 2,600; result **0**.
- **In the centre** (tissue): the green band covers the tissue **and its 3-voxel halo**; the values are kept, and the dark dip in the middle stays a dip.

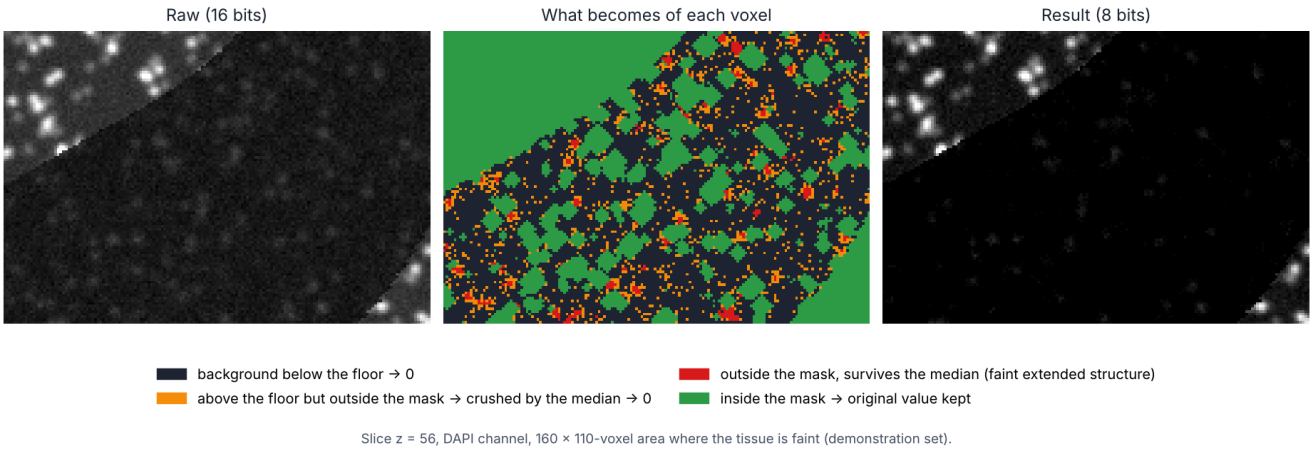
## A line through a faint area




▲ Same reading, in an area where the tissue is barely above the noise (enlarged axes). Red dotted line: what the window alone would give.

- Only the peaks that form a **small cluster** (at least a few voxels in all 3 directions) enter the mask and are kept.
- Narrow peaks (around x = 405–412), even at 7,000, are **removed**: the red dotted line shows what would have been seen without the median.

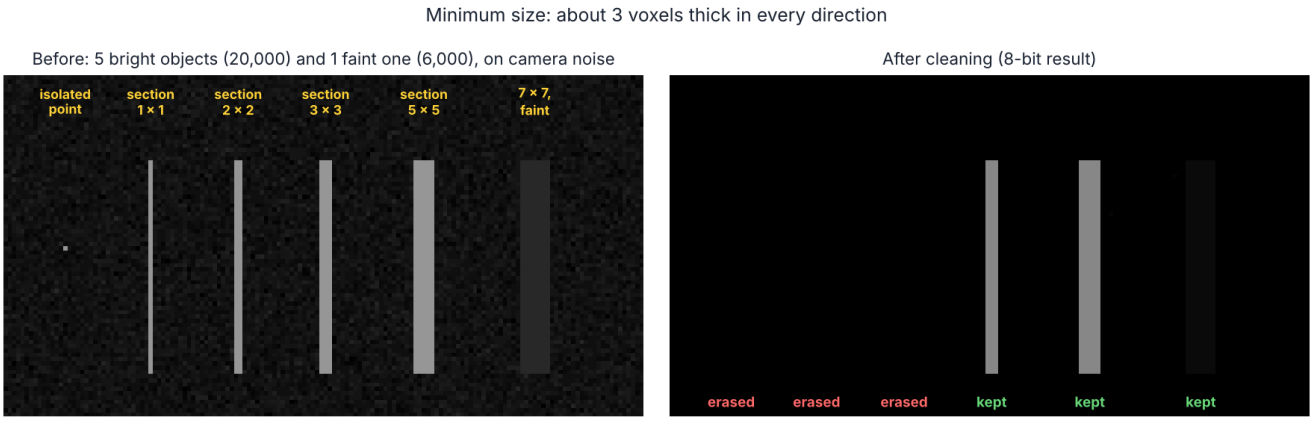
## The map of fates




▲ Left the raw slice, centre what becomes of each voxel, right the result. In a faint area the mask (green) keeps only the clusters; orange is crushed by the median; red, rare, survives the median.

## 5.10 Minimum size of an object

A direct consequence of the opening and the median: **an object that is too thin is erased, even if it is very bright.**




Synthetic test volume, middle slice: each bar has a square cross-section (same thickness in X and in Z). Same floor and ceiling as the demonstration DAPI channel.

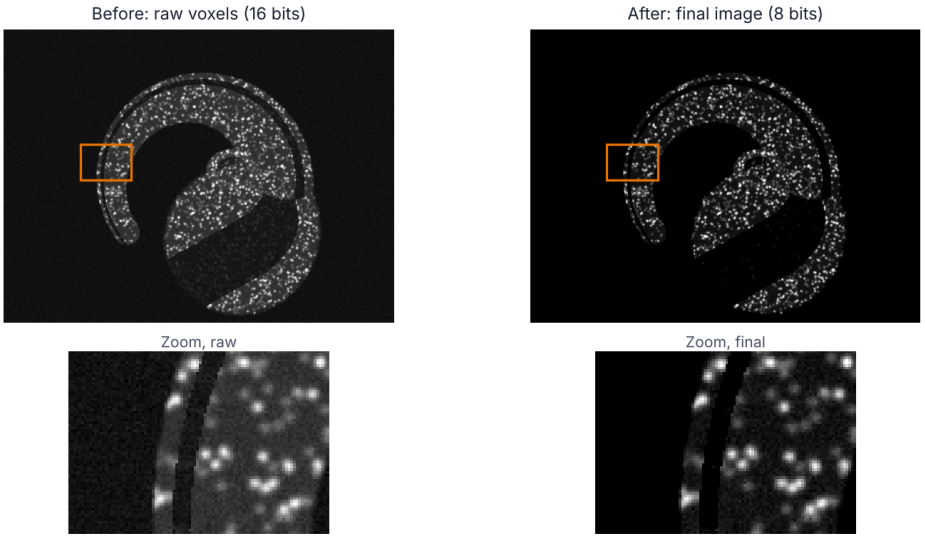
▲ Bars with a square cross-section, at 20,000 (bright) or 6,000 (faint). The 1 × 1 and 2 × 2 cross-sections disappear; from 3 × 3 on, the object is kept intact, even when faint.

| SHAPE OF THE OBJECT (IN VOXELS) | RESULT | WHY |
|---|---|---|
| **isolated point, 1 × 1 thread, sheet 1 voxel thick** | **erased** | the opening removes it; the median of its 27 neighbours is background |
| **2 × 2 thread, small 2 × 2 × 2 cube** | **erased** | same reason: fewer than 14 of the 27 neighbours are bright |
| **sheet 2 voxels thick** | kept | the opening removes it, but the median stays bright (18 neighbours out of 27) |
| **at least 3 voxels in every direction** | **kept intact** | it holds a complete cross: it enters the mask |

⚠️ **WARNING**

**Order of magnitude: about 3 voxels in every direction.** With the voxels of the demonstration dataset (1.2 × 1.2 × 3.0 µm), that is **3.6 µm in X and Y and 9 µm in Z**. An object present on only one or two slices is therefore at risk of being erased, however bright. Do the calculation with **your** voxel sizes; when in doubt, compare with the original `.ims`. In practice the microscope's optical blur spreads most objects over several voxels, and a thin object **touching** a thick one (within 3 voxels) is protected by the halo.

## 5.11 Before and after




Same slice z = 56, DAPI channel (demonstration set). The raw one is displayed from 0 to sig_max for a fair comparison.

▲ The same slice before (raw voxels displayed from 0 to the ceiling) and after cleaning. Bottom: zoom on the edge of the embryo. The grey grain of the background has disappeared; the nuclei are intact (demonstration dataset).

Look at the zoom: left of the edge, the grainy grey background has become pure black; on the right, the bright nuclei have the same shape and the same contrast.

## 5.12 What was tried, then abandoned

The pipeline once had more sophisticated methods, removed in version 0.12.0:

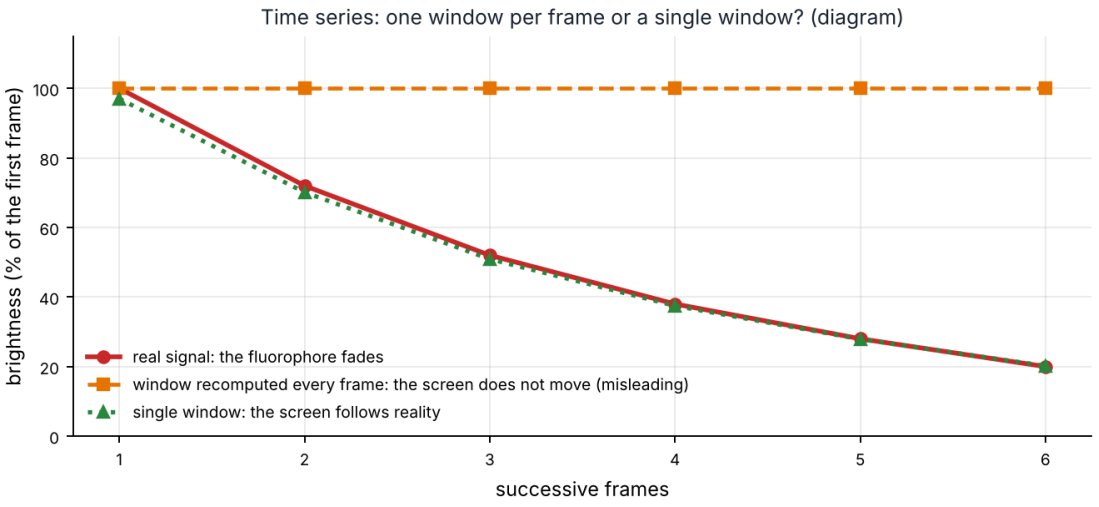
| METHOD | WHY ABANDONED |
|---|---|
| **Otsu threshold** | Mask subtraction: created artificial **large coloured blobs** |
| **Noise2Void (neural network)** | Same artefactual result; the embryo lost its natural texture |

❓ **WHY THIS CHOICE?**

Current choice: the pipeline **removes only the camera's global background** and keeps the microscope's native intensities. No neural network, no flat-field correction. A test on a reference dataset recovered the 97 SHA-256 fingerprints of the production packs, byte for byte.

## 5.13 Time series: a single window

For a movie (several images), should there be one window per image? **No: a single one for the whole series**, for each channel. Otherwise the image "flickers".




▲ Diagram (illustrative figures). A fluorophore fades (photobleaching). If each image is stretched over its own window, the screen stays constant while the signal collapses: that is wrong. With a single window, the screen dims like the real signal.

How is the single window measured?

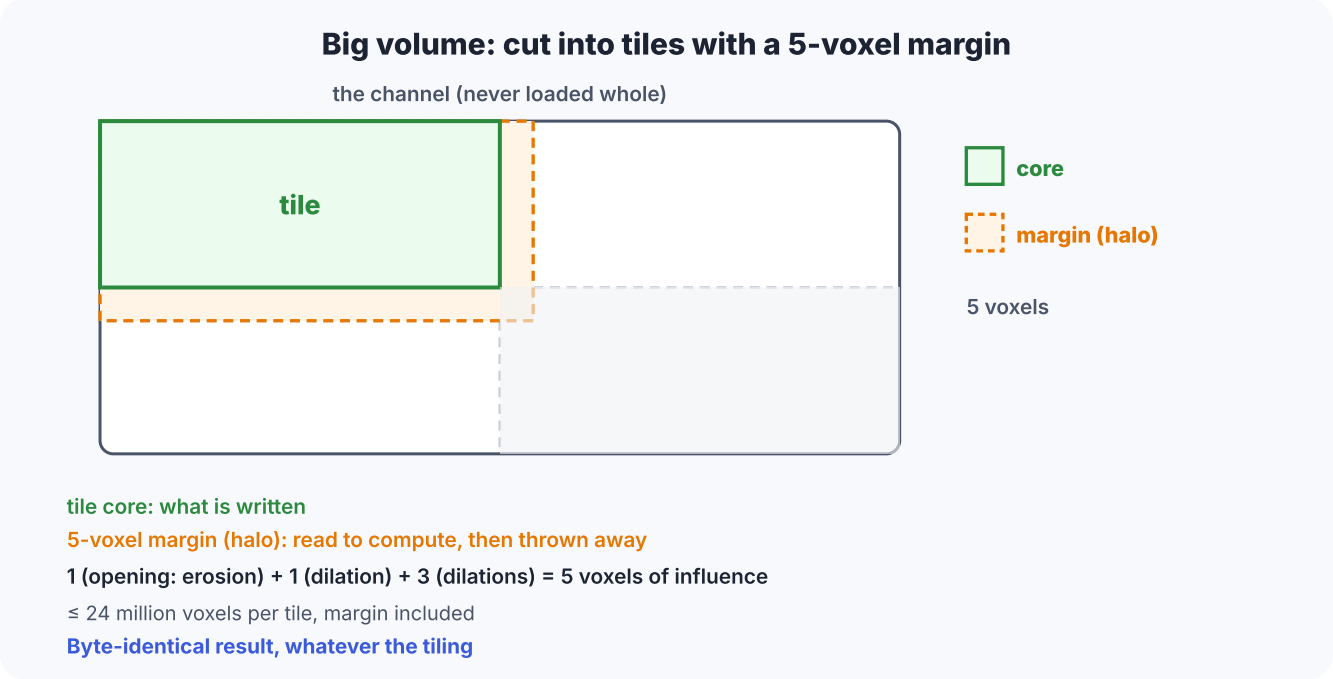
- up to **8 images** are sampled (evenly spaced, always including the first and the last);
- `bg_floor` = 99th percentile of the corners of all these images together;
- `sig_max` = 99.9th percentile of **only the voxels above** `bg_floor`, if there are at least **1,000** of them.

> ❓ **WHY THIS CHOICE?**
>
> In a sparse series, the signal covers only ~0.4% of the voxels: the 99.9th percentile of the whole volume would fall **inside the background** and saturate 15% of the real signal. By ranking only the voxels above the background, the saturated share falls to about 0.06% (measurements from the version 0.15.0 log).

Photobleaching is **measured, never erased**: the pipeline records one signal level per image and per channel (99.9th percentile of the raw data) in `metadata.json`. No viewer setting uses these levels yet: a fading series therefore really does look darker, and nothing “catches it up” on screen.

## 5.14 Large volumes: in tiles, without changing the result

A channel of 3789 × 3789 × 178 voxels is 2.56 billion voxels, i.e. 10.2 GB as 32-bit floats: it does not fit in memory. The pipeline therefore reads it **in tiles** of at most **24 million voxels** (margin included).




▲ Tiling. The 5-voxel margin gives the edge computations (opening, dilations) the neighbours they lack; only the core is written.

Why 5 voxels? The erosion (1) + the dilation of the opening (1) + the 3 dilations (3) spread a voxel's influence over 5 voxels of distance. The median needs 1 (covered by the same margin).

🔢 **WORKED EXAMPLE**

**Verified on the demonstration dataset** (3 channels): cut into 4 tiles (the default setting), then into **256 tiles** (0.5 M voxels), the 8-bit volume is **byte-for-byte identical** to the single-block computation, and to the dataset published by the pipeline (slices z = 0, 30, 56 and 100).

## 5.15 What this means for you

⚠️ **WARNING**

8-bit values are **relative**, not absolute. Each channel has its own window [ `bg_floor` ; `sig_max` ]; 128 on DAPI and 128 on Pecam1 do not represent the same quantity of light. Do not compare intensities between channels or between datasets.

- In a **time series**, values are comparable between images **of the same channel** (single window).
- The **original .ims file** is never modified; it is the source for any intensity measurement. It can be downloaded from the `download/` folder if the dataset was published with its original files.
- What can be erased: a **very weak** signal (below 1.1 × `bg_floor`) that does not form an extended area, and any object **thinner than about 3 voxels** in one direction, even a bright one (§ 5.10). That is the price of a perfectly black background. Inside the mask, however, the original values are kept.

📌 **REMEMBER**

1. `bg_floor` = 99th percentile of 8 corner cubes; `sig_max` = 99.9th percentile of one voxel in 4.
2. Mask = threshold 1.1 × `bg_floor` → opening (removes isolated points) → dilation × 3 (halo).
3. 3 × 3 × 3 median **outside the mask**; the signal stays intact. An object less than ~3 voxels thick is erased.
4. 8 bits = linear window [ `bg_floor` ; `sig_max` ] → [0; 255], truncated; background = exact 0.
5. One window for a whole movie; tiling does not change the result.

# 6. Shrinking the size without losing anything important

- An embryo of several gigabytes cannot travel over the Internet as it is: the pipeline **shrinks** it in four moves — 16 → 8 bits, a pyramid of resolutions, removal of empty bricks, **lossless** compression.
- **Format 5** adds, for display only, four lighter compressions (two of them lossy); what measures always reads the lossless version (section 7.11).
- None of these moves changes an intensity at the finest level: what you see at full resolution is **exactly** the result of the cleaning in chapter 5.
- On the demonstration dataset: **297 MB of raw voxels → 9.3 MB of bricks**, about 32 times less.

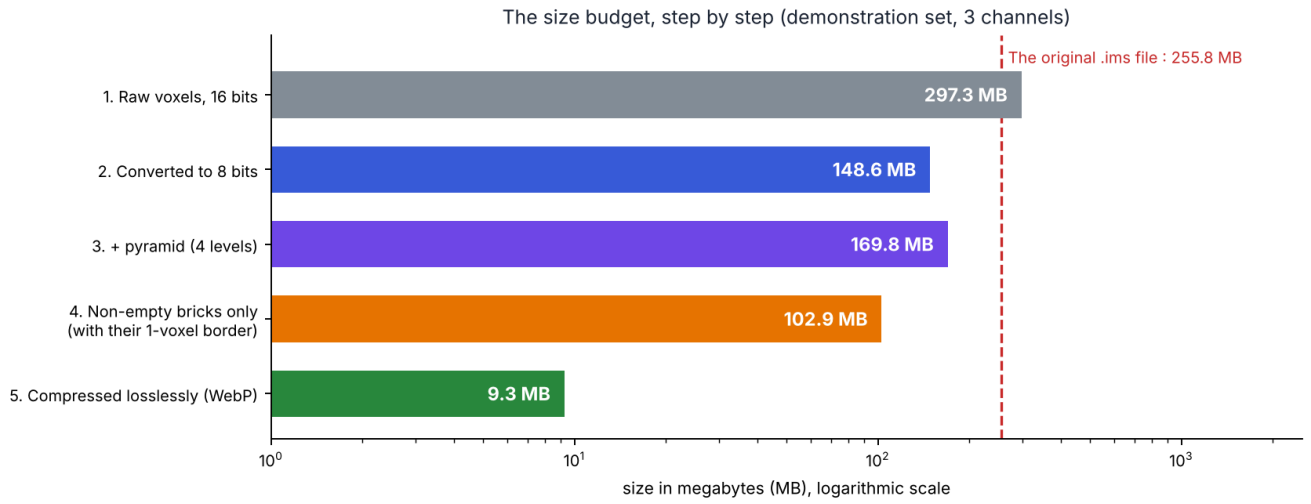
All the “demo” figures in this chapter are **measured** on the synthetic embryo `Embryo-E95-Em2-Pecam1-Sox2` (768 × 576 × 112 voxels, 3 channels), published by the real pipeline (format 4).

## 6.1 The size budget in numbers

Let us first take a **big** embryo from the lab: 3789 × 3789 × 178 voxels, 4 channels. That makes 10.2 billion voxels.

| STEP | CALCULATION | SIZE |
|---|---|---|
| **Raw voxels, 16-bit** | 10.22 G voxels × 2 bytes | **20.4 GB** |
| **Converted to 8-bit** | 10.22 G voxels × 1 byte | **10.2 GB** |
| **+ resolution pyramid** | × 1.286 for this layout | **13.1 GB** |
| **Real .ims file of this kind** | compressed by Imaris | 14.4 GB |

The last two steps (empty bricks removed, compression) depend on the image content. Let us measure them on the demonstration dataset, step by step:




▲ Size budget of the demonstration dataset. The axis is logarithmic: each tick is ten times the previous one.

**297 MB** raw 16-bit

**149 MB** as 8-bit

**103 MB** non-empty bricks

**9.3 MB** on disk

ℹ️ **GOOD TO KNOW**

The pyramid **adds** bytes (+14% here): it is the price of fast display. The gain comes from the next two steps: not storing emptiness, and compressing.

## 6.2 From 16 bits to 8 bits: dividing by two

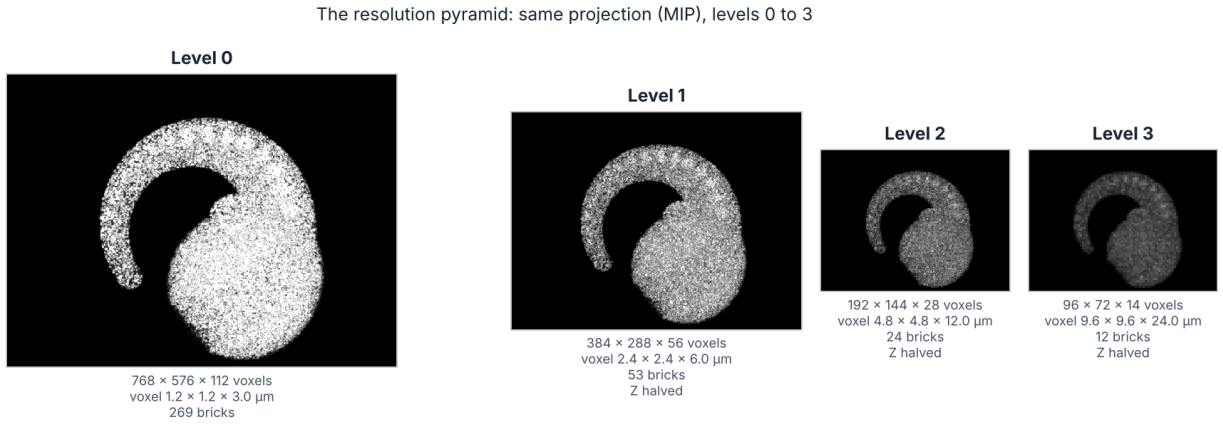
This is the window of chapter 5: a 2-byte voxel becomes a 1-byte voxel. We lose the fineness of the 65,536 levels, but the browser only shows 256: the screen cannot show more.

⚠️ **WARNING**

This is **the only step that loses information** (the nuances above 256 levels), and it takes place in chapter 5, not here. Everything that follows in this chapter is **lossless**.

## 6.3 The resolution pyramid

When you look at the whole embryo, you do not need all the voxels: the screen has only a few million pixels. The pipeline therefore prepares **several versions** of the image, smaller and smaller: the **levels**.




▲ The four levels of the demonstration dataset, drawn to scale (maximum-intensity projection of the DAPI channel). Same shape, fewer and fewer voxels.

> 💡 **A PICTURE TO UNDERSTAND**
>
> It is a **road map**: the map of the country for travelling, the map of the town for finding your way, the street plan for finding the door. We load only the map that is useful.

## The format 4 rule

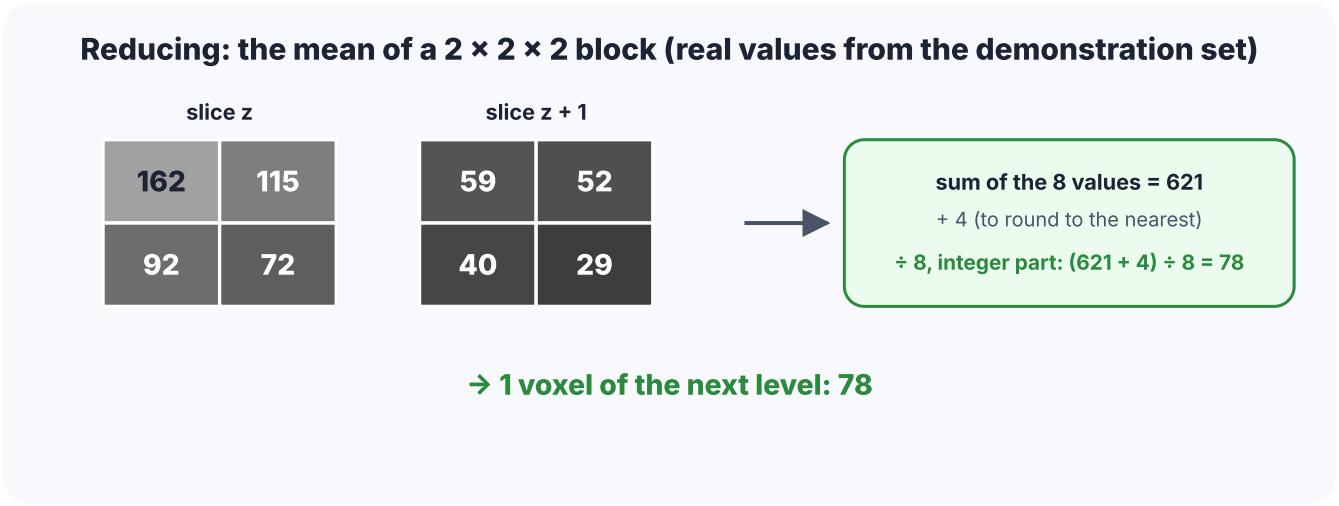
- **Level 0**: the cleaned image, **as it is** (never resampled).
- **Next level**: X and Y are **divided by 2** (rounded up) and their voxel doubles.
- **Z is divided by 2 only if** the voxel depth stays less than or equal to **1.5 times** the new XY voxel. Otherwise Z is kept.
- We stop when the longest XY side is **less than or equal to 128** voxels.

> ❓ **WHY THIS CHOICE?**
>
> A voxel that is too “flat” or too “elongated” distorts the image. Dividing Z only when it is still comparable to XY gradually brings the voxel back towards a **cube** (this is the convention of modern biomedical image formats, such as OME-Zarr).

## Computing one voxel of the next level

Each voxel of the next level is the **mean** of a 2 × 2 (× 2) block of voxels of the previous level. The computation is done in **integers**, **rounded to the nearest value, halves going up**: (sum + n ÷ 2) ÷ n, integer part, n being the number of voxels in the block.

> 🔢 **WORKED EXAMPLE**
>
> Real block from the demonstration dataset: 162, 115, 92, 72, 59, 52, 40, 29. Sum = **621**. We add 4 (half of 8), then divide by 8: (621 + 4) ÷ 8 = 78.1 → **78**. Without adding 4 we would have got 77: the bias would have darkened every level.




▲ Computing one voxel of the next level, on a real block from the demonstration dataset.

At the edge of the volume, a block may be only 1 voxel wide: we then average **only the voxels that exist**.

## The demonstration dataset's scale

Starting voxel: 1.2 × 1.2 × 3.0 µm. For each level, the Z rule is tested with the **new** XY voxel.

| LEVEL | VOXELS (X × Y × Z) | VOXEL (MM) | Z DIVIDED? |
|---|---|---|---|
| 0 | 768 × 576 × 112 | 1.2 × 1.2 × 3.0 | (base) |
| 1 | 384 × 288 × 56 | 2.4 × 2.4 × 6.0 | yes: 3.0 ≤ 1.5 × 2.4 = 3.6 |
| 2 | 192 × 144 × 28 | 4.8 × 4.8 × 12.0 | yes: 6.0 ≤ 1.5 × 4.8 = 7.2 |
| 3 | 96 × 72 × 14 | 9.6 × 9.6 × 24.0 | yes: 12.0 ≤ 1.5 × 9.6 = 14.4 |

We stop at level 3: 96 ≤ 128.

🔧 FOR THE CURIOUS

**When Z is not divided.** For a 3789 × 3789 × 178 stack with a voxel of (0.33; 0.33; 1.876 µm) (*illustrative* voxel sizes), level 1 keeps Z: 1.876 > 1.5 × 0.66 = 0.99. Level 2 divides it: 1.876 ≤ 1.5 × 1.32 = 1.98. The result is 3789 × 3789 × 178 → 1895 × 1895 × 178 → 948 × 948 × 89 → 474 × 474 × 45 → 237 × 237 × 23 → 119 × 119 × 12 (6 levels).

ℹ️ GOOD TO KNOW

I rebuilt this pyramid with the pipeline's functions from the result of chapter 5, then compared it with the published bricks: **358 bricks out of 358 identical**, voxel for voxel.

## And in format 5?

Format 5 (section 7.11) changes the pyramid rule:

| | FORMAT 4 | FORMAT 5 |
|---|---|---|
| **level sizes** | halved each time | long side **exactly** 4096, 2048, 1024, 512, 256 |
| **slices (Z)** | halved while the voxel stays ≈ cubic | **all kept** at every level |
| **computation** | average of 2 × 2 (× 2) voxels of the previous level | **exact area average** of the native voxels, from level 0 |
| **demonstration dataset** | 768, 384, 192, 96 (and 112, 56, 28, 14 slices) | 768 (native), 512, 256, always 112 slices |

In both cases, level 0 stays the cleaned image as it is.

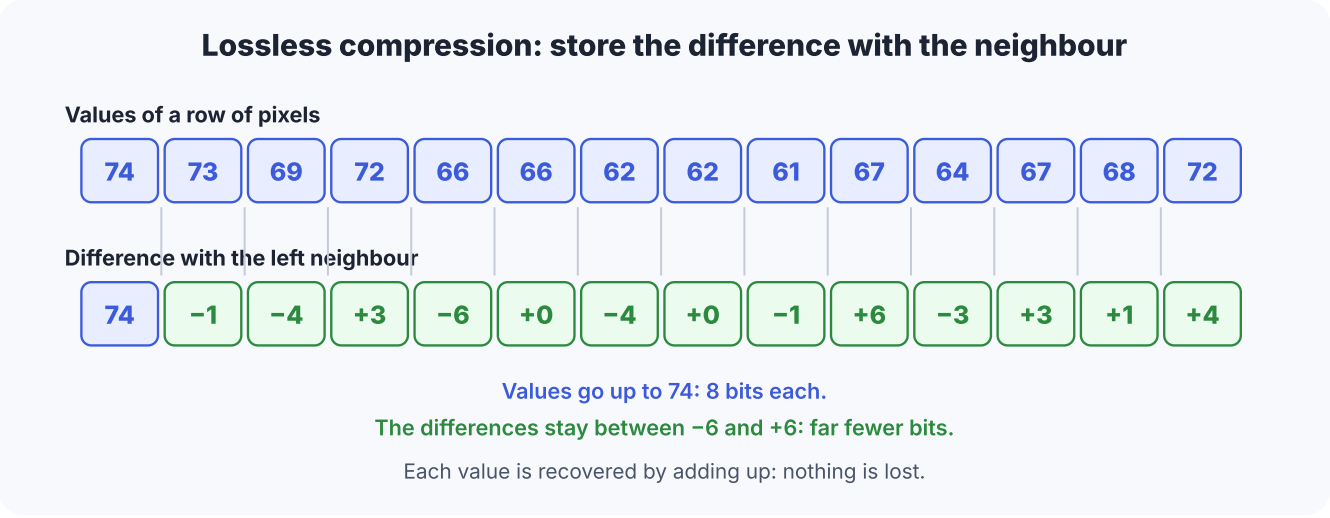
## 6.4 Compressing without losing anything

### Why “lossless”?

**Lossy** compression (JPEG) slightly changes the values to save space: acceptable for a holiday photo, not for a **measurement**. **Lossless** compression lets you recover every voxel identically.

### How it works, in plain words

The idea: a voxel looks like its neighbour. Rather than storing every value, we store **the difference from the neighbour**, which is small and compresses well.




▲ A real row of 14 voxels from the Sox2 channel (demonstration dataset). The differences are small; adding them up in order gives back the original values.

🔧 **FOR THE CURIOUS**

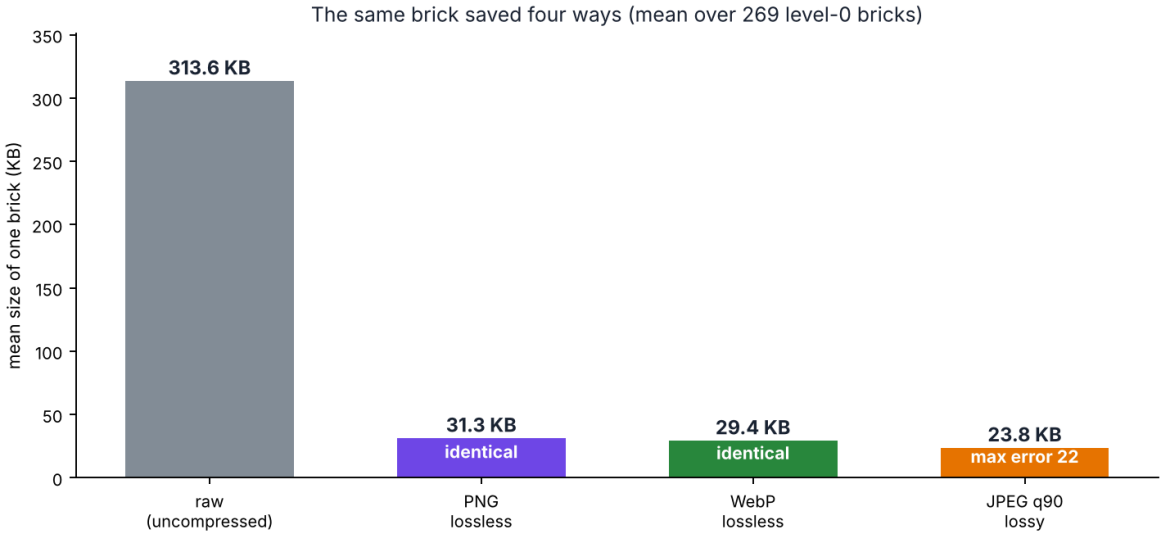
Real compressors are subtler: they predict a voxel from several neighbours (left, above…), then code the gaps economically, the most frequent first. The principle remains this one, and it is exactly reversible.

## Bricks are lossless WebP images

A 3D brick is not a 2D image. The pipeline therefore lays its 66 slices side by side in a **594 × 528 pixel mosaic** (9 columns × 8 rows; chapter 7), and saves the mosaic as **lossless WebP**.

## The measurement: four ways of saving the same brick

I took the **269 non-empty level-0 bricks** (3 channels) of the demonstration dataset and saved each 594 × 528 pixel mosaic in four ways.

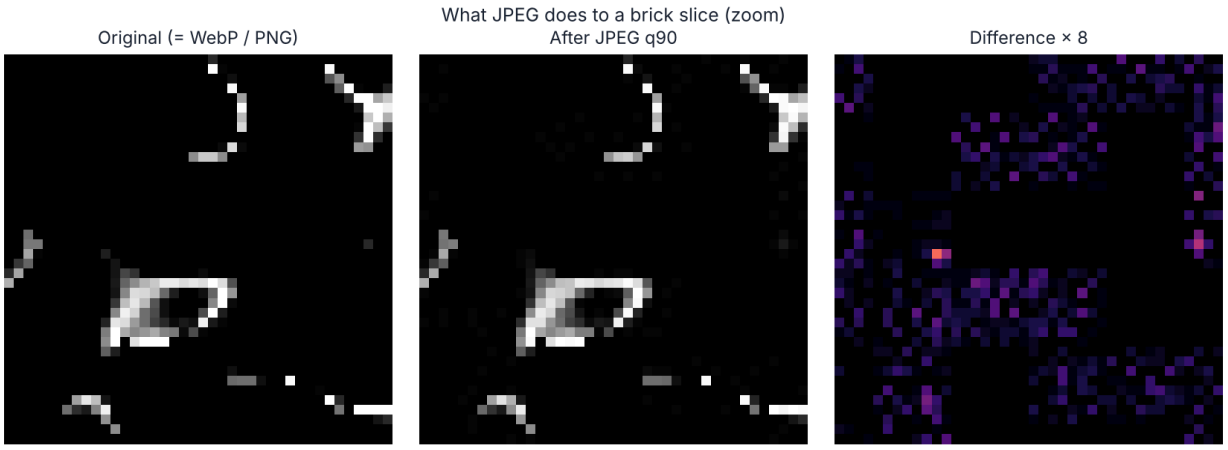



▲ Average size of a brick. Raw: 313.6 KB. PNG (best compression): 31.3 KB. Lossless WebP (pipeline setting): 29.4 KB. JPEG quality 90: 23.8 KB, but with an error of up to 22 grey levels.

| FORMAT | AVERAGE SIZE | EXACT? |
|---|---|---|
| **Raw (uncompressed)** | 313.6 KB | yes |
| **PNG, level 9 optimised** | 31.3 KB | yes |
| **Lossless WebP (pipeline's choice)** | **29.4 KB** | yes |
| **JPEG, quality 90** | 23.8 KB | **no** |

⚠️ **WARNING**

**JPEG is the smallest, but it is wrong.** On these bricks: a maximum error of 22 levels (out of 255), **11.7%** of voxels modified, and **1,644,202 voxels that were exactly 0 became non-zero** (2.2% of the background). Yet "exact 0" is what lets us recognise an empty brick (6.5): JPEG would destroy it.




▲ Zoom on a brick slice (Pecam1 channel): after JPEG, faint grey halos appear around fine structures, where the original is perfectly black. The difference is amplified 8 times on the right.

❓ **WHY THIS CHOICE?**

**Why WebP rather than PNG?** On these bricks, lossless WebP is only **6% smaller** than the best PNG (29.4 versus 31.3 KB). The code gives another main reason: the browser **decodes WebP natively** and fast ( `createImageBitmap` ), whereas no 3D image format exists. The 2D slices of `planes/` , on the other hand, are PNG: the viewer reads them with a small home-made PNG decoder, never through the browser's canvas, whose readback is not guaranteed to be exact.

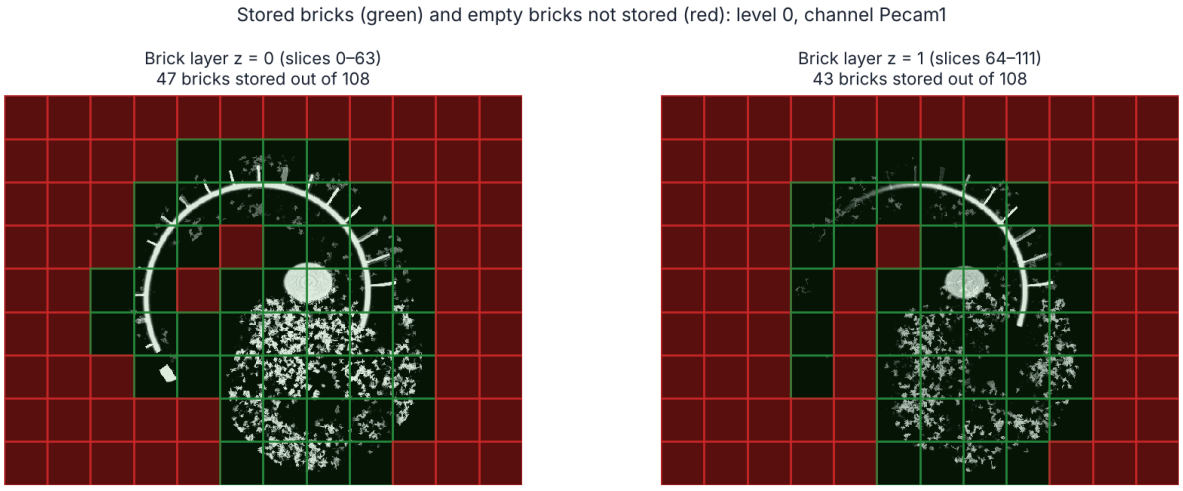
ℹ️ **GOOD TO KNOW**

The pipeline's bytes are reproducible: re-encoding the mosaics with the same settings (Pillow, lossless WebP, method 4), I got back **exactly the same bytes** as in the published files. Only the version of the WebP library matters: that is why Pillow is pinned.

## 6.5 Not storing emptiness

A mouse embryo does not fill its frame. Many bricks contain only black: why store them?

**The rule (format 4):** a brick is stored **if and only if** its 64 × 64 × 64 voxel interior contains **at least one voxel ≥ 1**. No tolerance: a single faint voxel is enough to keep it.




▲ Level 0, Pecam1 channel, two layers of bricks. Green: stored brick. Red: empty brick, not stored. The bricks are drawn over a maximum-intensity projection of the layer.

The exact count, read from `index.bin` of the demonstration dataset (3 channels):

| LEVEL | POSSIBLE BRICKS | STORED BRICKS |
|---|---|---|
| **0** | 648 (216 per channel) | **269** (DAPI 100, Pecam1 90, Sox2 79) |
| **1** | 90 | 53 |
| **2** | 27 | 24 |
| **3** | 12 | 12 |
| **Total** | **777** | **358** (46%) |

🔢 **WORKED EXAMPLE**

The Pecam1 channel has only **0.75%** non-zero voxels, yet **90 bricks out of 216** at level 0 are stored: a few fine vessels cross many bricks. A stored brick is not full; it contains at least one voxel.

ℹ️ **GOOD TO KNOW**

An **absent** brick is zero everywhere, including on its 1-voxel border. The viewer does not download it: empty space costs neither bytes nor transfer time (chapter 10).

## 6.6 Final size: from the .ims to the published folder

▲ From the original file to the published folder, for the demonstration dataset. Measurements in bytes on disk (sum of the files).

| CONTENT | SIZE |
|---|---|
| Original `.ims` file (compressed by Imaris) | 255.8 MB |
| `bricks/` (3D display) | 9.28 MB |
| `planes/` (native slices, PNG) | 8.64 MB |
| `mips/` (per-layer projections) | 0.47 MB |
| `thumbnail.webp` + `metadata.json` | 0.04 MB |
| **Published folder (excluding `download/`)** | **18.4 MB** |

**14×** smaller than the .ims

**28×** for the 3D view alone

**46%** bricks stored

⚠️ **WARNING**

The `download/` folder (original file, TIFF, projections…) is not counted: it is not used for display. It weighs 506 MB here, because it contains the .ims.

📌 **REMEMBER**

- 16 → 8 bits is the **only** loss, decided in chapter 5; everything else is exact.
- The pyramid: X, Y ÷ 2 at each level, Z ÷ 2 only while its voxel ≤ 1.5 × the XY voxel; integer mean rounded up; stop at 128 voxels.
- **Lossless WebP** compression: JPEG is smaller but damages the values and the “exact 0”.
- A brick is stored only if its interior contains a voxel ≥ 1: 358 bricks stored out of 777 possible.

# 7. Cutting into bricks

- A volume of several gigabytes cannot be loaded in one go: it is cut into **bricks** (small cubes of 64 × 64 × 64 voxels) that the browser fetches one by one.
- Each brick becomes **a single WebP image** (lossless); the bricks are stored in **packs**, and found again thanks to an **index** of a few kilobytes.
- Two side structures, `planes/` and `mips/`, make slices and projections fast; the whole forms **data format 4**.
- **Format 5** (section 7.11) stores the same volume as a **matrix**: several resolutions × five compressions, plus three copies for fast cuts.

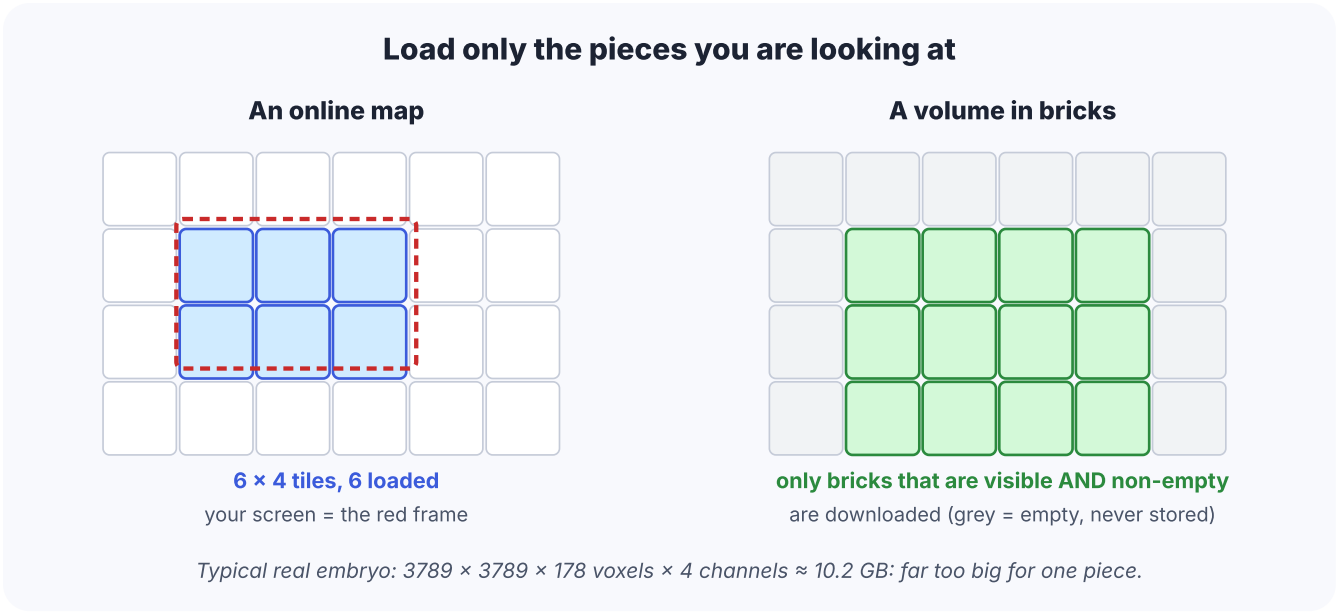
In this chapter, the data images (mosaic, seam, packs…) all come from the **demonstration dataset**: the synthetic embryo `Embryo-E95-Em2-Pecam1-Sox2` (768 × 576 × 112 voxels, 3 channels), published by the real pipeline. It is not lab data.

## 7.1 Why cut a volume?

A typical lab embryo is 3789 × 3789 × 178 voxels over 4 channels: about **10.2 GB** once reduced to 8 bits. No browser can swallow that in one go, and at any moment you are looking at only a small part of it.

💡 **A PICTURE TO UNDERSTAND**

**An online map.** When you zoom into a town, the site does not download the planet: it loads only the **tiles** that are in your window, at the right level of detail. Bricks are the tiles of your embryo, with one more dimension.



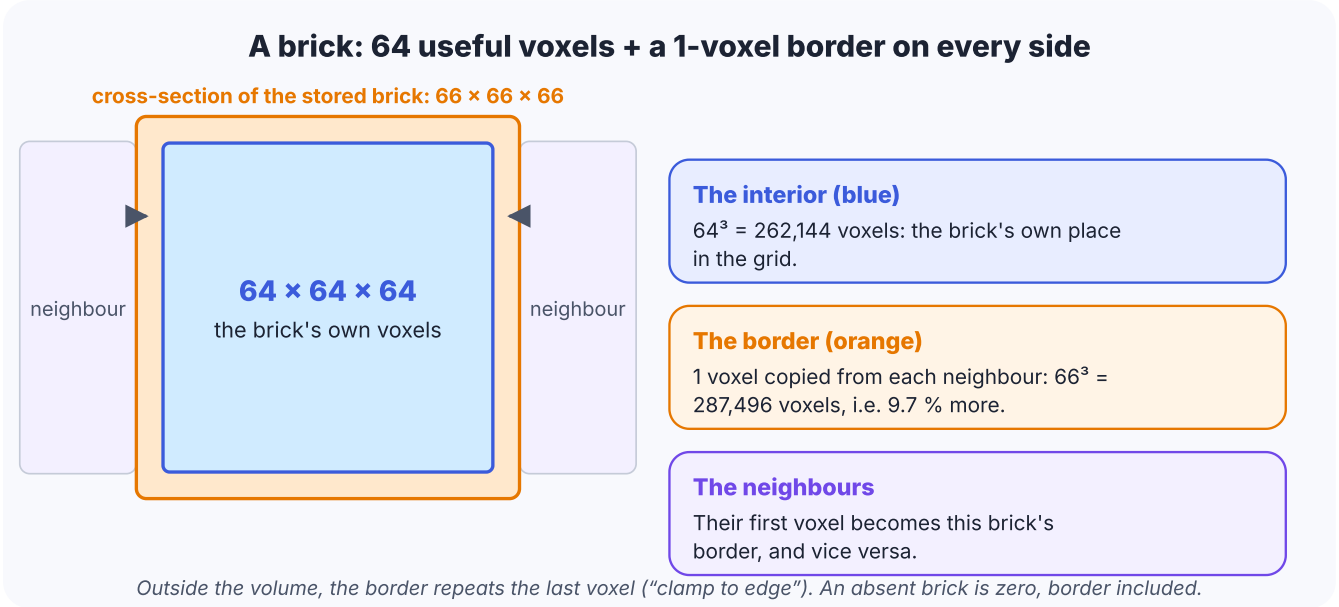

▲ Like the tiles of a map, only the visible, non-empty bricks are downloaded.

Three advantages, all visible on screen:

- **the start is fast**: the first bricks arrive in a fraction of a second;
- **memory is bounded**: the graphics card receives only what it can hold (chapter 10);
- **emptiness costs nothing**: a completely black brick is never written to disk.

## 7.2 A brick: 64 useful voxels and a border

A brick covers **64 voxels** in each direction. But it is stored with **one extra voxel on each side**: 64 + 1 + 1 = **66**. This border voxel is a copy of the neighbouring voxel, taken from the next brick over.

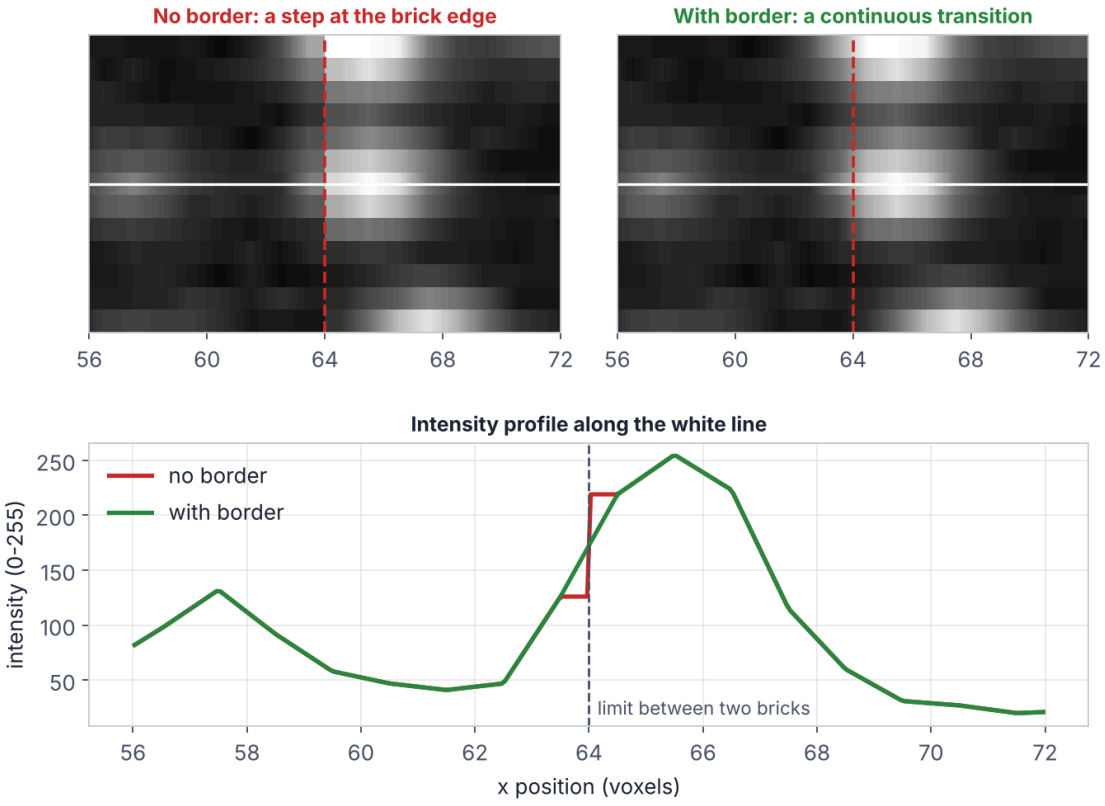



▲ The core of the brick (blue) is surrounded by a one-voxel border (orange) copied from the neighbouring bricks (violet).

❓ **WHY THIS CHOICE?**

**What is the border for?** To draw a smooth image, the graphics card *interpolates*: the value at a point is a weighted blend of the surrounding voxels. At the edge of a brick, the neighbour on the other side is missing. Without a border, the graphics card would have to repeat the last voxel: you would see a regular grid at the brick limits. With the border, the neighbour is there, and the image is continuous.

The next figure shows it on real data: the same intensity profile, straddling the limit between two bricks (at voxels 56 to 72).




▲ Without a border (red) the value plateaus then jumps; with a border (green) it varies without a break. Demonstration dataset, DAPI channel.

🔢 **WORKED EXAMPLE**

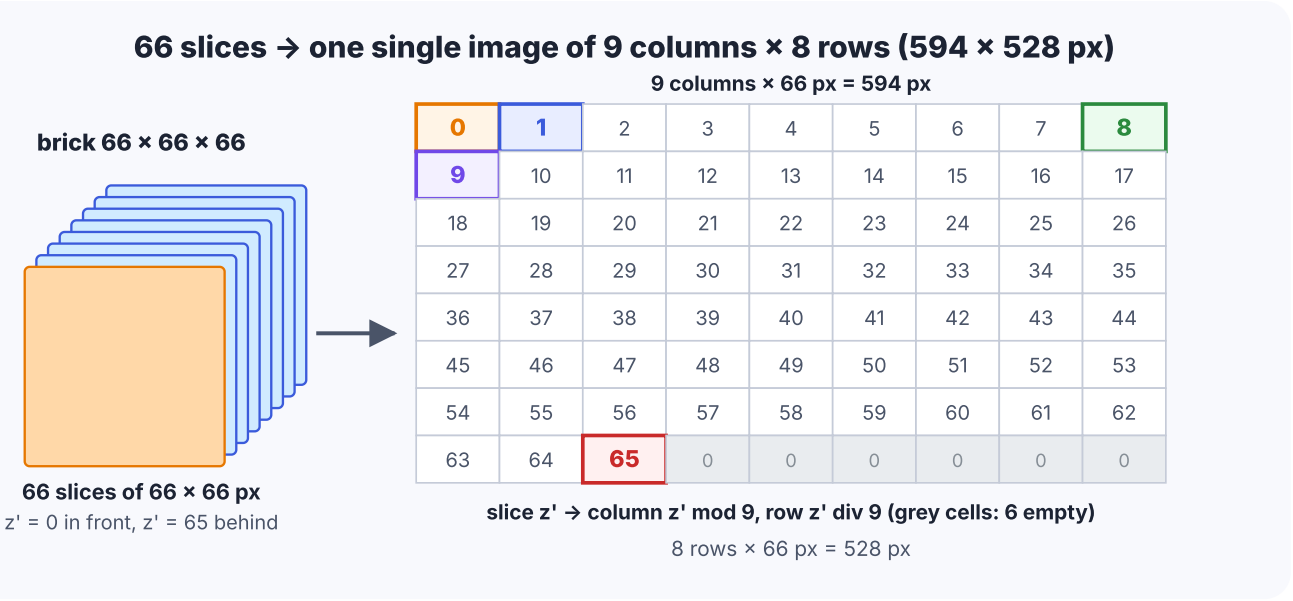
**The cost of the border.** $66^3$ = 287,496 voxels against $64^3$ = 262,144: **9.7% more** stored, in exchange for seamless images at every quality level.

Two rules for edge cases:

- **outside the volume**, the border repeats the last voxel (“clamp to edge”);
- an **absent brick** (empty) is zero everywhere, border included: a border is never rebuilt from the neighbouring bricks.

## 7.3 From a cube to an image: the mosaic

The browser can decode a 2D image very fast, but has no 3D image format. The trick: the brick's 66 slices are **laid flat** into a single image, like a contact sheet of 66 photos.




▲ The 66 slices of a brick arranged in 9 columns × 8 rows; the last 6 cells stay empty.

Each slice is 66 × 66 pixels; the grid is therefore **9 × 66 = 594** pixels wide and **8 × 66 = 528** tall, in 8-bit greyscale. The rule is simple:

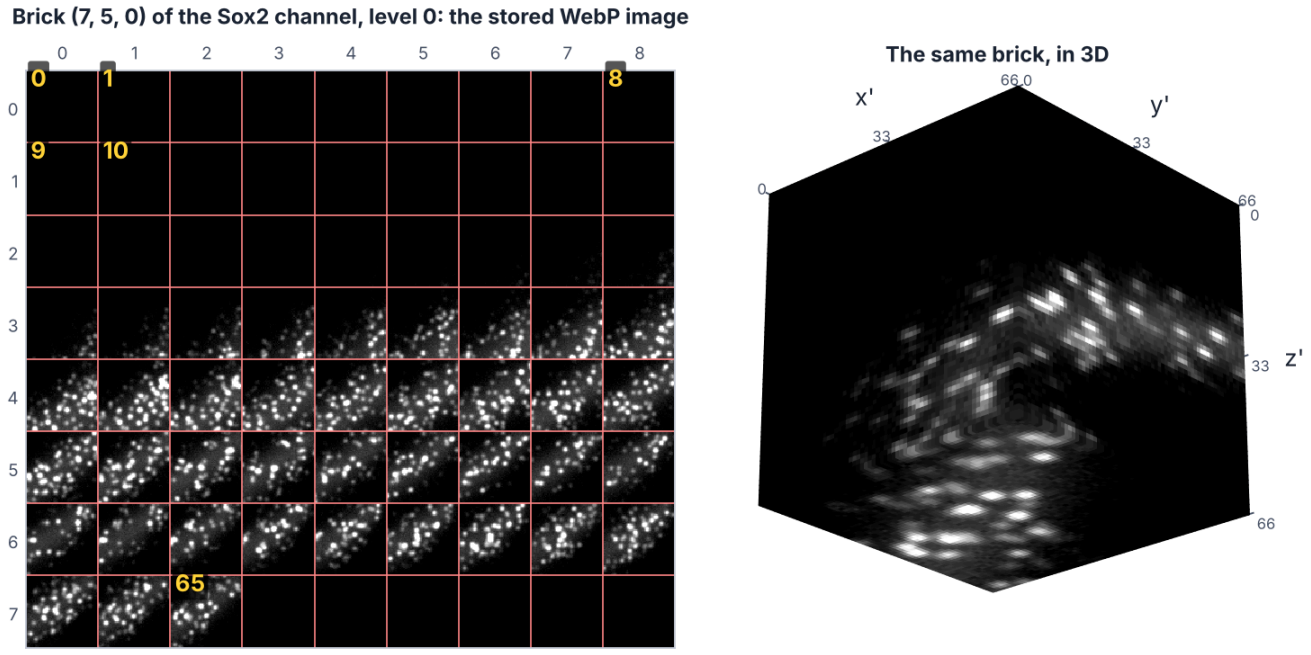
🔢 **WORKED EXAMPLE**

**Where is slice z′?** Column = z′ modulo 9, row = z′ divided by 9 (integer division).

| SLICE Z′ | COLUMN | ROW | TOP-LEFT CORNER (PX) |
|---|---|---|---|
| 0 | 0 | 0 | (0; 0) |
| 1 | 1 | 0 | (66; 0) |
| 8 | 8 | 0 | (528; 0) |
| 9 | 0 | 1 | (0; 66) |
| 65 | 2 | 7 | (132; 462) |

For a voxel (x′, y′, z′) of the $66^3$ brick, its pixel in the image is: **column = (z′ mod 9) × 66 + x′**, **row = (z′ div 9) × 66 + y′**. Interior voxel number *i* sits at index *i* + 1, because of the border.

Here is a real brick from the demonstration dataset, **as it is stored**, next to its 3D cube.



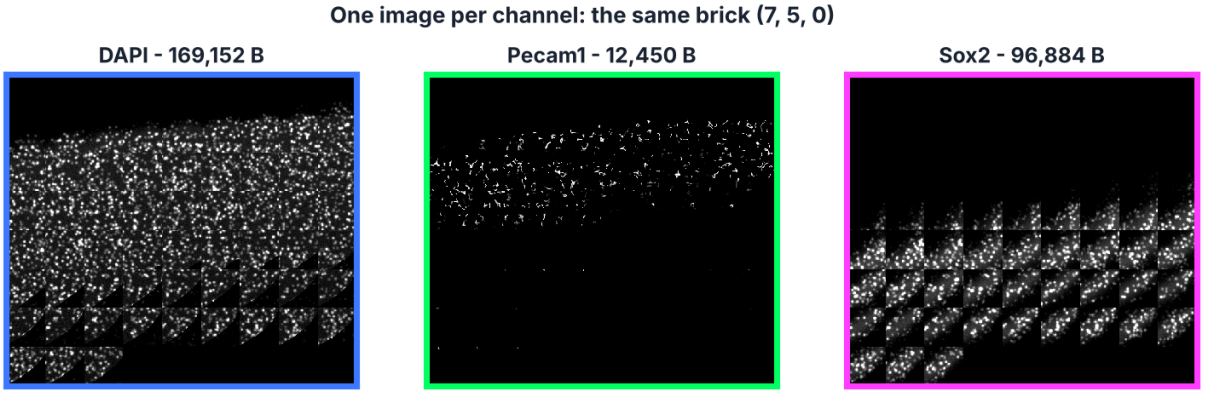

▲ Brick (7, 5, 0) of the Sox2 channel: the decoded WebP image, with the numbers of a few slices, and the reconstructed cube.

ℹ️ **GOOD TO KNOW**

The first slices are black because the demonstration embryo occupies only the bottom of this brick. Cells 66 to 71 (the last 6, bottom right) are always empty: 72 places for 66 slices.

## One image per channel

Each channel has its **own** bricks: brick (7, 5, 0) exists in three independent copies (DAPI, Pecam1, Sox2). The browser assembles them in colour at the last moment. A sparse channel, like Pecam1 here, gives a much smaller image.




▲ The three channels of the same brick: same geometry, different contents and weights.

🔧 **FOR THE CURIOUS**

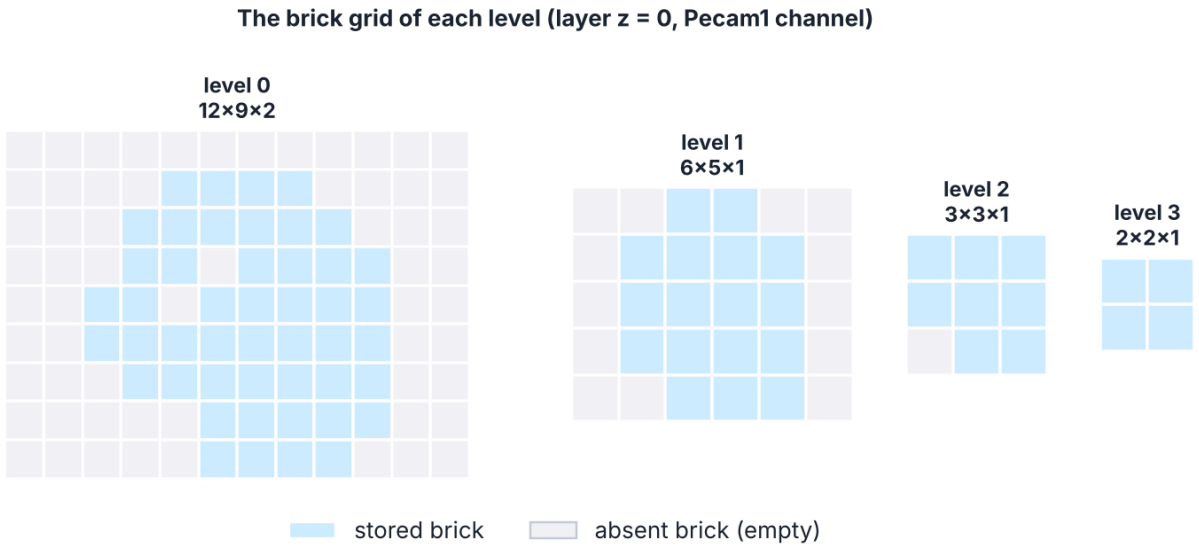
**Encoding.** Each mosaic is saved as *lossless* WebP (Pillow library, `lossless=True`, quality 75, method 4): the voxel read back is identical to the voxel written. A brick is kept only if its $64^3$ interior contains **at least one voxel ≥ 1**; otherwise it is "absent" (length 0 in the index). This rule is computed channel by channel.

## 7.4 The brick grid of each level

Chapter 6 built the resolution pyramid. At each level, the volume is cut into a **grid** of bricks: the number of voxels divided by 64, rounded up.

| LEVEL | VOXELS (X, Y, Z) | BRICK GRID | STORED BRICKS (3 CHANNELS) |
|---|---|---|---|
| 0 | 768 × 576 × 112 | 12 × 9 × 2 | 269 |
| 1 | 384 × 288 × 56 | 6 × 5 × 1 | 53 |
| 2 | 192 × 144 × 28 | 3 × 3 × 1 | 24 |
| 3 | 96 × 72 × 14 | 2 × 2 × 1 | 12 |

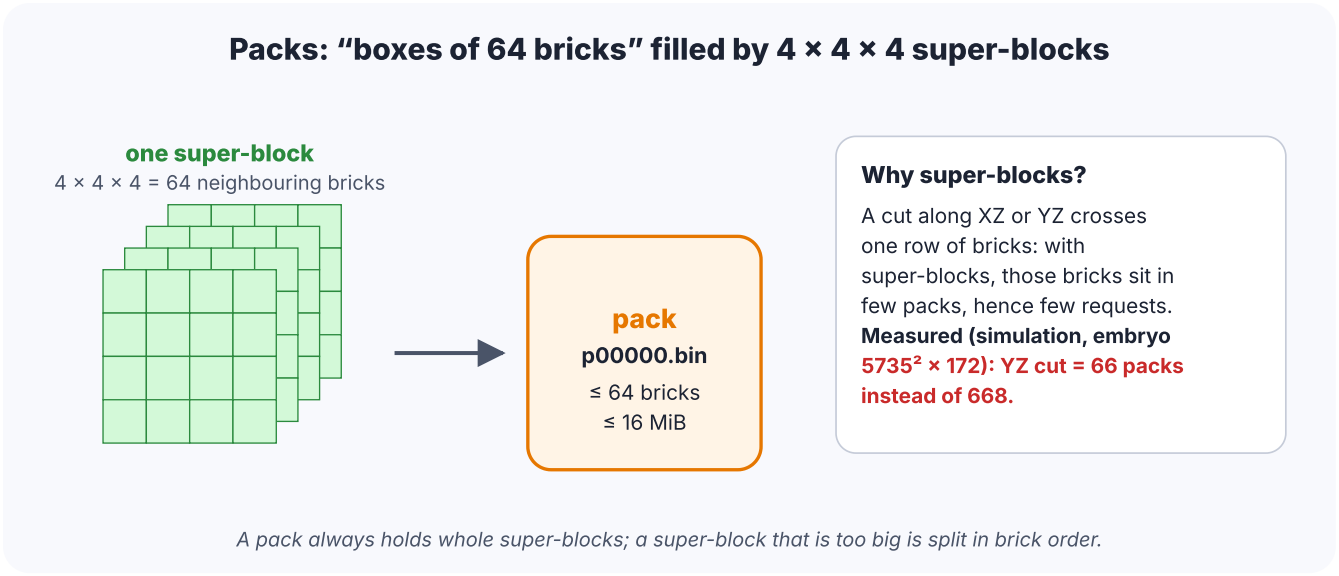
The level-0 grid has 12 × 9 × 2 = 216 cells per channel, but only 100 (DAPI), 90 (Pecam1) and 79 (Sox2) contain signal: the other corners are empty.




▲ The brick grid of each level for one layer and one channel: in grey, the bricks that were never written.

## 7.5 Packs: boxes of 64 bricks

Thousands of small files would be slow to serve. The bricks of the same level and the same channel are therefore grouped into **packs**: `bricks/l0/c1/p00000.bin` is the first pack of level 0, channel 1.



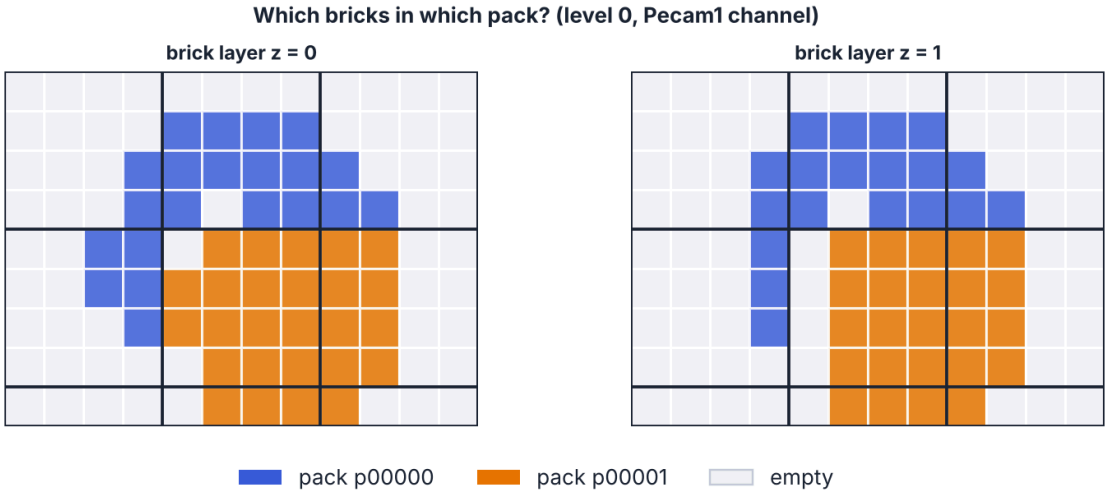

▲ A pack holds whole super-blocks of 4 × 4 × 4 bricks: a cut crosses few packs.

The filling rules:

- bricks are sorted into **super-blocks of 4 × 4 × 4** neighbouring bricks;
- a pack contains **whole super-blocks**, at most **64 bricks** or **16 MiB**;
- if a single super-block exceeds the limit, it is split in brick order.

> 💡 **A PICTURE TO UNDERSTAND**
>
> **A house move.** You pack boxes room by room (the super-blocks), not at random into the lorries. When you look for “everything in the kitchen”, you only need to open one lorry.

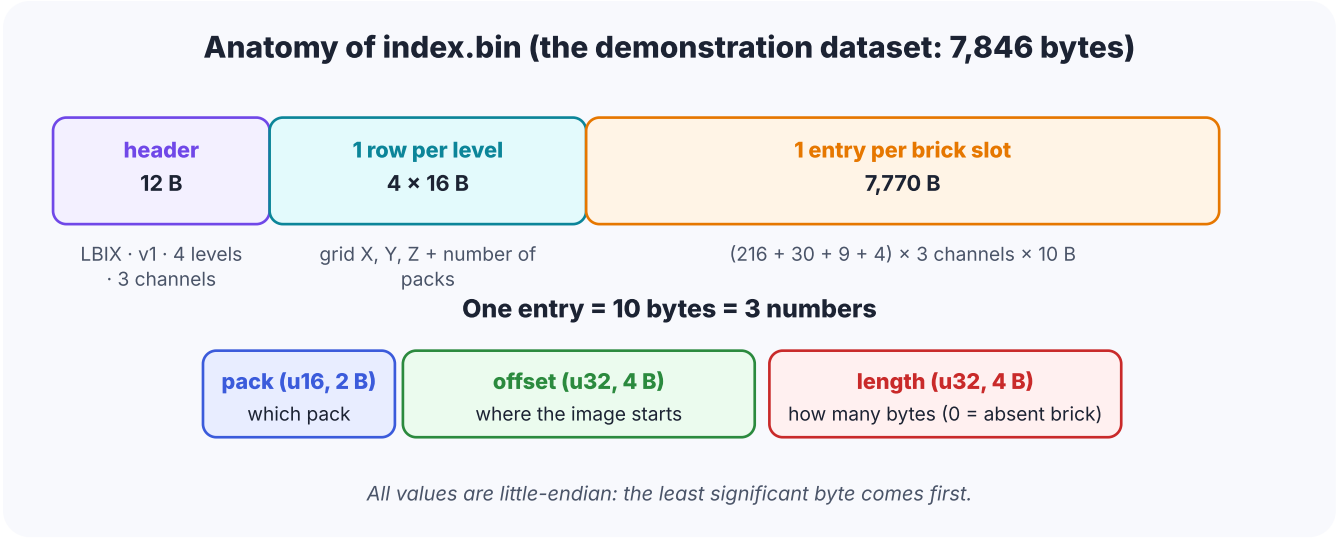
In the demonstration dataset, level 0 of the Pecam1 channel fits in **two packs** (blue and orange below): the black lines are the super-block limits.




▲ Which bricks are in which pack, for the two layers of level 0 (Pecam1 channel).

## 7.6 The index and the manifest

To fetch a brick, the browser must know **which pack** it is in, **where it starts** and **how much it weighs**. These three numbers are in `bricks/index.bin`, a small binary file: **10 bytes per brick cell**.




▲ An index.bin file: a header, one row per level, then a 10-byte entry per brick cell.

Here are the real bytes of the demonstration dataset, decoded by hand: the header, then brick (7, 5, 0) of each channel.

| WHAT WE READ | BYTES (HEXADECIMAL) | DECODED VALUE |
|---|---|---|
| **start of the file** | `4c 42 49 58` | "LBIX" (the signature) |
| **levels · channels** | `04 00` · `03 00` | 4 levels · 3 channels |
| **level 0** | `0c 00 00 00 09 00 00 00 02 00 00 00 06 00 00 00` | grid 12 × 9 × 2, 6 packs |
| **brick (7, 5, 0), DAPI** | `01 00 ce d3 07 00 c0 94 02 00` | pack 1, start 512,974, length 169,152 |
| **brick (7, 5, 0), Pecam1** | `01 00 88 1c 01 00 a2 30 00 00` | pack 1, start 72,840, length 12,450 |
| **brick (7, 5, 0), Sox2** | `00 00 a0 e2 03 00 74 7a 01 00` | pack 0, start 254,624, length 96,884 |

🔧 **FOR THE CURIOUS**

**Reading a number.** The bytes are "little-endian": `c0 94 02 00` is read backwards, `0x000294c0` = 169,152. A length of **0** means "absent brick" (the pack and the start are then 0 too). This file is 12 + 4 × 16 + 7,770 = **7,846 bytes**; on a large embryo, the former JSON manifest weighed 7.6 MB against 233 KB for the binary index.

Next to the index, `bricks/manifest.json` describes the brick set in plain text. Real excerpt (abridged):

```
{"schema":"iribhm-bricks-v3", "formatVersion":4, "channels":3,
 "brickSize":64, "apron":1,
 "brickPacking":{"mode":"grid","cols":9,"rows":8,"slice":66},
 "encoding":"webp-lossless",
 "levels":[{"level":0, "dimensions":{"x":768,"y":576,"z":112},
            "voxelSize":{"x":1.2,"y":1.2,"z":3.0},
            "gridSize":{"x":12,"y":9,"z":2}, "brickCount":269}, ...],
 "index":{"url":"index.bin", "bytes":7846, "sha256":"aea09808…"},
 "histograms":[...]}
```


The index's `sha256` signature is checked before use: a tampered index is refused. The `histograms` feed the contrast sliders (chapters 8 and 11).

## 7.7 The tree of a published dataset

Here is the real content of the demonstration dataset's folder (abridged; sizes are those on disk).

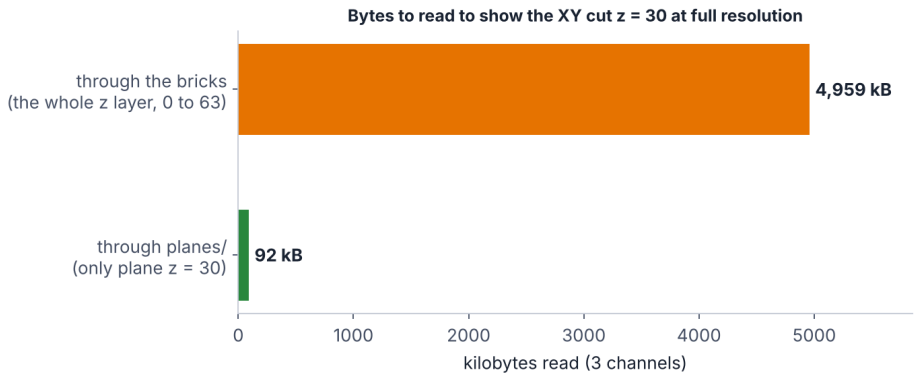
```
3d/Embryo-E95-Em2-Pecam1-Sox2/
├── metadata.json            1.8 KB   the dataset's facts
├── thumbnail.webp          33.6 KB   the thumbnail
├── bricks/                  9.3 MB   the bricks (format 4)
│   ├── manifest.json        3.7 KB
│   ├── index.bin            7.8 KB
│   ├── l0/ c0/ p00000.bin  1.4 MB    level 0, channel 0
│   │       │   p00001.bin  3.9 MB
│   │       c1/ …  c2/ …
│   └── l1/ l2/ l3/ …                 coarser levels
├── planes/                  8.6 MB   one file per Z plane
│   ├── manifest.json
│   └── z00000.bin … z00111.bin       (112 files)
├── mips/                  473 KB     one file per layer of 64 planes
│   ├── manifest.json
│   └── l00000.bin  l00001.bin
└── download/                         (see chapter 8)
```

> ℹ️ **GOOD TO KNOW**
>
> The levels break down as follows: level 0 = 7.9 MB, level 1 = 1.1 MB, level 2 = 0.18 MB, level 3 = 0.03 MB. The coarse levels are tiny: that is why a first image appears almost instantly.

## 7.8 `planes/`: an XY slice without reading 64 planes

Bricks are cubes. To display **one** XY slice at full resolution (for example for a Studio figure), you would have to read all the bricks of the layer, that is **64 planes of data** to use just one.

The `planes/` folder therefore holds level 0 **re-cut plane by plane**: one `zNNNNN.bin` file per Z plane (`z00000.bin`, `z00001.bin`...). Each plane is cut into **512 × 512** pixel tiles, saved as lossless PNG; a completely black tile takes no bytes at all.




▲ Bytes read for slice z = 30 of the three channels: 5,078 KB through the bricks against 94 KB through planes/ (54 times less).

🔢 **WORKED EXAMPLE**

**The demonstration dataset's figure.** The XY slice z = 30 of the three channels takes 5,078,326 bytes via the bricks of layer 0, against **94,053 bytes** via `planes/z00030.bin`. On a large lab embryo, a slice like this represented about 190 MB, i.e. 40 seconds at 5 MB/s.

The result is **pixel for pixel identical** to the bricks, which remain the fallback. `planes/` is used only for XY slices; the 3D view and the XZ, YZ and oblique slices still use the bricks. Price: about 1.3 times the size of the native level on disk.

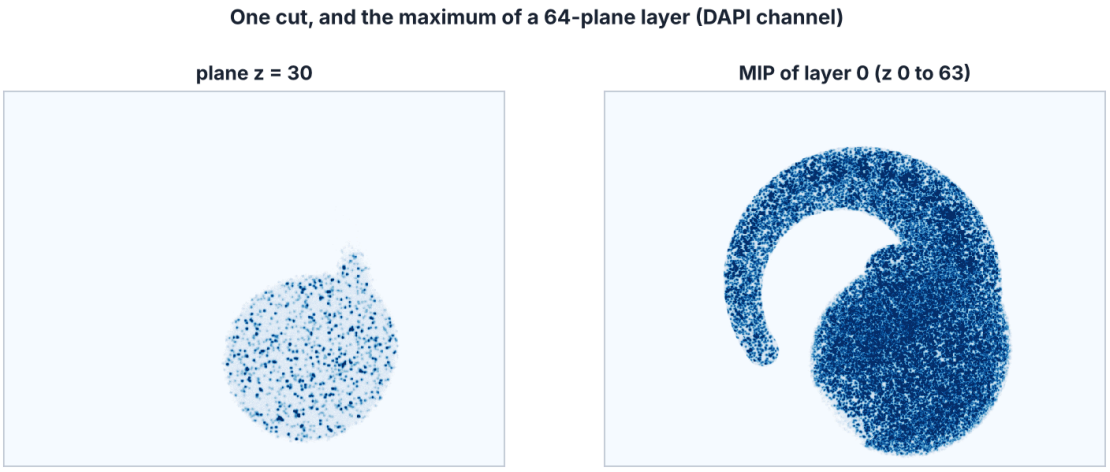
🔧 **FOR THE CURIOUS**

A plane file starts with a `LPLN` header (16 bytes), followed by a table (8-byte offset + 4-byte length per tile and per channel), then greyscale PNGs, "None" filter, zlib level 6. The browser decodes them with its own PNG decoder (never by reading a canvas, which is not guaranteed to be exact).

## 7.9 `mips/`: the maximum of a layer of 64 planes

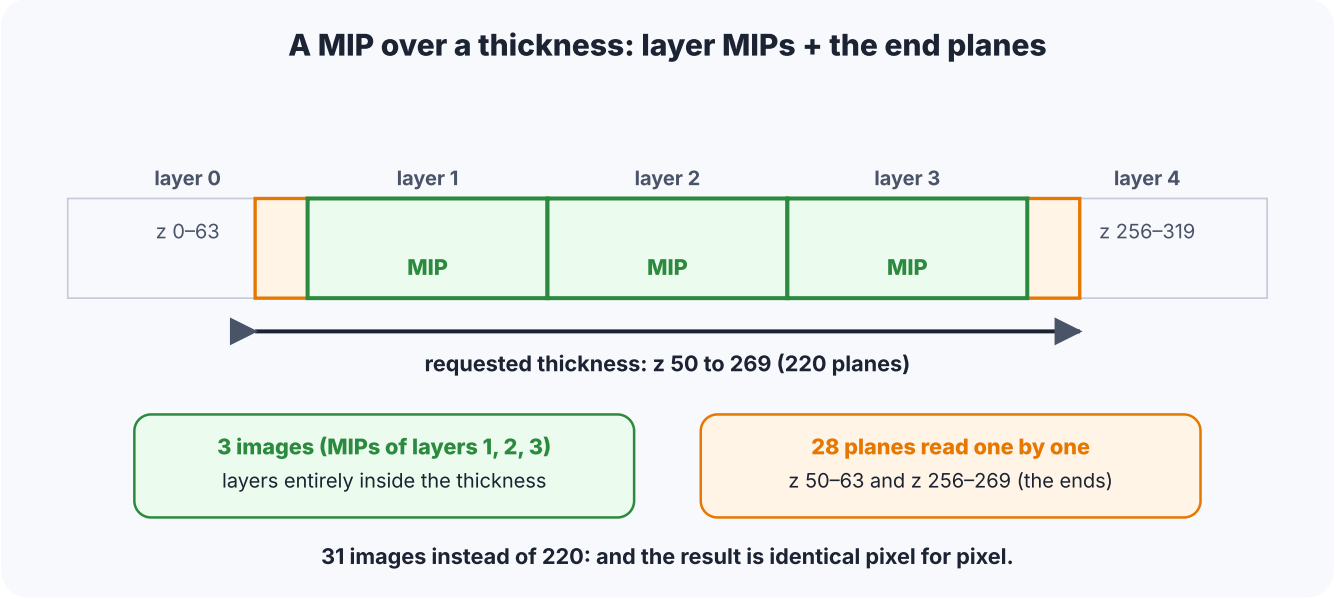
A **maximum-intensity projection** (MIP) keeps, for each pixel, the brightest value met along Z. For a thick stack, computing it on demand would read every plane.

The `mips/` folder therefore keeps a ready-made copy **for each layer of 64 planes** (`l00000.bin` = planes 0 to 63, `l00001.bin` = planes 64 to 111…), at full XY resolution.




▲ A slice (left) and the MIP of the 64-plane layer (right). Demonstration dataset, DAPI channel.

For an arbitrary thickness, the Studio combines: the **MIPs of the layers entirely contained** in the thickness, and the **individual planes** at the two ends.




▲ A thickness of 220 planes: 3 layer MIPs + 28 end planes, i.e. 31 images instead of 220, with an identical result.

## 7.10 Data formats 1 to 5

Over the versions, the file organisation has evolved. Each dataset carries a number, `formatVersion`, in its `metadata.json`. Format **5** is the current format (platform 1.65).

| FORMAT | WHAT IT ADDS | MIGRATION |
|---|---|---|
| 1 | bricks only (v2: $64^3$ without border, 8 × 8 mosaic, one JSON manifest) | — |
| 2 | `planes/` : fast XY slices | `m002-planes` |
| 3 | `mips/` : MIP per layer of 64 planes | `m003-layer-mips` |

| FORMAT | WHAT IT ADDS | MIGRATION |
|---|---|---|
| 4 | v3 bricks: $66^3$ border, 9 × 8 mosaic, packs in super-blocks, `index.bin` | `m004-bricks-v3` |
| 5 | the **matrix**: levels at $256 \cdot 2^k$ that keep every slice, **five compressions** per level, three copies for fast cuts (section 7.11) | `m005-matrix` |

The pipeline (version 0.22.0) writes **format 4 by default**, and format 5 on request ( `--format 5` ). A dataset published in format 4, or older, is not lost: the platform knows how to upgrade it, up to 5.

### "Data updates"

Like a software update, the **Data updates** tab of the admin panel (chapter 17) detects the datasets that are behind and converts them **in place**, one by one. The native level is kept voxel for voxel: nothing is recomputed from the original.

**In this browser**
Your browser does the work (WebP decoding and encoding) and sends the files to the server. On a PHP host, it is the **only** way to reach format 5.

**On the server**
The server converts by itself, if its hosting can do it (a Python server for format 5).

**Pause and resume**
The work is cut into small units noted in a journal: you can pause, close, resume, even changing who does the work.

## 7.11 Format 5: a resolution × compression matrix

⏱ IN 30 SECONDS

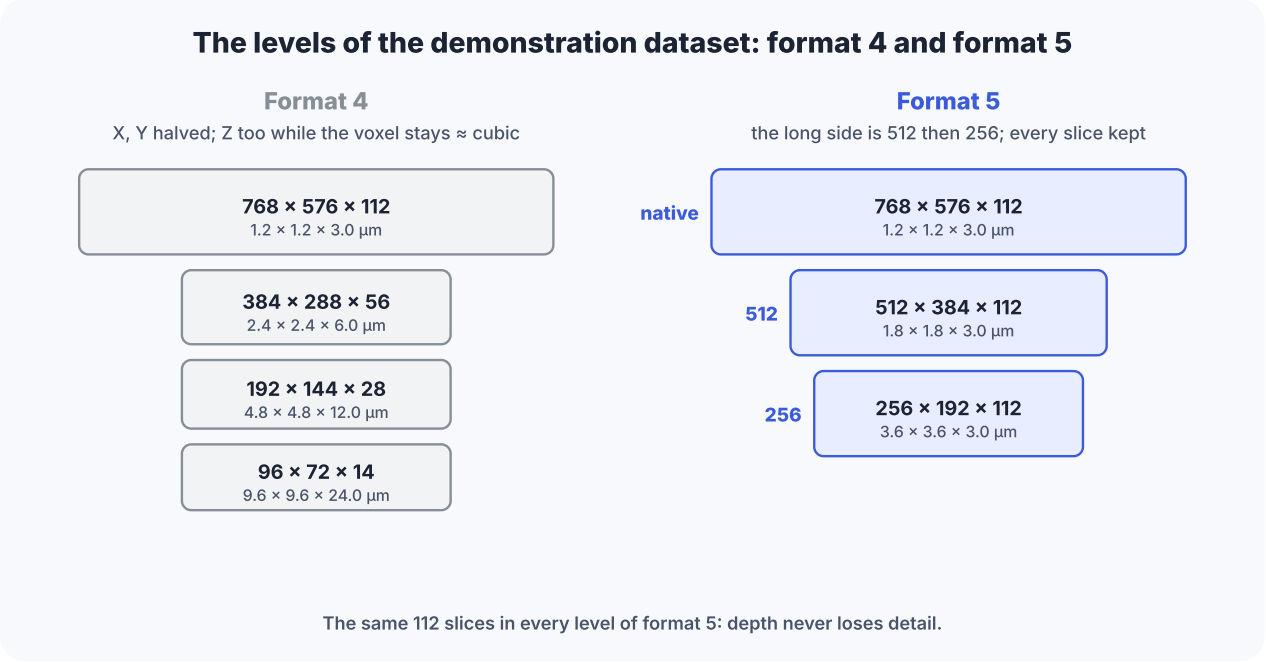
- The same volume is stored **several times**: at several **resolutions** (the rows) and in **five compressions** (the columns). The visitor picks a cell.
- The reduced levels **keep every slice**; their long side is exactly 4096, 2048, 1024, 512 or 256.
- Three **re-cut copies** (XY, XZ / YZ, oblique) make the Studio's cuts fast, and everything that **measures** always reads the voxels **losslessly**.

💡 A PICTURE TO UNDERSTAND

A **photo library** keeps each picture in several sizes and several qualities: the thumbnail to browse, the original to print. Format 5 does the same with a volume: you **look** at a light version, you **measure** and **export** from the original.

The principle: **disk is spent; the network and the graphics card are not.** What is displayed may be compressed with loss, as long as the picture is not visibly altered; what measures reads the original.

## The rows: levels that keep every slice




▲ The levels of the same demonstration dataset in format 4 and in format 5 (real dimensions).

- **Level 0** is the native volume, never resampled.
- The other levels have an XY long side of **exactly** 256, 512, 1024, 2048 or 4096 voxels: every one of these powers of two smaller than the native size is made. A 5192 × 5192 × 312 dataset therefore has six rows: native, 4096, 2048, 1024, 512, 256.
- **Z is never reduced**: every level keeps the 312 slices (here 112). In format 4, the coarse levels lost slices.
- Each voxel of a level is the **exact area average** of the native voxels it covers, in integers, rounded once. Each level is computed **directly from the native level**, never from another level.

🔢 **WORKED EXAMPLE**

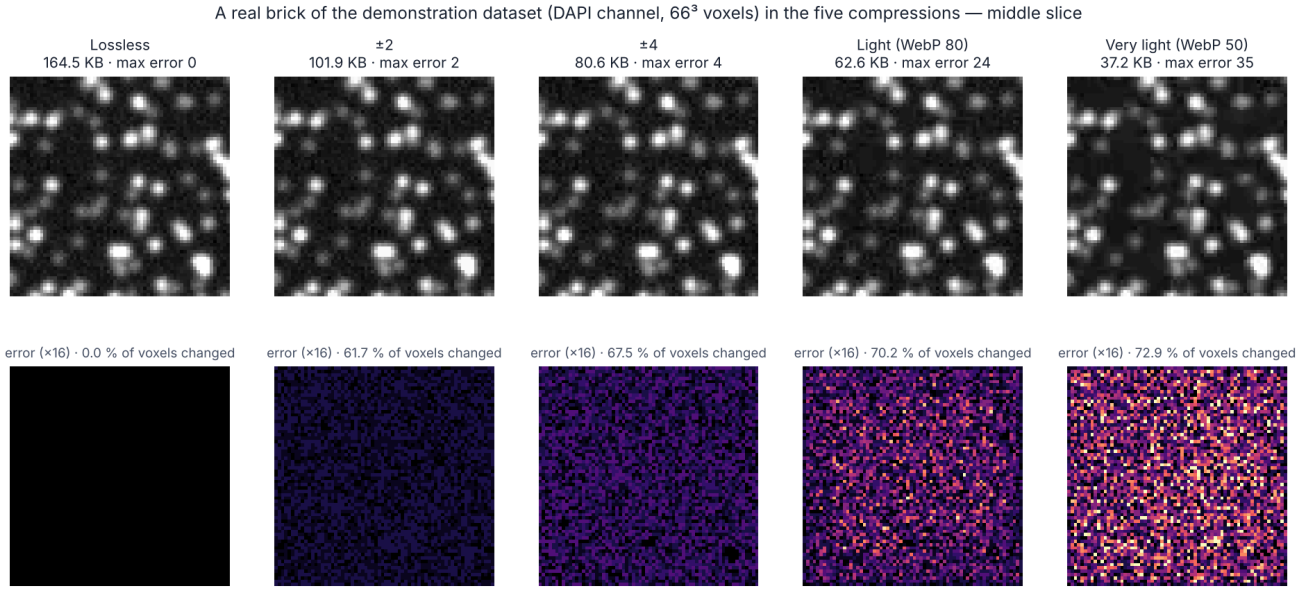
To go from 768 to 512 voxels, one reduced voxel covers 1.5 native voxels. The pipeline cuts the line into whole pieces (768 × 512 units), counts how much of each reduced voxel every native voxel covers, and takes the weighted average: reduced voxel 0 takes all of native voxel 0 and half of voxel 1. No voxel is forgotten or counted twice.

## The columns: five compressions

| COLUMN | ON SCREEN | WHAT IS GUARANTEED | SIZE (RELATIVE TO "LOSSLESS") |
|---|---|---|---|
| **Lossless (** `ll` **)** | **Lossless** | every voxel exact (as in format 4) | 1 |
| **±2 (** `b2` **)** | **±2** | no voxel moves by more than **2** out of 255 | 0.62 to 0.82 |
| **±4 (** `b4` **)** | **±4** | no voxel moves by more than **4** | 0.50 to 0.69 |

| COLUMN | ON SCREEN | WHAT IS GUARANTEED | SIZE (RELATIVE TO “LOSSLESS”) |
|---|---|---|---|
| **Light ( `q80` )** | Light | lossy WebP, quality 80 (no bound; ≈ 36 dB) | 0.35 to 0.61 |
| **Very light ( `q50` )** | Very light | lossy WebP, quality 50 (≈ 31–33 dB) | 0.22 to 0.40 |

Sizes measured on 1,673 native bricks of three of the lab’s embryos (specification, § 14.11).

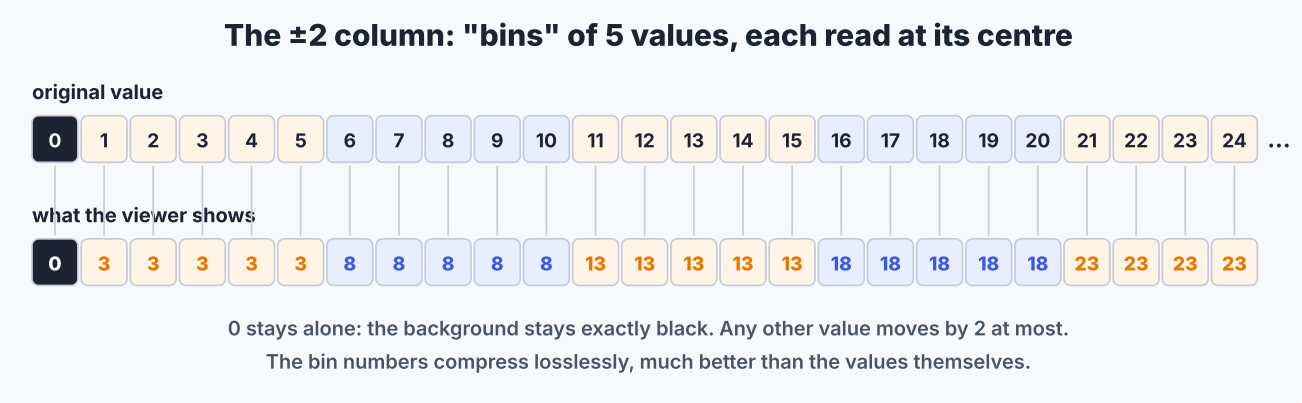



▲ The same real brick in the five compressions: top, the middle slice; bottom, the difference from the original, multiplied by 16 to be visible.

🔢 **WORKED EXAMPLE**

**Read the figure.** On this dense brick (164 KB lossless), ±2 weighs 102 KB and changes no voxel by more than 2; “Very light” weighs 37 KB (4.4 times less) but a voxel can be off by 35. By eye, the five top slices are almost identical: that is the promise of the light columns. To measure, the platform **never** uses them (section “What measures reads the original”).

### ±2 and ±4: “bins” of values

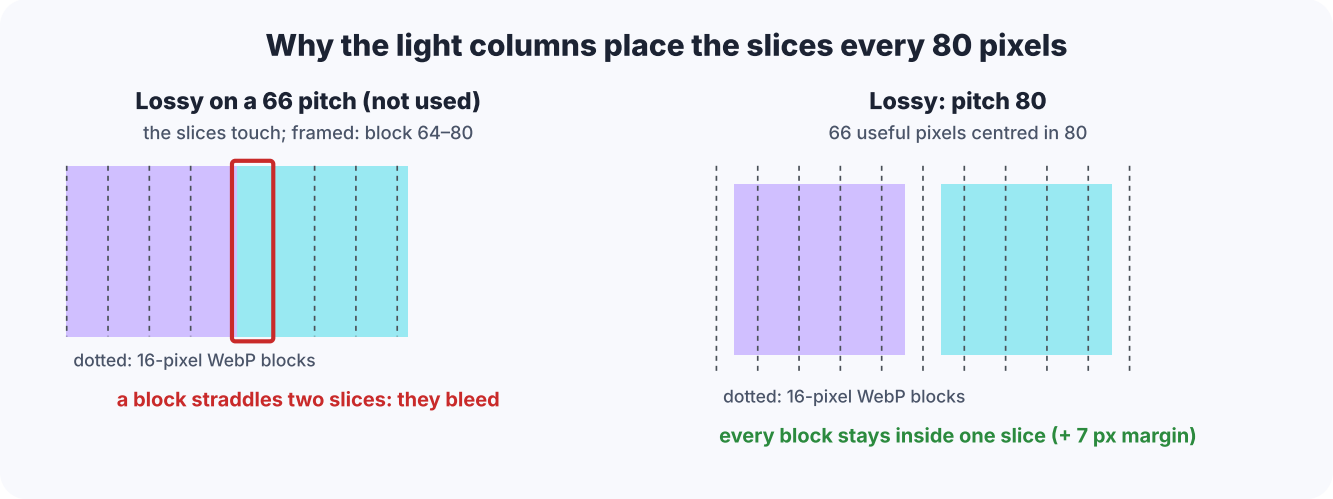



▲ The ±2 column: values are grouped in bins of 5; the viewer shows the bin’s centre.

- The value **0 stays alone** in its bin: the background stays exactly black, empty bricks stay empty.
- The other values are grouped by 5 (±2) or by 9 (±4); **the bin number** is stored, losslessly. Fewer distinct values → the image compresses better.
- The manifest carries the "bin → value" table; the viewer applies it **together with** its display setting, at no extra cost.

**Light and Very light: lossy WebP, but watched**

1. **A dead zone.** A lossy compression "bleeds": a few voxels of the black background take small values. The encoder, which knows the original, chooses a threshold τ (0 to 12) below which everything goes back to 0, and writes it in **one byte** at the end of the brick.
2. **Never heavier than a faithful column.** On an almost empty brick, the lossy image can weigh more than the lossless one. The encoder then keeps the smallest of three: the lossy image, the ±4 image or the lossless image. The final byte says which (0 to 12 = τ; 254 = ±4; 255 = lossless).
3. **A mosaic on an 80-pixel pitch.** Lossy WebP works in blocks of 16 pixels. On a 66 pitch, a block would straddle two neighbouring slices and mix them. On an 80 pitch, each 66-pixel slice is centred with a 7-pixel margin: no block touches two slices.




▲ Why the lossy columns place the slices every 80 pixels.

🔢 **WORKED EXAMPLE**

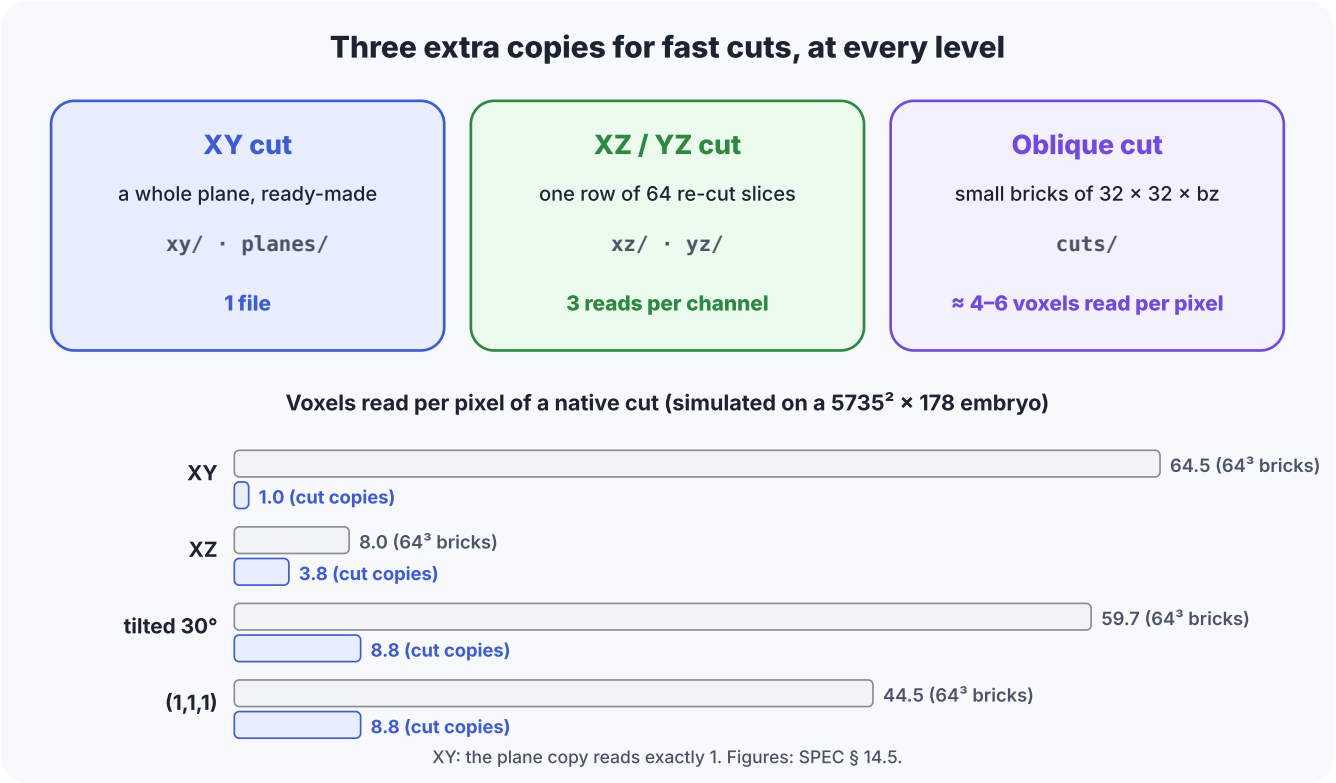
**Measured on the DAPI channel of the demonstration dataset** (100 native bricks, Very light column): without the dead zone, 3.9 % of the black voxels become non-zero; with it, 1.8 %. And 12 bricks out of 100 are stored as ±4 rather than lossy WebP, because that was smaller.

❓ **WHY THIS CHOICE?**

**Why not AVIF or JPEG XL?** AVIF gives the same size / quality trade-off as WebP within a few percent, encodes twice as slowly and has no encoder in browsers, which therefore could not do the conversion. JPEG XL is not decoded by Chrome or Firefox by default (October 2026). Measurements: specification § 14.3.

## Three copies for fast cuts

A cut drawn from $64^3$ bricks downloads far more than it shows: an XY cut one voxel thick reads 64 slices. Format 5 adds, **at every level**, three re-cut copies, all **lossless**:




▲ The three cut copies, and what they save.

| CUT | READ FROM | WHAT IS READ |
|---|---|---|
| XY | `planes/` (native) or `xy/` (reduced levels) | **one** plane |
| XZ or YZ | `xz/` or `yz/` : packs of 64 re-cut rows | one row: three small reads per channel |
| **oblique** | `cuts/` : small bricks of 32 × 32 × *bz* voxels, without border | only those the plane crosses |

*bz* is chosen so that the small brick is roughly cubic **in micrometres**: 32 × 32 × 16 on the demonstration dataset (1.2 × 1.2 × 3 µm voxel), 32 × 32 × 4 on an embryo at 0.344 × 0.344 × 3 µm.

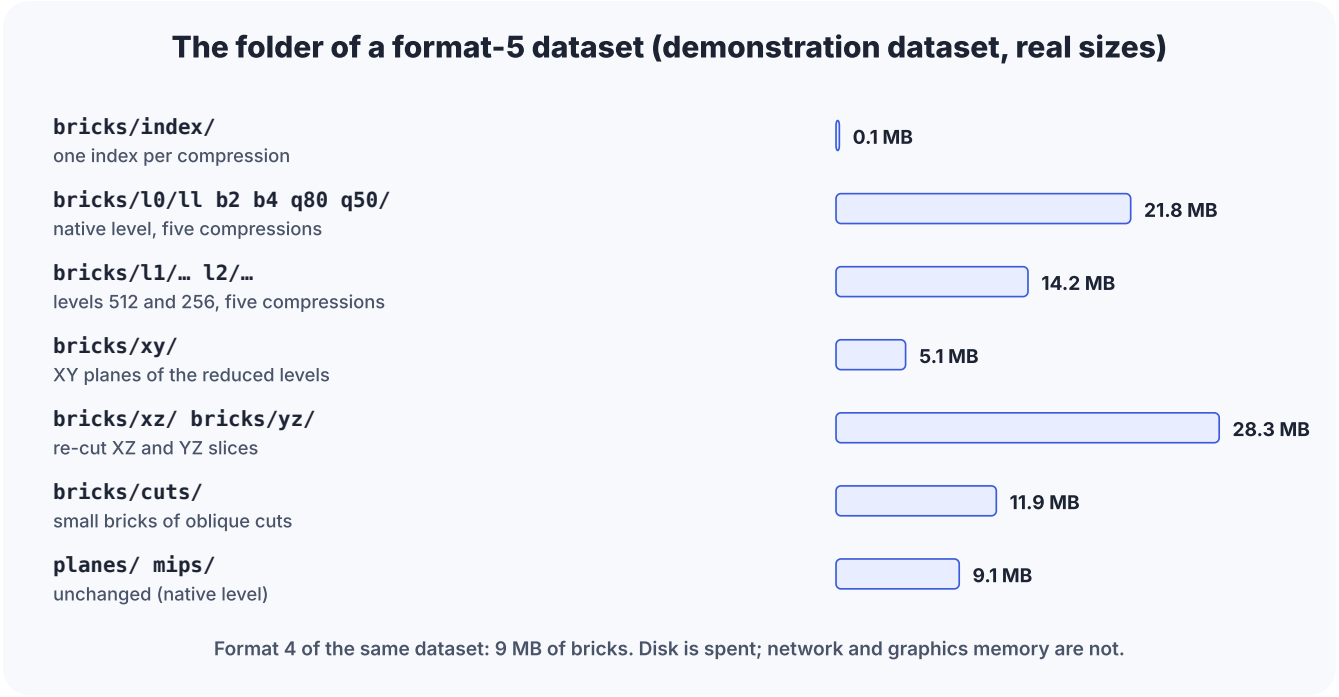
🔢 **WORKED EXAMPLE**

**Measured on a 1946² × 72-voxel embryo** (platform 1.64): a native oblique cut downloaded **23 MB in 15 requests** instead of 60 MB in 30 through the bricks, and every picture is identical voxel for voxel.

## What measures reads the original

Every measurement, every plugin that reports a number and every “native” export reads the **lossless column of the native level**, whatever cell is displayed (chapters 12 and 15). A light display setting therefore never skews a number.

## The tree of a format-5 dataset




▲ The `bricks/` folder of the demonstration dataset after conversion, with real sizes.

- Everything format 5 adds lives **inside** `bricks/` : one manifest ( `iribhm-bricks-v5` ) describes it, and one rename installs it.
- `bricks/index/ll.bin` , `b2.bin` …: one index per column (same structure as format 4's `index.bin` ). The viewer reads the lossless index when the dataset opens, the others only when their column is chosen.
- The packs of the lossless column of level 0 **are** those of format 4: the migration links them (hard link) instead of re-encoding them.
- `planes/` and `mips/` do not change: they already hold the native level.

> ⚠️ **WARNING**
>
> **Disk use grows a lot**: on the demonstration dataset, 9 MB of bricks in format 4 become about 80 MB in format 5 (28 MB of which are XZ / YZ copies). That is intended: what costs is what each visitor downloads and what their graphics card keeps, and format 5 reduces that. Check the free space of the host before converting large datasets.

> 📌 **REMEMBER**
>
> - A brick = $64^3$ useful voxels + 1 border voxel = **$66^3$**, stored as **one lossless WebP image** of 594 × 528 pixels (9 × 8 slices).
> - The bricks are in **packs** (4 × 4 × 4 super-blocks); `index.bin` gives the pack, start and length of each (10 bytes per brick).
> - `planes/` speeds up XY slices, `mips/` projections.
> - Current format = **5**: levels at $256 \cdot 2^k$ that keep every slice, five compressions per level (lossless, ±2, ±4, light, very light), three re-cut copies for cuts; everything that measures reads the lossless column of the native level.

# 8. The end of the pipeline: metadata, cell tracking, publication

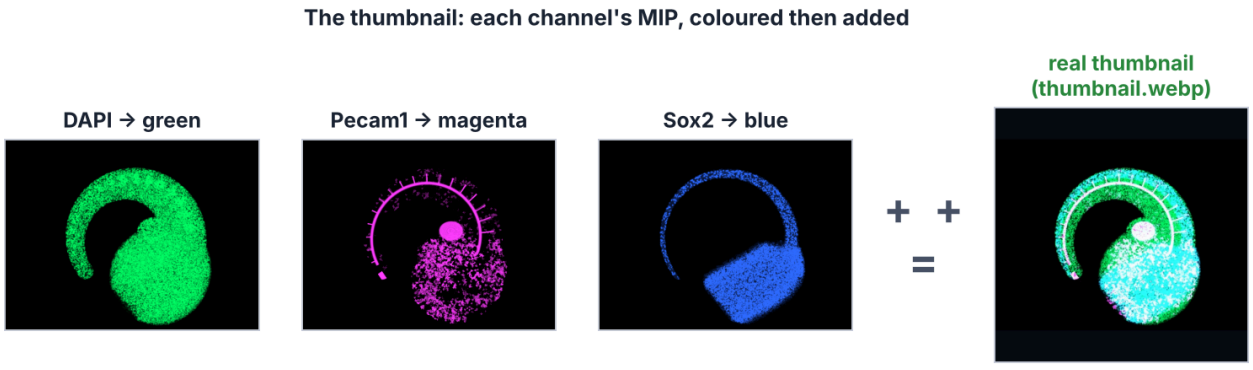
- After the bricks, the pipeline makes the **thumbnail**, the **histograms** and the `metadata.json` file (stage, embryo, calibration, channels), and, for a time series, the **cell tracking**.
- The result is published **"all or nothing"**: the dataset already online is never left half-replaced, and what you entered by hand is kept.
- You launch everything with `RUN.bat`, then copy the folder it produces to the server (FTP) or drag it into the **Import** tab.

The data images in this chapter come from the **demonstration dataset** (synthetic embryos generated for this documentation, processed by the real pipeline): they are not lab data.

## 8.1 The thumbnail

The thumbnail is the small image on the dataset's card in the explorer. It is made from the **first image** of the acquisition, using the finest resolution level whose long side does not exceed 1024 pixels.

For each channel, the pipeline computes a **MIP** (the brightest voxel across the whole thickness), colours it, then **adds** the channels: where two channels overlap, the colours add up.



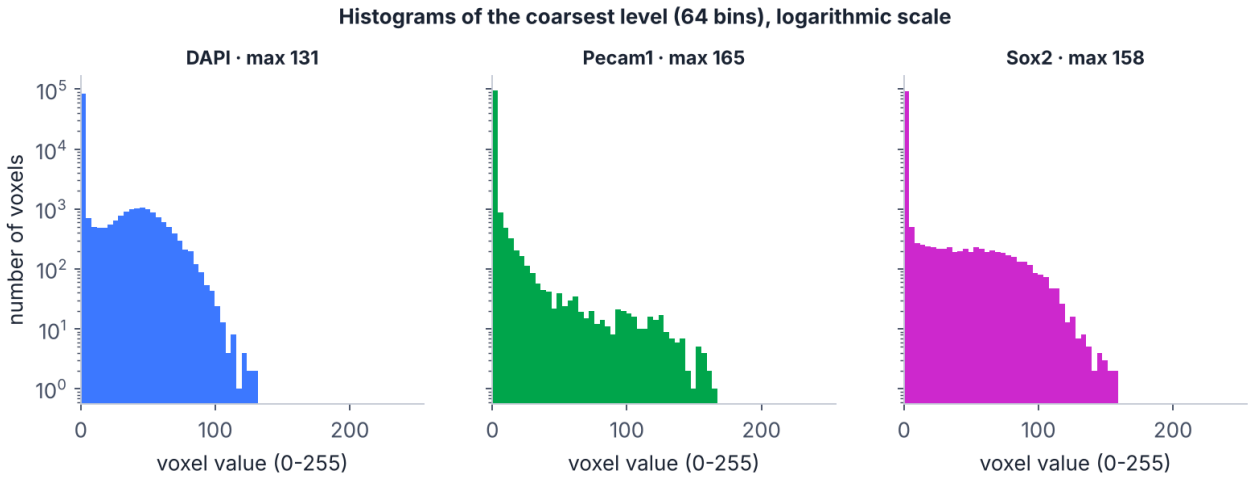

▲ Three coloured MIPs, added together, give the real thumbnail of the demonstration dataset.

🔧 **FOR THE CURIOUS**

Thumbnail colours, in channel order: green, magenta, blue, red, yellow, violet, cyan (repeating). They have no link with the colours chosen in the viewer. The image is reduced to 512 pixels on its long side (Lanczos interpolation), centred on a square of `#080a12` background, then saved as **lossy WebP**, quality 88 (33.6 KB here).

## 8.2 The histograms

For each channel, the pipeline counts how many voxels have each grey value, on the **coarsest level** (the fastest to read). The values 0 to 255 are spread over **64 bins** of width 3.98.




▲ The three histograms stored in the demonstration dataset's manifest (logarithmic scale: the background at 0 would flatten everything else on a linear scale).

They are stored in `bricks/manifest.json` (`counts`, `edges`, `total`, `max`, `mean`, `std`); the viewer shows them in the channel panel (chapter 11). For a time series there is **one set per image**, because photobleaching would change the scale of a shared set.

## 8.3 `metadata.json`: the dataset's identity card

This is the only place where the facts about a dataset are written; the site's catalogue is computed from it on every request. **Real** excerpt (abridged) from the demonstration dataset:

```
{
  "id": "3d/Embryo-E95-Em2-Pecam1-Sox2",
  "formatVersion": 4,
  "type": "3d",
  "stage": "E9.5", "stageNumeric": 9.5, "embryo": "Em2",
  "dimensions": { "x": 768, "y": 576, "z": 112, "c": 3, "t": 1 },
  "voxel_size": { "x": 1.2, "y": 1.2, "z": 3.0 },
  "calibrationStatus": "exact",
  "channels": [ { "name": "DAPI", "color": "#3D7BFF",
                  "min": 0.0, "max": 1.0, "gamma": 1.0 }, ... ],
  "description": "Confocal imaging stack: E9.5 fixed embryo, ...",
  "volumeSources": [ { "kind": "bricks", "multiscale": true, ... } ],
  "hidden": false
}
```

| FIELD | WHAT IT IS |
|---|---|
| `id`, `type` | the type (`3d` or `live`) and the folder: `3d/<name>` |
| `formatVersion` | the data format (chapter 7); written by the pipeline, never edited by hand |
| `stage`, `embryo` | read from the file name (next section) |

| FIELD | WHAT IT IS |
| --- | --- |
| `dimensions` , `voxel_size` | size in voxels and in micrometres, read from the Imaris file |
| `calibrationStatus` | `exact` if the three voxel sizes are known, otherwise `metadata-missing` : no scale is invented |
| `channels` | name, colour, window ( `min` , `max` ) and starting gamma of each channel |
| `volumeSources` | where the viewer finds the bricks |

ℹ️ **GOOD TO KNOW**

For a time series ( `live` ), the file also contains `timeline` (number of images, interval, timestamps) and `intensityNormalization` (the normalisation window shared by the whole series, and one signal level per image to measure photobleaching; the viewer does not use it yet, see chapter 5).

ℹ️ **GOOD TO KNOW**

The pipeline writes **all** the channels of the original file; the viewer **displays only the first four** (limit of the RGBA graphics texture, chapter 11).

## Stage and embryo, read from the file name

The `.ims` file name is **interpreted**. The stage is an `E…` token surrounded by hyphens, underscores or spaces; the embryo is an `Em<n>` token or, failing that, the number that directly follows a whole-number stage.

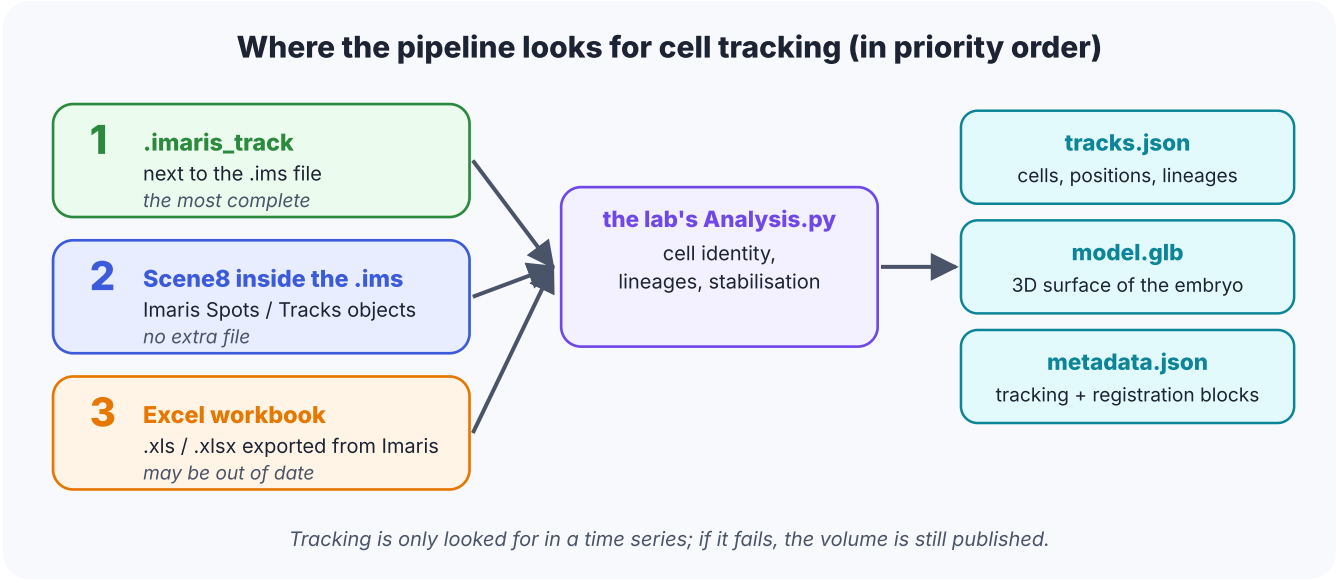
| NAME (EXCERPT) | STAGE | EMBRYO | NOTE |
| --- | --- | --- | --- |
| `…-E8.5-…` or `E8,5` | E8.5 | — | point or comma |
| `…-E8-5-…` | E8.5 | — | hyphen: confirmed by the lab |
| `…-E85-…` | E8.5 | — | compact form |
| `…-E105-…` | E10.5 | — | two-digit day if it starts with 1 |
| `…-E95-Em2-…` | E9.5 | Em2 | explicit embryo |
| `…-E8-1-DAPI-…` | E8 | Em1 | `-1` is **not** a fraction: after a hyphen, only .25, .5 and .75 count as one |
| `…-E95-1-…` | E9.5 | Em1 | the number after the stage |
| `Embryo-Pecam1` | unknown | — | no `E…` token |

⚠️ **WARNING**

The token must be surrounded by hyphens, underscores or spaces: `-E85-Em1-` is read correctly, `E85Em1` is not. If the stage is wrong, correct it in the **Datasets** tab: the pipeline will keep it on later runs (section 8.6).

## 8.4 Cell tracking (time series)

For a time series only, the pipeline looks for a **tracking analysis** (cells, trajectories, divisions) and attaches it to the dataset. It looks in three places, from best to least good.




▲ The three possible sources, from the most complete file to the most fragile workbook.

❓ **WHY THIS CHOICE?**

**Why does Scene8 come before Excel?** An exported workbook can be out of date: on a real dataset it held 155 cells while the volume counted 172. The objects stored in the `.ims` are always up to date. Tracking is looked for automatically (option `--tracking auto`); if something fails, the volume is published anyway.

Tracking brings three files or blocks:

- `tracks.json` (and its compressed copy `tracks.json.gz`): for each cell, its identifier, its region, its positions at each image in **µm** (stabilised and raw), its markers (red = division, black = fusion), its mother cell and its daughters;
- `model.glb`: the embryo's **3D surface** at each image, rebuilt from the cells (at least 4 per image);
- in `metadata.json`, the `tracking` block (cell count, regions, data origin) and the `registration` block (the stabilisation transform, below).

🔢 WORKED EXAMPLE

**The demonstration dataset over time.** 50 cells, 4 images, 3 regions (Posterior 18, Anterior 16, Lateral 16), 1 division and 0 fusions; `model.glb` weighs 412 KB and `tracks.json` 32.7 KB. Positions are numbered from 1 (as in Imaris) and aligned to the volume's images, numbered from 0 (offset −1).

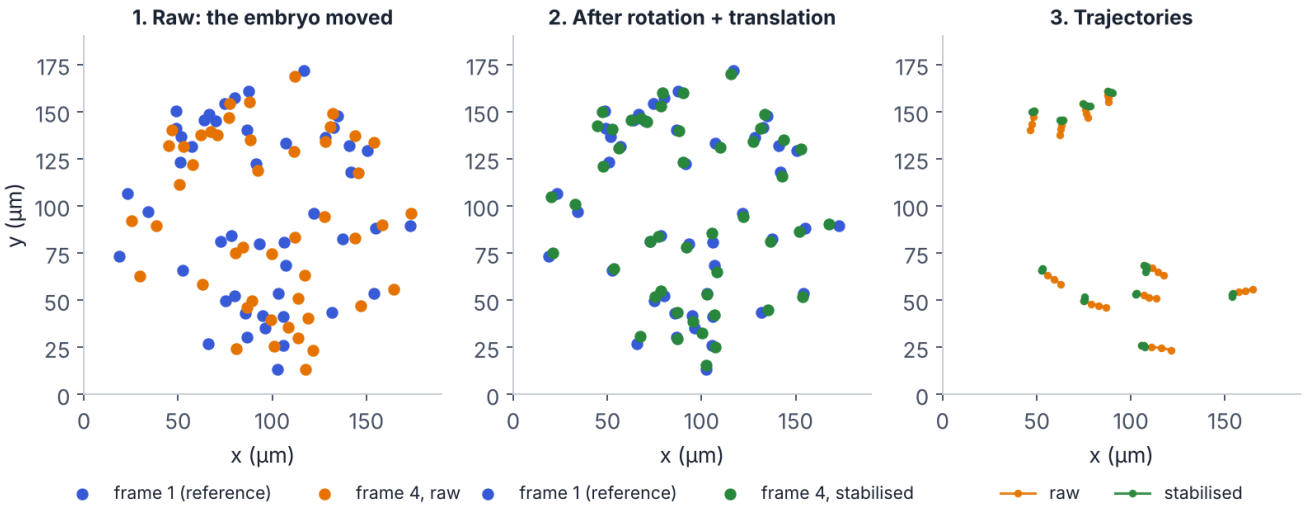
## Stabilising: cancelling the embryo's movements

Over hours, the embryo drifts and rotates a little in the field. The pipeline **puts it back in the same position** at each image, so that the visible movement is that of the cells and not that of the sample.

💡 A PICTURE TO UNDERSTAND

**A transparent sheet.** Draw the cells of image 1 on one sheet, and those of image 4 on another. To overlay them, you slide the sheet and rotate it, but you never stretch it: that is exactly a "rigid" motion.

On the demonstration dataset, here is the same idea with the real cells (47 cells present in images 1 and 4, projected onto the x-y plane).




▲ Left, the cells of image 4 have drifted; centre, after a rotation of 7.03° and a translation, they overlay those of image 1; right, the raw trajectories drift, the stabilised ones stay in place.

The algorithm used is called **Kabsch** (singular value decomposition). It chooses **reference cells** (those that do not divide, present in at least two images) and looks, image after image, for the rotation and translation that best overlay them; at least 3 in common are needed.

🔧 FOR THE CURIOUS

**The maths.** Let P be the stabilised positions of image n−1 and Q the raw positions of image n (reference cells), centred on their centroids $c_P$ and $c_Q$. We form $H = Q_c^T P_c$, decompose it as $H = U S V^T$, then $\mathbf{R = V \cdot diag(1, 1, sign(det(V U^T))) \cdot U^T}$ (a true rotation, never a mirror) and $\mathbf{t = c_P - R c_Q}$. The transform (R, t) is applied to **all** the cells of image n. The first image is the reference.

### The `registration` block: applying the same thing to the images

The tracking is stabilised, but **the images are not**. Since the motion is rigid, the same transform lets the viewer overlay the trajectories on the images simply by shifting the reading coordinates, **without resampling** a single voxel. For each image, the file keeps a 4 × 4 matrix, the angle and the translation:

| IMAGE | ROTATION | TRANSLATION (MM) | RESIDUAL (MM) |
|---|---|---|---|
| 1 | 0° | (0; 0; 0) | $4 \times 10^{-14}$ |
| 2 | 2.33° | (−5.65; 5.01; −0.37) | $6 \times 10^{-14}$ |
| 3 | 4.66° | (−11.09; 10.60; −1.01) | $4 \times 10^{-14}$ |
| 4 | 7.03° | (−16.14; 16.39; −1.79) | $8 \times 10^{-14}$ |

📌 **REMEMBER**

**Rigidity test.** The pipeline checks that the transform really is rigid: the largest residual must stay **≤ 0.05 µm**. Here it is $7.6 \times 10^{-14}$ µm (zero, to within rounding error), so the stabilisation is applied to the images. If the test failed, a warning would say so and it would not be applied.

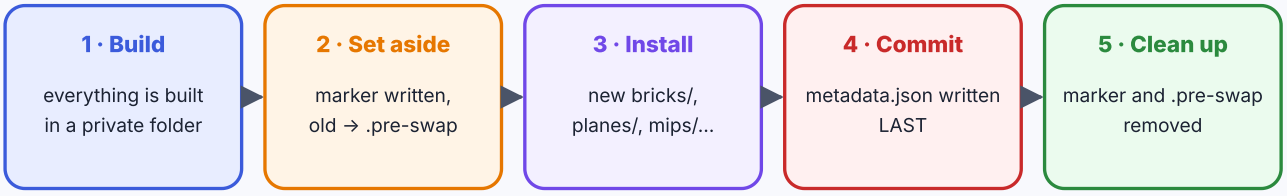
## 8.5 Publishing “all or nothing”

A run sometimes lasts hours. Meanwhile, **the dataset already online keeps working**: everything is built in a private working folder, on the same disk location as `DATA_WEB`, and is installed only at the end.

💡 **A PICTURE TO UNDERSTAND**

**A shop window.** You prepare the new window in the back room; you take the old one out only once the new one is completely set up. If a problem occurs, customers still see the old one.

**“All or nothing” publication: the shop window changes only when the new one is ready**




**Failure BEFORE step 4: everything is put back; the published dataset never stopped working.**

Crash in the middle: at the next launch, the marker lets the pipeline finish or undo cleanly,

depending on whether metadata.json (its sha256 is stored in the marker) is already in place.

▲ The five stages of publication; `metadata.json` is written last, it is the point of no return.

A **new** dataset arrives in a single folder rename. For an **existing** one, only the pipeline’s own items are replaced ( `bricks/` , `planes/` , `mips/` , the thumbnail, and `tracks.json` , `tracks.json.gz` , `model.glb`

if a new tracking exists). Everything else in the folder, such as `download/` or `gallery/`, is never touched.

## 8.6 What the pipeline keeps when you reprocess a dataset

Did you correct a name, orient the embryo, add captions in the administration? A new pipeline run **does not overwrite it**: just before installation, it merges the fields entered by hand into the new `metadata.json`.

- **Identity**: `name`, `description`, `stage`, `stageNumeric`, `embryo`, `line`, `staining`, `reporter`, `tags`, `notes`, `created`;
- **Display**: `hidden`, `orientation`, `orientationAxes`, `upsideDown`, `defaultView`, `exposure`;
- **Links and images**: `gallery`, `linkedTrackingId`, `relatedIds`, and any field the new run does not produce (for example `tracking`).

Channel colours and settings are also kept, as long as the **number of channels** has not changed. Conversely, `formatVersion` is **never** carried over: it always reflects the structure actually written on disk.

> ⚠️ **WARNING**
>
> If a crash occurs during installation (power cut), the `.swap-in-progress` marker lets the next run **finish or cleanly undo** the change. Do not delete this file by hand.

## 8.7 The `download/` folder

With the `--with-downloads` option, the pipeline prepares, **after** publication (so a failure here never undoes the dataset), a folder of files to download. Real content of the demonstration dataset:

```
download/
├── Embryo-E95-Em2-Pecam1-Sox2.ims                255.8 MB  the original
├── Embryo-E95-Em2-Pecam1-Sox2.tif                254.6 MB  ImageJ composite
├── Embryo-E95-Em2-Pecam1-Sox2_C1_DAPI_MIP.png      368 KB  one MIP per channel
├── …_C2_Pecam1_MIP.png  …_C3_Sox2_MIP.png
├── Embryo-E95-Em2-Pecam1-Sox2_web.zip             18.4 MB  the web dataset
└── README.txt                                              provenance
```

- **The original** `.ims` is a **hard link**: it appears in `download/` without taking any extra space; it is never modified.
- **The** `.tif` is an ImageJ/Fiji composite in the **native format** (8 or 16 bits, raw values), calibrated in µm, with each channel's colour and display range. It is taken from the Imaris resolution level closest to 2048 pixels.
- **The PNG MIPs** give a per-channel preview (normalised between the 1st and 99.9th percentiles).
- **The** `_web.zip` archives the served folder, without recompression (WebP and PNG already are).
- **The** `README.txt` recalls dimensions, voxel sizes, channels and how to cite the dataset.

For a time series, the `.tif` and the MIPs show only **one image** (the first); the `.ims` contains them all.

## 8.8 2D photographs, in half a page

Photographs (stereomicroscope, TIFF exported from a Leica file) have their **own tool**, `2d_importer.py`: one TIFF file gives one `2d/<name>/` folder.

| FILE | ROLE | DEMONSTRATION DATASET |
|---|---|---|
| `image.webp` | the image at native size | 1920 × 1440 px, 191 KB |
| `preview.webp` | 640 px preview, displayed first | 10.6 KB |
| `thumbnail.webp` | 512 px thumbnail | 12.8 KB |
| `metadata.json` | stage, line, date, pixel size (2.0 µm here) | written last |

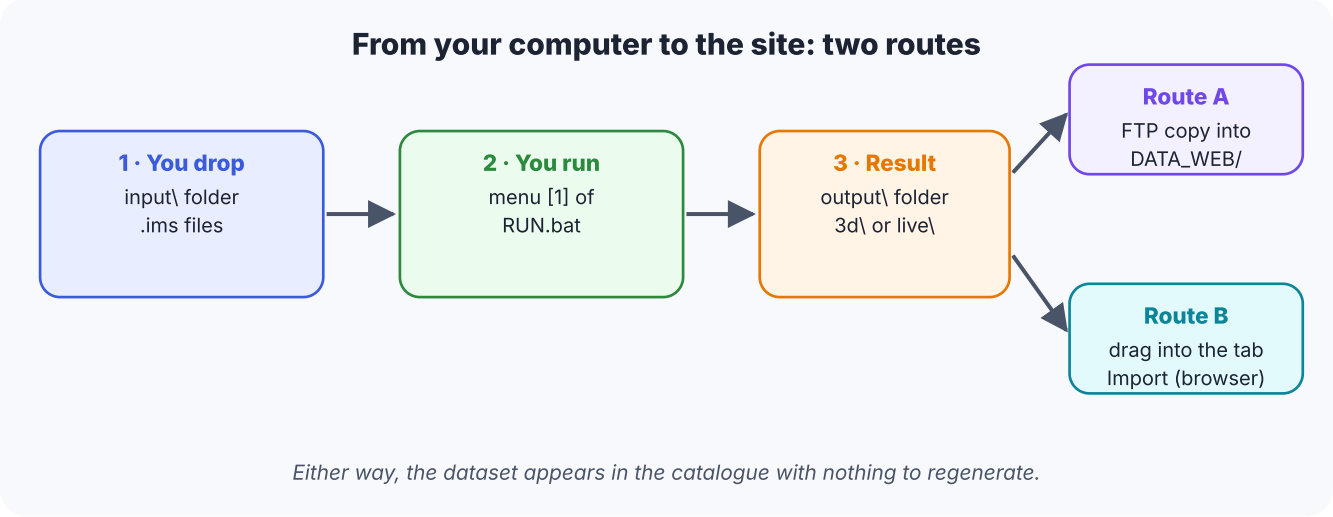
❓ **WHY THIS CHOICE?**

**Why "lossy" WebP by default?** It divides the size by about 40 compared with the original TIFF (8.3 MB against 191 KB here), at a quality of 90. But lossy WebP **subsamples colour** (chrominance is stored with half as many pixels). Now the viewer's X-gal stain isolation reads the **blue/red ratio** pixel by pixel: on sharp edges, this ratio is slightly disturbed. The `--lossless` option keeps every pixel exact, at the price of bigger files.

The pixel size is read only if the TIFF declares a unit in microns; otherwise `pixelSizeUm` stays empty and the viewer shows **neither scale nor measurement**. The stage, zoom and dissection date are read from the file name, with the same stage reader as the volumes.

## 8.9 Launching all this: `RUN.bat`

The **Pipeline pack** (a folder to unzip on a Windows PC) contains everything. Double-click `RUN.bat`: it checks the pack's integrity (SHA-256 fingerprints), prepares Python and the libraries, then shows a menu.




▲ From your computer to the site: you drop the files, run, then copy or drag the result.

| CHOICE | WHAT THE MENU DOES |
|---|---|
| `[1]` | Preprocessing of Imaris volumes (.ims → `output\`, tracking included) |
| `[2]` | Imaris tracking analysis (Excel → `tracking\OUTPUT\`) |
| `[3]` | Import of 2D photographs (.tif → `output\2d\`) |
| `[4]` | Attach a tracking to an already processed dataset |
| `[5]` | Check the environment only |
| `[0]` | Quit |

The menu asks a few questions (folder of the `.ims` files, output folder, download archives or not) then launches `run_preprocess.py`. You can also call it yourself:

```
python run_preprocess.py --input input --output DATA_WEB --only "*E95*" --with-downloads
```

| OPTION | EFFECT |
|---|---|
| `--input` | folder of the `.ims` files (not recursive, one file at a time) |
| `--output` | the `DATA_WEB` root: the result goes to `3d/<name>/` or `live/<name>/` |
| `--only` | filter on file names (e.g. `"*E8*"`) |
| `--with-downloads` | also prepares the `download/` folder (slower) |
| `--tracking` | `auto` (default), `off`, or the path of a tracking file |

The type is decided automatically: more than one image in the file means `live`, otherwise `3d`. `Ctrl+C` asks for confirmation before stopping, and cleans up the temporary files.

## How the result reaches the server

- **Route A, by FTP**: copy the produced folder (`3d\<name>` or `live\<name>`) into the server's `DATA_WEB`. It appears in the catalogue at once; there is nothing to regenerate.
- **Route B, without FTP**: in the admin panel, open the **Import** tab and drag the folder into the **Drop a folder here** area. The transfer picks up where it stopped if you restart it; then click **Publish**.

> ⚠️ **WARNING**
>
> A dataset published through the **Import** tab is **hidden by default**: go to the **Datasets** tab and turn its visibility on so that it appears in the public explorer (chapter 14).

📌 **REMEMBER**

- The thumbnail, the histograms and `metadata.json` describe the dataset; cell tracking is a **layer** of a time series, stabilised by a rigid motion.
- Publication is **all or nothing**, and what you corrected by hand is kept on reprocessing.
- `RUN.bat` → choice `[1]` → `output\` folder → FTP or the **Import** tab.

# 9. The 3D viewer: Three.js and ray marching

- Every pixel on the screen is computed by **one ray** that travels through the volume: this is **ray marching**, and it runs on the graphics card.
- **Three.js** provides the scene, the camera and the textures; the part that matters for the science (the three render modes) is **custom-written** and stays readable.
- While you rotate the embryo, sharpness drops for a moment to keep things smooth; at rest, everything returns to full quality.

## 9.1 Two brains: the processor and the graphics card

Your computer has two kinds of “brain”. The **processor** (CPU) is a generalist. The **graphics card** (GPU) is specialised in a single thing: doing the same small calculation on thousands of points at once.

**Why the computation runs on the graphics card**

**THE PROCESSOR (CPU): one chef**

**Very skilled, but alone**
prepares the plates one by one

1,920 × 1,080 pixels = 2 million plates

→ too slow to rotate an embryo live

**THE GRAPHICS CARD (GPU): kitchen hands**

**Each makes the same small gesture**
... each on its own pixel, all at the same time

the small gesture = the "shader" (the pixel program)

→ a smooth image, aiming for 60 times a second

▲ One head chef against thousands of kitchen hands: the graphics card wins whenever the same gesture must be repeated millions of times.

💡 **A PICTURE TO UNDERSTAND**

A star chef prepares one perfect plate in two minutes. To serve 2 million guests, it would take years. A thousand kitchen hands, each following **the same simple recipe**, serve the whole room in an instant. A 1,920 × 1,080 pixel screen is exactly that room.

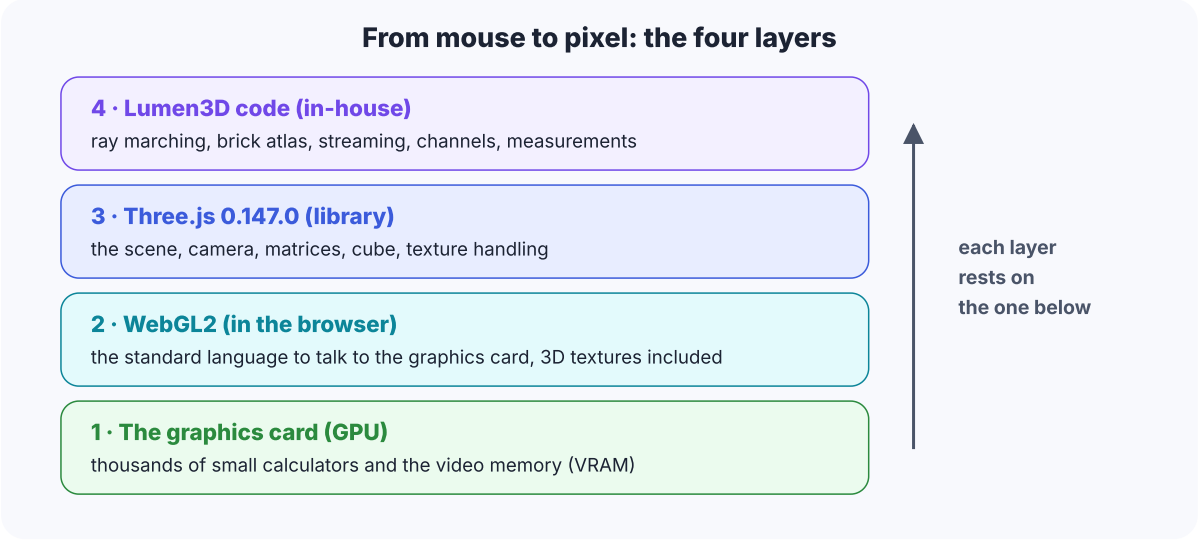
In Lumen3D, the “recipe” is called a **shader**: a small program written once and run simultaneously for every pixel. All the volume computation happens there.

## 9.2 WebGL2 and Three.js: who does what

**WebGL2** is the standard language browsers understand to talk to the graphics card. Among other things, it can handle **3D textures**: blocks of voxels stored directly in video memory. This is essential for

a volume.

**Three.js** (version **0.147.0**) is a library that sits on top of WebGL2 and takes care of the “plumbing”. It is **hosted on the platform’s own server** (the `js/vendor/` folder), not downloaded from the Internet.




▲ The four layers between your mouse and the displayed pixel.

| PROVIDED BY THREE.JS | CUSTOM-WRITTEN FOR LUMEN3D |
|---|---|
| **The drawing engine (the “renderer”), the scene** | The ray-marching **shaders** (3 render modes) |
| **The perspective camera (45° field of view)** | The brick **atlas** and its page table |
| **The cube (a six-faced box that is the starting point of the rays)** | Brick **streaming** (chapter 10) |
| **Matrices, quaternions, vectors** | Adaptive quality and the memory budget |
| **The wrapper around 3D textures** | Depth reading under the mouse (measurements) |
| **The 3D model loader (cell-tracking surface)** | Channels, histograms, Gaussian filter |

❓ **WHY THIS CHOICE?**

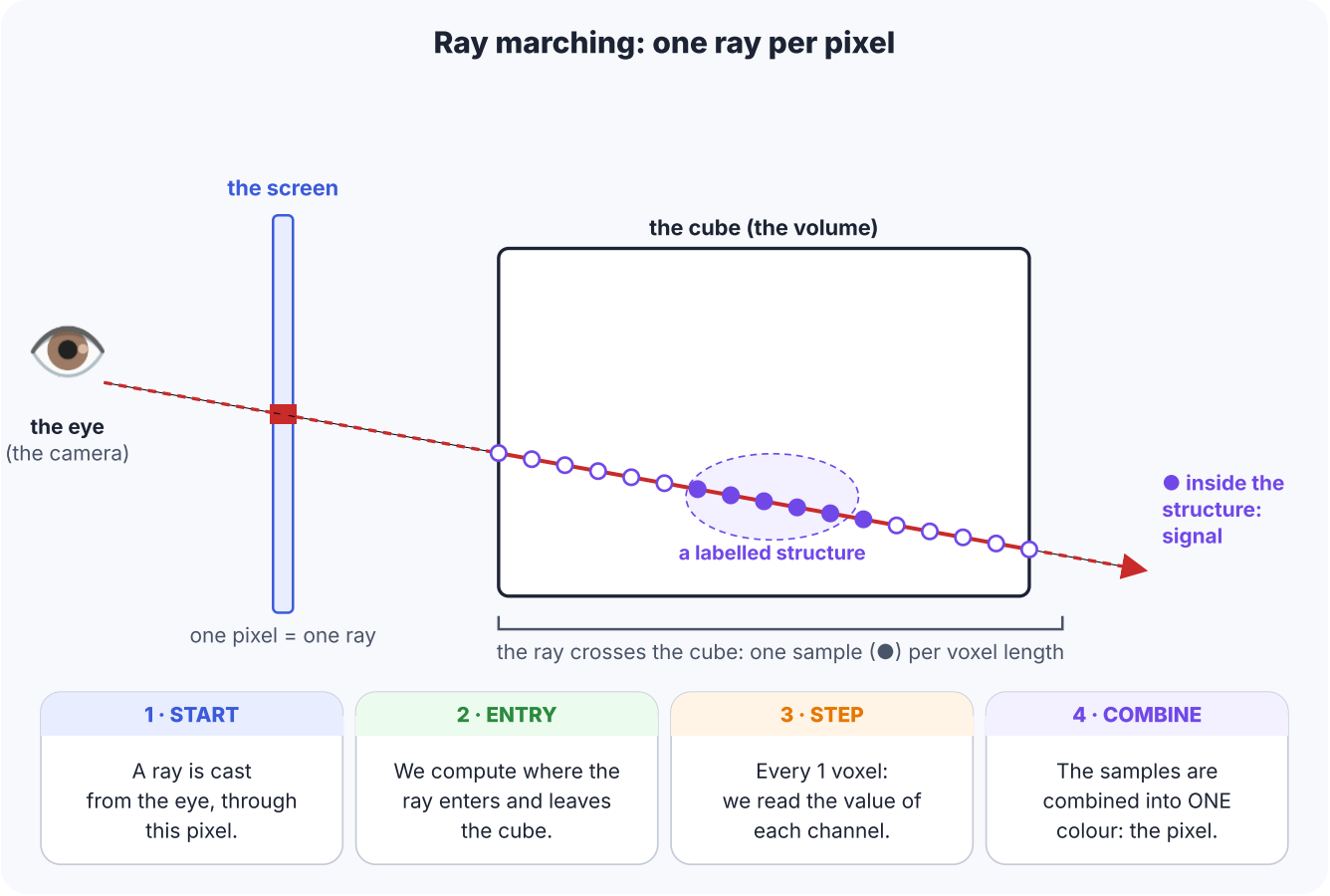
- **Mature and light**: Three.js is very widely used and needs no assembly tool (no “build”): one file, one browser.
- **Offline**: copied onto the server, it works without Internet.
- **Auditable**: the part that determines what you see (the rendering formulas) is lab code, commented line by line. A full game engine was not needed: it would have brought thousands of functions that are useless for microscopy.

ℹ️ **GOOD TO KNOW**

The viewer requires **WebGL2** (there is no degraded WebGL1 version). If the graphics card cannot create 3D textures, loading is refused with a clear message, never by crashing the tab.

## 9.3 Ray marching, step by step

The volume is an enormous array of values. To turn it into an image, we work **pixel by pixel**. Each screen pixel casts a **ray** from the eye through the cube. We read the volume along that ray, then summarise what we met into a single colour.




▲ One ray per pixel: entering the cube, one sample per voxel, then combining.

1. **Start.** The pixel defines a direction: the one that leaves the eye and passes through this pixel.
2. **Entry and exit.** The shader computes where the ray enters the cube and where it leaves it (the "ray / box intersection"). Outside the cube, the pixel stays transparent.
3. **Regular step.** We advance by **one voxel** each time and read the value of every channel.
4. **Combination.** The readings are summarised into one colour, according to the render mode (section 9.6).

### One sample per voxel

At rest, the viewer takes **exactly one sample per voxel length**, whatever the direction of the ray. This guarantees that **a structure one voxel wide never slips through the net**.

🔢 **WORKED EXAMPLE**

The E9.5 demo dataset is 768 × 576 × 112 voxels. A ray crossing the whole cube lengthwise (X axis) therefore takes **768 samples**. A diagonal ray, which is longer ($\sqrt{768^2 + 576^2 + 112^2} \approx 966$ voxels), takes **about 966**.

🔧 **FOR THE CURIOUS**

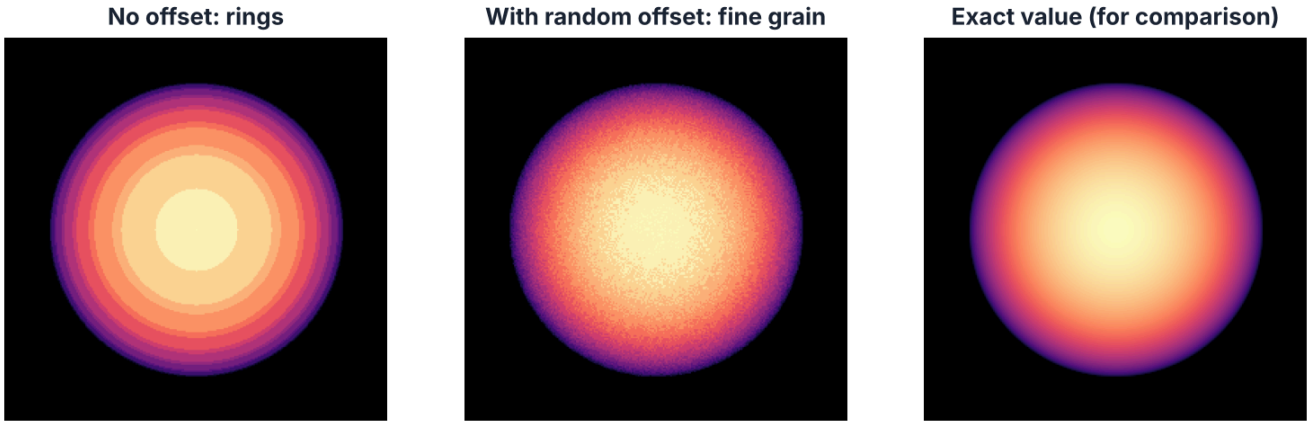
The step (in cube units) is `delta = 1 / (rate × nu)`, where `nu = ‖direction × (number of voxels of the level)‖` measures how many voxels the ray crosses per unit of length and `rate = 1.0` at rest. The step is therefore uniform in voxels, not in micrometres. If a ray would need more than 4,096 steps, they are spread out more widely (see 9.9).

## 9.4 Random grain against banding

With a fixed step, all neighbouring rays sample at the **same depth**. The result is **rings** (“banding”), like the contour lines of a map. They do not exist in the specimen: they are created by the calculation.

The remedy is simple: the **first sample of each ray is shifted** by a random fraction of the step (a pseudo-random number computed from the pixel’s position). The rings turn into a **fine grain**, which the eye averages out and ignores.

A uniform sphere, one sample every 14 pixels (simulation)

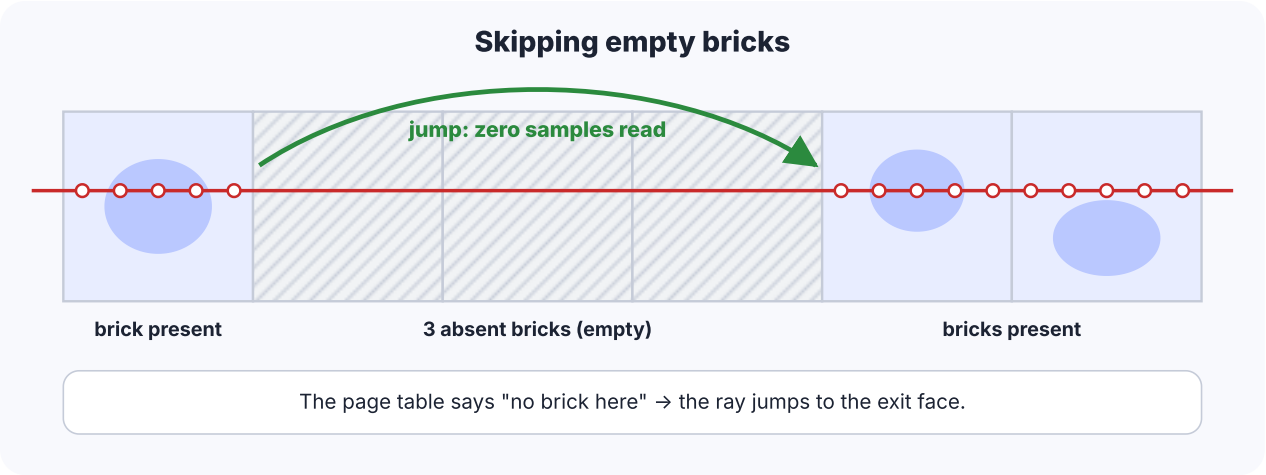



▲ Simulation: uniform sphere, one sample every 14 pixels. On the left, the rings; in the middle, the grain after random offset.

The same offset is used when exporting an image: it carries on from one tile to the next, with no visible seam.

## 9.5 Skipping the empty space

An embryo fills only part of its box. Bricks with no signal are not even stored (chapter 7). When a ray reaches an **absent** brick, there is nothing to read: it **jumps straight** to the exit face of that brick.




▲ The ray crosses three empty bricks without reading a single voxel.

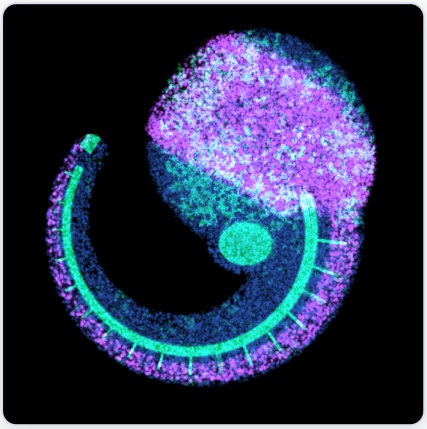
The jump follows the same random grid as the rest of the ray: the image does not change, it is just computed faster.

ℹ️ **GOOD TO KNOW**

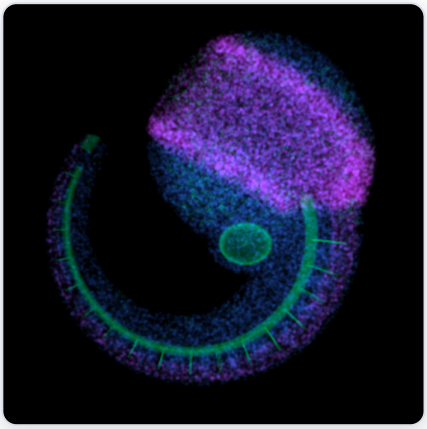
An absent brick reads as **zeros**: black that costs almost nothing, not a hole in space.

## 9.6 The three render modes

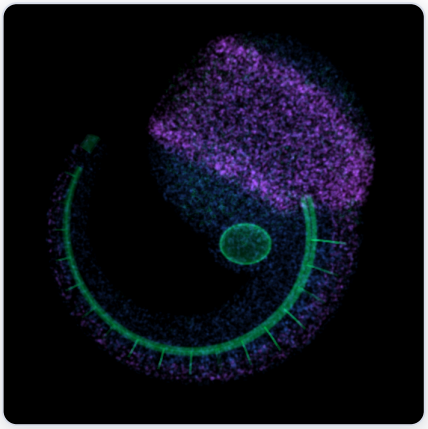
The **Render mode** menu (left-hand panel) offers three ways of summarising the values along a ray. Here is **the same viewpoint** of the E9.5 demo dataset (DAPI, Pecam1, Sox2), rendered three times.



▲ **Fluorescence (Imaris-like)**



▲ **Natural fluorescence (depth)**



▲ **Structure (DVR)**

ℹ️ **GOOD TO KNOW**

The middle and right-hand captures are lit with a higher **Visibility (exposure)** slider (see the caption of the figure further down): at equal exposure, these two modes naturally produce a darker image.

| MODE | WHAT IT DOES ALONG THE RAY | DEPTH? |
|---|---|---|
| **Fluorescence (default mode)** | keeps the **strongest** value of each channel | no, everything is superimposed |
| **Natural fluorescence** | every point **glows**; what is in front **darkens** what is behind | yes |
| **Structure (DVR)** | stacks **semi-transparent** layers from front to back | yes |

## Mode 1: Fluorescence (the maximum)

This is the “classic” rendering of microscopy software. For **each channel separately**, the viewer keeps the **maximum value** met along the ray. The channel colours are then **added together**.

🔢 **WORKED EXAMPLE**

A ray crosses green at 0.85 and then magenta at 0.75. The pixel receives 0.85 of green **and** 0.75 of magenta: the magenta is not hidden by the green in front of it. Green + magenta added together give white where both are strong.

🔧 **FOR THE CURIOUS**

`mip = max(mip, v)` for each channel; final colour = Σ `mip_i × colour_i` × exposure. The ray stops early when **every** active channel has reached its ceiling (opacity × 0.999). A pixel whose maximum is below 0.004 stays transparent.

## Mode 2: Natural fluorescence (emission and absorption)

Mental picture: each fluorophore is a **small lamp** glowing in its own colour, and the dense matter in front of it **filters** the light. You get relief and depth while keeping faithful colours.

🔧 **FOR THE CURIOUS**

For each sample of density `d` = value of the strongest channel (if `d > 0.0025`):

- segment opacity (Beer-Lambert law): `a = 1 − exp(−absorption × delta × d)` with `absorption = 1.8` ;
- emitted light: `T × emissionGain × exposure × delta × Σ v_i × colour_i` with `emissionGain = 2.2` , where `T` is the **transmittance** (the share of light not yet absorbed, from 1 down to 0);
- then `T ← T × (1 − a)` ; stop when `T < 0.004` .

At the end, the **luminance** (Rec. 709 formula) is compressed with an extended Reinhard curve (white point 2.0) and the three components are multiplied by **the same ratio** (“chromaticity lock”): a very bright fluorophore stays green and never drifts to white. Only the overlap of 3 or 4 channels is lightened (by at most 50%). A saturation of 1.18 is applied at constant luminance.

⚠️ **WARNING**

The absorption is **not a measurement**: it is a display constant (1.8), independent of the specimen. It is there to give relief, not to quantify the real opacity of the tissue.

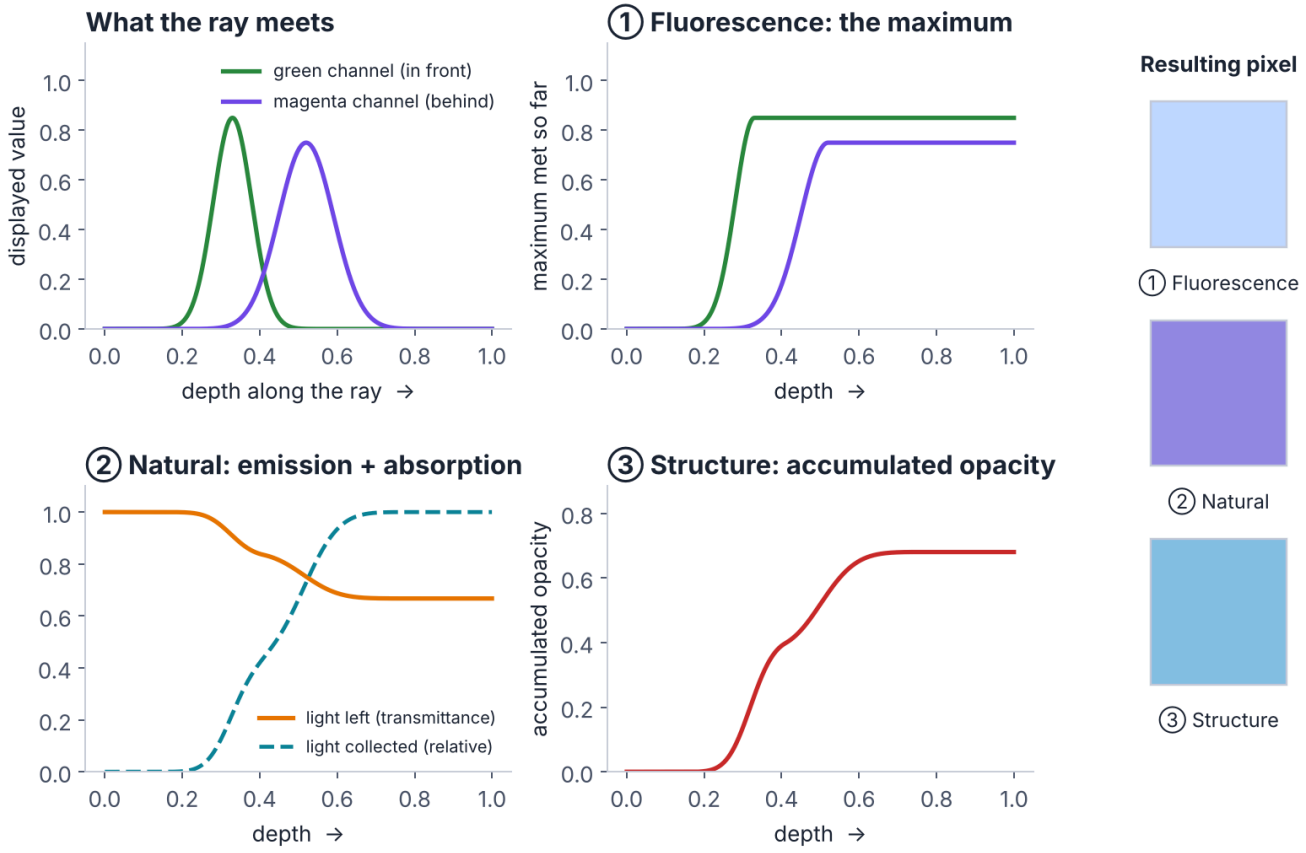
## Mode 3: Structure (DVR)

DVR (“Direct Volume Rendering”) stacks **semi-transparent** layers like tinted panes of glass, starting from the nearest. Once the stack is almost opaque (**97%**), the ray stops: what lies behind is no longer visible.

> 🔧 **FOR THE CURIOUS**
>
> For each sample of local opacity `a = max of the channels` (if `a > 0.01`): `step = 1 − (1 − 0.05 × a)^(delta / 0.01)` then `colour += (1 − alpha) × step × local_colour` and `alpha += (1 − alpha) × step`.
>
> The exponent makes a column of matter become opaque **according to its length**, not to the number of samples: the image does not change when the viewer lightens the step during rotation.

## One ray, three answers

Imagine a ray that first meets a green cloud and then, a little further back, a magenta cloud that partly overlaps it.




▲ The same ray, summarised by the three modes (computed with the viewer’s formulas; exposure slider 1 for mode 1, 4 for mode 2 and 3 for mode 3, as in the captures).

- **Fluorescence**: we keep the strongest value of each channel, **regardless of order**.
- **Natural**: the light from the magenta cloud is somewhat **attenuated** by the green cloud in front (the transmittance falls to about 0.67).
- **Structure**: the green cloud partly **hides** the magenta; the accumulated opacity ends up around 0.68.

## 9.7 Smoothing between voxels

Between two voxels, the viewer does not show staircase steps: it **interpolates**. On **format 4** (the one used by all the demo datasets), the graphics card blends the eight neighbouring voxels: this is **trilinear interpolation**.

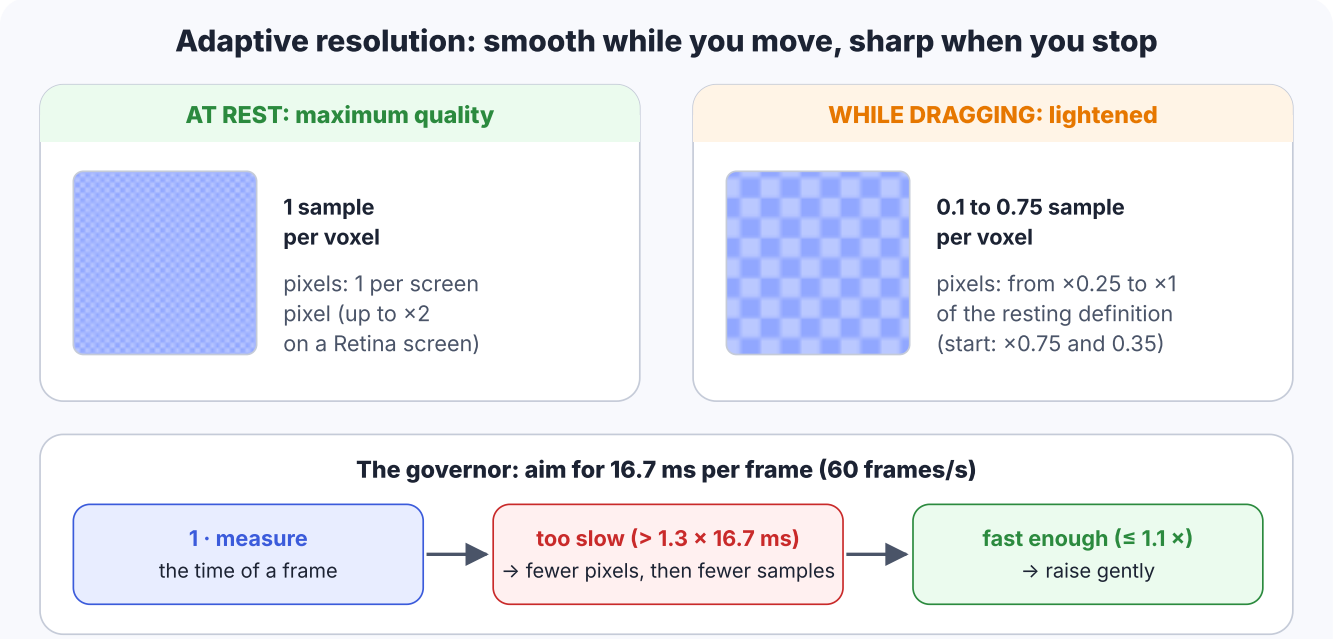
So that this blending never spills from one brick into its neighbour, each brick is stored with a **1-voxel border** ($66^3$ instead of $64^3$). At the boundaries between bricks the image is therefore continuous, with no seam.

📌 **REMEMBER**

- **Format 4** and a full 3D texture: trilinear interpolation (smooth).
- **Old format 2**, when the atlas is split into bricks: the nearest voxel is displayed as it is (blocks visible when zoomed in).

## 9.8 Smooth when you move, sharp when you stop

Rotating a volume at 60 frames per second means computing each frame in under **16.7 ms**. The viewer times every frame while you drag and **adjusts two settings** to stay within that budget.




▲ While dragging, the image definition and the number of samples are tuned continuously; as soon as you stop, everything returns to maximum quality.

| SETTING | WHILE DRAGGING | AT REST |
|---|---|---|
| **Image definition** | from ×0.25 to ×1 (start ×0.75) | 1 screen pixel (up to ×2) |
| **Samples per voxel** | from 0.1 to 0.75 (start 0.35) | **1.0** |

- Interaction is considered finished **250 ms** after the last movement.
- If a frame takes more than 1.3 × 16.7 ms, the definition is lowered first, then the samples; if it stays under 1.1 ×, they are raised gently.

- Time is measured by the **graphics card's timer** when the browser offers one, otherwise by the interval between two frames.

> ℹ️ **GOOD TO KNOW**
>
> Nothing is drawn continuously: the viewer computes a frame only when something changes (camera, setting, arrival of bricks). During loading, redraws caused by new bricks are grouped **at most every 250 ms**.

## 9.9 The safeguard against graphics-card crashes

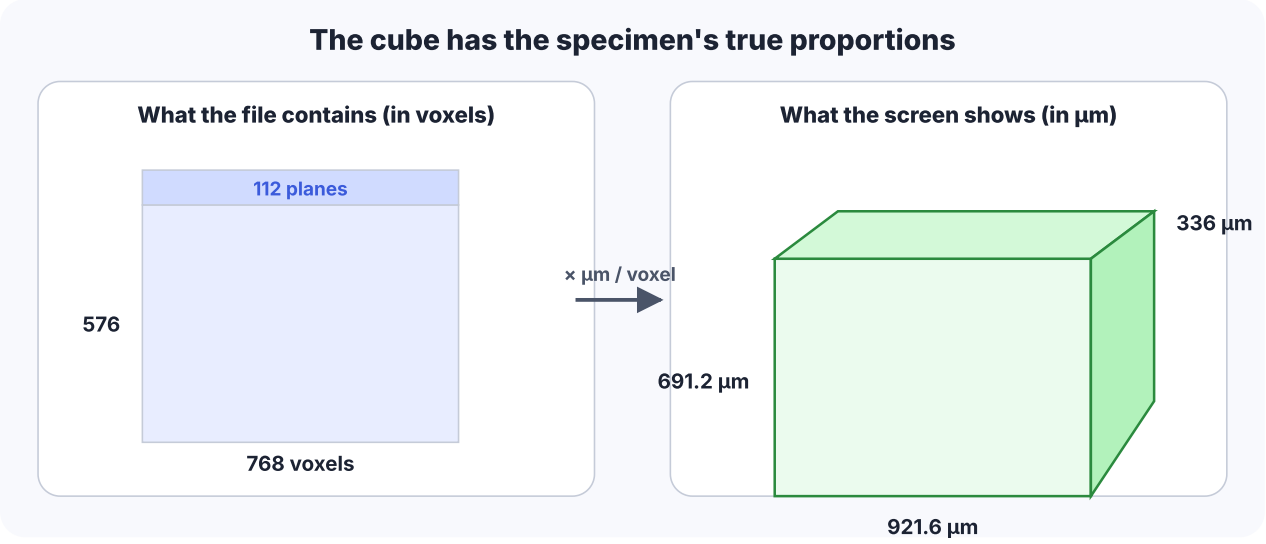
On Windows, if a single graphics-card computation lasts more than about **2 seconds**, the system resets the card: this is a "TDR" and the screen flickers. The viewer protects itself against it.

> 🔧 **FOR THE CURIOUS**
>
> For an image **at rest**, the maximum number of samples per ray is `clamp(⌊6 × 10⁹ / number of pixels⌋, 256, 4096)`. Example: a 1,920 × 1,080 screen gives ⌊6 × 10⁹ / 2,073,600⌋ = **2,893** samples at most per ray; the same screen with doubled pixels (Retina, 8.3 million pixels) gives 723.
>
> This ceiling is adjusted during use: multiplied by `max(0.5; 150 / duration)` if an image at rest takes more than 200 ms, raised by 10% if it stays under 60 ms, and halved after a graphics context loss (chapter 10). During a drag, it is one tenth of this value.

For you, this means that an exceptionally long ray may, on a very large screen, be sampled a little more coarsely than once per voxel. In normal use (volumes of a few hundred voxels), the ceiling is never reached.

## 9.10 True proportions: calibration and Z stretch

The cube is not drawn from the **number of voxels** but from the **real dimensions in micrometres**. Voxels are often thicker in Z than in X and Y; without correction, the embryo would look squashed.




▲ The file counts voxels; the screen shows micrometres. E9.5 demo dataset: voxels of 1.2 × 1.2 × 3.0 µm.

🔢 **WORKED EXAMPLE**

E9.5 dataset: 768 × 576 × 112 voxels of 1.2 × 1.2 × 3.0 µm.

- X: 768 × 1.2 = **921.6 µm**; Y: 576 × 1.2 = **691.2 µm**.
- Z: (112 − 1) × 3.0 + optical section thickness 3.0 = **336 µm**.
- The longest side (X) is set to 1; the cube therefore measures **1 × 0.75 × 0.365** (336 / 921.6).

The viewer sorts the calibration into three states: **exact** (voxel sizes and section thickness known), **estimated**, or **absent**. Without calibration, the 3D **scale bar** is hidden rather than wrong. This status is **recomputed by the viewer** from the metadata it receives (“exact” only if a section thickness is known); the pipeline itself writes only `exact` or `metadata-missing` (chapter 8).

In the **Physical Scale** panel, the **Display Z override** slider (from ×0.25 to ×2.0) stretches only the **display** in Z so that thin layers are easier to see; the **1:1** button resets it. Measurements in µm ignore this stretch.

⚠️ **WARNING**

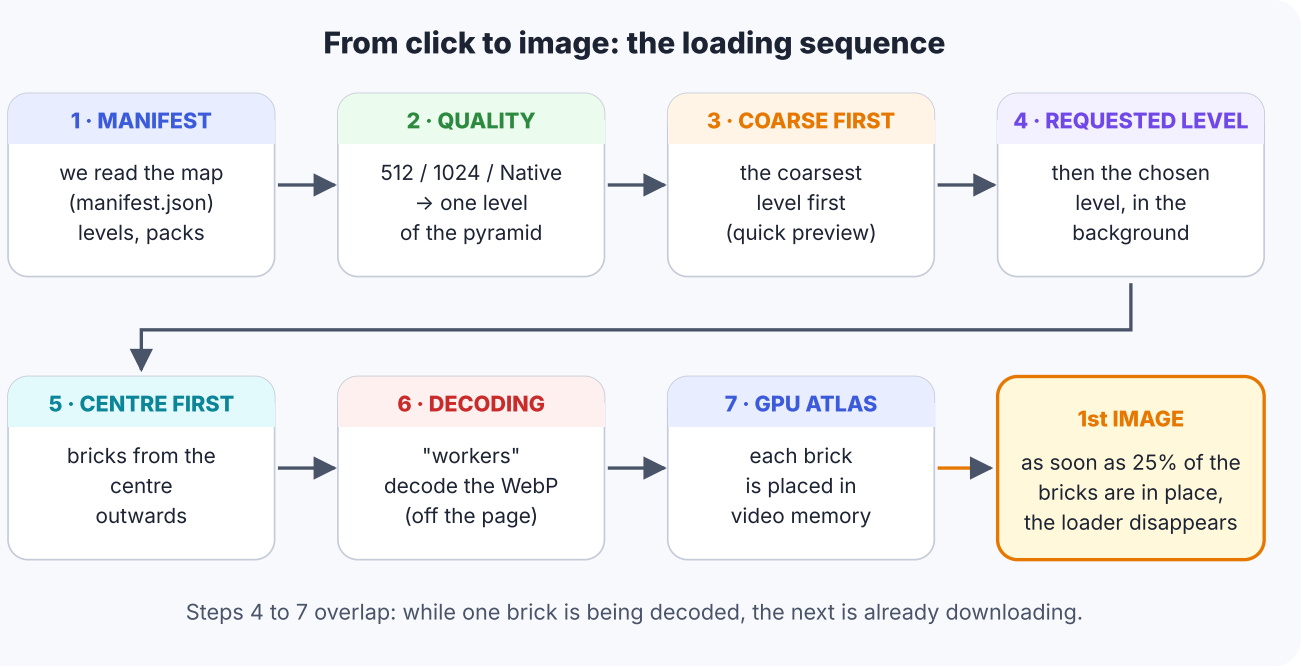
The 3D scale bar is exact for whatever lies **at the depth of the specimen’s centre**. The view is a perspective one: what is closer looks larger, and what is further away looks smaller.

# 10. Displaying gigabytes: streaming

- The browser **never** loads the whole volume: it downloads **bricks** (small cubes of $64^3$ voxels) from the centre outwards and stores them in the graphics card's memory.
- A first image appears as soon as **25%** of the bricks are in place; quality then climbs without blocking the screen.
- If graphics memory is insufficient, the viewer automatically picks a **coarser level** and tells you so.

## 10.1 The loading sequence

When you open a dataset, seven steps follow one another. You do not see all of them, but they explain why the image becomes sharp **progressively**.




▲ From click to image: seven overlapping steps.

1. **Manifest.** The viewer reads the dataset's "map" (`bricks/manifest.json`): pyramid levels, brick size, pack files.
2. **Quality.** The **Render Quality** menu is translated into a pyramid level (section 10.2).
3. **Coarse first.** On first opening, the **coarsest** level is loaded first: a few bricks, an image in the blink of an eye.
4. **Requested level.** The chosen level then loads in the background.

5. **Centre outwards.** Bricks are ranked by their distance from the centre, **measured in micrometres** (so that the large voxel thickness in Z does not distort the “centre”).

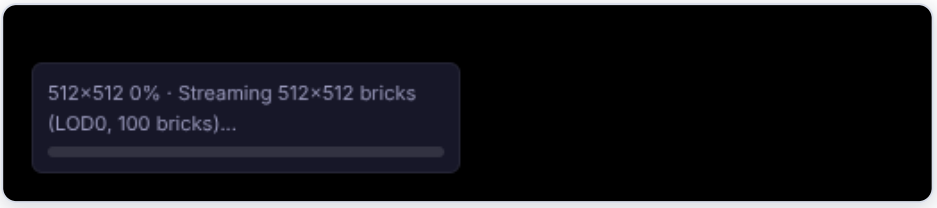
6. **Decoding.** “Workers” decode each brick outside the page (section 10.5).

7. **Atlas.** The decoded brick is copied to its place in the graphics card’s memory (section 10.3).

📌 **REMEMBER**

The **first image** is displayed when **25%** of the bricks of the current level are on the graphics card; the “loading” card then disappears. Redraws caused by incoming bricks are grouped (at most one every 250 ms).

### Where to see progress

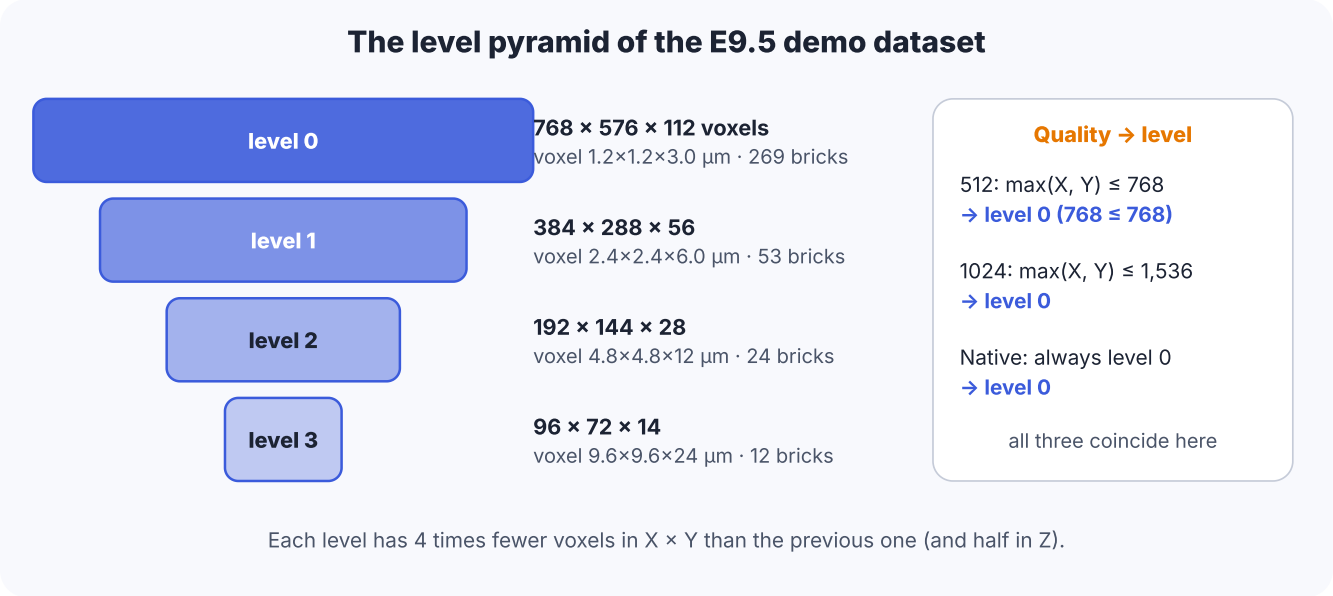



▲ The progress line at the bottom left of the viewer: quality, percentage of bricks processed and current step.

The percentage is the share of bricks **already processed** (placed or abandoned), not of bytes downloaded. The line disappears when the level is complete.

## 10.2 The quality menu: 512, 1024, Native

The pyramid of levels was built by the pipeline (chapter 6): each level has **4 times fewer voxels in X × Y** than the previous one. The menu picks **the finest level that remains reasonable** for the requested setting.




▲ The real pyramid of the E9.5 demo dataset and the rule behind the three qualities.

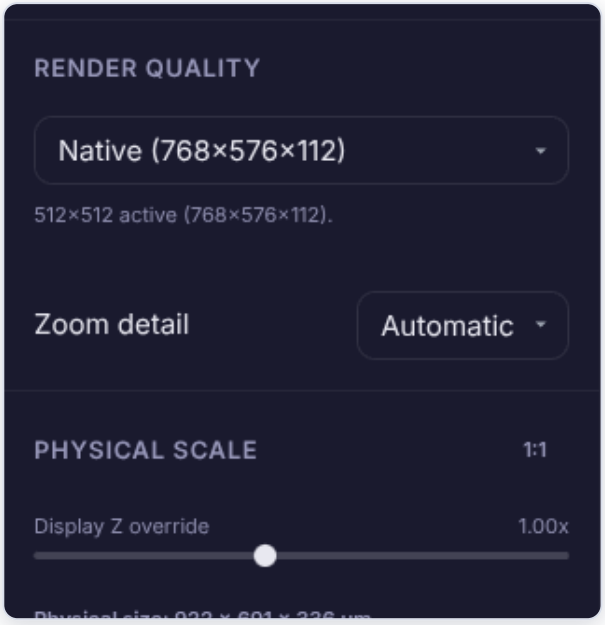
| SETTING | LEVEL CHOSEN (FORMAT 4) |
|---|---|
| **512 (default)** | the finest level whose longest XY side is **≤ 768 voxels** |
| **1024** | the finest level whose longest XY side is **≤ 1,536 voxels** |
| **Native** | always **level 0** (full resolution) |

🔢 **WORKED EXAMPLE**

A **fictional** dataset of 3,072 × 2,304 voxels in XY, with levels at 3,072, 1,536, 768 and 384:

- **512** → level at 768 (≤ 768); **1024** → level at 1,536 (≤ 1,536); **Native** → 3,072.

On the E9.5 demo dataset, the longest side is 768: all three settings land on level 0. The menu then **shows only one option**, with the real dimensions: **Native (768×576×112)**.




▲ The **Render Quality** panel: one line per genuinely distinct level (here a single one, with its dimensions), the status line ("512×512 active"), and, below it, the **Zoom detail** selector (section 10.8).

ℹ️ **GOOD TO KNOW**

The viewer opens at **512** quality. A quality change that fails leaves the previous volume on screen and puts the menu back as it was.

## On a format-5 dataset: the matrix

On a format-5 dataset (section 7.11), the menu gives way to a **table**: one row per level, one column per compression. Each cell is a way of displaying the same volume; **a click loads it**.

▲ The **Render Quality** panel of a format-5 dataset (demonstration dataset; sidebar widened to show the five columns).

| N | WHAT IT IS |
|---|---|
| 1 | The **rows**: one level, with its real dimensions (here native 768 × 576 × 112, then 512 and 256). |
| 2 | The **columns**: **Lossless**, **±2**, **±4**, **Light**, **Very light**. A tooltip on the header explains each compression. |
| 3 | The level's **graphics memory** ("GPU 21.0 MB"), with a coloured mark. |
| 4 | The **active cell**, framed: here **512 · ±2**, the opening setting. Each cell shows the **size to download** and, once the throughput is measured, the **estimated time**. |

| COLOUR | MEANING |
|---|---|
| **green** | fits within three quarters of the estimated graphics memory |
| **amber** | tight: above three quarters, or only thanks to graphics compression (section 10.3) |
| **red** | does not fit: the overview stays at the finest level that fits, and **zoom detail** (section 10.8) fetches the chosen cell around what you look at |

🔢 **WORKED EXAMPLE**

**Demonstration dataset** (read on the capture). The lossless native weighs **7.5 MB**; the opening cell, 512 · ±2, **2.3 MB**, three times less; 256 · Very light, **0.4 MB**. The graphics memory depends only on the row: 189 MB at native, 84 MB at 512, 21 MB at 256.

- The chosen cell is kept in the workspace, the page address ( `512x512@b2` , `native@ll` ...) and this browser.
- **Zoom detail** stays in the **same column** as the chosen cell.

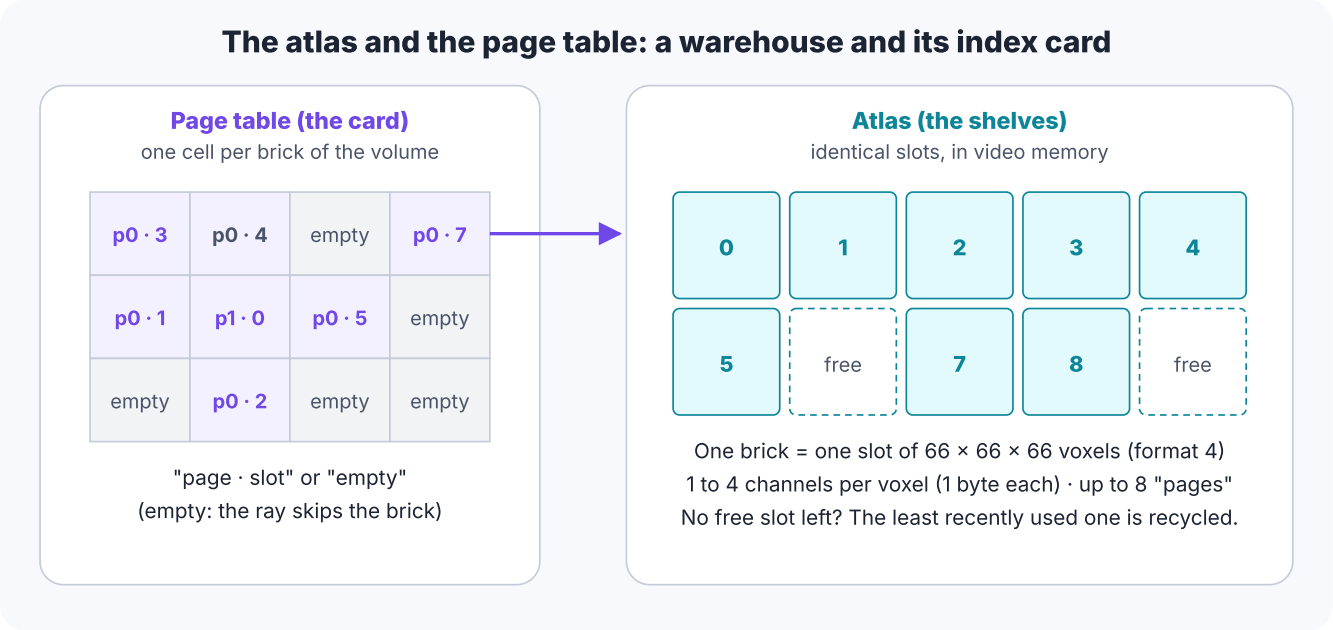
- The **Compare** page still shares memory by rows; each panel keeps its column.
- The sidebar is 320 pixels wide: the last columns are reached by **scrolling the table to the right**.

> ⚠️ **WARNING**
>
> A lossy column **changes the displayed voxels**, not the data: measurements, computing plugins and “lossless” exports always read the lossless column of the native level (chapters 12 and 15).

## 10.3 The GPU atlas and the page table

A brick of $64^3$ voxels is too small to deserve its own texture; creating hundreds of them would be unmanageable. The viewer therefore reserves in video memory **large blocks of identical slots**: the **atlas**.

> 💡 **A PICTURE TO UNDERSTAND**
>
> The atlas is a **warehouse** whose shelves are all the same size. The **page table** is the **index card** at the entrance: “brick no. 17 is on shelf 4”. When the ray needs a voxel, it consults the card, then goes straight to the right shelf. If the card says “empty”, it does not even move.




▲ The atlas and its page table.

- **One table cell per brick** of the volume: page number + slot, or “empty”.
- Up to **8 atlas pages**, each 4, 8 or 16 slots per side.
- **1 to 4 channels** per voxel, one byte each: 1 channel = 1 byte per voxel, 2 channels = 2, 3 or 4 channels = 4.
- When all slots are taken, the **least recently used** one is recycled.
- The atlas is **sized before it is allocated**: no texture is ever created at random for a brick.

🔧 FOR THE CURIOUS

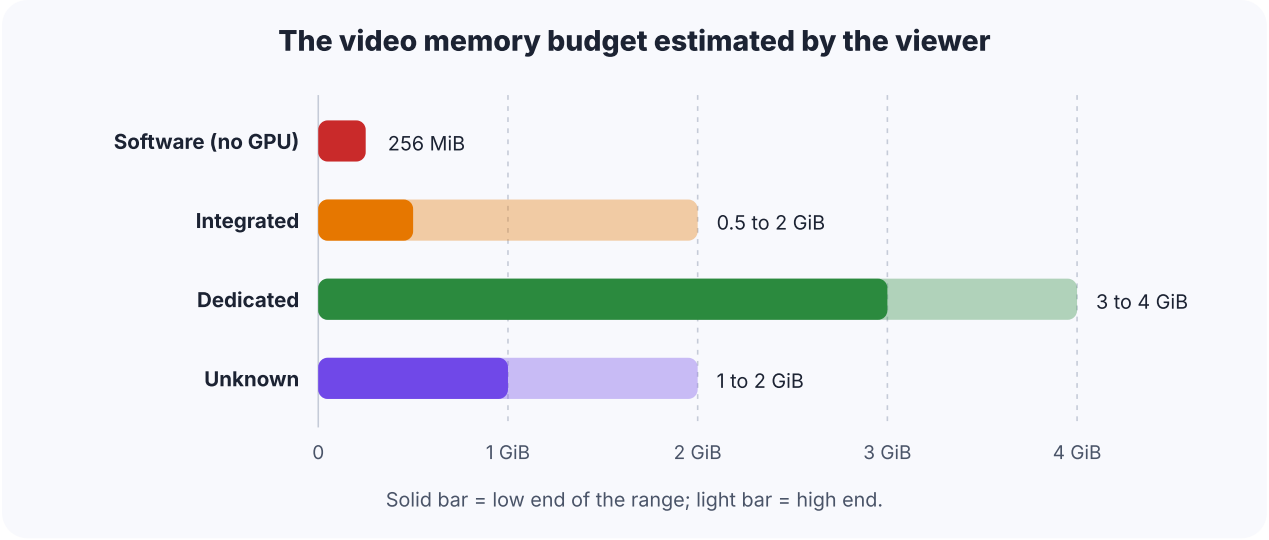
A small volume stays in one piece: if the level fits in less than **512 MiB** (at 4 bytes per voxel) and within the card's limits, it is stored in **a single dense 3D texture**. The E9.5 dataset at level 0 is an example: 768 × 576 × 112 × 4 bytes ≈ **189 MiB**. The paged atlas is used only for large levels.

### Graphics compression (timelapses)

Since platform 1.60, the frames of a **timelapse** can be kept on the graphics card in **compressed** form (BC4 / BC5 blocks): **0.5 byte** per voxel and channel instead of 1, so about **twice as many time points** in memory. It is display only (displayed values differ by about 1 to 2 grey levels); the Studio and zoom detail stay exact. Setting **GPU compression** of the panel: **Timelapses** (default), **On** or **Off**. On a software renderer the option is unavailable and the panel says so.

## 10.4 The video memory budget

No browser tells you how much video memory is left. The viewer **estimates it** from the type of graphics card and, if available, the device's memory.




▲ The estimated budget by graphics-card type (1 GiB = 1,024 MiB).

| GPU TYPE | ESTIMATED BUDGET |
|---|---|
| **Software (no real graphics card)** | **256 MiB** |
| **Integrated (Intel, Apple…)** | **0.5 to 2 GiB** depending on the device's memory |
| **Dedicated (NVIDIA, Radeon…)** | **4 GiB** (3 GiB if the device reports less than 8 GB) |
| **Unknown** | 1 or 2 GiB |

Each graphics **context loss** in the past week **halves the budget** (floor: 256 MiB). The administrator can set it by hand if needed.

### When it does not fit

Before allocating anything, the viewer computes the size of the requested level. If it exceeds the budget, it tries the **next coarser** level, and so on.

🔢 **WORKED EXAMPLE**

**Fictional** example: you ask for **Native** and level 0 needs 1.9 GiB for a budget of 1 GiB: the viewer moves to level 1 (about 4 times smaller), which fits. A notice appears:

"{requested} resolution exceeds available GPU memory — displaying {actual} instead. ({needed} MB needed, {budget} MB available for the volume.)"

If the card **refuses** an allocation despite the calculation, the viewer also falls back to a coarser level; the notice says so: "(The GPU refused the allocation.)". The volume **always stays visible**, never a crashed tab.

## 10.5 Downloading and decoding without blocking the page

Decoding a brick (a WebP image to be unpacked into voxels) is heavy work. It is handed to a **pool of workers** that run **alongside the page**: the interface stays smooth.

- **Number of workers**: `min(8, number of cores − 1)`, at least 1.
- **Faithful decoding**: no browser colour management is applied; the bytes are **measured intensities**, not colours.
- **Mosaic**: a $66^3$-voxel brick is a WebP image of 594 × 528 pixels (9 × 8 tiles of 66 × 66); an image of any other size is rejected as corrupt.
- **No needless copying**: the decoded bytes are transferred as they are to the page, then sent to the graphics card.
- A worker that crashes or freezes (30 s) is **replaced**; the brick is retried up to 3 times, then counted as missing.

⚠️ **WARNING**

A corrupt brick is **never** displayed: it stays empty and the viewer reports it (for example: "N bricks could not be loaded"). An incomplete stream is not kept in memory: reload to try again.

## 10.6 Packs: an ordered download

Bricks are stored in **packs** (`.bin` files). Downloading a whole pack is far more efficient than thousands of small requests.

- **"Pack by pack" order**: the viewer starts from the central brick and moves on pack after pack; each pack is downloaded **only once** and released as soon as its bricks are cut out.
- **At most 4 pack downloads at the same time**, because a browser opens 6 connections per server: the page's own calls must not wait behind big files.
- **Precise cuts**: for a slice or a Studio image, only the useful **byte ranges** are requested.

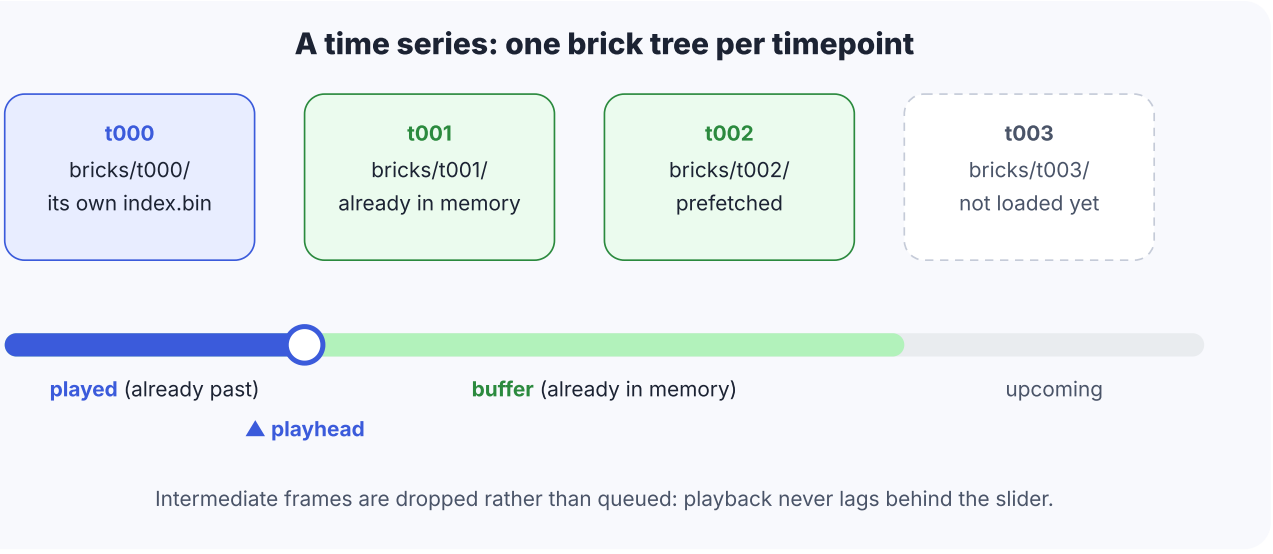
| CACHE (COMPRESSED BYTES) | SIZE |
|---|---|
| Whole packs, while a load still needs them | 192 MiB |
| Byte ranges (slices) | 64 MiB |
| Prefetch (neighbouring frames of a series) | 32 MiB |

🔧 FOR THE CURIOUS

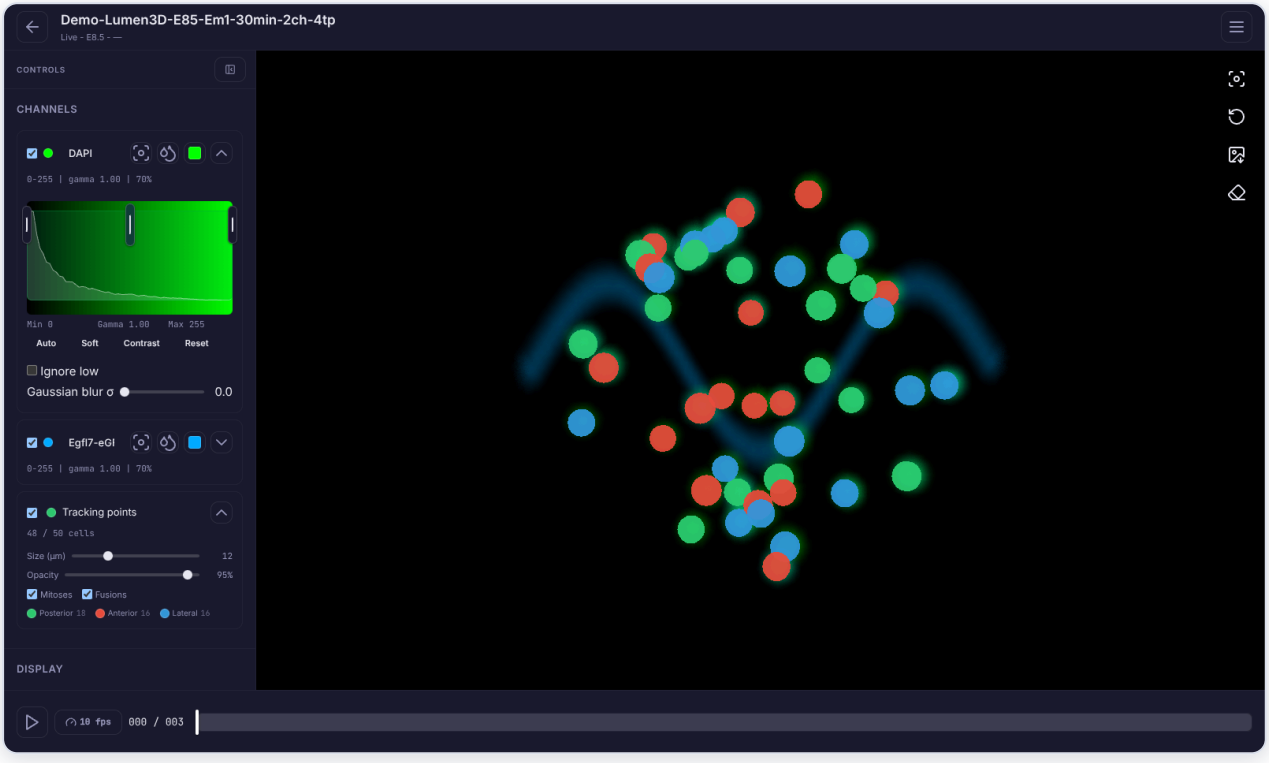
**Decoded** bricks are never kept on the page side: the GPU atlas is their only home. Each pack address carries a `?v=` stamp (a fingerprint of the manifest's packs); the server then answers "keep for one year": if the dataset changes, the fingerprint changes and the browser downloads again.

## 10.7 Time series

A **live** dataset (time series) contains **one brick tree per timepoint**. Each timepoint is loaded as a small volume in its own right.




▲ Each timepoint has its own tree; the buffer shows the frames already in memory.



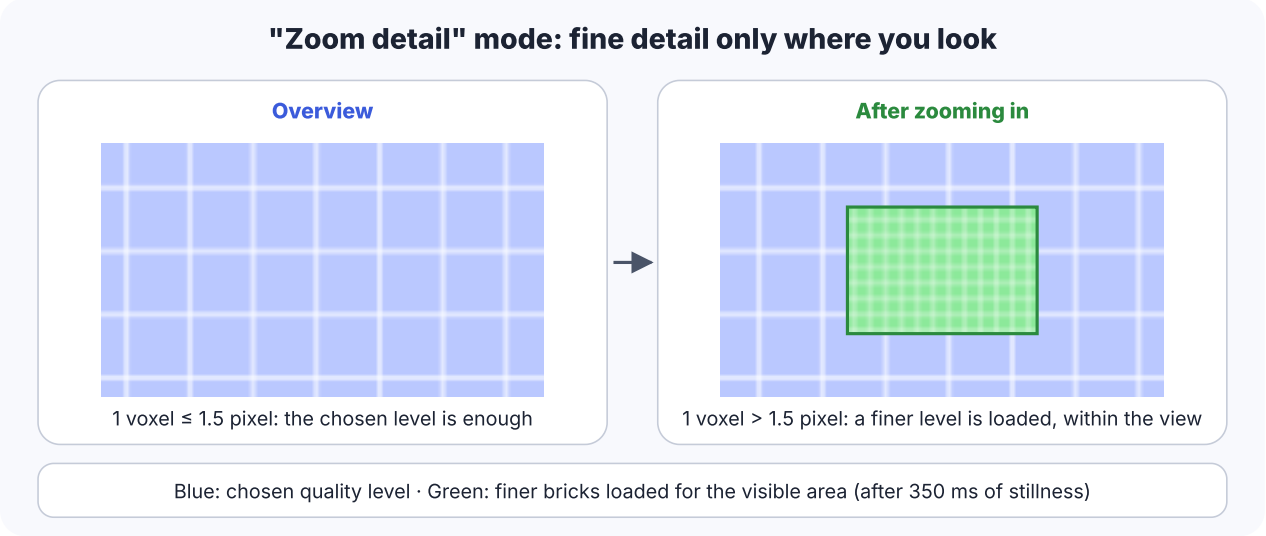

▲ The timeline of the Demo-Lumen3D-E85-Em1-30min-2ch-4tp demo dataset: playback, speed, time slider and buffer bar.

- **Playback**: speeds of **0.5 – 1 – 2 – 5 – 10 – 20 frames per second** (10 by default, remembered in your browser).
- **One request at a time**: if you move the slider quickly, only the **last** request is served.
- **Playback waits for the frame**: the clock is suspended while a frame loads; playback runs at the real speed of arrival.
- **Buffer**: the viewer keeps neighbouring frames in memory (up to `min(768 MiB, max(128 MiB, 40% of the budget))`). The timeline bar shows those **actually present** at the current quality. A text says so if the series does not fit entirely: “Buffer n/total · quality — the whole series does not fit in memory (… frames max)”.
- **Prefetch**: during playback, the packs of the following frames (t+1, t+2…) are prefetched in spare time.

> ℹ️ **GOOD TO KNOW**
>
> Full prefetching is only possible for levels stored as a **dense texture**: a level held in a paged atlas is not prefetched.

## 10.8 The “Zoom detail” mode

You zoom in hard on an area: the voxels of the chosen level become larger than the screen needs, and the image looks blurred. The **Zoom detail** mode then loads **finer bricks, only in the visible area**.




▲ The chosen level everywhere (blue), and finer bricks only in the view (green).

| OPTION | EFFECT |
|---|---|
| **Automatic** | active if the memory budget exceeds **1 GiB** |
| **On** | active even below 1 GiB |
| **Off** | never |

- **Trigger**: a voxel of the current level covers **more than 1.5 screen pixels**.
- The level chosen is **the coarsest** of the finer levels that drops below 1.5 pixels per voxel (the fewest bytes possible).

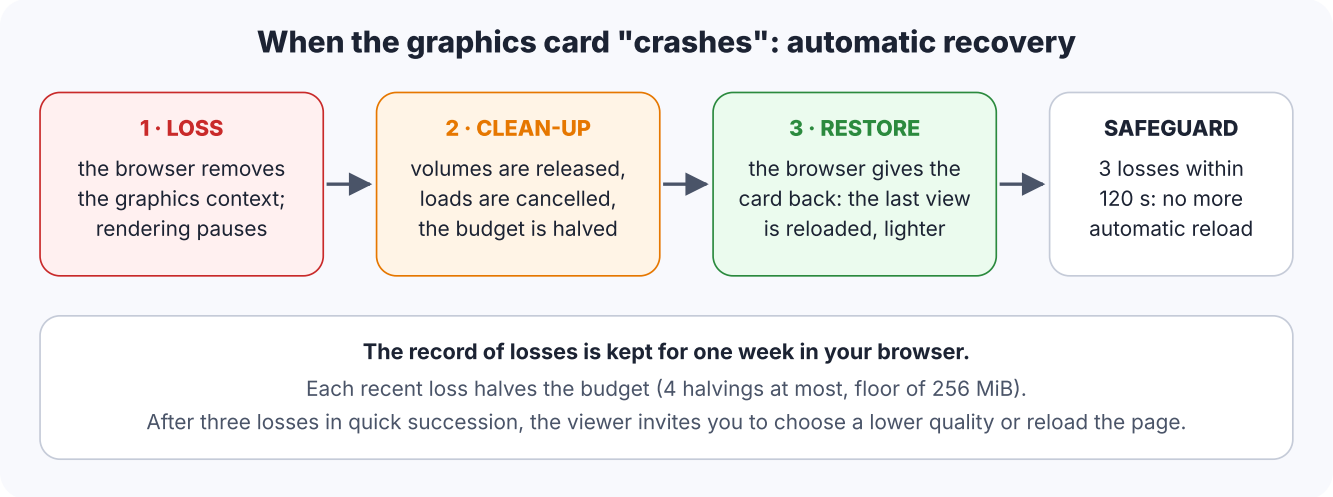
- Loading starts **350 ms** after the camera has stopped, 8 bricks at a time, **the largest and most central first**. It uses at most 768 MiB and 75% of the remaining memory.
- At the boundary of the fine area, the image switches from one level to the other **at a brick face**, with no cross-fade.

> ⚠️ **WARNING**
>
> The mode exists only for **formats 4 and 5** (bricks with a border); in format 5 it stays in the chosen compression column. It is suspended while a volume is loading and as soon as you apply a **Gaussian blur** to a channel (chapter 11), because raw bricks on a blurred channel would give a different image.

The status text shows, depending on the case: “Detail: N bricks of level L in view”, “Detail: loading x of y bricks of level L”, or “Detail unavailable: not enough GPU memory left”.

## 10.9 When the graphics card crashes

A browser can take the graphics card away from a page (a struggling driver, too much memory requested). The viewer knows how to **restart on its own**.




▲ Recovery after a graphics context loss.

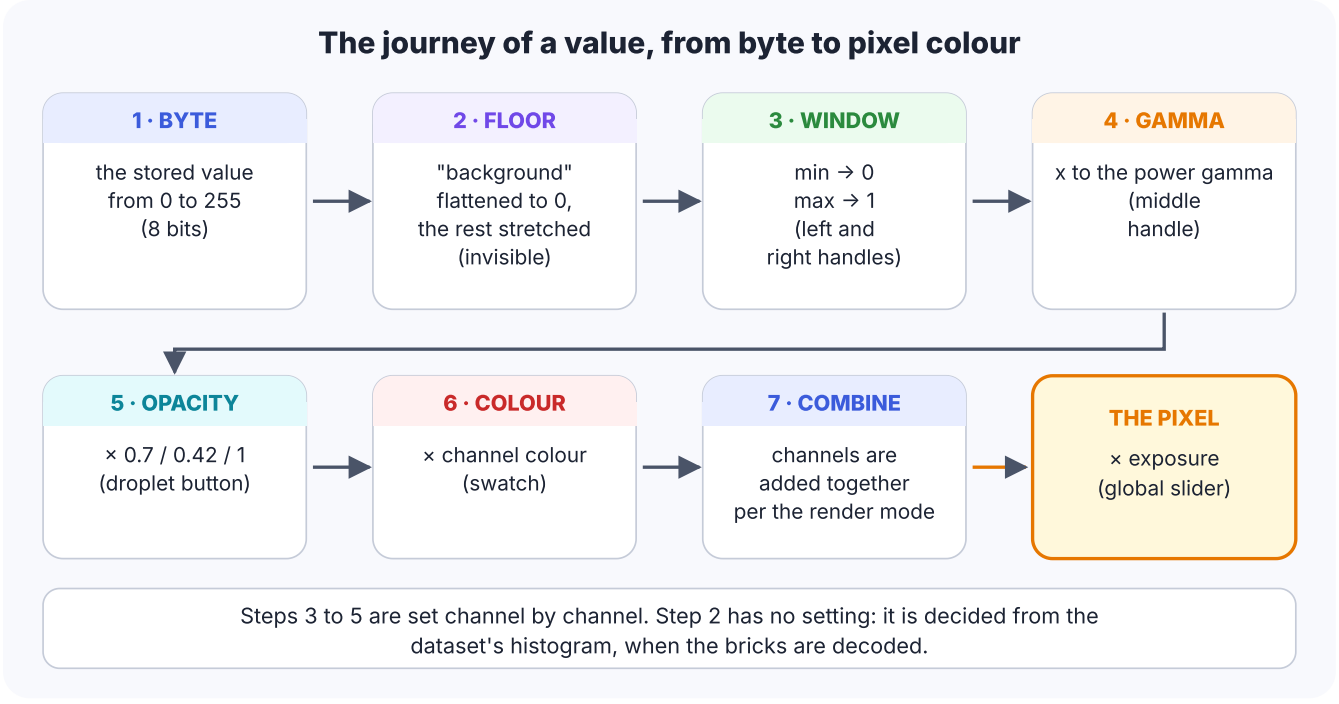
You first see “GPU context lost: rendering is paused until the browser gives it back.”, then the volume reloads in a lighter form. After **three losses within 120 seconds**, it stops and invites you to choose a lower quality or reload the page.

# 11. Colours, contrast and histograms

- Every channel follows the same chain of settings: **invisible floor → min / max window → gamma → opacity → colour**, then the channels are combined.
- The **histogram** shows how the 256 grey levels are distributed; its three handles set the window and the gamma.
- Grey levels are **relative** to one dataset: they are not calibrated fluorescence intensities.

## 11.1 The processing chain of a channel

Each voxel is stored as one **byte** (a number from 0 to 255). Before becoming a coloured pixel, it goes through seven steps. Five are within your reach; only one is hidden.



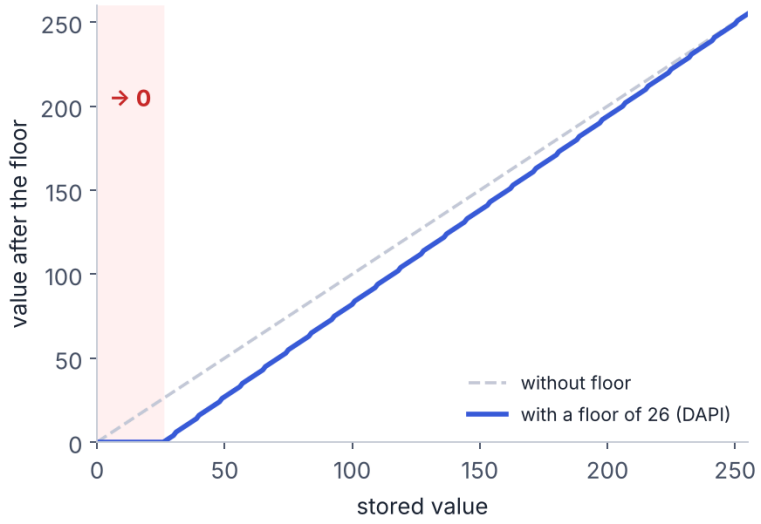

▲ The journey of a value: from the stored byte to the pixel colour.

| STEP | WHO DECIDES? | WHERE TO SET IT |
|---|---|---|
| **Floor** | automatic (histogram) | nowhere |
| **Min / max window** | you | left and right handles |
| **Gamma** | you | middle handle |
| **Opacity** | you | droplet button (3 values) |
| **Colour** | you | colour swatch |

| STEP | WHO DECIDES? | WHERE TO SET IT |
|---|---|---|
| Exposure | you | **Visibility (exposure)** slider, global |

## 11.2 The hidden step: the background floor

Before you touch any setting, the viewer **flattens the background**: every value less than or equal to a threshold, the **floor**, becomes **0**, and the remaining values are **stretched** to fill the whole 0–255 scale.




▲ The effect of a floor of 26 (the estimated real value for the DAPI channel of the E9.5 dataset).

🔧 **FOR THE CURIOUS**

The floor `f` is estimated from the dataset's histogram: we look for the first level where the cumulative count reaches **bin 0 + 20% of the non-zero voxels**, take the upper edge of that bin, add 2 and clamp the result between **6 and 48**. Then: `value' = 0` if `value ≤ f`, otherwise `round((value − f) × 255 / (255 − f))`.

🔢 **WORKED EXAMPLE**

Floor of 26 (DAPI): a voxel stored at **120** becomes (120 − 26) × 255 / 229 ≈ **105**; a voxel at 20 becomes **0**. For the E9.5 dataset the estimated floors are 26 (DAPI), 10 (Pecam1) and 22 (Sox2).

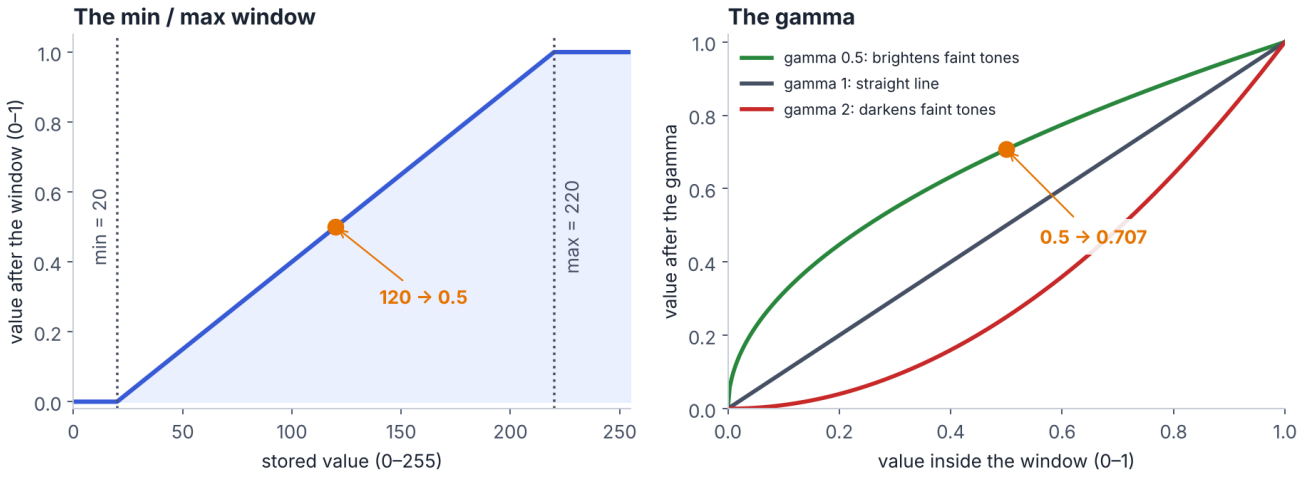
ℹ️ **GOOD TO KNOW**

The values you read on the handles (Min 0, Max 255…) are already **after** this floor. You never see it, but it explains why the background is so clean.

## 11.3 The min / max window and gamma

The **window** chooses which portion of the scale goes "from black to full brightness". Values below **Min** become black, those above **Max** saturate, and between the two the transition is **linear**.

**Gamma** then bends this scale: a gamma below 1 **brightens** the weak tones (useful for faint signals), a gamma above 1 **darkens** them.




▲ On the left, the window min 20 / max 220; on the right, the effect of gamma 0.5 / 1 / 2.

🔢 **WORKED EXAMPLE**

A voxel of value **120**, with Min = 20, Max = 220, gamma = 0.5 and opacity = 70%:

1. Window: (120 − 20) / (220 − 20) = **0.5**
2. Gamma: 0.5 to the power 0.5 = **0.707**
3. Opacity: 0.707 × 0.7 = **0.495**
4. Colour: on a green channel (0; 1; 0), the voxel contributes (0; **0.495**; 0), before exposure.

### The middle handle

The middle handle does not set the gamma directly: it designates the level (**within the window**) that should be displayed at **50%**. The viewer deduces the gamma from it, clamped between **0.18 and 5.5**:

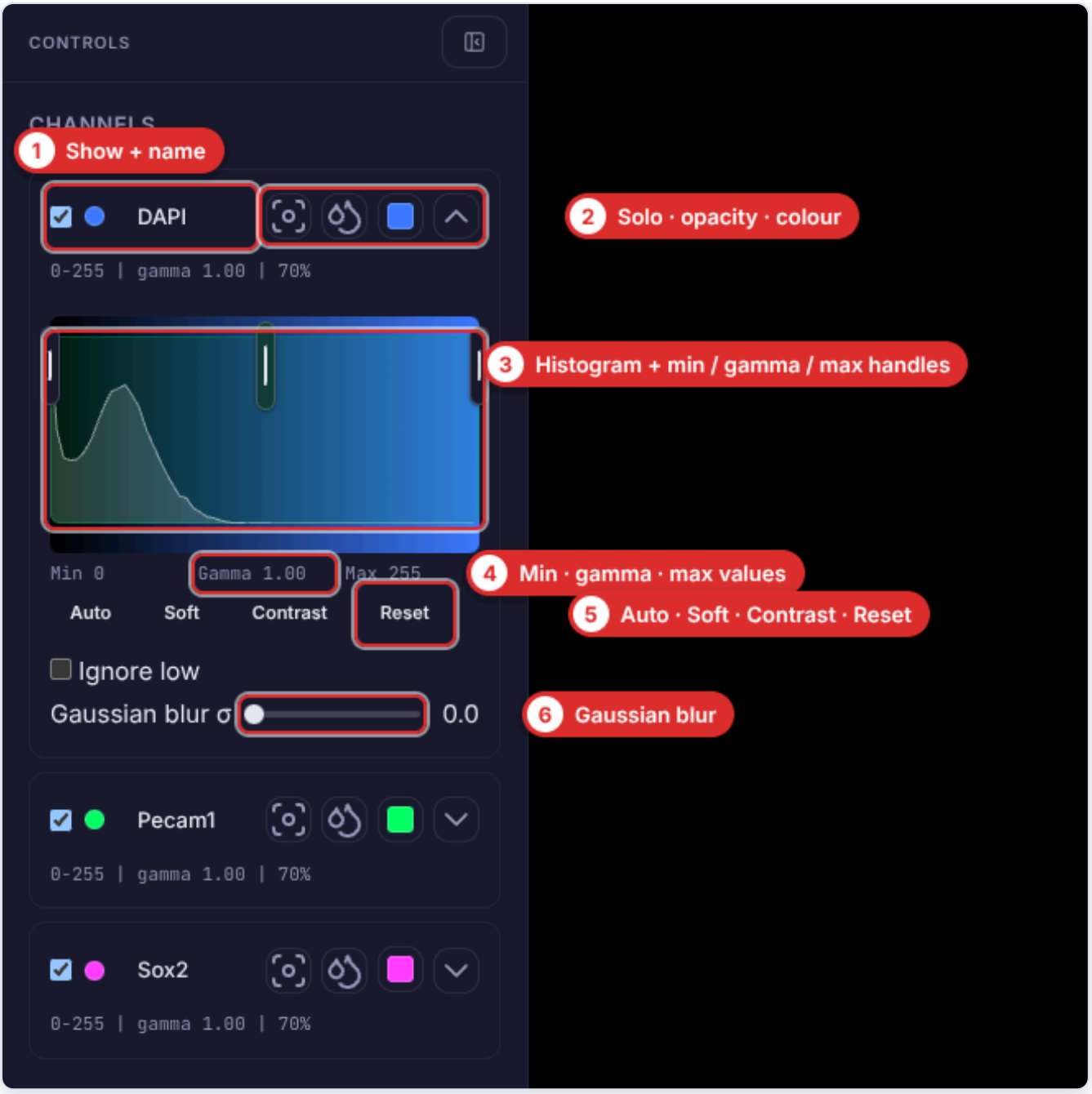
`gamma = ln 0.5 / ln m`, where `m` is the relative position of the handle between Min and Max.

🔢 **WORKED EXAMPLE**

Handle at a quarter of the window (m = 0.25): gamma = ln 0.5 / ln 0.25 = **0.5**. With Min = 20 and Max = 220, this quarter corresponds to the value 20 + 0.25 × 200 = **70**: a value of 70 is displayed at 50%. Handle in the middle (m = 0.5): gamma = 1.

## 11.4 A channel's panel

Each channel has its own card in the **Channels** panel. The summary line (for example `0-255 | gamma 1.00 | 70%`) recalls the window, gamma and opacity even when the card is collapsed.




▲ An expanded channel card (DAPI, E9.5 demo dataset).

*(numbers: see the legend below.)*

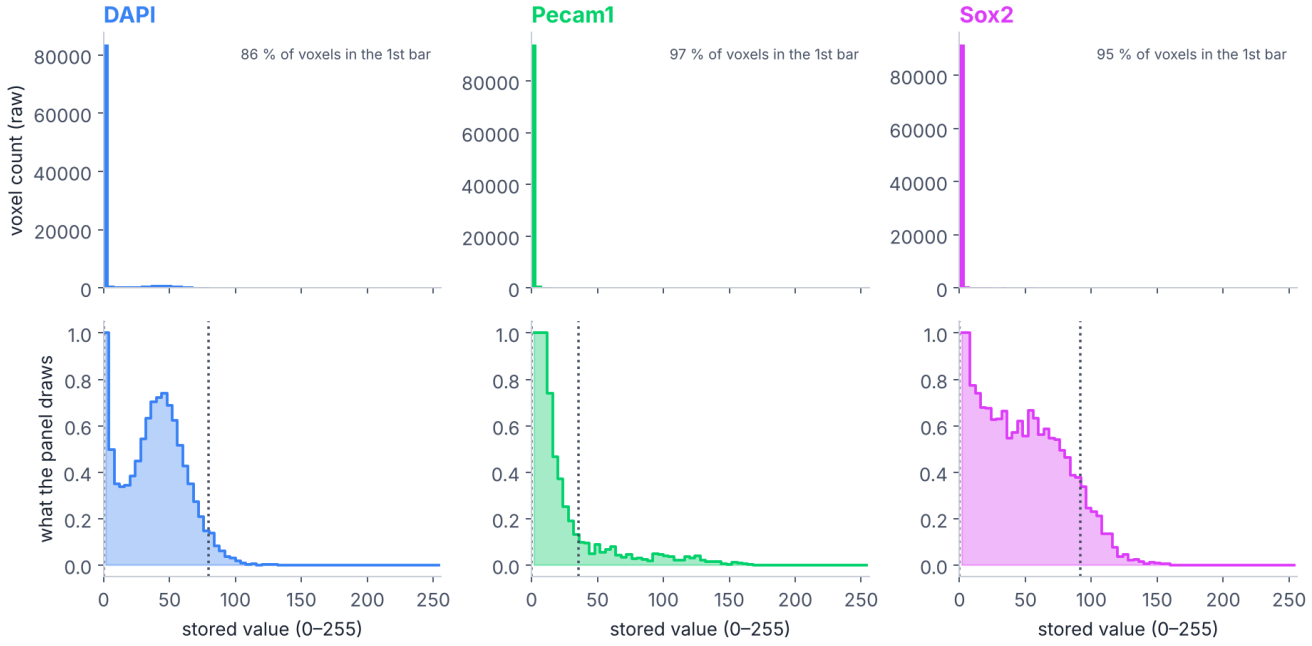
| N | WHAT IT IS |
|---|---|
| 1 | checkbox (show / hide the channel) and channel name, editable |
| 2 | buttons: **Solo channel**, opacity (droplet), colour (swatch) |
| 3 | histogram, with the three handles: Min (left), gamma (middle), Max (right) |
| 4 | values read out: Min, Gamma, Max |

| N | WHAT IT IS |
|---|---|
| 5 | quick-setting buttons: Auto, Soft, Contrast, Reset (see below) |
| 6 | **Gaussian blur σ** slider (section 11.8) |

## 11.5 Histograms

A **histogram** counts how many voxels have each value. The panel's histogram has **64 columns** (4 grey levels per column).

- It is computed **by the pipeline**, on the **coarsest level** of the pyramid, and saved in the manifest. The browser does not recompute it.
- It covers **all** voxels, background included: that is why the first column is huge (86 to 97% of the voxels of the E9.5 dataset).
- In a **time series**, each timepoint has its own histogram; the panel follows it when you change frame.
- It represents the **stored bytes** (before the floor and the window).




▲ The real histograms of the three channels of the E9.5 demo dataset. Top: the raw counts. Bottom: what the panel draws. The dotted lines mark the range chosen by **Auto**.

### How the panel draws the histogram

A giant column at level 0 would crush everything else. The panel therefore applies three rules:

- **linear** scale, normalised by the tallest column;
- the **lowest 4% of columns** (the first 3 out of 64) are **capped at 1.35 times** the tallest column that follows them;
- "comb gaps" (a column below 20% of both its neighbours) are filled with the average of its neighbours, to hide a WebP decoding artefact.

ℹ️ GOOD TO KNOW

The **Ignore low** checkbox changes only the **drawing**: it multiplies the displayed columns by $(i / 63)^2$ to flatten the background bump. It **does not modify the image**.

## 11.6 The quick-setting buttons

The buttons under the histogram set Min, Max and the midpoint in one go. They are called **Auto**, **Soft**, **Contrast** and **Reset** in the panel.

| BUTTON | MIN | MAX | RESULTING GAMMA |
|---|---|---|---|
| **Auto** | 0.5th percentile | 99.5th percentile | 1 (middle of the window) |
| **Soft** | 5 | 240 | 0.83 |
| **Contrast** | 20 | 209 | 1.22 |
| **Reset** | 0 | 255 | 1 |

🔧 FOR THE CURIOUS

**Auto** walks through the histogram (64 columns): Min = last column whose cumulative count stays ≤ 0.5% of the voxels, Max = end of the last column whose cumulative count stays ≤ 99.5%. Without a histogram: Min 0.01 and Max 0.98 (out of 1). The presets are expressed on the 0–1 scale: Soft = 0.02 / 0.94 with midpoint at 0.42; Contrast = 0.08 / 0.82 with midpoint at 0.5; Reset = 0 / 1 / 0.5.

🔢 WORKED EXAMPLE

**Auto** on the E9.5 dataset gives Max = 80 for DAPI, 36 for Pecam1 and 92 for Sox2, and Min = 0: since 86 to 97% of the voxels are background, the lowest 0.5% falls entirely within the first column. **Reset** puts back 0 / 255.

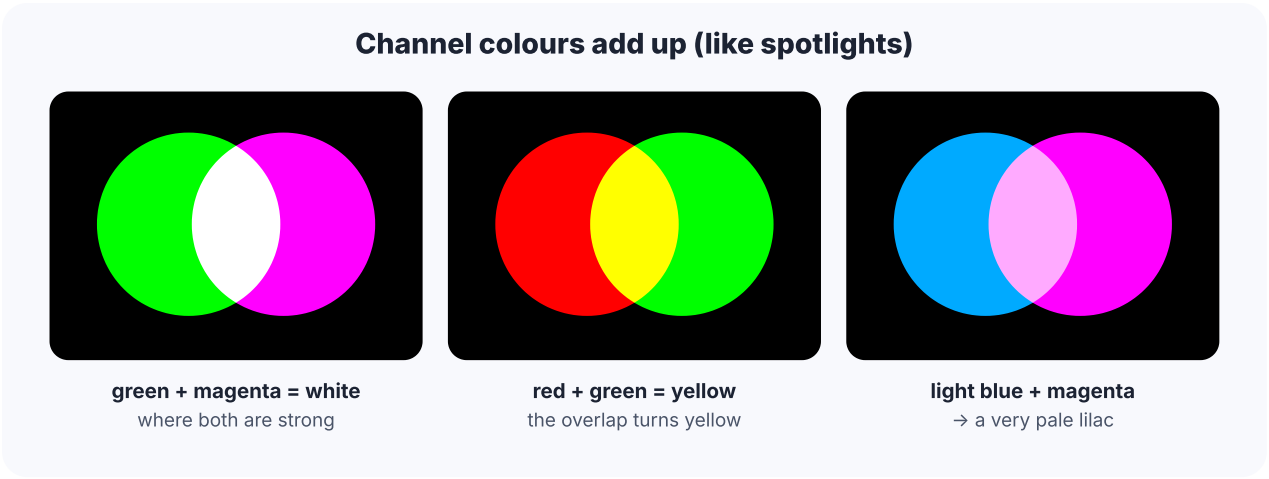
⚠️ WARNING

Auto reflects the **distribution of all voxels**, background included. On a rare signal, it may place Max very low: the image then saturates, deliberately. Keep an eye on the histogram and adjust the handles by hand if needed.

## 11.7 Colours, opacity, exposure, solo

### Colour: an addition of lights

A channel's colour is a simple **RGB multiplier**. Channels are **added together**, like spotlights: where two channels are strong, the colours mix, and anything above 1 is clipped on screen.




▲ Additive mixing: green + magenta gives white.

- The picker offers **27 colours**: three rows (vivid, light, dark) of nine hues.
- By default (when the metadata gives no colour): green, light blue, magenta, red.
- The header's menu of **colour-vision deficiency simulations** applies a filter to the whole screen: it is for checking that a figure stays legible, and does not change the data.

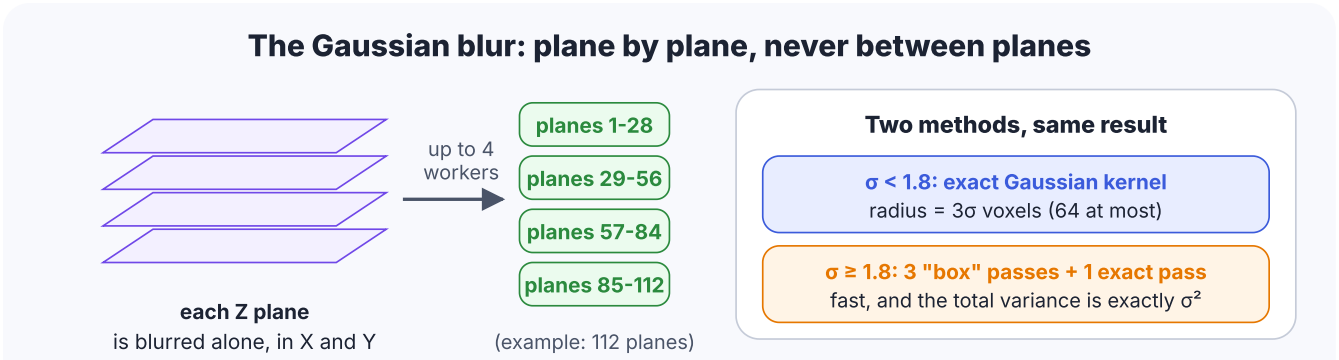
> 👉 **TIP**
>
> Avoid red and green side by side for figures meant for a wide audience: choose green and magenta instead, which remain distinct for most deficiencies.

### Opacity, exposure, solo

- **Opacity**: the droplet button cycles through **70%, 42%, 100%**. It is an intensity multiplier (0.7; 0.42; 1), not a physical absorption.
- **Exposure**: the **Visibility (exposure)** slider (from 20 to 500%, i.e. ×0.2 to ×5) multiplies the final colour; in the natural mode, it multiplies the emission.
- **Solo**: **Solo channel** shows only that channel; a second click (**Show all channels**) restores the previous state.
- A dataset with **more than 4 channels** shows only **4** (a limit of the graphics texture).

## 11.8 The Gaussian blur

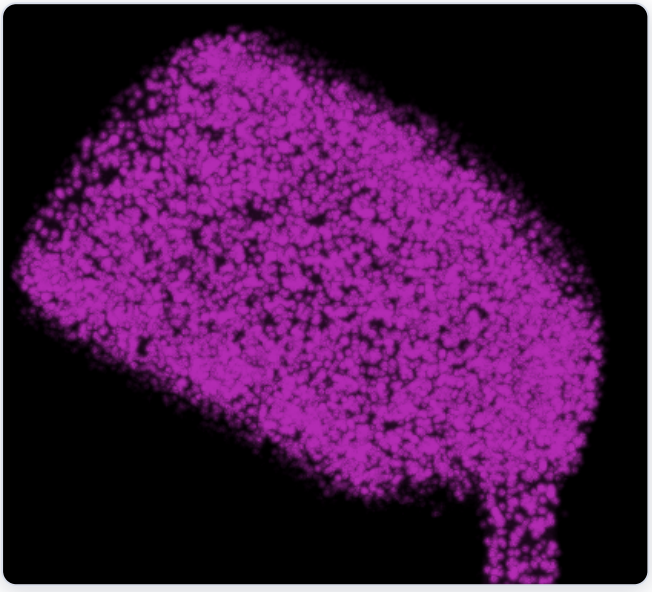
The **Gaussian blur σ** slider (from 0 to 5, in steps of 0.1) smooths the noise of **one channel**. The effect is applied **when you release** the slider; setting σ back to 0 restores the original channel. The setting changes only the display.



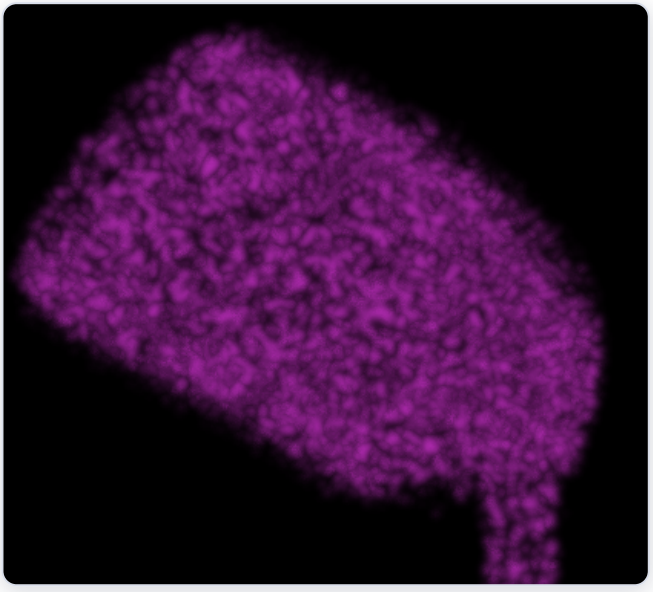

▲ Plane by plane, with several workers; two calculation methods depending on σ.

📌 **REMEMBER**

- **2D in the plane**: each Z plane is blurred separately, never from one plane to another.
- σ is in **voxels of the displayed level**.
- Computed by a pool of **up to 4 workers**, outside the page; a bubble indicates that the calculation is in progress.



▲ Sox2 channel alone, σ = 0.



▲ Same area, σ = 2.0.

⚠️ **WARNING**

The blur is only available if the displayed level fits in **a single dense texture**. For a large level stored in an atlas, the slider is greyed out with the message: “Not available at this quality (sparse brick atlas): choose a lower quality to use the blur.” Using the blur also disables the **Zoom detail** mode (chapter 10).

## 11.9 A safeguard: what the grey levels mean

⚠️ **WARNING**

**The grey levels displayed are relative, not calibrated.** The pipeline brought the original intensities (12 or 16 bits) down to 0–255, between an estimated background (99th percentile of the volume’s corners) and a maximum signal (99.9th percentile); the viewer adds its floor on top. A value of 120 therefore corresponds to **no number of photons**.

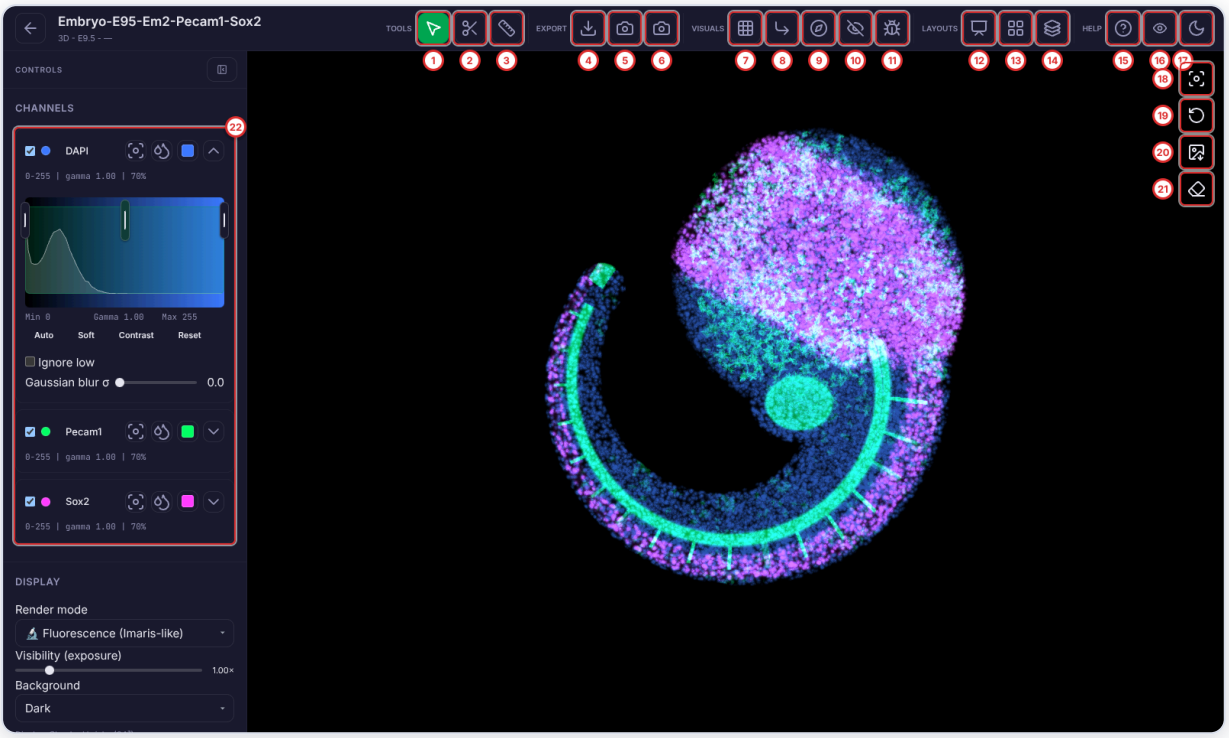
- Comparing **two channels** of the same dataset is delicate: each is stretched over its own scale.
- Comparing **two datasets** by their grey levels is not valid, even with identical settings.
- In a **time series**, the same window is shared by all the timepoints of a channel: intensity variations over time remain comparable with one another.
- For a **quantitative measurement**, go back to the original data (the Imaris file).

# 12. The tools, one by one

- This chapter is the **user manual** for every button of the viewer, the Compare page, the timeline and the 2D page.
- Each tool follows the same plan: **what it is for**, **how to use it**, **what it really does** (in plain words) and **its limits**.
- Every screenshot shows the **demo dataset** (synthetic embryos generated for this documentation, not real laboratory data).

## 12.1 The toolbar at a glance

Hover over a button and a tooltip gives its name. The bar is arranged in **five groups** (Tools, Export, Visuals, Layouts, Help). When the window is narrow or the title very long, the bar folds behind the ☰ button.




▲ The viewer toolbar, numbered (demo dataset).

| N | WHAT IT IS |
|---|---|
| 1 | **Navigate** (V or Esc) |
| 2 | **Slice through volume** (C) |
| 3 | **Measure distance** (M) |

| N | WHAT IT IS |
|---|---|
| 12 | **Presentation mode** |
| 13 | **Decompose by channel** |
| 14 | **Z-Stack Browser** |

| N | WHAT IT IS |
|---|---|
| 4 | **Download Center** |
| 5 | “Sandboxed” capture (demonstration plugin) |
| 6 | **Screenshot** |
| 7 | **Toggle Grid (None / Normal / Fine)** |
| 8 | **Toggle Axes** |
| 9 | **Toggle Orientation Axes** |
| 10 | **Hide / Show 3D Volume (keep projections)** |
| 11 | **Chunk debug (brick boundaries)** |

| N | WHAT IT IS |
|---|---|
| 15 | **Help and methods**: a link to the About page (presentation and methods) |
| 16 | **Colorblind filters** |
| 17 | **Toggle theme** (light / dark) |
| 18 | **Center sample** |
| 19 | **Reset View** |
| 20 | **Export the 3D view as PNG** |
| 21 | **Reset the workspace (view, tools, measurements)** |
| 22 | The **Channels** panel (see chapter 11) |

ℹ️ GOOD TO KNOW

**Your toolbar may look different.** The tools are *plugins* that the administrator installs from the catalogue (chapter 15). Chunk debug, the “sandboxed” capture and the 2D tools are optional. The cell-tracking tools appear only on a tracked time series.

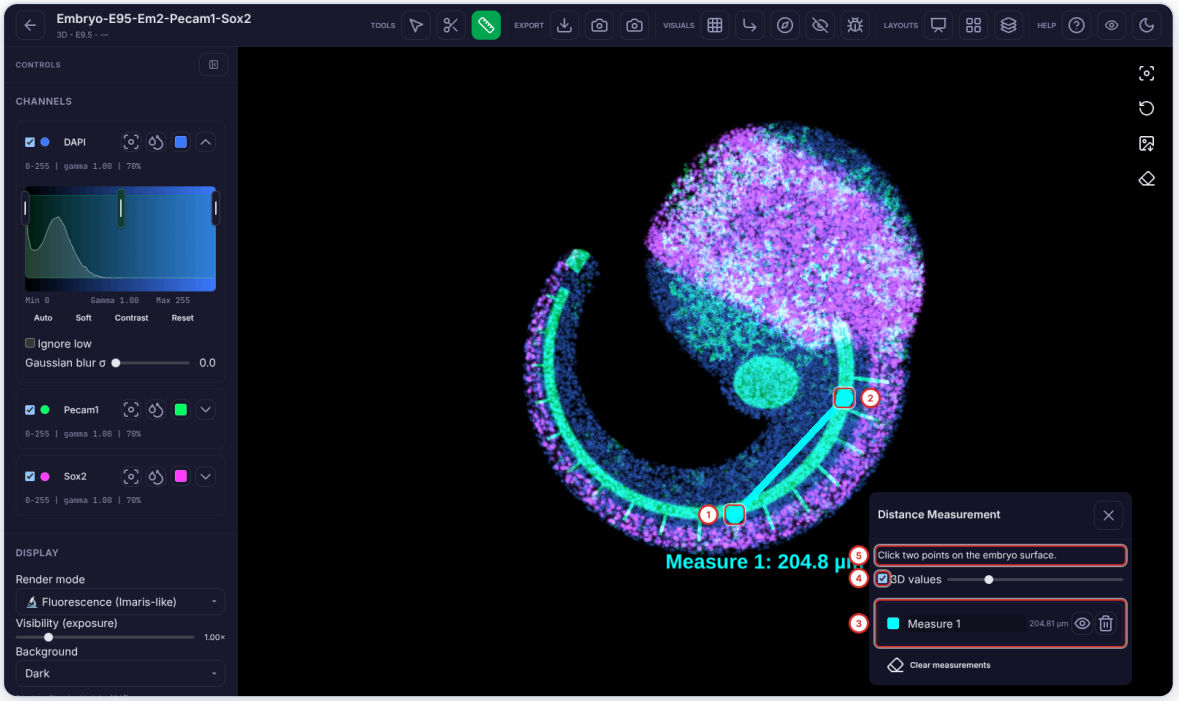
## I want to… → I use…

| I WANT TO… | I USE… | SHORTCUT |
|---|---|---|
| Measure a distance in µm | **Measure distance** | M |
| See a slice, even an oblique one | **Slice through volume** | C |
| Page through the planes one by one | **Z-Stack Browser** | — |
| Know where the front of the embryo is | **Toggle Orientation Axes** | — |
| An annotated figure for a paper | The Studio (§ 12.7) | — |
| A very large or transparent 3D image | **Export the 3D view as PNG** | — |
| A quick screen capture | **Screenshot** | — |
| One panel per channel | **Decompose by channel** | — |
| Get the original file | **Download Center** | — |
| Compare two embryos | The Compare page (§ 12.8) | — |
| Follow cells over time | The five tracking tools (§ 12.9) | I, D |
| Share what I am seeing | Copy the page address (§ 12.11) | — |

## 12.2 Measuring a distance

**⏱ IN 30 SECONDS**

- **What it is for**: finding, in micrometres, the distance between two points of the volume.
- The point is not dropped “at random”: it lands on **the first bright structure** under your cursor.
- A click in empty space is **refused**.




▲ Two points (1, 2), the segment and the list of measurements. Demo dataset, 204.8 µm.

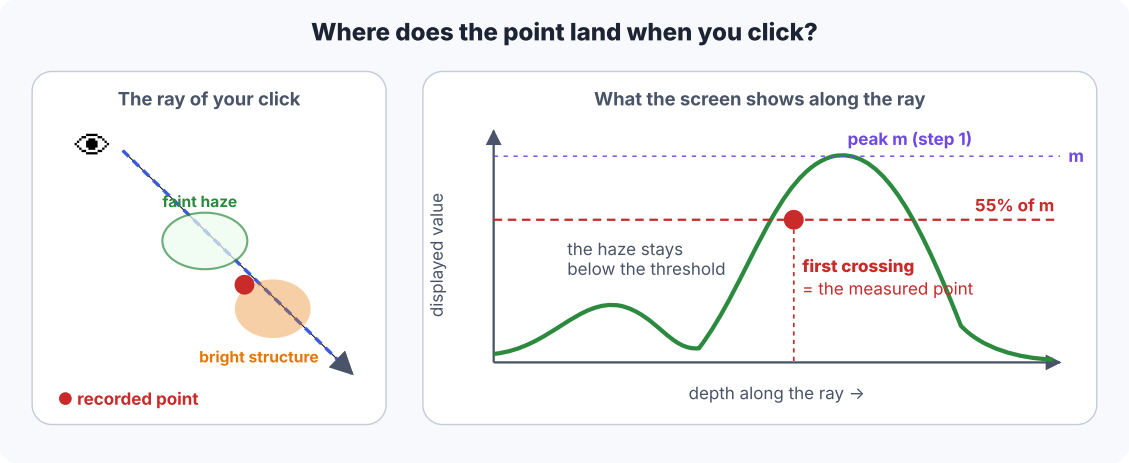
| N | WHAT IT IS |
|---|---|
| 1, 2 | Points A and B, placed by your two clicks |
| 3 | The list: colour, editable name, value, eye (hide), bin |
| 4 | **3D values** checkbox: shows or hides the label in the view, with its size slider |
| 5 | The instruction of the **Distance Measurement** panel |

1. Press M (or click the ruler). The **Distance Measurement** panel opens.
2. Click point A on a **visible** part of the specimen.
3. Click point B: the distance and the Z difference are displayed, and a coloured segment appears.
4. A third click starts a new measurement. Drag to rotate the embryo without placing a point.

### How the point is found

The viewer keeps **no copy** of the volume in main memory: it asks the graphics card. For your click it redraws a mini-render of **a single pixel** with exactly the same settings as the screen (window, gamma,

channels switched on, cropping).




▲ The measured point is the first structure that reaches 55% of the displayed maximum.

🔧 **FOR THE CURIOUS**

Two passes along the ray. Pass 1: the displayed maximum `m` . Pass 2: the first point where the displayed value reaches `max(0.02, 0.55 × m)` , interpolated between the two samples on either side of the threshold. The 0.02 floor stops the viewer from "measuring" noise when everything is almost black. Consequence: **switching a channel off or changing its window moves the point** (you measure what you see).

⚠️ **WARNING**

A click with no structure beneath it (black background) is **refused**: `No structure under the cursor: click on a visible part of the specimen.` Without physical calibration in the metadata, the measurement is refused too. On a **wall of the grid** (§ 12.5), the point is placed on the wall, not on the specimen.

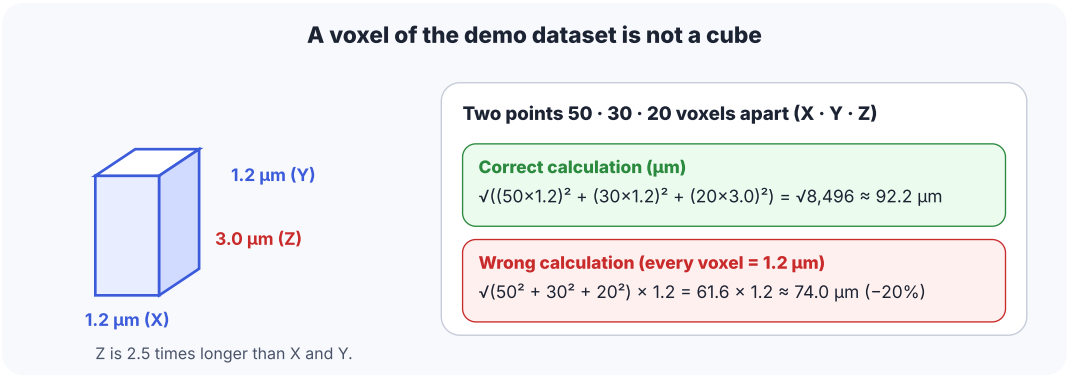
## The calculation, with real numbers

The normalised coordinates (0 to 1) are converted to micrometres with the physical size of the volume (number of voxels × voxel size). Then, on the **physical** coordinates:

$$d = \sqrt{(\Delta x)^2 + (\Delta y)^2 + (\Delta z)^2}$$

🔢 **WORKED EXAMPLE**

The points of the screenshot: A = (498.5, 163.4, 211.1) µm and B = (639.3, 311.9, 203.2) µm. The differences are Δx = 140.8, Δy = 148.5, Δz = 8.0 µm. So d = √(19,838 + 22,043 + 63) = √41,944 ≈ **204.8 µm**, the value displayed.



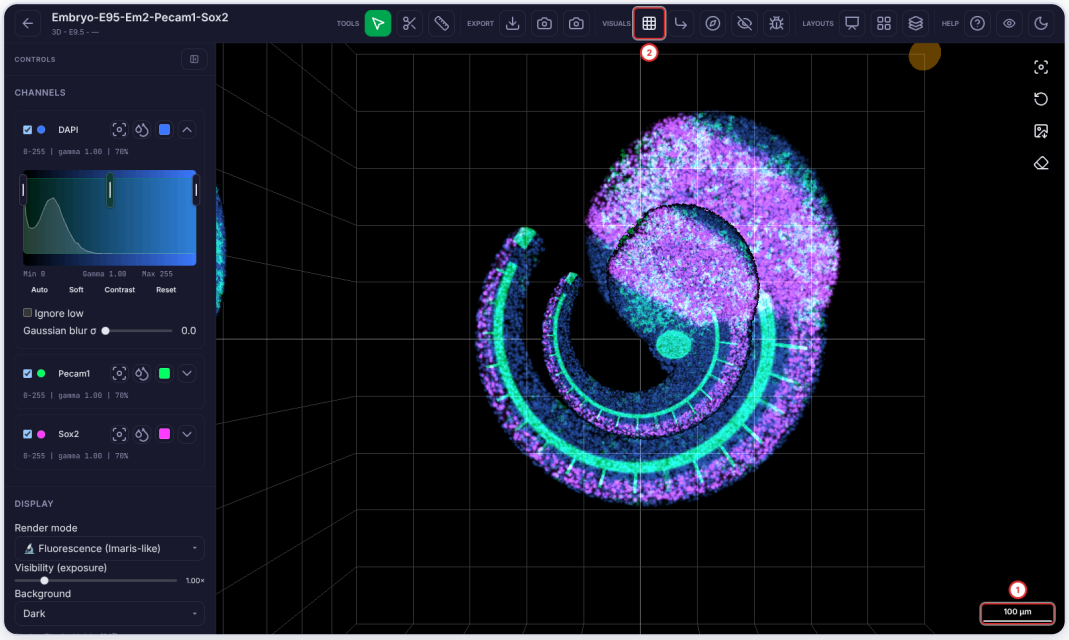

▲ Why you cannot “count voxels”: Z is 2.5 times longer.

> ❓ **WHY THIS CHOICE?**
>
> The demo dataset has voxels of 1.2 × 1.2 × 3.0 µm. Counting voxels as if they were cubic would underestimate a diagonal distance here by 20%. The viewer always does the calculation in µm, never in voxels.

**Where are the measurements kept?** Only **in the memory of the page**: closing the tab erases them. To keep them, copy the page address (they travel in the link, § 12.11) or download **Measurements CSV** from the Download Center.

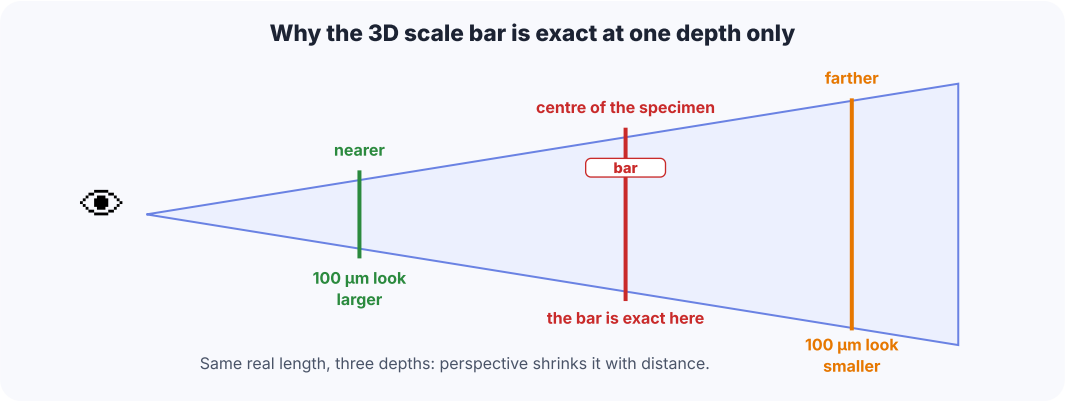
## The scale bar of the 3D view

Switch the grid on (button 7): a scale bar appears at the bottom right, with a “round” length (the **1-2-5** rule: 1, 2, 5, 10, 20, 50, 100 µm…). It aims at about 20% of the width of the view, between 60 and 200 pixels.




▲ The grid and the scale bar (1): 100 µm. The grid button is number 2.

Because of **perspective**, a distant object looks smaller: the bar is exact only at the depth of the **centre of the specimen**.




▲ Same length, three depths: the bar holds for the central plane.

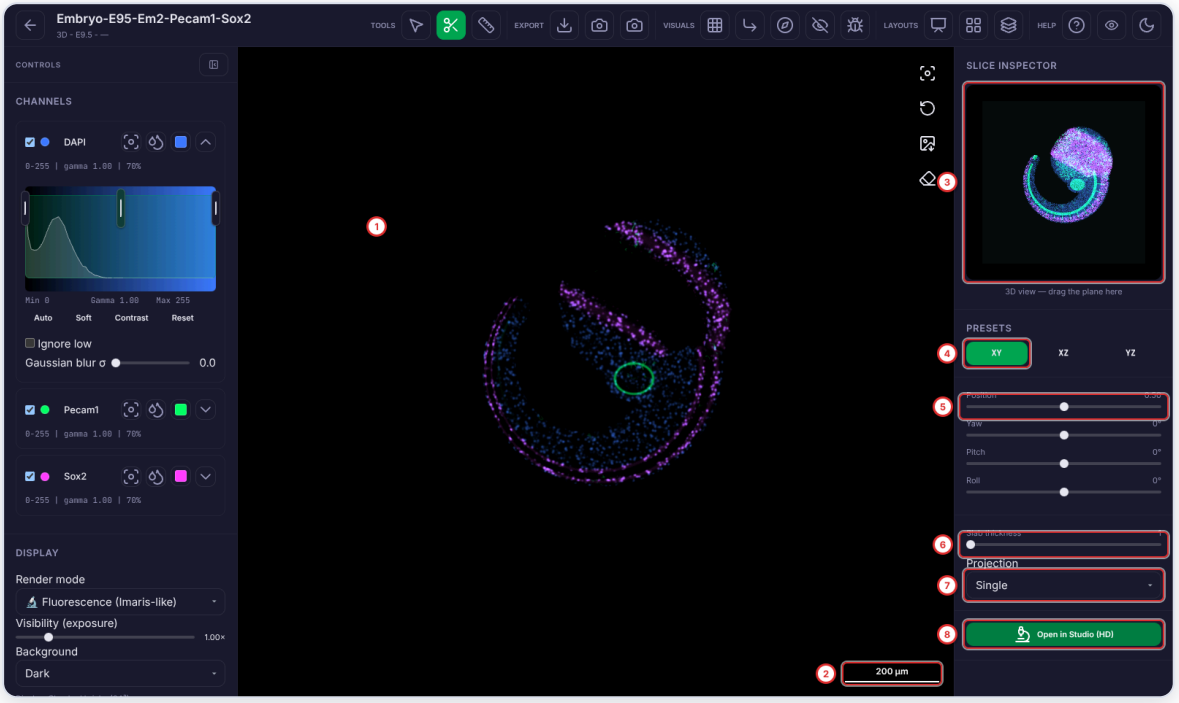
🔧 **FOR THE CURIOUS**

µm per pixel = 2·tan(field of view/2) · depth · (physical size X / cube scale) / height of the view in pixels. The bar is hidden without calibration or when the grid is off.

## 12.3 The oblique slice

⏱ **IN 30 SECONDS**

- **What it is for**: seeing a flat slice of the volume, straight or tilted, instead of the overall view.
- While the tool is open, **the two views swap places**: the slice fills the screen and the 3D view becomes a small window.
- It cannot be used at the same time as the Z-stack browser.




▲ XY slice through the middle of the volume (demo dataset).

| N | WHAT IT IS |
|---|---|
| 1 | The slice, computed by the graphics card, in the colours of your channels |
| 2 | Exact scale bar of the slice (200 µm here) |
| 3 | The volume in miniature: the plane is moved there |
| 4 | **XY**, **XZ**, **YZ** presets |

| N | WHAT IT IS |
|---|---|
| 5 | Plane **Position**, then **Yaw**, **Pitch**, **Roll** |
| 6 | **Slab thickness**: number of samples in the slab |
| 7 | **Projection**: **Single**, **MIP** or **Average** |
| 8 | **Open in Studio (HD)**: sends the slice to the Studio (§ 12.7) |

1. Press C: a translucent plane appears and the slice fills the screen.
2. Choose a preset, or move the plane in the miniature (mouse wheel on the plane: 2% of the volume per notch).
3. The **Yaw**, **Pitch** and **Roll** sliders make the plane **oblique**.
4. Thicken the slab and choose how to flatten it: a single slice, the brightest value (MIP) or the average.

🔧 **FOR THE CURIOUS**

The slice reads the **same atlas** as the 3D view, so the windows, gamma and colours are the same. The plane is the rotation Ry(−yaw)·Rx(−pitch)·Rz(roll) in the frame with physical proportions; because voxels are longer in Z, the plane's normal is corrected ($n_{tex} \propto S^{-1} \cdot n$). While you move the plane the image is 1,024 px; once it is still (160 ms) it is refined up to 2,048 px. A thickness counts in samples (at most 1,024), not in µm.

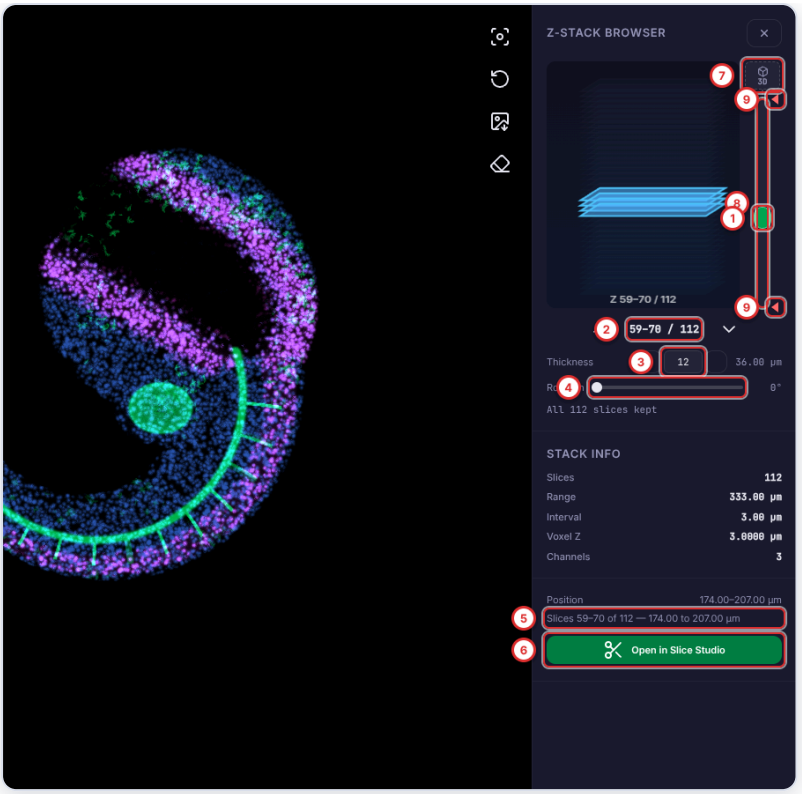
⚠️ **WARNING**

The XY/XZ/YZ presets follow the axes **of the file**, not those of the embryo, unless the dataset has been calibrated (§ 12.5).

## 12.4 The Z-stack browser

⏱ **IN 30 SECONDS**

- **What it is for**: paging through the planes of the stack as in a microscopy program.
- On opening, the stack **lays itself flat** in 1.5 s, upper face towards you.
- The cursor can show one slice, several slices (a "thickness") or the whole stack in 3D.




▲ Z-stack in slice mode: 12 slices (59 to 70), i.e. 36 µm.

| N | WHAT IT IS |
|---|---|
| 1 | The **cursor**: its height = number of slices shown |
| 2 | **59–70 / 112**: slices displayed / total |
| 3 | **Thickness**: 12 slices = 36.00 µm |
| 4 | **Rotation** 0–360° |
| 5 | Position: 174 to 207 µm deep |
| 6 | **Open in Slice Studio**: sends the slab to the Studio |
| 7 | The **3D notch**: cursor here = the whole stack in 3D |
| 8 | The **track**: one position per slice |
| 9 | **Trim triangles** (above / below) |

1. Click **Z-Stack Browser**: the stack lays itself flat, in 3D mode (notch 7).
2. Drag the cursor **down** into the track (8): the view locks from above onto the chosen slice.
3. Drag an **edge** of the cursor, or the **Thickness** button, to show several slices.
4. The triangles (9) hide slices at the top or bottom (**All 112 slices kept** becomes **Kept …**). Move the cursor back up into notch 7 to return to 3D.

🔧 **FOR THE CURIOUS**

A single operation crops the volume in Z: `setClipRange_z` with the interval `[start / z ; (end+1) / z]`. The render calculation is confined to this box and corrects the emission (“thin-slab gain”) so that a single slice is not black. Depth = index × voxel size in Z (3 µm here). From the keyboard, with the cursor selected: ↑ ↓ one slice, Page Up/Page Down 10% of the stack, Home = 3D notch, End = last slice, + / − thickness.

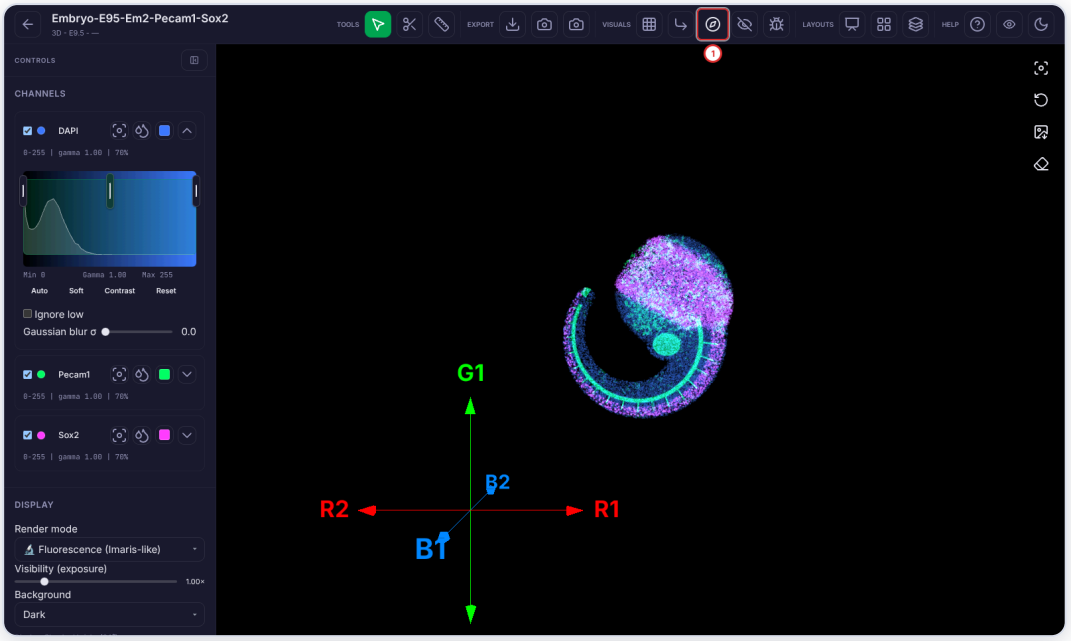
**An “upside-down” specimen.** Some Imaris files show the specimen seen from below. The administrator records this in the metadata (`upsideDown`): the “upper face” is then the −Z face, and the Z-stack, like the Studio figures, presents it the right way up. The orientation calibration does not depend on it.

# 12.5 Orientation axes, the grid and the displays

⏱ IN 30 SECONDS

- **Orientation axes**: a compass for the embryo (R1/R2, G1/G2, B1/B2).
- **Grid**: three graduated walls with projections; **Axes**: a movable XYZ marker.
- **Volume**: hides the 3D volume while keeping the projections.

## The orientation axes




▲ The compass: red R1/R2 (±X), green G1/G2 (±Y), blue B1/B2 (±Z).

Each arm has a **short name** (R1, R2, G1, G2, B1, B2). The administrator can rename them (“Anterior”, “Dorsal”…) or hide some. Drag the small central sphere to move the compass; its position is kept in the workspace.

💡 A PICTURE TO UNDERSTAND

**An orientation calibration is a hand gesture.** The microscope records the embryo in the orientation it was lying in, not “head up”. The calibration says: “to put the embryo the right way up, turn the file in **one** movement, around **this** axis, by **this many** degrees”. The computer stores that gesture as four numbers (a *quaternion*). You have nothing to set: the administrator does it once, in the administration panel.

**The default view** is a pose of the embryo (for example “ventral face, anterior up”) saved with the dataset: it is applied **before** the first brick is loaded, and the **Reset View** button returns to it. The presets offered are ventral, dorsal, left, right, anterior, posterior.

🔧 FOR THE CURIOUS

The file has its rotation `Q_base` (file axes → anatomical axes). The compass is drawn with `Q_anat = Q_cube · Q_base⁻¹`, and a default view is stored as `Q_anat` and applied as `Q_cube = Q_anat · Q_base`. The calibration can be edited only from the administration (preview ↔ viewer by messages); a visitor can neither change it nor save a default view.

### The grid, the axes, the volume

- **Grid** (7): each click cycles through *None* → *Normal (10 divisions)* → *Fine (40)* → *None*. The three walls show a **projection** of the volume with the same render mode. An orange handle resizes a wall (double-click: back to normal).
- **Axes** (8): a coloured XYZ marker, not to be confused with the compass. Drag its sphere to move it.
- **Volume** (10): hides the ray-marched volume; the grid, the axes, the measurements and the tracking remain.

## 12.6 Capture, export, presentation, download, channels, chunks

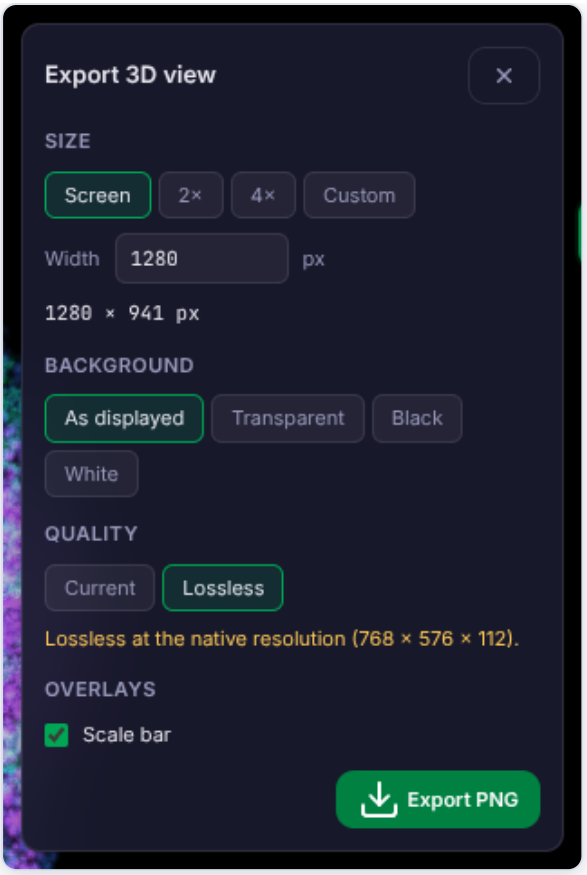
⏱ IN 30 SECONDS

- Three ways to get an image: **Screenshot** (quick), **Export the 3D view as PNG** (large size), the Studio (annotated figure).
- The **Download Center** does not give captures: it gives the **files** of the dataset.

### Screenshot and 3D view export

The **screenshot** (button 6) saves what is on screen in one click, as a PNG (`<dataset>_screenshot.png`), at the screen's resolution.

The **3D view export** (button 20) redoes the render **at any size**.




▲ The 3D view export window (button 20 of the toolbar), with the quality choice.

- **Size**: **Screen**, 2×, 4×, **Custom** (width from 16 to 16,384 px).
- **Background**: as displayed, **Transparent**, black, white.
- **Scale bar**: disabled without calibration (it would count voxels, not µm).
- **Quality**: **Current** (the displayed cell) or **Lossless**. *Lossless* loads the lossless column of the native level, or of the finest level the graphics card can hold (the dialog says which), renders the picture, then puts the previous quality back.
- File: `<dataset>_3d_<W>x<H>.png`.

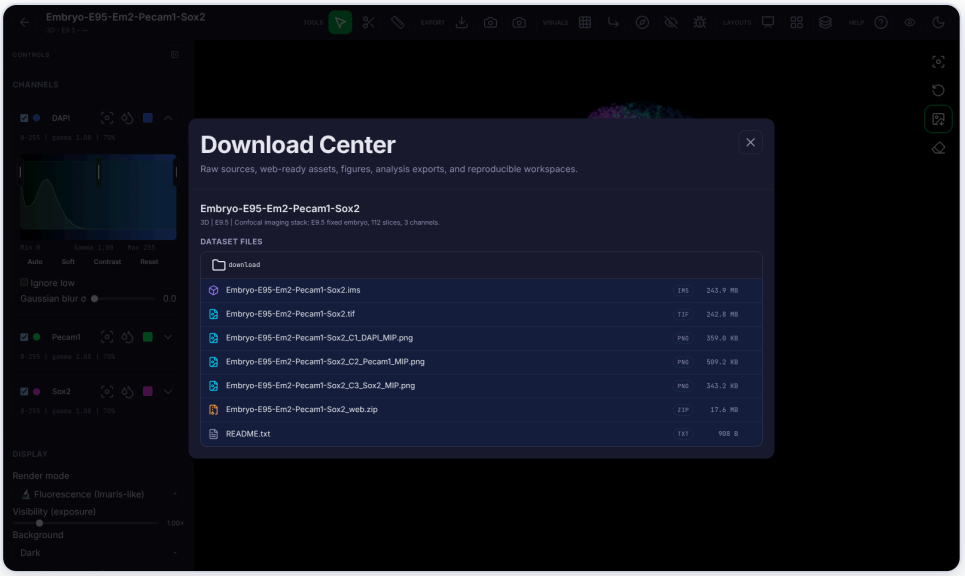
🔧 **FOR THE CURIOUS**

A graphics card cannot draw a giant image in one go. The viewer waits for loading to finish, then renders the view **in tiles** of at most 2,048 px (each is a window of the field of view, one tile per task so the page does not freeze) with the number of steps of a render at rest. The transparent background writes an opacity per pixel `a = max(opacity, max(channels))` ; over black, it gives back the screen. If the display changes during the render, it starts again (3 attempts). Ceiling: 16,384 px per side, 268 megapixels.

## Presentation mode

Button 12 hides the interface so that the volume fills the screen: useful for projecting or filming. Press the same button to go back.

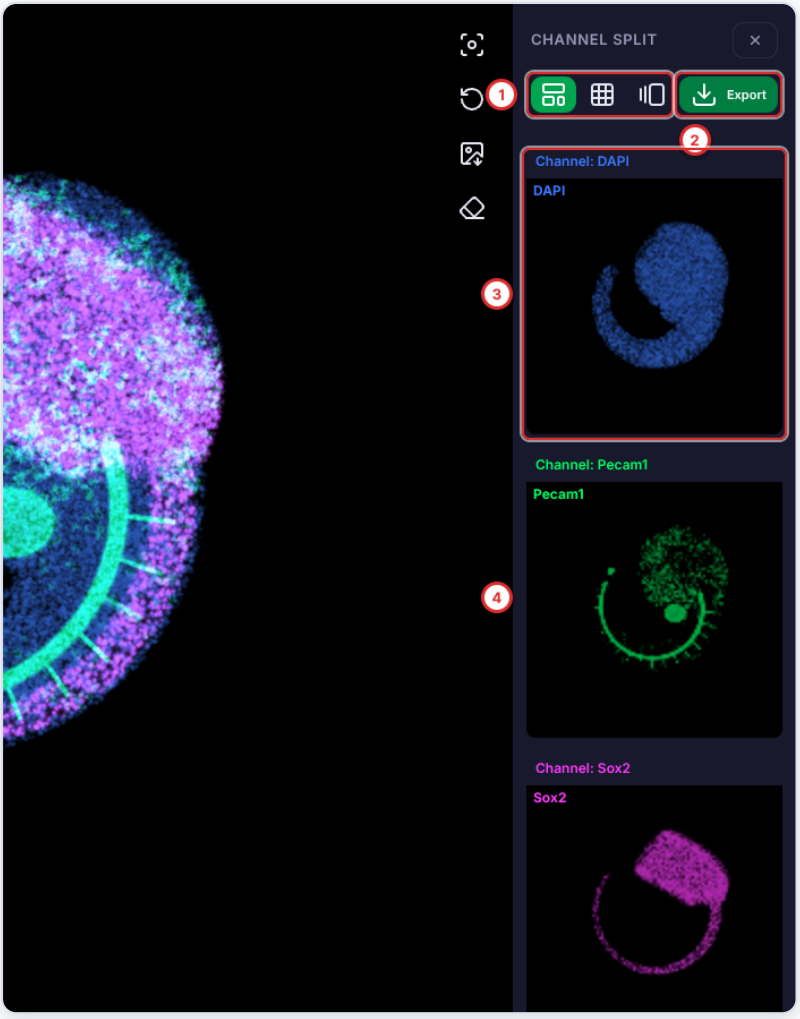
## Download Center




▲ The files of a dataset's `download/` folder (demo dataset).

In the viewer it shows a **list of files**: the original Imaris file, the calibrated TIFF, the per-channel projections (PNG), the web archive and a README file. They exist only if the pipeline was run with downloads (chapter 8). These files are always served as an **attachment**. If there are measurements, a **Measurements CSV** button appears (columns: label, type, distance, unit, point A, point B...).

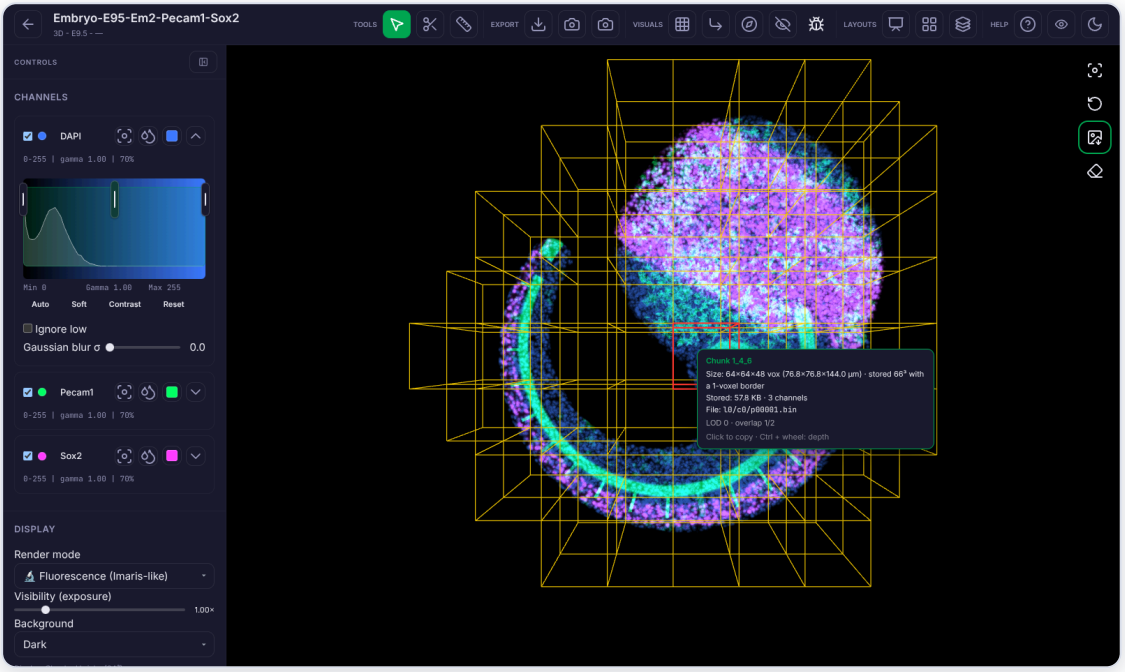
## Decompose by channel




Each channel that is switched on gets its **thumbnail** (the same volume, with that channel alone). Clicking a thumbnail lets you adjust that channel in the left-hand panel ( **Done** to leave, **Reset to original** ). The three buttons (1) choose the layout; **Export** (2) writes a PNG `decomposition_<layout>_<date>.png`. Each thumbnail is rendered at the size of the volume (512 to 4,096 px) and the viewer refuses an image too large for the graphics card (“Too many views to export as one image”).

▲ One thumbnail per channel: DAPI, Pecam1, Sox2.

## Chunk debug



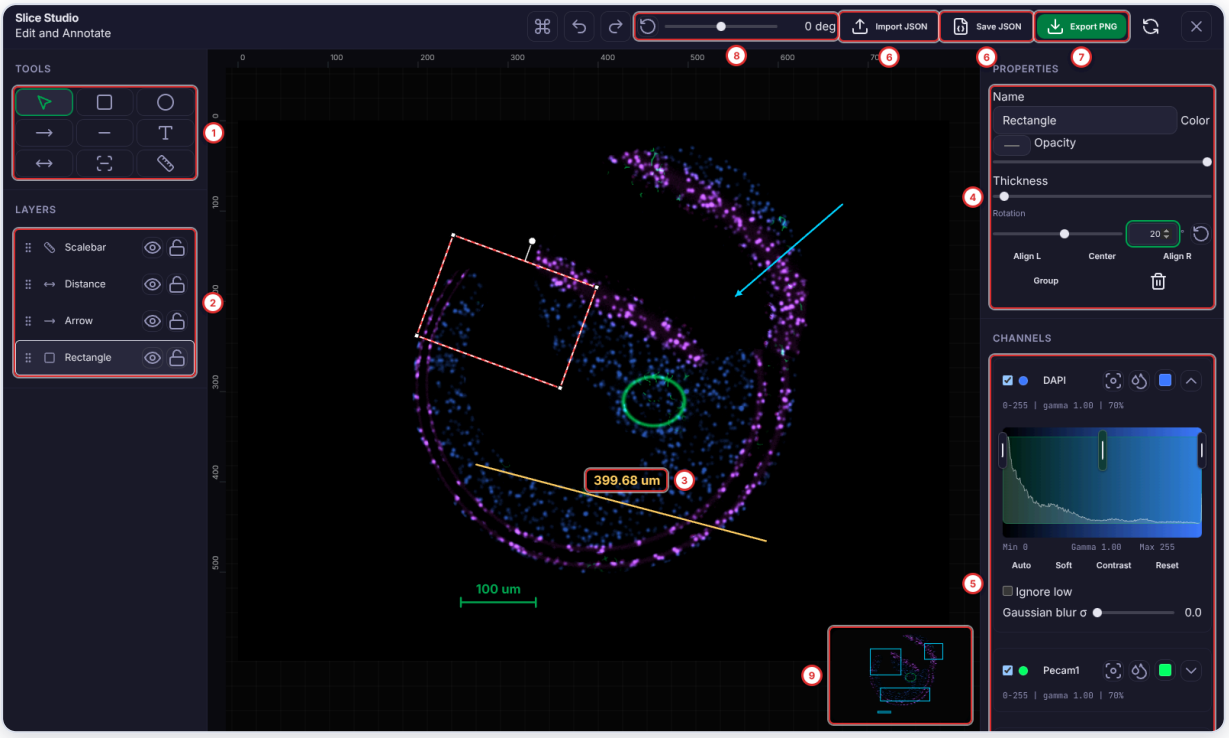

▲ The $64^3$-voxel bricks drawn in yellow; hovering over one of them.

The most instructive tool of chapter 7: it **draws the edges of every non-empty brick** of the displayed level. Hover over a brick to see its identifier, its size (here 64×64×48 voxels, 76.8 × 76.8 × 144 µm), its stored size ($66^3$ with a one-voxel border), its pack file and its level of detail. Click: copies this information. Ctrl + wheel: steps through the overlapping bricks under the cursor.

## 12.7 The Studio: annotating and exporting a figure

**⏱ IN 30 SECONDS**

- **What it is for**: making a publication figure from a slice (scale bar, arrows, measurements, text).
- It opens **immediately** on a preview, then reloads the slice at **native resolution**.
- Channels are **recoloured** in the Studio without re-rendering the volume.




▲ The Slice Studio with a rectangle rotated by 20°, an arrow, a distance and a scale bar.

| N | WHAT IT IS |
|---|---|
| 1 | Tools: select, rectangle, ellipse, arrow, line, text, distance, angle, scale bar |
| 2 | Layers (reorder, hide, lock) |
| 3 | A distance measurement: 399.68 µm (label computed in µm; the figure writes the unit as “um”) |

| N | WHAT IT IS |
|---|---|
| 6 | Import JSON / Save JSON |
| 7 | Export PNG |
| 8 | Rotation of the **view** (display only) |
| 9 | Mini-map |

| N | WHAT IT IS |
|---|---|
| 4 | **Properties** of the chosen layer: name, colour, opacity, thickness, **Rotation** (−180 to 180°) |
| 5 | **Channels**: colour, min/max, gamma, on/off, with their histograms |

1. From the oblique slice or the Z-stack, click the button that opens the Studio.
2. Annotate: each tool has a letter (table in § 12.11). A scale bar is placed on opening if the volume is calibrated.
3. Recolour the channels if needed (right-hand column).
4. **Export PNG** for the image, **Save JSON** to be able to resume later.

## The layers

- **Distance**: the label is `√((Δx·px_x)² + (Δy·px_y)²)` µm, with the pixel size of the figure. **Angle**: three clicks, computed in µm. **Scale bar**: its length (µm, mm, cm or px) is converted to pixels.
- **Layer rotation**: a rectangle, ellipse or text turns about its centre; a line, arrow or distance has its points rotated (the distance keeps its value in µm). Slider, field, handle above the selection (Shift = 15° steps), or [ / ].
- The history keeps 80 steps (Ctrl+Z).

## The “native” pass

The preview (at most 2,048 px) comes from the graphics card. The native pass fetches the **voxels at maximum resolution** for this exact plane, and replaces the image as they arrive. A bar shows bytes, chunks and time remaining, with **Stop here**.

- An **XY** slice of a format 2 (or later) dataset reads the `planes/` folder: only the voxels of the plane.
- An **oblique** plane or an old volume goes through the bricks (targeted byte reads, chapter 10).
- A brick that is absent because it is empty counts as **zero**, never as “missing”.
- If a figure needs more than 256 MB, a dialogue offers native, reduced resolution or preview.
- On a **format-5** dataset, the native pass reads the **cut copies** (section 7.11): an XZ or YZ cut = one row of the `xz/` or `yz/` pack, three small reads per channel; an oblique cut = only the small 32 × 32 × *bz* bricks the plane crosses. If a copy is missing or fails, the pass falls back to the bricks. The pictures are identical, voxel for voxel.

> ℹ️ **GOOD TO KNOW**
>
> On the demo dataset (768 × 576 px), the native version is as small as the preview: the progress bar has almost nothing to load. It comes into its own on a volume of several gigabytes.

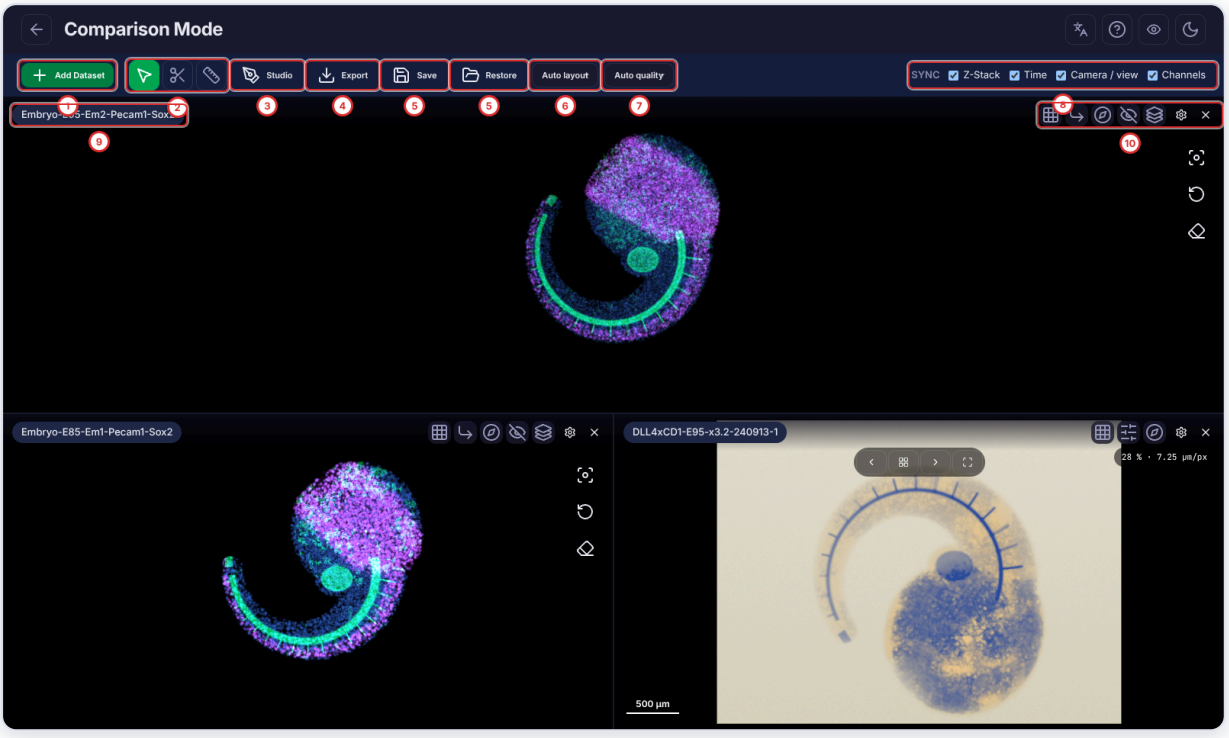
### Exports and limits

- **PNG**: native size, black background, with a **caption line always burned in** at the top left (dataset | plane | size | pixel). Refused beyond 16,384 px per side or $2^{28}$ pixels.
- Before the PNG export, a question: **Native lossless** or **This picture**. *Native lossless* waits for the native pass (or runs it again if you had stopped it); the layers are rescaled to the new picture and **keep their place**. *Decompose by channel* asks the same question.
- **JSON** ( `<dataset>_studio.json` ): layers, guides, channels, plane, calibration. **Never any pixels.** On import, a file larger than **5 MB**, or with more than 2,000 layers or 10,000 points, is refused; a file from another figure applies only its layers.
- Beyond 4 channels, only the first 4 are shown. Without calibration: no scale bar, and measurements are in pixels.

## 12.8 The Compare page

**⏱ IN 30 SECONDS**

- **What it is for**: displaying up to **four** datasets side by side, with the views kept in sync.
- Each panel is the **real page** of the viewer (or of the 2D page): same tools, same shortcuts.
- A single graphics memory is **shared** between the panels (automatic quality).




▲ Three panels: two volumes and a photograph. (1) add, (2) common tools, (8) synchronisations.

| N | WHAT IT IS |
|---|---|
| 1 | **Add Dataset** (search, filter by type) |
| 2 | Common tools: what the open pages offer |
| 3 | **Studio**: one figure with all the panels |

| N | WHAT IT IS |
|---|---|
| 6 | **Auto layout**: Auto, columns, rows, grid (adjustable gutters) |

| N | WHAT IT IS |
|---|---|
| 4 | **Export** : PNG / WebP figure of the grid |
| 5 | **Save** / **Restore** the workspace |

| N | WHAT IT IS |
|---|---|
| 7 | **Auto quality** : auto, 512, 1024, native |
| 8 | SYNC: **Z-Stack** , **Time** , **Camera / view** , **Channels** |
| 9 | The name of the panel's dataset |
| 10 | The toggle buttons **described by the page** of this panel |

## What is synchronised

| OPTION | WHAT TRAVELS BETWEEN PANELS |
|---|---|
| **Z-Stack** | Z-stack slice, plane of the oblique slice |
| **Time** | position in the series, **as a fraction** (frame = fraction × (N − 1)): 100 frames and 10 frames stay aligned |
| **Camera / view** | 3D viewing angle (corrected for each panel's calibration); for a photograph, the physical scale |
| **Channels** | state and exposure, **by channel name** |

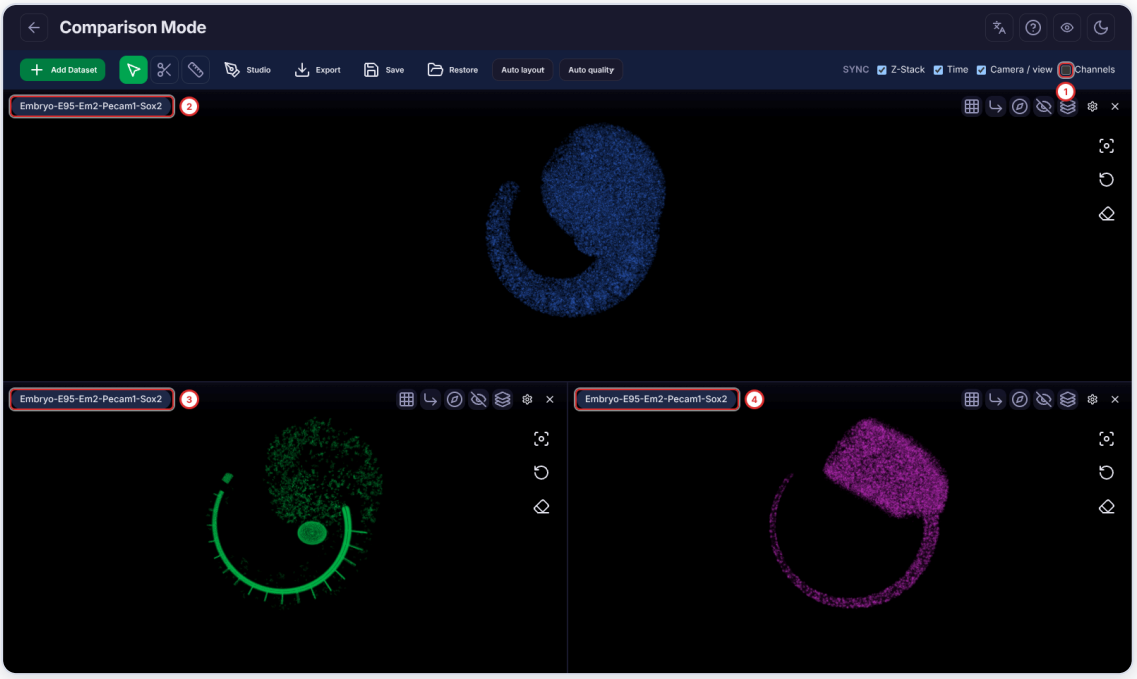
A panel added late is "caught up": it receives the latest camera, the latest time, and so on.

## The shared memory

> 💡 **A PICTURE TO UNDERSTAND**
>
> **A buffet on a fixed budget.** Compare keeps a **fixed** shared budget of 1.5 GiB for all the panels (it is not a measurement of your graphics card). Each volume says what each quality costs. The viewer lowers the hungriest one while the total is too high, then raises the most modest one while it still fits, never going above 1024 in automatic mode. This setting only concerns **volumes**: a 2D photograph has no quality.

Panels start at 512; quality is raised **one panel at a time**; a panel that does not settle within 180 s is abandoned (message in the console) and the next one is handled. Choosing 512/1024/Native by hand forces the level.

### Decompose, figure, workspace



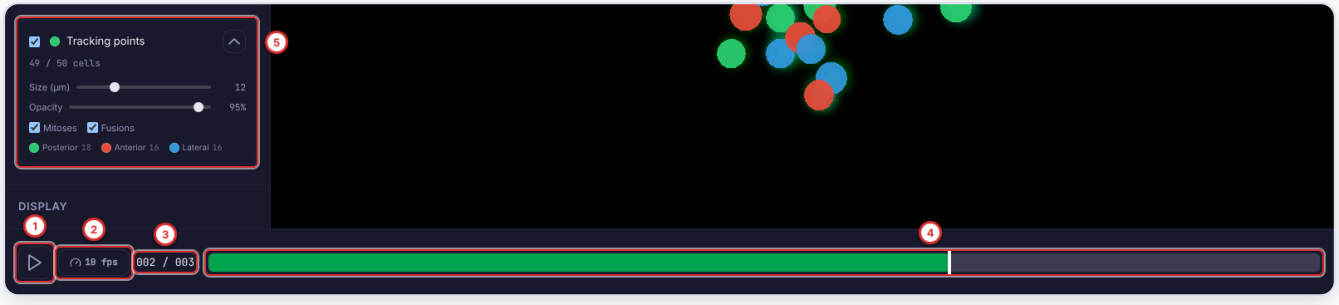

▲ A three-channel volume, **Decompose**: three panels, one channel each. The **Channels** sync (1) was unticked automatically.

- **Decompose** appears only when there is **one** multi-channel volume panel: it clones it into panels (4 at most), one channel each, keeping time and camera in sync.
- **Studio**: one figure with all the panels, in **Visual Size** or **Physical Scale** (a single scale in µm/px for all, never upsampled).
- **Export** composes the visible grid (4,096 px per side at most).
- **Save** keeps the workspace in the browser; **Restore** reloads it. The page address also contains the state (`#state=…`).

## 12.9 The timeline and cell tracking

**⏱ IN 30 SECONDS**

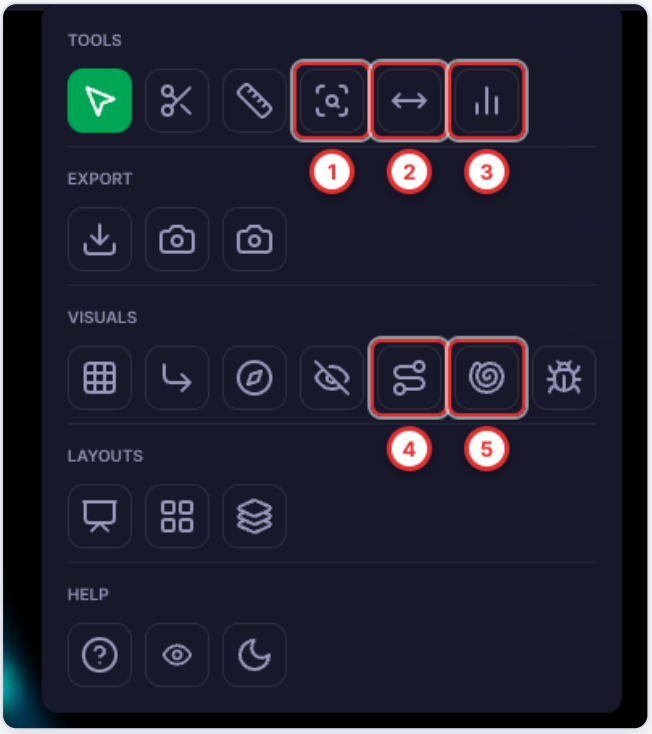
- A **time series** (*Live* type) adds a timeline at the bottom of the screen to play it.
- If it has cell tracking, **five tools** are added: trajectories, surface, inspector, cell distance, charts.
- Example: the 4-frame demo dataset with tracking (50 cells).




▲ The timeline: play (1), speed (2), counter (3), track (4); and the **Tracking points** layer (5).

| N | WHAT IT IS |
|---|---|
| 1 | Play / pause |
| 2 | Speed: each click cycles through 0.5 → 1 → 2 → 5 → 10 → 20 fps |
| 3 | Counter: current frame / last frame (000 / 003 = 4 frames) |
| 4 | Track: drag to go to a frame; the band shows the frames already loaded |
| 5 | Tracking points layer: size, opacity, mitoses, fusions, legend per region |

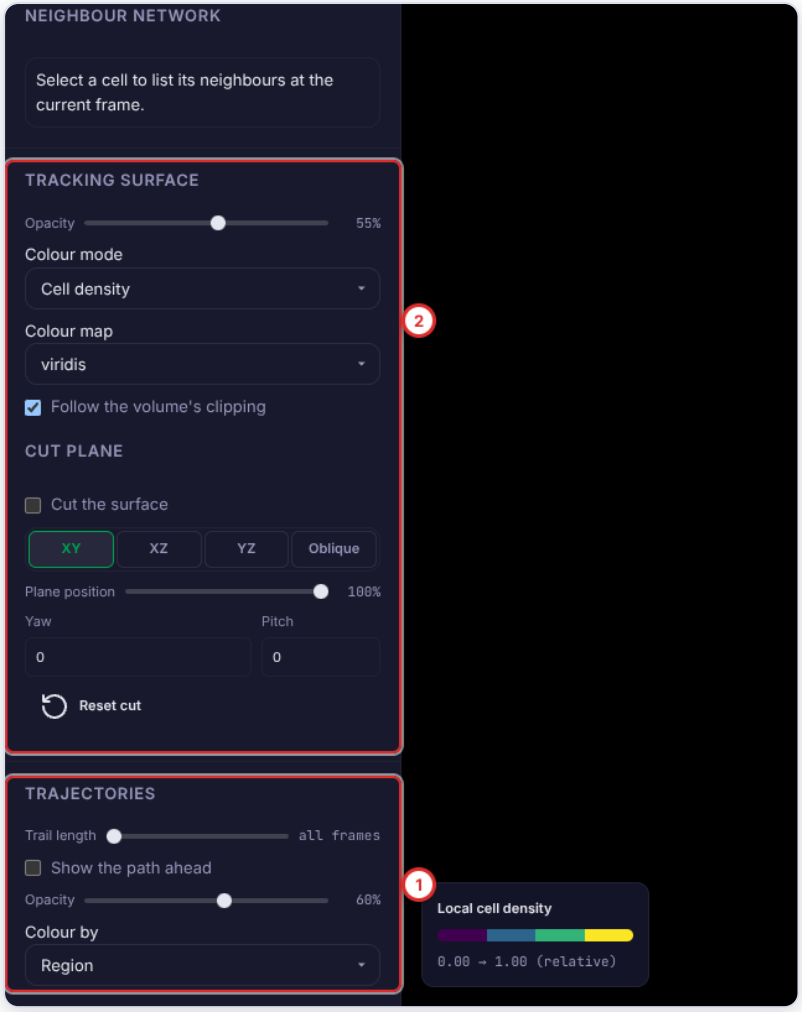
- Playback skips frames rather than piling them up; the clock **waits** for a frame that has not arrived (up to 15 s).
- The following frames are preloaded. **Buffer 4/4 · 512×512** shows what fits in memory.
- There is no clock in minutes on the timeline: the display is a frame number.




▲ On this series the bar is folded: the ☰ menu groups the tracking tools.

| N | WHAT IT IS |
|---|---|
| 1 | **Inspect cell** (I) |
| 2 | **Cell distance** (D) |
| 3 | **Tracking charts** |
| 4 | **Trajectories** |
| 5 | **Tracking surface** |

## Trajectories and surface




▲ **Trajectories** (1) and **Tracking surface** (2) in the left-hand column, last frame.

- **Trajectories**: trail length (0 = all frames), path ahead, opacity, colour by **region** or by **speed** (blue slow → red fast, fixed scale across the whole series).
- **Tracking surface**: the tissue surface ( `model.glb` ) inside the volume, coloured uniformly, by **cell density** or by region, with a palette of your choice and a **cutting plane** (XY, XZ, YZ or oblique).

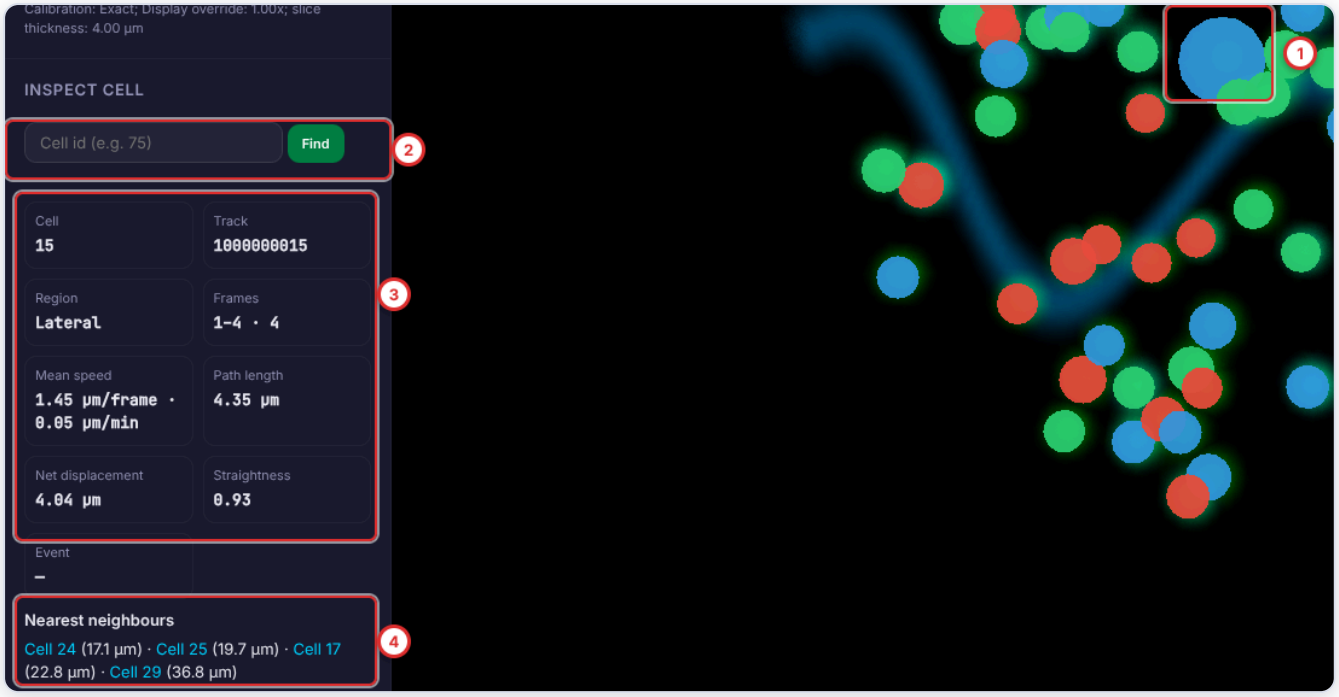
🔧 **FOR THE CURIOUS**

Local density at a vertex: sum of Gaussians $\exp(-|v-c|^2/2\sigma^2)$ over the cells of the current frame, with σ = 0.52 × neighbourhood radius, bounded between 8 and 54 µm, then two smoothing passes over the mesh. Speed of a segment: |Δposition| in µm / Δt.

⚠️ **WARNING**

On the demo dataset, the cells move only a few micrometres between two frames: no line was legible in our captures in software rendering. The sliders and the summary do apply; try it on a real series.

## Inspecting a cell



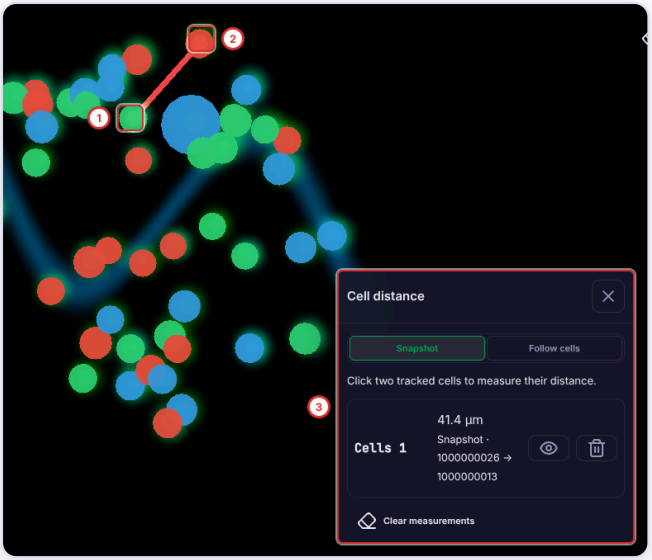

▲ Cell 15 selected (1). Left-hand column: identifier (2), metrics (3), neighbours (4).

1. Press `I`, then click a sphere (or type its number and **Find**).
2. Read the tiles: track, region, frames, mean speed, path length, net displacement, straightness, event.
3. Explore the lineage (mother, daughters) and the **neighbours** within 35, 55, 85 or 120 µm.

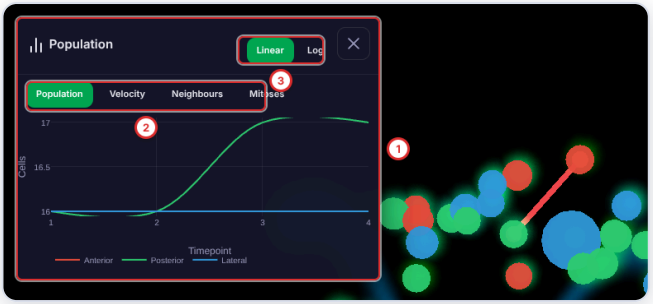
> 🔧 **FOR THE CURIOUS**
>
> Path length = Σ |p(f+1) − p(f)|; net displacement = |p(last) − p(first)|; **straightness = displacement / length** (1 = straight line). Mean speed = length / duration, in µm/frame, and in µm/min if the interval is known. Here 4.35 µm of path for 4.04 µm of displacement: straightness 0.93.

Export buttons: **Track CSV**, **Neighbours CSV**, **Lineage JSON**.

### Cell distance and charts




▲ Two cells (1, 2) and the panel (3): 41.4 µm.



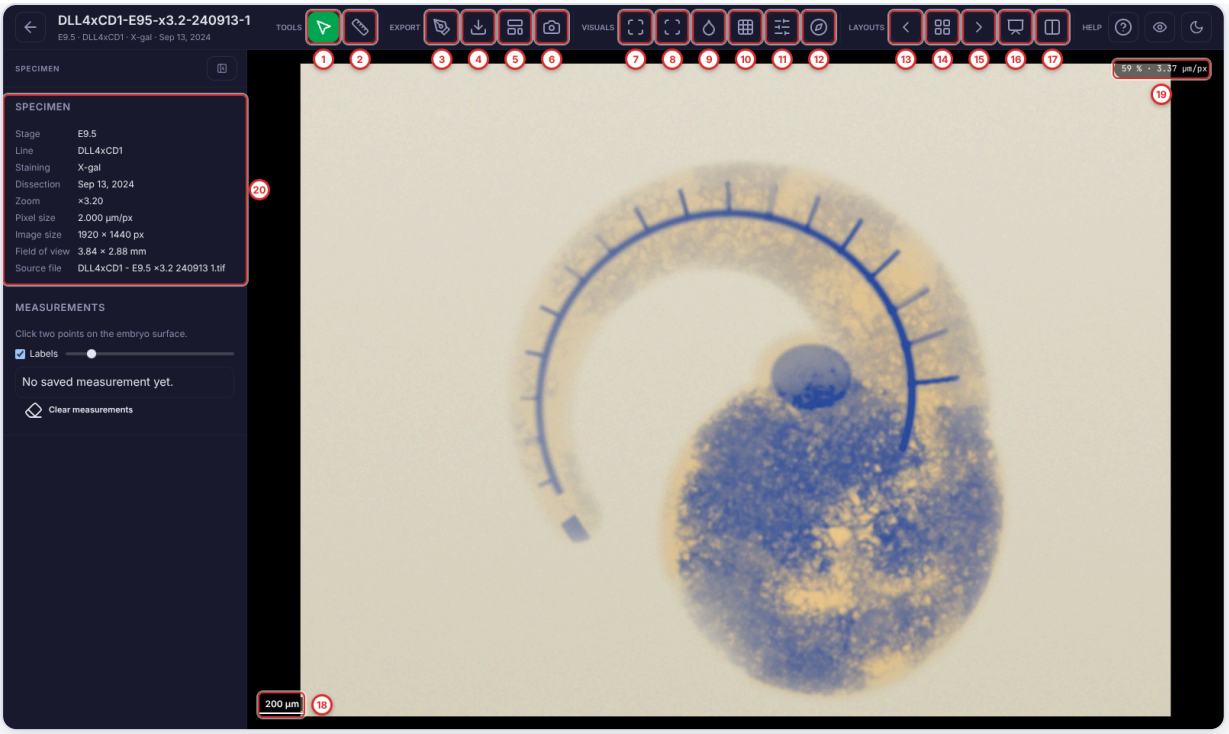

▲ Population by region (demo dataset).

- **Cell distance** (D): click two tracked cells. **Snapshot** mode (positions kept) or **Follow cells** (re-read at each frame; "Out of frame" if one disappears). Formula: $\sqrt{(\Delta x^2 + \Delta y^2 + \Delta z^2)}$ in µm.
- **Tracking charts**: **Population**, **Velocity**, **Neighbours**, **Mitoses**, **Linear** or **Log** scale, one curve per region.

## 12.10 The 2D page: photographs

**⏱ IN 30 SECONDS**

- **What it is for**: displaying a calibrated photograph (stereomicroscope, here X-gal), measuring it and comparing it.
- The engine is simpler than in 3D: a canvas you pan and zoom.
- **Isolating the staining** is a visual aid: nothing that is measured reads from it.

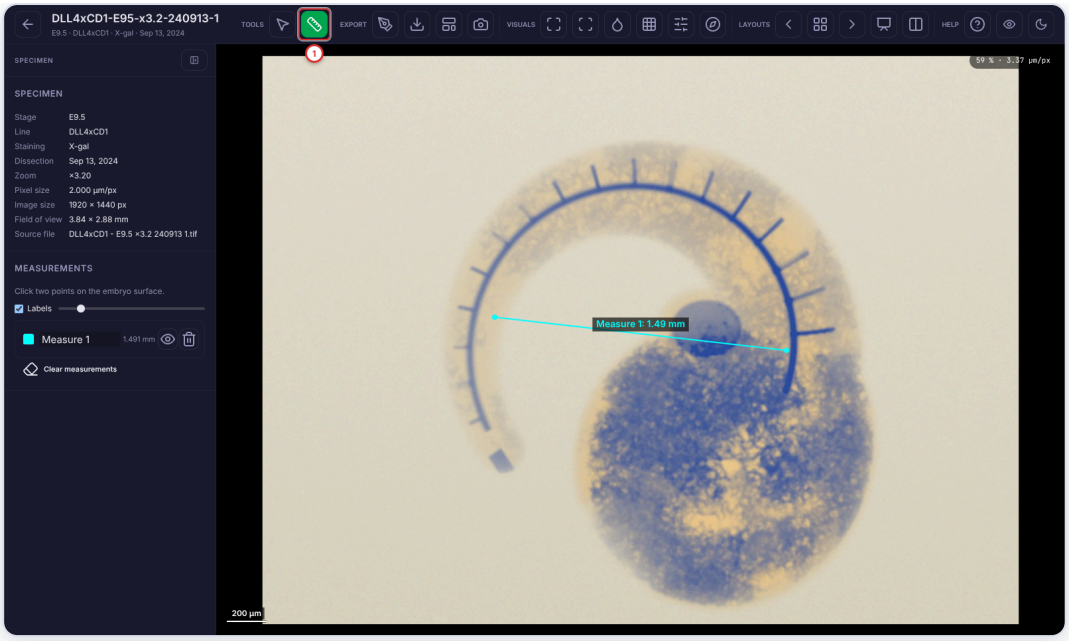



▲ The 2D page (1 to 17: the buttons; 18: scale bar; 19: zoom and µm/px; 20: specimen sheet).

| N | WHAT IT IS |
|---|---|
| 1, 2 | **Navigate**, **Measure distance** |
| 3 | **Open in Studio (annotate)** |
| 4, 5, 6 | Download, **Figure panel builder**, capture |
| 7, 8 | **Fit to screen (F)**, **Native resolution 1:1** |

| N | WHAT IT IS |
|---|---|
| 9 | **Isolate the staining** |
| 10, 11, 12 | **Calibrated grid**, **Display adjustments**, **Orientation** |
| 13, 14, 15 | Previous photograph, browse the collection, next |
| 16, 17 | Presentation mode, **Split view** |

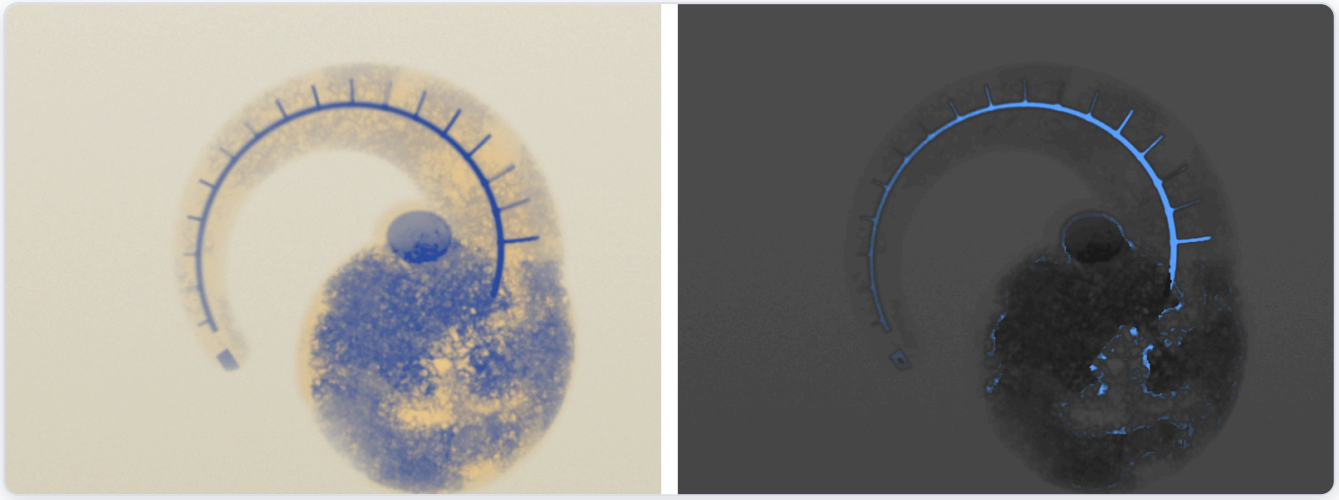
- **Navigation**: wheel to zoom around the cursor, drag to pan, double-click to fit. Zoom ranges from 0.25 × the fit up to 32 screen pixels per image pixel.
- **Two-stage loading**: a preview (≈ 640 px), then the native image.
- **Scale bar** at the bottom left, 1-2-5 value, computed with the pixel size (here 2.000 µm/px). Without calibration: **Uncalibrated**.
- **Browse the collection** (B) opens a contact sheet that can be filtered by stage and line.




▲ A measurement: 1.58 mm (1.576 mm in the list).

The measurement is `√(Δx² + Δy²) × pixel size`: the photograph is flat, there is no Z. Labels can be moved with the mouse.

## Isolating the staining (X-gal)



▲ Before (left) and after (right): the blue staining stands out, the rest turns dark grey.

X-gal makes a pixel **bluer** than the neighbouring tissue. But the matt background also carries bluish grains. To avoid lighting them up, the tool requires the blue to be **inside yellow tissue**.

### “Isolate the staining”: three steps for every pixel

**1 · Where is the yellow tissue?**

a pixel's “yellowness” =
**(red + green) / 2 − blue**

It is averaged over a square about 40 px wide (image reduced 4 times, blur radius 5).

matte background: 2 to 6
broad yellow trunk: 6 to 9

**2 · Is the pixel blue?**

ratio blue / (red + 1)

Unstained tissue: ≈ 0.6
Strongly stained core: 2 to 4

**strength v: 0 if ratio ≤ 1.0**
**1 if ratio ≥ 1.8, linear between**

and only if the local “yellowness” exceeds 5 (otherwise v = 0)

**3 · Repaint**

grey = brightness × 0.35 × (1 − v)

**red = grey + 90 × v**
**green = grey + 160 × v**
**blue = grey + 255 × v**

v = 0: dark grey
v = 1: vivid blue (90, 160, 255)

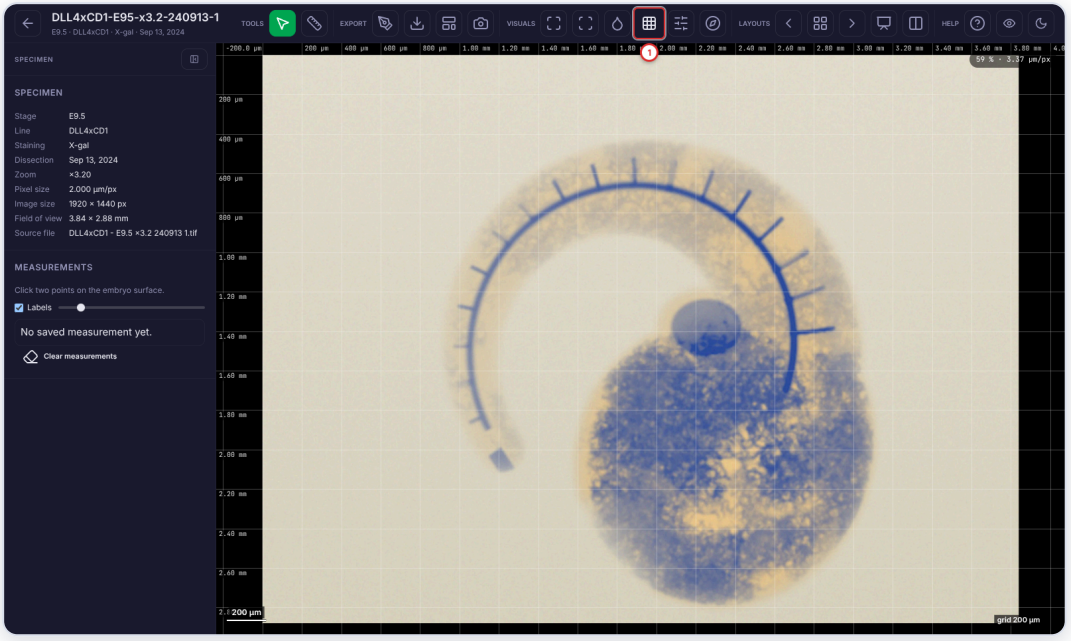
**Worked example**
Pixel (R, G, B) = (60, 90, 170): 170 / 61 = 2.8 → v = 1, in tissue with “yellowness” 8: it turns vivid blue.
Pixel (200, 190, 120): 120 / 201 = 0.6 → v = 0: it turns dark grey (0.35 × its brightness).

▲ The calculation, pixel by pixel.

🔧 **FOR THE CURIOUS**

Constants from the code: low ratio 1.0, high 1.8; context at 1/4 resolution, blur of radius 5 (≈ 40 px), threshold 5. Brightness = 0.299 R + 0.587 G + 0.114 B. All of it runs in a *worker* (the page stays smooth), only the latest request is processed, and the context map is kept as long as the image does not change. The tool is tuned for X-gal on yellow tissue: other stains or backgrounds may be detected badly.

## Display, grid, orientation

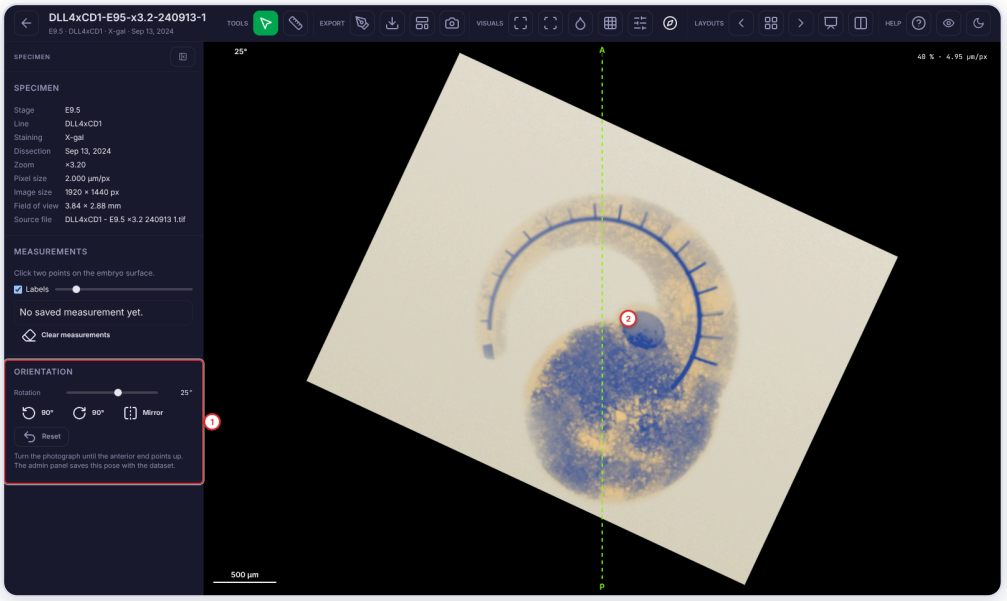



▲ **Calibrated grid** (200 µm): a reading scale over the photograph.

▲ **Display adjustments** (**Display** panel).

- **Display adjustments**: **Brightness**, **Contrast**, **Gamma**, **Red** / **Blue** balance, **Flatten the background (vignetting)**. *Display only*: measurements read the untouched photo. Per channel: `x = (v/255 · balance − ½)(1 + contrast/100) + ½ + brightness/100`, then `x^(1/gamma)`.
- **Flatten the background**: a smoothed (quadratic) surface is fitted to the darkest 40% of pixels (the background) and the corresponding gain is applied, between 0.5 and 3.

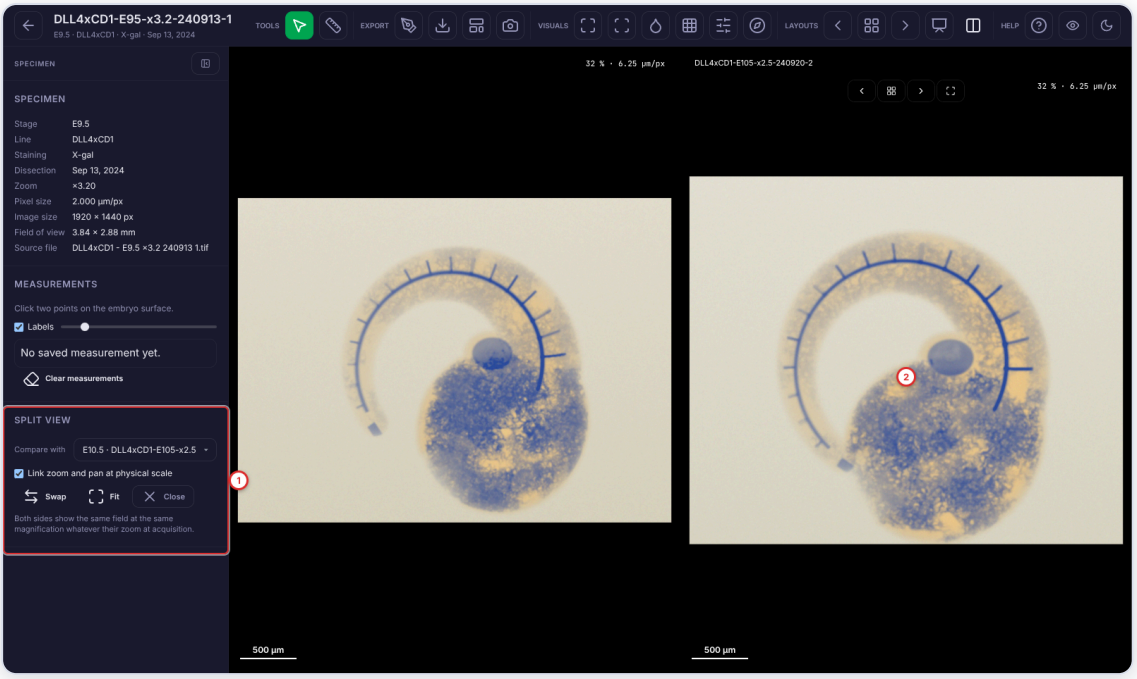
- **Calibrated grid**: one click = none → wide (≈ 120 px) → fine (≈ 60 px), not in 1-2-5 µm steps.




▲ **Orientation**: rotation of 25°, compass A (anterior) / P (posterior).

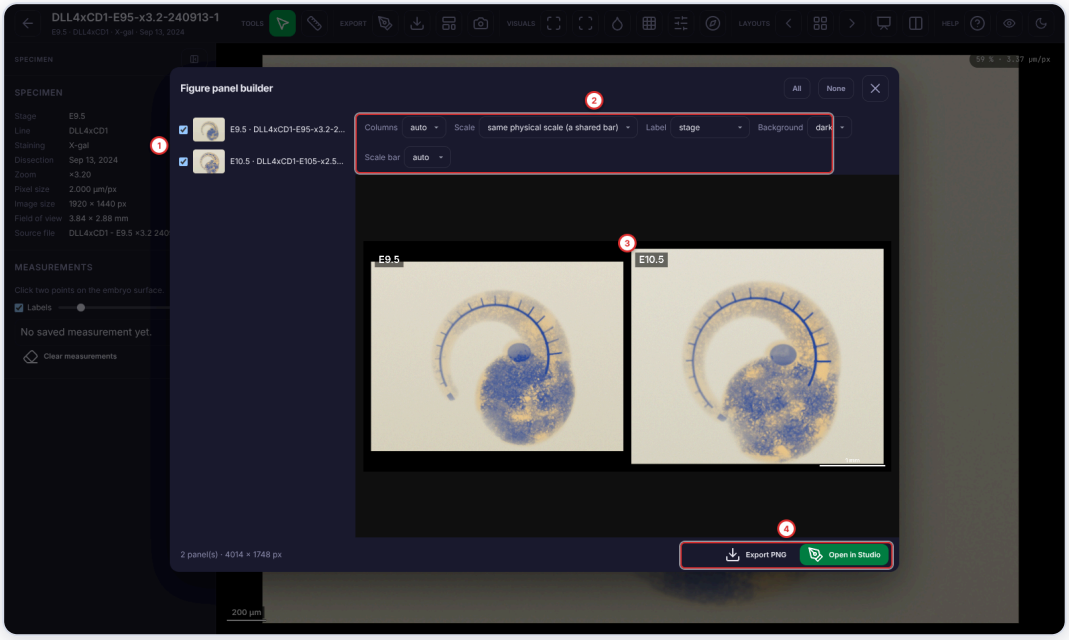
**Orientation** turns the image without changing the scale (rotation, ±90°, mirror). The administrator saves the pose with the dataset; it is reapplied each time the photograph is opened.

## Split view and figure panel




▲ **Split view**: two embryos at the **same physical magnification**.

The split view exchanges a **physical** view (µm per screen pixel + position) between the two sides: two photos taken at different zooms are at the same scale, and moving one moves the other by the same real distance.




▲ The **Figure panel builder**: list (1), options (2), preview (3), export (4).

The panel assembles several photographs into one PNG: **same physical scale** (one common bar; each image is brought to the coarsest pixel, never enlarged) or **same image size** (one bar per panel). Limits: 6,000 px wide, 120 megapixels. It also opens in the Studio.

# 12.11 Keyboard shortcuts and sharing a view

⏱ IN 30 SECONDS

- Shortcuts are ignored while you are typing in a field.
- **Sharing a view** = copying the page address: the state is in the link.

### 3D viewer

| KEY | ACTION |
|---|---|
| V / Esc | Navigate |
| C | Slice |
| M | Measure |
| I | Inspect a cell |
| D | Distance between cells |

### 2D page

| KEY | ACTION |
|---|---|
| ← / → | Previous / next photo |
| B | Contact sheet |
| F | Fit to screen |

### Studio

| KEY | ACTION |
|---|---|
| V R E | Select, rectangle, ellipse |
| A L T | Arrow, line, text |
| D G S | Distance, angle, scale bar |
| Space | Pan (held down) |
| Ctrl + K | Command palette |
| Ctrl + Z / Ctrl + Shift + Z | Undo / redo |
| Delete | Delete the layer |
| [ / ] | Rotate by −15° / +15° |

**Mouse**: left-drag = rotate; Shift or right/middle click = pan; wheel = zoom.

## Sharing a view

Every second the page writes its state into the address ( `#state=…` , compressed): camera, channels, active tool, cutting plane, **measurements**, frame of the series, quality, tool settings. Simply copy the address.

- Whoever opens the link sees: **This link carries a saved view** with **Open the saved view** or **Open the dataset as new** .
- **Reset the workspace** (button 21) puts view, tools, channels and measurements back to how the dataset opened; the **quality** is not changed.
- The viewer has no “Save” button: the address is the save. The Compare page, however, has **Save** / **Restore** (§ 12.8).

📌 REMEMBER

- A measurement is calculated in **µm** on what **you see**; a click in empty space is refused.
- The oblique slice and the Z-stack exclude each other.
- The Studio exports at native resolution; its JSON never contains pixels.
- Compare = the real pages in frames, with a shared memory budget.
- “Isolate the staining” is a visual aid, not a measurement.

# 13. The tools under the bonnet

- Chapter 12 taught you **how to use** the tools. This one lifts the bonnet: **how they work**, with the numbers and formulas of the code.
- Nine topics: cell tracking and stabilisation, the Studio, Compare, the 2D page and its importer, the timeline, shared state (link, browser memory), the measurement algorithms, the gallery and linked datasets, orientation.
- None of this is needed to use the platform. Each section reads on its own; the *For the curious* boxes can be skipped.

ℹ️ **GOOD TO KNOW**

Every screenshot and every figure comes from the **demo dataset** (synthetic embryos and photographs generated for this documentation). When an example uses values invented for clarity (a memory budget, two pixel sizes), the text says so.

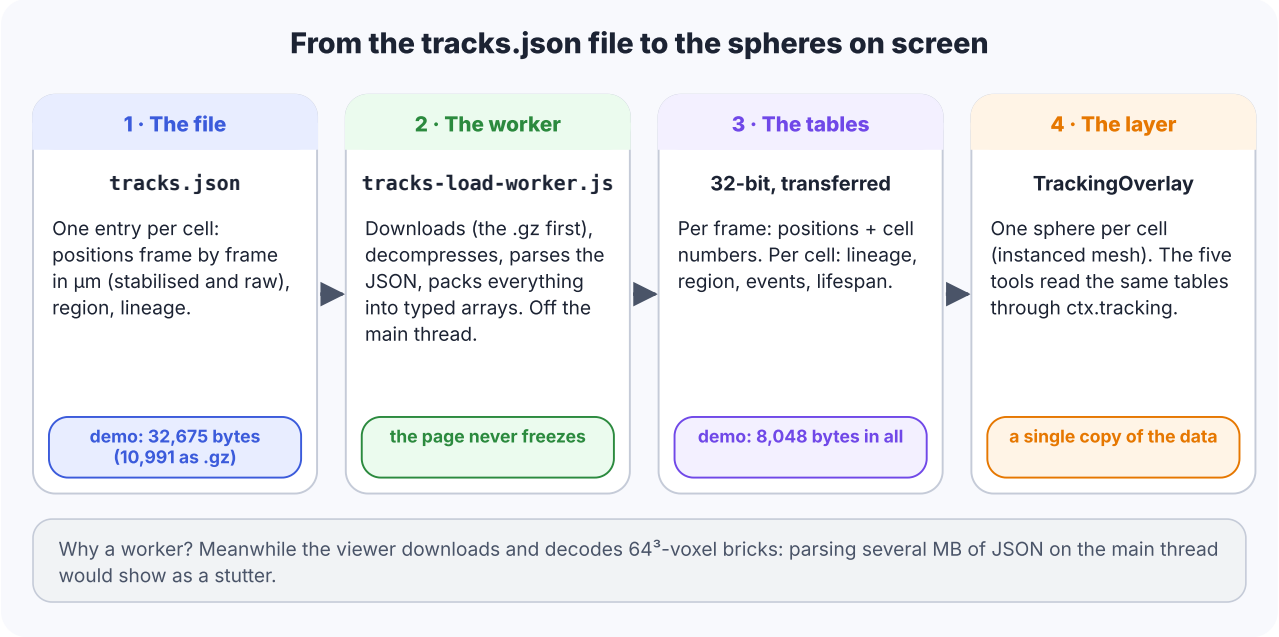
## 13.1 Cell tracking in the viewer

⏱ **IN 30 SECONDS**

- Tracked cells are drawn as **spheres** (one per cell), read from **arrays of numbers** prepared by a *worker*.
- Frame **stabilisation** never alters a single voxel: the shader only changes **where it reads** the texture, thanks to one 4 × 4 matrix per frame.
- Five tools share **one copy** of the data through a facade named `ctx.tracking`.

Chapter 8 (§ 8.4) explained how the pipeline computes the tracking and the Kabsch rotation. Chapter 12 (§ 12.9) showed the buttons. This is **the middle part**: what the viewer does with those files, frame after frame.

## From file to spheres




▲ The path of the tracking data, from the file to the screen.

The `tracks.json` file is not read by the page: a **worker** (a small programme running alongside the page, § 10.5) downloads it. It tries the compressed copy `tracks.json.gz` first and decompresses it with the browser. If that copy is missing, or the server has already decompressed it, the worker quietly falls back to the plain file.

The worker draws nothing. It puts everything into **typed arrays** (fixed-size runs of numbers) that it *transfers* to the page without copying. The viewer keeps these arrays **exactly once**: the drawing, the five tools and the exports all read the same ones.

> 💡 **A PICTURE TO UNDERSTAND**
>
> **A phone book and a metro map.** The JSON file is a phone book: pleasant to read, slow to leaf through. The arrays are the metro map: everything sits at a fixed address. “Where is cell 15 at frame 3?” becomes **one read in an array**, not a search.

## The arrays: two families

**Per frame** (for drawing, 30 times a second)

| ARRAY | CONTENT |
|---|---|
| `posStab` | stabilised positions, frame by frame |
| `posRaw` | the same, raw |
| `cellIdx` | number of the cell in each slot |
| `counts` | number of cells per frame |

**Per cell** (for analysis)

| ARRAY | CONTENT |
|---|---|
| `parent`, `daughterStart/Idx` | lineage (mother; list of daughters) |
| `flags`, `regionIdx` | mitosis / fusion; region |
| `firstFrame`, `lastFrame` | lifespan |
| `cellFrameSlot` | slot of the cell **at each frame** (−1 = absent) |

The last array is the most useful: `cellFrameSlot[cell × frames + frame]` answers “where is this cell at this frame?” in **a single read**. It is what makes selection, trajectories and instant distances possible.

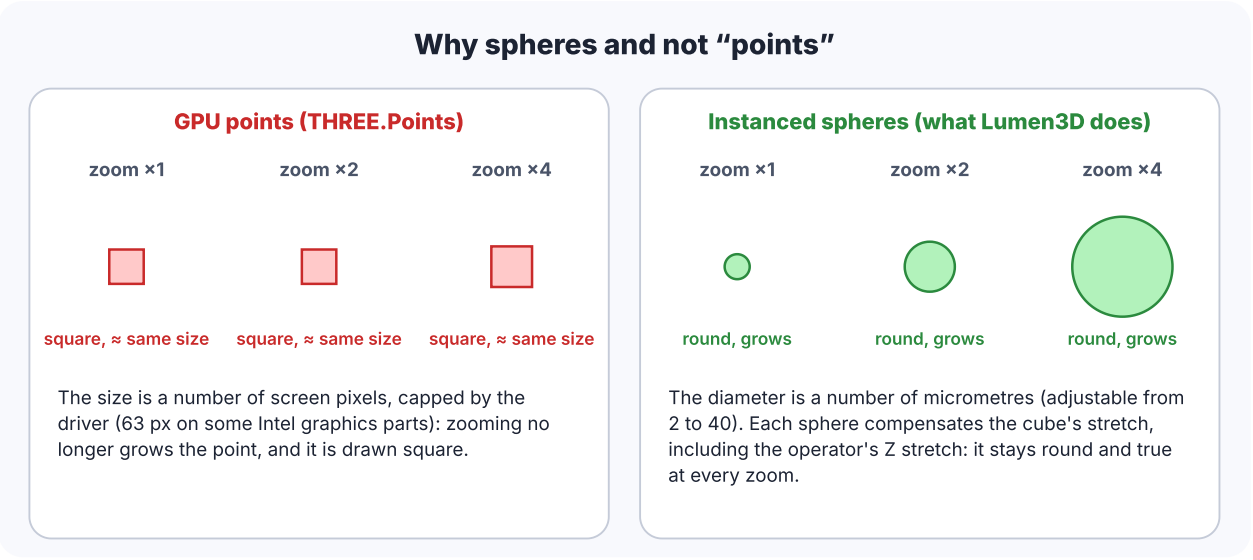
🔢 **WORKED EXAMPLE**

On the demo dataset (50 cells, 4 frames) the worker produces 8,048 bytes of arrays from a 32,675-byte file (10,991 compressed). The frames do not all hold the same number of cells: **48, 48, 49, 49**. Free slots are never drawn; they do not stay frozen at their last position.

🔧 **FOR THE CURIOUS**

- **32 bits everywhere**: counters and indices are `Uint32Array` / `Int32Array`. A 16-bit array would silently overflow beyond 65,535 cells; here nothing overflows.
- Cells are identified in the file by the text of their frame (“1”, “2”..., Imaris counts from 1) whereas bricks count from 0. Rather than adding 1 by hand, the worker **builds the mapping** from the file’s sorted `timepoints` list.
- An incomplete or non-finite position is **skipped and counted** (`malformed`): one faulty row never hides the whole layer.
- The `cellFrameSlot` table is the only one that grows with cells × frames. It is refused beyond $2^{28}$ entries (1 GiB) with a message that says why, instead of an opaque out-of-memory error.
- A lineage may designate a cell by its key, its `id` or its `track_id` (the exporter has used all three); the worker tries them in that order, then **makes the link symmetric** (mother ↔ daughters).

## Why spheres and not “points”




▲ GPU points versus instanced spheres.

A sphere costs a little more than a point, but its diameter is set **in micrometres** (2 to 40, 12 by default) and stays correct at any zoom. The layer is an *instanced mesh*: a single reference sphere, drawn as many times as there are cells, with one matrix per instance.

🔧 **FOR THE CURIOUS**

- The spheres are **children of the volume's cube**: orbit, pan, physical proportions and the operator's Z stretch apply with no extra code.
- The world diameter is `diameter_µm × (cube scale_x / acquisition size_x in µm)`. The sphere's scale is then **divided by the cube's own scale on each axis**: without that division, the Z stretch would turn it into an ellipsoid.
- `depthTest = false` : the volume is an additive ray-marcher **without depth**; a depth test would hide every centroid behind the cube's front face.
- The test "is this sphere inside the cropped box?" is **the same** as the shader's: the layer follows the Z-stack slab and the crop sliders instead of floating over a nearly hidden volume.
- A selected cell is drawn 1.8 times larger. A cell's colour comes from its region (one colour per **cell**, not per cell and per frame: thirty times fewer bytes).

## Two frames of reference, one dictionary switch

The file carries **two sets of coordinates** for each cell: stabilised ( `posStab` ) and raw ( `posRaw` ). The layer never applies the stabilisation matrix to points: it **picks the right array**.

⚠️ **WARNING**

Applying the matrix to points that are already stabilised would send them into a third frame of reference. At frame 1 the transformation is the identity, so you would see nothing. At frame 30 of a real series the error would reach about 250 µm and look like a general alignment fault. Hence the rule: **a dictionary, not a multiplication**.

- Stabilised volume on screen: the layer reads `posStab` .
- Raw volume: it reads `posRaw` .
- If the file does **not** have raw coordinates for every cell and the displayed frame is raw, the layer **hides itself** ("Raw coordinates missing: tracking hidden on a non-stabilised volume"). A plausible but wrong layer would be worse than no layer. Your own visibility setting is not touched.

## Clicking a sphere

`pick()` casts a ray through the clicked point and tests it against the mesh **as it was drawn**: a filtered (mitoses, fusions) or cropped cell is never hit, and a hidden layer cannot be clicked. The result is the cell number, or −1.

## The `ctx.tracking` facade

The five tools (trajectories, surface, inspector, charts, cell distance) never touch the mesh or the arrays directly. They go through a **facade** that the page builds.

| IT PROVIDES… | EXAMPLES |
|---|---|
| **data** | `getData()` , `positionUm(cell, frame)` , `cellsAt(frame)` , `whenLoaded()` |
| **shared state** | **one** selection, **one** neighbour radius (5 to 500 µm, 55 by default) |
| **services** | `pick(x, y)` , `umToObject()` , `isInsideClip()` , `getCamera()` |

| IT PROVIDES... | EXAMPLES |
|---|---|
| events | `loaded`, `frame`, `refresh`, `selection`, `options`, `style` |

❓ **WHY THIS CHOICE?**

**One selection, not five.** If the inspector, the trajectories and the charts each had their own, clicking a cell would light up three different things. With the facade, a chosen cell lights up everywhere, and the neighbour radius sets the inspector, the charts and the surface density in one go.

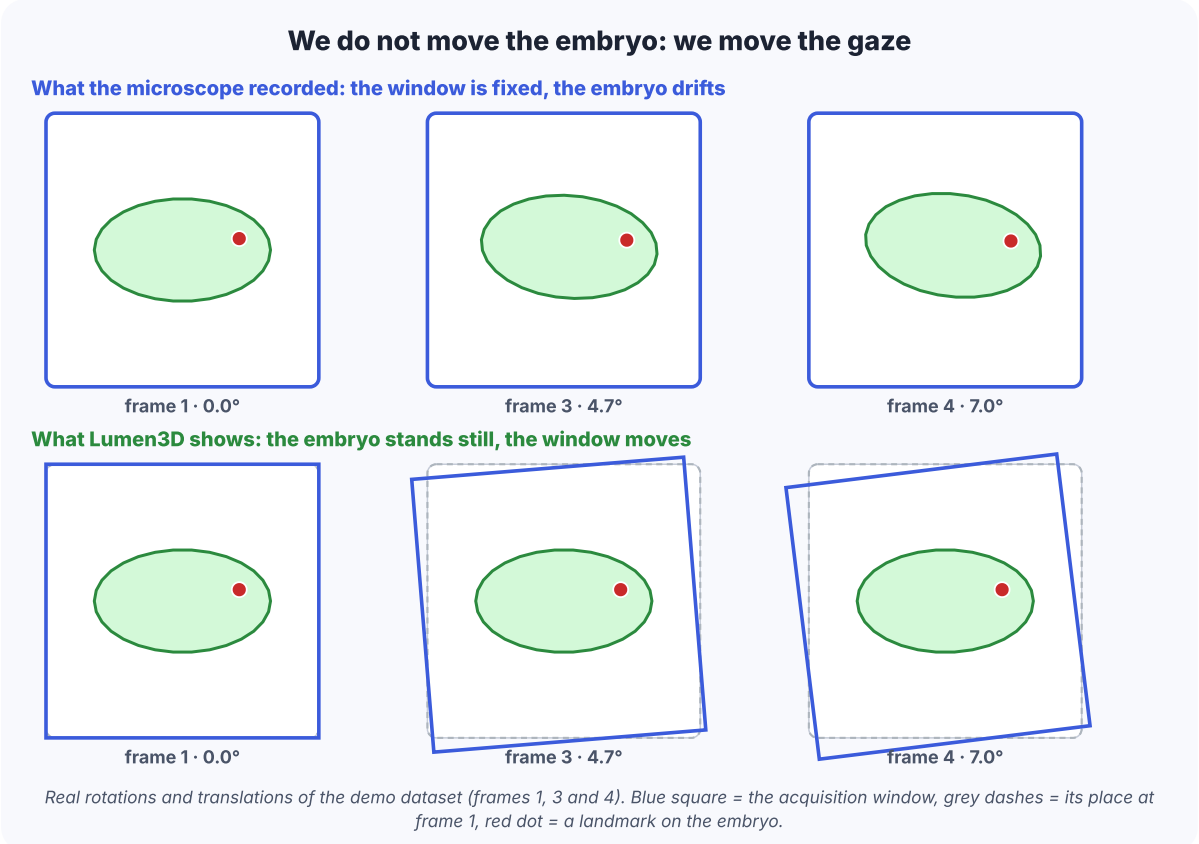
The `frame` event is emitted **after** the layer has moved: a tool that draws over the points never runs one frame ahead of them.

## 13.2 Stabilisation: moving the gaze

⏱ **IN 30 SECONDS**

- An embryo drifts and turns for hours. Stabilisation **cancels that movement on screen**.
- The viewer manufactures no new image: the shader reads each voxel **at another address** of the texture.
- It is applied only if the pipeline has checked that the motion is **rigid**.

### The principle in three pictures




*Real rotations and translations of the demo dataset (frames 1, 3 and 4). Blue square = the acquisition window, grey dashes = its place at frame 1, red dot = a landmark on the embryo.*

▲ Top row: what the microscope recorded. Bottom row: what the viewer shows.

The microscope always looks through the same window. If the embryo drifts, **it** moves within the window. The viewer reverses the point of view: the embryo stays put on screen, and it is **the window** (the blue square) that turns and slides around it, like a camera following the animal.

💡 **A PICTURE TO UNDERSTAND**

**A camera operator, not a set decorator.** To keep a dancer in the middle of the picture, you can redraw the scenery around them in every shot (resampling), or simply **turn the camera**. Lumen3D turns the camera: the scenery, meaning the measured voxels, is never retouched.

### Two ways to “stabilise” images

**Resample (not used)**

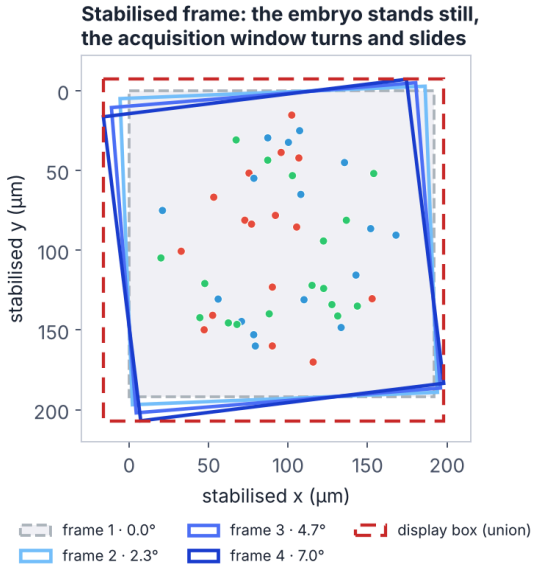
- Build, for each frame, a rotated and shifted copy of the volume.
- Every voxel is recomputed by interpolation: slightly blurred, so no longer the measurement.
- Cost: the whole volume, at every frame.
- Irreversible once the bricks are written.

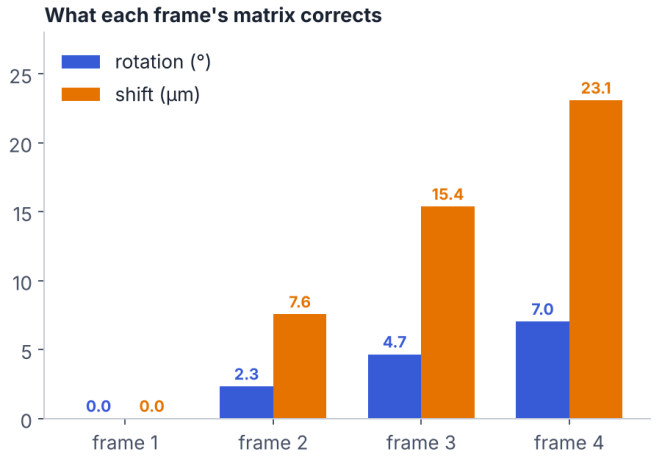
**Move the gaze (Lumen3D)**

- The texture is never touched: the voxels stay those of the measurement.
- Only the reading point changes: each ray is sent through the frame's matrix.
- Cost: one 4 × 4 matrix per frame.
- Reversible in one click (stabilised ⇄ raw).

▲ Two ways of stabilising, and the one Lumen3D chooses.

## The frames of reference, with real numbers




*Demo dataset (synthetic): rotations 0; 2.3; 4.7; 7.0°; dots = frame-4 cells coloured by region.*

▲ Four acquisition windows (one per frame) in the embryo’s frame, and the box that contains them all.

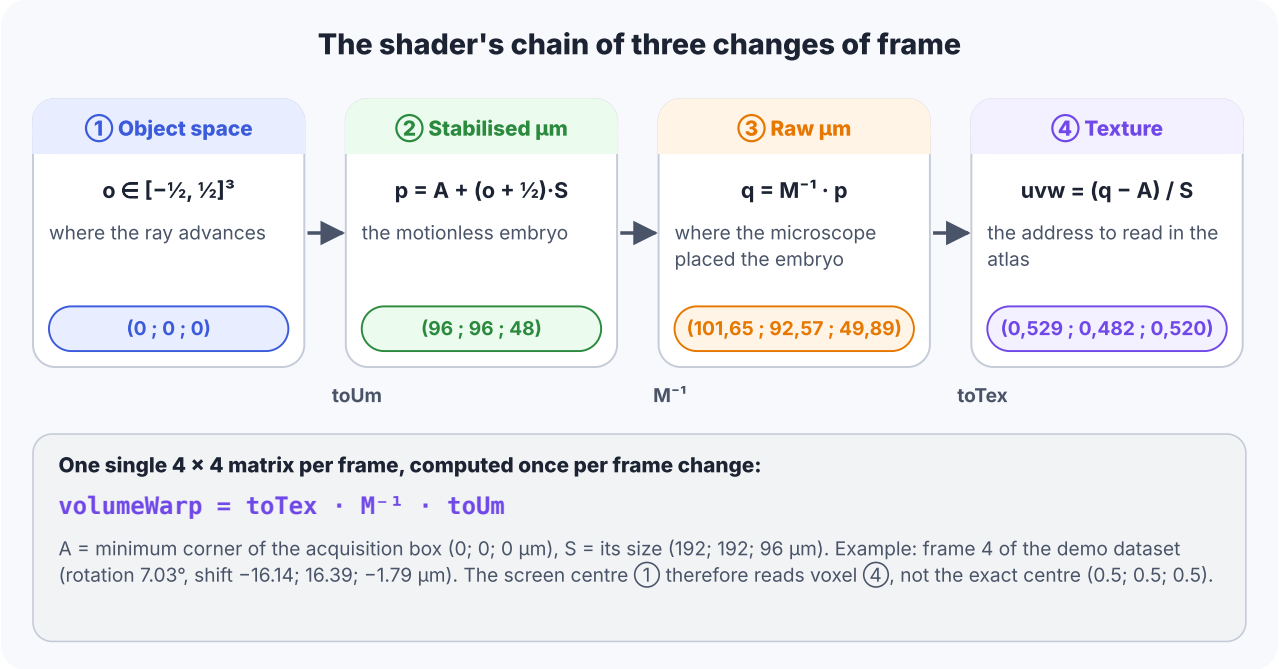
Three frames of reference are involved:

- the **raw frame**: the microscope’s own; the 192 × 192 × 96 µm window is fixed;
- the **stabilised frame**: the embryo’s, motionless; each frame has its own window there, rotated by 0; 2.3; 4.7; 7.0°;

- the **display box**: the smallest block that contains **all** the windows, here from (−16.3; −7.1; −2.4) to (197.9; 207.3; 96) µm.

The display box is what the geometric cube has to cover: without it, the window of frame 4 would stick out of the cube. The cube therefore grows, **without changing** its scale (the grid, the scale bar, the cutting plane and the camera framing keep relying on it).

## The shader: a chain of three changes of frame




▲ The point that is read goes through three frames, with the example of frame 4.

For every point where the ray takes a sample, the shader computes:

```
volumeWarp = toTex · M⁻¹ · toUm
```

1. `toUm` converts the object coordinate (from −½ to +½) into **stabilised micrometres**.
2. `M⁻¹` goes back to **raw micrometres**: it is the inverse of the matrix the tracking applied to the frame.
3. `toTex` converts to a **texture address** (from 0 to 1).

> 🔢 **WORKED EXAMPLE**
>
> **Frame 4 of the demo dataset.** The exact centre of the screen (object 0; 0; 0) is (96; 96; 48) stabilised µm. After `M⁻¹` (rotation 7.03°, offset −16.14; 16.39; −1.79 µm) it becomes (101.65; 92.57; 49.89) raw µm, hence the texture address (0.529; 0.482; 0.520). A voxel slightly off-centre is read: **that is the entire "work"** of stabilisation.

The matrix is computed **once per frame change** and passed to the shader as a constant. On the very first pass the shader is recompiled with the `VOLUME_WARP` option; after that, each frame only changes a number.

🔧 FOR THE CURIOUS

- **Ray entry and exit.** The ray / box intersection is computed in the space of the **source texture** (where the acquisition box is exactly the unit cube), by `hitBoxWarped`. Sampling density therefore does not depend on how far the embryo has drifted: the image is no brighter or grainier at frame 30 than at frame 1.
- **Crop and cut.** The crop sliders, the Z-stack slab and the cutting plane act in the **display box**: you cut what you see, not the acquisition frame.
- **Safeguards.** A non-finite matrix, or one with a near-zero determinant ($|\det| \le 10^{-9}$), is **ignored**: the frame is shown undeformed and a warning appears in the console. A rigid rotation has a determinant of ±1: anything that strays from that is suspect.
- **Orbit untouched.** The warp changes neither the cube's position nor its rotation: those are the mouse controls and are saved in workspaces. Mixing a per-frame matrix into them would break both.
- **The oblique cut and the Studio** sample through the same warp (`samplingSpace`, `planeSweep`): a cut through a stabilised series shows exactly the slice of the screen.

## When is it applied?

| CONDITION | IF IT IS MISSING |
|---|---|
| `registration.appliedToVolume` **is true (the rigidity test passed: residual ≤ 0.05 µm)** | the image stays **raw**, only the tracking is stabilised |
| `acquisitionExtentUm` **is present** | same, with a warning |
| **the frame's matrix is valid** | that frame is shown undeformed |

The pipeline's rigidity test (chapter 8) is therefore **the only green light**. On the demo dataset the residual is $7.6 \times 10^{-14}$ µm: stabilisation is applied.

ℹ️ GOOD TO KNOW

There is **no "stabilised / raw" button** in the interface: stabilisation is on as soon as it is applicable. The switch exists in the page's API (`ViewerApp.setVolumeStabilized`); the points layer and the tools follow the change on the next frame, because they read the displayed frame of reference from the frame-ready event, never by asking again.

⚠️ WARNING

**On the demo dataset the image has not really drifted**: only the tracking's raw coordinates were generated with a drift (up to 23 µm, 7°). Applying stabilisation therefore shifts the volume slightly. This is why the figures in this section are **diagrams computed with the real matrices**, and not "before / after" screenshots that would suggest a visible gain.

## The surface, provenance, measurements

- **The surface** `model.glb` contains one mesh **per frame**, in the raw frame and in the stabilised frame. It is a child of the cube like the spheres: the same `o = (p − A)/S − ½` places it, so orbit, Z stretch and stabilisation apply. The GLB reader is Three.js's own, **loaded on first use** with its integrity hash (SRI): the viewer page does not carry it.
- **Three colourings**: uniform, local density, region. A vertex's density is the sum of Gaussians `exp(−|v − c|² / 2σ²)` over the frame's cells, capped at 2.8 σ and stored in a uniform grid (a

vertex only visits its neighbourhood), then smoothed twice over the mesh. The result is remembered as long as neither the frame, nor the neighbour radius, nor the palette changes.

- **The surface cut** uses Three.js clipping planes, which are in **world space** whereas the cube rotates: the planes are therefore re-expressed just before each draw. A solid cap closes the cut.
- **Provenance.** `tracking.surfaceOrigin` says whether the surface was **exported** by the analysis or **rebuilt** from the cells (the demo dataset's case).
- **Alignment.** On import the pipeline checks that the tracking really lies inside the volume: at least 50% of the raw positions inside the acquisition box enlarged by 10%, and a spread of at least 1% of the longest side. Otherwise it suggests the probable unit (mm, nm, m), without converting anything. The demo dataset: 100% inside, spread 0.83, `aligned: true`.
- **Units.** Positions are brought to µm on import; `tracking.provenance.unit` keeps the declared unit, the factor and its status (*native*, *converted*, *assumed*, *unknown*).
- **Measurements** of distances between cells are stored under the `tracking` scope of the measurement store, and their export notes the frame of reference (*stabilized* or *raw*).

📌 **REMEMBER**

- Cells are **instanced spheres** read from **32-bit arrays**, in a single copy.
- Stabilisation is **a change of reading address**: `volumeWarp = toTex · M⁻¹ · toUm`.
- Nothing is resampled: it is reversible and exact.
- The only green light is the pipeline's **rigidity test**.

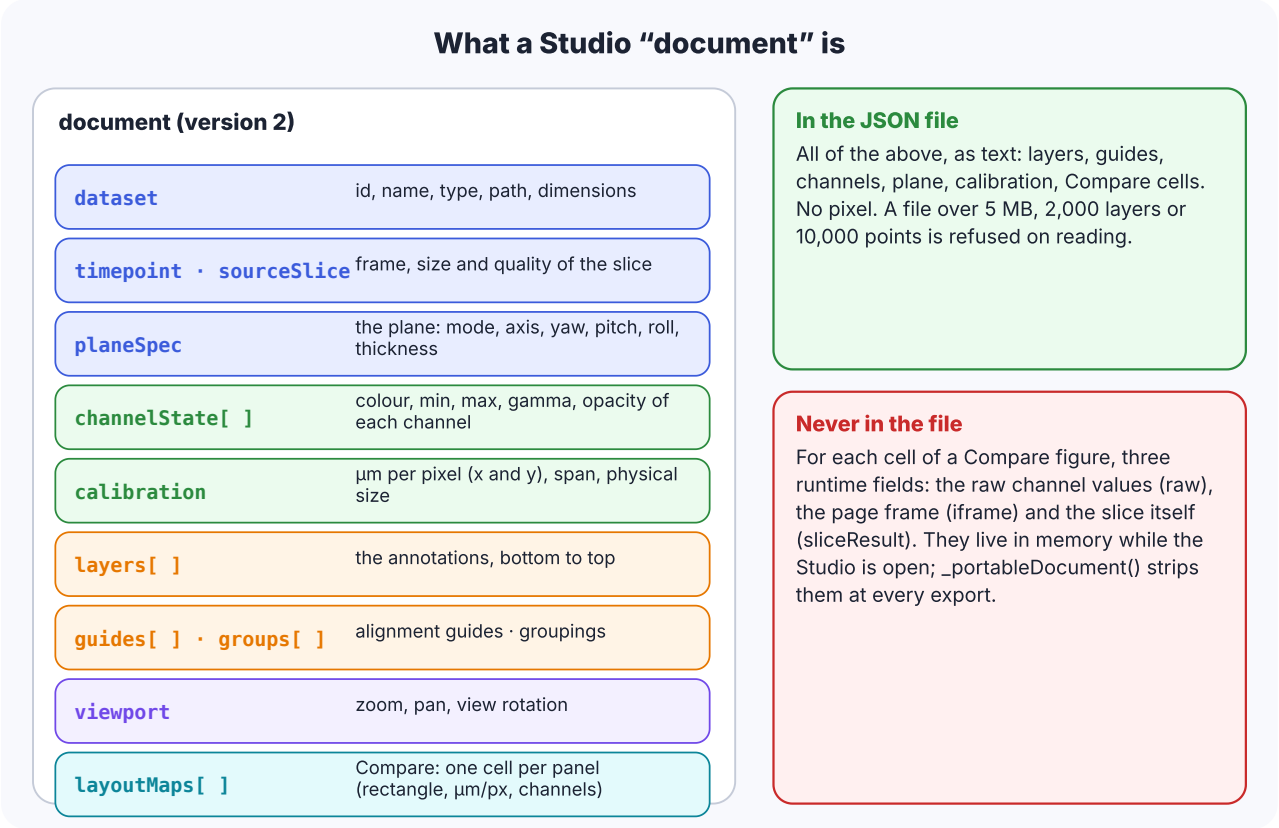
## 13.3 The Studio in depth

⏱ **IN 30 SECONDS**

- A Studio figure is a **document**: layers, guides, channel settings, a plane, a calibration. Never pixels in the JSON file.
- Channels are **recoloured without a new render**, with the same arithmetic as the slice shader.
- The **native** resolution arrives after opening, through a "plane" or an "atlas" path, under the protection of a document **token**.

Chapter 12 (§ 12.7) showed how to annotate. Here is what happens when you click.

## The document

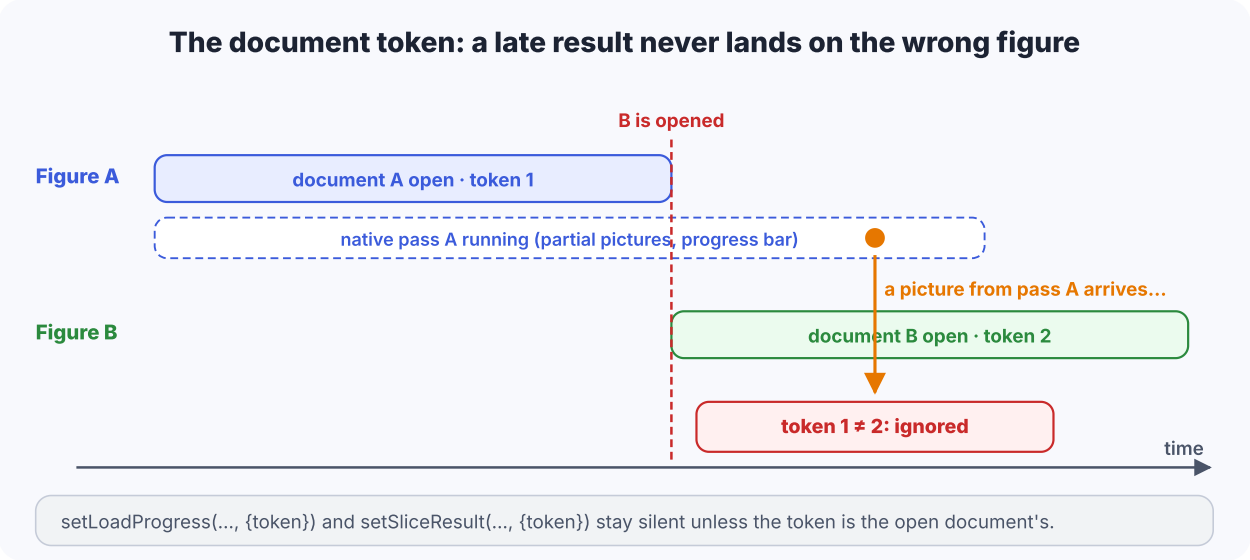



▲ The anatomy of a Studio document, and what the JSON file never contains.

The document fits in one object. The Studio **copies it entirely** at each history step, with two consequences: undo is instant and free of surprises, and the document must not contain large objects. In a Compare figure each cell carries three runtime fields (the raw values, the page frame, the cut): `_portableDocument()` strips them from every export, and the history copy keeps them **by reference**, not by duplication.

- **The history** keeps 80 steps. A channel change is a step like any other: undo reverts the **last** change, whatever it is, and undoing an annotation never restores the earlier channels.
- A burst of changes from one panel (a slider being dragged, one message per frame) forms **a single step**, closed by release, by a one-second pause, or by another action.
- **Layer types**: rectangle, ellipse, text, line, arrow, distance, angle, scale bar. Each layer has a unique identifier, a name, two flags (visible, locked), an optional group, a **rotation** in degrees (from −180 exclusive to 180) and a style.
- Coordinates are in **image pixels**. When the native pass replaces the preview with a larger image, every coordinate is multiplied by the size ratio: a layer stays on the structure it marks (line widths and font sizes, however, stay in screen pixels).

## The document token



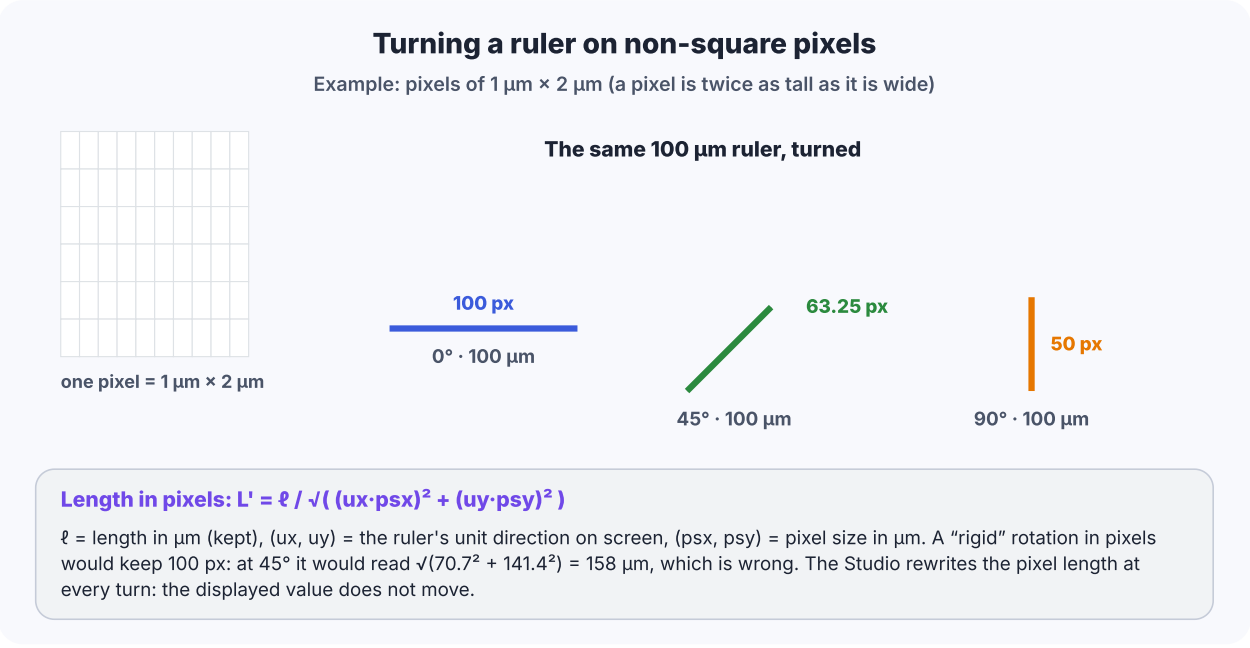

▲ Why a late native pass cannot damage the next figure.

Opening a figure returns a **token** (a number that increases with every document). The native pass, which can last several minutes on a large volume, presents it with each of its updates. If you have closed the figure and opened another, the token no longer matches: the late image is **ignored**, and so is its progress bar.

Closing the Studio frees the native image, the history and the Compare frames, and cancels the transfer in progress.

## Rotating a layer without falsifying a measurement

A rectangle, an ellipse or a text are **turned at display time** around their centre: only the rotation is stored. A line, an arrow, a distance, an angle or a scale bar are made of **points**: the rotation is *baked* into the points, because their label is computed from them.




▲ A 100 µm ruler rotated on 1 × 2 µm pixels: its length in pixels is recomputed.

- **Line, arrow**: rigid rotation of the points around their centre.
- **Angle**: turned in **micrometre** space ( $p' = c + S^{-1} \cdot R(\theta) \cdot S \cdot (p - c)$ , with $S$ the pixel size). A rigid rotation in pixels would change the measured angle: on 1 × 2 µm pixels, a right angle turned by 45° would read 53.1°.
- **Distance**: the direction turns, the length in pixels is **recomputed** to keep the same length in µm.
- **Scale bar**: its direction *is* its rotation; the length in pixels is the one that measures exactly its value.

🔢 **WORKED EXAMPLE**

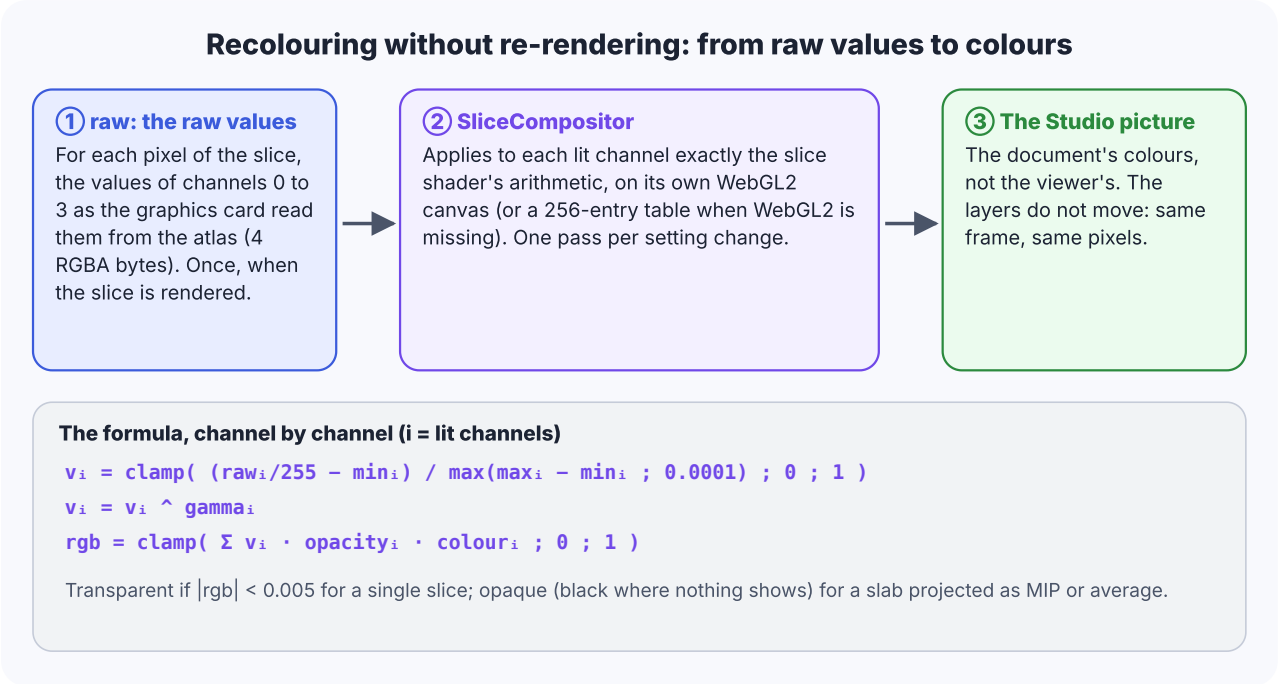
A 100 µm distance on 1 × 2 µm pixels. Horizontal: 100 px. At 45°: 100 / √(0.707² × 1² + 0.707² × 2²) = **63.25 px**. Vertical: **50 px**. The displayed value stays 100 µm.

## Which pixel size for a layer?

A single slice has just one calibration. A **Compare figure** has **one per cell** (a panel = a cell with its rectangle and its µm/pixel). A point layer takes the calibration **of the cell that contains the middle of its points** (the nearest one, if the middle falls in a gutter). That is also the centre of its rotations: turning a layer therefore never hands it over to the neighbouring cell.

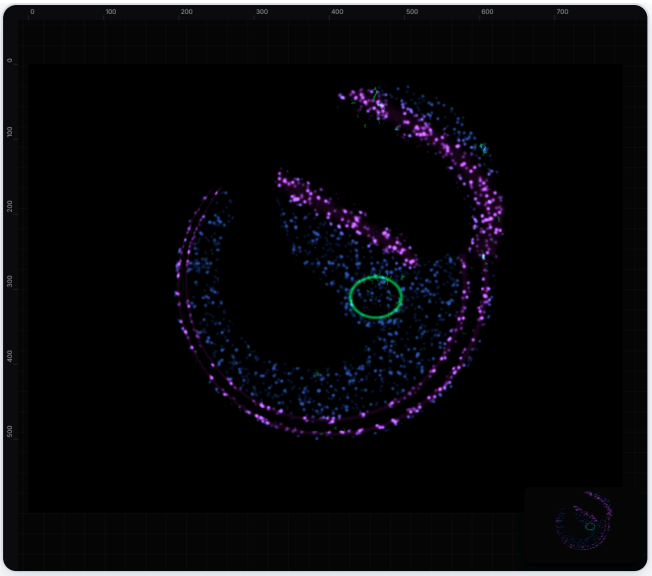
The scale bar is subtler: its middle depends on the length it takes, which depends on the cell. The Studio therefore looks for a **self-consistent** cell (whose own length puts the middle inside that same cell); failing that, the cell of the starting point decides, because it does not move when the other end is rewritten.

## Recolouring without re-rendering

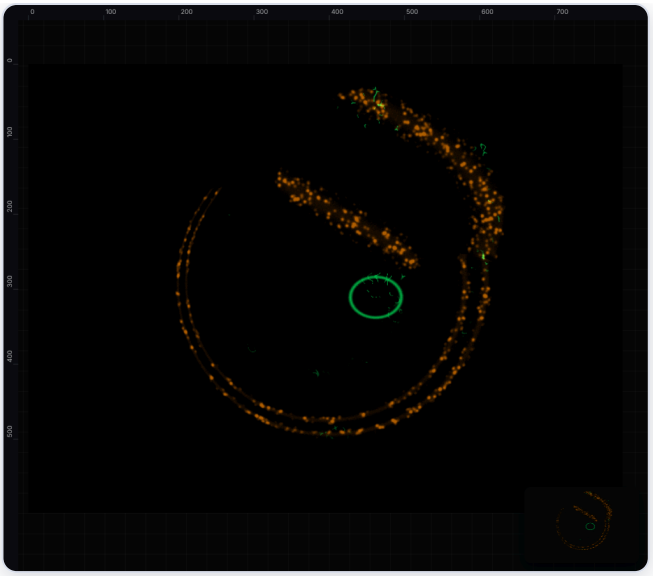



▲ The path from raw values to colours.

When the Studio opens a slice, it receives two things: a coloured image, and the **raw values** of the channels ( `raw` : four bytes per pixel, one per channel 0 to 3). The volume that produced them is a disposable atlas, gone after the pass. Recolouring therefore starts again from the raw values.



▲ XY slice of the demo dataset, viewer colours.



▲ Same slice: DAPI off, Sox2 in orange. The layers have not moved.

The **compositor** ( `SliceCompositor` ) applies the slice shader's formula to every channel that is on: window `(raw/255 − min) / max(max − min; 10⁻⁴)` , then gamma, then `opacity × colour` , and a sum. It runs on **its own WebGL2 canvas** (the Compare page does not load Three.js), with a 256-value table per channel as a fallback.

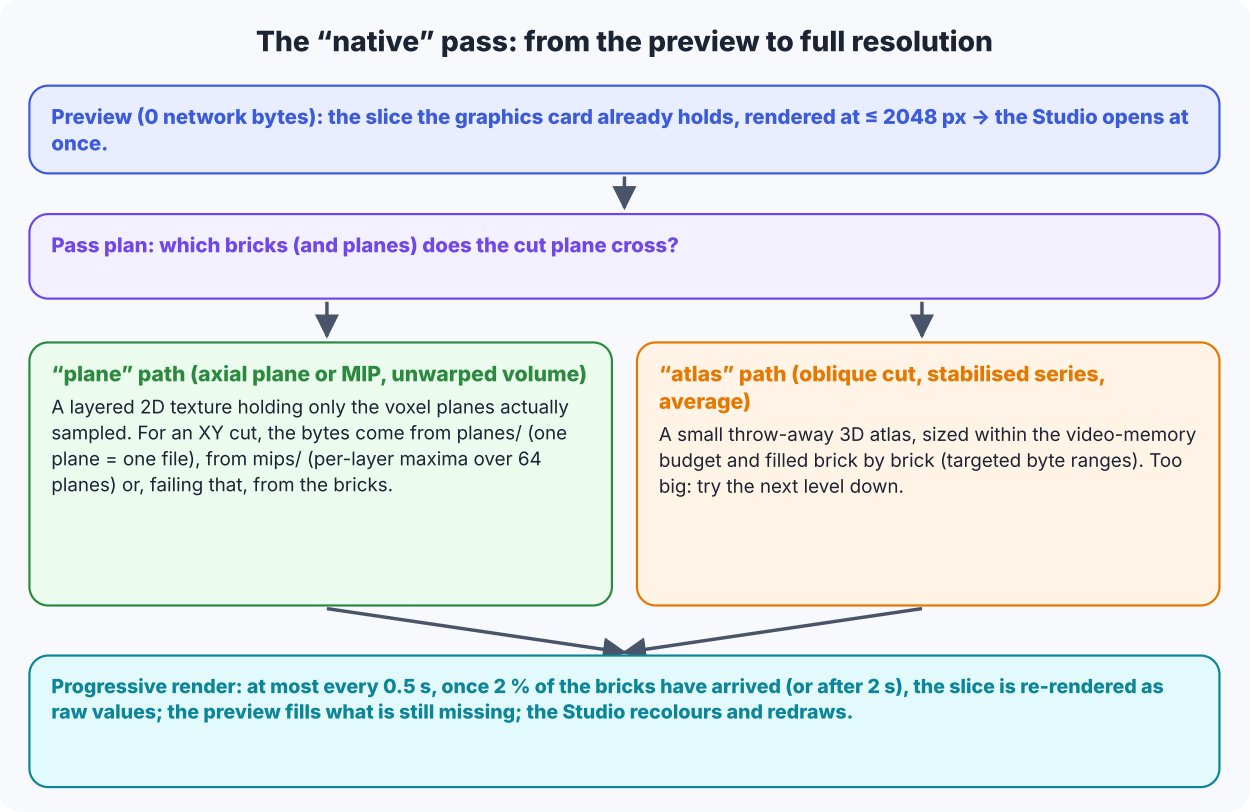
🔧 **FOR THE CURIOUS**

- The output colour is `clamp(Σ vᵢ·opacityᵢ·colourᵢ; 0; 1) × 255` , **transparent** below |rgb| < 0.005 for a single slice (the slice shader discards those fragments), **opaque** (black if nothing shows) for a slab projected as MIP or average.
- A **four-channel** slab has no spare byte to say where the volume exists: it carries a separate **coverage mask** of one byte per pixel, read from the alpha of the colour render of the same plane.
- The raw-value textures are kept within a budget of **256 MiB** (those of the masks in a separate 64 MiB budget, so that they do not evict each other). A raw of the same size is **rewritten in place** at each refinement of the native pass ( `raw.version` number).
- The table fallback computes in double precision; the graphics card, in 32-bit floats, may differ by **one unit** on a byte at a rounding boundary.

**The Studio's histograms do not have the same origin as the viewer's** (§ 11.5). They are computed **on the displayed slice** (up to 4 million samples, a single computation per raw), not over the whole volume. That is consistent: you set the contrast of what you see in the figure.

## The native pass

A volume of several gigabytes has no full-resolution slice "within reach". The Studio therefore works in two steps.




▲ From the preview to native resolution.

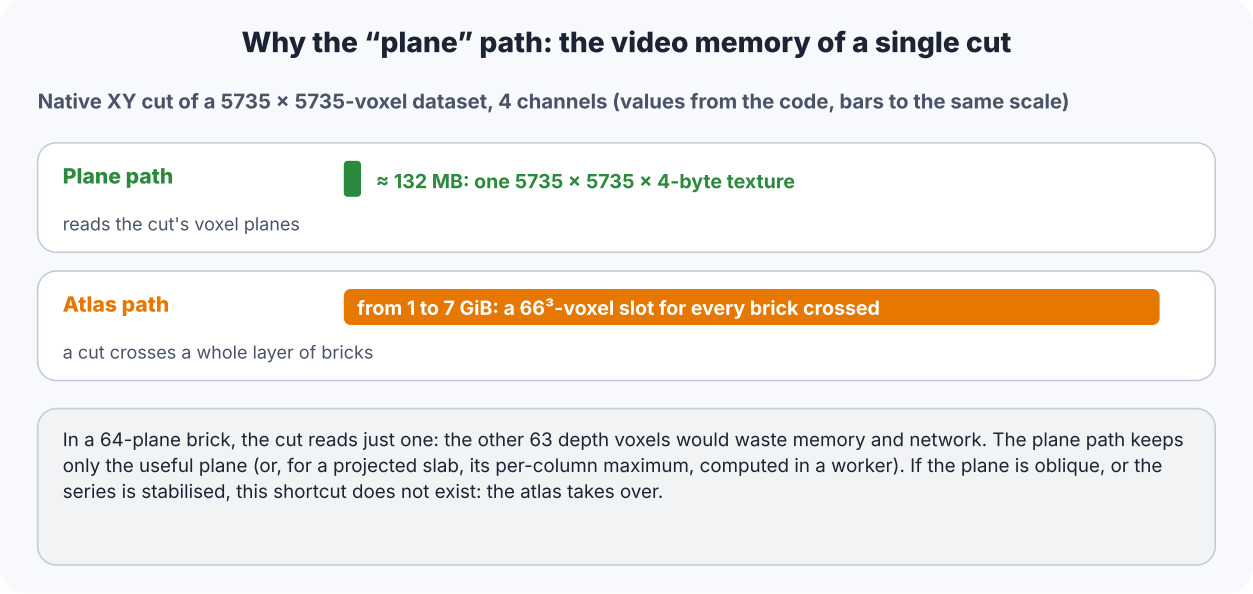
1. **The preview**: the slice the graphics card already holds, rendered at 2048 px at most, with no network bytes at all. The Studio opens instantly.
2. **The native pass**: the viewer works out which bricks the plane crosses, fetches them at maximum resolution (level 0), and re-renders the slice in raw values **at regular intervals**.

The pass’s image is framed like the preview (same region of the plane). Wherever a brick is still missing, the **preview fills the hole**: what you see is always a complete image, getting sharper and sharper. The render size is roughly **one pixel per voxel of the longest axis**, extended to the slice frame (× 1.5): from 512 to 16,384 px.

🔢 **WORKED EXAMPLE**

Demo dataset, 768 voxels along X: 768 × 1.5 = **1,152 px** of render. Each pixel covers 1.5 × 921.6 µm (the largest physical extent) / 1,152 = **1.2 µm**, which is exactly the voxel size: this dataset’s native is as small as the preview.

## Two paths for the same plane




▲ Why an axis-aligned plane does not need an atlas.

- **“Plane” path**: for a plane aligned with an axis (XY, XZ, YZ) of an undeformed volume. For each pixel the shader reads the voxel `floor(uvw × dim)`; along the normal, the coordinate does not depend on the pixel, so the voxel planes that are read are **known in advance**. A layered 2D texture holds only those. For a MIP slab, each box of bricks is reduced to its per-channel maximum in a *worker* (`studio-plane-worker.js`), then the maxima of the bricks in a column are merged.
- **“Atlas” path**: oblique cut, stabilised series, averaged slab. A disposable 3D atlas, sized **before** it is allocated against the video-memory budget (chapter 10); if it does not fit (`SVR_OVER_BUDGET`), the level just below is tried and the progress label says so.

Where do the bytes of an XY plane come from?

| SOURCE | WHEN | WHAT IT SAVES |
|---|---|---|
| `planes/` **(format 2 and later)** | XY slice, level 0 | the other 63 planes of each brick |
| `mips/` **(format 3 and later)** | MIP slab over the whole stack | one maximum per 64-plane layer instead of 64 planes |
| **bricks by byte ranges** | everything else | the bricks not crossed and the whole packs |

The formats are detailed in chapter 7 (§ 7.8 and 7.9); here the Studio only **picks the most economical one** that exists, and falls back to the bricks if the folder or a file is missing.

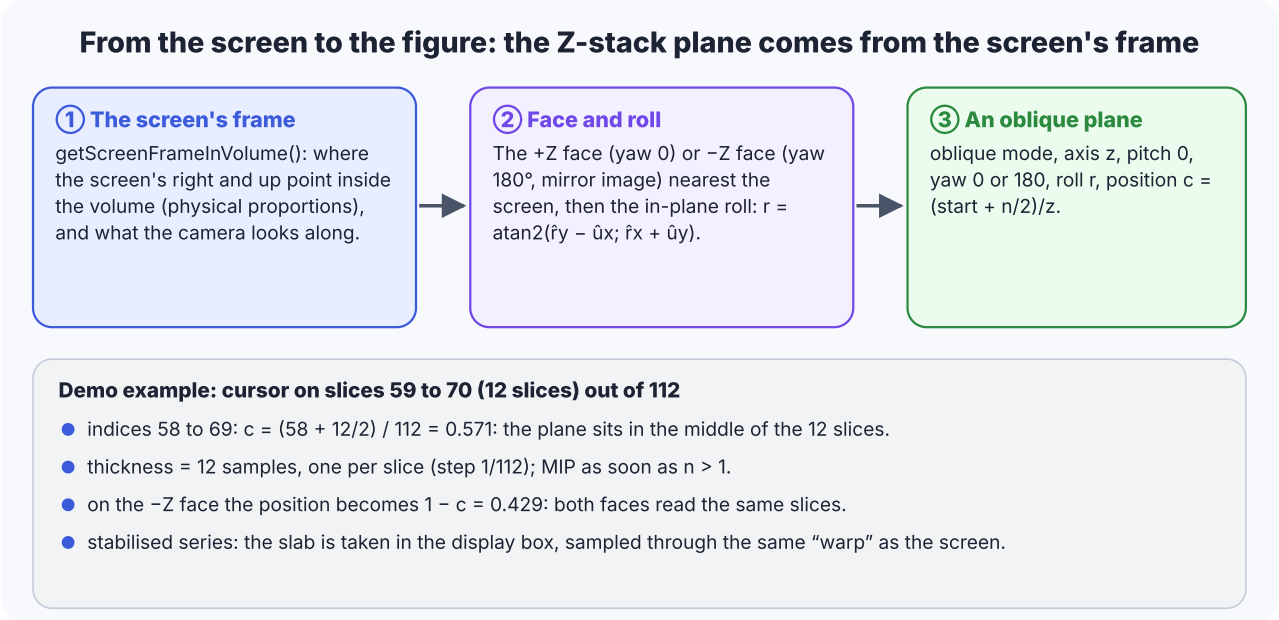
🔧 FOR THE CURIOUS

- **Empty brick ≠ missing brick.** A brick that the pipeline did not store (all its voxels are 0) is **zero**, never "pending": the atlas path points its entry at a single slot of zeros; the plane path marks its texels present from the start. Otherwise the preview would replace for ever every column that crosses an empty brick.
- **Progressive refinement**: at most every 0.5 s, as soon as 2% of the pass's bricks have arrived (or after 2 s regardless). A render is not restarted more often than a small multiple of the cost of the previous one.
- **Failure**: a chunk that fails twice keeps the preview's pixels and is counted ( `missingChunks` ). The final image is then labelled "partial native" and the Studio says so.
- **Channels switched off in the viewer**: when transport is per channel, they are not downloaded; switched on later in the Studio, they appear at preview resolution rather than black.
- **Independence**: the pass's brick batch has its own group and its own cancellation signal: a load started by the viewer does not cancel it, closing the Studio does.

⚠️ WARNING

**More than 256 MiB.** A thick Z-stack figure may ask for every brick of all its slices. Beyond an estimated 256 MiB, a dialogue offers three choices: load the native (with the number of MB), load the lower level (lighter), or keep the preview. The estimate goes through `planes/` and `mips/` when they exist: it can be far lower than that of the bricks, and the dialogue never offers "lighter" for more bytes.

## A Z-stack figure




▲ The plane of a Z-stack figure is born from the screen's frame of reference.

When you open the Studio from the Z-stack browser, the plane is not the oblique-slice one: it is built so that the figure **is** the screen. The viewer asks where the screen's right and up point in the volume ( `getScreenFrameInVolume` ), picks the nearest **+Z** or **−Z** face (hence a yaw of 0 or 180°), then the **roll** that locks the image to the screen. The cursor's `n` slices become a slab of `n` samples, one per slice; beyond one slice, it is a MIP.

### Exporting, importing, and the limits

| | RULE |
|---|---|
| **PNG** | figure size, black background, **caption always burned in** (dataset · plane · size · pixel). Refused beyond 16,384 px a side or $2^{28}$ pixels |
| **JSON** | layers, guides, channels, plane, calibration, Compare cells. Never a pixel |
| **Reading a JSON** | refused beyond 5 MB, 2,000 layers, 10,000 points; text limited to 2,000 characters |
| **History** | 80 steps |
| **Channels** | 4 at most (the texture is RGBA) |

**Importing a JSON** does not replace the open figure: it **applies** to it. The file is checked in full before anything changes (known layer types, numeric geometry, colours, unique identifiers).

- The open figure keeps its image, its frame, its calibration and its cells; it takes the file's **layers, guides and groups**, with the measurements re-read against the open figure's calibration (a scale bar keeps its value, a distance its points).
- It is **the same figure** if the file has the same size, names the same dataset (identifier or path), shows the same plane (within a tolerance) and the same frame of the series, and, for Compare, the same cells.
- In that case the file's **channel settings** also apply, and only if the slice can be recoloured without changing frame. Otherwise (another figure), only the **layers** go through, and a message says so.

📌 **REMEMBER**

- A document = layers + guides + channels + plane + calibration, **without pixels**.
- Recolouring starts from the **raw values**: no new render of the volume.
- The native pass chooses **plane** or **atlas**, reads `planes/` and `mips/` when they exist, and protects itself with a **token**.
- Rotating a layer **recomputes** pixel lengths: the measurement does not move.

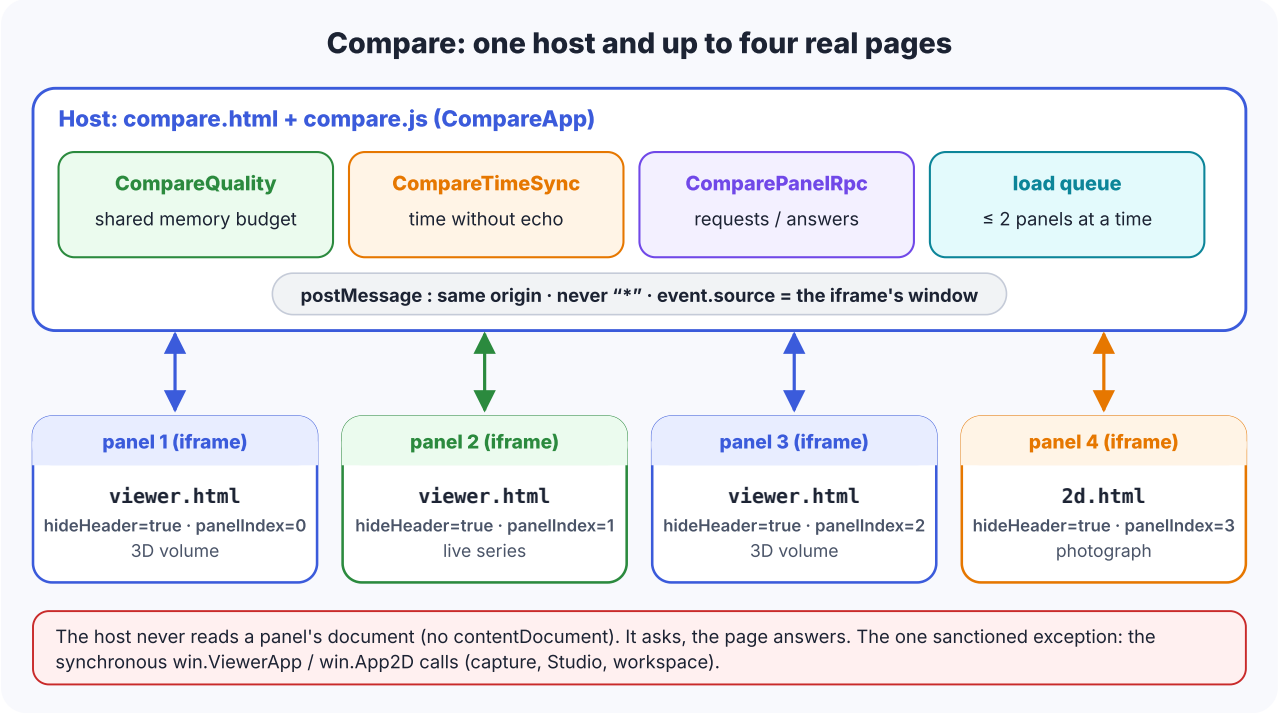
## 13.4 Compare under the bonnet

⏱ **IN 30 SECONDS**

- Compare is a **host** that drives up to four **real pages** ( `viewer.html` , `2d.html` ) locked in frames, by messages.
- Each page **describes** what it can do; the host draws the buttons. It never rummages in their document.
- A **common memory budget** of 1.5 GiB is shared between the panels by a small algorithm, and time is aligned by **fraction**.

Chapter 12 (§ 12.8) described the page. Here is the protocol that holds it together.

## The architecture



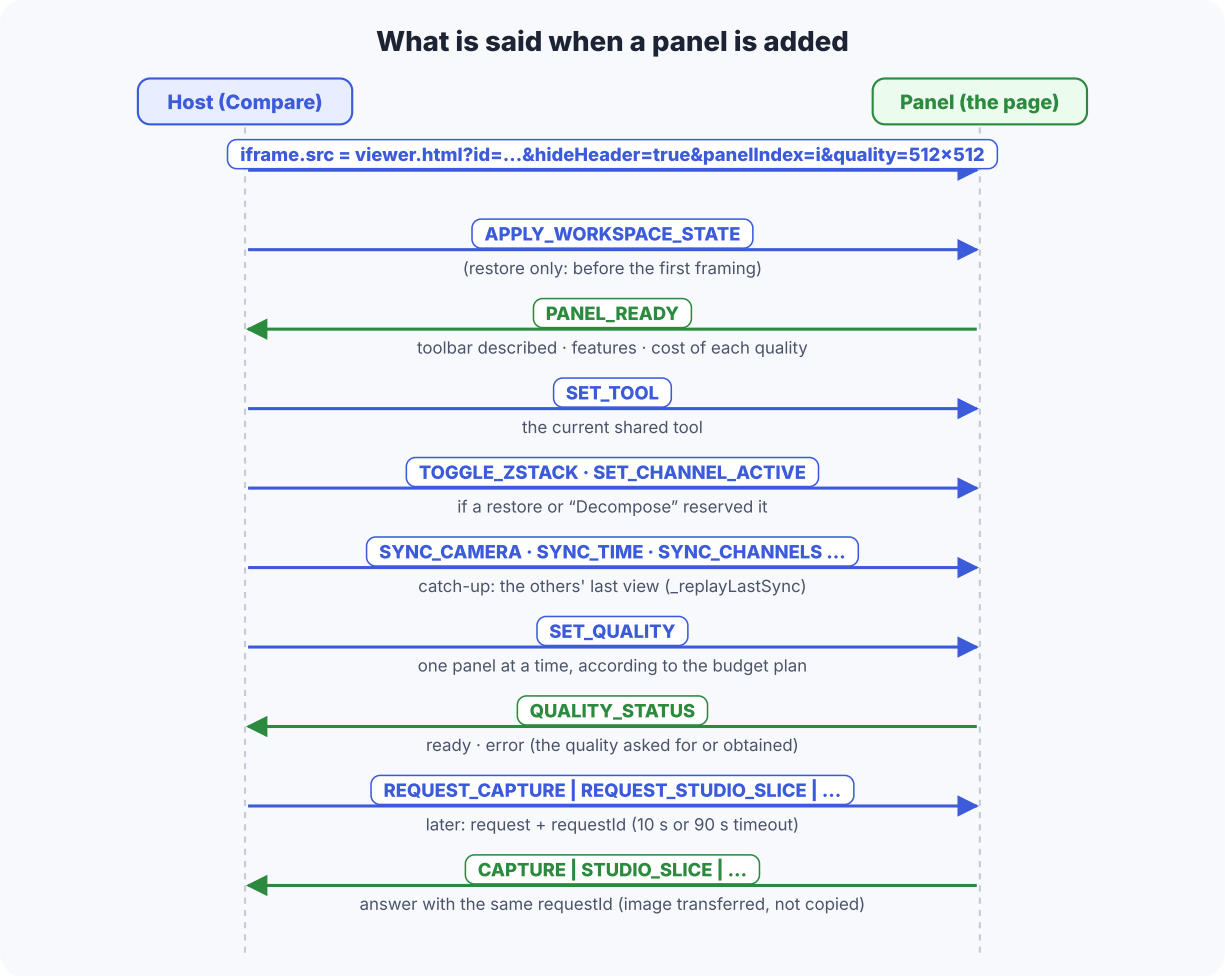

▲ One host, four frames, and a single channel: messages.

Each panel is a **complete page** opened without its header ( `hideHeader=true` ) with a number ( `panelIndex` ). Since it is chrome-less, its toolbar is invisible: the page therefore **describes** its toolbar to the host, which draws the buttons in the panel's header.

❓ **WHY THIS CHOICE?**

**Why whole pages and not a shared engine?** Because every tool, every shortcut, every setting already exists in the page. Compare has **nothing to reimplement**: a new tool becomes available in the panels as soon as it declares that it can be driven from outside ( `contexts: ["page", "panel"]` , chapter 15).

## A five-step handshake




▲ What is said when a panel is added.

1. The host gives the frame its address (with `quality=512x512` for a volume: we always start small).
2. As soon as the page is mounted, it sends `PANEL_READY`: name, type, **described toolbar** (tools and toggle buttons, with their state), features (volume, photograph, timeline, number of channels, tracking), quality, and **the cost of each quality** in bytes of graphics memory.
3. The host answers with the common state: the current tool, then the catch-up of the synchronisations (see below).
4. It sets the quality, **one panel at a time**.
5. Later, it sends **requests** (capture, slice for the Studio, state…) that the page answers.

A page that does not send `PANEL_READY` within **180 s** is declared failed; the panel shows a **Retry** button that reloads the frame from scratch. An error after set-up never removes the "ready" state: the volume already displayed is intact. At most **two** panels load at once; the others queue.

## The catalogue of messages

| DIRECTION | MESSAGES |
|---|---|
| **panel → host** | `PANEL_READY`, `PANEL_ERROR`, `PANEL_DATASET`, `PLUGIN_STATE`, `TOOL_CHANGED`, `QUALITY_STATUS`, `SIDEBAR_CLOSED`, `REQUEST_COMPARE_STUDIO` |

| DIRECTION | MESSAGES |
| --- | --- |
| **synchronisations (panel → host → others)** | `SYNC_CAMERA`, `SYNC_CHANNELS`, `SYNC_EXPOSURE`, `SYNC_Z`, `SYNC_TIME`, `SYNC_SLICER_SPEC`, `SYNC_ZSTACK_SLICE`, `WM_PHYSICAL_VIEW` |
| **host → panel** | `SET_TOOL`, `PLUGIN_ACTIVATE`, `SET_QUALITY`, `TOGGLE_SIDEBAR`, `TOGGLE_ZSTACK`, `SET_CHANNEL_ACTIVE`, `APPLY_WORKSPACE_STATE`, `PANEL_HELLO`, `WM_SET_PHYSICAL_VIEW` |
| **requests (host → panel)** | `REQUEST_CAPTURE`, `REQUEST_STUDIO_SLICE`, `REQUEST_WORKSPACE_STATE`, `REQUEST_CHANNEL_STATE` |
| **responses (panel → host)** | `CAPTURE`, `STUDIO_SLICE`, `WORKSPACE_STATE`, `CHANNEL_STATE` |

## Who may speak

- **Never a wildcard origin.** Messages go to the page's origin (`Utils.trustedTargetOrigin()`), never to `*`.
- **On receipt**, the host checks the origin, then that `event.source` really is **the window of the frame it mounted** at that number. A page nested inside a panel, carrying the same number, is ignored.
- A **response** is accepted only from the panel to which the request was sent, with the right type, **once**.

## Requests and responses

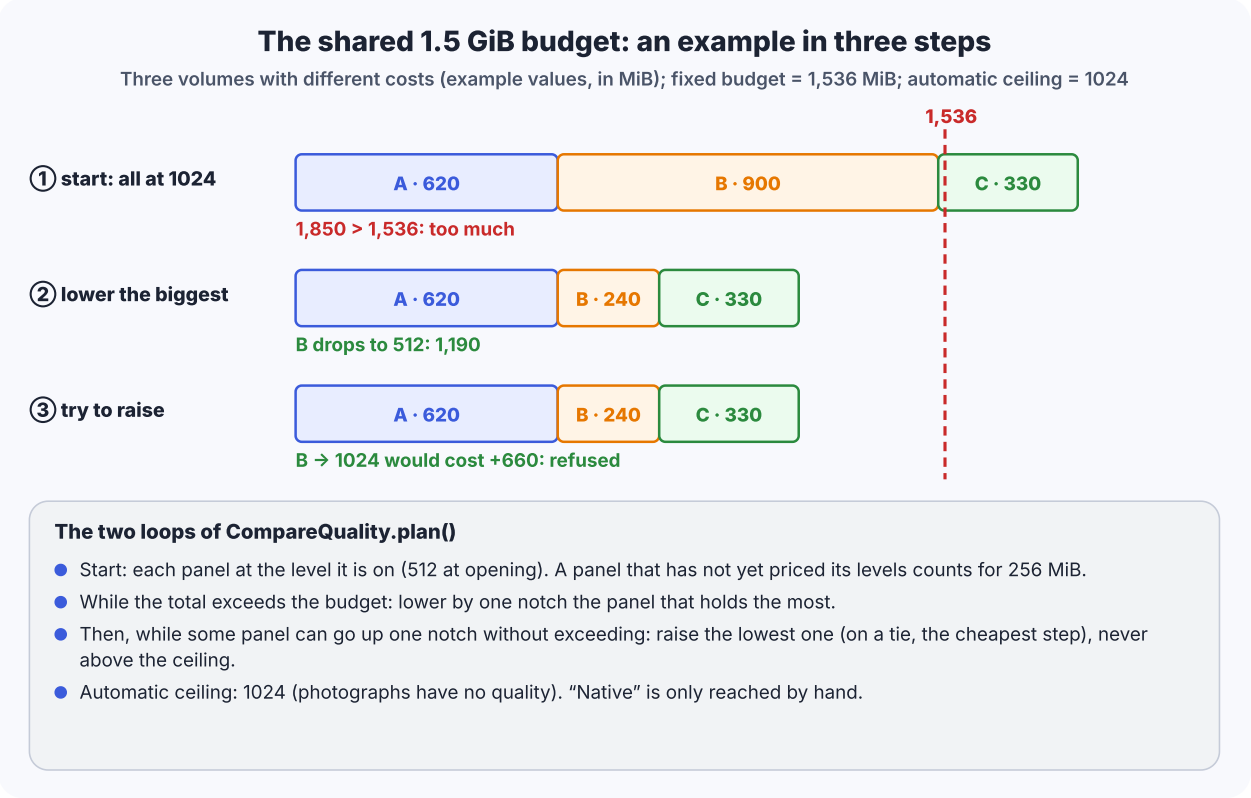
Each request carries a `requestId` (random prefix + counter). The response repeats it. If it does not arrive in time, the request fails with a precise code: `COMPARE_TIMEOUT`, `COMPARE_UNREACHABLE` (panel closed or unreachable), `COMPARE_PANEL_ERROR` (the page answered "impossible").

| REQUEST | TIMEOUT |
| --- | --- |
| **capture, workspace state, channel state** | 10 s |
| **slice for the Studio (may render a native slice)** | 90 s |

Images travel as **transferred** (not copied) `ImageBitmap`s: the panel renders its view and the photograph **in the same task** (its WebGL canvas does not keep its image), then transfers it. If the response arrives too late, its images are released instead of leaking.

## The common memory budget

The panels share **a single graphics processor**, but each believes it has all the memory to itself. "1024" may be worth a few hundred MB or more than a gigabyte depending on the dataset. Counting panels is therefore not enough: every volume **announces the price of each quality** (`qualityBytes`, atlas bytes), and the host shares out a **fixed budget of 1.5 GiB** (a constant in `compare.js`, not a measurement of your card).




▲ An example in three steps with three volumes (example values).

The algorithm (`CompareQuality.plan`) comes down to two loops:

1. **Lower**: as long as the total exceeds the budget, the panel that takes the most steps down one notch.
2. **Raise**: as long as a panel can go up one notch without exceeding it, the lowest goes up (on a tie, the cheapest step), without exceeding the ceiling.

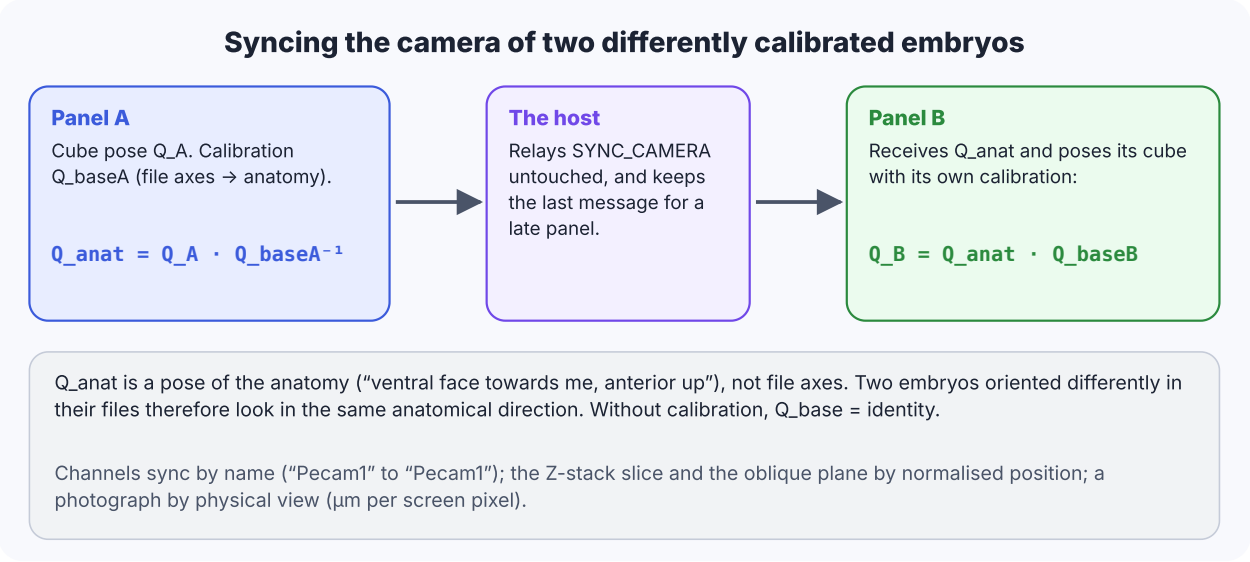
🔢 **WORKED EXAMPLE**

Three volumes at 1024 cost 620 + 900 + 330 = 1,850 MiB, against a budget of 1,536. The biggest (900) drops to 512 (240): total 1,190. Raising it again would cost +660: 1,850 > 1,536, refused. Result: A and C at 1024, B at 512.

- The **automatic ceiling** is 1024: “native” is only obtained by choosing it by hand. **Photographs have no quality** and do not count.
- A panel that has not yet priced its levels **reserves 256 MiB**.
- Raises happen **one panel at a time**: the next waits for the previous one’s `QUALITY_STATUS`. A panel that does not settle within **180 s** is abandoned (console message), and the level it did not reach is not requested again.
- If the card’s memory made a panel go **lower** than requested, the requested level also counts as failed: otherwise the host would request it endlessly and reload the volume on every pass.
- An older page that does not price its levels follows the old **counting** rule: 1024 up to two volumes, 512 beyond, and it only goes up.
- Choosing 512, 1024 or Native by hand forces that level on every volume.

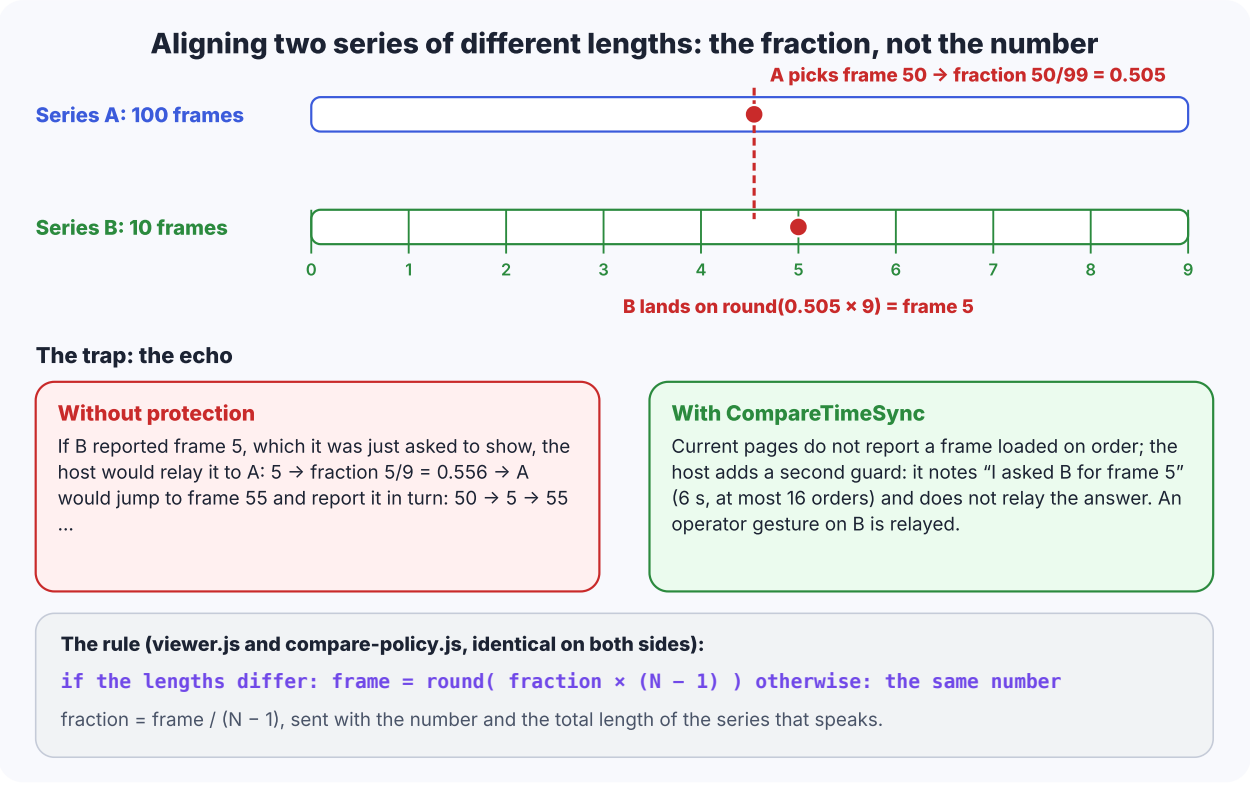
## Keeping the panels in the same place

**The camera.** Each dataset has its own `Q_base` calibration (§ 13.9). Two embryos oriented differently in their file therefore do not have the same cube pose for the same anatomical view. The panel that moves sends $Q_{anat} = Q_{cube} \cdot Q_{base}^{-1}$; the receiving panel sets its cube to $Q_{anat} \cdot Q_{base}$. While the Z-stack locks the top view, the received orientation is ignored but the zoom is kept.




▲ The camera is synchronised in anatomical pose, not in file pose.

**Time.** Two series of different lengths are aligned on the **fraction** of their duration.




▲ Why the fraction, and how the echo is avoided.

The rule is the same on both sides: if the lengths differ, `frame = round(fraction × (N − 1))`, otherwise the same number. It is backed by a guard: the host notes every frame it asked a panel to

show (6 s window, 16 orders at most). The panel's announcement of that frame is the **response** to the order, not an operator action: it is not relayed.

**Channels** are synchronised **by name** ("Pecam1" with "Pecam1"), exposure included. **The Z plane, the oblique plane and the Z-stack slab** travel as normalised positions. **A photograph** exchanges a **physical** view (§ 13.5); an uncalibrated photo neither sends nor receives one.

Each family can be set with a switch in the header (Z-Stack, Time, Camera / view, Channels).

### A panel that arrives late

The host **remembers the last message of each family** (camera, physical view, time, each channel, exposure, Z plane, oblique plane, Z-stack) and the name of the panel that produced it. When a new panel becomes ready, `_replayLastSync` replays these messages to it: it opens where the others are, inside the stack, with the same channels switched off. If a panel is closed, what it produced is **forgotten**: a panel added later would not open on a dataset that no longer exists.

A workspace restoration replays nothing: the panel's saved state takes priority.

### The grid figure

The export (§ 12.8) composes **the visible grid** on a canvas of 2× the screen size, capped at **4096 px** a side. For each panel the host sends `REQUEST_CAPTURE`, draws the received image in its cell, and adds the title and a signature. A panel that does not answer is replaced by a grey frame with its name, and the number of stand-in panels is counted. The format is PNG or WebP (quality 0.95); the background follows the theme.

**Compare's Studio** reuses the Studio of § 13.3: one cell per panel. In **physical scale**, every calibrated panel is drawn at **a single µm per pixel**, that of the coarsest, and is never enlarged: a scale bar is true for all of them.

### The workspace

The **Save** button stores, in the browser, the state of **every panel** plus that of the host:

`panels` · `panelTypes` · `layoutMode` · `layoutWeights` · `sync` · `tool` · `quality` · `panelZstackStates` · `panelSoloChannels` · `iframeStates`.

- The host **waits for every panel to be ready**; a failed panel is kept in the list (reopening the workspace retries it) with an empty state. Before that, the state is "not yet available" and the address is not updated.
- The **brick cache** is removed from a panel's state: it fills by itself during streaming and would change the address every second.
- On **restoration**, a panel's state is sent as soon as its document exists (the frame's `load` event), so that the viewer skips its first camera framing. An unknown quality is refused: it would never be acknowledged and would block the quality queue.
- The address (`#state=…`, § 13.7) is written only after an **operator change**: panels opened by the address itself (`?add=`) are the page's intact state, and the comparison point is taken again when they are all ready.

📌 REMEMBER

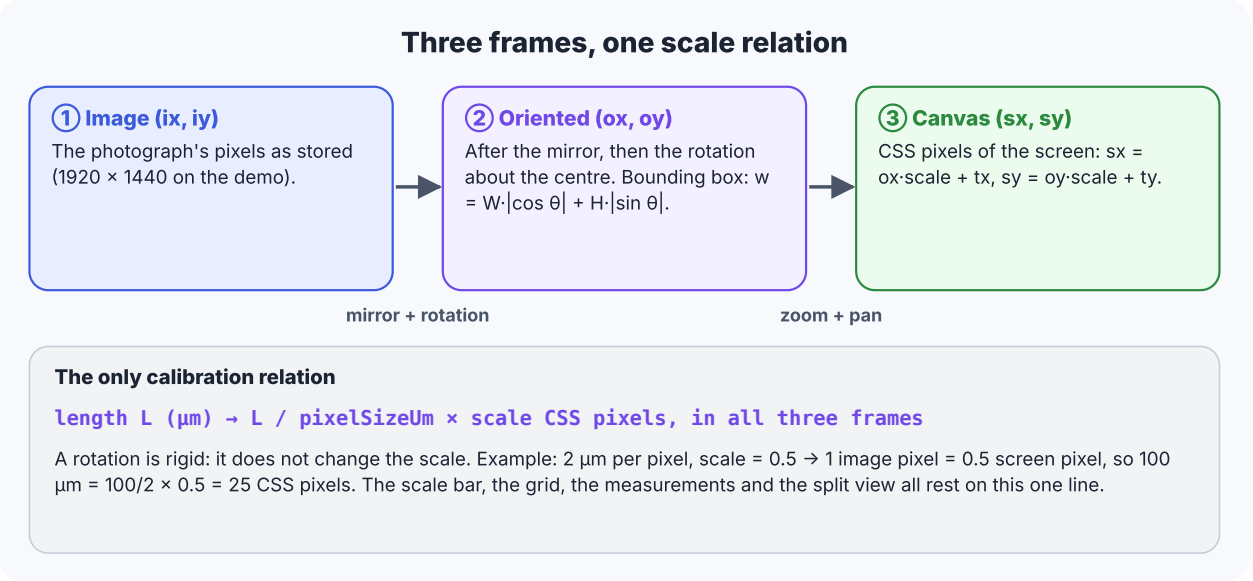
- The panels are the **real pages**; the host asks, the page answers.
- Explicit origin, `event.source` checked, requests with `requestId` and timeout.
- Quality: **fixed 1.5 GiB budget**, lower the biggest then raise the lowest.
- Time: **fraction**, with an echo guard. Camera: **anatomical** pose.

## 13.5 The 2D page in depth

⏱ IN 30 SECONDS

- The engine is a **canvas** that you pan and zoom: three frames of reference, one scale relation.
- Display settings and stain isolation run in a **worker**, on a copy; the measured photograph is never modified.
- The importer reads the **pixel size** from the TIFF tags and the **file name** for the stage.

### Three frames of reference



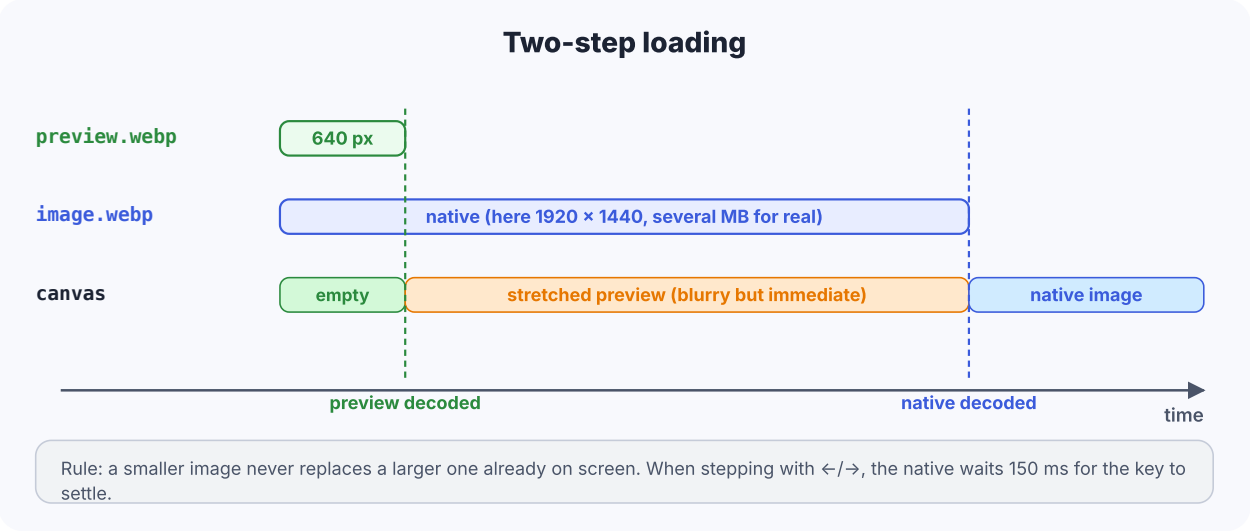

▲ Image, oriented, canvas: and the single calibration relation.

The `Viewer2D` engine does not use Three.js. It draws a 2D canvas with the transformation `sx = ox·scale + tx`. The mirror is applied **before** the rotation, around the image centre; the bounding box of the rotated image is used for framing. The angle is normalised to [−180; 180), and a “fitted” view stays fitted when the angle changes.

🔧 FOR THE CURIOUS

- `scale` is the number of **CSS pixels per image pixel**. It is at most **32** and at least **0.25 × the fit**. The fit leaves a 2% margin on each side (factor 0.96).
- The wheel zooms around the cursor by a factor of 1.1 per 100 units; a single event never zooms by more than 3 notches, and Firefox, which counts in "lines", is converted to pixels (1 line = 33 px).
- A gesture of less than 4 pixels is a click. Two fingers make a zoom and a pan. A double-click fits. **Native resolution 1:1** sets `scale = 1 / devicePixelRatio` : one image pixel per physical screen pixel.
- Measurement keeps the same contract as the volume's: `{ normalized, physicalUm, screen }` , with `physicalUm = pixel × pixel size` .

## Loading in two steps

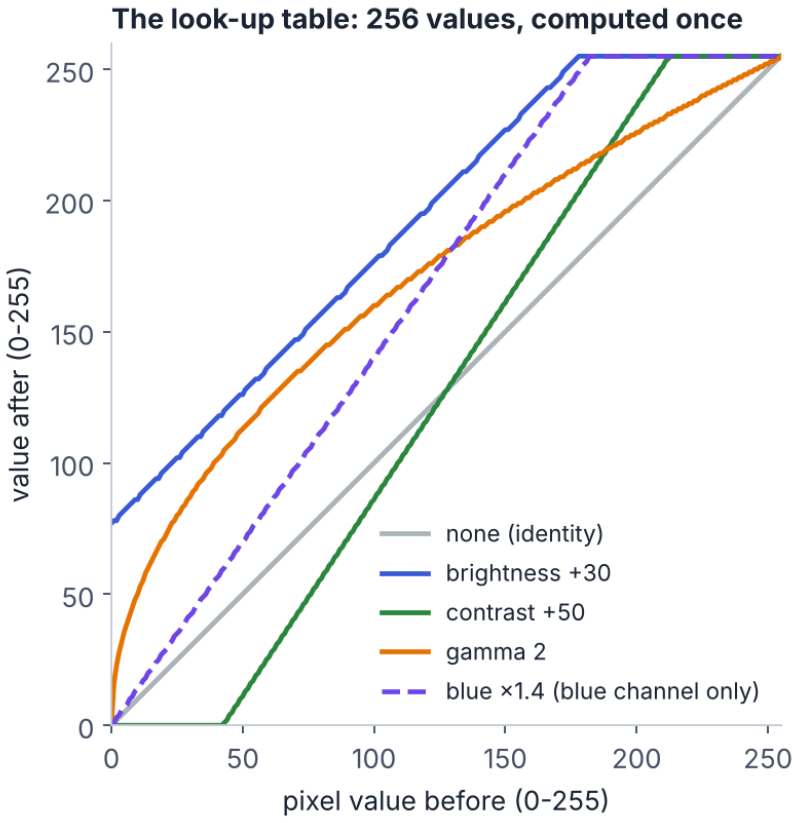



▲ The preview first, the native afterwards.

Both files are requested **together**; the first one decoded is drawn. A smaller image never replaces a larger one already on screen. When stepping with ←/→, the native request waits 150 ms for the key to settle: holding the arrow only downloads the photograph you stop on. A capture taken before the native one carries the enlarged preview: the Studio is warned ( `quality: 'preview'` ).

The page's loading badge distinguishes *preview*, *native*, and *native error* (the preview stays displayed).

## Display settings and isolation




▲ A channel's lookup table: 256 values computed once.

For each channel, the setting is a **table of 256 values**: `x = (v/255 · balance − ½)(1 + contrast/100) + ½ + brightness/100`, clamped to [0; 1], then `x^(1/gamma)`. Balance only applies to red and blue. Moving a slider therefore only recomputes the table, and the loop over the pixels is a simple lookup.

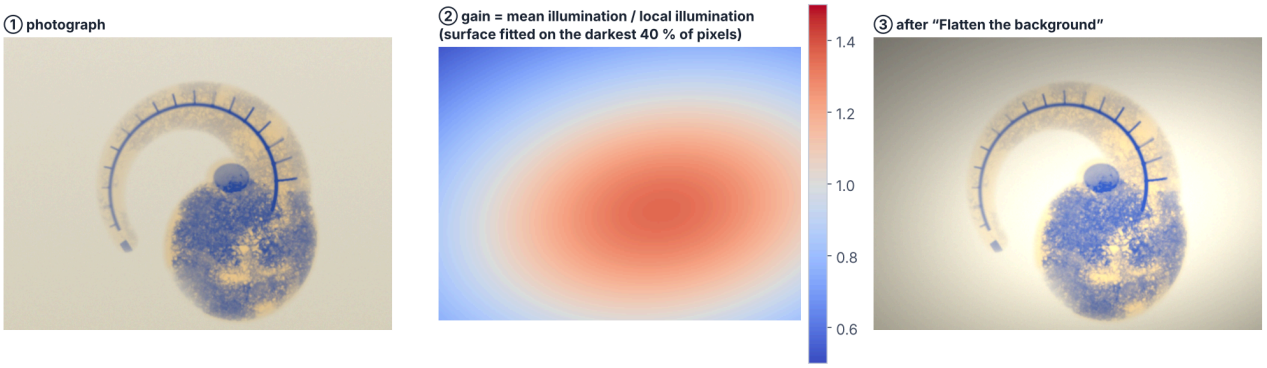
| SETTING | RANGE |
|---|---|
| **Brightness, Contrast** | −100 to 100 |
| **Gamma** | 0.1 to 10 |
| **Red, blue balance** | 0.1 to 4 |

A value coming from a slider, a workspace file or a link is **sanitised**: unknown, ignored; out of range, brought back into range; non-numeric, refused. A booby-trapped value therefore cannot turn the photograph completely black.

> ⚠ **WARNING**
>
> **Stain isolation takes priority over the settings**: as long as it is on, the view shows the isolation result alone. The two do not stack.

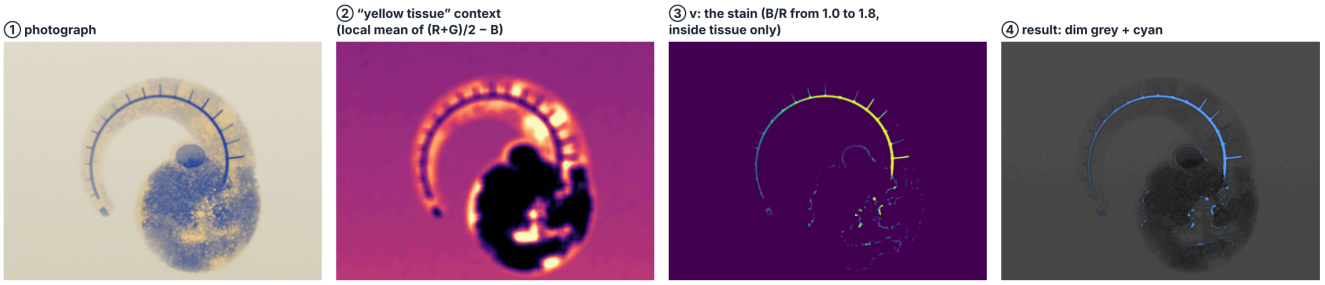
**Flattening the background.** A pixel's gain is "mean illumination / local illumination". The illumination is modelled by a **quadratic surface** $c_0 + c_1x + c_2y + c_3x^2 + c_4y^2 + c_5xy$ fitted by least squares on the map reduced to one eighth, keeping only the **darkest 40% of cells** (the matt background): a smooth

surface cannot follow the embryo, which therefore keeps its own contrast. The gain is bounded between 0.5 and 3; a singular system gives a flat surface.




▲ A photograph’s gain map and the resulting correction.

**Isolating the stain.** The computation comes down to four maps. The “yellow tissue” context is a local mean of `(R+G)/2 − B` at a quarter of the resolution (an 11 × 11 cell box, about 40 pixels, computed with a summed-area table, hence constant time per pixel). The stain `v` is `(B/(R+1) − 1.0) / 0.8` clamped to [0; 1], **only** if the context exceeds 5. The final pixel is a grey at 35% of its brightness, mixed with cyan `(90; 160; 255)` in proportion to `v`.




▲ The four maps of the computation, on the demo photograph.

🔧 FOR THE CURIOUS

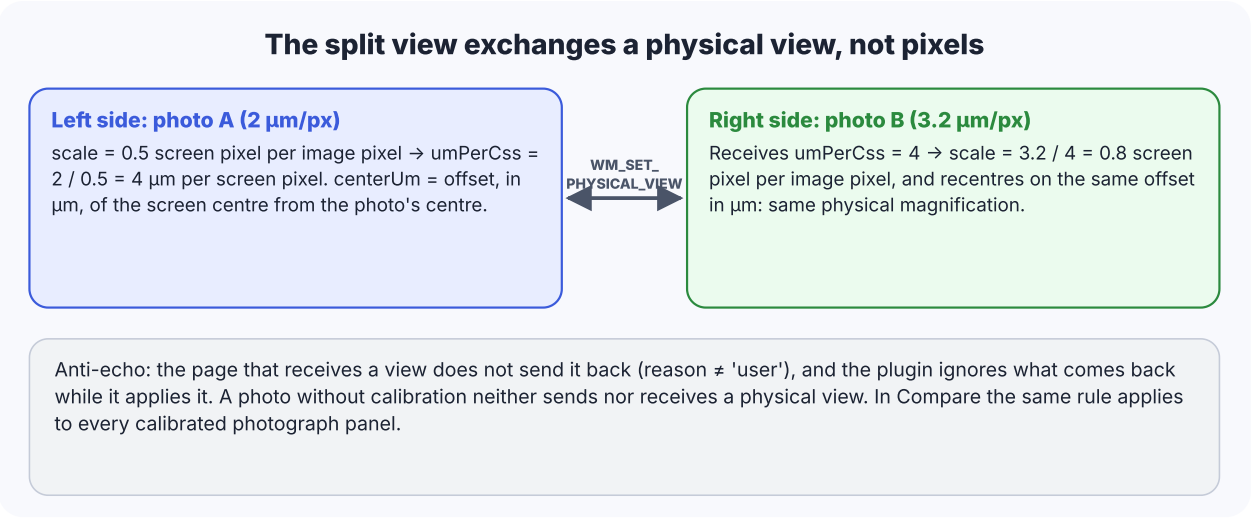
- **Why this threshold of 5?** A large stained trunk pulls the local mean to 6–9; the matt background never exceeds 2 to 6. The threshold is deliberately low: a stain is blue **within** a yellow tissue.
- **The worker.** The page transfers an `ImageBitmap` (no copy), the worker draws on an `OffscreenCanvas`, applies the operations of `pixel-ops-2d.js` and returns a bitmap. The illumination and context maps depend only on the image: they are **kept** as long as the image does not change. If requests pile up (a slider being dragged), **only the most recent** is run.
- Without Worker or `OffscreenCanvas`, or if the worker crashes, the same code runs on the main thread, one job at a time. A capture takes an **exact** version for the current settings, never a stale one.
- These two operations are **display only**: measurements and the grid read the intact photograph. The Studio, for its part, receives what is on screen (isolated or adjusted, mirror and rotation included), at native resolution, calibrated in µm/pixel.

## Orientation, grid, split view

- **Orientation.** The plugin drives `setOrientation({ rotationDeg, flipH })` and draws the compass (A/P); since the rotation is rigid, the scale does not change. The administrator saves the pose with

the dataset (`orientation2d`), through the same messages as the 3D orientation (§ 13.9).

- **Calibrated grid.** A **1-2-5** step nearest to 120 px (coarse) or 60 px (fine), anchored on the corner of the oriented image; the labels (distance from the corner) only appear if the step exceeds 44 px. Without calibration, it says so instead of drawing nothing.
- **Scale bar.** `length = nearest 1-2-5 (120 px / px per µm)`; it only exists if the photograph is calibrated.

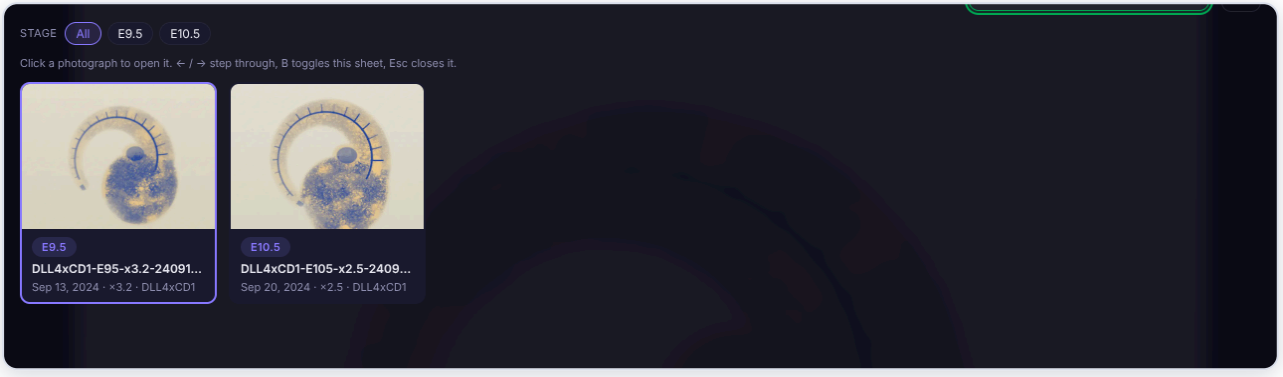



▲ The split view exchanges a physical view, not pixels.

The **physical view** is the pair `umPerCss = pixel size / scale` and `centerUm` (the offset of the screen centre from the photo centre, in µm). Two photos at 2.0 and 3.2 µm/px are thus at the same **real** magnification, and moving one moves the other by the same real distance.

## The collection

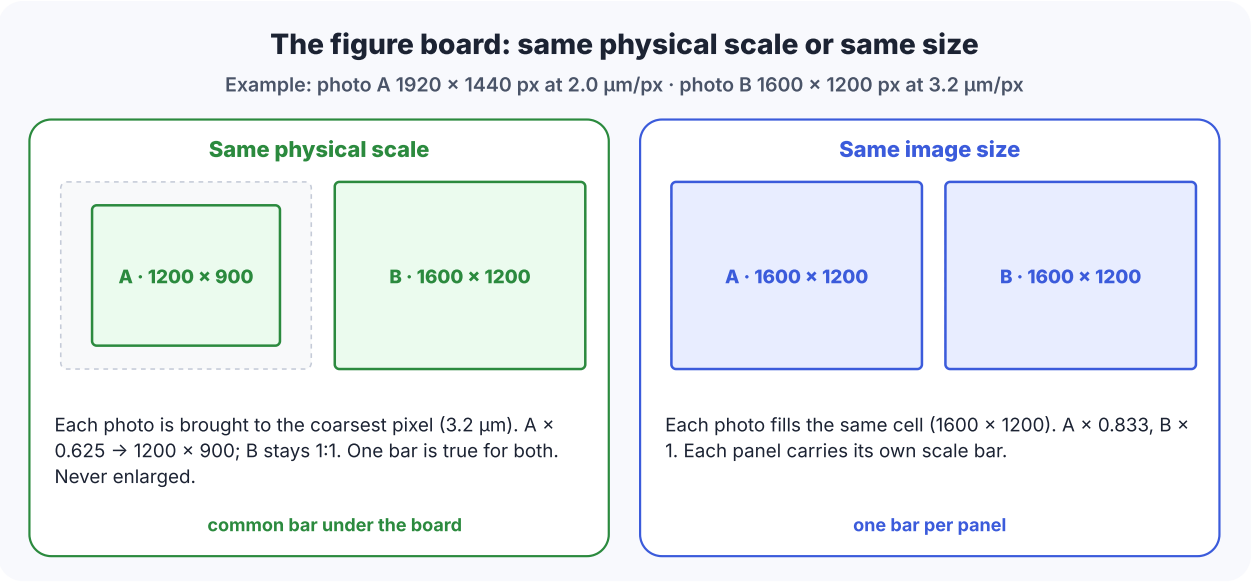
The contact sheet (B key) is sorted by stage, then dissection date, then name; it can be filtered by stage, by line and by text (name, stage, line, staining, description, date).




▲ The collection's contact sheet (two demo photographs).

- Once the photograph is on screen and the page is idle (400 ms), the **two neighbours** are preloaded (preview and native). Neither a Compare panel, nor the split view, nor the administration preview does so: it would multiply the downloads.
- Moving from one photo to another updates the address with the browser history (`pushState`); the Back button returns to the previous photograph without reloading the page.
- A photograph's **measurements** are kept in memory by identifier: returning to a photo restores them.

## The figure sheet

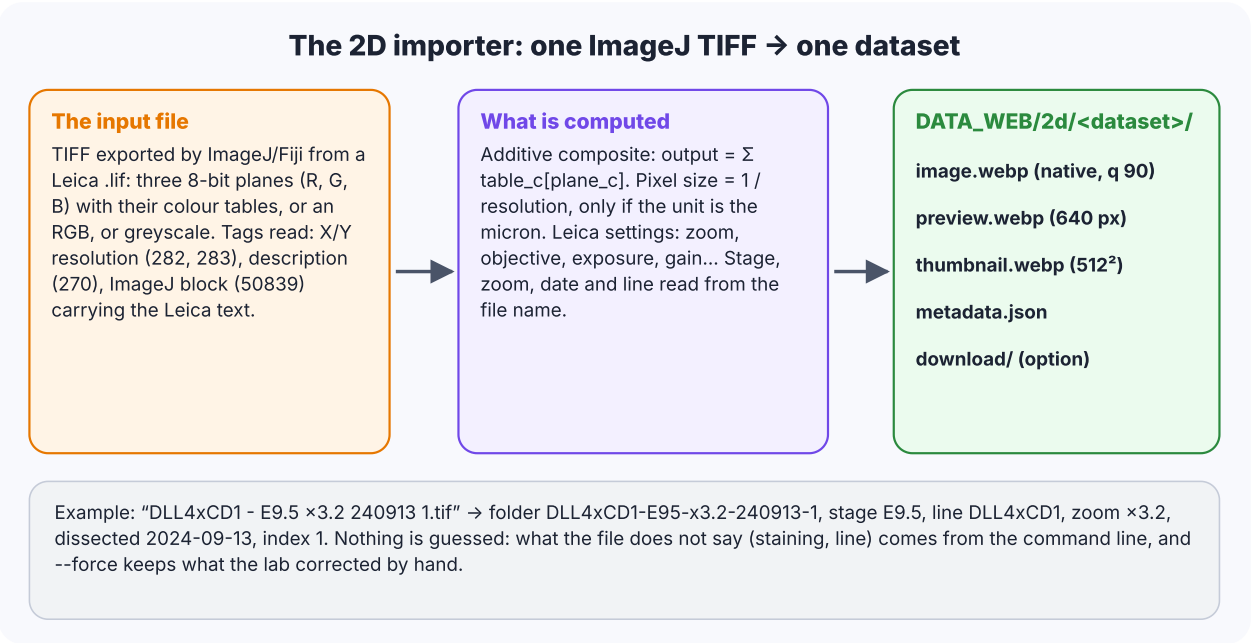



▲ Same physical scale, or same size: two ways of composing.

In “same physical scale” mode, each photograph is brought to the selection’s **coarsest pixel** ( `factor = px_i / px_max` , hence never an enlargement) and one common bar holds for all. In “same size” mode, each photograph fills the same cell, with its own bar. An **uncalibrated** photo does not take part in the common scale: it is fitted into the cell and marked as such.

The Studio receives **one calibration rectangle per panel**: a bar placed on a panel reads that panel’s µm/pixel, and follows it if moved to another. Limits: 6,000 px wide, 16,000 px a side, 120 megapixels (the figure is reduced to fit), 48 megapixels of decoded photographs kept between renders, 3 decodes in parallel.

## The 2D importer




▲ From the ImageJ export to the data folder.

`preprocess/2d_importer.py` (version 0.18.0) turns **one TIFF into one dataset**.

```
python 2d_importer.py --input <folder|file.tif> --output DATA_WEB \
    [--line DLL4xCD1] [--staining X-gal] [--only "*E8.0*"] \
    [--with-downloads] [--force] [--lossless]
```

| OPTION | EFFECT |
|---|---|
| `--line`, `--staining` | what the file does not say (line, staining) |
| `--only` | a name filter (“*E8.0*”) |
| `--with-downloads` | puts the original TIFF and a `README.txt` in `download/` |
| `--force` | re-imports while **keeping** what the laboratory has corrected |
| `--lossless` | lossless WebP (larger) |

**Reading the planes.** An ImageJ hyperstack stores its pages “channel first”: the first `channels=` pages form **one** composite; the following ones are other images, which are not added together. The composite is additive, exactly what ImageJ displays: `output = Σ table_c[plane_c]`, each 8-bit plane going through its 768-byte **colour table** (3 ramps of 256). Without a table, three planes are read as R, G, B and one plane as a grey. A 16-bit plane is brought down to 8 by its maximum.

**The pixel size.** ImageJ writes the resolution (tags 282 and 283) in **pixels per unit**. The importer only believes it if the description’s unit is the **micron**; a pixel’s size is then `1 / resolution`, in x and y (square if only the x resolution exists). Otherwise: `calibrationStatus: unknown`, no scale bar, no measurement, and the text says so.

**The Leica block.** ImageJ embeds the LAS X text. For the series that is read, the importer keeps the zoom, magnification, objective, numerical aperture, exposure (converted to ms), gain, microscope and camera. The text repeats each key per work block; the top-level line (the one for the exposure actually taken) wins, and padding “0”s never take precedence over a real value.

**The file name.** `<lif> - <stage> x<zoom> <date yymmdd> [<n> [<m>]]`. The stage goes through **the same parser as the volumes** (`E8-5`, `E8.5`, `E85` all mean E8.5), the date must be a real day, the line is read from the `.lif` name (`DLL4xCD1`). Nothing is invented: a missing part stays empty.

🔢 **WORKED EXAMPLE**

“DLL4xCD1 - E9.5 ×3.2 240913 1.tif” gives the folder `DLL4xCD1-E95-x3.2-240913-1`, stage E9.5, zoom ×3.2, dissected on 2024-09-13, index 1. Folder: `<line>-<stage without dot>-x<zoom>-<yymmdd>-<index>`.

**The files written.** `image.webp` (native, quality 90, method 6, or lossless), `preview.webp` (long side 640 px, LANCZOS, quality 80), `thumbnail.webp` (512 px square on a `#080a12` background), `metadata.json` (written **last**, atomically: a half-finished import is never mounted). Maximum side: 16,383 px (WebP’s limit).

❓ WHY THIS CHOICE?

**Why `--lossless` exists.** Lossy WebP subsamples colour (4:2:0), which disturbs the per-pixel B/R ratio that stain isolation reads, especially at sharp edges. For a stain analysis, keep the photo exact.

Two TIFFs that describe themselves in the same way (same line, stage, zoom, date, index) produce the same folder name: the second is **refused** with a message, rather than overwriting the first or being taken for already imported. An existing folder is not touched without `--force`.

📌 REMEMBER

- Three frames of reference (image, oriented, canvas) and **one** relation: `L / pixelSizeUm × scale`.
- Settings and isolation: a **256-value table** or a computation in a **worker**, on a copy.
- Isolation takes priority over the settings; nothing measured reads them.
- The importer guesses nothing: pixel size from the TIFF tags, stage from the name, the rest from the command line.

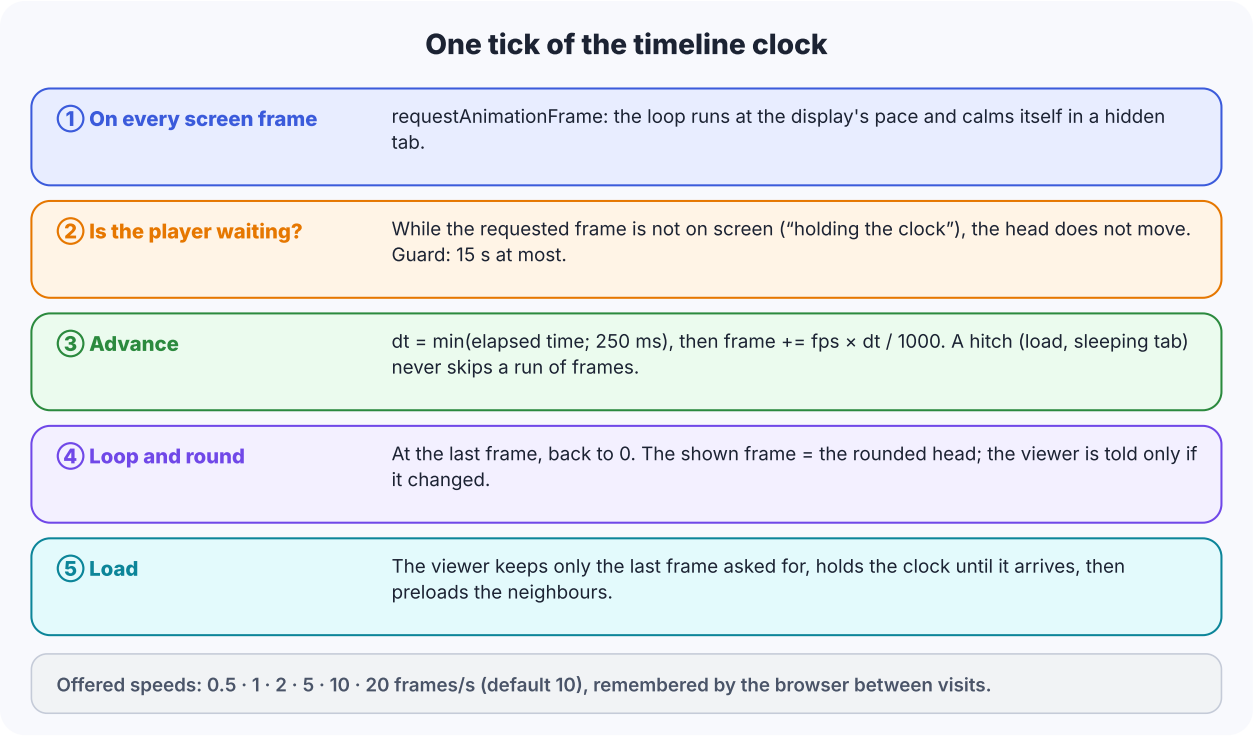
## 13.6 The timeline and live series

⏱ IN 30 SECONDS

- The timeline is a **self-contained component** ( `Timeline` ): a clock, a track, a buffer. The viewer tells it when to wait.
- The clock never **skips** a run of frames, and never **runs ahead of** a frame that has not arrived.
- Real time is not displayed, but the interval between frames is used for speeds in µm/min.

Chapter 10 (§ 10.7) explained how a series is loaded (buffer, preloading, last frame only). Here is the clock itself.

## One tick of the clock




▲ The five stages of a playback step.

🔢 **WORKED EXAMPLE**

At 10 frames per second, on a 60 Hz screen, each step lasts about 16.7 ms: the head advances by `10 × 0.0167 ≈ 0.17` frame. It takes **six steps** to change the displayed frame. A tab asleep for one second would not advance the head by 10 frames: the elapsed time is capped at **250 ms**, i.e. 2.5 frames at most.

| CONSTANT | VALUE |
|---|---|
| **Offered speeds** | 0.5 · 1 · 2 · 5 · 10 · 20 frames/s |
| **Default speed** | 10 frames/s |
| **Maximum step** | 250 ms |
| **Frame-wait guard** | 15 s |
| **Browser key** | `iribhm.viewer.playbackFps` |

A click on the speed moves to the **first speed strictly higher** than the current one, and wraps at the top. A remembered speed that is not in the list stays usable: it simply moves on to the next one.

## Waiting without blocking

A native frame may take 600 ms to stream. Without precaution, the clock would advance six frames in that time, and the loader would be sent chasing a frame nobody will see. The viewer therefore **holds the clock back** ( `setStalled` ) while it loads the frame it was asked for: the head no longer moves, but the loop keeps running.

❓ WHY THIS CHOICE?

**Why the 15 s guard?** A load may never finish (a background tab freezes the display, and the loader waits for a frame to draw). Without a cap, the timeline would stay frozen for ever. The viewer does not count loads in progress either: it releases the clock at **every** end of load, which at worst may free it a little early, never block it.

The track is drawn at `frame / (N − 1)`. When you grab it, playback **pauses** and the frame is the one under the cursor; releasing rounds to the whole frame. The Play button reacts to the press (not the release), and stays usable from the keyboard.

The counter text is `current / last`, on three digits: “000 / 003” means frame 0 of a 4-frame series. The frames **already in memory** are drawn as bands under the track: a series fills up around the head and loses frames on the way, so a simple width would not suffice. Frame `i` occupies the interval `[i − ½; i + ½]` so that the head stays inside the band that represents it.

### Real time and normalisation

- `timeline.intervalMinutes` and `timeline.timestamps` are in `metadata.json`. **The timestamps are not displayed**, but the interval lets the inspector give a cell’s speed in **µm/min** as well as µm/frame.
- The pipeline also records `intensityNormalization` (the signal levels of each channel and each frame) so that photobleaching can be **measured**. **No viewer setting uses it today**: a series that fades really does display darker.
- The component can also smooth (fractional positions) and display speed sliders; the viewer does not use them.

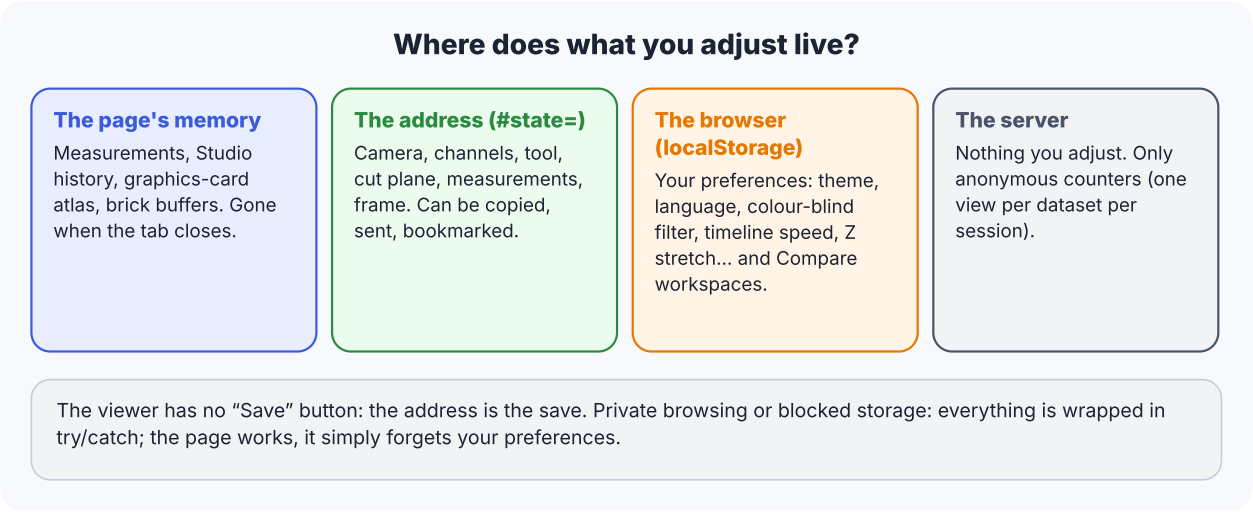
ℹ️ GOOD TO KNOW

Two series of different lengths in Compare are aligned by **fraction** of duration (§ 13.4): “half of the series” is half of each.

## 13.7 What the platform remembers: link, workspace, browser

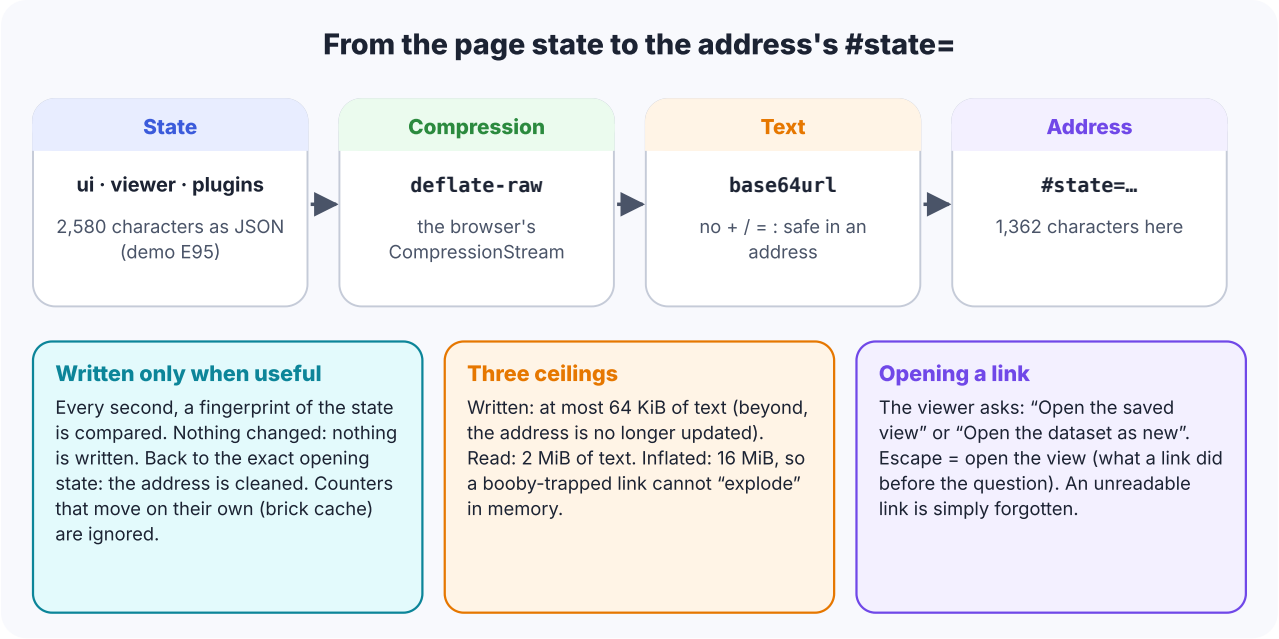
⏱ IN 30 SECONDS

- Four places: the **page’s memory**, the **address**, the **browser**, the server (which remembers none of your settings).
- The address carries the whole state of a view, compressed: it **serves as a backup**.
- Every browser access is protected: with storage blocked or private browsing, the page works and forgets.




▲ Where what you set lives.

## The `#state=` link




▲ From the page state to the address.

Every second, the page asks `_getWorkspaceState()` for its state. It gathers three blocks: `ui` (active tool, sidebar, background), `viewer` (camera, cutting plane, measurements, channels, exposure, grid, Z stretch, Z-stack, series frame, quality, calibration...) and `plugins` (the state each tool exposes through `getState()`).

🔢 **WORKED EXAMPLE**

The viewer state of the demo E95 dataset, at rest: **2,580 characters** of JSON, which become **1,362 characters** once compressed (deflate) and written in base64url. The camera fits on one line: `{"kind":"volume","cameraZ":1.13,"quaternion":[0,0,0,1],"position":[0,0,0],"zDisplayScale":1}`.

The state is **encoded** only if it has changed, according to a **fingerprint** (the JSON without the counters that move by themselves, like the brick cache). The dataset opened as-is is the reference: as long as

nothing has changed, the address stays clean; returning exactly to the opening state cleans it. The **Reset the workspace** button re-adopts the opening state as the new reference, after letting the end of a reload settle (two identical reads in a row, or a delay).

🔧 FOR THE CURIOUS

- **Writing**: at most 64 KiB of text, otherwise the address is no longer updated (an old `#state=` would be more wrong than none) and the console invites you to use "Save state".
- **Reading**: 2 MiB of text at most; decompression is read in chunks and **stopped at 16 MiB** (a booby-trapped link can inflate by a factor of ~1,000).
- **Generation**: any action that invalidates the address (clearing, new reference) increments a counter; a compression that finishes afterwards **discards** its result instead of resurrecting the cleared state.
- No network call: the link contains the state, the server never sees it.

When a link is opened, a dialogue asks: **Open the saved view** or **Open the dataset as new**. Escape and the backdrop mean "open the view" (what a link did before the question). Unreadable content is cleared without asking. In a Compare panel there is no question: the host decides.

## Browser workspaces

`WorkspaceState` stores a workspace under the key `iribhm.workspace.<scope>.<dataset identifier>` (scope: `viewer`, `compare`...), version 2, with the date. It is normalised into four blocks (`ui`, `viewer`, `tracking`, `compare`). Writing may **fail** (quota, blocked storage): the error is deliberately handed back to the caller, so that a workspace that was not saved is never announced as saved. Keys from before the renaming of the types (`fixed`, `wholemount`) are migrated once, in passing.

The viewer has **no** Save / Restore buttons: the address does that. They exist in the **Compare** page (Download Center) and in the Studio (JSON).

## What your browser keeps

| KEY | CONTENT |
|---|---|
| `iribhm-theme`, `iribhm-lang`, `iribhm-colorblind` | theme, language, colour-blind filter (shared between pages and frames by the `storage` event) |
| `iribhm.viewer.playbackFps` | timeline speed |
| `iribhm.viewer.zScale.<dataset>` | Z stretch of that dataset |
| `iribhm.workspace.<scope>.<dataset>` | a saved workspace |
| `iribhm-gallery-dock` | gallery dock state (open, wide, chip) |
| `lumen3d.vramBudgetMB` | imposed video-memory budget (operator) |
| `lumen3d.gpuContextLosses` | the last 8 graphics-context losses (one week) |
| `sessionStorage` `lumen_view_<dataset>` | a single view counted per session |

ℹ️ GOOD TO KNOW

**Measurements** are not in this list: they live in the page's memory ( `MeasurementStore` , a `Map` per scope and per dataset) and travel **in the link**. The administration has its own keys ( `adm-theme` , `adm-upload-dock-size` , `lumen-dupd-*` ), described in chapter 14.

The graphics budget is worth knowing about (§ 10.4): it is **halved for each graphics-context loss** of the past week (floor 256 MiB), on that browser profile. Clearing the site data resets it.

## 13.8 Measurement and pose algorithms

⏱ IN 30 SECONDS

- A measurement click reads the **depth on the GPU** in two one-pixel passes, with 16-bit precision.
- Laying a stack flat is **two rotations about the screen axes**: the stack never tumbles.
- Exporting a large image **starts over** if the volume or the settings change while it renders.

### The depth under the cursor

Chapter 12 (§ 12.2) described the logic: the ray's maximum, then the first crossing of 55%. Here is the mechanism.

**Reading the depth under the cursor: two one-pixel passes**

**Pass 0: x and y**
The shader renders ONE pixel (the camera window recentred on your click) and writes into its 4 RGBA bytes: x on 16 bits (high byte, low byte), y on 16 bits.

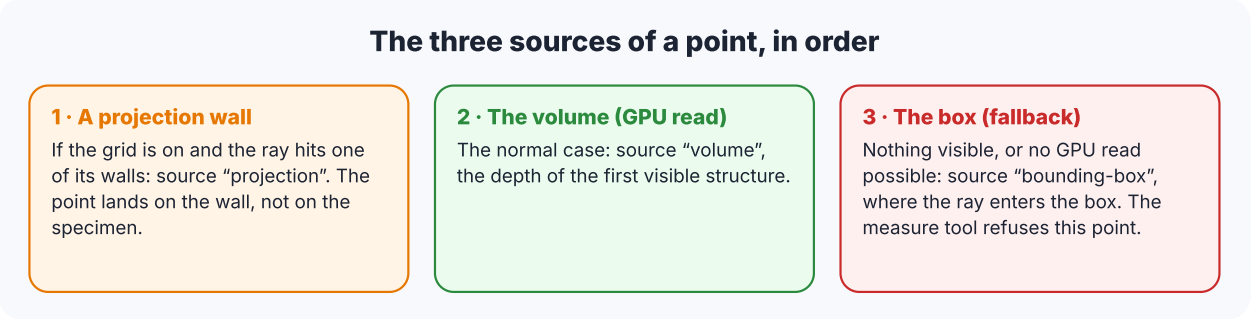
**Pass 1: z and the flag**
Same ray: z on 16 bits in R and G, 255 in B and A. An alpha of 0 means "nothing found": the click is refused (no structure under the cursor).

**Why two bytes: a single byte gives only 256 positions**

A position q ∈ [0; 1] becomes the integer N = round(q × 65,535), written as high byte = ⌊N / 256⌋ and low byte = N − 256 × high byte. Example: q = 0.7 → N = 45,875 = 179 × 256 + 51. The precision is therefore 1/65,535 of the box, i.e. 0.01 µm on a 600 µm box.

Along the ray: one sample every ½ voxel, at most 4,096 steps. Step 1: the maximum m of the displayed values. Step 2: the first point where the value reaches max(0.02; 0.55 × m), interpolated between the two neighbouring samples. Same window, channels, clipping, Z-stack slab and stabilisation as the screen.

▲ The two passes of a pixel and the 16-bit encoding.



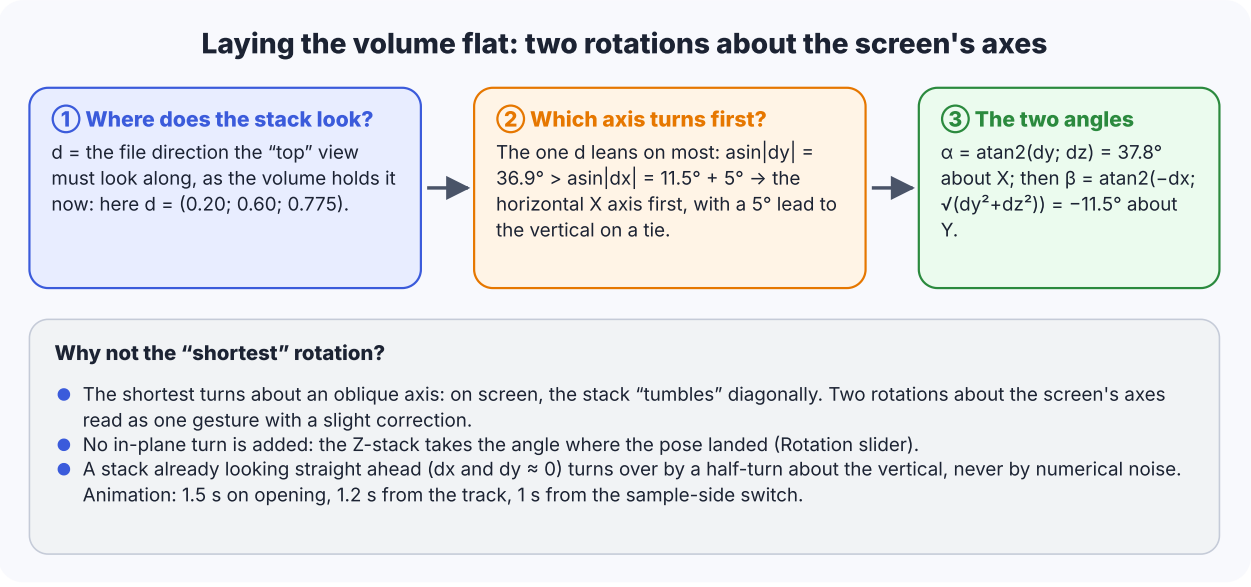

▲ The three sources of a point, in the order in which they are tried.

🔧 FOR THE CURIOUS

- The render of a pixel uses **the same uniforms** as the screen: same window, channels on, crop, Z-stack slab, stabilisation, bricks. It is framed by `camera.setViewOffset` on the clicked pixel.
- The constants are `PICK_SURFACE = 0.55`, `PICK_MIN_VALUE = 0.02`, `PICK_MAX_STEPS = 4096`; the step is half a voxel (`1 / (2·nu)`), lengthened if the ray would exceed 4,096 steps.
- Under stabilisation, the encoded position is that of the **display box** (`clipBoxMin + c · clipBoxSize`); without stabilisation, that of the unit box. The decoder handles both.
- A graphics-context loss, or the absence of an active volume, removes the GPU path: the point falls back to the box, and the measurement tool **refuses** that point.

## Laying the stack flat: opening the Z-stack

Chapter 12 (§ 12.4) says the stack “lays itself flat in 1.5 s”. Here is the geometry, which also serves the sample side (§ 13.9).




▲ Two rotations about the screen axes, computed for an example.

`setView(view, options)` accepts several ways of choosing **the in-plane angle** of the view:

| SPIN | CHOICE |
|---|---|
| `'frame'` **(default)** | the in-plane rotation nearest the calibrated frame |
| `'nearest'` | the one nearest the **current** pose (the smallest movement, usually about an oblique axis) |

| SPIN | CHOICE |
| --- | --- |
| `'tilt'` | **the one obtained by the two rotations** above; used by the Z-stack |
| **a number** | degrees counted from the frame |

The **side** ( `side` ) is `front` , `back` (the front pose turned a half-turn about the vertical), or `top` / `bottom` : the upper and lower faces of the specimen, resolved according to `upsideDown` . The animation is an interpolation from one end to the other, with an easing of $3t^2 - 2t^3$ ; the two rotations of `'tilt'` grow together so as to read as a single gesture. A camera is notified **only once**, on arrival, so that the Compare panels see the final pose; grabbing the mouse cancels the flight.

### Exporting a very large view

The export (§ 12.6) renders the view **in tiles** of at most 2048 px (less if the graphics card cannot, down to 256 px as a last resort). To make the large image consistent, `renderViewImage` :

- takes the **starting pose** once (the one where a flight in progress lands), with the aspect ratio of the requested image;
- waits for the volume to **finish streaming**, then renders one tile per task;
- checks before **each tile** that the active volume is the same and that the display fingerprint (channels, exposure, render mode, crop, grid and axes, Z stretch) has not changed;
- otherwise **clears and starts over**, three times at most (errors `unstable` or `display-changed` afterwards);
- continues the random grain from one tile to the next ( `fragCoordOffset` ) so that no seam shows, and writes a transparent background through a per-pixel opacity ( `exportAlpha` ).

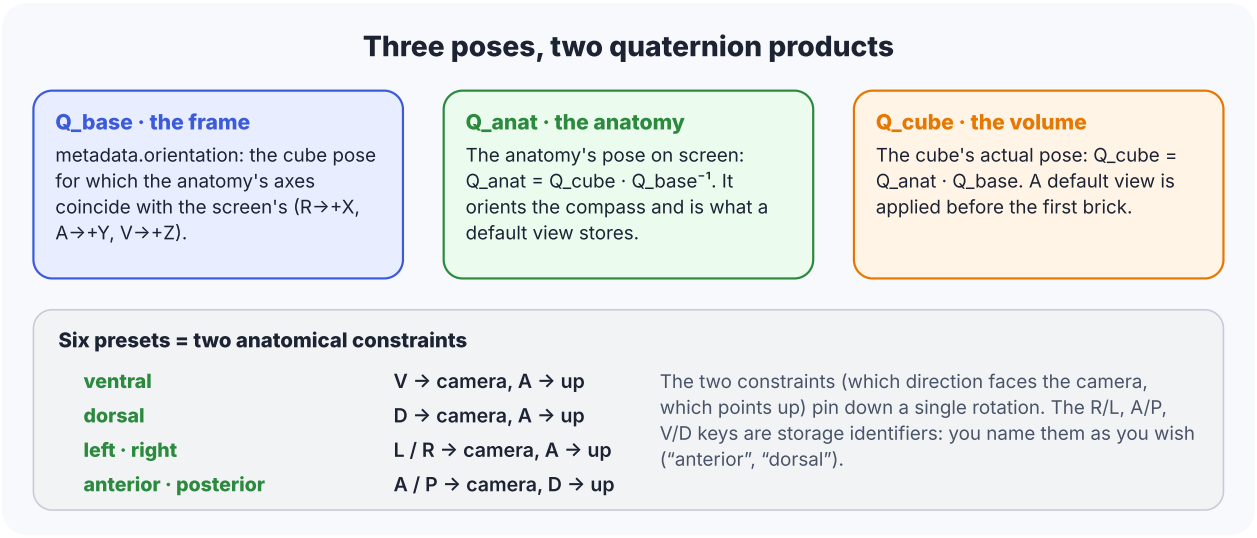
The other errors have a readable code: `busy` (an export is already running), `size` , `canvas` (the browser refuses an image that large), `gpu-memory` , `context-lost` .

## 13.9 Orientation in practice, and the sample side

**⏱ IN 30 SECONDS**

- Three poses: **Q_base** (the frame), **Q_anat** (the anatomy on screen), **Q_cube** (the volume). Two products link them.
- The administrator never computes a quaternion: they **ask** the plugin, which answers by message.
- The **sample side** corrects an Imaris file that shows the specimen seen from below.

Chapter 12 (§ 12.5) gave the theory in one sentence. Here is the complete mechanism, as the administrator handles it (chapter 14, and the administrator guide § 3.8).




▲ The three poses and the two quaternion products.

## What `metadata.json` stores

| FIELD | ROLE |
|---|---|
| `orientation` | **Q_base**: the cube pose for which the anatomy coincides with the screen axes |
| `orientationAxes.labels` | the names the operator gave the six arms (empty: R1/R2, G1/G2, B1/B2) |
| `orientationAxes.hidden` | the arms that are not drawn (they are not even **built**) |
| `orientationAxes.defaultView` | the opening view: `Q_anat` (a preset and/or a quaternion) |
| `upsideDown` | the file shows the specimen seen from below |

The arms have **storage identifiers** (R/L for ±X in red, A/P for ±Y in green, V/D for ±Z in blue) that impose no nomenclature: you name them as you wish.

## The conversation between the administration and the viewer

The administration preview is the real viewer, in a frame. The panel computes **no geometry**: it sends messages to the *Orientation Axes* plugin and stores what comes back. Only one implementation of the geometry exists.

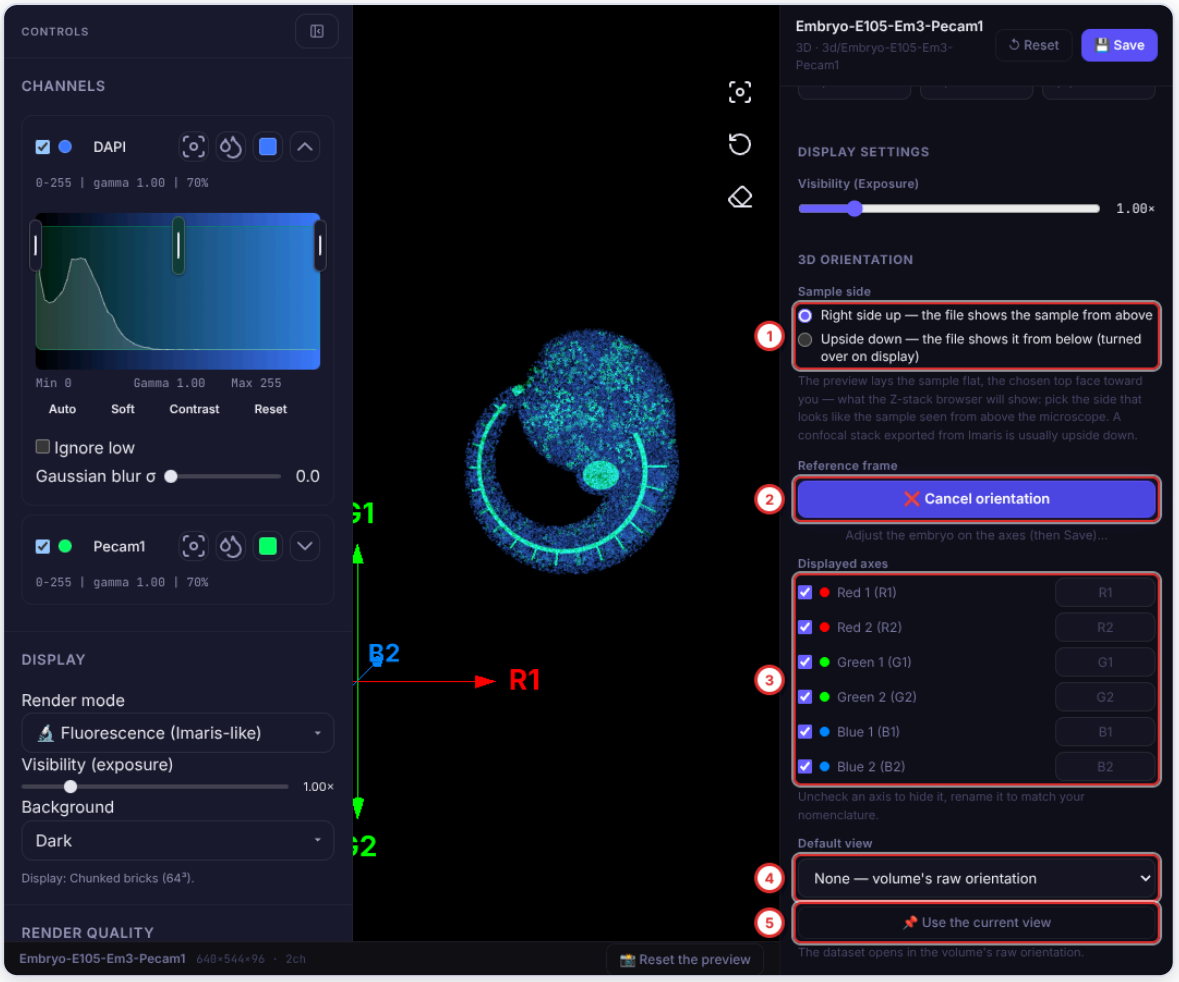
| THE PANEL SENDS… | THE PLUGIN ANSWERS / DOES |
|---|---|
| `CALIBRATE_ORIENTATION_START` | starts again from the **saved** alignment (you refine it, you do not start over), locks the compass to the screen axes, shows it |
| `GET_ORIENTATION` | `ORIENTATION_RESULT` : the cube's current pose, to be saved as `Q_base` |
| `SET_ORIENTATION_AXES` | live preview of the arms' names and visibility |
| `APPLY_ORIENTATION_VIEW` | poses the volume and returns `ORIENTATION_VIEW_RESULT` with the `Q_anat` to store |

| THE PANEL SENDS… | THE PLUGIN ANSWERS / DOES |
| --- | --- |
| `GET_ORIENTATION_VIEW` | captures the **current** pose as the default view (as `Q_anat`) |
| `SET_SAMPLE_UPSIDE_DOWN` | previews the upper face; answers `SAMPLE_SIDE_RESULT` |

🔧 FOR THE CURIOUS

- **The six presets** are each two constraints: which anatomical direction faces the camera (the screen's +Z), which points up (+Y). The single rotation that satisfies them is obtained by building the `(right, up, facing)` basis and taking the inverse. Ventral: facing V, up A; dorsal: facing D, up A; left / right: facing L / R, up A; anterior / posterior: facing A / P, up D.
- **The default view is applied before the first brick** ( `prepare()` , called between the viewer's initialisation and the first load): turning afterwards would show the volume in one pose and then snap it into another while the bricks arrive. It is recorded as the viewer's **home pose**: the **Reset view** button returns to it, not to the file's raw axes.
- **A default view is stored as Q_anat**, a statement about the anatomy ("ventral toward me"), so refining the calibration later does not make it stale.
- When a dataset has **no** calibration, the arms start from the file's raw pose (turned over if `upsideDown` ).
- The compass is driven by $Q_{cube} \cdot Q_{base}^{-1}$ ; its central sphere moves by dragging, and its position is in the workspace.

## The sample side




▲ The administration preview: 3D orientation and axes gizmo.

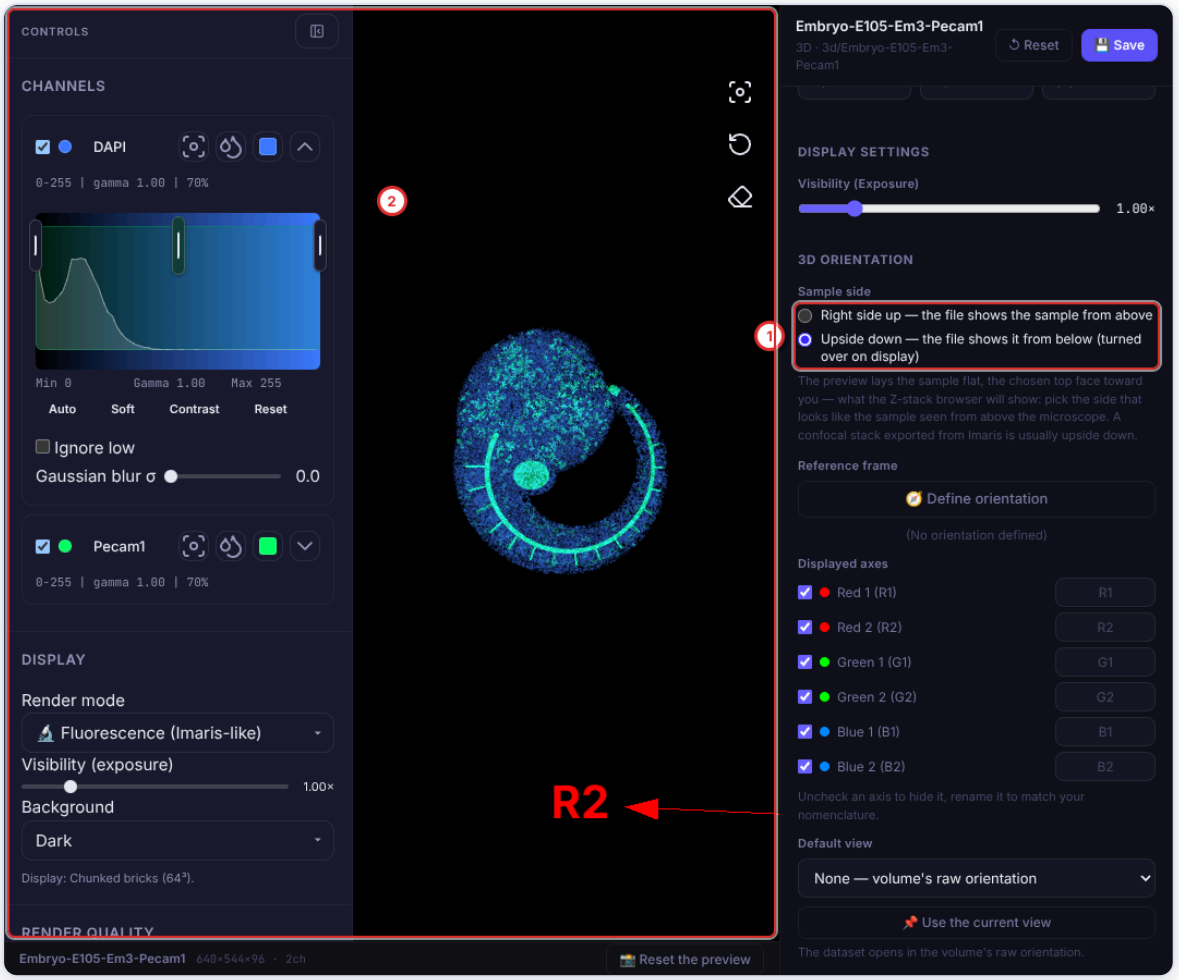
| N | WHAT IT IS |
|---|---|
| 1 | **Sample side**: two radio buttons |
| 2 | 🧭 **Define orientation**: places the axes gizmo in the preview |
| 3 | **Displayed axes**: tick, hide, rename |
| 4 | **Default view**: the dataset's opening pose |
| 5 | 📌 **Use the current view**: captures the preview's pose |

A confocal stack exported from Imaris is **usually seen from below**: an inverted microscope's objective is under the specimen. The viewer calls the −Z face the "top" in that case ( `upsideDown: true` ), and the raw pose of an uncalibrated dataset is turned over by a **half-turn about the screen's vertical**: left and right swap, up stays up.

❓ WHY THIS CHOICE?

**Why a half-turn about the vertical and not about the file's X axis?** As soon as a calibration has tilted the file's axes on screen, a half-turn about the file's X axis looks like a diagonal tumble. About **the screen's** vertical, it is always the same gesture, whatever the pose.

The administration's switch **previews** the upper face: it lays the volume flat ( `'tilt'` , 1 s) facing you. That is exactly what the Z-stack browser will then show.




▲ After choosing "Upside down": the preview lays itself flat, the chosen face toward you (same numbered callouts).

- **Neither Q_base nor a default view depends on the side**: both are poses of the anatomy; the editor only stores the flag.
- The side is taken into account by the initial view of an uncalibrated dataset, the reset button, the "3d" view, the Z-stack and the Studio figures (180° yaw on the −Z face).

⚠️ WARNING

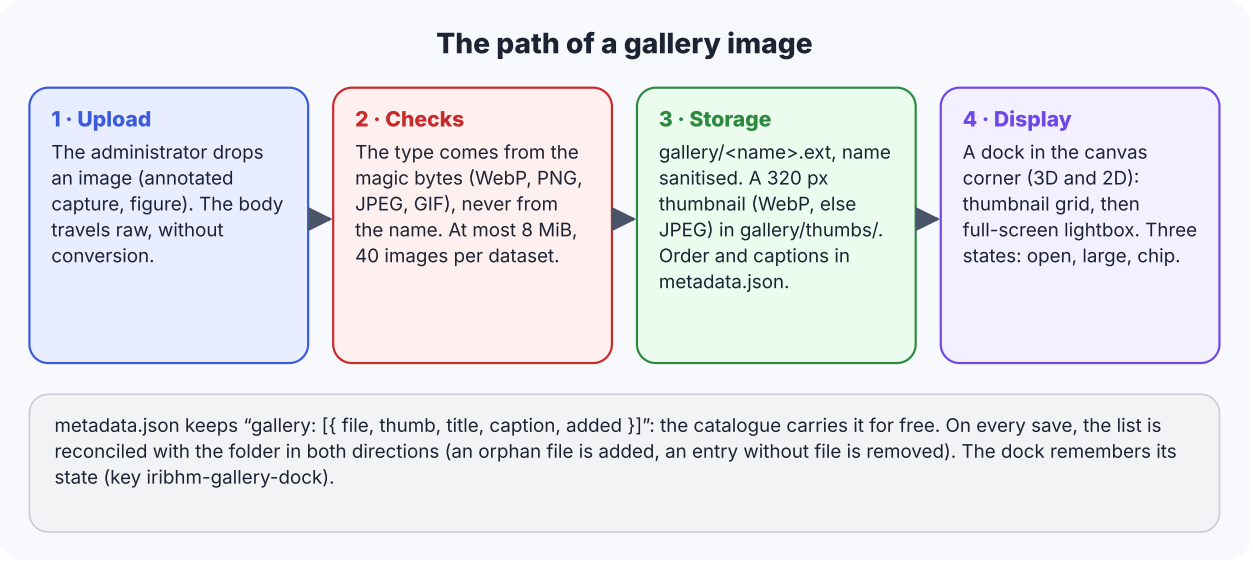
A dataset whose side was switched under platform version 1.55.7 has a distorted frame (it had been turned about the file's X axis). Its orientation must be **defined again**.

# 13.10 Gallery, linked datasets, thumbnail

⏱ IN 30 SECONDS

- The **gallery** adds images (annotated captures, diagrams) to a dataset; their type comes from the **magic bytes**, never from the name.
- **Linked datasets** are deduced from four rules, with nothing to enter.
- The explorer's **thumbnail** is a 512 × 512 capture that the administrator chooses.

## The image gallery



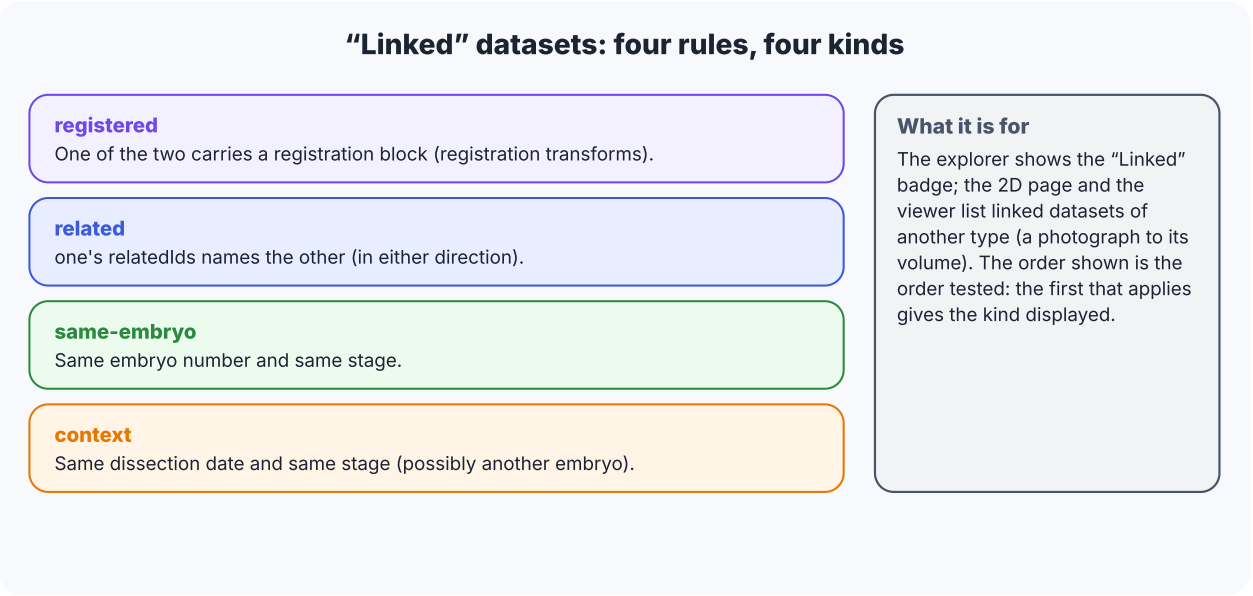

▲ The path of an image, from upload to the viewer.

The API accepts a **raw** image body (up to the server's limit) or, historically, a `data:` URL. In both cases the type is recognised from its **magic bytes** (WebP, PNG, JPEG, GIF); the file name is only used to make a readable name (letters, digits, hyphens, 60 characters at most, numbered in case of a duplicate). A file that is too large (8 MiB), a 41st upload or content that is not an image are refused.

The 320 px **thumbnail** serves the dock's grid: forty originals must not compete for the connections the brick packs need. Without Pillow or without WebP on the server, it falls back to JPEG, or is omitted. An image for a dataset **still being imported** is refused (409): publish first.

On the page side, `DatasetGallery` builds its DOM only with `createElement`, never by concatenating text: captions are operator text and file names come from the disk.

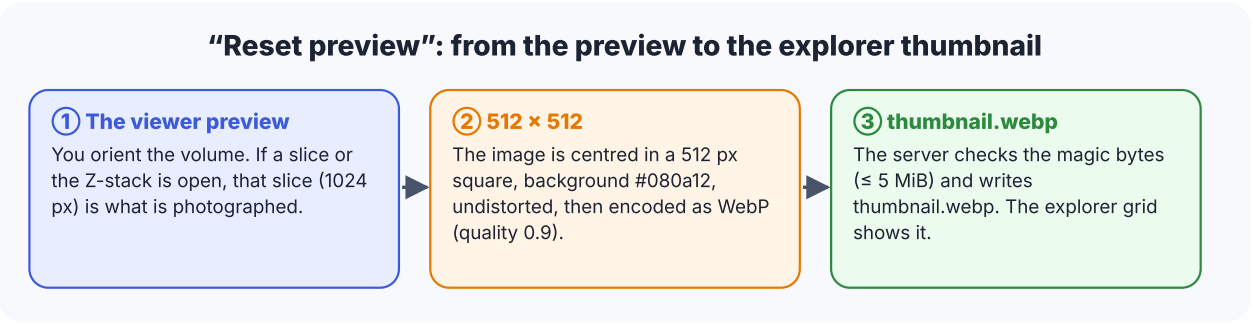
## Linked datasets




▲ The four natures of a link between two datasets.

`Catalog.getRelated(id)` returns the datasets that share **the same embryo and the same stage**, **the same dissection date and the same stage**, or that a `relatedIds` designates (in either direction). `getRelationMeta` then gives the **nature** of the link, testing in this order: *registered* (a registration block exists), *related* (explicit link), *same-embryo*, *context* (the rest). The explorer shows the "Linked" badge; the 2D page and the viewer list the linked datasets of another type: a photograph points to the volume of the same embryo.

## The explorer thumbnail




▲ From the administration preview to thumbnail.webp.

The **Reset the preview** button asks the preview viewer for a capture. If a slice or the Z-stack is open, **that** is what is photographed (rendered at 1024 px); otherwise the 3D canvas. The image is placed at the centre of a 512 × 512 px square with a `#080a12` background **without distortion**, then encoded as WebP (quality 0.9). The server checks the magic bytes and a size of at most 5 MiB before writing `thumbnail.webp`.

📖 **GOING FURTHER**

- The format of the `planes/` and `mips/` folders, and the migration journal: chapters 7 and 17.
- The plugin system (`contexts`, `dataTypes`) and trust: chapter 15.
- The exact names of the administration screens: chapter 14 and the administrator guide.

📌 **REMEMBER**

This chapter on one page:

- **Tracking**: instanced spheres, 32-bit arrays, one shared facade.
- **Stabilisation**: `volumeWarp = toTex · M⁻¹ · toUm`; no voxel retouched.
- **Studio**: a document without pixels, recolouring from raw values, native pass plane / atlas, token.
- **Compare**: real pages, verified messages, 1.5 GiB budget, time by fraction.
- **2D**: three frames of reference, a pixel worker, an importer that guesses nothing.
- **Timeline**: clock capped at 250 ms, held back during loading.
- **State**: the link is the backup; the browser only keeps preferences.
- **Measurement**: depth read on the GPU in two 16-bit passes; poses by two rotations.
- **Orientation**: Q_base, Q_anat, Q_cube; the sample side is a half-turn about the vertical.

# 14. The administration panel in brief

- The administration panel is Lumen3D's **engine room**: it is where you put data online, set the look of the site and update the platform.
- It has **15 tabs in 4 groups**. You will only use a few of them day to day.
- This chapter is an **overview**. The step-by-step manual is the "**Administrator guide**", published in the panel's own **Documentation** tab.

## 14.1 What the panel is for, and how to reach it

The public site is the shop window: visitors look at embryos. The administration panel is the **back office**. It lets you:

- **put datasets online** and describe them (name, stage, colours, orientation);
- **choose what the public sees** (visible or hidden);
- **customise the site**: name, colours, pages, legal notices;
- **install plugins** and **update the platform**.

ℹ️ **GOOD TO KNOW**

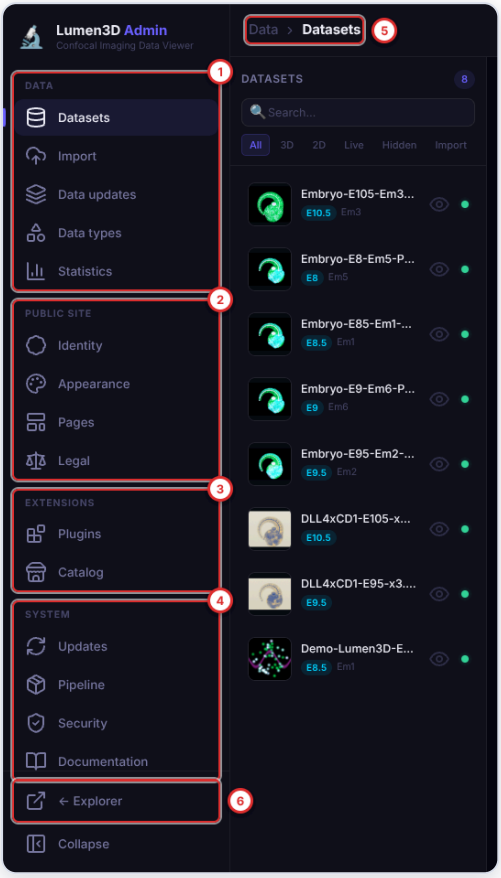
The panel has **no public link**. You open it by typing the address `admpan.html` after the site's address (for example `https://your-site/admpan.html`). It asks for a user name and a password. The password is created on the very first visit, using a 5-step wizard (account, identity, theme, texts, plugins).

## 14.2 The 15 tabs, in 4 groups

The left-hand sidebar arranges the tabs into four groups. The breadcrumb at the top tells you where you are (for example **Data › Datasets**).

| TAB | WHAT IT DOES | HOW OFTEN |
|---|---|---|
| **Data** | | |
| **Datasets** | List, preview, settings | often |
| **Import** | Upload a pipeline folder | for every new dataset |
| **Data updates** | Bring datasets to format 5 | rarely |
| **Data types** | Rename 3D, 2D, Live | once |
| **Statistics** | Visits, views, downloads | occasionally |
| **Public site** | | |

| TAB | WHAT IT DOES | HOW OFTEN |
|---|---|---|
| **Identity** | Name, vocabulary, SEO, menu | at installation |
| **Appearance** | Colours, font, rounding | at installation |
| **Pages** | Visual page editor | sometimes |
| **Legal** | Legal texts, per language | once |
| **Extensions** | | |
| **Plugins** | Enable, approve | sometimes |
| **Catalog** | Install signed plugins | sometimes |
| **System** | | |
| **Updates** | Update the platform | regularly |
| **Pipeline** | Download the pack (ch. 4 to 8) | for every new pack |
| **Security** | Password, permissions | rarely |
| **Documentation** | Read the published guides | on demand |




▲ The panel's sidebar (demo dataset).

- 1 **Data** group: the datasets
- 2 **Public site** group: appearance and texts seen by visitors
- 3 **Extensions** group: plugins
- 4 **System** group: updates, security, documentation
- 5 Breadcrumb: group, then current tab
- 6 Link to the public site (new tab)

At the top right: light/dark theme of the panel, language (French, English, Spanish, Dutch) and log out.

📌 **REMEMBER**

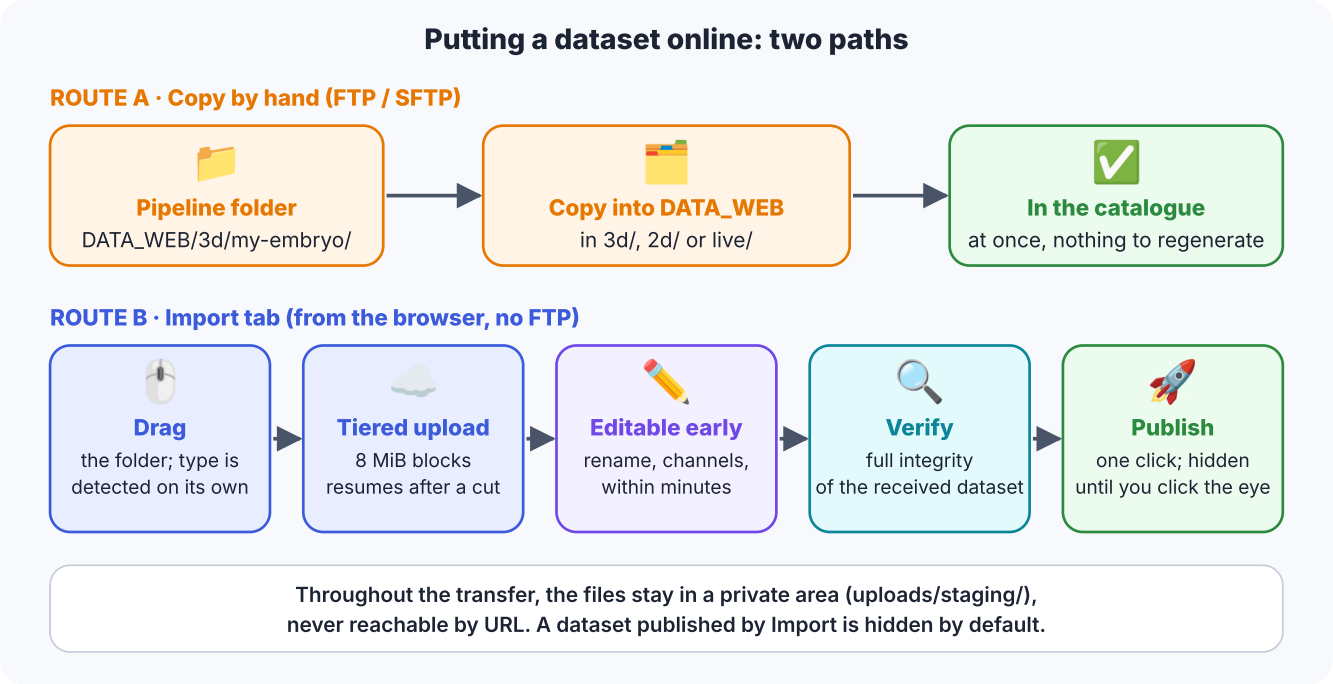
Two tabs start with “Updates”: **Data updates** (Data group: bring **your datasets** to the current format) and **Updates** (System group: update **the platform**). Do not confuse them.

👉 **TIP**

An orange “Unsaved changes” badge lights up at the top as soon as a tab holds changes that have not been saved yet. Ctrl + S saves the visible tab (Datasets, Data types, Identity, Appearance, Legal).

## 14.3 Putting a dataset online

The pipeline (chapters 4 to 8) produces **one folder**. There are two ways to put it on the server.

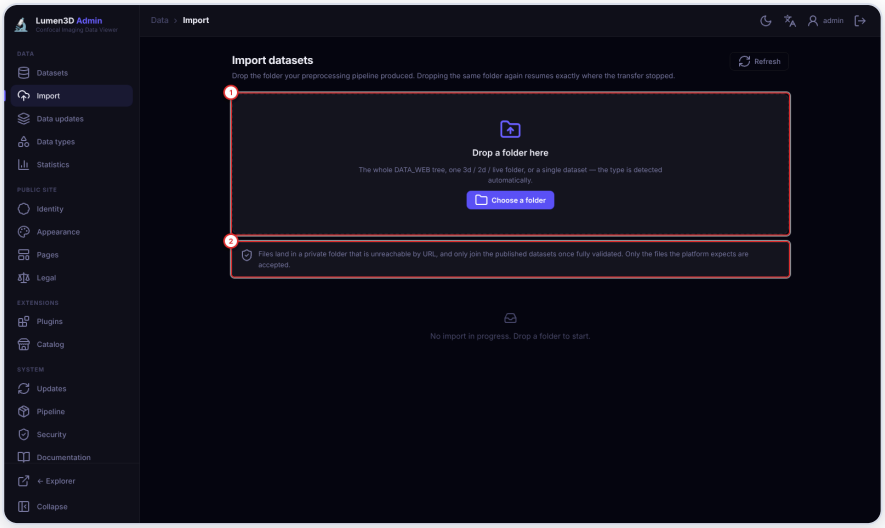



▲ Two paths to put a dataset online.

🔢 **WORKED EXAMPLE**

**Route A**: you copy the folder into `DATA_WEB/3d/` by FTP. The dataset appears straight away: the catalogue is recomputed on every request from the `metadata.json` files, so there is nothing to regenerate.

## The Import tab, step by step




▲ The Import tab, before any transfer (demo dataset).

1 Drop zone: drag the folder or click **Choose a folder** · 2 Reminder: transit through a private folder, only the expected files are accepted

Once a transfer is running, a card appears under the drop zone. It shows the overall progress (percentage, volume transferred, speed, time remaining), the dataset's state (see below) and the buttons **Edit**, **Verify**, **Publish** and **Delete**, depending on that state.

The states of a dataset being imported:

| STATE SHOWN | WHAT IT MEANS |
|---|---|
| Uploading — not editable | the files essential for opening it are not all there yet |
| Uploading — editable | can be opened at low resolution: rename it and set the channels while the rest arrives |
| Uploaded — ready to publish | transfer complete and verified; it only remains to publish |
| Interrupted | drag the same folder in again to resume; without a resume, it is purged after 7 days |

💡 **A PICTURE TO UNDERSTAND**

**A move by shuttle truck.** The folder leaves in small numbered boxes (8 MiB blocks) bound for a private warehouse. If the truck breaks down, we restart from the last box delivered. The most useful boxes (the card, the map, the coarse view) go first: you can start decorating before everything has arrived.

⚠️ **WARNING**

A dataset published by Import is **hidden by default**. Go to **Datasets**, then turn on the eye (or the **Visibility** switch) to show it to the public.

## 14.4 Editing a dataset

Click a dataset in the **Datasets** tab's list: the preview opens in the centre, the settings on the right. Here is what you can set there:

| SECTION | WHAT YOU DECIDE THERE |
|---|---|
| **Visibility** | visible or hidden in the explorer (**applied immediately**) |
| **Identification** | display name, stage, embryo, description |
| **Image gallery** | up to 40 images (8 MB max) with captions, shown in the viewer |
| **Physical calibration** | voxel size X, Y, Z in µm |
| **Display settings** | default exposure |
| **3D orientation** | sample side, reference frame, displayed axes, default view |
| **Channels (in the preview)** | name, colour, min, max, gamma of each channel |

📌 **REMEMBER**

What the editor saves is a **setting**: it is merged into the dataset's `metadata.json`. The panel never touches the pixels of the native level. The camera position, the render mode or the slice plane are not kept: they belong to the visitor.

The **Reset the preview** button (under the preview) makes the current view the explorer's thumbnail. The 3D orientation (axes, default view) is explained in chapter 12.

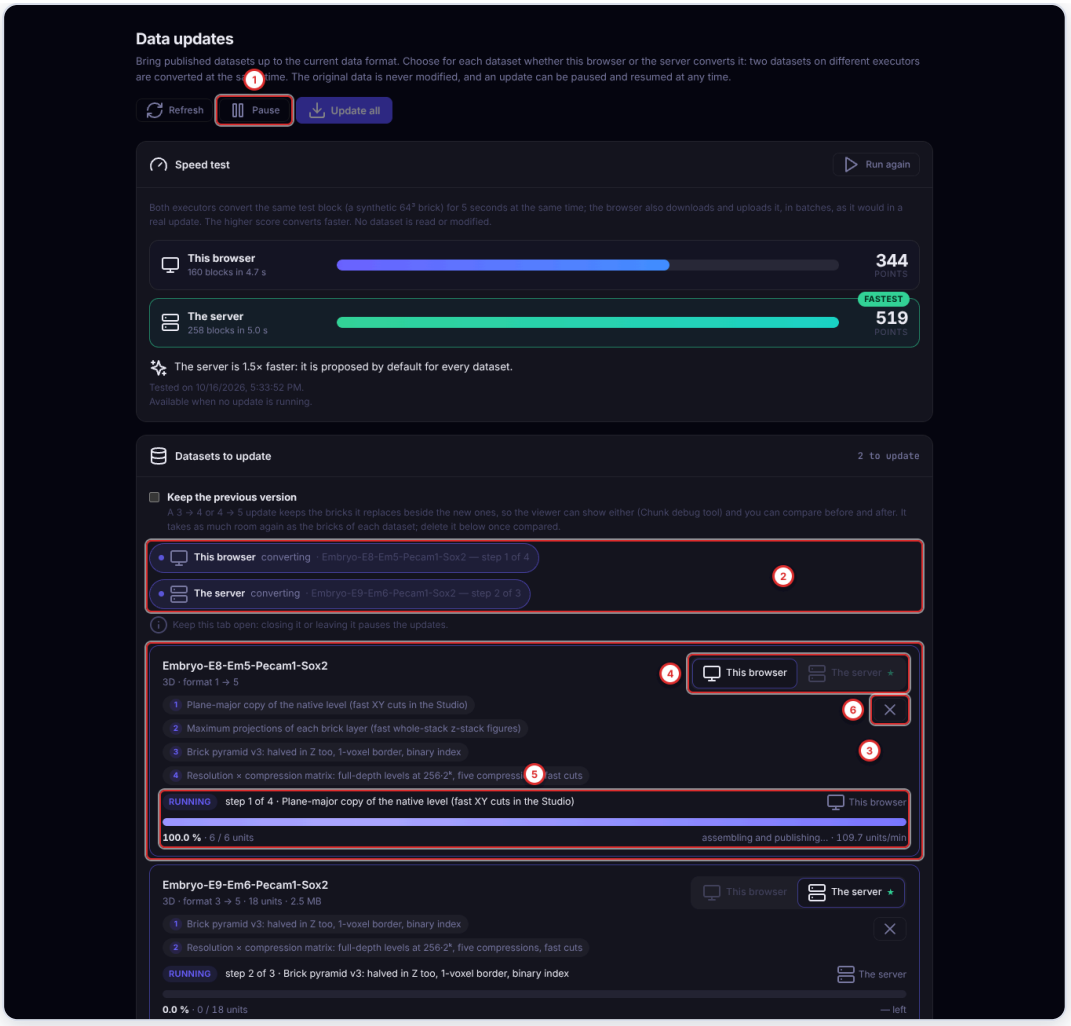
## 14.5 Data updates

The way datasets are stored has evolved five times (**formats 1 to 5**, see chapter 7). An old dataset remains readable, but it can be **brought to the current format without going through the pipeline again**, like updating software.

The four steps add: `planes/` (one file per plane, fast native XY slices in the Studio), `mips/` (maximum projection of each layer of 64 planes, fast z-stack figures), a v3 brick pyramid ($66^3$, one-voxel border, `index.bin`), then the **format-5 matrix** (levels that keep every slice, five compressions, cut copies; section 7.11).

❓ **WHY THIS CHOICE?**

The native level is kept voxel for voxel: nothing is lost. The old `bricks/` tree is deleted only after the dataset's version has changed. A dataset produced by pipeline 0.22.0 arrives in format 4 (in format 5 with `--format 5`): the tab then offers only the 4 → 5 step. On a PHP host, this step runs **in the browser**.




▲ The Data updates tab during a conversion to format 5 (demonstration dataset).

1 **Pause** · 2 The two queues: this browser and the server, each with its running dataset · 3 A dataset card: “format 1 → 5”, its steps · 4 Choice of executor, dataset by dataset (locked while running) · 5 The progress · 6 Cancel. The details are in the *Administrator guide*, chapter 5.

> ⚠ **WARNING**
>
> **Keep the tab open** during an update. Leaving it pauses the update; nothing is lost, you resume later. Do not open several tabs at once: a host has already blocked the address of an operator who sent too many requests. The tab’s network governor limits the rate to avoid this.

These updates are **not mandatory**: the viewer reads formats 1 to 5. They speed up some Studio operations and improve the display.

## 14.6 Plugins and the signed catalogue

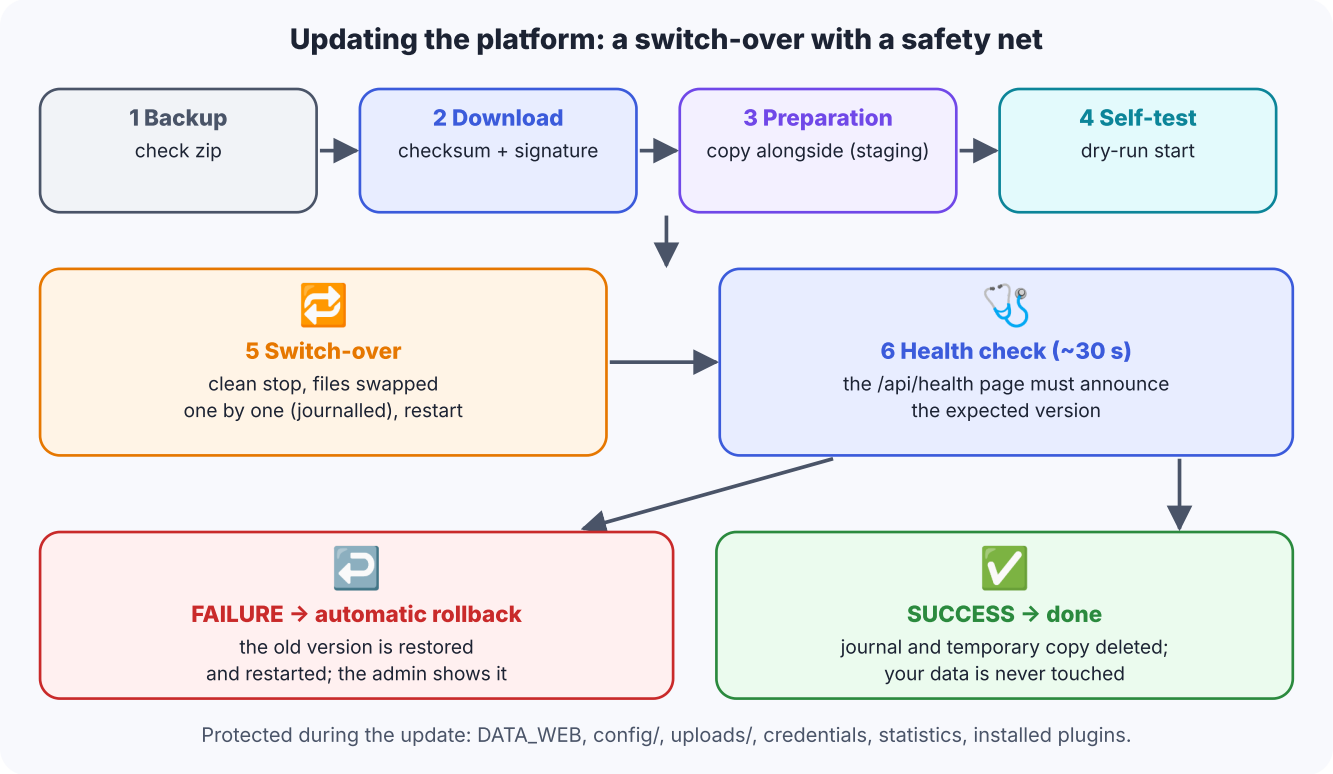
Plugins are the viewer’s tools (chapter 12). They are **not shipped with the platform**: you install them on demand from the **Catalog**, as in an app store.

- The catalogue is **signed**: an unverified plugin is never installed (chapter 19).
- Installation asks for your **password**, then the plugin is installed and approved.

- The **Plugins** tab shows three cards (Tools, Channels, Render modes): one switch per plugin and a trust label (`bundled`, `approved`, `sandbox`, `untrusted`...).
- Some plugins do not appear in the Datasets tab's preview, nor in the Compare page: they declare themselves compatible with the full page only. This is not a fault.

## 14.7 Updating the platform

From **System › Updates**, a button starts the update. What happens behind the scenes was designed so that **a site is never left broken**.




▲ The update switch-over and its safety net.

💡 **A PICTURE TO UNDERSTAND**

**The blue-green move.** The new flat is set up next to the old one; you check that the water and electricity work, and only then do you move in. If something is wrong, you go back to the old one, which was left untouched.

- The tab shows the **release notes** of every version that will be installed, and checks your plugins beforehand: any that would be incompatible are set aside, then re-enabled later. ::: warning This complete safety net (switch-over checked by a health probe, **automatic return** to the old version) exists on a **Python** server. On **PHP** hosting, the update is done in a single request, with the same SHA-256 and signature checks, but **without automatic rollback**: the files already copied stay in place. → chapter 18. :::
- It also shows the version of the **Pipeline pack** (**Pipeline** tab): this is a different piece of software, which is installed on the processing workstation, not on the server.

## 14.8 Customising the site (white label)

Lumen3D is not tied to one laboratory: everything the public sees can be set without writing code.

| TAB | WHAT YOU CUSTOMISE THERE |
|---|---|
| **Identity** | instance name, name of the “specimen” (embryo, organ…), tagline, footer, menu links |
| **Appearance** | brand colour, font, rounding, with live preview |
| **Pages** | visual editor: sections, columns, widgets; the draft is saved automatically |
| **Legal** | text sections, one language at a time |
| **Data types** | the **name** given to “3d”, “2d” and “live” in badges and filters |

ℹ️ **GOOD TO KNOW**

Renaming a data type changes only the **displayed text**: never the folders, identifiers or addresses. The names of the specimen and of the types adapt automatically throughout the site.

📖 **GOING FURTHER**

For the detail of every screen, error messages and special cases, see the “**Administrator guide**” (**Documentation** tab). For the panel’s security, see chapter 19.

# 15. Plugins, trust and the signed catalogue

- Almost everything you do in the viewer (measuring, capturing, adjusting a histogram, choosing a render mode) is provided by a **plugin**: a small folder that the platform discovers, checks and then plugs in.
- A plugin is **third-party code**. Lumen3D treats it like a phone app: its **identity is checked at installation**, it is given **precise permissions**, and it can be **locked in a box**.
- This chapter explains both sides of the system: **how a plugin works** (§15.1 to §15.11) and **why it can be trusted** (§15.12 to §15.19).

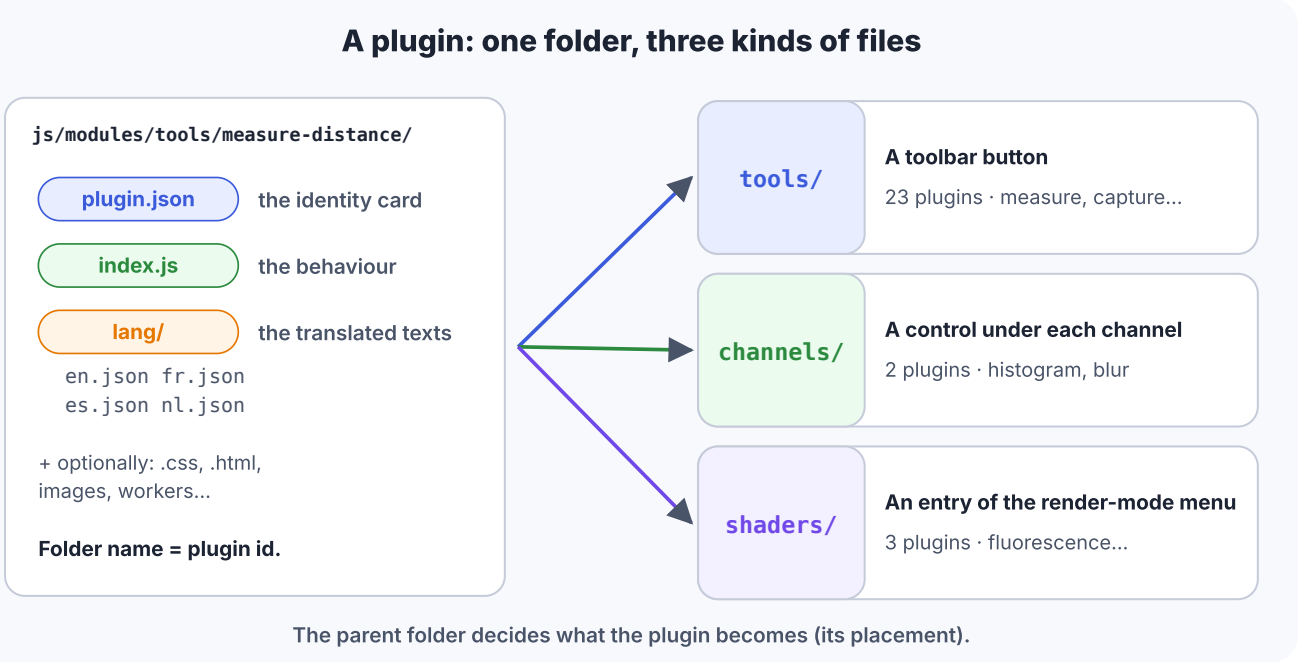
💡 **A PICTURE TO UNDERSTAND**

**A smartphone and its app store.** The platform is the phone. A plugin is an app. The signed catalogue is the official store: every app carries a seal. At installation, the phone checks the seal (signature), the file's fingerprint (hash) and compatibility with its own version (platformCompat). It then locks the app in a sandbox and gives it only the permissions it asked for (capabilities).

Chapter 12 explains **how to use** each tool, chapter 13 what is **under the bonnet** of each one, chapter 14 **the screens** of the administration panel. Here we look at **the system that carries them all**.

## 15.1 What is a plugin?

A plugin is a **folder** stored in `js/modules/<placement>/<id>/`. It contains three kinds of file.




▲ One folder, three kinds of file, three possible destinations.

| FILE | ROLE | ANALOGY |
|---|---|---|
| `plugin.json` | Who I am, where I go, what I require | The identity card |
| `index.js` | What I do (calls `PluginRegistry.implement`) | The know-how |
| `lang/<code>.json` | My texts in French, English, Spanish, Dutch… | The translated manual |

The **parent folder** decides what the plugin becomes. This is its **placement**.

| PLACEMENT | WHAT THE VISITOR SEES | NUMBER IN THE CATALOGUE |
|---|---|---|
| `tools` | A toolbar button (or an exclusive tool) | 23 |
| `channels` | A setting under each fluorescence channel | 2 |
| `shaders` | An entry in the **Render mode** menu | 3 |

Total: **28 plugins**, exactly the 28 entries of the signed catalogue (§15.17).

> ℹ️ **GOOD TO KNOW**
>
> **Automatic discovery.** There is no list to edit: drop a folder in and it appears; remove it and it disappears. The list is rebuilt every time a page loads (§15.3).

## 15.2 The identity card: `plugin.json`, field by field

Here is the real `plugin.json` of the measurement tool (its fields are explained in the next table).

```
{
  "id": "measure-distance",   "name": "Measure Distance",   "version": "1.1.4",
  "platformCompat": ">=1.53.0",
  "dataTypes": ["3d", "live", "2d"],   "contexts": ["page", "panel"],
  "creator": "IRIBHM",   "placement": "tools",
  "group": "tools",   "subtype": "tool",   "icon": "ruler",   "order": 20,
  "tool": "measure",   "shortcut": "m",
  "description": "Pick two 3D surface points to measure calibrated physical distance in µm.",
  "i18nTitle": "title",   "i18nLanguages": ["en", "fr", "es", "nl"]
}
```

The loader only **requires** the `id` (and that it equals the folder name). Everything else has a sensible default.

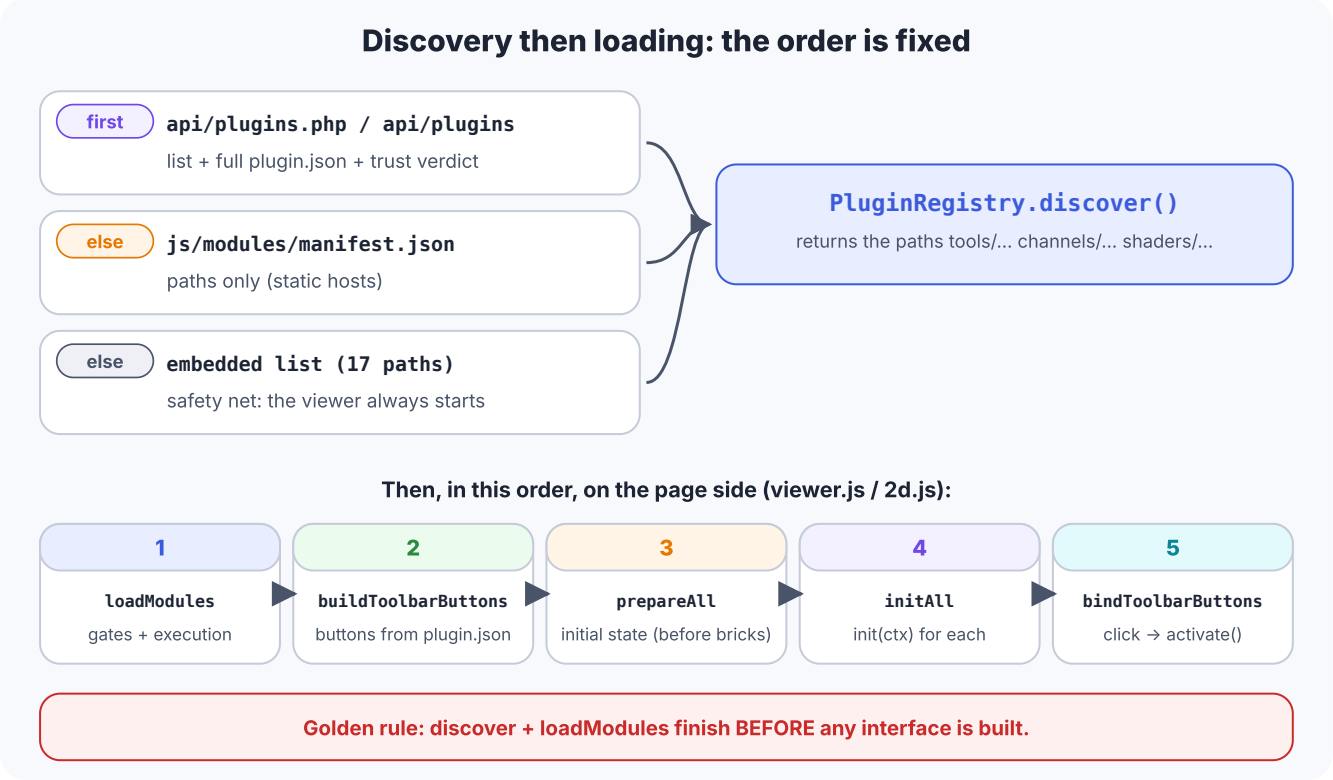
| FIELD | WHAT IT SAYS | VALUES AND RULES |
|---|---|---|
| `id` | Unique identity | **Equal to the folder name** (`[A-Za-z0-9_][A-Za-z0-9._-]*`), otherwise quarantine |
| `name`, `creator`, `description` | Display text (catalogue card, Plugins tab) | Free |

| FIELD | WHAT IT SAYS | VALUES AND RULES |
|---|---|---|
| `version` | **The plugin's** version (SemVer) | The catalogue compares it to offer an update |
| `placement` | Where it goes | `tools`, `channels` or `shaders`; must **equal the parent folder** |
| `platformCompat` | Accepted platform versions | Range or list (§15.15); absent = compatible |
| `dataTypes` | Data types covered | `3d`, `2d`, `live`; **absent** = old volume-viewer plugin |
| `contexts` | Where it can live | `page` (full page) and/or `panel` (embedded page); **absent** = `page` only |
| `requires` | Sources it needs | `bricks`, `webstack`, `tracking`; the button stays **hidden** while they are missing |
| `group` | Toolbar cluster | `tools`, `export`, `visuals`, `layouts` |
| `subtype` | Kind of button | `tool`, `toggle`, `action` (channels: `per-channel`) |
| `icon` | Pictogram | Name of a **Lucide** icon |
| `order` | Place in the cluster | Ascending; **999** if absent |
| `tool` | Name of the exclusive tool | Default = the `id`; only for `subtype: "tool"` |
| `shortcut` | Keyboard shortcut | One key, for an exclusive tool (`m` for measure, `c` for slice) |
| `buttonId` | HTML identifier of the button | Plain token, accepted only if no other element carries it |
| `i18nTitle`, `i18nAria` | Keys of the label and of the accessibility text | Looked up in the plugin's `lang/<code>.json` |
| `i18nLanguages` | Languages shipped | The server **recomputes** it from the `lang/` folder |
| `renderModeValue`, `default` | Render modes only | Ray-marcher mode number (0, 1, 2); `default: true` = mode chosen at opening |
| `sandbox`, `sandboxCapabilities` | Plugin written for the sandbox | §15.14 |

⚠️ **WARNING**

**A `trust` field in `plugin.json` is ignored.** The plugin is written by a third party: if it could declare its own trust level, the check would be pointless. The registry **deletes** this field and reads the verdict only from the server's answer.

## 15.3 How the platform finds plugins




▲ Three sources in order of preference, then five steps always in the same order.

The page asks `PluginRegistry.discover()` for the list. There are three sources; the first one that answers wins.

1. **The server** (`api/plugins.php`, then `api/plugins`: the `.php` address is tried first because it works on both kinds of server). For each plugin it returns the **complete** `plugin.json`, the translations already included and the **trust verdict** (§15.12).
2. **The file** `js/modules/manifest.json`: only `{path, placement, id}` triplets. The server **rewrites** it at every discovery, so that a purely static host inherits an up-to-date list.
3. **The list built into the code**: 17 paths of the historical core. It is a safety net: even if everything else fails, the viewer starts.

The server **filters** before answering: it removes plugins disabled by the administrator, **incompatible** plugins and **untrusted** plugins. An untrusted plugin is therefore not even a candidate for loading.

❓ WHY THIS CHOICE?

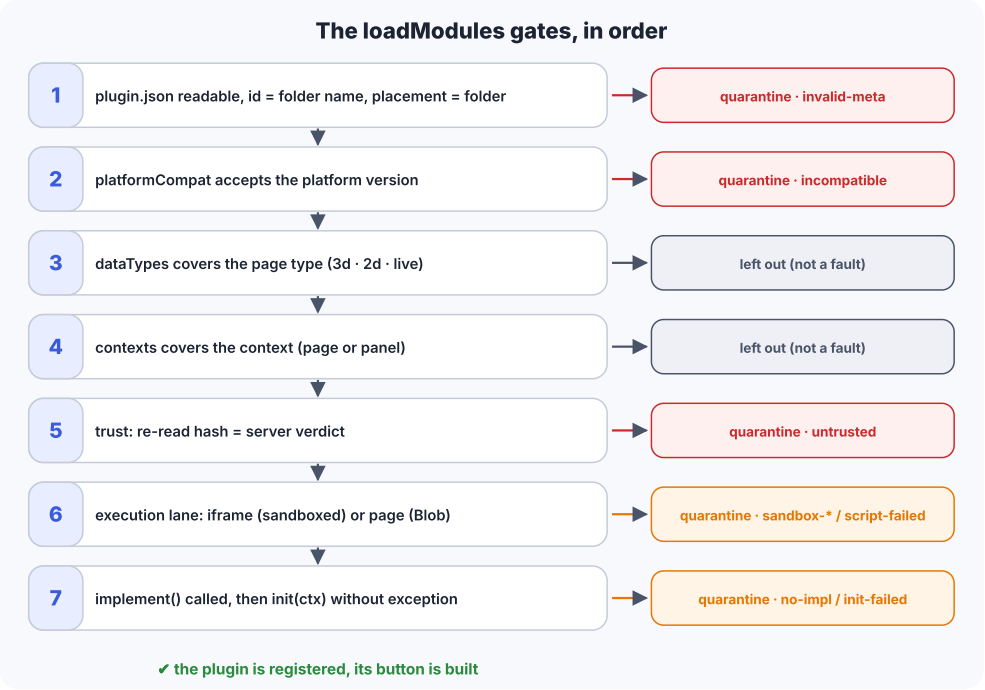
**Why wait for everything before drawing anything?** If the toolbar were drawn before the plugins had finished loading, it would appear empty. This rule was broken once (version 0.12.45) and the result was a viewer with no tools. Since then, `discover` and `loadModules` are **always finished** before any interface is built.

🔧 FOR THE CURIOUS

A folder is taken into account only if its name matches `^[A-Za-z0-9_][A-Za-z0-9._-]*$` (no `..`, no slash) and if its `plugin.json` is readable and does not contradict the parent folder. Otherwise the server **silently ignores** it. Translation tables are attached only if English is present. The answer also carries `devTrust` (is developer mode on?) and `trustEpoch` (§15.14).

## 15.4 Loading: seven barriers

Once it has the list, `loadModules` examines each plugin through a series of checks. The first four are **choices** (does this plugin target this page?); the rest are **safeguards**.




▲ Each plugin goes through the barriers in order; on the right, what happens to those that fail.

There are two kinds of failure, not to be confused:

- **Left out** (barriers 3 and 4): the plugin is not made for this page. A 2D photograph tool has no business on a 3D volume. This is **not a fault**: only an information message appears in the console.
- **Quarantined**: something is wrong. The reason is recorded in a register (`PluginRegistry.getQuarantined()`) and a warning appears in the browser console.

🔧 FOR THE CURIOUS

**Quarantine reasons** ( `reason` field): `meta-unreachable` (plugin.json not found), `invalid-meta` (unreadable JSON, id missing or different from the folder, inconsistent placement, plugin.json modified during verification), `incompatible` , `untrusted` , `trust-unavailable` (the trust module is not loaded: as a precaution nothing is run), `sandbox-unsupported-placement` , `sandbox-unavailable` , `sandbox-boot-failed` , `script-failed` (index.js does not load), `no-impl` (index.js never called `implement` ), `init-failed` (init threw an exception), `revoked` (approval withdrawn on the fly), `load-error` (other exception).

📌 REMEMBER

**A broken plugin never costs you the viewer.** Each plugin is loaded, initialised and activated in its own `try/catch` , and a global barrier surrounds the whole plugin phase: even if this entire subsystem failed, the 3D canvas would still start.

## 15.5 Who loads where? `dataTypes` and `contexts`

Two `plugin.json` fields decide whether the plugin is taken on a given page.

`dataTypes` : the dataset **type** ( `3d` , `2d` , `live` ).

- If present: the plugin runs on these types and **on no other**.
- If absent: it is an old plugin written for the volume viewer; only the 3D / live viewer accepts it (never the 2D page).

`contexts` : the page's **situation**.

- `page` : full page, with its toolbar.
- `panel` : **embedded** page without a toolbar (Compare panel, split-view pane, admin preview).
- If absent: `page` only (**default refusal** in an embedded page).

### What loads where? (28 plugins × type × context)

| | | 3d page | 3d panel | live page | live panel | 2d page | 2d panel |
|---|---|---|---|---|---|---|---|
| calibrated-grid | tools | · | · | · | · | ● | ● |
| chunk-debug | tools | ● | · | ● | · | · | · |
| decompose-channels | tools | ● | · | ● | · | · | · |
| display-adjust | tools | · | · | · | · | ● | ● |
| download-center | tools | ● | · | ● | · | ● | · |
| figure-panel | tools | · | · | · | · | ● | · |
| measure-distance | tools | ● | ● | ● | ● | ● | ● |
| orientation-2d | tools | · | · | · | · | ● | ● |
| orientation-axes | tools | ● | ● | ● | ● | · | · |
| presentation-mode | tools | ● | · | ● | · | ● | · |
| screenshot | tools | ● | · | ● | · | ● | · |
| screenshot-sandboxed | tools | ● | · | ● | · | · | · |
| slice-inspector | tools | ● | ● | ● | ● | · | · |
| split-view | tools | · | · | · | · | ● | · |
| toggle-axes | tools | ● | ● | ● | ● | · | · |
| toggle-grid | tools | ● | ● | ● | ● | · | · |
| toggle-volume | tools | ● | ● | ● | ● | · | · |
| tracking-charts | tools | · | · | ● | · | · | · |
| tracking-inspector | tools | · | · | ● | ● | · | · |
| tracking-measure | tools | · | · | ● | · | · | · |
| tracking-surface | tools | · | · | ● | ● | · | · |
| tracking-trails | tools | · | · | ● | ● | · | · |
| zstack-browser | tools | ● | ● | ● | ● | · | · |
| gaussian-filter | channels | ● | ● | ● | ● | · | · |
| histogram | channels | ● | ● | ● | ● | · | · |
| fluorescence | shaders | ● | ● | ● | ● | · | · |
| natural-fluorescence | shaders | ● | ● | ● | ● | · | · |
| structure-dvr | shaders | ● | ● | ● | ● | · | · |

● = loaded · = left out (computed from the real plugin.json files)

▲ Actual result for the 28 plugins, computed from their plugin.json files.

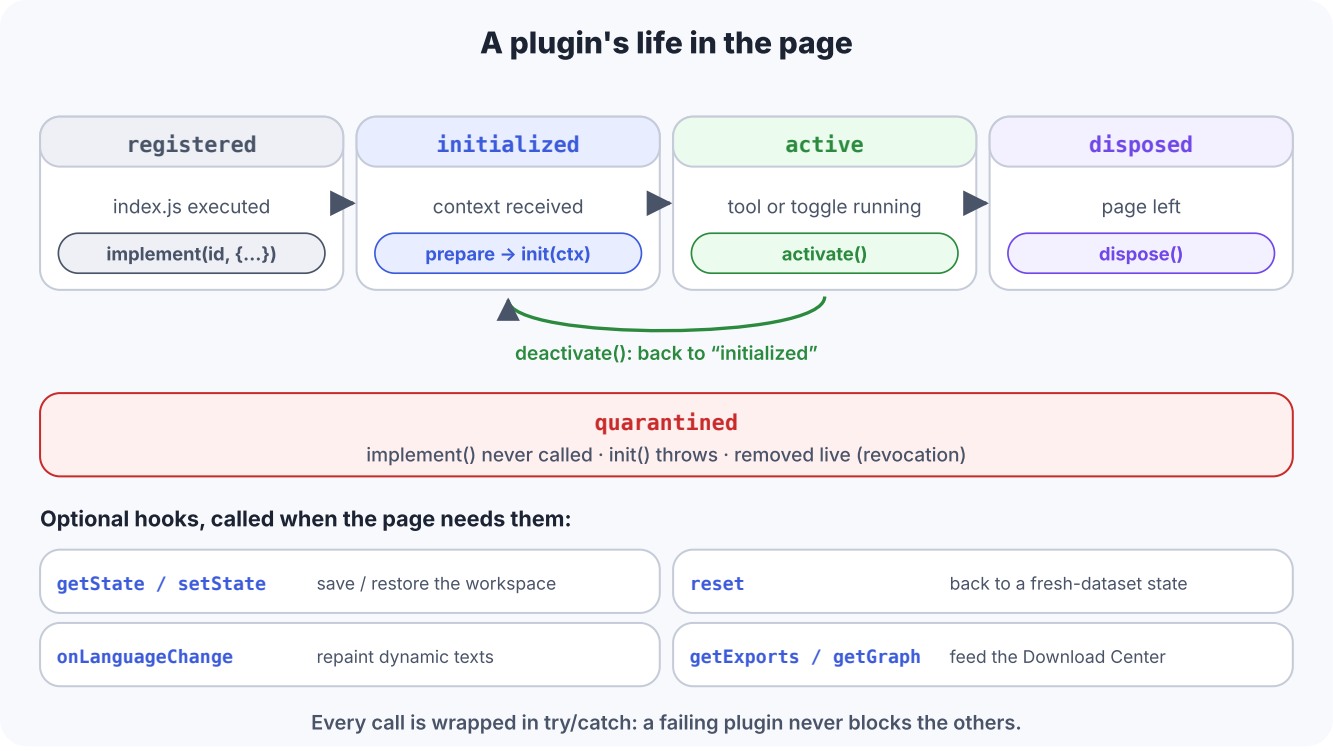
| PAGE | PLUGINS LOADED |
|---|---|
| 3D volume, full page | 18 |
| 3D volume, in a panel | 12 |
| Timelapse (live), full page | 23 |
| Timelapse (live), in a panel | 15 |
| 2D photograph, full page | 9 |
| 2D photograph, in a panel | 4 |

**Ten plugins are “page only”**: Chunk Debug, Decompose by Channel, Download Center, Figure Panel Builder, Presentation Mode, Screenshot, Screenshot (sandboxed), Split View, Tracking Charts, Cell Distance. A 300-pixel panel cannot host a full screen, a modal menu or a split view inside a split view. **This is not a fault**: the administrator sees it too in the preview of the Datasets tab.

🔢 **WORKED EXAMPLE**

On the 2D page, the viewer asks for `dataType = 2d`. `measure-distance` declares `["3d","live","2d"]`: taken. `zstack-browser` declares nothing: an old volume-viewer plugin, so **refused** by the 2D page (which does not accept plugins without `dataTypes`). `calibrated-grid` declares `["2d"]`: taken on 2D, absent from the 3D volume.

## 15.6 A plugin's life cycle




▲ From registration to the end of the page, with quarantine as the emergency exit.

A plugin is an object that `index.js` hands to the registry: `PluginRegistry.implement('my-id', { … })`. The hooks, all optional:

| HOOK | CALLED WHEN | DETAIL |
|---|---|---|
| `prepare(ctx)` | Before the first brick load | **Initial** state of the scene (e.g. Orientation Axes applies the default view). Synchronous |
| `init(ctx)` | After the volume has loaded | Receives its context (§15.7); may return its instance |
| `activate()` | Button click, or render-mode change | Returns `{ active, icon }` for a toggle |
| `deactivate()` | Deactivation | Back to the "initialized" state |
| `getState()` / `setState(s)` | Workspace save / restore | The states of all plugins are gathered into one object |

| HOOK | CALLED WHEN | DETAIL |
|---|---|---|
| `reset()` | **Reset the workspace** | Only plugins that declare it are affected |
| `onLanguageChange()` | Language change | Repaint dynamic texts |
| `getExports()` / `getGraph()` | Download Center opens | §15.9 |
| `dispose()` | End of page | Free resources |

The three **button kinds** ( `subtype` ) behave differently.

| SUBTYPE | BEHAVIOUR | EXAMPLE |
|---|---|---|
| `action` | One click, one function, nothing to remember | Screenshot |
| `toggle` | On / off, the state is remembered and given back to the button | Toggle Grid |
| `tool` | **Exclusive tool**: only one active at a time (managed by `ToolManager` ) | Measure Distance |

🔢 **WORKED EXAMPLE**

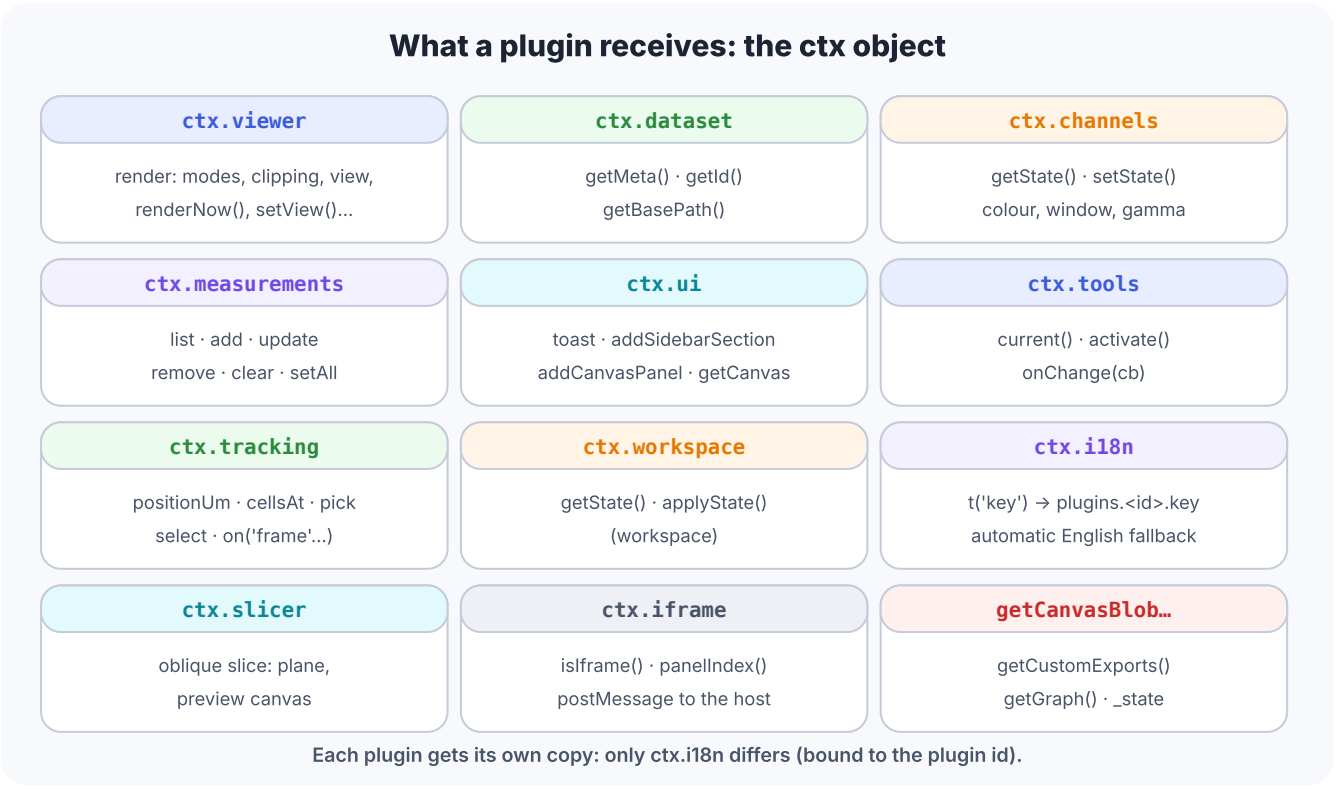
Here is the Toggle Grid plugin (condensed). Each click cycles the grid from “none” to “normal” to “fine”.

```
PluginRegistry.implement('toggle-grid', {
  init(ctx) { this._ctx = ctx; this._mode = 0; return this; },
  activate() {
    this._mode = (this._mode + 1) % 3;
    VolumeViewer.setGridMode(this._mode);
    return { active: this._mode > 0 };       // the button lights up
  },
  getState() { return { gridMode: this._mode }; },
  setState(s) {
    this._mode = s.gridMode;
    VolumeViewer.setGridMode(this._mode);
    PluginRegistry.syncToolbarButton('toggle-grid', { active: this._mode > 0 });
  },
  reset() { this.setState({ gridMode: 0 }); }
});
```

**Channel** and **render-mode** plugins have contracts of their own: a channel plugin supplies `getChannelUI(channel)` (the HTML) and then `bindChannelUI(…)` (the listeners); a render-mode plugin makes do with its `activate()` , which sets the ray-marcher’s mode (chapter 9).

## 15.7 What a plugin receives: the `ctx` object

At `init` , the page hands each plugin a **context**: a façade that gives access to what it needs, nothing more.




▲ The twelve sections of a plugin's context.

| SECTION | WHAT IT OFFERS |
|---|---|
| `ctx.viewer` | Render modes, slices, views, grid and axes, measurements, `renderNow()` (render the image right now before reading the canvas) |
| `ctx.dataset` | `getMeta()` (the `metadata.json`), `getId()`, `getBasePath()`; on the 2D page also the collection and `open(id)` |
| `ctx.channels` | The state of the channels (colour, window, gamma) |
| `ctx.measurements` | Measurements stored per dataset and per domain |
| `ctx.ui` | `toast`, `addSidebarSection`, `addCanvasPanel`, `getCanvas`, `openStudio`... |
| `ctx.tools` | The current exclusive tool, `activate`, `onChange` |
| `ctx.tracking` | The cell tracking of a timelapse (below) |
| `ctx.data` | The **exact voxels**, lossless, whatever quality is displayed (below) |
| `ctx.workspace` | Read / apply the workspace |
| `ctx.i18n` | `t('key')`: looks up `plugins.<id>.key`, automatic fallback to English |
| `ctx.slicer` | The oblique slice plane |

| SECTION | WHAT IT OFFERS |
|---|---|
| `ctx.iframe` | Whether the page is embedded; `postMessage` to the host |
| Miscellaneous | `getCanvasBlob` , `getCustomExports` , `getGraph` |

Each plugin receives **its own copy** of the context: only the `i18n` section differs, because it is tied to the plugin's identifier.

> 🔧 **FOR THE CURIOUS**
>
> **The 2D page's context** has the same shape, with the functions of a photograph: `viewer.getView/setView` , `addOverlay` , `setOrientation` , `setAdjustments` , `getPhysicalView` , `getNativeCanvas` , and `ui.openStudioWith` . A plugin aimed at both pages therefore reads its functions with care ( `ctx.viewer.renderNow?.()` ).

## Cell tracking: `ctx.tracking`

The five tracking plugins never read `tracks.json` themselves. The page keeps **a single copy** (as compact 32-bit tables) and lends it to them.

| FUNCTION | ROLE |
|---|---|
| `isAvailable()` , `whenLoaded()` | Does the dataset have tracking? Wait until it is loaded |
| `positionUm(cell, frame, options, output)` | Position of a cell, in µm, at a given frame |
| `cellsAt(frame)` | The cells present at that frame |
| `pick(x, y)` | Cell under the cursor (−1 if none) |
| `select(c)` , `getSelected()` | **One selection shared** by all plugins |
| `getOptions()` , `setOptions({ neighborThresholdUm })` | Shared neighbourhood radius (5 to 500 µm) |
| `on('loaded' · 'frame' · 'refresh' · 'selection' · 'options' · 'style', callback)` | Subscribe; returns the unsubscribe function |

Result: clicking a cell in the Inspector also selects it in the trajectories and the charts, without them talking to each other.

## The exact voxels: `ctx.data`

Since platform 1.64, a plugin that **computes** something (intensity profile, histogram of a region, measurement) goes through `ctx.data` : it reads the **lossless** column of the bricks, never the graphics card's image, which may be a lossy column (section 7.11).

| FUNCTION | ROLE |
|---|---|
| `screenToVoxel(x, y)` | The point under the cursor: `{ um, voxel, onBoundingBox }` (µm and native voxel) |
| `toVoxel(um, { level })` , `toUm(voxel, { level })` | Go from micrometres to the voxels of a level, and back |

| FUNCTION | ROLE |
| --- | --- |
| `locate(voxel, channel)` | Which file, and which bytes, hold this lossless voxel |
| `readRegion(box, channels, { level })` | The exact voxels of a box (decoded in a worker; refused above 512 MiB) |
| `valueAt(voxel)` | The exact value of each channel at a voxel |

🔢 **WORKED EXAMPLE**

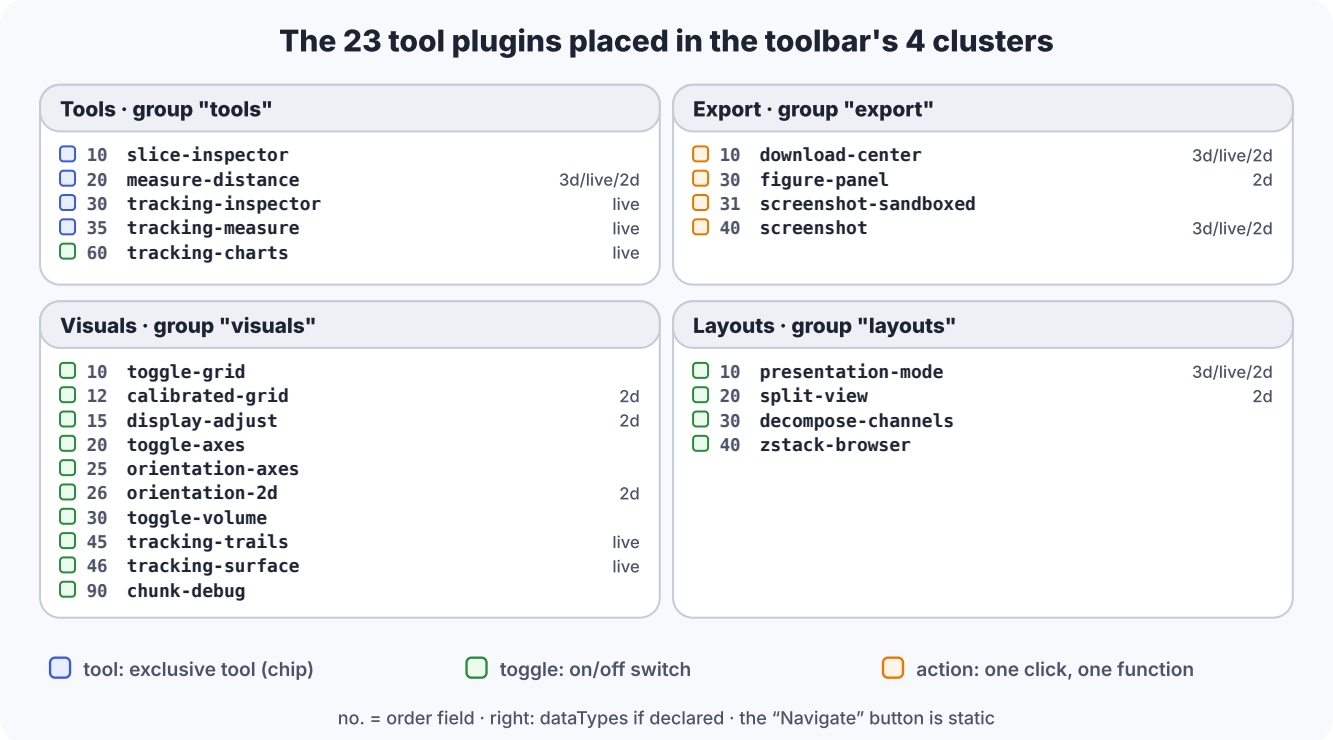
A plugin that shows “intensity at the clicked point” calls `ctx.data.screenToVoxel(x, y)`, then `ctx.data.valueAt(voxel)`. Even if the visitor is looking at the 256 · Very light cell, the value returned is the native voxel’s, exact. Works on formats 2 to 5.

### A plugin’s texts

A plugin brings its translations in `lang/<code>.json`. They are grafted into the platform’s translation tree under `plugins.<id>`. If a language is missing, **English** is shown: a plugin shipped in English only remains usable everywhere.

## 15.8 The toolbar is built, not written

The file `viewer.html` contains **no plugin button at all**. It contains four empty **clusters**: **Tools**, **Export**, **Visuals**, **Layouts**, plus one static “Navigation” button. `buildToolbarButtons` then creates one button per tool plugin, from its `plugin.json`.




▲ The 23 tool plugins, arranged by their group field and sorted by order.

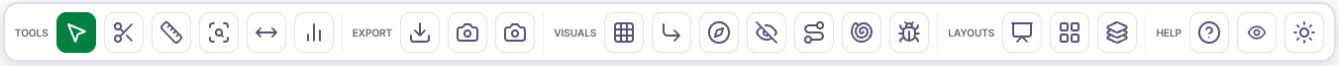
For each plugin, the builder:

1. chooses the **cluster** from `group` (a group with no cluster on the page: warning, button ignored);

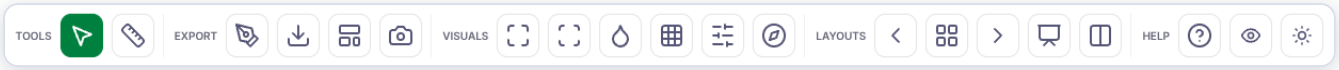
2. creates an **exclusive tool** button ( `subtype: "tool"` , wired by `data-tool` ) or an **action / toggle** button (wired by `data-plugin-id` );
3. sets the `icon` pictogram, the `i18nTitle` tooltip (translated, and translated again at every language change) and the accessibility text;
4. **hides** the button if `requires` is not satisfied ( `tracking` : the dataset has a tracking block; `bricks` / `webstack` : the source exists);
5. registers the `shortcut` with `ToolManager` .

Result: **the toolbar changes from one dataset to the next**, without a line of page code.

▲ Toolbar of a 3D volume (demonstration dataset).




▲ Toolbar of a tracked timelapse: three more tools (inspector, distance, charts) and two tracking toggles (demonstration dataset).




▲ Toolbar of a 2D photograph: the five 2D plugins replace the volume tools (demonstration dataset).

> 👉 **TIP**
>
> **Is your toolbar different from the manual's?** That is normal: the administrator has installed other plugins (chapter 14) or disabled some. A missing button is never a viewer fault.

## 15.9 Plugins and the workspace, exports, embedded pages

### Workspace

Each plugin's `getState()` is gathered into the **same object** as the camera and the channels: this is what the **Save state** and **Restore state** buttons of the Compare page's Download Center keep, and what the `#state=…` address contains. A plugin without `getState` is not affected. A plugin still "registered" (not yet initialised) is skipped.

### Exports

`getExports()` returns a list of `{ action, handler, … }` entries that the Download Center adds to its own exports. `getGraph()` returns an on-screen chart (Plotly) that the Center knows how to export. The registry **collects** these answers, ignoring any plugin that throws an exception: **a faulty export never makes you lose the platform's exports**.

### Embedded pages (Compare, split view, preview)

An embedded page has no visible toolbar. It therefore **describes** what it offers: `describeToolbar()` lists the exclusive tools and the toggles (with their current state) of only those plugins that accept the `panel` context. The host (the Compare page) **draws its own buttons** from this description, then sends `PLUGIN_ACTIVATE` or `SET_TOOL` ; the state comes back through `syncToolbarButton` . The protocol is detailed in chapter 13.

## 15.10 The 28 plugins of the catalogue

**Render modes** (placement `shaders`, loaded everywhere).

| PLUGIN | IN ONE LINE |
|---|---|
| **Fluorescence** | Default rendering: each channel emits its colour (mode 1) |
| **Natural Fluorescence** | Emission-absorption: dense structures hide what is behind them (mode 2) |
| **Structure (DVR)** | Volume rendering with depth and occlusion (mode 0) |

**Channels** (placement `channels`).

| PLUGIN | IN ONE LINE |
|---|---|
| **Histogram Controls** | Intensity histogram per channel and min / max / gamma sliders |
| **Gaussian Filter** | 2D in-plane Gaussian blur, per channel |

**Volume tools** (`3d` and `live`).

| PLUGIN | CONTEXT | IN ONE LINE |
|---|---|---|
| **Slice through Volume** | page + panel | Orientable plane slice; the slice takes over the canvas |
| **Z-Stack Browser** | page + panel | Browse the slices (exclusive with the oblique slice) |
| **Measure Distance** | page + panel | Two points → distance in µm |
| **Decompose by Channel** | page | One channel per panel |
| **Orientation Axes** | page + panel | Orientation gizmo and default view |
| **Toggle Grid · Toggle Axes · Hide/Show 3D Volume** | page + panel | Grid, axes, volume |
| **Chunk Debug** | page | Brick boundaries (requires `bricks`) |

**Tools common to all types.**

| PLUGIN | CONTEXT | IN ONE LINE |
|---|---|---|
| **Download Center** | page | Files of the dataset's `download/` folder, and above them the analysis exports of the page and its plugins (measurements, tracking tables, the chart on screen) |

| PLUGIN | CONTEXT | IN ONE LINE |
|---|---|---|
| **Screenshot** | page | PNG capture of the view |
| **Screenshot (sandboxed)** | page | The same, **in a sandbox** (reference plugin) |
| **Presentation Mode** | page | Full screen without interface |

**2D photograph tools** ( `2d` ).

| PLUGIN | CONTEXT | IN ONE LINE |
|---|---|---|
| **Calibrated Grid** | page + panel | Physical 1-2-5 grid in µm or mm |
| **Display Adjustments** | page + panel | Brightness, contrast, gamma, balance, flattening |
| **Orientation 2D** | page + panel | Rotation and mirror |
| **Split View** | page | A second photograph side by side |
| **Figure Panel Builder** | page | Compose a plate from several photographs |

**Cell-tracking tools** ( `live` , require `tracking` ).

| PLUGIN | CONTEXT | IN ONE LINE |
|---|---|---|
| **Tracking Trails** | page + panel | Trajectories over the volume |
| **Tracking Surface** | page + panel | Specimen surface ( `model.glb` ) |
| **Cell Inspector** | page + panel | Metrics, lineage, neighbours, velocity field |
| **Tracking Charts** | page | Population, speed and mitosis charts |
| **Cell Distance** | page | Distance between two cells |

## 15.11 Writing a plugin

1. Create `js/modules/tools/my-plugin/` (the folder name is the `id` ).
2. Write `plugin.json` (at least `id` , `placement` , `group` , `subtype` , `icon` , `order` ).
3. Write `index.js` with `PluginRegistry.implement('my-plugin', { … })` .
4. Add `lang/en.json` (mandatory as the fallback), then the other languages.
5. Reload the page: the button appears. On a production site, it first carries the **untrusted** label: approve it (§15.12).

A complete example: a button that shows the dataset's name.

```
{ "id": "hello-dataset", "name": "Hello Dataset", "version": "1.0.0",
  "platformCompat": ">=1.53.0", "placement": "tools",
  "group": "export", "subtype": "action", "icon": "info", "order": 90,
  "dataTypes": ["3d", "live"], "contexts": ["page"], "creator": "My lab",
  "description": "Shows the name of the open dataset.",
  "i18nTitle": "title", "i18nLanguages": ["en", "fr"] }
```

```
PluginRegistry.implement('hello-dataset', {
  init(ctx) { this._ctx = ctx; return this; },
  activate() {
    const meta = this._ctx.dataset.getMeta();
    this._ctx.ui.toast(this._ctx.i18n.t('hello') + ' ' + meta.name);
    return { active: false };                // an action: no state
  },
  dispose() { this._ctx = null; }
});
```

With `lang/en.json` : `{ "title": "Say hello", "hello": "Dataset:" }` .

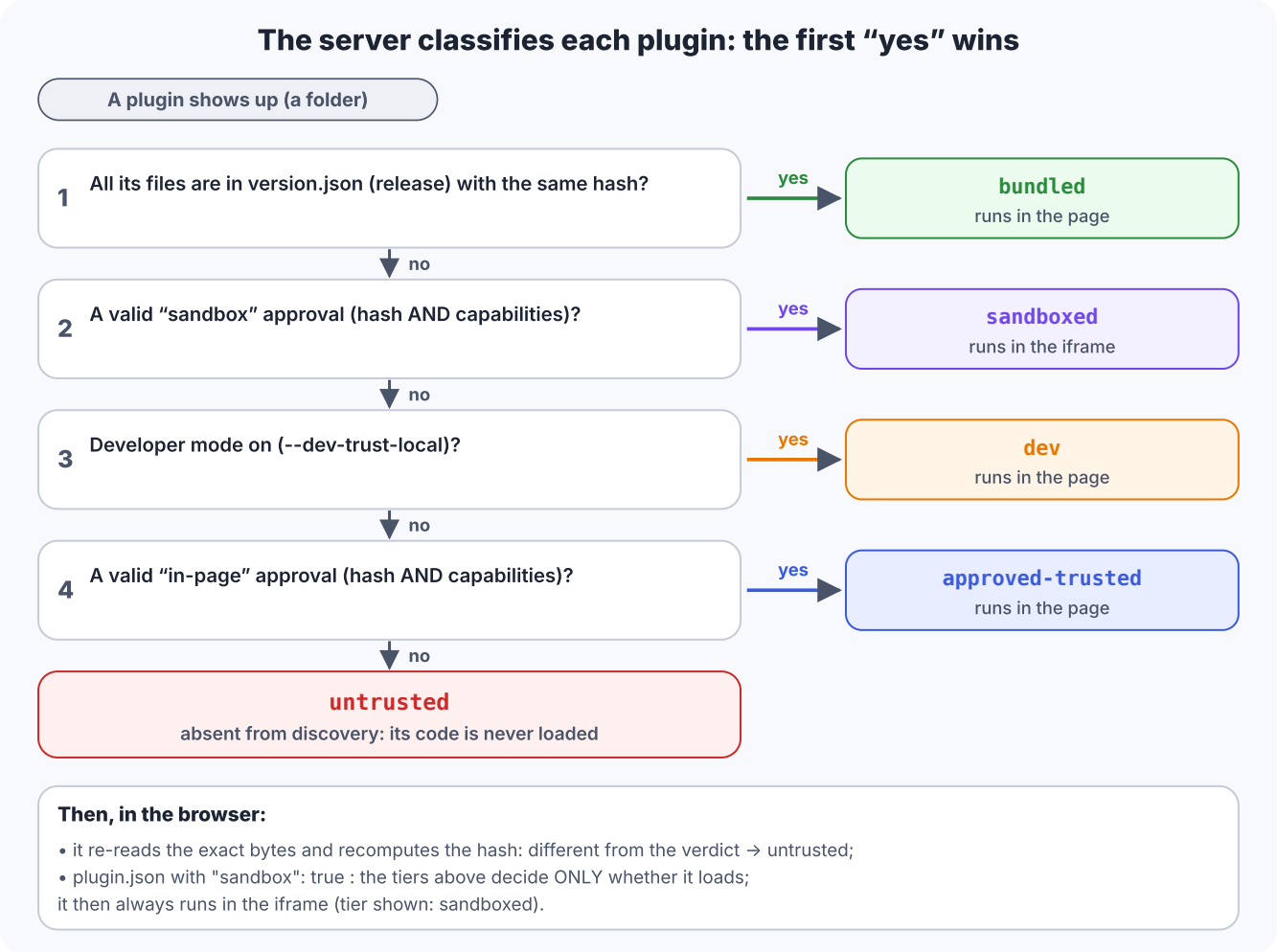
📖 **GOING FURTHER**

The complete guide for authors (channel and render-mode contracts, examples of exclusive tools, good practice) is `DOCS/plugins/guide-creation-plugins.pdf` (the `.tex` source is in the same folder). The tests of this chapter's rules: `tests/js/test_plugin_context_gate.mjs` , `test_plugin_datatype_gate.mjs` , `test_plugin_autonomy.mjs` , `test_plugin_lang.mjs` .

## 15.12 Why be wary of a plugin

A plugin is a JavaScript file: **real code that runs in the browser of your visitors and of the administrator**. Before version 1.6.0, a plugin had the **full powers of the page**: access to the whole interface, to cookies, and even to the calls reserved for administration. A malicious plugin could have taken control of the site.

Since then, the rule has been **reversed**: *a plugin is not allowed to run until it has been recognised.*




▲ The server applies these four questions in order; the first “yes” gives the trust level.

The five levels, from most to least trusted:

| LEVEL | HOW IT IS OBTAINED | WHERE IT RUNS |
|---|---|---|
| `bundled` | Every file of the folder appears, with **the same fingerprint**, in a release’s `version.json` | In the page |
| `sandboxed` | The operator approved it in **sandbox** mode (or the plugin declares `sandbox: true` ) | In an isolated **iframe** |
| `dev` | The server was launched with `--dev-trust-local` | In the page |

| LEVEL | HOW IT IS OBTAINED | WHERE IT RUNS |
|---|---|---|
| `approved-trusted` **("approved")** | The operator approved it in **in-page** mode | In the page |
| `untrusted` | **Everything else** | **Nowhere** |

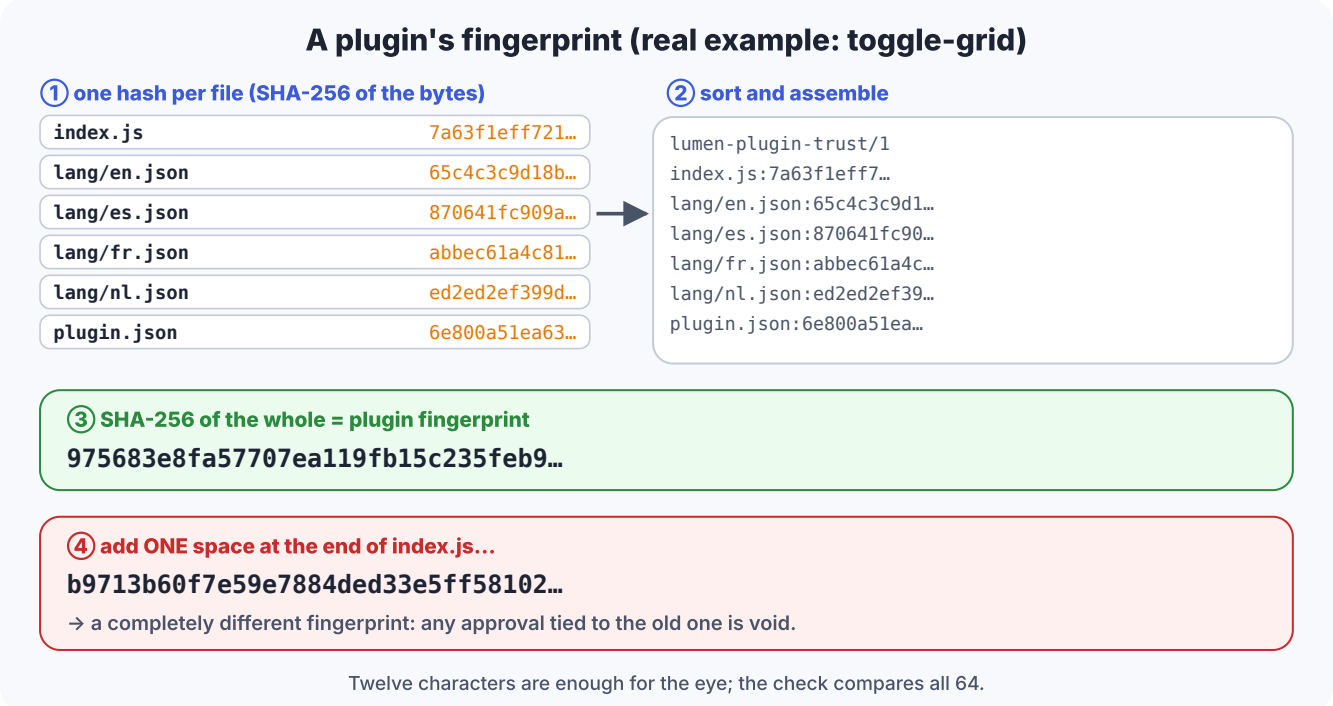
Three clarifications about the tree:

- Trust depends on the **content** of the files, never on their path or on the absence of a file. A plugin with the same name as an official plugin but different content is **not** `bundled`.
- A "sandbox" approval **wins** over developer mode: if the operator chose to lock a plugin up, a development machine does not free it.
- The `dev` level is a **positive signal** from the operator (a flag). The presence of a `.git` folder is not one: a production install behind a relay looks exactly the same. Without the flag, a Git clone has **no** trusted local plugin.

> 📌 **REMEMBER**
>
> **The server is the authority.** It classifies; the browser **re-checks**. The plugin itself has no say in the matter.

## 15.13 The fingerprint: a signature of the exact content

To recognise a plugin, the system computes its **fingerprint** (hash): a 64-digit hexadecimal number specific to its content.




▲ From content to 64 characters, and why a single space changes everything (real example).

💡 A PICTURE TO UNDERSTAND

**A digital wax seal.** Change one comma in the letter and the seal no longer matches. It is impossible to make two different plugins that give the same fingerprint.

The exact rules (identical in the three implementations: JavaScript, Python, PHP):

1. Every file of the folder with extension `.js`, `.json`, `.mjs`, `.css` or `.html` gets its SHA-256, computed on the **raw bytes** as served (no line-ending or BOM conversion). Hidden files (leading dot) are ignored; images do not count.
2. The lines `path:fingerprint` are formed, **sorted** by path.
3. The plugin's fingerprint is the SHA-256 of `lumen-plugin-trust/1` + a line feed + these lines.

The **translations** (`lang/*.json`) count: changing one sentence changes the fingerprint. A set of test vectors (`tests/plugin-trust-vector.json`) checks that the three implementations give exactly the same result, including on tricky cases (Windows line endings, BOM).

### Anti-TOCTOU: what is checked is what runs

❓ WHY THIS CHOICE?

**TOCTOU** stands for "time of check, time of use": a file is checked, **then** run, and in between it has changed. It is the classic trick: show a clean file to the inspector and a booby-trapped file at execution.

Lumen3D closes the gap as follows:

1. The browser downloads the bytes of each file **only once**, with no cache for anything that can run.
2. It recomputes the fingerprint on **those** bytes and compares it with the server's verdict. If different: `untrusted`, the plugin is not run.
3. It runs **those same bytes**, through a temporary `Blob` address, never by asking for the file again by its URL.
4. The `plugin.json` that shaped the toolbar and the barriers must be the document that was hashed, otherwise quarantine.

🔧 FOR THE CURIOUS

Three special cases, for honesty. (1) On an **insecure** address (HTTP on a local-network IP), the browser does not provide `crypto.subtle`: it cannot recompute the fingerprint and relies on the server's verdict, with a console warning. (2) On a **static** host with no trust authority, only `bundled` plugins are recognised, by comparison with `version.json`. (3) With no trust module loaded, nothing is run (fail closed).

## 15.14 Operator approval, and the sandbox

### Approving: tying a YES to precise content

When a plugin is dropped in by hand (by FTP, for example), it is `untrusted`. The administrator approves it in the **Plugins** tab (chapter 14): **Approve (sandboxed)** or **Approve (in-page)**.

The approval is **pinned**:

- the server **recomputes the fingerprint itself** from the disk and requires it to equal the one the operator saw (otherwise “the plugin’s content has changed”);
- the operator **types their password again**: approving is the only action that lets foreign code run;
- it also records the **capabilities** granted (next section);
- it is written to `api/plugin-trust.json` : `{ path, sha256, mode, caps, date, author }`.

If the content **or** the requested capabilities change, the approval **lapses** (“approval cancelled: content or capabilities modified”) and the plugin becomes `untrusted` again. The message is not a fault: it is the system doing its job.

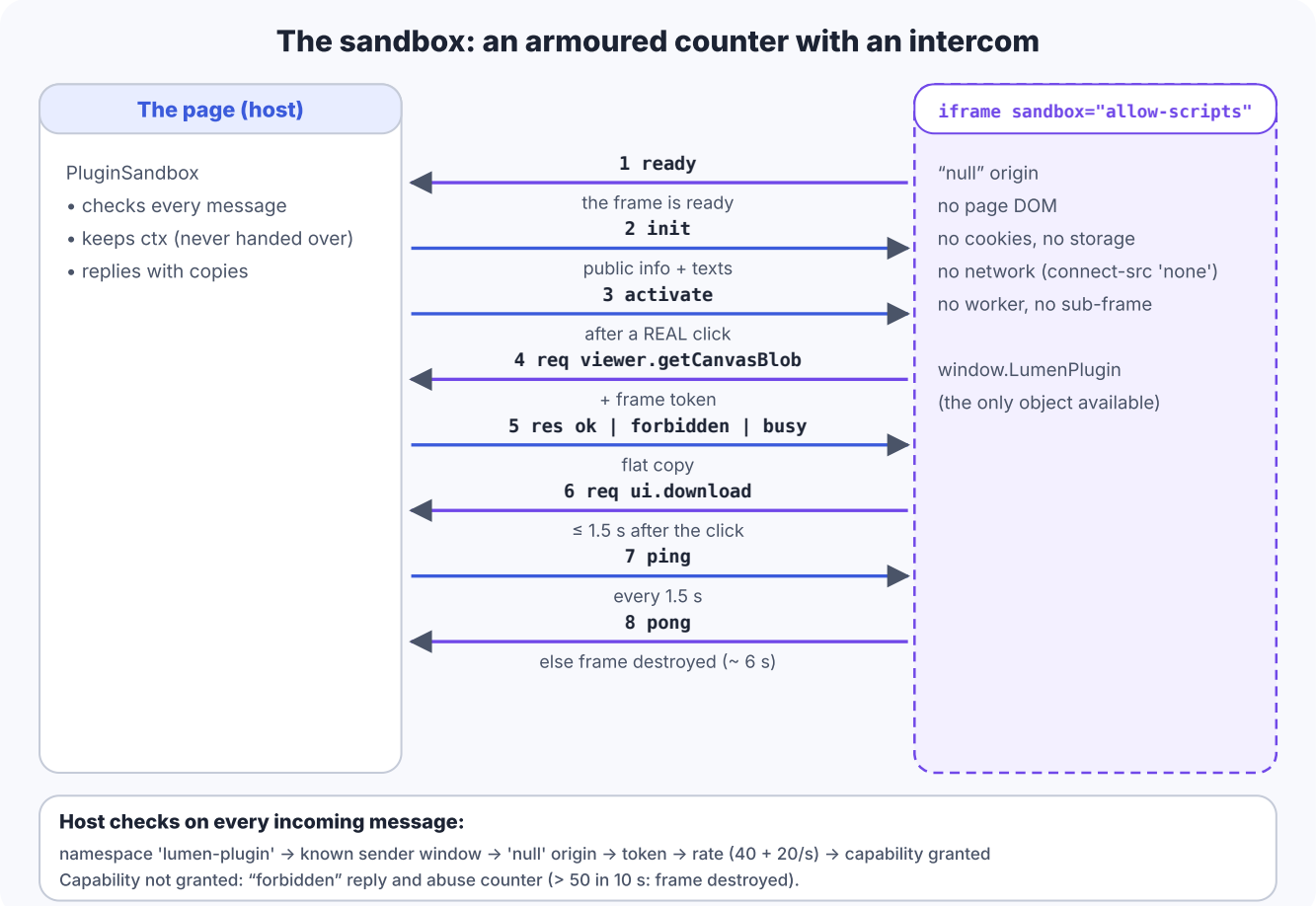
> 🔧 **FOR THE CURIOUS**
>
> The file `api/plugin-trust.json` is **never served** over HTTP, it is protected against updates, and a release that contained one is **refused**: a booby-trapped version therefore cannot pre-approve an attack plugin. Every approval, revocation, installation or uninstallation increments a counter, `trustEpoch` .

### Revocation on the fly

A viewer that is **already open** polls the server ( `api/health` , every 8 seconds and when you return to the tab). If `trustEpoch` has changed, it asks again for the list of recognised plugins; any sandboxed plugin that is no longer on it is **destroyed on the spot** and its button disappears.

For an “in-page” plugin, code that has already run cannot be undone: the revocation takes effect **at the next load**. And on a PHP host, which does not emit `trustEpoch` , revocation also takes effect only at the next load.

## The sandbox: an armoured counter with an intercom




▲ An empty iframe, a monitored intercom, eight message types.

💡 **A PICTURE TO UNDERSTAND**

**An armoured counter.** The plugin works in a closed booth behind it. It cannot see the room (the DOM) and has no telephone (no network). It has an intercom: it asks a question, the clerk (the page) checks that it is entitled to, then answers with **a copy** of the information, never the original.

**The booth**: an `<iframe sandbox="allow-scripts">` **without** `allow-same-origin`. The browser gives it a “null” origin: no access to the page’s DOM, cookies, storage or administration calls. An internal policy (`default-src 'none'`, `connect-src 'none'`) forbids any network, worker or sub-frame. A continuous-integration test rejects any change that would add `allow-same-origin`.

**The intercom**: `postMessage` in a `lumen-plugin` namespace, never confused with messages between pages. Each message carries `{ ns, v, dir, id, plugin, token, type, payload }`.

**The plugin’s only object**: `window.LumenPlugin`.

| PLUGIN CALL | CAPABILITY REQUIRED | EFFECT |
|---|---|---|
| `addButton({label, icon})` | `toolbar.addButton` | Declares its button |
| `toast(text)` | `ui.toast` | Message (200 characters at most) |
| `getInfo()` | `viewer.getInfo` | Name, dimensions, voxel, number of channels |

| PLUGIN CALL | CAPABILITY REQUIRED | EFFECT |
|---|---|---|
| `getCanvasBlob(opts)` | `viewer.getCanvasBlob` | PNG or JPEG image of the view (one request at a time) |
| `download(name, type, data)` | `ui.download` | Download, **on a real click only** |
| `setRenderMode(mode)` | `viewer.setRenderMode` | Change render mode (known modes only) |
| `getChannels()` | `channels.getState` | Copy of the channel state |
| `on('render' · 'channels-updated' · 'camera', f)` | `events.subscribe` | Receive events, as **flat copies** |
| `saveState(s)`, `t('key')` | (none) | Memory for the workspace (256 KiB at most), translation |

The plugin declares in `sandboxCapabilities` what it needs; the operator grants a subset; the **effective** capabilities are the **intersection** of three lists: what the disk declares, what the operator approved, and the closed list of the eight capabilities in the table.

**The checks on every message received**, in this order:

1. the namespace is `lumen-plugin`;
2. the **sending window** is that of a known booth (this is the main authentication: the window's identity is trusted, not the declared origin);
3. the origin is indeed "null";
4. the message's shape and the frame's random **token** are correct;
5. the **rate** is respected: a reserve of 40 messages, refilled at 20 per second;
6. the capability requested has been **granted**; otherwise the answer is `forbidden`.

**Lifetime safeguards**:

| SAFEGUARD | VALUE |
|---|---|
| **Booth start-up (`init`)** | 10 s at most, otherwise destroyed |
| **Heartbeat (`ping`)** | every 1.5 s; booth destroyed after about 6 s without an answer |
| **Abuse (invalid or forbidden messages)** | more than 50 in 10 s: destroyed |
| **Download** | one per activation, within 1.5 s of a real click, 32 MiB at most, PNG / JPEG / JSON / CSV / text types, extension imposed by the type |

> ⚠️ **WARNING**
>
> **Only "action" and "toggle" tools can be locked in.** A render mode compiles GLSL code on the graphics card at every frame: there is no asynchronous boundary behind which a booth could stand. A channel plugin receives a piece of the page's DOM directly, exactly the privilege the sandbox removes. These two placements therefore stay in **full trust** (reserved for official plugins), and approving one of them in sandbox mode puts it in quarantine.

ℹ️ GOOD TO KNOW

A plugin declared `sandbox: true` **always runs in the booth**, whatever its trust level. Conversely, a plugin written for the page (which calls `PluginRegistry`) cannot work in the booth, which has only the SDK. The reference plugin is `screenshot-sandboxed`: it captures and downloads without ever touching the DOM or the context.

## 15.15 Compatibility: `platformCompat`

A plugin says which platform versions it works with. If the platform changes version, this field stops a plugin that is too old (or too new) from breaking the interface.

| DECLARATION | MEANING | PLATFORM 1.59.2 |
|---|---|---|
| **absent** | No constraint | compatible |
| `"*"` or `"x"` | Any version | compatible |
| `">=1.53.0"` | At least this version | compatible |
| `">=1.60.0"` | | **refused** |
| `"^1.4.0"` | From 1.4.0 inclusive to 2.0.0 exclusive | compatible |
| `"~1.58.0"` | From 1.58.0 inclusive to 1.59.0 exclusive | **refused** |
| `"1.x"` or `"1.59"` | Prefix | compatible |
| `["1.58", "1.59.x"]` | List: **any one** of the values is enough | compatible |
| `">=1.51.0 <1.59.0"` | Several comparators: **all** must hold | **refused** |
| `">=banana"`, `42` | Unreadable | **refused** |

Rules: a string separated by spaces is an **AND**; an array is an **OR** (“bare” values only, no operator); three numbers = exact version, fewer numbers = prefix; a `-rc1` suffix is ignored.

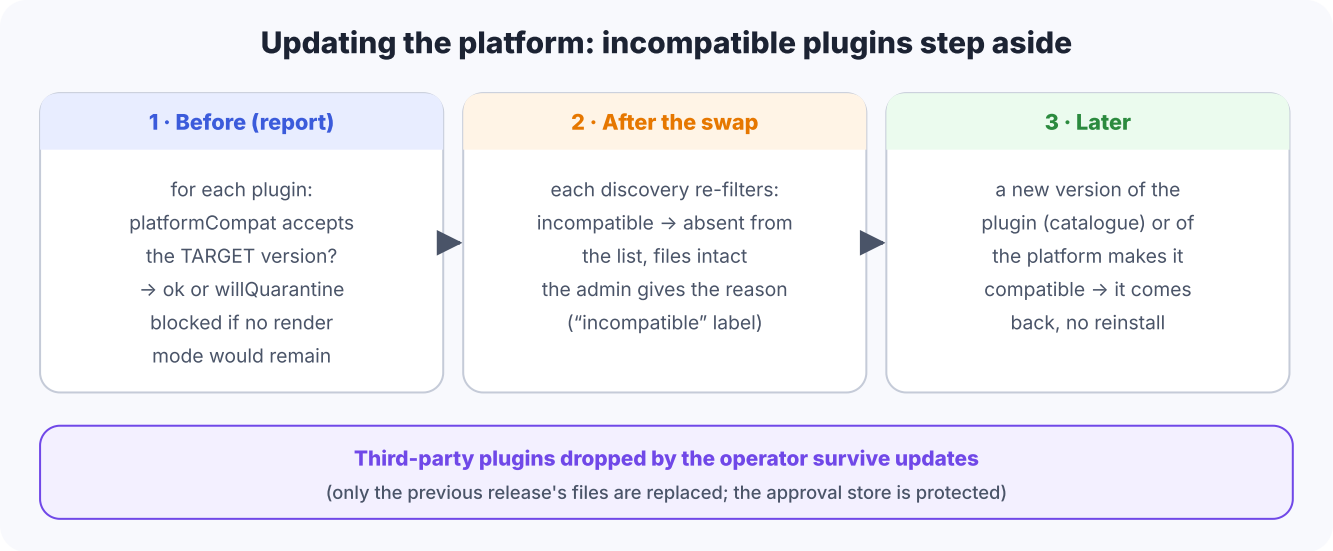
📌 REMEMBER

**Unreadable = incompatible** (fail closed). A declaration that cannot be read never lets anything through. The **only** exception: if the platform’s version is unknown (a clone with neither `version.json` nor a development server), the barrier is neutralised and says so in its reason.

🔧 FOR THE CURIOUS

**Three twins, one behaviour**: `js/core/compat.js` (browser), `dev_server.py` (`_compat_satisfies`), `api/_admin_lib.php` (`admin_compat_satisfies`). They are all validated against `tests/compat-vector.json` (44 cases); any change of semantics must pass all three. The platform version is read from `version.json` (releases), otherwise from `api/health` (development server).

## 15.16 Updating the platform: reversible erasure




▲ Before, after, later: an incompatible plugin is never deleted.

When the administrator prepares an update (chapter 18), the **verification report** applies `platformCompat` to the **target** version, plugin by plugin, **before** anything is installed:

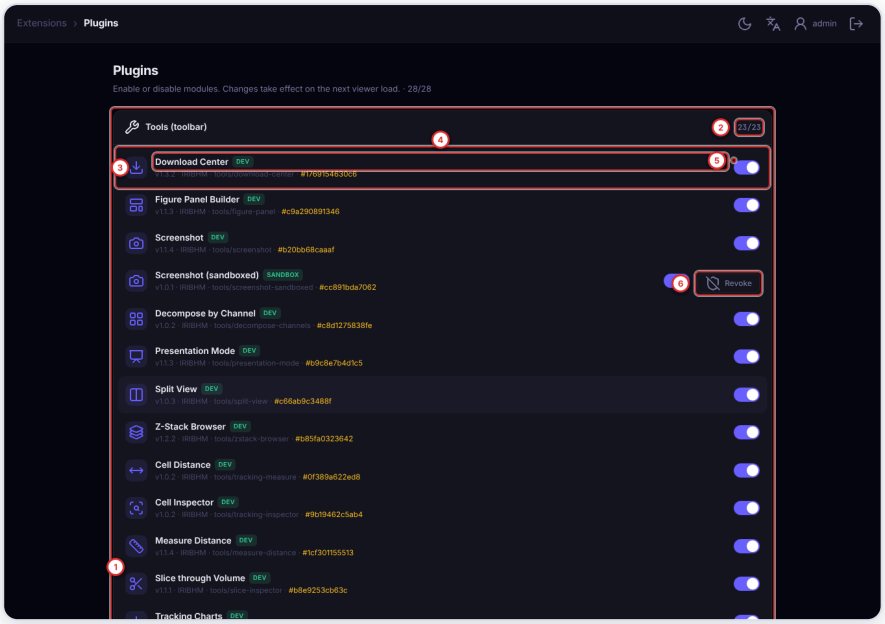
- **compatible**: nothing to do;
- **quarantined** ( `willQuarantine` ): they would no longer work;
- **blocking**: if no **render mode** would remain compatible, the viewer would be unusable: the update cannot be confirmed.

After the switch, nothing is deleted. Discovery **re-filters on every request**: an incompatible plugin disappears from the list (its files stay), and **comes back by itself** as soon as a compatible version of the plugin or of the platform is installed. A plugin dropped in by hand by the operator survives the update: only the files of the **previous release** are replaced.

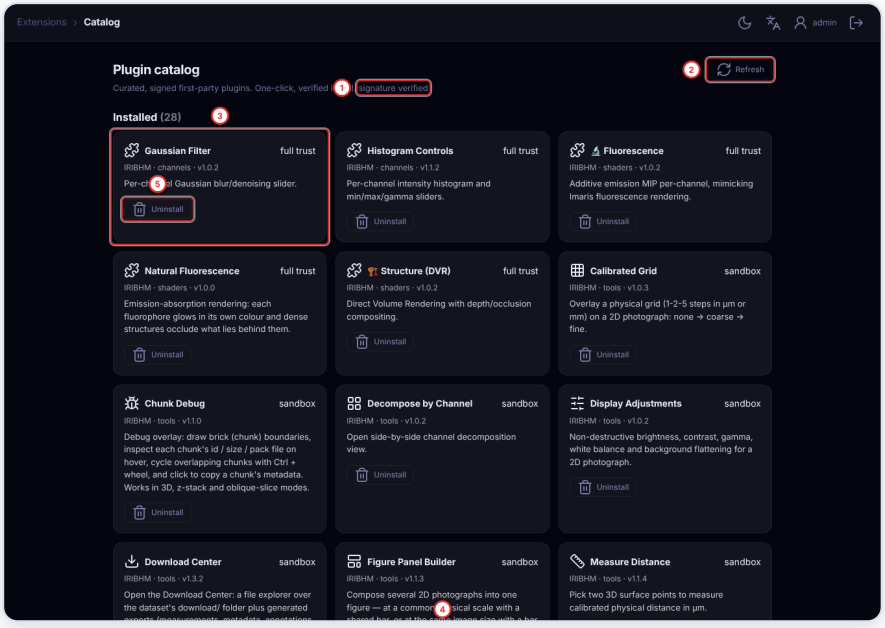
> ℹ️ **GOOD TO KNOW**
>
> Official plugins do **not travel** with the platform: a fresh install starts with no plugin and installs them from the catalogue (§15.17). Compatibility is what keeps this separation safe.

## 15.17 The signed catalogue: the official store

The **catalogue** is the list of plugins the platform offers to install, from the **Catalog** tab or from the first-install wizard (the “recommended” plugins are pre-ticked there). It is **curated**: only the platform’s publisher publishes to it.




▲ The administration Plugins tab: three cards (tools, channels, render modes), a switch and a trust level for each plugin (“dev” on this demonstration machine).




▲ The Catalog tab: “signature verified” at the top, one card per plugin with its mode (“full trust” or “sandbox”).

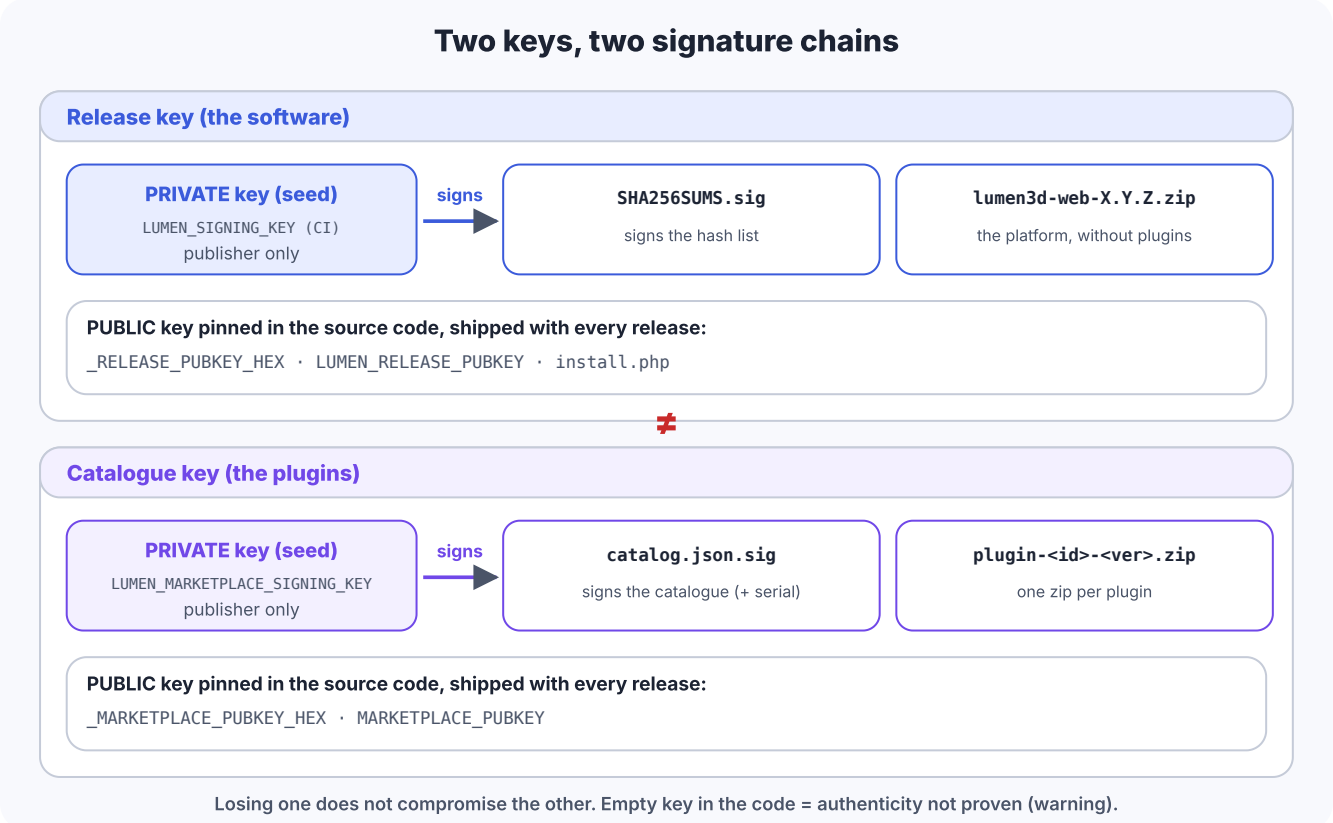
### What the catalogue contains

A file `marketplace-catalog.json`, signed by `marketplace-catalog.json.sig`. It carries `version`, a **serial number** (`serial`), the issue date (`issuedAt`) and one entry per plugin:

| ENTRY FIELD | ROLE |
|---|---|
| `id`, `name`, `placement`, `subtype`, `description`, `creator` | The card displayed |
| `latestVersion`, `platformCompat` | Latest version, compatibility |
| `sandboxCapabilities`, `recommended` | Capabilities requested; pre-ticked or not at first install |
| `assetUrl`, `sumsUrl`, `sigUrl` | Where to find the zip, its fingerprints and their signature |
| `sha256` | The **zip's** fingerprint (authenticated by the catalogue's signature) |

Each plugin is published as a **deterministic** zip (sorted entries, date fixed at 1980, permissions 0644: the same content always gives the same bytes) accompanied by `SHA256SUMS`, `SHA256SUMS.sig` and `version.json`.

## Two keys, never the same




▲ Two independent signature chains: compromising one gives nothing on the other.

💡 **A PICTURE TO UNDERSTAND**

**Two different wax seals.** One authenticates the software's official letters, the other the store's. Stealing or losing one does not let anyone forge the other, and one can be changed without touching the other.

- The signature is an **Ed25519** signature (RFC 8032 standard), verified by a purpose-written module with no dependencies in Python, and by the libsodium library in PHP.
- The **private key** (the seed) never leaves the publisher: environment variable or file outside the Git repository.
- The **public key** is **pinned in the source code** ( `dev_server.py` , `api/_admin_lib.php` , `install.php` for releases): it travels with every version and survives updates. A server trusts only what verifies with that precise key.
- **Empty** key in the code: the signature cannot be proven, only the fingerprint is checked, with a warning. **Key filled in**: a missing or false signature **refuses** the catalogue.

### Anti-rollback: the serial number

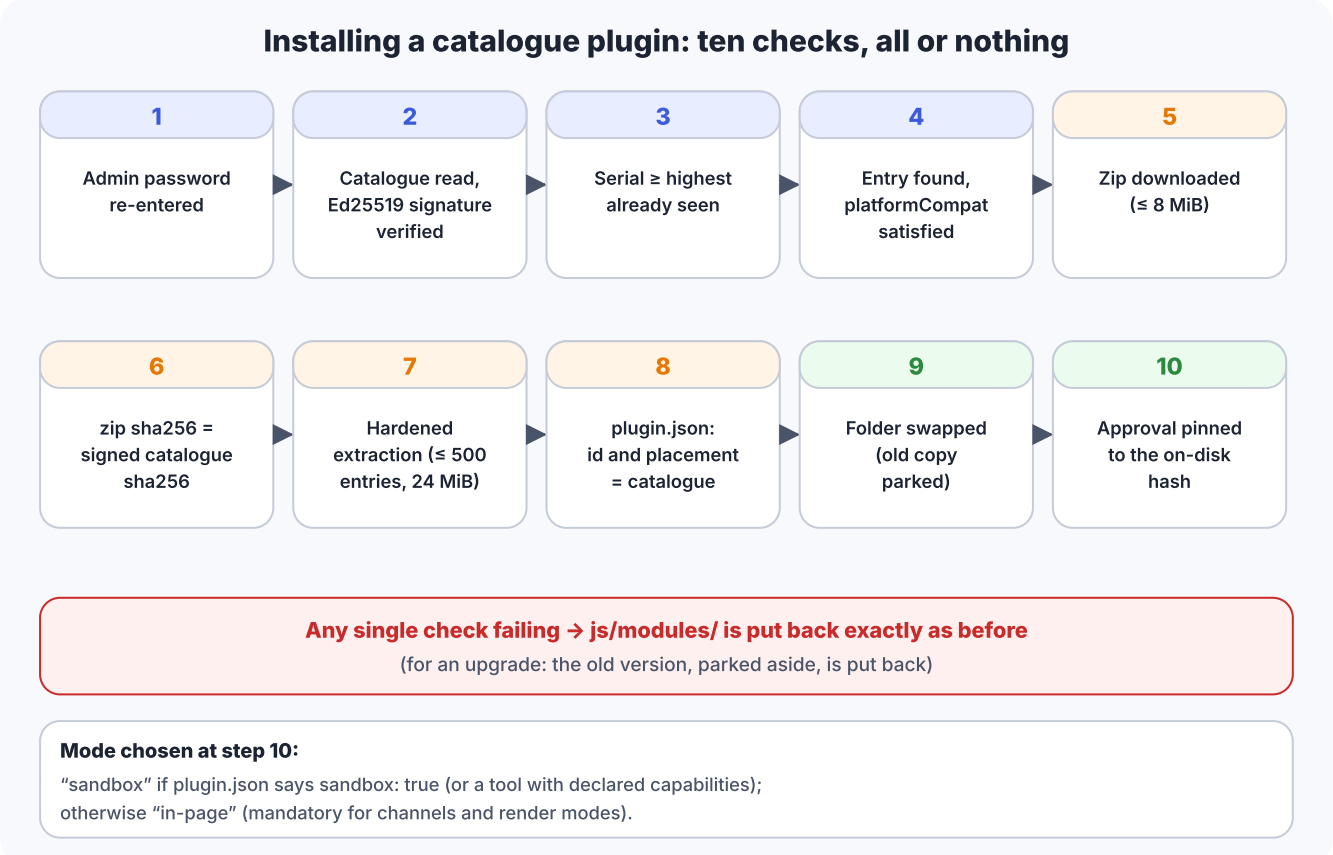
A stale but still **validly signed** catalogue could be used to reinstall a version of a plugin whose flaw has since been fixed. Each publication therefore **increases** the `serial` . Each server remembers the highest number it has accepted ( `api/marketplace-state.json` ) and **refuses** a catalogue with a lower number. Only a catalogue whose signature is proven can raise this counter: a false, gigantic number cannot freeze a server.

The administrator then sees “Catalogue refused: it is older...”: there is nothing to do on their side.

🔢 **WORKED EXAMPLE**

At the time of writing, the repository’s catalogue is at `serial` 2. A server that has accepted 2 will refuse a catalogue with serial 1, even if its signature is perfectly valid.

## 15.18 Installing, updating, uninstalling




▲ Ten checks; at the slightest error, everything is put back as it was.

Installation is started **by the operator** (password re-entered), never automatically. The steps, in the order of the code:

1. password verified;
2. the catalogue is downloaded (1 MiB at most) and its **signature** verified;
3. its **serial** is compared with the highest already seen;
4. the entry is found, `platformCompat` is compared with the platform version ("incompatible" otherwise);
5. the zip is downloaded, **8 MiB at most**;
6. its SHA-256 must equal that in the **signed catalogue**; without a key, it is compared with `SHA256SUMS` (itself verified);
7. **hardened extraction**: no more than 500 entries or 24 MiB, no absolute path, none with `..`, backslash or drive letter; only plugin file extensions are accepted (never `.php`, `.py`, or a hidden file: the folder is served to the web);
8. `plugin.json`: the `id` and `placement` must equal those of the catalogue;
9. the folder is installed; in the case of an **update**, the old copy is **set aside**, not deleted;
10. the server **recomputes the folder's fingerprint** and **approves** the plugin: `sandbox` if it declares `sandbox: true` (or if it is a tool with capabilities), otherwise `in-page`.

If **any** step fails, the `js/modules/` folder is put back **exactly** in its initial state, including the old version in the case of an update.

**Uninstalling** deletes the folder **and** the approval. The operation is idempotent, and refuses to remove the **last active render mode**.

> ⚠️ **WARNING**
>
> A "full trust" plugin installed from the catalogue **runs in the page**. It is approved because **the publisher signed it and the server verified the signature**, not because it would be harmless. This is also why the catalogue is never opened to third parties.

## 15.19 Publishing a plugin (publisher side)

A single command prepares and publishes:

```
python tools/publish_plugin.py path/to/my-plugin --push
```

It packages and signs the plugin, adds its entry to the catalogue, **increases the `serial`**, **re-signs** the catalogue and then, with `--push`, commits only the `marketplace/` folder. A plugin is **online** only when these files are on the `main` branch (the catalogue is served from GitHub, `main` branch). The tool refuses a seed whose public key is not the one pinned in the code, because every server would reject the catalogue.

Other options: `--recommended false` (published but not pre-ticked), `--remove <id>` (unpublish), `--resign` (reissue the catalogue under the next serial).

To change a plugin that is already published: **raise `version`** (and raise `platformCompat` if the plugin uses a recent core function with no fallback). Republishing unchanged content produces nothing.

## 15.20 The core modules that make all this work

None of these files is a plugin: they **run** the plugins.

| MODULE | ROLE |
|---|---|
| `js/core/plugin-registry.js` ( `PluginRegistry` ) | Discovery, barriers, life cycle, toolbar, quarantine, revocation on the fly, workspace state |
| `js/core/plugin-trust.js` ( `PluginTrust` ) | Fingerprint, re-reading the bytes, verdict (client twin of the server) |
| `js/core/plugin-sandbox.js` ( `PluginSandbox` ) | Create the booths, capability broker, rate limit, heartbeat |
| `js/core/compat.js` ( `Compat` ) | `platformCompat` resolver; platform version |
| `js/core/tool-manager.js` ( `ToolManager` ) | **Only one exclusive tool** active at a time; keyboard shortcuts; defaults `v` /Esc (navigate), `c` (slice), `m` (measure) |
| `js/core/ui-actions.js` ( `UiActions` ) | Header buttons through a `data-action` attribute rather than code in the HTML (the security policy forbids inline code) |
| `js/core/i18n.js` ( `I18n` ) | Grafts plugin dictionaries under `plugins.<id>`, falls back to English |
| `js/core/export-manager.js` , `workspace-state.js` | Consumers of `getExports()` and `getState()` |
| `js/components/channel-panel.js` | Host of channel plugins ( `getChannelUI` , `bindChannelUI` ) |
| `js/pages/viewer.js` , `js/pages/2d.js` | Build the `ctx` and call the registry in the order of §15.3 |

On the server side: `dev_server.py` ( `_list_plugins` , `_classify_plugin` , `_approve_plugin` , `_marketplace_*` , `_install_marketplace_plugin` ) and their PHP twins in `api/_admin_lib.php` and `api/plugins.php` . The **list** of utility modules unrelated to plugins ( `Utils` , `Catalog` ...) is in the appendix, chapter 20.

## 15.21 A plugin does not show up: what to check?

1. Open the browser **console**: every quarantine writes `[PluginRegistry] Quarantined "tools/…" (reason): detail` there.
2. Reason `untrusted` : **Plugins** tab, approve it (or it has changed since its approval).
3. Reason `incompatible` : compare its `platformCompat` with your version (§15.15).
4. No quarantine message, just “left out”: it does not target this page ( `dataTypes` , `contexts` , §15.5).

5. Button missing but plugin loaded: an unsatisfied `requires` (no tracking, no bricks), or a `group` with no cluster.

6. Missing from the Plugins tab: the folder is badly named, or its `plugin.json` is unreadable or contradicts the parent folder.

📌 **REMEMBER**

- A plugin = a folder ( `plugin.json` + `index.js` + `lang/` ); the **parent folder** fixes its placement.
- The page **discovers, filters, verifies, then initialises**, always in this order; a broken plugin never costs you the viewer.
- **Default refusal**: a plugin is run only if it is `bundled` , approved, or developer mode is on.
- Approval is **tied to the exact content** (fingerprint) and to the capabilities; what is checked is what is run.
- The **sandbox** locks in action and toggle tools; channels and render modes stay in full trust.
- The **catalogue** is signed by **its own key**, protected against rollback (serial), and installed **all or nothing**.

# 16. Customising and translating

- Lumen3D **repaints itself in your colours without a line of code**: name, vocabulary, theme, pages, legal notices. It all fits in a few small JSON files in the `config/` folder.
- The site speaks **four languages** (English, French, Spanish, Dutch) with **1,943 texts** per language; adding a fifth language means dropping in a file.
- The site also looks after its visitors: a **light or dark theme** with no flicker, **colour-blindness simulation**, use on a phone, and **statistics with no personal data**.

📖 **GOING FURTHER**

This chapter explains the **mechanics**: where each setting lives and how it reaches the screen. The step-by-step for each tab is in the "Administrator guide" (chapters 6 to 11); the overview is in chapter 14.

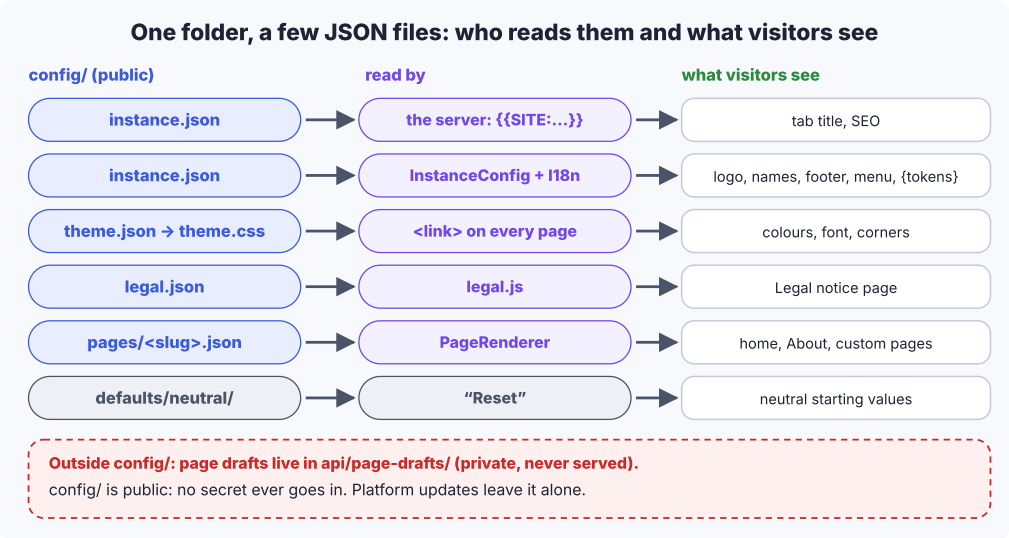
## 16.1 One platform, many homes

Lumen3D was born to look at mouse embryos. Today it serves other objects: organs, samples, tissues. The **engine** does not change; what visitors read does.

💡 **A PICTURE TO UNDERSTAND**

**A hotel chain.** The walls, the wiring and the lift are the same everywhere (the engine). The reception, the sign, the restaurant menu and the decoration carry the name of the establishment (the configuration).

Everything specific to *your* site lives in the **`config/`** folder:




▲ The config/ folder: each file, who reads it, and what the visitor sees of it.

📌 **REMEMBER**

`config/` is **public**: any visitor can read these files. So **never put a secret** in it (passwords live in `api/`, which is never served).

### The files, one by one

| FILE | WHAT IT CONTAINS | EDITED IN THE TAB... |
|---|---|---|
| `instance.json` | name, vocabulary, SEO, footer, menu, type names, variables | **Identity**, **Data types**, **Pages** |
| `theme.json` | the colour, font and corner values you changed | **Appearance** |
| `theme.css` | the style sheet **compiled** from `theme.json` | (generated, never by hand) |
| `legal.json` | the sections of the legal notice, per language | **Legal** |
| `pages/<slug>.json` | one page per file: its published version | **Pages** |
| `defaults/neutral/` | the neutral starting values, shipped with the platform | **Reset** button |
| `uploads/` | the images sent from the page editor | page editor |

Next to it, outside `config/`: page **drafts** live in `api/page-drafts/` (private, see 16.6).

🔧 **FOR THE CURIOUS**

**Updates and starting values.** `instance.json`, `theme.json`, `theme.css`, `legal.json` and `pages/` are on the list of what a platform update never touches: your settings survive. Uploaded images are part of no shipped version, so they are never replaced. The files in `defaults/neutral/`, on the other hand, travel with every version.

**The floor with no file.** If `instance.json` cannot be found or read, the browser falls back on a neutral vocabulary built into the code ("Lumen3D", "sample"). The server lets the fallback texts written in the pages play. The site does not break.

**Images.** They are limited to 8 MiB and to the PNG, JPG, WebP, GIF and AVIF formats. **SVG is deliberately refused**: served from the same domain, it could contain a script.

## 16.2 `instance.json`, field by field

This is the richest file. Here is what each block really controls.

### Identity and organisation

| FIELD | WHERE IT APPEARS | SET IN |
|---|---|---|
| `brand.name` | `{brand}` token; title of the `page.html` page | **Identity** · *Instance name* |
| `brand.shortName` | `{brandShort}` token; title of the administration panel | *Short name* |
| `brand.productName` | `{product}` token | *Product name* |

| FIELD | WHERE IT APPEARS | SET IN |
|---|---|---|
| `brand.monogram` | the letters of the logo in the top bar (home, About…) | *Monogram* |
| `brand.logoEmoji` | the logo icon of the Explorer and of the administration panel | *Logo emoji* |
| `brand.organization` | **the text next to the logo** in the top bar; `{orgShort}` token | *Organization* |
| `brand.tagline` | `{tagline}` token (per language) | *Tagline* |
| `org.name` | `{org}` token | (first-install wizard) |

🔢 **WORKED EXAMPLE**

In the demonstration, the top bar shows "**IRIBHM — ULB**": that is `brand.organization`, not `brand.name` ("IRIBHM Microscopy Platform"). The monogram "**IR**" is in the green badge.

## Vocabulary, search-engine optimisation, footer, menu

| FIELD | WHERE IT APPEARS | SET IN |
|---|---|---|
| `specimen.singular`, `specimen.plural` | `{specimen}`, `{specimenPlural}` tokens and their capitalised versions | *Terminology* (per language) |
| `datasetTypes.<type>.label`, `.title` | badges, filters, home-page cards | **Data types** |
| `seo.description`, `seo.keywords` | the `<meta>` tags of every page | *Tagline & SEO* |
| `pageTitles.<page>` | the browser-tab title of each page (`home`, `explorer`, `viewer`, `compare`, `2d`, `about`, `admin`, `legal`) | (file) |
| `footer.copyright` | the "© …" line of the footer | *Footer* |
| `nav.showExplorer`, `showCompare`, `showAbout` | show or hide the link in the bar | *Navigation* |
| `nav.showLegal` | adds the **Legal** link **in the footer** | *Navigation* |
| `nav.customPages` | the pages you created (`slug`, label, visible) | added at first publication |

ℹ️ **GOOD TO KNOW**

**Unticking hides the link, not the page.** If you untick **Show "Compare"**, the address `compare.html` stays valid: only the link disappears from the bar. The pages you create are added after the links, in list order, with the label in the visitor's language.

### The organization link and the footer links

ℹ️ GOOD TO KNOW

- `footer.links` (*Footer → Links*): the public footer (home, Explorer, About, Legal, custom pages) shows your links **before** **Legal**. Only `http(s)://`, `mailto:` and relative addresses are accepted; an external link opens in a new tab.
- `org.url` (*Organization link*): after the copyright, the footer adds " · " followed by the organization's name, linked to that address (the address's host name if no organization name is set; nothing if `org.url` is empty).
- A copyright entered **per language** is displayed in the visitor's language.

### Settings with no administration screen

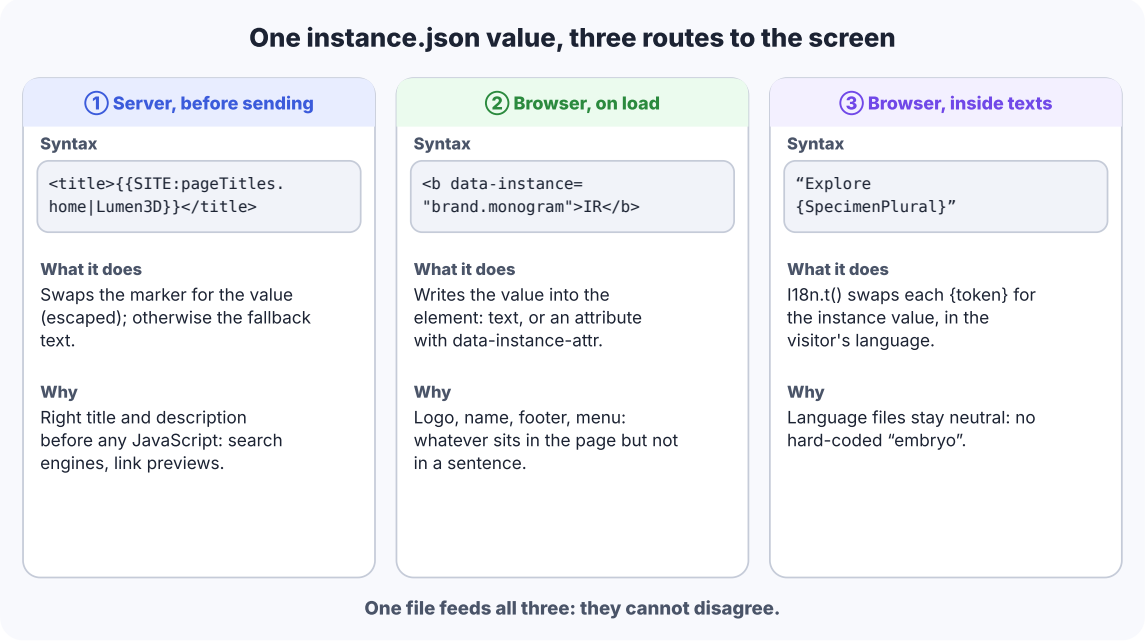
| FIELD | WHAT IT IS FOR |
|---|---|
| `channelColorPresets` | the starting colour of a channel according to its name (see 16.8) |
| `variables` | your custom `{name}` variables (managed in the page editor, Variables tab) |

ℹ️ GOOD TO KNOW

Each file carries its version identifier ( `"$schema": "lumen3d-instance/1"`, `lumen3d-theme/1`, `lumen3d-legal/1` ).

## 16.3 How a value reaches the screen

Your institution's name appears in many places: in the browser tab, in the top bar, in the welcome sentence. Three mechanisms share the work, **all fed by the same file**.




▲ Three paths, one file: they cannot contradict each other.

### ① The server fills in the header

In every HTML page, the title and the description tags contain **markers**:

```
<title>{{SITE:pageTitles.home|Lumen3D — 3D Imaging Data Viewer}}</title>
```

Before sending the page, the server replaces `{{SITE:path|fallback}}` with the value found in `instance.json`. If there is nothing (or not a character string), it puts the **fallback text** written after the `|`.

> ❓ **WHY THIS CHOICE?**
>
> **Why on the server?** Search engines and link previews (messaging apps, social networks) read the header **without running any JavaScript**. Filling in the header before sending gives the right title from the very first byte, with no flicker.

> 🔧 **FOR THE CURIOUS**
>
> - The value is **escaped** ( `& < > "` ) before entering the page: a name containing an angle bracket cannot break the HTML.
> - Only **strings** are substituted: a "per-language" value ( `{ "en": …, "fr": … }` ) is not resolved here (the server does not know the visitor's language); the browser resolves it afterwards.
> - The file is re-read only when it changes (its modification date is the key). Both servers (Python and PHP) apply the same rule.

### ② The browser fills in the page

The logo, the footer and the menu carry `data-instance` attributes:

```
<span data-instance="brand.organization">Lumen3D</span>
<a data-instance-attr="title:brand.tagline">…</a>   <!-- variant for an attribute -->
```

On load, `InstanceConfig` reads `instance.json` and writes the values: `data-instance` fills an element's **text**, `data-instance-attr` fills **attributes** ("attribute:path", separated by semicolons; the shipped pages do not need it today, but the mechanism is ready). It also applies the titles, the `<meta>` tags and the menu entries ( `nav.*` ).

> 🔧 **FOR THE CURIOUS**
>
> The text between the tags is the **fallback**: if the file cannot be found (offline, fresh install), the page shows this fallback instead of a hole. The application can be called again (after a language change, or when the administration preview reloads the configuration).

### ③ Tokens in sentences

The language files **never contain** the word "embryo". They write a **token** between braces, which `I18n.t()` replaces at display time:

| TOKEN | COMES FROM | VALUE IN THE DEMONSTRATION |
|---|---|---|
| `{brand}` | `brand.name` | IRIBHM Microscopy Platform |

| TOKEN | COMES FROM | VALUE IN THE DEMONSTRATION |
|---|---|---|
| `{brandShort}` | `brand.shortName` | Lumen3D |
| `{product}` | `brand.productName` | Lumen3D |
| `{tagline}` | `brand.tagline` | Confocal Imaging Data Viewer |
| `{org}` | `org.name`, otherwise `brand.organization` | IRIBHM — Université Libre de Bruxelles |
| `{orgShort}` | `brand.organization`, otherwise `org.name` | IRIBHM — ULB |
| `{specimen}`, `{specimenPlural}` | `specimen.singular`, `.plural` **in the visitor's language** | embryo, embryos |
| `{Specimen}`, `{SpecimenPlural}` | the same, **first letter capitalised** | Embryo, Embryos |
| `{type3d}`, `{type2d}`, `{typeLive}` | the displayed name of each type | 3D, 2D, Live |

Thirteen tokens in all. The page editor lists them (in the **Variables** tab), with a button to copy each one:

🔧 FOR THE CURIOUS

**Order of resolution of a token**: first a value passed by the calling code ( `{count}` , `{name}` ...), then the instance token, otherwise the brace **stays as it is** on screen (an unknown token shows).

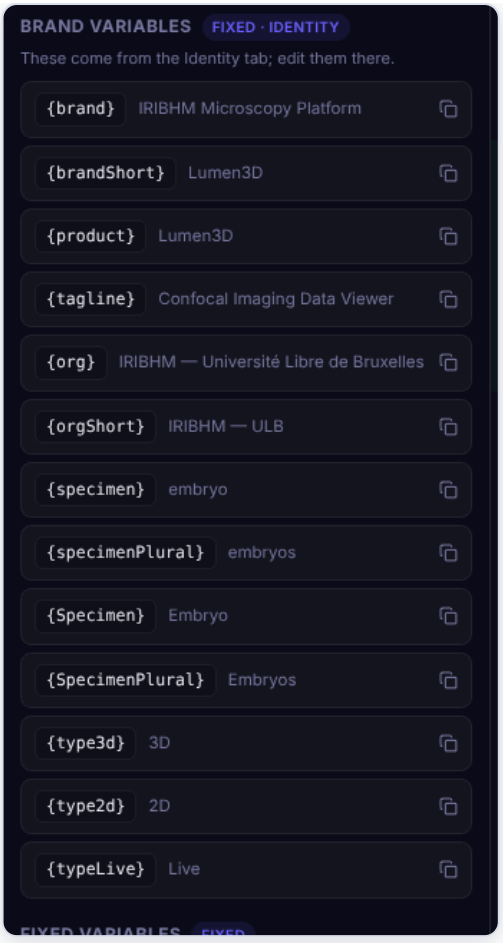
**A trap avoided.** The default type names themselves live in the language files ( `types.3d.label` ...). Reading them through `I18n.t()` would make the function that serves `t()` call `t()` : an endless loop. The code therefore reads this string through `I18n.raw()` , the same lookup *without* token replacement.

🔢 **WORKED EXAMPLE**

The key `landing.heroTitle` is `Explorez les {specimenPlural}` in French and `Explore {SpecimenPlural}` in English.

- with `specimen.plural.en = embryos` → “**Explore Embryos**”;
- if the lab writes “organoids” → “**Explore Organoids**”, without touching a single language file.

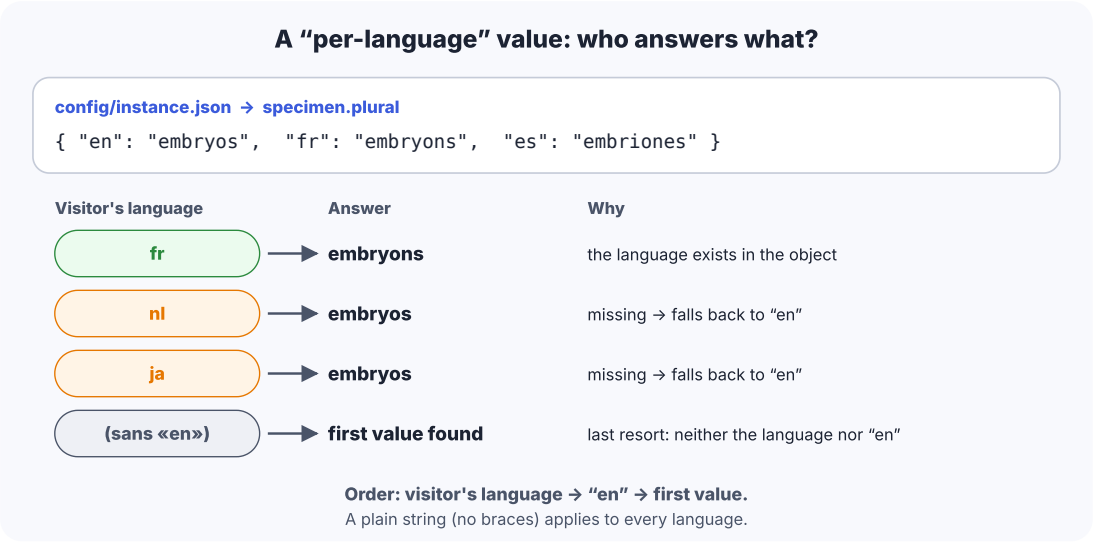
The same tokens work in the **page texts** you write (16.6). The editor adds **dynamic variables** there: year, date, time, and the catalogue counters.




▲ The instance tokens, seen from the page editor (demonstration dataset).

## A “per-language” value

A specimen’s name, a tagline, an SEO description: these texts depend on the language. They are written either as **a simple string** (identical everywhere) or as an **object per language**.

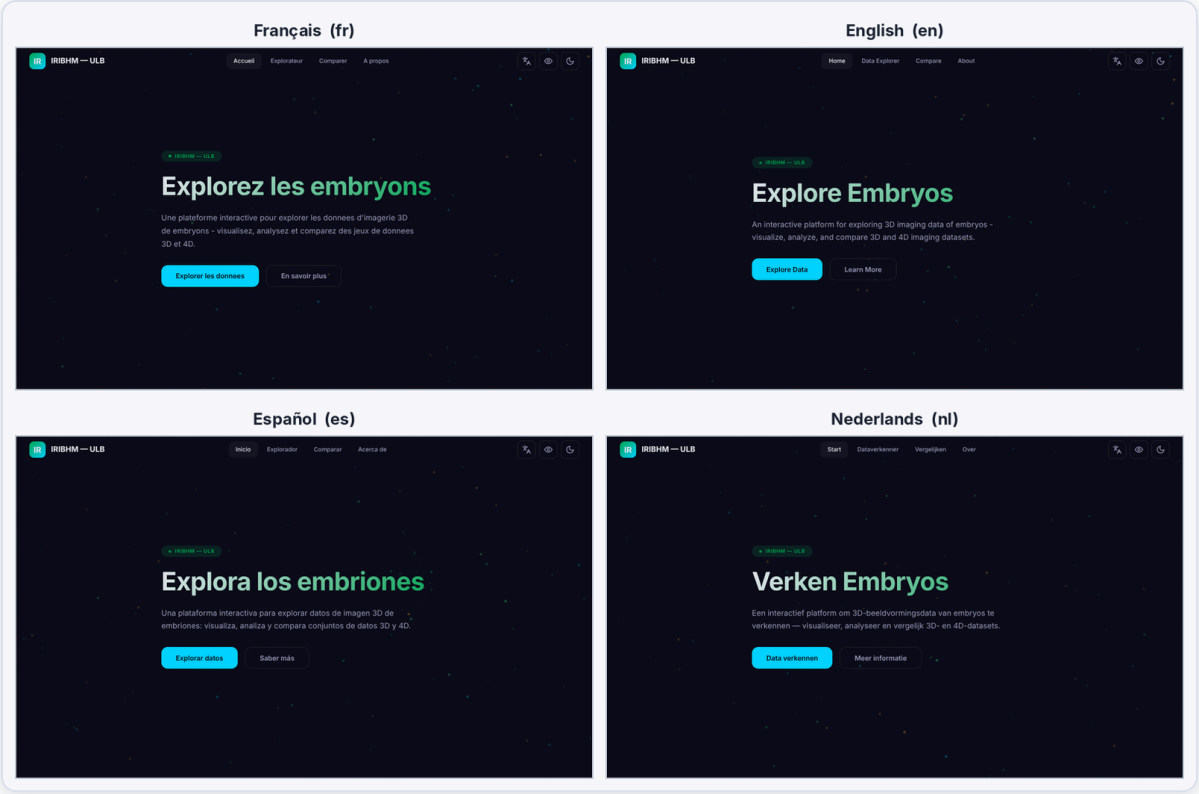



▲ A per-language value: answer with the visitor’s language, otherwise English, otherwise the first one found.

🔢 **WORKED EXAMPLE**

**The missing Dutch.** In the demonstration, `specimen.plural` exists in English, French and Spanish, but not in Dutch. A Dutch visitor therefore sees: “Verken **Embryos**”: the sentence is Dutch, the noun is the English fallback, and it is capitalised because the token is `{SpecimenPlural}`.

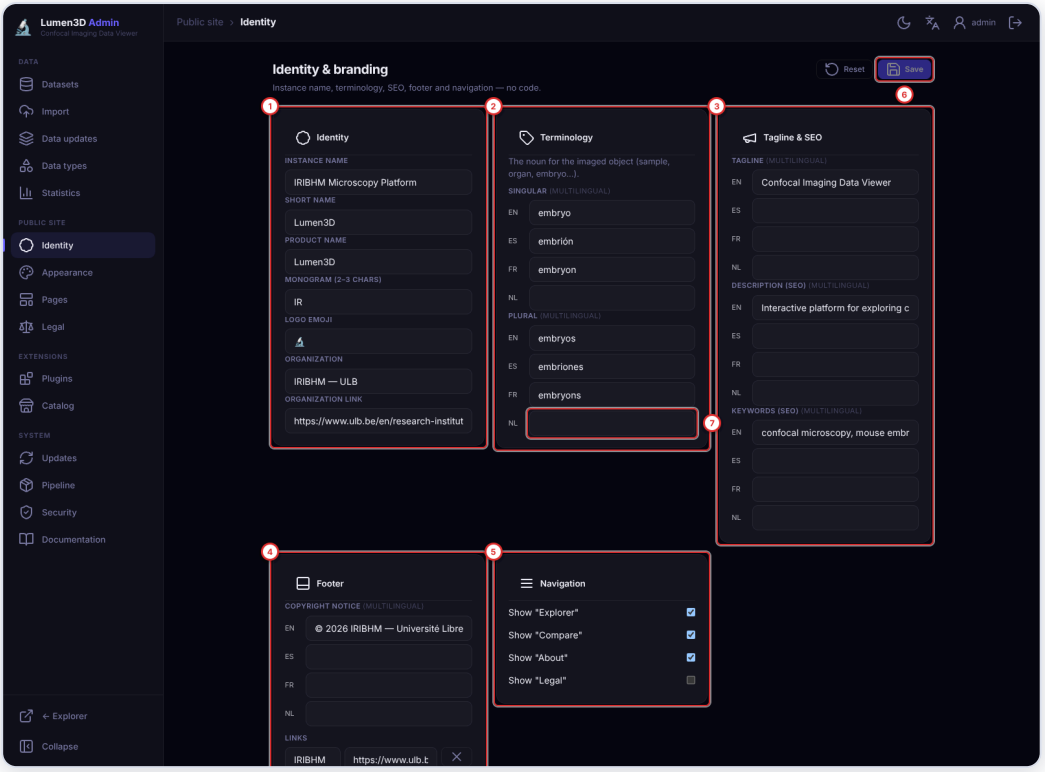
This is not a fault, it is the fallback working: the fix is to fill in the **NL** line in the **Identity** tab.




▲ The home page in the four languages. The name of the object follows the language; in Dutch, it falls back to English for lack of an entry.

# 16.4 The customisation screens

## The Identity tab

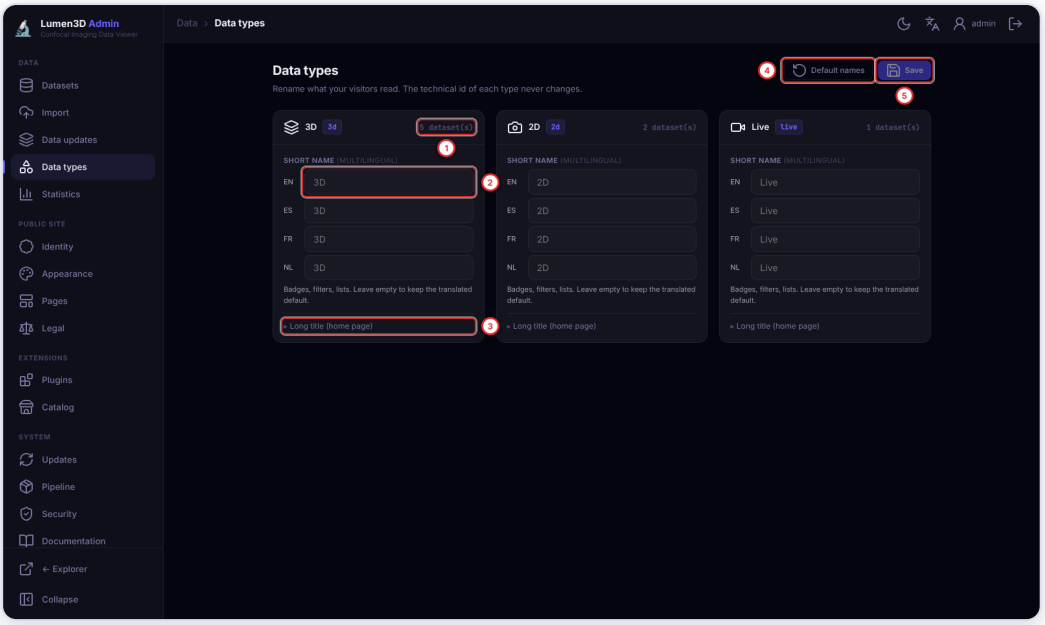



▲ The Identity & branding tab (demonstration dataset).

| N | WHAT IT IS |
|---|---|
| 1 | **Identity** card: names, monogram, emoji, organisation. |
| 2 | **Terminology** card: the word for your objects of study, singular and plural. |
| 3 | **Tagline & SEO** card: tagline, description and keywords, per language. |
| 4 | **Footer** card: copyright (per language) and links, shown in the public footer (see 16.2). |
| 5 | **Navigation** card: one box per entry of the public menu. **Show "Legal"** is unticked by default. |
| 6 | **Save**: active only when a field has changed. |
| 7 | A **multilingual** field: one line per available language. Here the **NL** line is empty. |

Each multilingual field shows **as many lines as the platform has languages**: if you add a language (16.9), a line appears by itself.

## The Data types tab




▲ The Data types tab.

| N | WHAT IT IS |
|---|---|
| 1 | The number of datasets of this type (here 5 3D datasets). |
| 2 | **Short name**: one line per language. Empty = keep the default translation. |
| 3 | **Long title (home page)**: collapsible panel, for the large cards. |
| 4 | **Default names**: **empties** the fields (you then have to save). |
| 5 | **Save**. |

Renaming “3d” to “Volumes” changes the **displayed word**, never the `DATA_WEB/3d/` folder, nor the addresses, nor the identifiers. The chain is: name entered → otherwise the `types.<type>` translation → otherwise the identifier itself.

> 🔧 **FOR THE CURIOUS**
>
> **A good practice in the code.** No screen writes a type name by hand: everything goes through `Utils.datasetTypeLabel()` and `datasetTypeTitle()`. An HTML element carrying `data-dataset-type="3d"` has its text rewritten by this mechanism, including at every language change.

## Why two tabs do not overwrite each other

Three tabs write into the same `instance.json` file: Identity, Data types and the page editor (variables, menu). A tab left open for an hour must not erase the others’ work.

> 💡 **A PICTURE TO UNDERSTAND**
>
> **A shared notebook.** Everyone writes **on their own lines**: the server agrees to modify only the "paths" announced ( `?merge=brand,specimen,…` ) on the document **as it is now**, under a lock. There is no stale photocopy.

For **pages**, one more protection: every read carries a **revision** (a fingerprint of the document). A save that arrives with a stale revision is **refused** (409 "stale") rather than overwriting a newer version.

### The first-install wizard

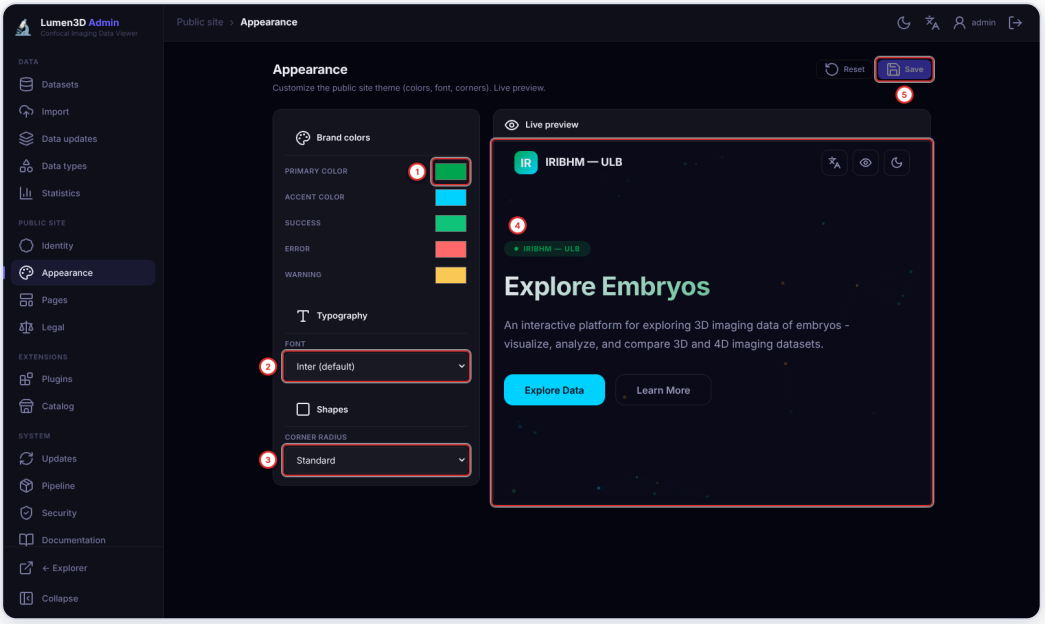
On the very first visit (no password exists), the panel opens a wizard in **five steps**. Only the first is mandatory.

1. **Account**: the administrator's password.
2. **Identity**: instance name, organisation, name of the object (singular / plural).
3. **Theme**: a brand colour from six (green, blue, violet, turquoise, orange, crimson).
4. **Texts**: tagline and copyright line.
5. **Plugins**: the signed catalogue (chapter 15).

At the end, the wizard writes only what you filled in: `brand` (the monogram takes the first two characters of the name, in capitals), `specimen`, possibly `org.name` and `footer.copyright`, then the colour tokens chosen, in `theme.json`.

## 16.5 The theme: colours, font, corners

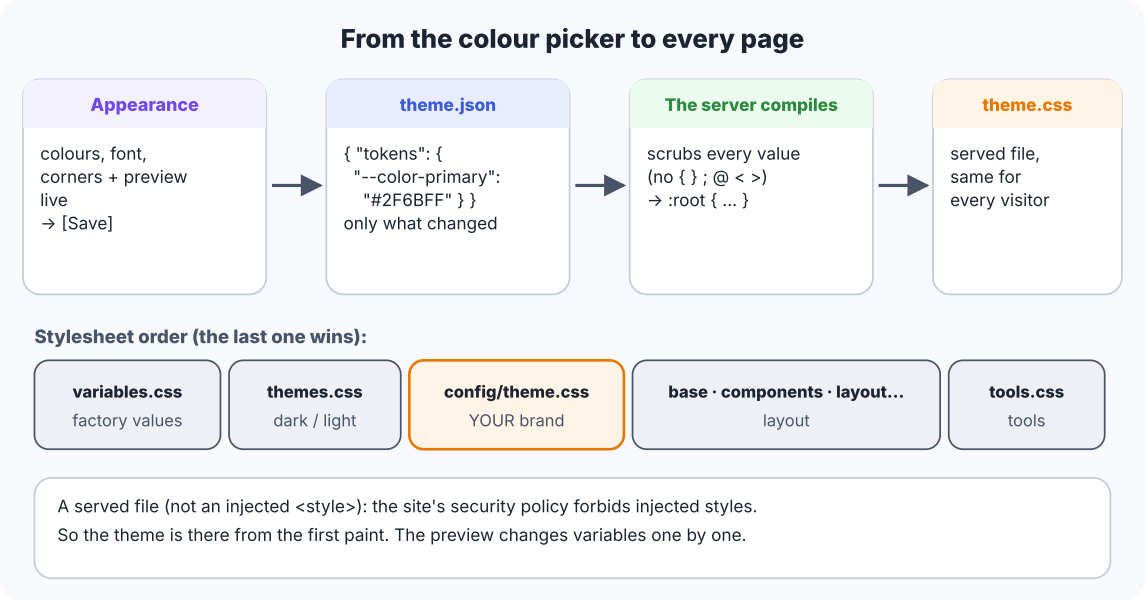
The look of the site is driven by **CSS variables** (style "tokens"): `--color-primary`, `--font-sans`, `--radius-md` … The **Appearance** tab changes only a handful of them, on purpose.




▲ The Appearance tab: five colours, a font, a corner radius, and a preview of the real site.

| N | WHAT IT IS |
|---|---|
| 1 | The **five brand colours**: primary, accent, success, error, warning. |
| 2 | The **font**: Inter (default), System, Grotesque, Serif, Rounded. |
| 3 | The **corner radius**: Standard, Sharp, Soft, Round. |
| 4 | The **live preview**: the real site in a frame, which changes instantly. |
| 5 | **Save**: nothing is applied to the site before this click. |

## From the click to every page




▲ The path of a theme setting, and the order of the style sheets.

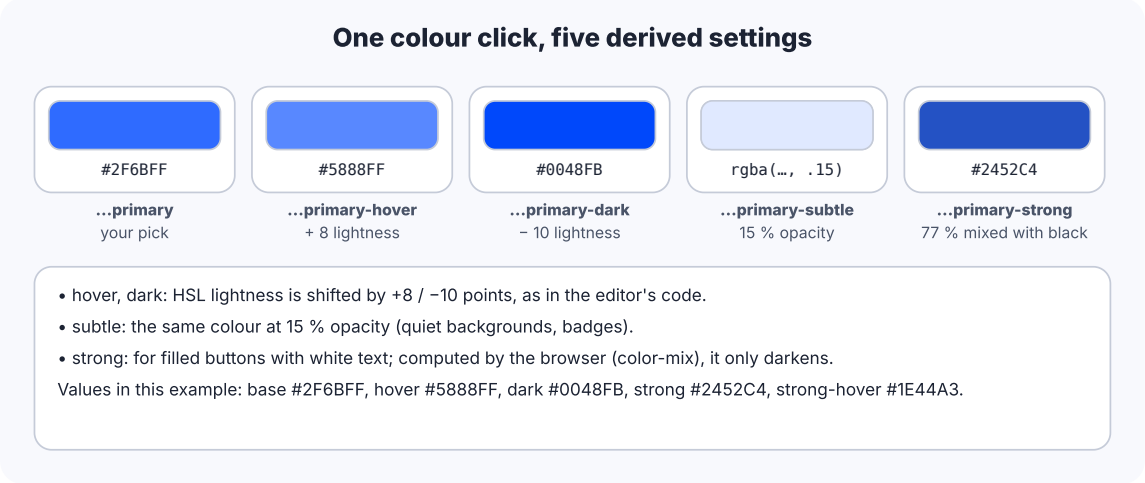
**📌 REMEMBER**

The server lets through only variables whose name starts with `--`, and strips from each value the characters that could break a style sheet ( `{ } ; < > \ @` ); a value is cut at 200 characters. **A badly entered theme cannot destroy the whole sheet.**

The `theme.css` sheet is loaded **after** `themes.css` (the factory dark/light theme) and before the layout sheets: it therefore **wins** over the factory values, but only for what you changed. The rest keeps coming from `variables.css`.

## One colour, five settings

A single colour picker is not enough: a button needs a hover shade, a dark shade, a subtle background. The editor **derives** them for you.



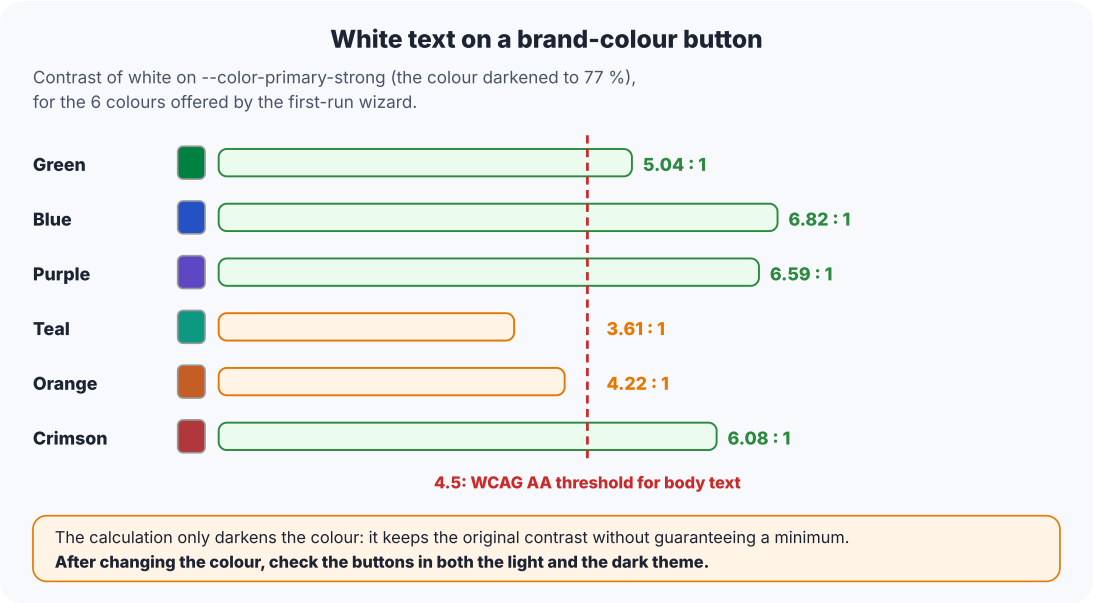

▲ A chosen colour (here #2F6BFF) and its derivatives.

| SETTING | CALCULATION |
|---|---|
| `--color-primary-hover` | lightness (HSL) **+ 8** points |
| `--color-primary-dark` | lightness **− 10** points |
| `--color-primary-subtle` | the same colour at **15%** opacity (accent: 12%) |
| `--color-primary-strong` | **77%** of the colour over black, then darkened one point at a time until white text reaches **4.5 : 1** (hover: 64/77 of that factor) |

Success, error and warning derive in the same way (hover, subtle background at 12%).

## The contrast of white text

Solid buttons carry **white text** on `--color-primary-strong`. This is the whole point of the “strong” setting: the factory green (#00A654) gives only **3.19 : 1** under white, which is insufficient for text; darkened, it rises to **5.05 : 1**. The WCAG AA rule asks for **4.5 : 1**.




▲ Contrast of white text on the button, for the six colours offered by the wizard.

ℹ️ **GOOD TO KNOW**

The figure shows the plain 77% mix. As soon as a primary colour is set, the server's **theme compiler** computes the filled-button colour itself: it starts at 77% and darkens one point at a time until white text reaches **4.5 : 1**. The AA threshold is therefore **guaranteed** for every colour it can parse ( `#rgb` , `#rrggbb` , `rgb()` / `rgba()` ). Shades that are already dark do not move; **orange** #FF7A2F (4.22 at 77%) gives #BD5A23 (**4.52 : 1**) and **turquoise** #0FC5A8 (3.61 at 77%) gives #0A8672 (**4.50 : 1**). The Appearance tab's preview shows the same colours before saving, and an existing `theme.css` is recompiled once, automatically, after the update. With no colour set, the factory green comes from the stylesheet (#00803F, 5.05 : 1).

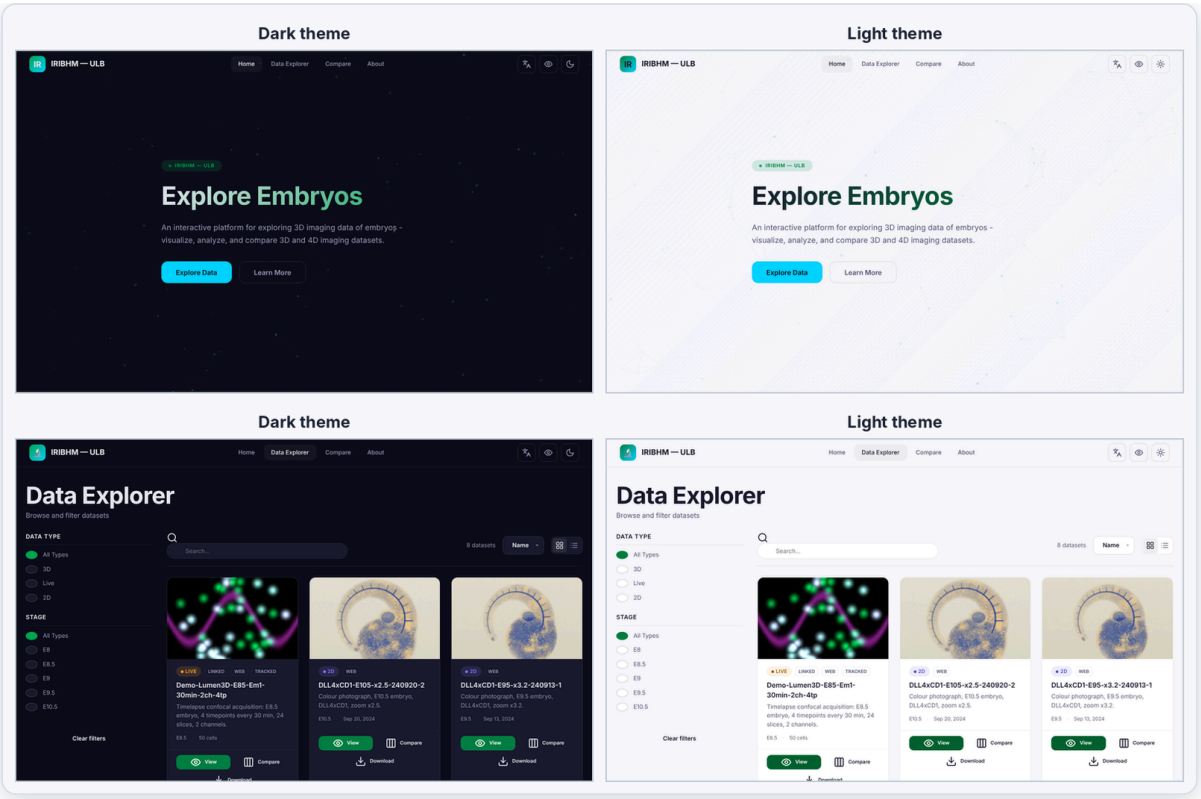
| WIZARD COLOUR | BUTTON (HOVER) | CONTRAST |
|---|---|---|
| **#00A654** | #008041 (#006A36) | 5.04 : 1 |
| **#2F6BFF** | #2452C4 (#1E44A3) | 6.82 : 1 |
| **#7C5CFF** | #5F47C4 (#4F3BA3) | 6.59 : 1 |
| **#0FC5A8** | #0A8672 (#097060) | 4.50 : 1 |
| **#FF7A2F** | #BD5A23 (#9E4C1D) | 4.52 : 1 |
| **#E5484D** | #B0373B (#932E31) | 6.08 : 1 |

🔧 **FOR THE CURIOUS**

Contrast is the WCAG relative-luminance ratio, `(L1 + 0.05) / (L2 + 0.05)` , where `L` is computed on the **linearised** RGB components (sRGB law). The figures above are recomputed by the figure script from the wizard's colours ( `DOCS/documentation/img/ch16/make_figures.py` ).

## Light or dark

The **dark** theme is the default, by choice: a fluorescence image reads better on a black background. The visitor switches with one click; their choice is remembered **in their browser** ( `iribhm-theme` ).




▲ The same site in the dark theme (left) and the light theme (right).

> **ℹ️ GOOD TO KNOW**
>
> The theme is **not** tied to the system setting on the first visit. If the visitor has **never** chosen, a change of their system's light/dark setting is followed; as soon as they have chosen, their choice is what counts.

Each theme also sets `color-scheme`: native drop-down menus, scroll bars and date pickers take the right look, with no light text on a white background.

## The 3D viewer's background: a separate setting

The **volume's background** does not follow the site's theme: it is a viewer setting, in the sidebar (**Background**), remembered with the working state.

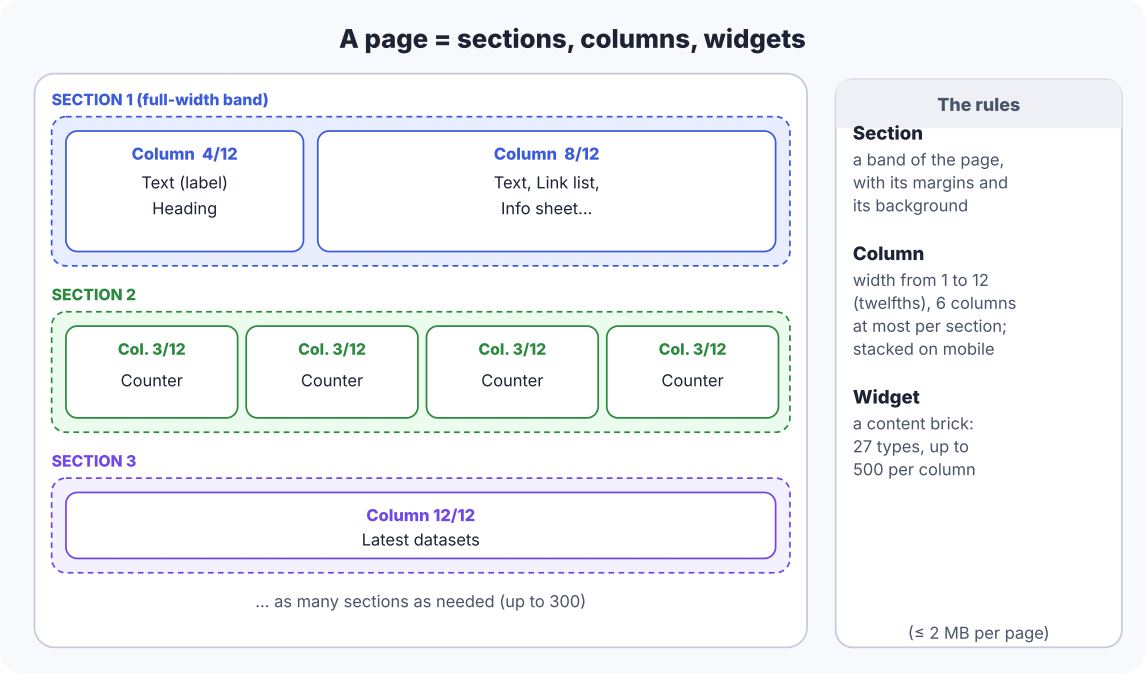
| CHOICE | COLOUR |
|---|---|
| **Dark** (default) | `#000000` |
| **Light** | `#f4f6fb` |
| **Paper** | `#f8f5ec` |
| **Transparent** | no background (PNG exports keep the transparency) |
| **Custom** | the colour you choose ( `#1a1d27` to start with) |

👉 TIP

A **light** or **paper** background suits printed figures. An invalid colour code is refused with a console warning: the viewer keeps the choice's default colour rather than showing a random shade.

## 16.6 The page builder

Home, About, or any page you create (protocols, team, contact…) are built **with the mouse**, as in a page-layout program.




▲ A page = sections (bands), columns (twelfths), widgets (bricks).

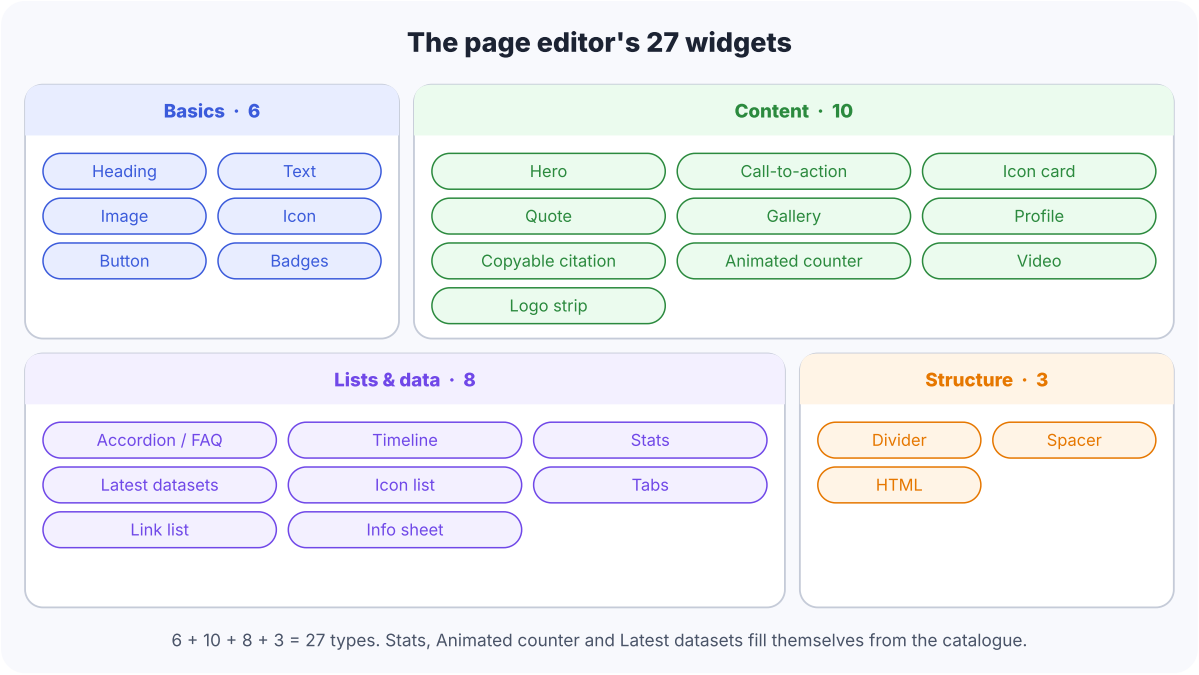
### The model: sections, columns, widgets

- A **section** is a band of the page: background, margins, maximum width.
- A **column** takes up **1 to 12 twelfths** of it; a section has **at most 6 columns** in the editor. On a phone, the columns **stack**.
- A **widget** is a brick of content. There are **27 types**.

💡 A PICTURE TO UNDERSTAND

**A newspaper.** The page is an issue, the sections are the horizontal bands, the columns are the newspaper's columns, the widgets are the articles, the photos and the boxed items.

## The 27 widgets




▲ The 27 widgets, arranged as in the editor's palette.

## What each widget does

| WIDGET | WHAT IT IS, AND ITS SETTINGS |
|---|---|
| **Basics** | |
| **Heading** | a section heading; heading level and alignment |
| **Text** | a paragraph; mini formatting `**bold**`, `*italic*`, `[link](address)` |
| **Image** | an image with caption and link; fit (fill or contain), width, height, alternative text |
| **Icon** | a pictogram chosen from the icon library (with search); size, colour |
| **Button** | label and link; variant, icon on the left or right, size, outline, full width |
| **Badges** | small labels; coloured dot, monospaced font, colours |
| **Content** | |
| **Hero** | the big home banner: title, subtitle, two buttons, badge, decorative glow, background |
| **Call-to-action** | a box that invites a click: title, subtitle, two buttons |
| **Icon card** | icon, image or monogram + title + text + link; horizontal or vertical layout |
| **Quote** | text, author, role, photo; as a bar or a card |
| **Gallery** | images in a grid; columns, captions, zoom on hover |
| **Profile** | a person's card: name, role, description, media |
| **Copyable citation** | a reference with a "copy" button and a collapsible block (BibTeX...) |
| **Animated counter** | a number that counts up; fixed value **or** catalogue source; prefix, suffix |

| WIDGET | WHAT IT IS, AND ITS SETTINGS |
|---|---|
| **Video** | an `.mp4` / `.webm` file, or a YouTube/Vimeo link that opens **in a new tab**; preview image, loop, muted autoplay |
| **Logo strip** | a row of partner logos; desaturated with colour on hover, light plate |
| **Lists & data** | |
| **Accordion / FAQ** | questions that unfold; only one open at a time, the first open |
| **Timeline** | a series of dated steps |
| **Stats** | a row of key figures; fixed value **or** catalogue counter (datasets, specimens, cells, regions) |
| **Latest datasets** | the cards of the most recent datasets; number, columns, type and date shown or not |
| **Icon list** | a bulleted list with illustrated bullets; vertical or horizontal |
| **Tabs** | content spread over tabs |
| **Link list** | rows separated by thin rules, with an arrow: the sober way to present links |
| **Info sheet** | a “label: value” table |
| **Structure** | |
| **Divider** | a line; thickness, width, style (solid, dashed, dotted), colour |
| **Spacer** | an adjustable gap from 0 to 400 pixels (32 by default) |
| **HTML** | free HTML, **cleaned by an allow-list** (see below) |

🔧 FOR THE CURIOUS

**Content security.** Text is always written with `textContent` (never interpreted as HTML). The HTML widget goes through an **allow-list**: scripts, event handlers, SVGs and `javascript:` links are removed; every `style` attribute is filtered declaration by declaration (no `position: fixed`, no `z-index`, no exotic URL). Links you enter refuse the `javascript:`, `vbscript:` and `data:` schemes. External videos are **not** embedded in the page: the site’s security policy forbids third-party frames, hence the simple link.

## An element’s style

Each **widget**, **column** and **section** has its style settings, in seven families:

| FAMILY | WHAT YOU SET THERE |
|---|---|
| **Text** | colour (or a **gradient** painted into the letters), size, weight, line height, letter spacing, italic, capitals, alignment |
| **Background & border** | background colour or image, overlay, corner radius, border (thickness, colour, line), shadow (light, medium, large, glow), opacity |
| **Spacing** | margins and padding, side by side or linked |
| **Size** | maximum width, minimum height |

| FAMILY | WHAT YOU SET THERE |
|---|---|
| **Effects** | on hover: lift, glow or zoom |
| **Visibility** | hide on mobile, or on desktop |
| **Custom CSS** | a few CSS declarations, **sanitised** before being applied |

🔧 **FOR THE CURIOUS**

The style is compiled into **inline** CSS ( `style` attribute), with no injected style sheet: the security policy forbids `<style>` elements added after the fact. Only the rules that really need a sheet (the `:hover` effect, the `@media` hiding by screen size) live in `css/pages.css` , a file of the site. Every value goes through a filter that removes anything that could break out of a declaration.

## Variables, tokens and backgrounds

- **Dynamic variables**: `{year}` , `{date}` , `{time}` , and the counters `{datasetCount}` , `{specimenCount}` , `{cellCount}` , `{regionCount}` (computed at display time, in the visitor's language).
- **Instance tokens**: the thirteen of section 16.3 also work in page texts.
- **Your variables**: a name (a letter, then letters, digits or `_` , 32 characters at most) and a value, per language if needed. If their name is that of an existing variable, yours **wins**.
- **Animated backgrounds**: ten presets (floating particles, waves, aurora, starry sky, pulsing grid, constellation, orbs, cursor ripples, force field, halo). Five stand alone, five react to the mouse. With "reduce motion" turned on in the system, a single **still** drawing replaces the animation.

👉 **TIP**

`{…}` variables work in **all** page texts, but not in the language files (which have only the instance tokens).

## Draft and publication

▲ Two files, two audiences: the draft is private, the published version is public.

📌 **REMEMBER**

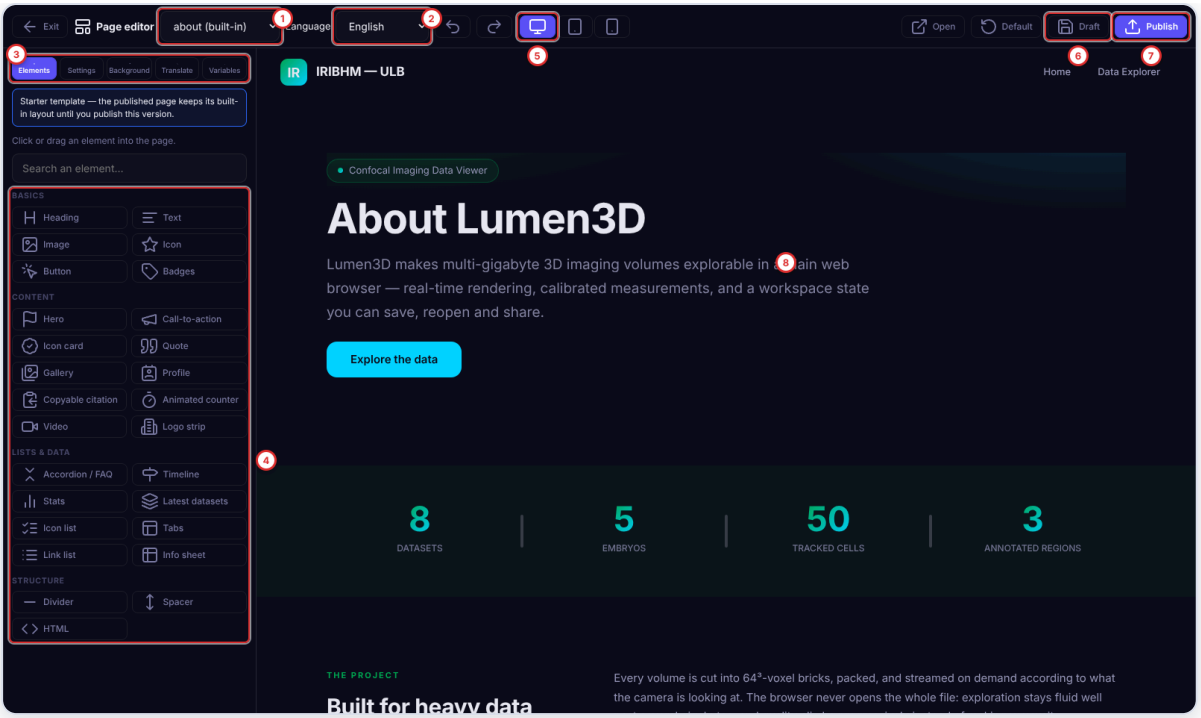
**Nothing is public before** `Publish`**.** While you work, visitors see the old version. The draft is saved automatically, but it is stored **outside** `config/`: that folder can be read without a password, and the editor saves about once a second while you write. A draft left in `config/` would have let anyone watch the operator write.

This also sets common-sense rules for the editor:

- **"Publish"** first saves the draft, then *promotes* it to the public version.
- **"Default"** (original pages) returns to the shipped template; on a page of your own, it abandons the draft and returns to the published version.
- **One editing tab per page**: the second falls silent until you "take back control".
- The editor keeps **60 steps** of undo / redo.
- At the first `Publish` of a page you created, it is added to the **menu** automatically.

## The editor on screen

The editor opens **in its own tab** (`admpan.html?editor=<page>`): the **real** page appears in a frame, with its real menu and its real theme; the editor adds a layer of handles on top.




▲ The page editor, on the “About” page (demonstration dataset).

| N | WHAT IT IS |
|---|---|
| 1 | The **page** being edited (the “built-in” label: it still uses the shipped template). |
| 2 | The **language** you are writing. |
| 3 | The five panels: **Elements**, **Settings**, **Background**, **Translate**, **Variables**. |
| 4 | The **palette** of the 27 widgets, in four families. A click or a drag. |
| 5 | The **desktop / tablet / mobile** preview. |
| 6 | **Draft**: saves without publishing. |
| 7 | **Publish**: puts your version online. |
| 8 | **The real page**: click an element to select it. |

The page’s widgets are described by the **same code** for the editor and for the site (`PageRenderer`): what you see in the editor is exactly what will be published.

## The About page template

The **Home** and **About** pages exist from the start. As long as you have published nothing, the page shown is a **template** written in the same format as your pages (`js/core/page-templates.js`). It is both:

- what **visitors see** (there is no longer any fallback HTML hidden in `about.html`);
- the editor’s **starting point**, which opens it with the “Starter template” label.

Its sentences are already written in English, French and Spanish, with tokens (`{brandShort}`, `{SpecimenPlural}`, `{org}`, `{year}`…): the page therefore carries **your** name before you have changed a word.

### Translating a page

The **Translate** tab lists **all the texts** of the page, one field per language, and flags those that are missing ("24 texts · 7 missing translations"). There is **no machine translation**: nothing leaves your server.

> 👉 **TIP**
>
> Recommended method: write the whole page in **one** language, then go to the Translate tab. Do not forget English: it is the fallback of all the other languages.

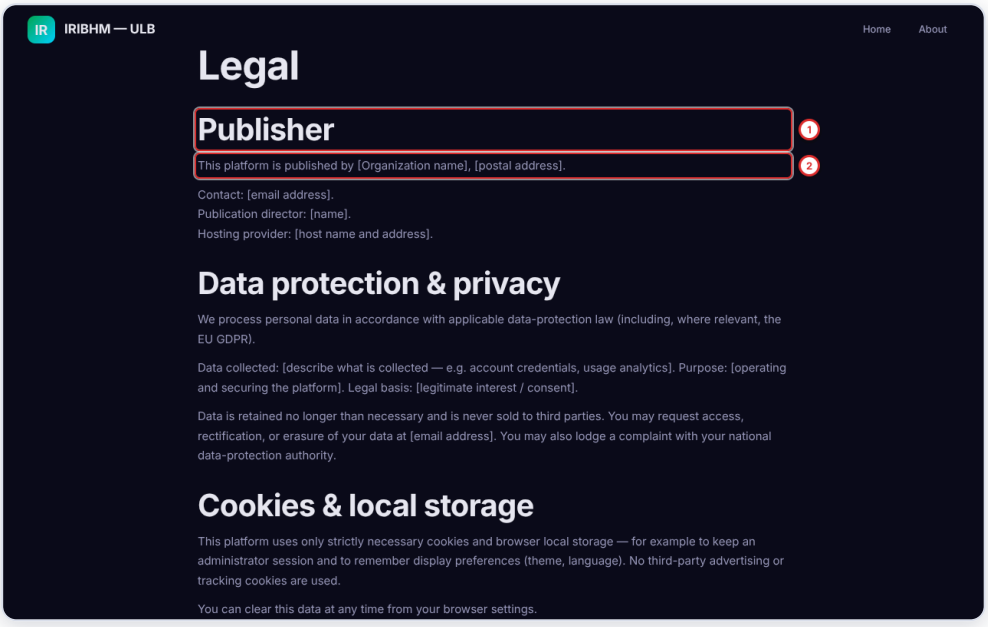
### The limits

| LIMIT | VALUE |
|---|---|
| **size of a page** | 2 MB |
| **sections per page** | 300 |
| **columns per section** | 12 on the server, 6 in the editor |
| **widgets per column** | 500 |
| **width of a column** | 1 to 12 twelfths |
| **address of a page ( `slug` )** | lower-case letters, digits, `-`, `_`; 64 characters at most |

A document outside the limits is **refused as a whole**, never half saved (only a column's width is simply brought back between 1 and 12). A deleted or unknown page sends you back to the home page rather than showing an empty page.

## 16.7 The legal notice

A public site needs a **publisher**, a **host**, a **data** policy. The **Legal** tab is a deliberately simple editor: a list of **sections**, each with a title and a text, **per language**.




▲ The legal.html page, as a visitor sees it (neutral template, in English).

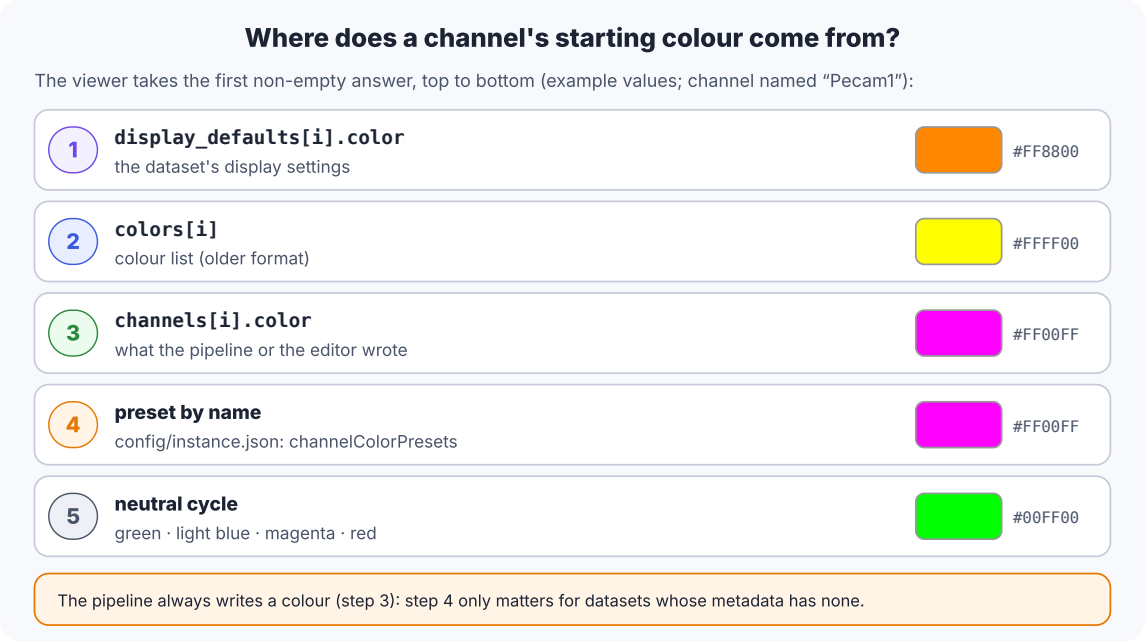
| N | WHAT IT IS |
|---|---|
| 1 | The **title** of a section (“Publisher”). |
| 2 | The **text**: one paragraph per blank line. The `[square brackets]` are to be replaced by your information. |

- As long as you have published nothing, the page shows the shipped **neutral template** (publisher, data protection, cookies and local storage, intellectual property, disclaimer, contact). It is written in **English, French, Spanish and Dutch**.
- The text is displayed with `textContent`: no HTML, no formatting, one section per block.
- The link appears in the footer only if the **Show “Legal”** box is ticked (unticked by default).

> ⚠️ **WARNING**
>
> These texts are **starting points**, not legal advice. Have them reviewed for your country. Remember that a web server generally keeps an **access log** (IP addresses) depending on its configuration: this is your host’s business, and it is declared here (see 16.11).

## 16.8 Where does a channel’s starting colour come from?

When you open a dataset, every channel already appears in colour. Where does this colour come from? Chapter 11 shows how to **change** it; here is where it **comes from**.




▲ The first non-empty answer wins.

The panel takes the **first** answer available:

1. the dataset’s **display settings** (`display_defaults`);
2. a `colors` list (old format);
3. the channel’s colour in `metadata.json` (`channels[i].color`): the one the **pipeline** writes at the start, or that the **dataset editor** has saved;
4. a **preset by name** from the instance;

5. a **neutral cycle**: green, light blue, magenta, red.

### The preset by name

In `instance.json`, a `channelColorPresets` list associates a piece of name with a colour:

```
{ "match": "dapi",  "color": "#00AAFF" },
{ "match": "pecam", "color": "#FF00FF" }
```

The channel name is lower-cased; **the first** `match` contained in the name wins. The demonstration contains eight: `gfp` (green), `dapi` and `hoechst` (light blue), `pecam` and `picam` (magenta), `rfp`, `mcherry` and `alexa` (red).

> ⚠️ **WARNING**
>
> Since the pipeline **always** writes a colour (step 3), the presets are in practice used only for datasets whose metadata has none. They have no administration screen: they are edited in `instance.json`.

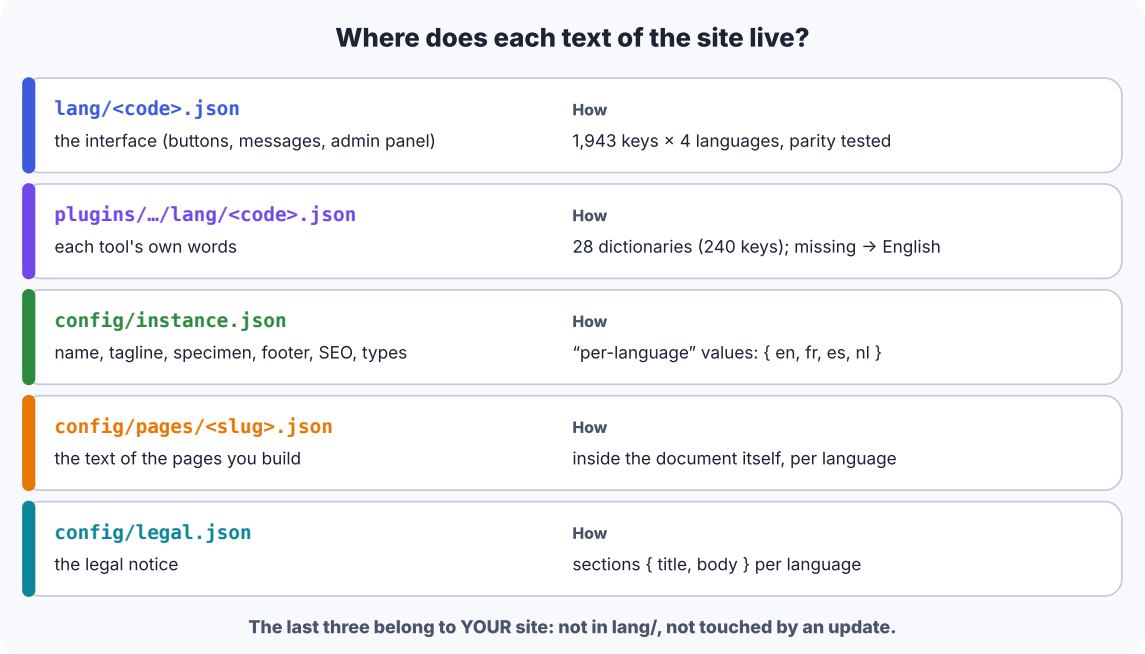
### Four colour lists, four uses

| WHERE | COLOURS |
|---|---|
| **the pipeline (`metadata.json`)** | `#00FF00`, `#00AAFF`, `#FF00FF`, `#FF0000`, `#FFFF00`, `#00FFFF` |
| **the dataset editor (if nothing)** | `#00FF66`, `#FF3DFF`, `#2F6BFF`, `#FF3030` |
| **the channel panel (neutral cycle)** | `#00FF00`, `#00AAFF`, `#FF00FF`, `#FF0000` |
| **the dataset's thumbnail (Explorer image)** | green, magenta, blue, red, yellow, violet, cyan |

> ℹ️ **GOOD TO KNOW**
>
> These lists are not synchronised: they serve at different times. The colour **that counts** for the visitor is that of the channel panel, which starts from `metadata.json`. The picker's 27 colours, for their part, are fixed (chapter 11).

## 16.9 Translating: how the site speaks four languages

The public site, the viewer, the administration and even the words of each tool are translated. It all rests on **one file per language** and a single function, `t('key')`, which returns the text in the visitor's language.




▲ Where each text of the site lives.

## The numbers

| | |
|---|---|
| **languages shipped** | **4**: English, French, Spanish, Dutch |
| **texts per language** | **1,943** (measured on the four `lang/*.json` files) |
| **tool dictionaries** | **28**, i.e. 240 texts in English |
| **fallback language** | **English**, always |

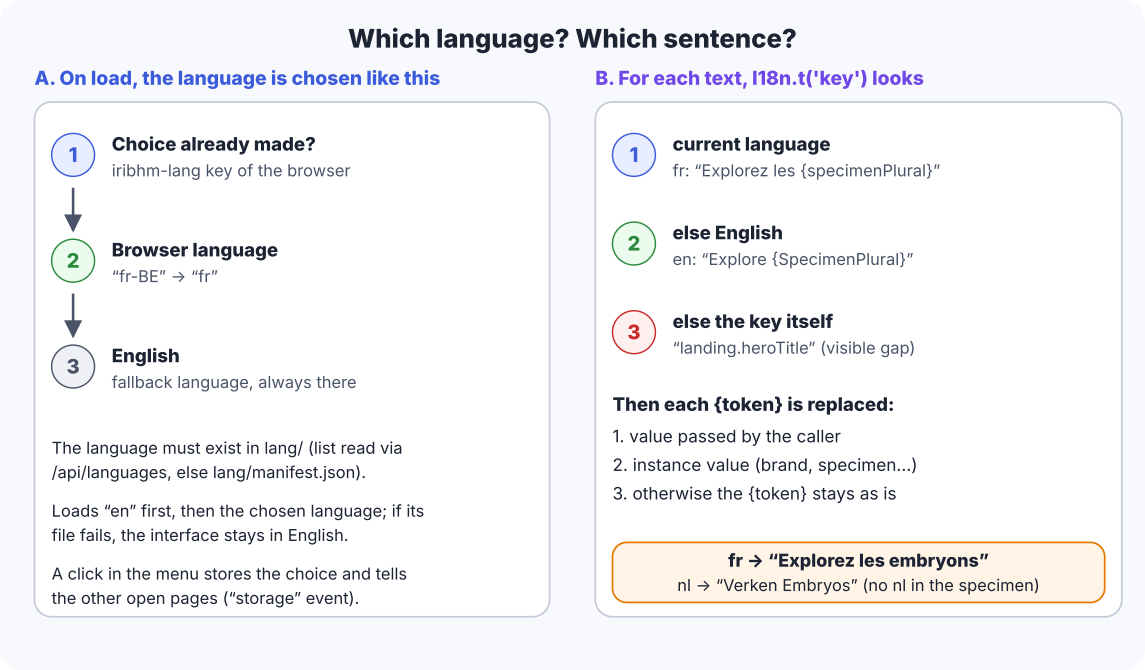
The 1,943 texts are distributed as follows:

| KEY FAMILY | TEXTS | WHAT IT CONTAINS |
|---|---|---|
| `pages` | 485 | the page editor |
| `admin` | 365 | the administration panel |
| `viewer` | 182 | the viewer |
| `dupd` | 119 | the Data updates tab |
| `upl` + `upload` | 104 | the import |
| `compare` , `about` , `studio` , `tips` … | 688 | the rest |

📌 **REMEMBER**

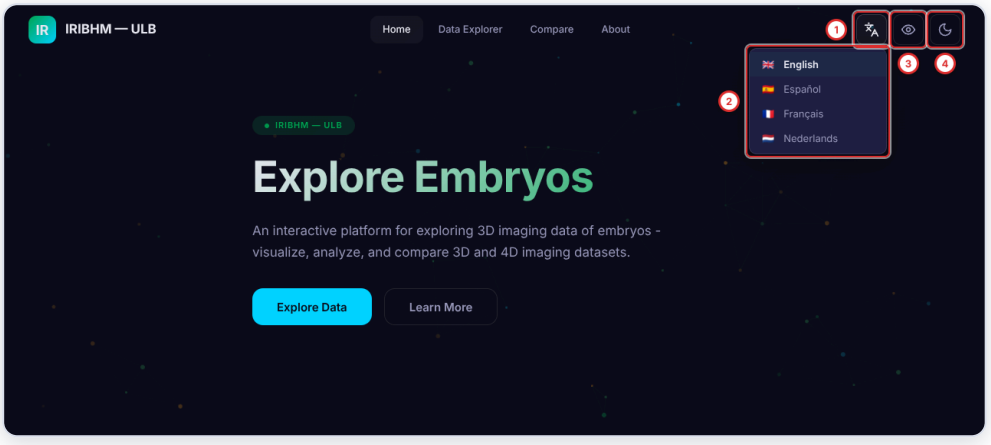
A key of one language **exists in all four**, with the **same `{…}` tokens**. An automatic test checks it at every version ( `tests/js/test_build_lang_parity.mjs` ): a missing or empty key, or one whose tokens change, makes the delivery fail.

## Which language? Which sentence?




▲ Left: the choice of language at load. Right: the lookup of a text.

- **The choice.** The browser's `iribhm-lang` key (written when the visitor clicks in the menu); otherwise the browser's language (“fr-BE” becomes “fr”); otherwise English.
- **The loading.** English is read **first** (always), then the chosen language. If that file fails, the interface stays in English.
- **The propagation.** A click in the menu notifies the **other open pages** of the same origin ( `storage` event): in Compare, the embedded panels change language with the host page.
- **The document.** The page's `lang` attribute follows the language; `dir="rtl"` is set for a right-to-left language (none is shipped).




▲ The language menu of the top bar: one button per language, with its flag and native name.

| N | WHAT IT IS |
|---|---|
| 1 | The **Language** button. |
| 2 | The menu: generated **from the languages discovered**, English first, then alphabetical order. |
| 3 | The filter for colour-blind people (see 16.10). |

| N | WHAT IT IS |
|---|---|
| 4 | The light / dark theme. |

### Language discovery

The list of languages is **written nowhere by hand**: it is discovered, from the most to the least direct way.

1. `GET api/languages.php` : the server lists the `lang/<code>.json` files (the name must be of the form `fr` or `pt-BR` ).
2. Otherwise `lang/manifest.json` , the index that the Python server **rewrites** when the list changes, for static hosts.
3. Otherwise a fallback list built into the code: English, French, Spanish.

> 🔧 **FOR THE CURIOUS**
>
> English is **always added** to the list, even if it was missing: the menu cannot be empty. Dutch is not in the fallback list: if both discoveries fail, it disappears from the menu.

### The five ways to attach a text to an element

In the HTML, an attribute says which text the element displays:

| ATTRIBUTE | WHAT IT FILLS |
|---|---|
| `data-i18n="key"` | the element's text |
| `data-i18n-placeholder` | the greyed text of an input field |
| `data-i18n-title` | the tooltip |
| `data-i18n-aria` | the label read by a screen reader ( `aria-label` ) |
| `data-i18n-html` | a text that contains markup (rarely) |

The replacement takes place only if the key **exists**: otherwise the text written in the page (English) is kept, never a bare key.

### The tools' words

Each plugin carries its own texts: `js/modules/<family>/<id>/lang/<code>.json` . On load, they are filed **under** `plugins.<id>` in the same tree as the platform's texts.

- A plugin's list of languages is declared in its `plugin.json` ( `i18nLanguages` ): no attempt is made to load a file that does not exist.
- For each plugin, the **fallback is its own English**. The "sandboxed" screenshot plugin has no Spanish; a Spanish visitor sees it in English, without an error.
- On the Python server, the dictionaries travel **in the plugin-discovery answer** (one request fewer per plugin and per language); elsewhere, they are read file by file, and only English and the chosen language.

- A plugin that offers a language the platform does not have does not add it to the menu: only the platform decides.

### Adding a language

1. **Copy** `lang/en.json` to `lang/de.json` (example: German) and translate the **values**, never the keys or the `{tokens}`.
2. **Reload**: the language appears in the menu (the code `de` is already known: “Deutsch”, flag). An unknown code works too, with its abbreviation in capitals and a neutral flag.
3. **Plugins**: add `lang/de.json` to the tools you want to translate and complete `i18nLanguages`; without this, English.
4. **Your content**: a `DE` line appears by itself in the Identity, Data types, Legal tabs and in the page editor.
5. **Check** with `node tests/js/test_build_lang_parity.mjs` (keys, empties, tokens).

> ℹ️ **GOOD TO KNOW**
>
> Languages written right to left (Arabic is known to the code) receive `dir="rtl"`, but none is shipped: the layout has not been validated for them in this repository.

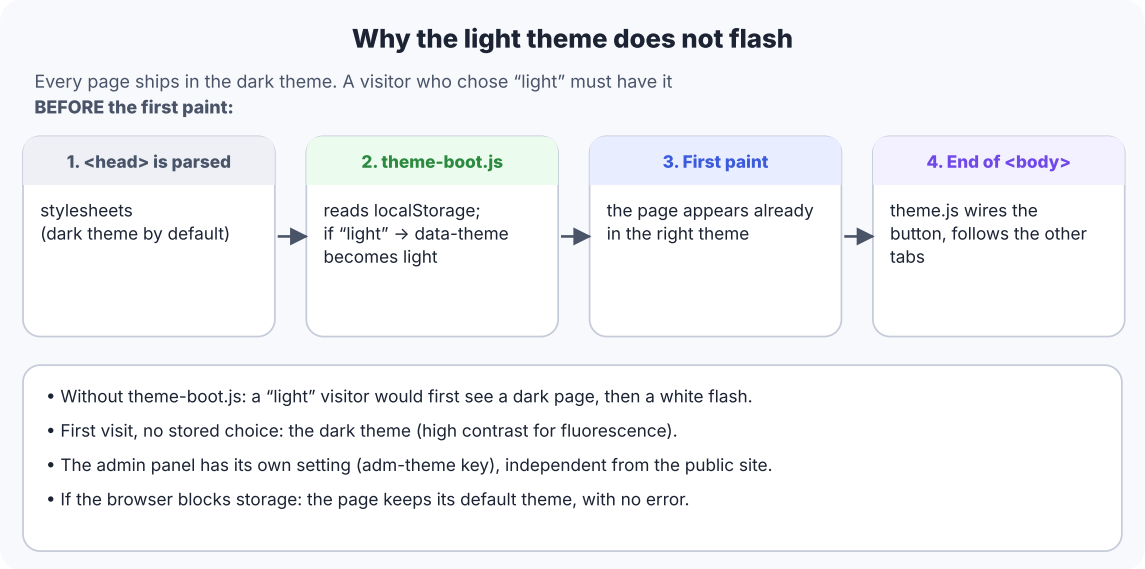
### What happens if...?

| SITUATION | WHAT THE VISITOR SEES |
|---|---|
| a key is missing in French | the **English** text |
| a key is also missing in English | the **key itself** (“landing.heroTitle”) |
| `specimen.plural` has no `nl` line | the **English** name in the Dutch sentence |
| a language’s value is empty | English (an empty line counts as absent) |
| a language file is unreadable | the interface stays in **English** |
| a plugin does not have the language | the **plugin’s** English |
| a `{xyz}` token does not exist | `{xyz}` is displayed as it is |

## 16.10 Accessibility, theme and mobile

The accessibility of an imaging tool has limits (a 3D image remains an image). Here is what the code really does.

## Before the first display: the theme




▲ How the light theme arrives before the first display.

**🔧 FOR THE CURIOUS**

A small blocking script ( `theme-boot.js` , a few lines) is placed in the `<head>` , after the style sheets. It reads the theme key and, if it is “light”, changes the `data-theme` attribute **before** the page is painted. The administration panel loads it with its own key ( `adm-theme` ).

## With a keyboard and a screen reader

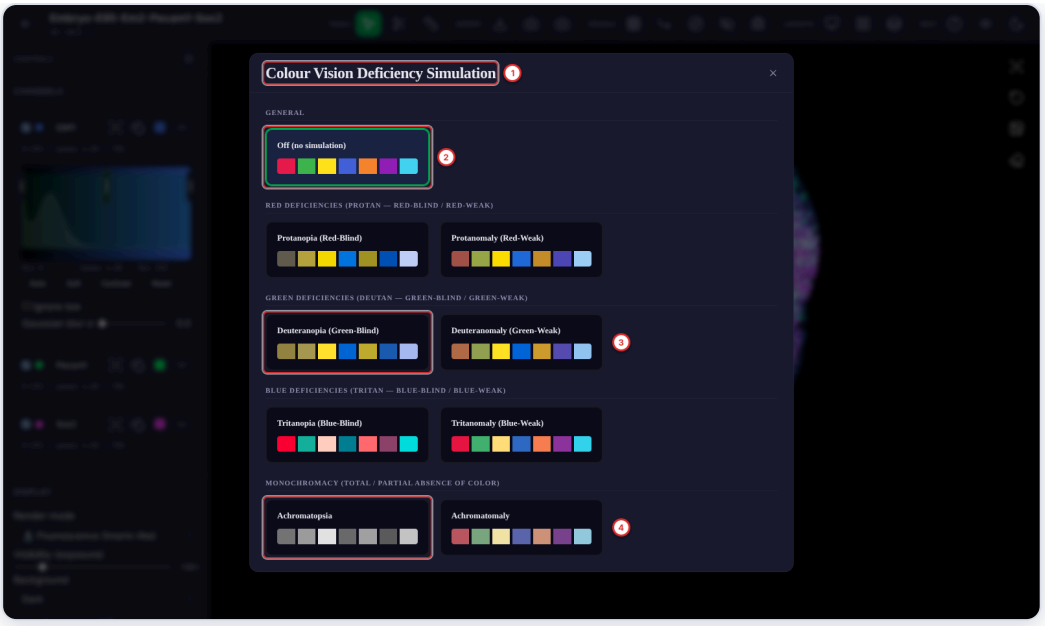
- A **“Skip to content”** skip link is the first focusable element of every public page (9 pages out of 10: only the widget demonstration page, not shipped, has none); it leads to `<main id="main">`.
- Icon buttons carry a name ( `aria-label` , or a translated tooltip); the text comes from the language files.
- Modal windows ( `Dialog` ) are declared `aria-modal` ; when they offer a “cancel” way out, Esc or a click on the backdrop chooses it. Their text is written with `textContent` .
- The viewer’s tools have shortcuts (V, C, M, I, D…): see chapter 12.
- The style sheets contain 25 occurrences of `:focus-visible` : the element that has keyboard focus is outlined.

## Reduced motion

If the system asks for “reduce motion”, **all** the site’s CSS animations and transitions are brought down to almost zero, and the pages’ animated backgrounds become a still drawing. The viewer’s camera movements (WebGL) are not CSS: they are not affected.

## The filter for colour-blind people

The eye-shaped button in the top bar opens the **“Colour Vision Deficiency Simulation”** menu: 9 choices in 5 groups.

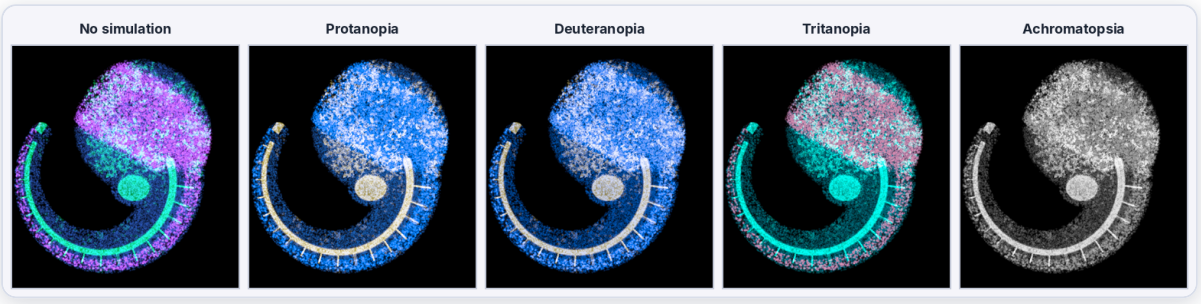



▲ The simulation menu: for each choice, seven test shades already filtered.

| N | WHAT IT IS |
|---|---|
| 1 | The title: it is indeed a **simulation**. |
| 2 | The active choice (here “Off”). |
| 3 | Deuteranopia: no sensitivity to green. |
| 4 | Achromatopsia: no colour at all. |

| GROUP | COMPLETE DEFICIENCY | PARTIAL DEFICIENCY |
|---|---|---|
| **Red (protan)** | protanopia | protanomaly |
| **Green (deutan)** | deuteranopia | deuteranomaly |
| **Blue (tritan)** | tritanopia | tritanomaly |
| **Monochromacy** | achromatopsia | achromatomaly |

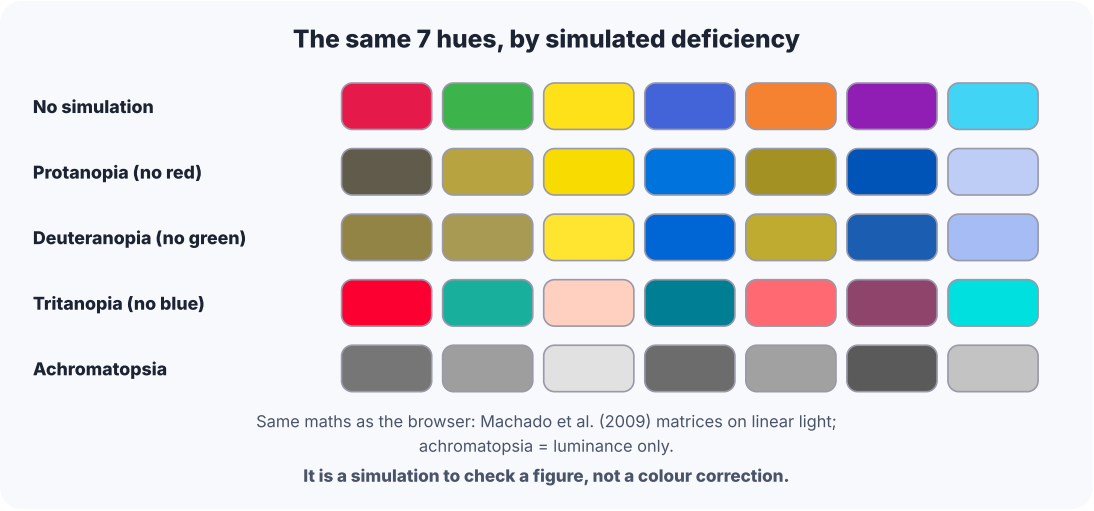
The filter applies to the **whole page**, including the 3D viewer, to see what a person with the deficiency would see:




▲ The same embryo (demonstration dataset) with the DAPI, Pecam1, Sox2 channels, under four simulations.

🔢 WORKED EXAMPLE

In this demonstration, the green and magenta of the normal view become, in protanopia and deuteranopia, **blues and beiges**: the red-green opposition that separated the two stainings disappears. In tritanopia, green becomes cyan and magenta pink. This is exactly what a publication figure should be checked for.




▲ The same seven shades, computed by the script from the code's matrices.

🔧 FOR THE CURIOUS

- **The calculation.** Each simulation is a 3×3 matrix by **Machado, Oliveira and Fernandes (2009)**, applied to **linear** light ( `color-interpolation-filters="linearRGB"` ). The "-opia" types are severity 1.0 (cone absent); the "-omaly" types, severity 0.5. Each row sums to 1: a grey stays grey. Achromatopsia replaces the three channels by the luminance (0.2126 R + 0.7152 G + 0.0722 B); achromatomaly is the half-and-half mix.
- **Where it is attached.** The filter is placed on the root `<html>` element, not on `<body>` : a filter on any other element would make it the reference frame of its `position: fixed` descendants (top bar, windows, notifications), which would scroll with the page.
- **During the choice.** The page's filter is lifted while the menu is open, so that the previews (which have their own filter) are not filtered twice.
- **Memory.** The browser's `iribhm-colorblind` key; the Compare panels follow it through the `storage` event.

⚠️ WARNING

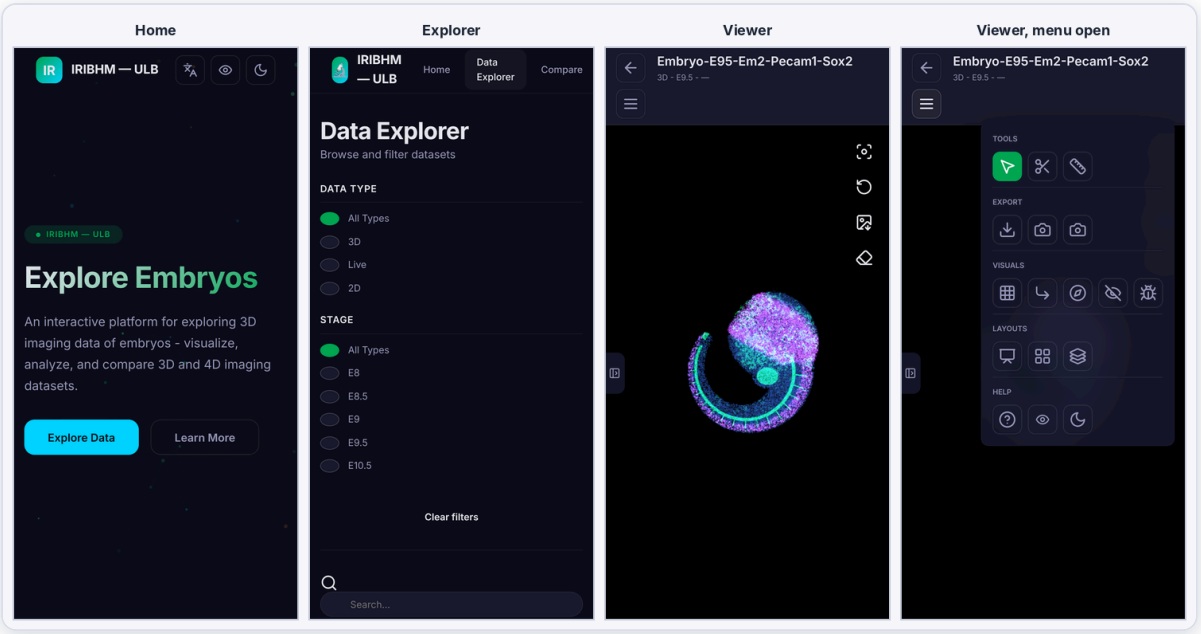
**It is not a correction.** The filter does not recolour the page to make it readable for a colour-blind person: it **shows** what they see, so that you can judge *your* figures. Nor is there any "colour-blind-friendly" palette shipped: channel colours are free. Advice from chapter 11: prefer green + magenta, or blue + yellow, to red + green.

ℹ️ GOOD TO KNOW

The simulation is a **display** effect: exports (capture, figure) are made from the render, not from the filtered screen. To judge a figure meant for print, export it and then look at the image under a simulation.

### On a phone and a tablet

The site is **usable on a phone**, with limits that the viewer cannot lift.




▲ The home page, the Explorer, the viewer and its menu, on a screen 390 pixels wide (the last image shows the ☰ menu open).

- **Touch.** One finger rotates the volume; **two fingers** move it and bring it closer (pinch). No modifier key is needed.
- **Collapsed toolbar.** When the buttons no longer fit on one line, they are tucked behind a ☰ button (third and fourth images).
- **Wider handles** on touch screens: the histogram handles go from 12 to 22 pixels.
- **Home.** Below 900 pixels, the links of the bar disappear (the bar keeps only language, filter and theme); you go through the page's buttons.
- **Studio.** Below 720 pixels, its panels stack under the image.

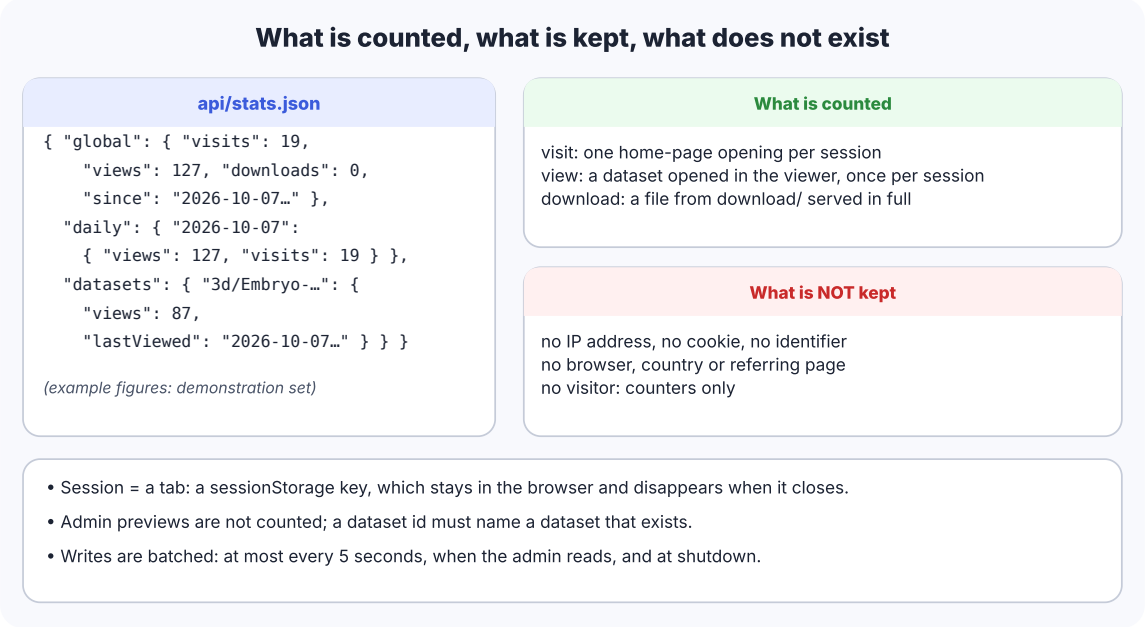
> ⚠️ **WARNING**
>
> **The limits are those of the hardware.** The viewer requires WebGL2; a phone's graphics memory is counted tightly (chapter 10). A large dataset at maximum quality may be refused there with a message: the site drops to the coarser level rather than crash.

### Fonts

The site's only remote dependency is **Google Fonts** (Inter, JetBrains Mono). The link is loaded **deferred** (`media="print"`, switched on by `font-loader.js`): the page does not wait for it to display. Offline, or if the font is blocked, the browser uses its sans-serif font: the site stays readable.

## 16.11 Usage statistics

A lab wants to know whether its datasets are being looked at. Lumen3D tells it **without knowing anything about you**.




▲ What is counted, and what is not kept.

## What is counted

| COUNTER | WHEN IT GOES UP |
|---|---|
| **Visits** | once per **browsing session** (a tab), on the **first public page** opened, whichever it is (home, explorer, viewer, 2D page, compare, about, legal, custom page…) |
| **Views** | once per session and per dataset, when a dataset opens (a volume in the viewer or a photograph in the 2D page) |
| **Downloads** | when a file of an existing dataset's `download/` folder (the one the Download Center offers) is served **in full**: a complete `GET`, not a `HEAD`, not a partial resume (`Range`) |

🔧 FOR THE CURIOUS

- The beacons are `POST` requests sent by `navigator.sendBeacon`, without waiting for an answer (they survive the closing of the tab). A `GET` is **refused** (405): a third-party image cannot inflate your figures.
- "Once per session" means: a `sessionStorage` key (`lumen_visit`, `lumen_view_<dataset>`) remembers that it has been done. It stays in the browser and disappears when the tab is closed.
- **Administration previews** (`mode=admin`) do not count: editing a dataset does not inflate its views. The admin panel counts no visit, nor does a page embedded in another (a panel of the Compare page, the admin preview, the page editor).
- A visitor who arrives **directly** at a dataset (a link received by email) counts one **visit** and one **view**: a deliberate choice, the visit is counted on the landing page, whichever it is.
- Downloads are counted **by both servers**, with the same rule, at the moment the file is served (a partial resume is not counted again). On an Apache/PHP host, the root `.htaccess` routes the request through `api/download.php`, which counts it and then redirects (302) to the same file marked `?lumen_dl=1`, which Apache serves itself (resume, ranges and the "attachment" header unchanged); under `php -S`, `router.php` counts it.

## What is kept: `api/stats.json`

Nothing but **numbers** and a date:

- `global`: visits, views, downloads, and the start date (`since`);

- `daily`: one line per **day** (never purged);
- `datasets`: per dataset (`3d/Embryo-…`), its views, its downloads, its last consultation.

**No IP address, no cookie, no visitor identifier, no browser, no country.** The file is protected from updates (your history survives) and is never served (it lives under `api/`).

🔧 **FOR THE CURIOUS**

- A dataset's identifier must designate a dataset that **exists**: otherwise the beacon counts only globally. Without this check, anyone could have added thousands of fake datasets to the file.
- The figures accumulate in memory and are written **at most every 5 seconds**, when the administration reads them and when the server stops; the write is atomic (a temporary file replaces the old one).

## Protecting against fraud: two token buckets

▲ Each beacon costs a token from two buckets; when empty, it receives 429.

💡 **A PICTURE TO UNDERSTAND**

**A ticket counter.** Each visitor has a small book of 60 tickets, refilled at one per second. The whole counter has a book of 600, refilled at 20 per second. Without a ticket in either, you are not served.

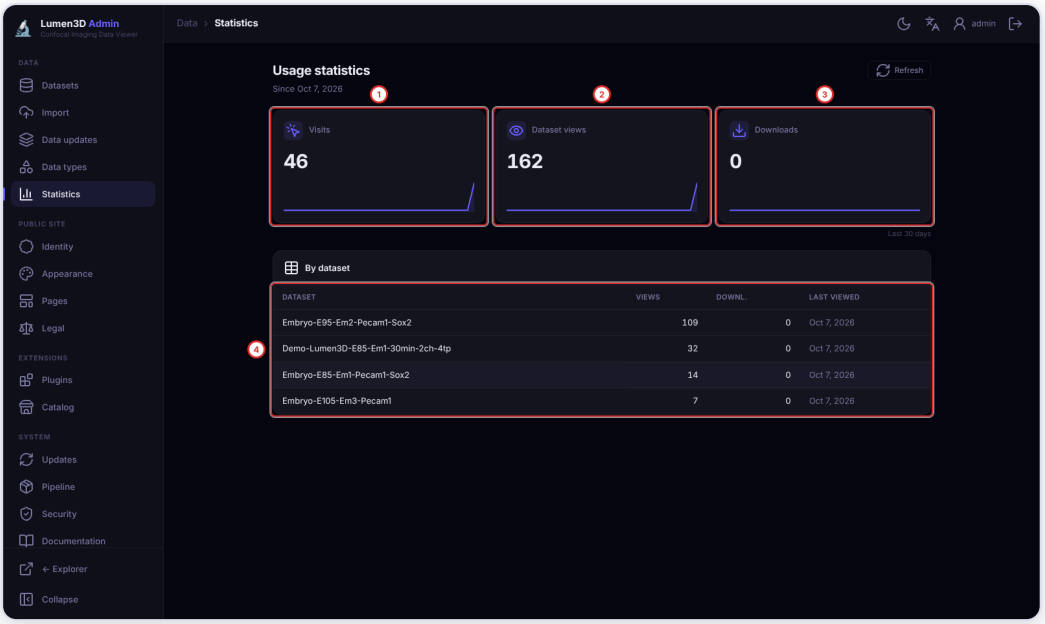
The visitor's address is used **only** to choose one slot among the 4,096 of an in-memory table (its SHA-256 fingerprint is reduced to a number); it is **neither written to disk nor into** `stats.json`. Even the table never grows: two addresses that fall on the same slot simply start from a fresh book.

⚠️ **WARNING**

The counters keep no personal data. This says nothing about the **access logs** of the web server itself (host, Apache…), which may contain IP addresses depending on its configuration: this must be declared in your legal notice (16.7).

### Reading them

The **Statistics** tab shows three cards (with the **last 30 days** plot drawn by hand in SVG, with no charting library) and a **By dataset** table that you sort by clicking a header.




▲ The Statistics tab (test figures of the demonstration dataset).

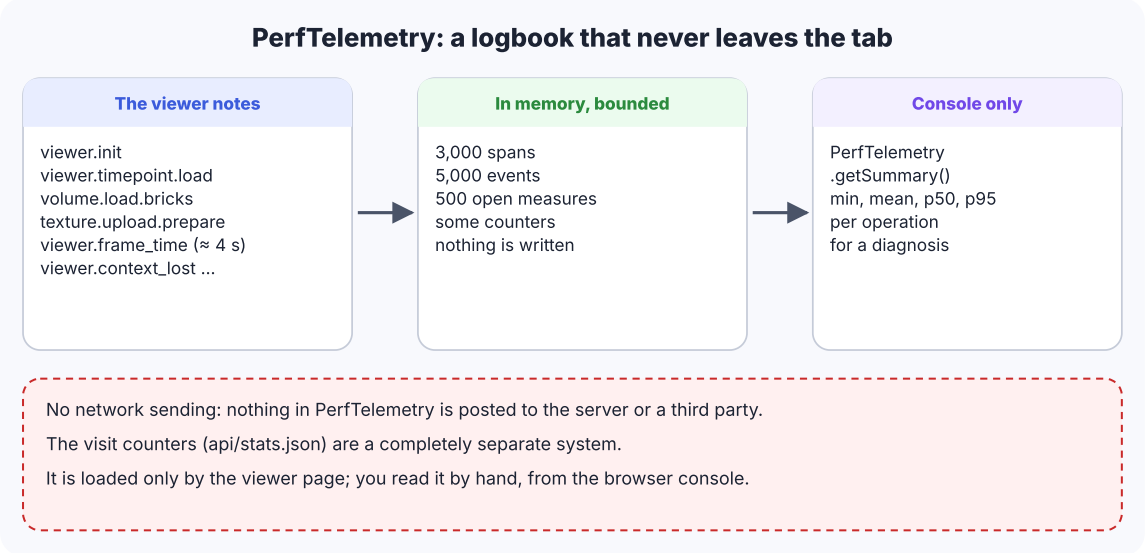
| N | WHAT IT IS |
|---|---|
| 1 | **Visits** (one per tab, on any public page), with the curve of the last 30 days. |
| 2 | **Dataset views**: the most telling indicator. |
| 3 | **Downloads**. |
| 4 | The detail per dataset: views, downloads, last view. Click a header to sort. |

Under each figure, a line recalls its definition (“One per browser tab, on any public page (admin panel and embedded pages excluded).”, “One per dataset opened in a browser tab.”, “One per complete download of a file from a dataset’s download/ folder.”).

Changing the **name** of a data type changes none of these figures. When moving from the old type vocabulary, the counters of the old keys are **added** to those of the new ones.

## 16.12 `PerfTelemetry`: the viewer’s logbook

Not to be confused with the statistics. `PerfTelemetry` is an internal stopwatch that the viewer fills in to **diagnose** its performance.




▲ PerfTelemetry: a logbook that never leaves the tab.

📌 **REMEMBER**

**Nothing is sent anywhere.** The logbook lives in the tab's memory, bounded (3,000 durations, 5,000 events, 500 measurements open at once), and disappears when the tab closes.

## What the viewer records

| MEASUREMENT | WHAT IT TIMES |
|---|---|
| `viewer.init` | opening the page until the volume is ready |
| `viewer.timepoint.load` | loading one timepoint of a series |
| `volume.load.bricks` | loading the bricks of a quality level |
| `texture.upload.prepare` | preparing the texture before sending it to the graphics card |
| `viewer.frame_time` | smoothness: mean, median and 95th-percentile frame duration (every 4 seconds, after 45 frames) |
| `viewer.context_lost` / `restored` | a loss and a recovery of the graphics context |

## Reading it

In the browser console (F12): `PerfTelemetry.getSummary()`. You get, **per operation**, the number of occurrences, the minimum, the mean, the maximum, the median (p50) and the 95th percentile (p95), plus the last 80 events. It is a developer's tool, not a visitor's.

ℹ️ **GOOD TO KNOW**

Only the viewer page loads `PerfTelemetry`. The progress line you see while loading a volume ("37% — level 2") is something else: a display, not a log.

# 16.13 Summary

## I want to… where do I go?

| I WANT TO… | I USE |
|---|---|
| change the name, the organisation, the logo | **Identity** |
| change the word “embryo” everywhere | **Identity** › *Terminology* |
| rename “3D”, “2D”, “Live” | **Data types** |
| change the site’s green, the font, the corners | **Appearance** |
| edit the home page, the About page | **Pages** |
| create a “Protocols” page | **Pages** › **New page** |
| write the legal notice | **Legal**, then tick the box in **Identity** |
| add a language | drop in `lang/<code>.json` |
| give channels a colour according to their name | `channelColorPresets` in `instance.json` |
| find out how many people look at my data | **Statistics** |

## Frequent mistakes, explained

| OBSERVATION | EXPLANATION |
|---|---|
| “My Appearance change does not show” | it is applied only after **Save**; clear the cache if the browser keeps the old `theme.css` |
| “The Legal link has disappeared” | the box is unticked by default in **Identity** |
| “The Dutch talks about embryos” | the **NL** line of *Terminology* is empty |
| “My page is saved but invisible” | the draft is saved; you must **Publish** |
| “The editor says the page is open elsewhere” | another tab is editing it: take back control or close the other |

## Going further

📖 GOING FURTHER

- **Step by step**: Administrator guide, chapters 6 (Data types), 7 (Statistics), 8 (Identity), 9 (Appearance), 10 (Pages), 11 (Legal).
- **The tests** that lock this chapter down: `tests/js/test_build_lang_parity.mjs` (language parity), `tests/js/test_plugin_lang.mjs` (plugin languages), `tests/test_dev_server_site.py` (configuration store, `{{SITE:…}}` markers, theme compilation), `tests/js/test_admin_pages_save.mjs` (draft, revision, lock), `tests/js/test_admin_branding_merge.mjs` (saving by paths), `tests/test_v3_minor_telemetry.py` (token buckets).
- **The security** of these mechanisms (allow-lists, content security policy, file permissions): chapter 19. **Plugins** and their languages: chapter 15. **Updates** that preserve `config/` : chapter 18.

# 17. Data in depth: files, formats, migrations, import

- A dataset is **a folder**: every file has an author (the pipeline, the operator or the platform), a reason to exist and, most of the time, a fingerprint that lets you check it.
- There are **five data formats**. A data update moves a dataset from one to the next **in place**, in small units recorded in a journal, with a final atomic swap.
- The browser import sends a folder **block by block**, verified and resumable, into a private area; the files that make the dataset openable go first.

Chapters 7 and 8 showed **what** bricks, `planes/`, `mips/` and `metadata.json` look like. Chapter 14 showed **where to click** in the administration panel. This chapter lifts the bonnet: it says **exactly** what each file contains, who writes it, how it changes format and how it reaches the server.

### This chapter, on one map

Three questions, three parts: what a dataset is made of, how it changes format, how it arrives.

**A — The files**

**What is a dataset made of?**

17.1 the three annotated trees
17.2 who writes what
17.3 metadata.json, field by field
17.4 manifests, index, tracks.json
17.5 catalogue and vocabulary
17.6 download/ and gallery/

**B — Formats and migrations**

**How does a dataset change format?**

17.7 formats 1 to 4
17.8 three migrations
17.9 units and journal
17.10 two executors
17.11 finalize
17.12 governor, slow hosts

**C — The browser import**

**How does a dataset arrive without SFTP?**

17.13 the journey
17.14 the allowlist
17.15 the tiers
17.16 chunks and journal
17.17 states and purge
17.18 validate, publish, replace

**Common thread: a dataset is a folder, and every file has an author, a purpose and an integrity check.**

▲ The chapter map: three parts, eighteen sections.

ℹ️ **GOOD TO KNOW**

Every worked number comes from the **demonstration datasets** (synthetic, produced by the real pipeline) or from **real trials** run on test datasets in a temporary folder. No laboratory data is shown.

## 17.1 One dataset, one folder: the three trees

A dataset lives in `DATA_WEB/<type>/<folder>/`. The **type** (`3d`, `2d` or `live`) is the name of the parent folder; the folder itself is the dataset's identifier: `3d/Embryo-E95-Em2-Pecam1-Sox2`.

> 💡 **A PICTURE TO UNDERSTAND**
>
> **A travel suitcase.** Open it and every item has its place: the identity card (`metadata.json`), the passport photo (the thumbnail), the contents (the bricks) and some souvenirs to take away (`download/`). The coloured dots in the figures tell you **who packed** what.

### The tree of a 3D dataset (demo dataset)

Dots say who creates each element; sizes are those on disk.

| | | | |
|---|---|---|---|
| **3d/Embryo-E95-Em2-Pecam1-Sox2/** | 549 MB | | one folder = one dataset |
| ├ metadata.json | 1.8 kB | 🔵🟠 | the identity card (see 17.3) |
| ├ thumbnail.webp | 33.6 kB | 🔵🟠 | thumbnail; the operator may redefine it |
| ├ **bricks/** | 9.3 MB | 🔵🟢 | v3 brick pyramid (format 4) |
| │ ├ manifest.json | 3.7 kB | 🔵🟢 | levels, histograms, index fingerprint |
| │ ├ index.bin | 7.8 kB | 🔵🟢 | pack + start + length of each brick |
| │ ├ l0/c0/p00000.bin p00001.bin … | 7.9 MB | 🔵🟢 | level 0 (native), one folder per channel |
| │ └ l1/ l2/ l3/ | 1.4 MB | 🔵🟢 | ever coarser levels |
| ├ **planes/** | 8.6 MB | 🔵🟢 | fast XY cuts (format 2) |
| │ ├ manifest.json | 360 B | 🔵🟢 | lumen-planes-v1 |
| │ └ z00000.bin … z00111.bin | 112 files | 🔵🟢 | one file per Z plane |
| ├ **mips/** | 473 kB | 🔵🟢 | per-64-plane-layer MIP (format 3) |
| │ └ manifest.json l00000/1.bin | | 🔵🟢 | lumen-mips-v1 |
| ├ **download/** | 530 MB | 🔵 | always served as an attachment |
| │ ├ ….ims ….tif _web.zip README | | 🔵 | original (hard link), composite, archive |
| │ └ …_C1_DAPI_MIP.png (one per channel) | | 🔵 | projection previews |
| ├ **gallery/** | (absent here) | 🟠 | attached images: ≤ 40 images of ≤ 8 MiB |
| │ └ a.png thumbs/a.png.webp … | | 🟠 | extension from the magic bytes |

🔵 written by the pipeline 🟠 entered / added by the operator 🟢 set by the platform (migration, import)

▲ The tree of a 3D dataset: from the identity card to the bricks, planes, mips and downloads.

Three things to notice in this tree:

- the `bricks/`, `planes/` and `mips/` folders carry **a blue and a green dot**: the pipeline writes them directly (format 4), or the platform built them through a migration (section 17.8);
- `gallery/` exists only if the operator attached images;
- all of `DATA_WEB/` is served as it stands, except `download/`, which is **always** served as an attachment (section 17.6).

### The tree of a “live” timelapse (demo dataset)

Same idea as a 3D dataset, but every frame (tNNN) has its own tree; the tracking lives at the root.

| | | | |
|---|---|---|---|
| **live/Demo-Lumen3D-E85-Em1-30min-2ch-4tp/** | 5.8 MB | | timelapse: 4 timepoints, 2 channels |
| ├ metadata.json | 7.7 kB | ● ● | + timeline, tracking, registration… |
| ├ thumbnail.webp | 10.3 kB | ● ● | |
| ├ tracks.json (+ .gz) | 32.7 kB / 11.0 kB | ● | tracking: 50 cells (JSON + gzip copy) |
| ├ model.glb | 412 kB | ● | tracking 3D surface (binary glTF) |
| ├ **bricks/** | 233 kB | ● ● | one brick tree PER timepoint |
| │ ├ manifest.json | 10.7 kB | ● ● | timepoints[]: one index.bin per frame |
| │ ├ t000/ index.bin l0/c0/ c1/ | 52.1 kB | ● ● | frame 0 (108-byte index) |
| │ └ t001/ t002/ t003/ | 170 kB | ● ● | the other frames, same shape |
| ├ **planes/** | 251 kB | ● ● | one sub-folder per frame |
| │ └ t000/ z00000.bin … z00023.bin | 60.5 kB | ● ● | 24 planes per frame |
| ├ **mips/** | 31.6 kB | ● ● | |
| │ └ t000/ manifest.json l00000.bin | 7.7 kB | ● ● | one layer (24 planes < 64) |
| └ download/ | 4.8 MB | ● | the .ims holds all 4 frames |

● written by the pipeline ● entered / added by the operator ● set by the platform (migration, import)

▲ The tree of a time series: one brick tree per frame, tracking at the root.

In a `live` series, **each frame** has its own tree: `bricks/t000/`, `bricks/t001/`… each with its `index.bin`. The `planes/` and `mips/` folders follow the same split (`planes/t000/`, `mips/t000/`). Cell tracking (`tracks.json`, `model.glb`) sits at the root of the dataset: it describes the whole series.

### The tree of a 2D photograph (demo dataset)

No bricks, no channels: three copies of one image and an identity card.

| | | | |
|---|---|---|---|
| **2d/DLL4xCD1-E95-x3.2-240913-1/** | 8.5 MB | | one stereomicroscope photograph |
| ├ metadata.json | 1.3 kB | ● ● | image{} block, pixelSizeUm, acquisition{}… |
| ├ image.webp | 191 kB | ● | the displayed photo (lossy WebP, q90) |
| ├ preview.webp | 10.6 kB | ● | 640 px, painted first |
| ├ thumbnail.webp | 12.8 kB | ● | square, for the explorer |
| ├ **download/** | 8.3 MB | ● | the original TIFF (untouched) and a notice |
| │ └ ….tif (original) README.txt | | ● | |
| └ gallery/ | (absent here) | ● | possible for a photo too |

● written by the pipeline ● entered / added by the operator ● set by the platform (migration, import)

▲ The tree of a 2D photograph: three copies of one image and an identity card.

A 2D photograph has neither bricks nor channels. The file `image.webp` is the display copy; the original TIFF stays untouched in `download/` (it is a hard link: it takes no extra space).

## Work-in-progress folders

During an operation the platform builds **next to** the dataset, then swaps. These temporary folders have recognisable names; a folder whose name starts with a dot is **never** listed in the catalogue.

| NAME | CREATED BY | DISAPPEARS WHEN |
|---|---|---|
| `.planes-incoming/`, `.mips-incoming/`, `.bricks-incoming/` | a migration (assembly) | the swap is done |
| `.planes-old/`, `.mips-old/`, `bricks.v2-old/` | a migration (previous state set aside) | the version is raised |
| `.incoming-<dataset>-<date>/` | the import (copy between two disks) | the final rename, or the next start-up |
| `.replaced-<dataset>-<date>/` | the import (old dataset being replaced) | publication is finished |
| `.swap-in-progress` | the pipeline (installing a new run) | the installation is finished |
| `.lumen-tmp-*` | any “all or nothing” file write | the final rename |

📌 **REMEMBER**

A file is never written “in place”: a neighbouring temporary file is written, then **renamed**. A rename is atomic: the visitor sees the old file or the new one, never half of each.

## 17.2 Who writes what: three authors

A single `metadata.json` is touched by three authors. So that each respects the others’ work, **every field has an owner**.

### Three authors for one metadata.json

Every field has an owner; the other two respect its choices.

**1. The pipeline measures**

id, type, formatVersion
dimensions, voxel_size
physicalSizeUm, acquisitionExtentUm
calibrationStatus / Note
channels[] (starting values)
volumeSources[], thumbnail
timeline, intensityNormalization
tracking, registration
stage, embryo (read from the name)
created, description (initial)

**2. The operator adjusts**

name, description
stage, stageNumeric, embryo
voxel_size (hand calibration)
channels[]: name, colour, window
exposure
orientation, orientationAxes
upsideDown, orientation2d
gallery[] (captions, order)
hidden (visibility)
line, staining, reporter, tags...

**3. The platform sets**

id, type, folderName (every save)
configured = true
lastModified
formatVersion (migration)
hidden = true (on publish)
gallery[]: thumb, added
never: computed fields (staging...)

**When the same dataset is redone or replaced**

| | |
|---|---|
| **Pipeline redoes it:** | measurements from the new run; the 20 “curated” keys of the old file win. |
| **Import replaces it:** | the new file wins; a curated key is carried only if it is missing. |
| **In both cases:** | formatVersion is never taken from the old file (it describes the disk). |

▲ The pipeline measures, the operator tunes, the platform stamps; two merge rules.

### The pipeline measures

The pipeline writes everything that can be **measured** in the acquisition: dimensions, voxel sizes, initial channels, volume sources and, for a series, `timeline`, `intensityNormalization`, `tracking` and `registration`. It also writes the starting value of fields the operator may correct (name, stage, description).

### The operator tunes

The administration editor (chapter 14, section 14.4) sends the fields it modifies. The server **merges** them into the existing file, under a lock, then imposes `id`, `type`, `folderName`, `configured` and `lastModified` itself. The gallery is **reconciled** with the `gallery/` folder: the submission decides the order and the captions, the folder decides which files exist.

### The twenty "curated" keys

When a dataset is **rebuilt** or **replaced**, a fixed list of twenty keys is protected, both in the pipeline and in the import (`CURATED_KEYS`):

| FAMILY | KEYS |
|---|---|
| **Identity** | `name`, `description`, `stage`, `stageNumeric`, `embryo`, `line`, `staining`, `reporter`, `tags`, `notes`, `created` |
| **Display** | `hidden`, `orientation`, `orientationAxes`, `upsideDown`, `defaultView`, `exposure` |
| **Links and images** | `gallery`, `linkedTrackingId`, `relatedIds` |

### Two merge directions

The protection does not work the same way in the two cases, and the difference matters.

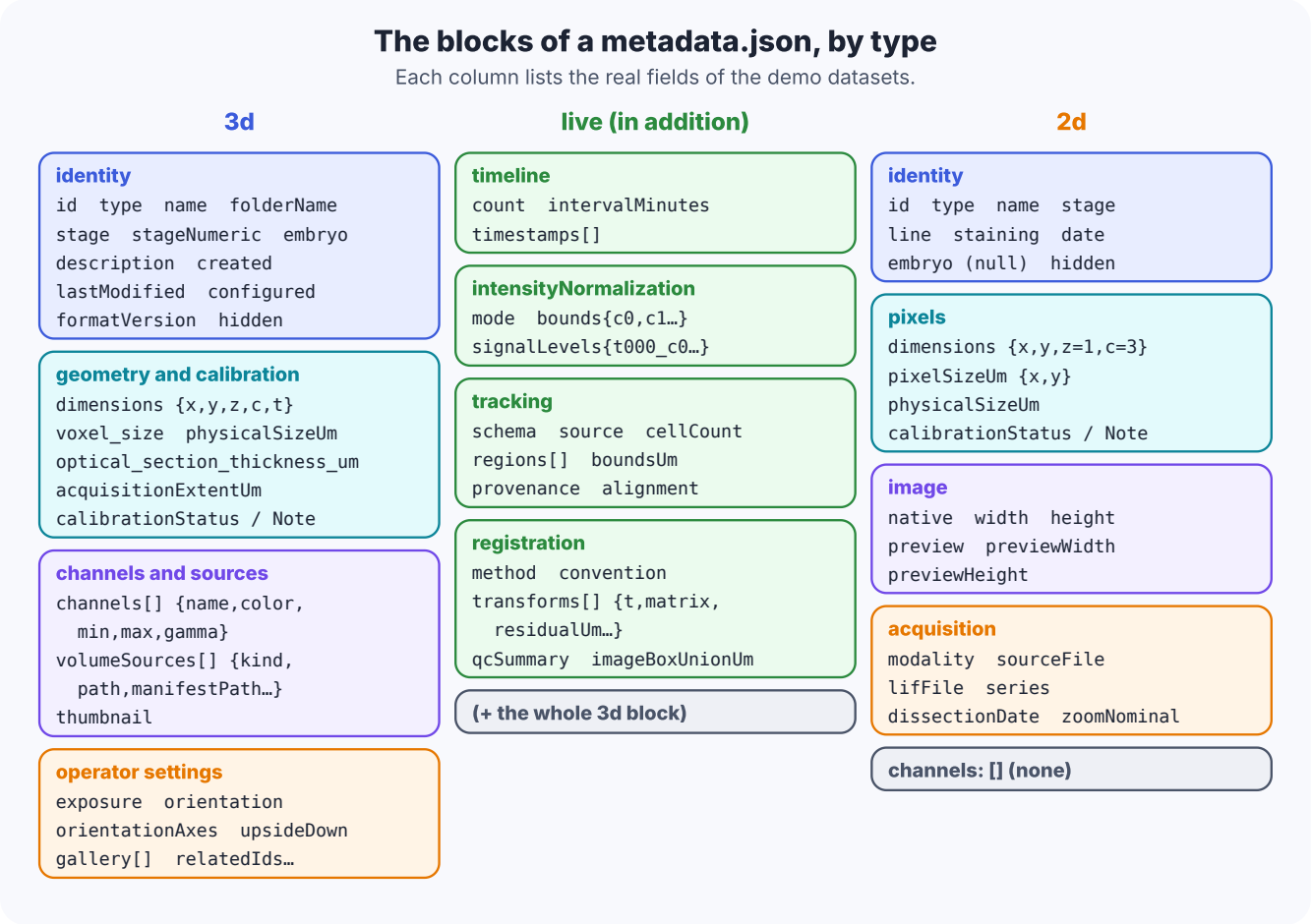
| SITUATION | WHO WINS | CONSEQUENCE |
|---|---|---|
| **The pipeline rebuilds a published dataset** | the **old** file, for the 20 curated keys | your corrected name, your orientation and your gallery survive |
| **The import replaces a published dataset** | the **new** file; a curated key is carried over **only if the new file does not contain it** | `orientation` and `gallery` survive; a `name` that the new file states **wins** |

🔢 **WORKED EXAMPLE**

**A real trial** (import engine, temporary folder). A published dataset had been edited: `orientation`, a gallery, a corrected name. It was replaced with the pipeline's folder. The server's answer: `carriedKeys = [gallery, orientation]` and `carriedGallery = true`. The name, on the other hand, **went back to the pipeline's**: the new `metadata.json` contains it, so it wins. And `formatVersion` is **4** (the new file's value), never the old number.

> ⚠️ **WARNING**
>
> `formatVersion` is **never** carried over from the old file, in either case: it describes what is really on disk.

## 17.3 `metadata.json`, field by field




▲ The blocks of a metadata.json: the same skeleton, three shapes.

The tables below give, for each field, **what it means** (with the real value from the demonstration dataset) and **who writes it**.

### Identity and state

| FIELD | WHAT IT IS | WRITTEN BY |
|---|---|---|
| `id` | `"3d/Embryo-E95-Em2-Pecam1-Sox2"` : type and folder | pipeline; **imposed** at every save |
| `type` | `3d` , `2d` or `live` ; always equal to the parent folder | pipeline; imposed |
| `name` | displayed name | pipeline (folder name), then operator |
| `folderName` | folder name | pipeline; imposed |

| FIELD | WHAT IT IS | WRITTEN BY |
|---|---|---|
| `stage` , `stageNumeric` | `"E9.5"` and `9.5` , read from the name (chapter 8) | pipeline; the editor recomputes `stageNumeric` when you change `stage` |
| `embryo` | `"Em2"` | pipeline; operator |
| `description` | starting sentence, for example “Confocal imaging stack: E9.5 fixed embryo, 112 slices, 3 channels.” | pipeline; operator |
| `created` , `lastModified` | ISO dates; `created` is protected; `lastModified` is refreshed at every save | pipeline; platform |
| `configured` | `true` ; decides whether the dataset enters the catalogue (section 17.5) | pipeline; imposed |
| `formatVersion` | `4` or `5` ; absent = 1 | pipeline, or the end of a migration |
| `hidden` | `true` = invisible to the public, visible in the administration panel | operator; **the import always publishes hidden** |

## Geometry and calibration

| FIELD | WHAT IT IS | WRITTEN BY |
|---|---|---|
| `dimensions` | `{x:768, y:576, z:112, c:3, t:1}` : voxels in X, Y, Z, number of channels, number of frames | pipeline |
| `voxel_size` | `{x:1.2, y:1.2, z:3.0}` in µm | pipeline; the operator may correct it by hand |
| `physicalSizeUm` | `{x:921.6, y:691.2, z:336.0, sliceThickness:3.0, voxelX, voxelY, voxelZ}` : real extent in µm | pipeline |
| `optical_section_thickness_um` | `3.0` : section thickness, declared **equal to the Z step** | pipeline |
| `acquisitionExtentUm` | `{unit:"um", min:[0,0,0], max:[921.6,691.2,336]}` : the acquisition box, in Imaris’s coordinate frame | pipeline |
| `calibrationStatus` , `calibrationNote` | `exact` or `metadata-missing` , with a one-sentence explanation | pipeline |

🔧 FOR THE CURIOUS

**Why declare `sliceThickness` equal to the Z step?** The viewer models depth as (D − 1) steps plus one section thickness. Without an explicit value it guesses `min(Z step, voxel X)`, which underestimates the depth of an anisotropic stack (329.50 µm instead of the 333.87 µm Imaris declares). Yet everything recorded in Imaris's coordinates (the tracked cells) must land on the right voxels.

The **calibration status** has two vocabularies: the pipeline writes only `exact` or `metadata-missing`; the viewer recomputes its own status from what it receives (`exact` only if a section thickness is present, otherwise `estimated` or `metadata-missing`, chapter 9).

## Channels and sources

| FIELD | WHAT IT IS | WRITTEN BY |
|---|---|---|
| `channels[]` | `{name:"DAPI", color:"#3D7BFF", min:0.0, max:1.0, gamma:1.0}`; the editor adds `active` | pipeline (starting values), then operator |
| `volumeSources[]` | `{kind:"bricks", label, priority:-1, available:true, multiscale:true, path, manifestPath}`: where the viewer finds the bricks; the known kinds are `bricks` and `webstack` | pipeline |
| `thumbnail` | path of the thumbnail, or `null`; the catalogue **recomputes** it from whether the file exists | pipeline |

Channel settings are kept when a dataset is rebuilt **as long as the number of channels has not changed**: with a different set of channels, the old settings would belong to other data.

## Operator settings

| FIELD | WHAT IT IS | WRITTEN BY |
|---|---|---|
| `exposure` | exposure multiplier at opening (the slider runs from 20 % to 500 %, i.e. 0.2 to 5) | operator |
| `orientation` | quaternion `Q_base`: it takes the file's axes to the anatomy (chapter 12) | operator, via the orientation tool |
| `orientationAxes` | `{labels:{A:"Rostral"}, hidden:["D"], defaultView:{preset, quaternion}}`: arm names, hidden arms, default view | operator |
| `upsideDown` | `true` if the raw file shows the sample from below | operator (**Sample side**) |
| `gallery[]` | `[{file, added, thumb, title, caption}]` | operator; the platform creates the thumbnail |

| FIELD | WHAT IT IS | WRITTEN BY |
|---|---|---|
| `relatedIds`, `linkedTrackingId` | links between datasets, entered by hand; the vocabulary conversion (section 17.5) rewrites them | operator |
| `line`, `staining`, `reporter`, `tags`, `notes` | free information; `line` and `staining` are set by the photograph importer | 2D importer, operator |

### What a `live` series adds

| BLOCK | REAL CONTENT OF THE DEMONSTRATION DATASET |
|---|---|
| `timeline` | `{count:4, intervalMinutes:30.0, timestamps: ["2026-05-12T09:00:00", …]}` |
| `intensityNormalization` | `{mode:"global", bounds:{c0:{bgFloor, sigMax}, c1: {…}}, signalLevels:{"t000_c0":33107.0, …}}` : the common window of the whole series and one signal level per frame and channel |
| `tracking` | `{schema:"iribhm-tracks-v1", source, tracksPath:"tracks.json", surfacePath:"model.glb", surfaceOrigin:"reconstructed", cellCount:50, timepointCount:4, mitosisCount:1, fusionCount:0, regions:[{name, cells, color}], boundsUm: {stabilized, raw}, provenance:{…}, alignment:{…}}` |
| `registration` | `{method:"tracking-procrustes", coordinateSpace:"acquisition-um", timepointOffset:-1, appliedToVolume:true, transforms:[…], qcSummary:{…}, imageBoxUnionUm: {…}}` |

ℹ️ **GOOD TO KNOW**

**About `intensityNormalization`.** The pipeline records these levels “so that an optional setting can one day compensate for photobleaching”. **No viewer setting reads them yet**: a series whose signal fades really does display darker.

The `tracking` block has three useful sub-blocks:

- `provenance`: where the tracks come from (`kind: "scene8"` = the Imaris objects inside the `.ims`, or `excel` = an exported workbook), the number of points (`spots: 194`) and tracks (`tracks: 48`), the declared unit and its conversion to µm (`toUm: 1.0`, `status: "native"`), the sphere radius, the extraction tool;
- `alignment`: the check made at import — `insideAcquisitionBox: 1.0` (100 % of the positions fall inside the volume’s box), `spanRatio`, `aligned: true`;
- `surfaceOrigin`: `exported` if the `.glb` file comes from the tracking software, `reconstructed` if the platform rebuilt it from the cells.

The `registration` block says **whether the stabilisation can be applied to the image**: `appliedToVolume: true` only if the fit of the tracks is **rigid** (residual ≤ 0.05 µm; here $7.6 \times 10^{-14}$ µm). Each `transforms[]` entry gives the 4 × 4 matrix (column by column, for `THREE.Matrix4.fromArray`), the angle (`rotationDeg: 7.0317` at the last frame) and the displacement in µm. Chapter 8 explains the calculation; chapter 12 shows what the viewer does with it.

### What a `2d` photograph changes

| FIELD | WHAT IT IS |
|---|---|
| `dimensions` | `{x:1920, y:1440, z:1, c:3, t:1}` : three colour channels, never any depth |
| `pixelSizeUm` , `physicalSizeUm` | `{x:2.0, y:2.0}` µm per pixel and `{x:3840, y:2880}` µm: read from the TIFF's resolution tags |
| `image` | `{native:"image.webp", width, height, preview:"preview.webp", previewWidth:640, previewHeight:480}` : **fixed** file names, the import demands them as they are |
| `acquisition` | `{modality:"brightfield-stereo", sourceFile, series, dissectionDate, zoomNominal:3.2…}` : read from the file name and the Leica block |
| `line` , `staining` , `date` | `"DLL4xCD1"` , `"X-gal"` , `"2024-09-13"` |
| `orientation2d` | `{rotationDeg, flipH}` : set by the operator with the 2D orientation tool |
| `channels` | `[]` : a photograph has no channels |

## 17.4 Manifests and description files

### `bricks/manifest.json` (format 4)

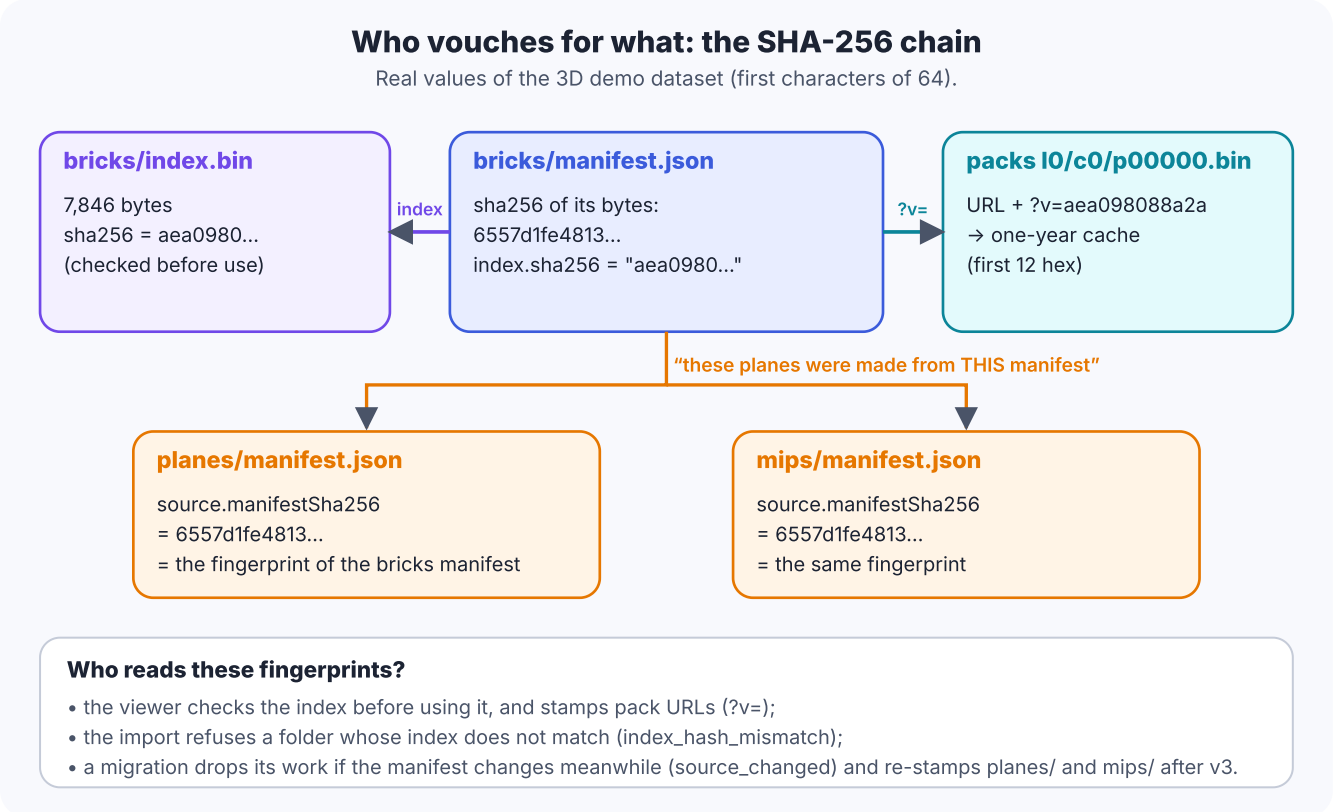
This is the table of contents of the bricks. Here are **all** its fields, with the values of the 3D demonstration dataset.

| FIELD | WHAT IT IS |
|---|---|
| `schema` , `version` , `formatVersion` | `"iribhm-bricks-v3"` , `3` , `4` : the reader picks its method from `schema` |
| `dataset` | dataset name |
| `channels` | `3` |
| `brickSize` , `apron` | `64` and `1` : a 64-voxel core with a one-voxel border (stored brick: $66^3$) |
| `brickPacking` | `{mode:"grid", cols:9, rows:8, slice:66}` : the mosaic of 66 slices of $66^2$ |
| `encoding` | `"webp-lossless"` |

| FIELD | WHAT IT IS |
|---|---|
| `levels[]` | `{level, dimensions, voxelSize, gridSize, brickCount}` per level; e.g. level 0: `768×576×112`, voxel `1.2×1.2×3.0`, grid `12×9×2`, `269` bricks present |
| `timepoints` | `null` for a 3D dataset; for a series, a list `[{path:"t000", index:{url, bytes, sha256}}, …]` |
| `index` | `{url:"index.bin", bytes:7846, sha256:"aea0980…"}`; **absent** for a series (each frame carries its own) |
| `histograms[]` | per channel: `counts` (64 bins), `edges` (65 bounds), `total`, `max`, `mean`, `std`, `backgroundFloor` |
| `timepointHistograms` | for a series: the same histograms, frame by frame |
| `producer`, `createdAt` | `"pipeline"` (or a migration executor) and the date |

🔧 FOR THE CURIOUS

**The histograms are computed on the coarsest level** (here `total: 96768` = 96 × 72 × 14 voxels). `backgroundFloor` is always `0` in the file: the viewer estimates the noise floor itself (chapter 11).




▲ Who guarantees what: the manifest's fingerprint signs the planes and MIPs, the index's fingerprint signs the packs.

## `bricks/manifest.json` (format 5)

The format-5 manifest (`"schema": "iribhm-bricks-v5"`, `"version": 5`) describes the whole matrix in one file. What it adds to format 4:

| FIELD | WHAT IT IS (CONVERTED DEMONSTRATION DATASET) |
|---|---|
| `levels[]` | one row per level, with `target` : `"native"` , then `512` , `256` ; dimensions `768×576×112` , `512×384×112` , `256×192×112` |
| `columns[]` | the five compressions: `ll` , `b2` , `b4` (with their 256-value `lut` table), `q80` , `q50` |
| `stores` | the cut copies: `xy` (512² PNG tiles), `xz` / `yz` (packs of 64 rows), `cuts` (32 × 32 × `brickZ` bricks, here 16 or 32 depending on the level) |
| `index` | one index per column ( `index/ll.bin` ...) and one per cut level ( `cuts/l0/index.bin` ...), each with its size and SHA-256 fingerprint |
| `bytes` | the bytes of each cell, per level, column and channel: what the viewer's matrix shows **without downloading anything else** |

Pack addresses carry `?v=` followed by the first 12 characters of **their own** index's fingerprint: changing an index changes the address, so the browser cache never serves an old pack.

### `index.bin`

The index is explained byte by byte in chapter 7 (section 7.6). Three additions:

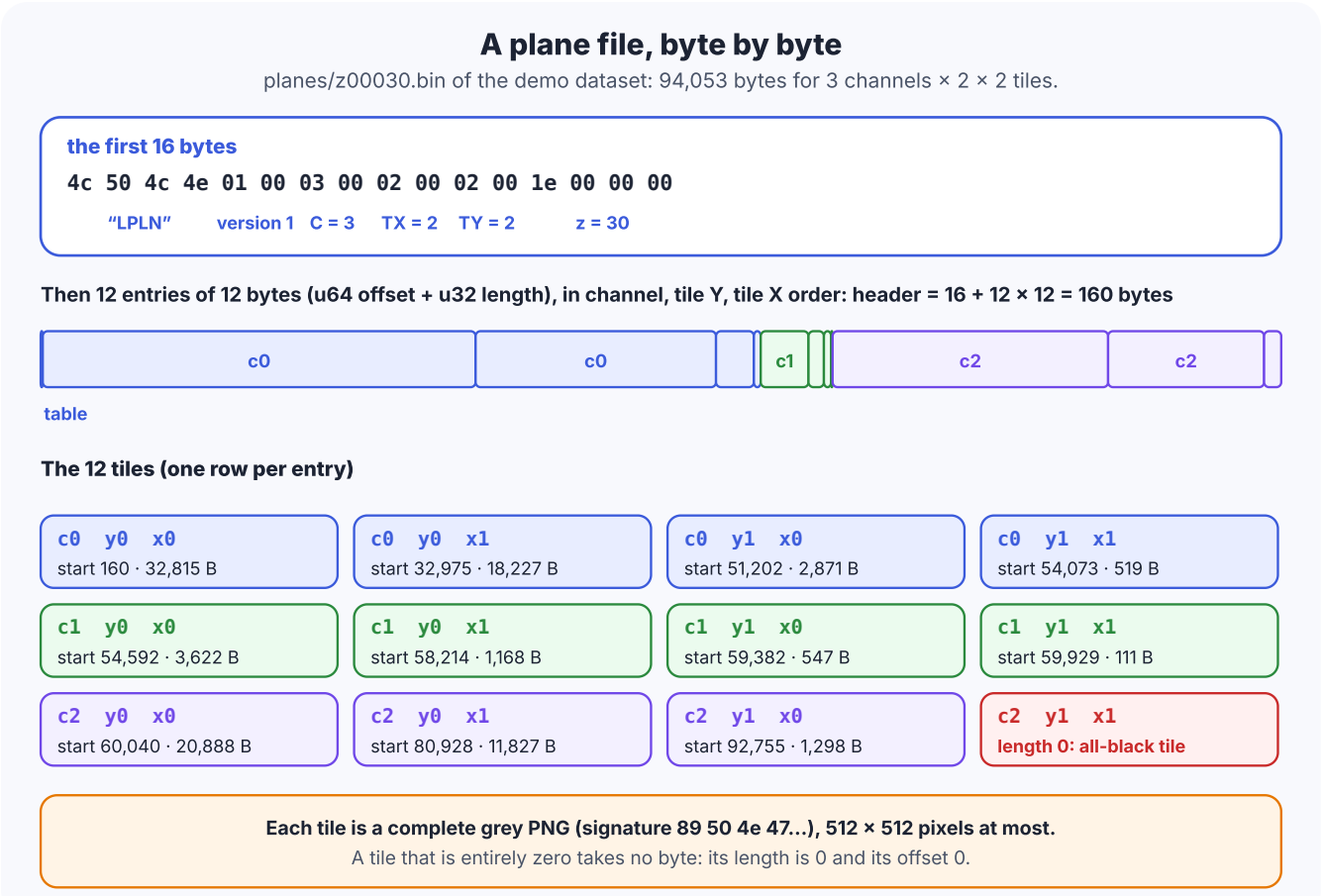
- its **size is predictable**: `12 + 16 × (number of levels) + 10 × (channels) × (sum of the grid cells)` . For the 3D dataset: 12 + 16 × 4 + 10 × 3 × 259 = **7,846 bytes**, exactly what the manifest says;
- packs are named **without any string in the index**: pack *p* of level *k* and channel *c* is called `l{k}/c{c}/p{p:05d}.bin` ;
- the import checks the **exact** length, the fingerprint and the consistency with the manifest's levels (section 17.18).

### `planes/manifest.json` and `mips/manifest.json`

Both have the same shape; `mips/` adds two fields.

| FIELD | REAL VALUE ( `PLANES/` ) | MEANING |
|---|---|---|
| `schema` | `"lumen-planes-v1"` ( `"lumen-mips-v1"` ) | |
| `formatVersion` | `2` ( `3` ) | the format that introduced the structure |
| `level` | `0` | always the native level |
| `dimensions` , `channels` | `{768, 576, 112}` , `3` | equal to those of the bricks |
| `tileSize` , `tiles` | `512` , `{x:2, y:2}` | 512² tiles: 768 = 512 + 256, 576 = 512 + 64 |

| FIELD | REAL VALUE ( `planes/` ) | MEANING |
|---|---|---|
| `codec` | `"png-gray8"` | 8-bit grey PNG, “None” filter |
| `packPattern` , `headerBytes` | `"z{z}.bin"` ( `"l{l}.bin"` ), `160` | `16 + 12 × channels × Y tiles × X tiles` |
| `layers` , `layerDepth` | (mips) `2` , `64` | one projection per layer of 64 planes |
| `source.manifestSha256` | `"6557d1fe…"` | fingerprint of the bricks manifest they come from |
| `producer` , `createdAt` | `"pipeline"` | `"pipeline"` , `"migration-browser"` , `"migration-server"` or `"mixed"` |




▲ A real plane file, byte by byte: the header, the table of 12 entries, then the PNGs.

A `MIP` file (signature `LMIP` ) has **exactly the same layout** as a plane file: its header’s “z” field holds the layer index.

## “v2” bricks in five lines

Before format 4, `bricks/manifest.json` listed **every brick** in `brickTransport.brickToPack` : a key `lod0/c0/x000_y000_z000.webp` for an entry `{url:"lod0/c0/pack_00.bin", offset, length}` . Bricks were $64^3$ with no border, arranged in an 8 × 8 mosaic ($512^2$ image); packs were called `lodN/cC/pack_NN.bin` ; a series had a `timepoints` dictionary rather than a list. The migrations of part B turn these files into their modern equivalent.

### `tracks.json`

A series' tracking lives in a separate file (`tracks.json`, and its compressed copy `tracks.json.gz`). Real structure from the demonstration dataset:

| KEY | CONTENT |
|---|---|
| `schema`, `source`, `sourceId`, `generated` | `"iribhm-tracks-v1"`, source name, unique identifier, date |
| `timepoints` | `[1.0, 2.0, 3.0, 4.0]` : the frame numbers (Imaris counts from 1) |
| `cells` | one object per cell, indexed by its number (50 cells) |
| `layout` | `{x_range, y_range, z_range}` : the extent of the stabilised positions |

One cell (number 23, which divides):

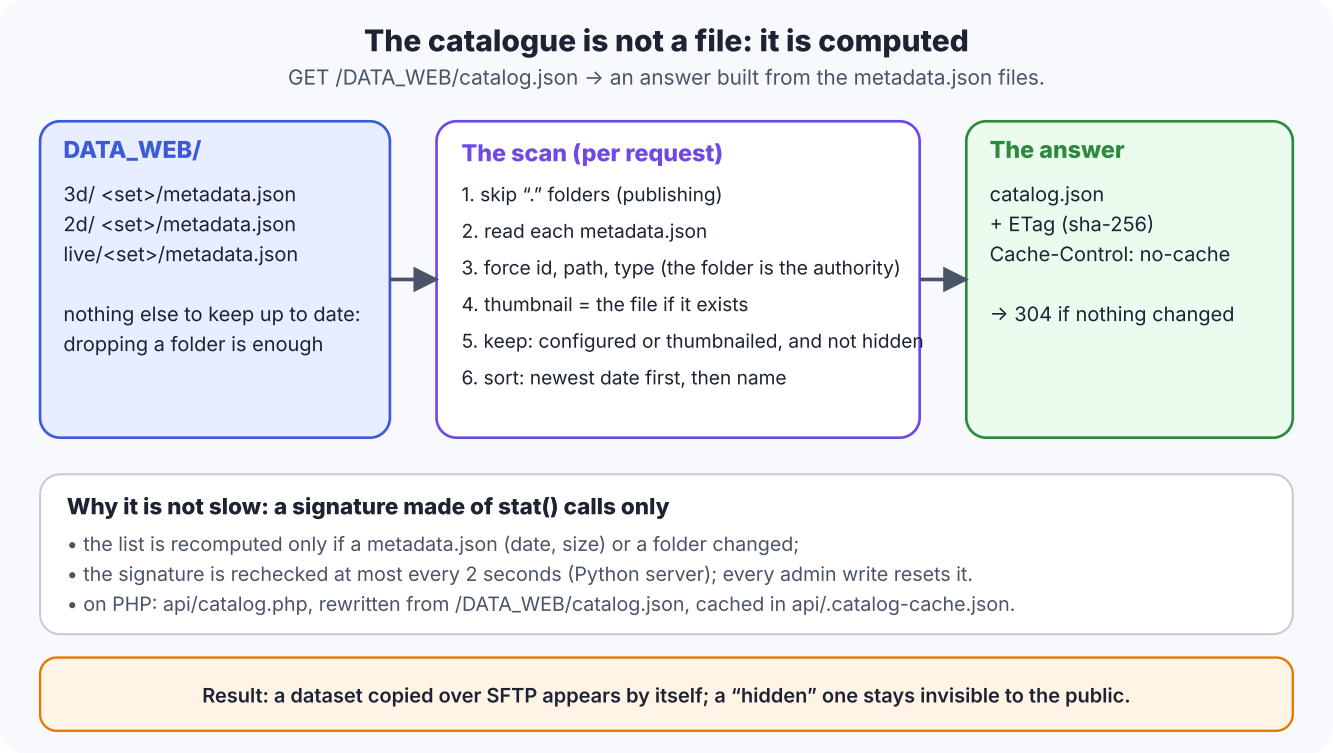
```
{ "id": "23", "track_id": 1000000023, "region": "Posterior", "color": "#2ecc71",
  "positions": { "1": [119.6, 121.3, 69.5], "2": [119.4, 121.9, 68.6] },
  "raw_positions": { "1": [119.6, 121.3, 69.5], "2": [120.3, 121.6, 69.4] },
  "parent": "", "daughters": ["24", "25"],
  "is_mitosis": true, "is_fusion": false, "markers": { "2": "red" } }
```

`positions` are the **stabilised** coordinates in µm, `raw_positions` the acquisition coordinates; `parent` and `daughters` form the **lineage** (a mitosis: one mother, two daughters that share the same `track_id`). The viewer loads this file in a Web Worker that turns it into compact 32-bit arrays.

## 17.5 The catalogue and the type vocabulary

### A catalogue that is not a file

The list of datasets the public sees (`/DATA_WEB/catalog.json`) is **stored nowhere**: it is computed on every request from the `metadata.json` files.




▲ The catalogue is computed on demand, with a signature made of stat() calls to stay fast.

The exact rules of the construction:

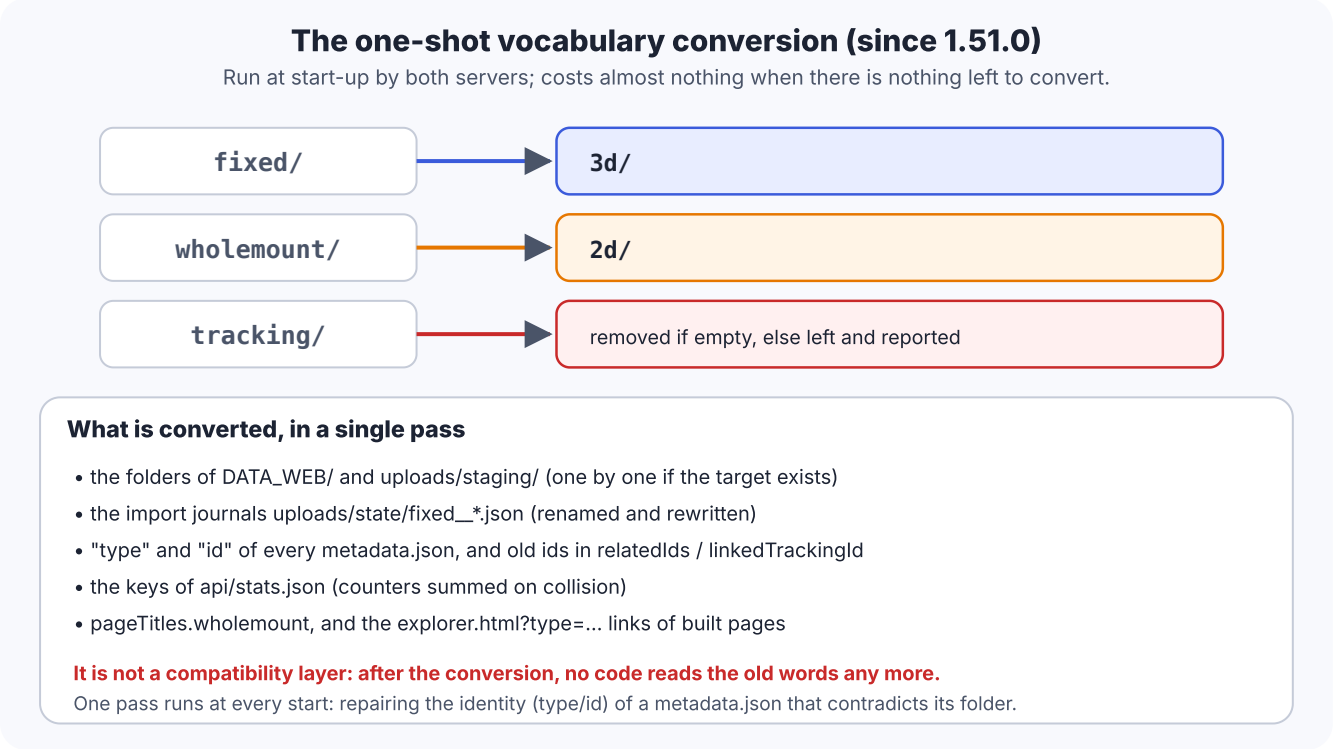
- a folder whose name starts with a dot is **never** a dataset (it is a work-in-progress folder);
- a folder **without** `metadata.json` (processing under way) gives an "unconfigured" row, with no thumbnail — so it is absent from the public catalogue;
- `id`, `path`, `type` and `folderName` are **imposed** from the location; `thumbnail` is the file if it exists, `null` otherwise;
- datasets that are **configured or have a thumbnail**, and that are **not hidden**, are kept;
- the most recent date comes first; datasets without a date (volumes rarely have one) follow, by descending name.

❓ **WHY THIS CHOICE?**

**Why not a file?** The catalogue used to be a file that only the administration panel rewrote: a dataset copied over SFTP stayed invisible until someone clicked a regenerate button. Now the catalogue **holds no information of its own**: nothing to keep up to date, so nothing to forget.

## One vocabulary, three words

Since version 1.51.0 a dataset type is written `3d`, `2d` or `live`, and that same word is at once the folder under `DATA_WEB/`, the start of the identifier, the `type` field, the value of a URL filter... The old words (`fixed`, `wholemount`, `tracking`) no longer exist anywhere in the code.




▲ The vocabulary conversion: done once at start-up, by both servers.

An old deployment is converted **once**, by an **idempotent** pass (replaying it changes nothing) run at the start of each server. It leaves **no marker**: a few folder-existence tests are enough to know there is nothing left to do.

> 🔧 **FOR THE CURIOUS**
>
> **One pass runs at every start-up.** It repairs the identity of a `metadata.json` whose `type` or `id` contradicts its folder. It exists because, before 1.54.1, a PHP host had written the editor's payload (which does not contain `type`) in place of the file. It rewrites only the inconsistent files.

What the operator **sees** is never the technical word: the displayed names ("3D", "Photo", "Live"...) come from the site configuration (chapter 16).

## 17.6 `download/` and `gallery/`

### `download/`: the souvenirs

This folder is filled by the pipeline (option `--with-downloads`, chapter 8). Three platform rules concern it:

- **everything** in it is served **as an attachment**: an HTML or XML report dropped there never displays as a page of your site;
- it is **excluded** from platform updates and data migrations: it is never touched;
- the import accepts only **data** extensions there: `ims tif tiff png jpg jpeg webp gif zip txt md csv json pdf gz h5 hdf5`.

### `gallery/`: attached images

The operator can attach images to a dataset (annotated captures, figures). They live **inside** the dataset's folder, so they travel with it.

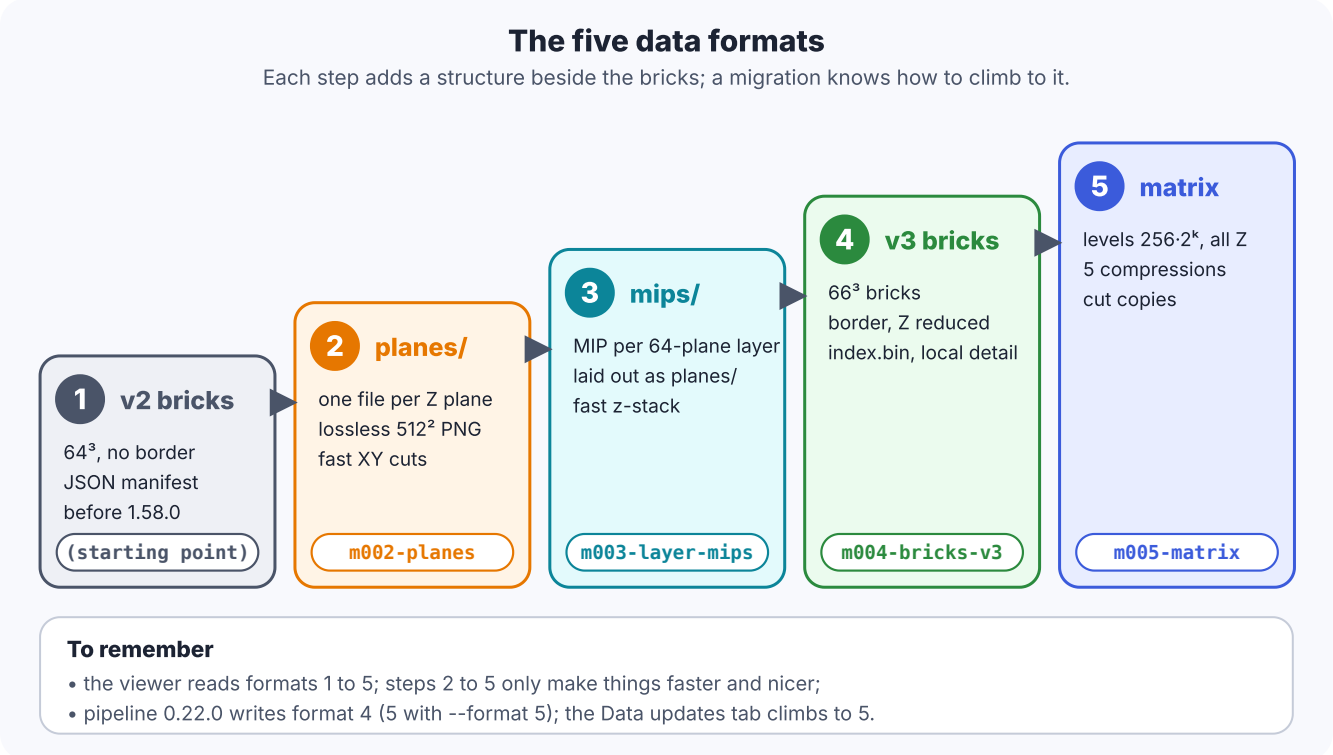
| RULE | VALUE |
|---|---|
| **Maximum number** | **40** images per dataset |
| **Maximum size** | **8 MiB** per image |
| **Formats** | WebP, PNG, JPEG, GIF; the extension is deduced from the **magic bytes**, never from the name sent |
| **Thumbnails** | `gallery/thumbs/<file>.webp` (320 px at most, never enlarged) for the grid; the viewer loads the original |
| **Order and captions** | in `metadata.json`, key `gallery` : `{file, added, thumb, title, caption}` |
| **Caption** | 400 characters at most; title 120 |

A file named `a.php.png` whose bytes are not those of a PNG is **refused**: only the bytes count.

> ⚠️ **WARNING**
>
> The gallery **does not travel** with the import (no rule of the allowlist accepts it). When a dataset is replaced, it is the platform that **moves** the gallery from the old folder to the new one (section 17.18).

## 17.7 Formats 1 to 5

A published dataset carries a **format** number (`formatVersion`). The visitor has nothing to do: the viewer reads all five formats. But the recent formats open possibilities the old ones lack.




▲ Five steps: each adds a structure; a migration knows how to climb to it.

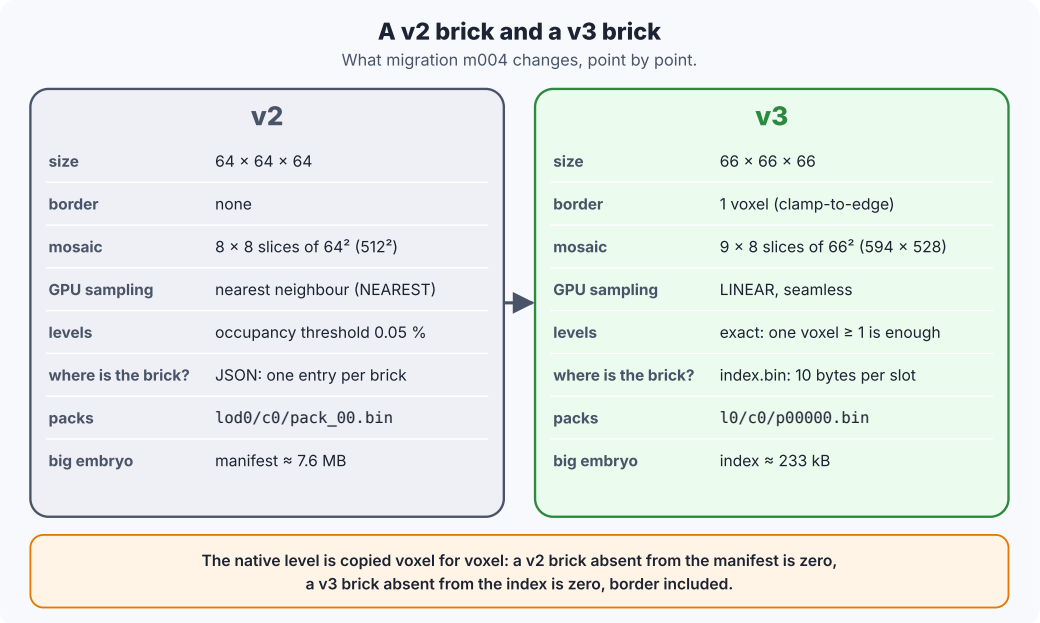
What each step unlocks:

| FORMAT | WHAT IT ALLOWS | WHERE IT IS EXPLAINED |
|---|---|---|
| 2 ( `planes/` ) | a **native** XY cut in the Studio reads one plane instead of a layer of 64; z-stack figures open fast | chapters 7 and 12 |
| 3 ( `mips/` ) | a maximum projection of the whole stack reads one image per layer | chapter 7 |
| 4 (v3 bricks) | **seamless** filtering between bricks, **local detail** (“Zoom detail”), lighter atlases for 1 or 2 channels, exact quality levels | chapters 9 and 10 |
| 5 (matrix) | levels that keep **every slice**, **five compressions** to choose from, fast native XZ / YZ / oblique cuts, exports and measurements always **lossless** | chapters 7, 10 and 12 |

## What a format number promises

A dataset **never declares a structure it does not have**. The pipeline writes the highest format whose structures are **all** complete on disk (every tree of a series included); the platform does the same. If a dataset says “format 3” but `mips/` is damaged, the **Data updates** tab offers it for **repair**.

## A v2 brick and a v3 brick




▲ What migration m004 changes, point by point.

## When does a level also reduce Z?

In the old format, reduction was mostly in XY. Format 4 applies a precise rule: the native level is copied **as it is**, then each level divides X and Y by 2; it **also** divides Z only if the voxel is not already much thicker in Z than in XY ( `vz ≤ 1.5 × vxy` of the next level).

### When does a level also halve Z?

Rule: Z is halved only if vz ≤ 1.5 × (the XY size of the next level).

**3D demo dataset (768 × 576 × 112; 1.2 × 1.2 × 3 µm)**

| level 0 | level 1 | level 2 | level 3 |
|---|---|---|---|
| 768×576×112 | 384×288×56 | 192×144×28 | 96×72×14 |
| vxy 1.20 | vxy 2.40 | vxy 4.80 | vxy 9.60 |
| vz 3.00 | vz 6.00 | vz 12.00 | vz 24.00 |
| native (verbatim) | Z halved | Z halved | Z halved |

**Large anisotropic embryo (3789 × 3789 × 257; 0.43 × 0.43 × 2.06 µm)**

| level 0 | level 1 | level 2 | level 3 | level 4 | level 5 |
|---|---|---|---|---|---|
| 3789×3789×257 | 1895×1895×257 | 948×948×129 | 474×474×65 | 237×237×33 | 119×119×17 |
| vxy 0.43 | vxy 0.86 | vxy 1.72 | vxy 3.44 | vxy 6.89 | vxy 13.77 |
| vz 2.06 | vz 2.06 | vz 4.11 | vz 8.23 | vz 16.46 | vz 32.91 |
| native (verbatim) | Z kept | Z halved | Z halved | Z halved | Z halved |

**Why? So that a coarse level has nearly cubic voxels:**
as long as the voxel is much thicker in Z than in XY, only XY is reduced; once comparable, all three axes are.
We stop when max(X, Y) ≤ 128 voxels. The mean is an integer, rounded to nearest (half up).

▲ The Z-reduction rule, on the demonstration dataset and on a large anisotropic embryo.

🔢 WORKED EXAMPLE

**The large embryo (3789 × 3789 × 257, voxel 0.43 × 0.43 × 2.06 µm).** Level 1: 0.86 µm in XY; 2.06 > 1.5 × 0.86 = 1.29, so Z is **kept**. Level 2: 1.72 µm in XY; 2.06 ≤ 1.5 × 1.72 = 2.58, so Z is **reduced** (129 planes). Six levels in all, down to 119 × 119 × 17.

The reduction is an **integer mean**, rounded to the nearest (halves round up): `(sum + n/2) // n`. The same arithmetic is used by the pipeline, Python, PHP and the browser: that is what makes their results **identical voxel for voxel**.

## 17.8 Four migrations, one registry

A **migration** transforms a dataset from version *v* to version *v + 1*. The four migrations form an **ordered registry**, identical in three languages.

| IDENTIFIER | FROM → TO | WHAT IT CREATES | APPLIES TO |
|---|---|---|---|
| `m002-planes` | 1 → 2 | `planes/` : the native level re-cut into XY planes | `3d` , `live` |
| `m003-layer-mips` | 2 → 3 | `mips/` : the maximum of each layer of 64 planes | `3d` , `live` |
| `m004-bricks-v3` | 3 → 4 | a new `bricks/` tree ($66^3$, binary index) | `3d` , `live` |
| `m005-matrix` | 4 → 5 | a new `bricks/` tree: the matrix and the cut copies | `3d` , `live` |

Each entry carries an identifier, the versions, the types concerned and a **title and a description in four languages** (this is what the tab displays).

> 🔧 **FOR THE CURIOUS**
>
> **The three twins.** The registry exists in Python ( `dataset_migrations.py` ), in PHP ( `api/_migrations_lib.php` , `_migrations_formats.php` , route `api/migrations.php` ) and in the browser (one module per migration: `js/migrations/m002-planes.js` ...). The shared **contract**, including every byte of the formats, is the specification `DOCS/dataset-migrations/SPEC.md` . Adding a migration = adding an entry in the three twins, its split into units, its two executors and a browser module; nothing else changes.

## Which dataset needs what?

A dataset at version *v* receives, **in order**, every migration whose "from" is ≥ *v*; a migration applies only if the dataset is **exactly** at its starting version. A format 1 dataset therefore gets m002, then m003, then m004, then m005.

For each **published** `3d` or `live` dataset (2D photographs are always "up to date"; a dataset still being imported is not concerned), the tab decides as follows:

| FINDING | WHAT THE TAB OFFERS |
|---|---|
| **`formatVersion` lower than the current one** | the missing migrations, in order |
| **the version announces a structure that is absent or invalid ( `planes/` missing, `mips/` damaged...)** | a **repair**: the migration that produces that structure is redone |
| **version 4 but damaged v3 bricks** | a **problem** ( `bricks_v3_invalid` ), not a repair: the old tree no longer exists |
| **version 5 but damaged v5 tree** | a **problem** ( `bricks_v5_invalid` ), for the same reason |
| **unreadable or missing manifest ( `manifest_invalid` , `no_manifest` )** | a problem flagged on the dataset's row |
| **a series whose frames do not all have the same dimensions** | refusal ( `trees_differ` ): m004 cannot convert it |

Before starting, the server **refuses a job the disk cannot hold** (error 507, like the import): it estimates the space needed for the result — about 1.3 times the native level for `planes/` , 5 % of the size of `planes/` for `mips/` , about 1.3 times the old pyramid for v3 bricks — and requires it to fit both in the working store and next to the dataset (twice on a single disk), with a 512 MiB reserve.

## A real end-to-end trial

To check every claim in this chapter, a **synthetic format 1** dataset (520 × 520 × 130 voxels, 2 channels, random noise) was converted by the real Python engine in a temporary folder.

| STEP | UNITS | TIME | SIZE BEFORE → AFTER |
|---|---|---|---|
| **m002 → format 2** | 24 (1 of them empty) | 2.4 s | `planes/` : 42.0 MB (the bricks: 40.1 MB) |
| **m003 → format 3** | 24 (1 of them empty) | 0.6 s | `mips/` : 0.70 MB |

| STEP | UNITS | TIME | SIZE BEFORE → AFTER |
|---|---|---|---|
| **m004 → format 4** | 30 | 16.3 s | `bricks/` : 40.1 MB → 48.9 MB |

> ⚠️ **WARNING**
>
> These sizes are **not representative**: random noise cannot be compressed, and format 4 adds 9.7 % of border. On real images (mostly black, with smooth signal) the v3 pyramid is generally lighter. Remember the **units** and the **mechanics**, not the megabytes.

### m005: four kinds of units

The migration to format 5 reads the **lossless native level** (the `planes/` folder and the format-4 bricks), never the `.ims` file. Its work is cut into four stages, A to D:

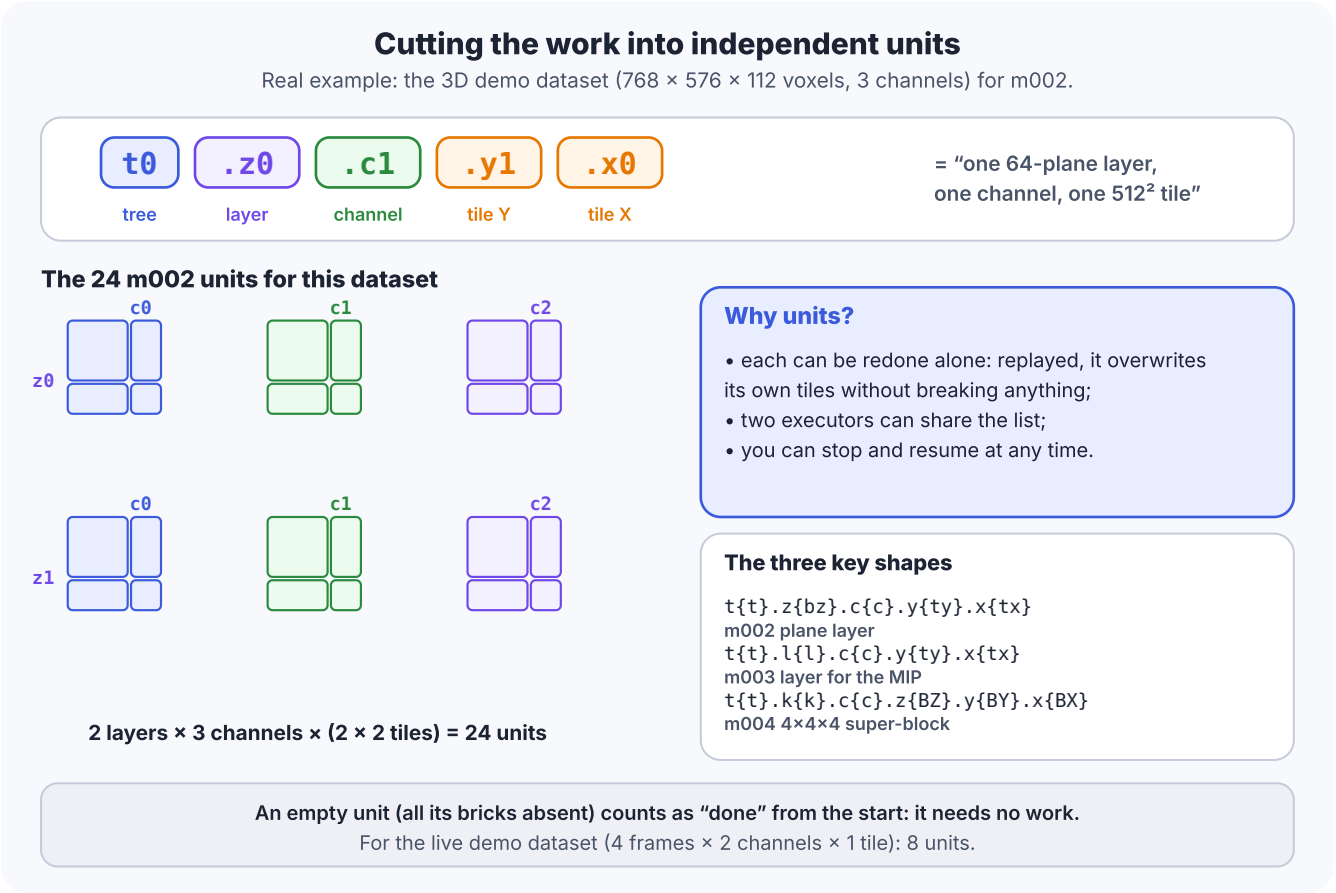
| STAGE | ONE UNIT = | WHAT IT PRODUCES |
|---|---|---|
| **A** | one native plane of one channel | that plane, reduced at every level (exact area average) |
| **B** | a 4 × 4 × 4 super-block of bricks of one level and channel | those bricks in the five columns (at level 0, all but “lossless”, taken over from format 4) |
| **C** | 64 XZ rows (or 64 YZ columns) of one level and channel | an `xz/` or `yz/` pack |
| **D** | an 8 × 8 × 8 super-block of small cut bricks | a `cuts/` record |

- Stages B, C and D of a reduced level **wait** for stage A of their channel (they read its reduced planes); those of the native level start at once.
- A unit whose area holds **no** format-4 brick is “empty”: marked done at planning, never computed.
- At the end, `finalize` assembles the new tree in `.bricks5-incoming/`, **checks it whole**, then swaps it with `bricks/`; the old one is deleted (or kept as `bricks.previous/` if the **Keep the previous version** option is ticked).

> 🔢 **WORKED EXAMPLE**
>
> **Measured in this container** (Python executor, 22 cores, `tools/convert_dataset.py`): the E9.5 demonstration dataset (768 × 576 × 112, 3 channels) goes from format 4 to format 5 in **546 units and 38 seconds**; E10.5 in 28 s; E8.5 in 11 s. Its bricks grow from 9 MB to about 80 MB on disk (section 7.11).

## 17.9 Work units and the journal

Converting a 10 GB dataset in one piece would fail at the first incident. The platform therefore cuts it into independent **units**.




▲ One unit = one layer of 64 planes, one channel, one 512² tile: here, the 24 units of the demonstration dataset.

For m002, a unit processes **one layer of bricks** (at most 64 planes in Z), **one channel** and **one tile** of 512 × 512 pixels (= 8 × 8 bricks): it decodes the matching bricks, then emits up to 64 PNG tiles, one per plane.

🔢 **WORKED EXAMPLE**

**The 24 units of the 3D demonstration dataset.** 112 planes = 2 layers (64 + 48); 768 × 576 pixels = 2 × 2 tiles; 3 channels. 2 × 3 × 4 = **24 units**. For the `live` dataset: 4 frames × 1 layer × 2 channels × 1 tile = **8 units**.

The three migrations do not have the same unit shape:

| MIGRATION | UNIT KEY | WHAT THE UNIT DOES |
|---|---|---|
| m002 | `t{t}.z{bz}.c{c}.y{ty}.x{tx}` | decodes the bricks of a layer, produces PNG tiles |
| m003 | `t{t}.l{l}.c{c}.y{ty}.x{tx}` | reads the ≤ 64 plane tiles of a layer, keeps the **maximum** pixel by pixel |
| m004 | `t{t}.k{k}.c{c}.z{BZ}.y{BY}.x{BX}` | builds the v3 bricks of a **4 × 4 × 4 super-block** of level *k* |

A unit whose bricks are **all** absent is **empty**: it is counted "done" from the start and needs no work.

### The next level waits for the previous one

For m004, level *k + 1* is computed from level *k*: it can therefore start **only once the previous level is complete**. The server returns the units that are **runnable now** ( `runnable` ) and the total remaining ( `pending` ).

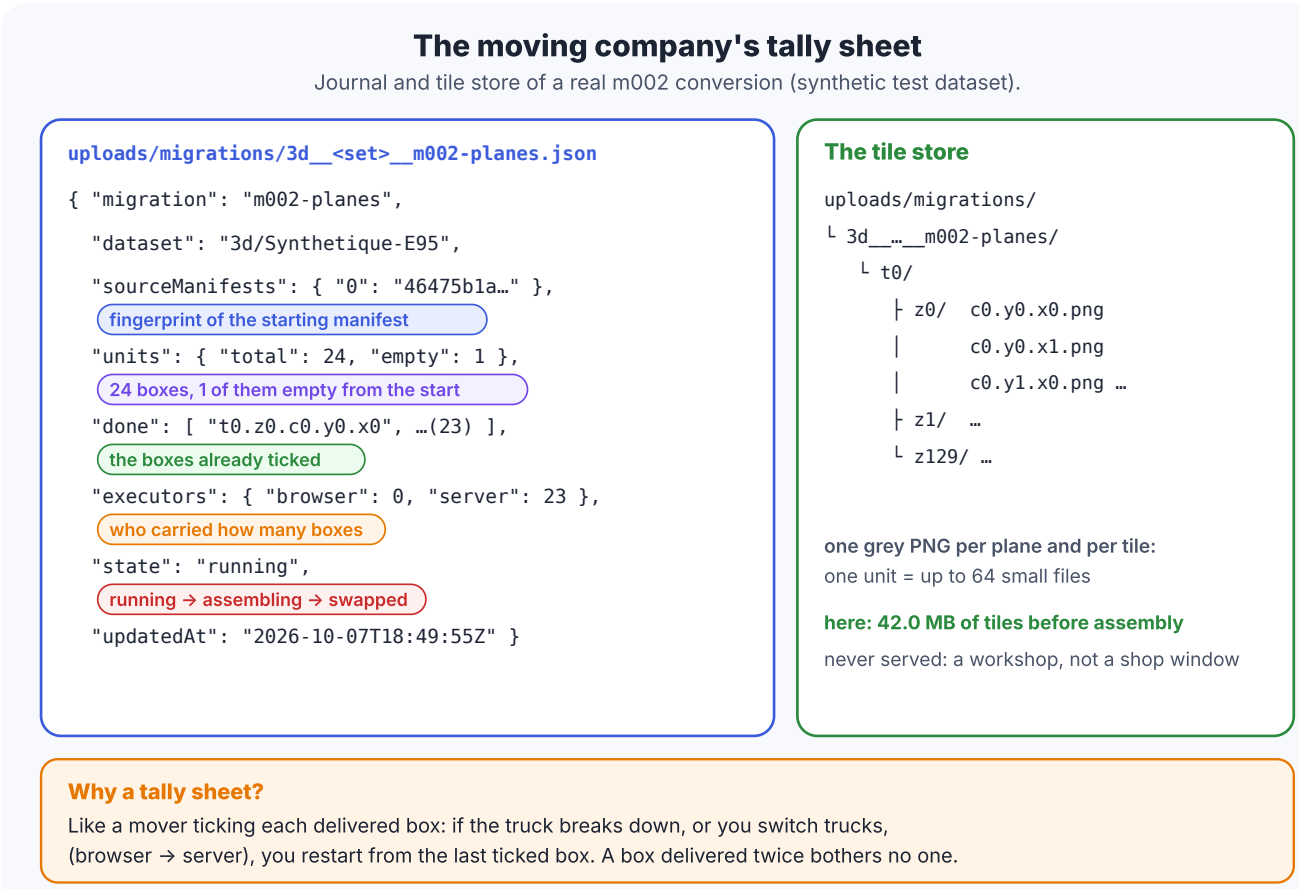
| LEVEL (REAL TRIAL) | DIMENSIONS | UNITS |
|---|---|---|
| 0 | 520 × 520 × 130 | 18 |
| 1 | 260 × 260 × 65 (Z reduced) | 8 |
| 2 | 130 × 130 × 33 | 2 |
| 3 | 65 × 65 × 17 | 2 |

At the start, 18 units out of 30 are runnable; the others wait. The browser **re-plans in waves** until none are left.

### The tally sheet

> 💡 **A PICTURE TO UNDERSTAND**
>
> **A house move.** Each unit is a **box**. The mover ticks each delivered box on a sheet. If the van breaks down, or you switch vans, you start again from the last ticked box. If a box is delivered twice, nobody notices: the second one overwrites the first.

This sheet is the **journal**, one JSON file per migration and per dataset, in `uploads/migrations/` (an area that is **never served**). Next to it, a **tile store** keeps the results of finished units.




▲ The journal of a real m002 conversion and its tile store.

| JOURNAL FIELD | MEANING |
|---|---|
| `migration`, `dataset` | which migration, on which dataset |
| `sourceManifests` | `{tree: sha256}` of each tree's starting manifest |
| `inputManifests` | (m003) fingerprint of the `planes/` manifest used as input |
| `units` | `{total, empty}`: number of units and number of empty units |
| `done` | the list of keys of finished units |
| `executors` | `{browser: n, server: n}`: who carried how many units |
| `state` | `running`, `assembling`, `swapped` or `failed` |
| `producer`, `assembledAt`, `assembly` | from assembly on: who produced, when, and the progress |
| `error`, `createdAt`, `updatedAt` | cause of a failure; dates |

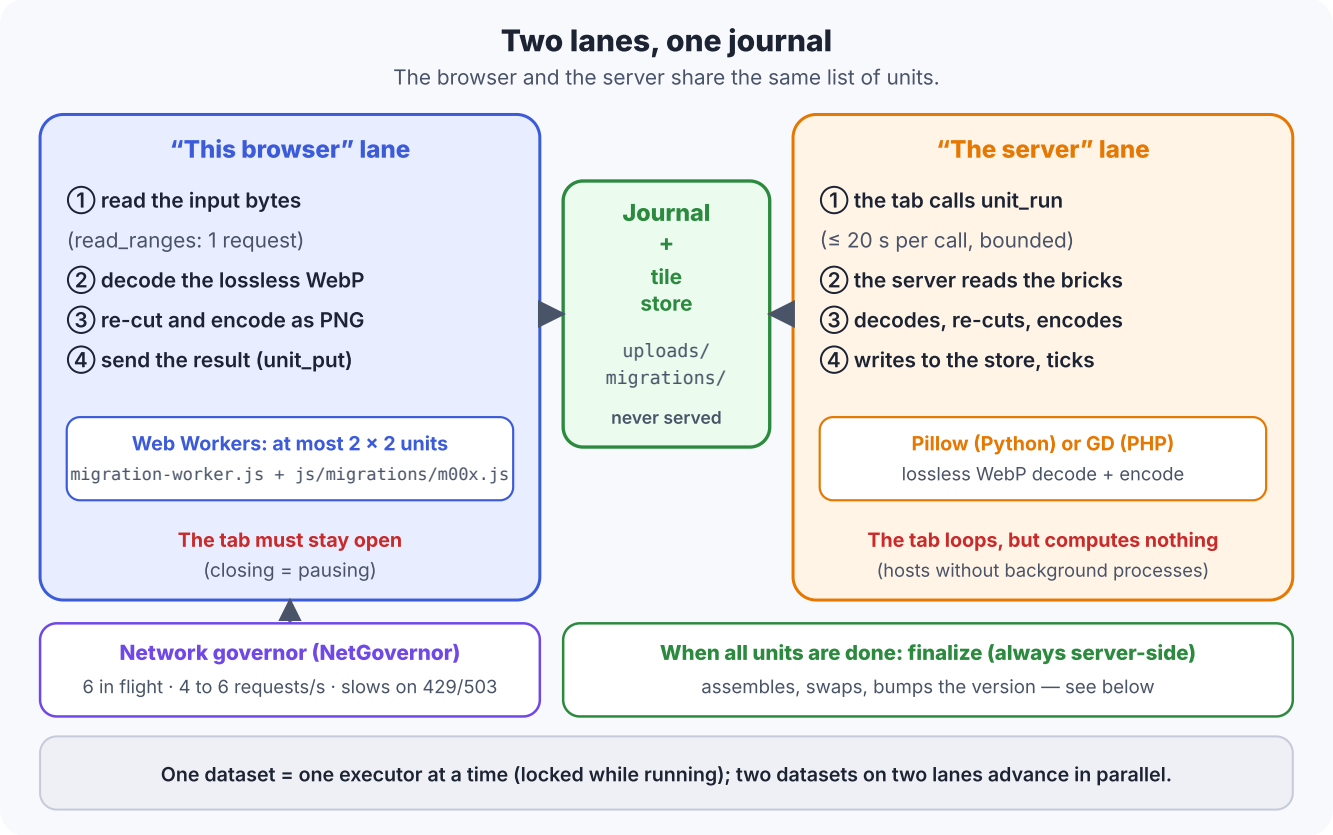
The properties that make this mechanism solid:

- the journal is written **"all or nothing"**, under a lock: two simultaneous writes do not mix;
- a **replayed** unit is harmless: it replaces its own tiles;

- if the **bricks manifest changes** during the work (the dataset was reprocessed meanwhile), the work is invalidated ( `source_changed` ): two versions of the same dataset are never mixed;
- a dataset deleted along the way is **never recreated** by a job that finishes.

## 17.10 Two executors

The work of each unit can be done by **two executors** that write into **the same journal**.




▲ Two lanes, one journal: the browser and the server share the list of units.

### The browser

A **Web Worker** ( `js/workers/migration-worker.js` ) does all the computing, with a small module specific to the migration ( `js/migrations/<id>.js` ). For each unit:

1. **read** the input bytes from the packs, by byte ranges, in **a single request** ( `read_ranges` );
2. **decode** the lossless WebP bricks, exactly as the viewer does;
3. **re-cut** and **encode**: PNG tiles (m002, m003) or v3 WebP bricks (m004);
4. **send** the result in one request ( `unit_put` , 32 MiB at most).

The tab launches **at most 2 workers × 2 units** (the number of workers is half the cores, from 1 to 2): a unit holds up to about 40 MiB in memory. The server **re-checks** everything it receives: each PNG must start with the right signature and have the right dimensions; each v3 brick must be a lossless WebP of 594 × 528 pixels, with the 6 unused cells at zero.

### The server

The tab calls `unit_run` **in a loop**; each call lasts at most 20 seconds (less on a host that limits time) and processes as many units as it can. The server itself reads the bricks and writes into the store. A 120-second "lease" prevents two calls from taking the same unit.

> ❓ **WHY THIS CHOICE?**
>
> **Why a loop driven by the browser?** Shared hosts have no background task: every computation must fit **inside one request**. It is the open tab that restarts it, hence the instruction not to close it.

### The case of format 5

- **The browser** does not plan an m005 unit itself: it asks the server **what to read** ( `unit_inputs` ), reads those bytes in grouped requests, computes the unit with the **same arithmetic** as the Python engine, and sends it. Its lossless images decode to **exactly** the same voxels as the server's; its lossy images stay within the same bounds.
- **The Python server** converts several units at once (in threads).
- **A PHP host** has **no m005 executor**: the tab shows "on this server, this update runs in the browser", and **This browser** does the work. The PHP server prepares each unit's inputs, checks what it receives and assembles the final tree, byte for byte as Python would.
- The browser probe also checks that it can encode **lossy WebP** (otherwise: "this browser cannot encode lossy WebP images").

### Converting on your own computer

For a very large dataset, or a slow host, the conversion can run **locally**: `python tools/convert_dataset.py DATA_WEB/3d/<folder>` applies every pending migration with the Python executor ( `--list` shows them without doing anything; `--keep-previous` keeps the old tree). The journal is the same as the tab's: an interrupted conversion **resumes**. The converted folder is then dragged into the **Import** tab of the other site.

### Who can do what: the probes

An executor is offered only if it **knows how** to do the step. The capability is measured **per migration** and **per executor**, by a small real trial.

| REASON SHOWN ON SCREEN | CODE | WHAT IS MISSING |
|---|---|---|
| **the server cannot decode lossless WebP images** | `no_webp_decode` | Pillow without WebP, or `imagecreatefromwebp` absent / faulty |
| **the server cannot encode lossless WebP images** | `no_webp_lossless_encode` | PHP < 8.1 with WebP, or Pillow without the lossless option |
| **the server has no NumPy** | `no_numpy` | (Python, m004) |
| **the server has no zlib compression** | `no_zlib` | |
| **the server allows less than 128 MiB of memory per request** | `low_memory` | |

| REASON SHOWN ON SCREEN | CODE | WHAT IS MISSING |
|---|---|---|
| the server stops requests after less than 10 seconds | `exec_time_too_short` | |
| this browser has no compression streams | `no_compression_stream` | `DecompressionStream` |
| this browser does not round-trip PNG tiles exactly | `no_png_codec` | |
| this browser cannot encode lossless WebP images | `no_webp_lossless_encode` | |

The server's probe decodes a tiny **built-in** WebP and compares every pixel value; the m004 one encodes a test brick, then decodes it and compares **every voxel**. The browser's probe, for m004, checks that an `OffscreenCanvas` really produces **lossless** WebP (by reading it back).

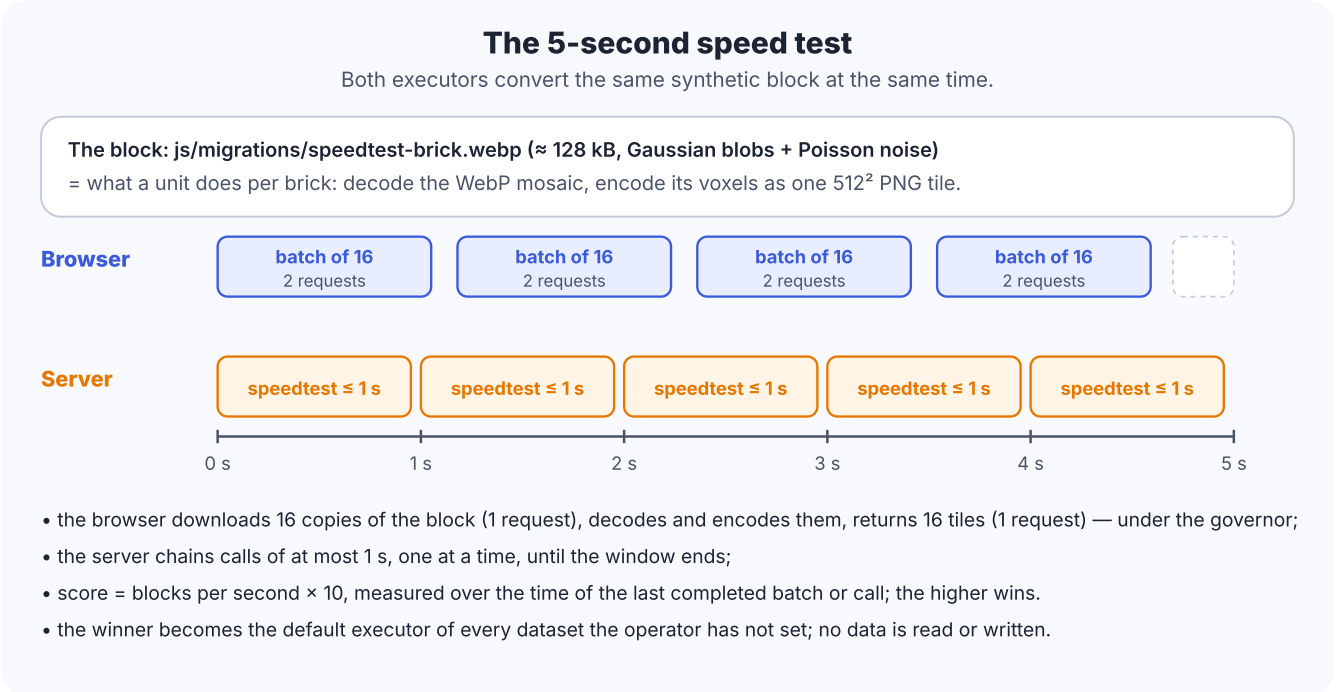
## Who is chosen when nobody chooses?

The choice is made **per dataset**, before the start, and it is **locked** while the job runs. The order of decision:

1. the operator's manual choice (remembered in this browser);
2. otherwise the **speed-test winner**;
3. otherwise the browser;
4. otherwise the server.

Each executor has **its own queue**: a dataset given to the browser and another given to the server move forward **in parallel**; two datasets given to the same executor go one after the other. A step the chosen executor cannot do is handed to the other, **in the same queue**: a dataset's steps stay in order.

## The speed test




- the browser downloads 16 copies of the block (1 request), decodes and encodes them, returns 16 tiles (1 request) — under the governor;
- the server chains calls of at most 1 s, one at a time, until the window ends;
- score = blocks per second × 10, measured over the time of the last completed batch or call; the higher wins.
- the winner becomes the default executor of every dataset the operator has not set; no data is read or written.

▲ Five seconds, one synthetic block, two executors.

The test reads and writes **no dataset**: it uses a fixed block shipped with the platform. The **score** is the number of blocks converted per second, multiplied by 10, measured over the duration of the last completed batch or call: a batch cut off by the deadline does not skew the measurement. If the browser completes no batch, it is given one more window. The result is remembered in this browser.

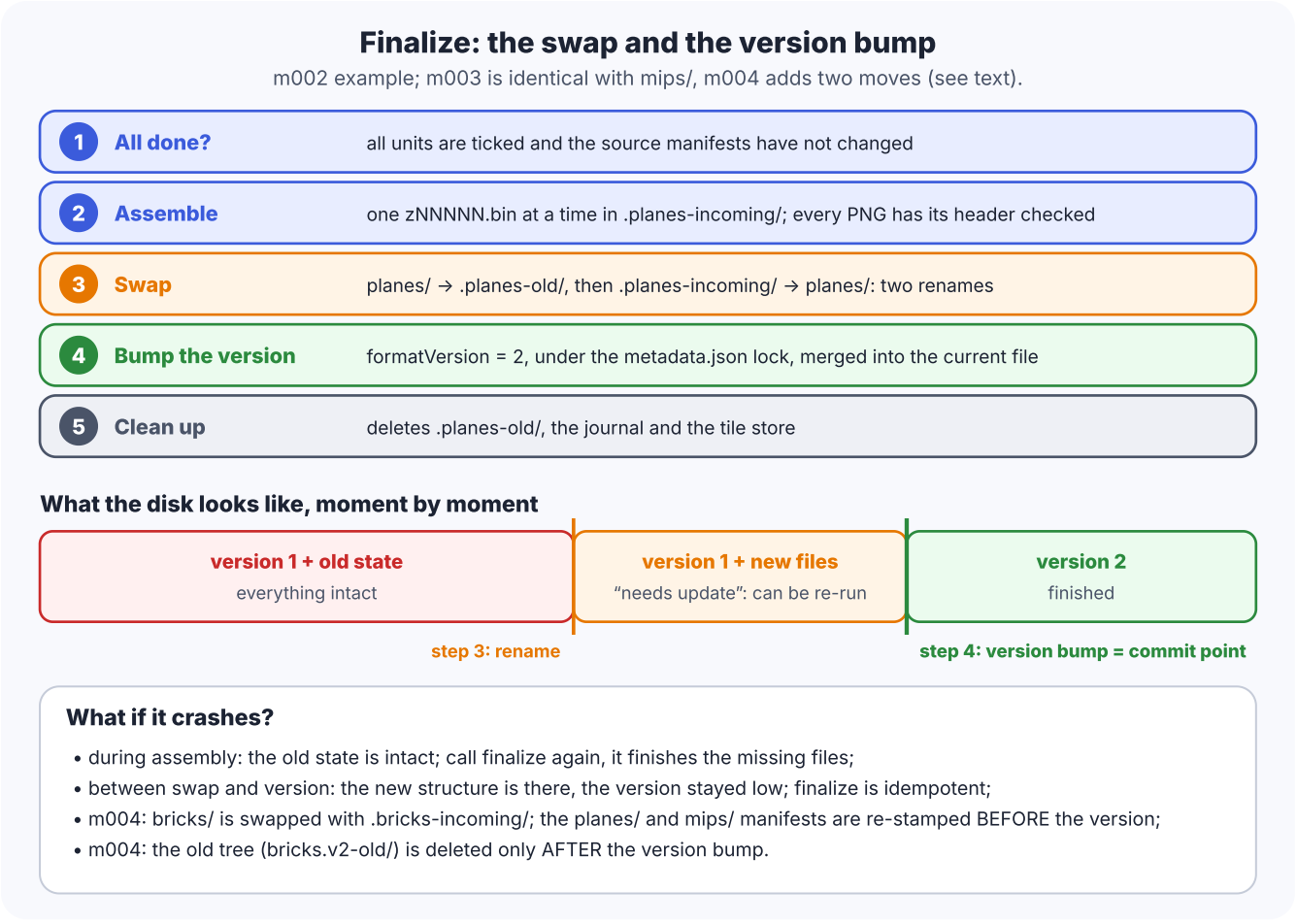
### The API actions

All of them require the administration session; those that modify something also require the anti-CSRF header and the POST method (chapter 19).

| ACTION | ROLE |
|---|---|
| `status` | the whole state: registry, datasets, server capabilities |
| `plan` | creates or resumes a job; returns the runnable units |
| `unit_put` | receives the result of a unit done by the browser |
| `unit_run` | does units on the server side, in a bounded time |
| `finalize` | assembles, swaps, raises the version (can be called again) |
| `cancel` | deletes journal and store |
| `bench` | measures the server on a few units (tool) |
| `speedtest`, `speedtest_put`, `speedtest_sample` | the three pieces of the speed test |
| `read_ranges` | returns up to 1,024 byte ranges (32 MiB) of packs, in one answer |
| `store_get_many` | returns up to 128 v3 bricks from the store, in one answer |
| `unit_inputs`, `store_get` | lists a unit's inputs; reads one stored brick |

## 17.11 `finalize`: the swap and the version change

When all units are done, what remains is to **assemble** the results into a complete structure, **install** it and **change the version number**. That is the job of `finalize`, **always run by the server**, whichever executor did the units.




▲ The five moves of finalize, and what the disk looks like at each moment.

Two ideas make it robust.

**It is resumable.** Assembly writes the files one by one into the neighbouring folder; if it is interrupted (time limit, power cut), it answers `complete: false` with its progress and you **call it again**. It resumes where it stopped.

🔢 WORKED EXAMPLE

**A real trial.** With a tiny time budget, `finalize` answered `{complete: false, assembly: {planes: 130, written: 0}}` : 130 planes to write, none written. The next call answered `{complete: true, formatVersion: 2}` . The journal went through the `assembling` state, with `producer: "migration-server"` .

**The version change is the commit point.** Throughout assembly, and even after the swap, the old number stays in place. If the server stops between the swap and the version change, the dataset has its **new files** and the **old number**: the tab offers it again, and `finalize` — idempotent — finishes the job.

## The moves specific to each migration

- **m002 and m003**: `planes/` (or `mips/` ) is renamed to `.planes-old/` , the neighbouring folder replaces it, then the old one is deleted.
- **m004**: `bricks/` is swapped with `.bricks-incoming/` ; the manifests of `planes/` and `mips/` — which had noted the fingerprint of the old manifest — are **re-signed** with the new one's (their

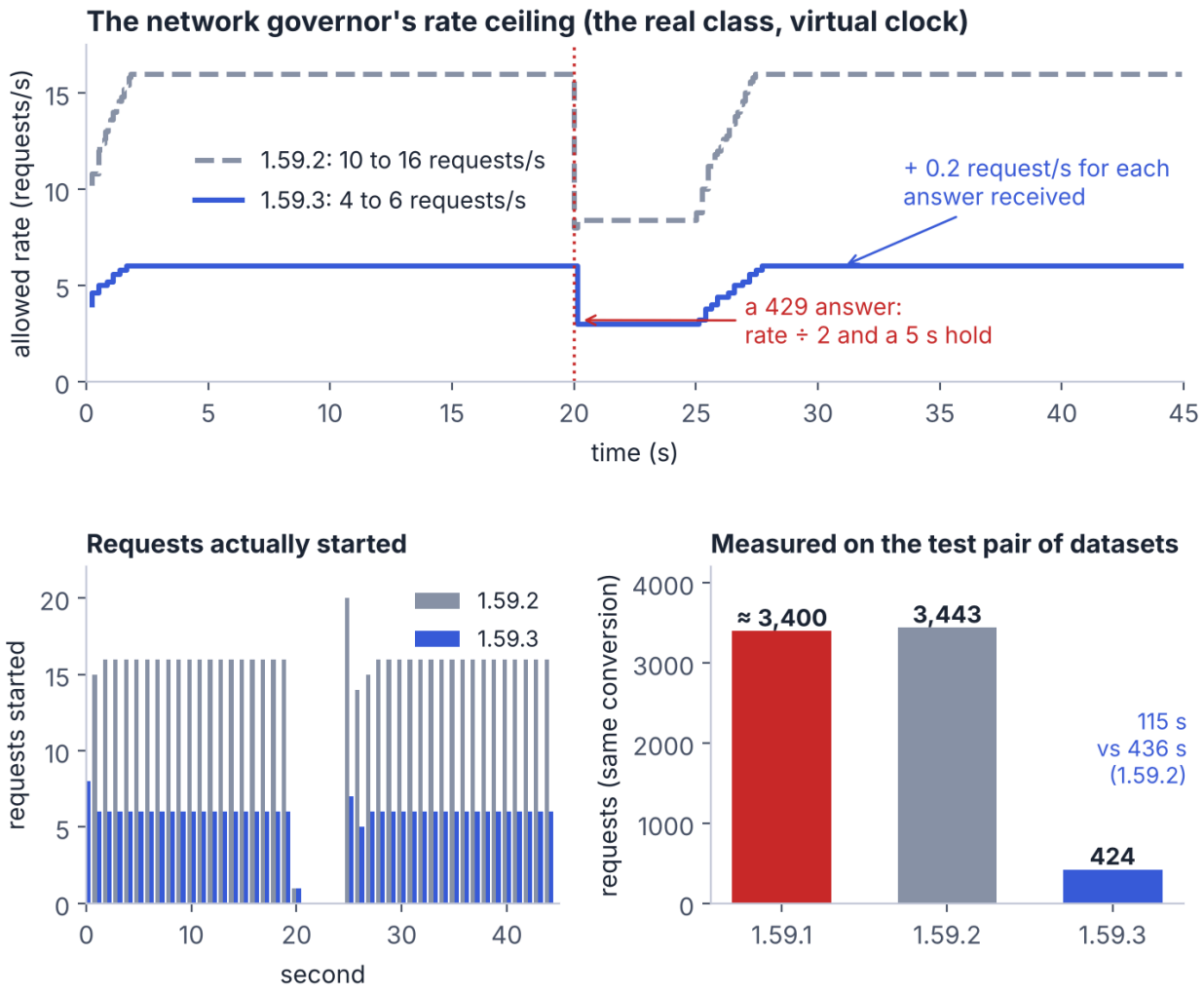
voxels, those of the native level, have not changed); the version goes up; **only then** is `bricks.v2-old/` deleted.
- before swapping, every assembled structure is **re-read and validated** (`assembly_invalid` otherwise);
- the version is raised **under the lock of** `metadata.json`, by **merging** into the current file: a save made in the editor meanwhile is not lost, and no other field is touched;
- without a journal (lost in a crash), `finalize` checks whether the structure is already there and valid: it then just re-signs and raises the version.

> 🔧 **FOR THE CURIOUS**
>
> **Why `bricks.v2-old/` stays until the end.** The viewer may still be reading the old tree when the swap happens. The file names of v3 packs (`l0/c0/p00000.bin`) look nothing like those of v2 (`lod0/c0/pack_00.bin`) and the manifest carries a different `schema`: a reader never confuses the two.

## 17.12 The network governor and slow hosts

### An incident behind the governor

In version 1.59.1, a flood of requests (two groups of workers, one request per stored brick, an unbounded speed test) led a host to **ban the operator's address**. Since then, **all** the tab's requests — its own calls and those of its workers — go through **a single governor** (`NetGovernor`).




▲ The governor (real class, virtual clock) and what 1.59.3 gained on the test pair of datasets.

Its rules, measured in the code:

- **6 requests in flight** at most;
- a **token bucket**: it starts at **4 requests per second** and never exceeds **6** (1.59.2 ran from 10 to 16, before a unit's inputs travelled together);
- a request with **no answer**, a **429** or a **503**: the rate is **halved** (floor 0.5 request per second) and **every new request is held** for 5 seconds, then 10, 20… up to 60 as long as the failures continue;
- each answer received **raises** the rate by 0.2 request per second.

The tab says so: "The host is answering slowly: requests are paced down so that it does not block this address."

## About three requests per unit

Since 1.59.3, a unit reads **all** its input ranges in **one** answer ( `read_ranges` ), its level-*k* bricks in groups of 128 ( `store_get_many` ) and sends its result in **one** request. Measured on the same pair of test datasets (two 2-channel volumes, formats 1 → 4):

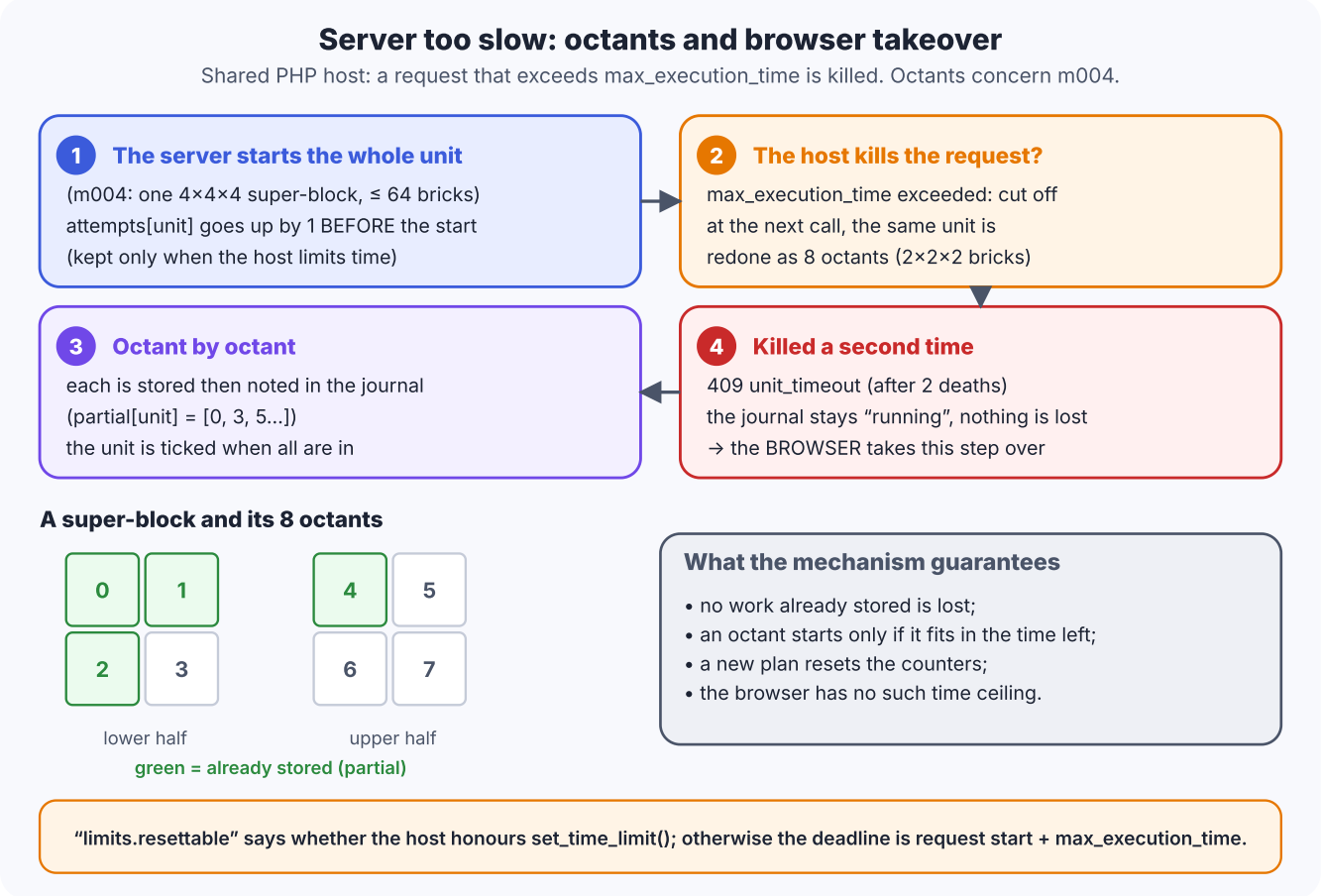
| VERSION | REQUESTS | AVERAGE RATE | DURATION |
|---|---|---|---|
| **1.59.1** | about 3,400 (over 40 per second) | — | — |

| VERSION | REQUESTS | AVERAGE RATE | DURATION |
|---|---|---|---|
| **1.59.2** | 3,443 | 8 per second | 436 s |
| **1.59.3** | **424** | 3.7 per second (11 at most in the worst second) | **115 s** |

A server that does not know `read_ranges` answers “unknown action” (400): the worker then reads range by range, the old way.

### When the server is too slow: octants

On a shared PHP host, a request that exceeds `max_execution_time` is **killed** by the host. The server cannot know in advance, so it protects itself.




▲ A unit that is too slow is split into 8 octants; two request deaths hand over to the browser.

Before each step, the server calls `set_time_limit()`; `status.server.limits.resettable` says whether the host honours it. Then:

1. **before** starting a unit, it writes `attempts[unit] + 1` in the journal;
2. for m004, a unit can be done in **8 octants** (2 × 2 × 2 bricks of the super-block), each stored and noted ( `partial[unit]` ) before the next: the unit is ticked when all are in. It does this automatically as long as no whole unit of the level has been timed, when the slowest (+ 25 %) would not fit in the remaining time, or after a request death;
3. a unit whose steps have killed the request **twice** gets the answer **409** `unit_timeout`. The journal stays `running`, **nothing is lost**, and the tab hands this migration to the browser, which has no such

ceiling.

### What the tab says when something goes wrong

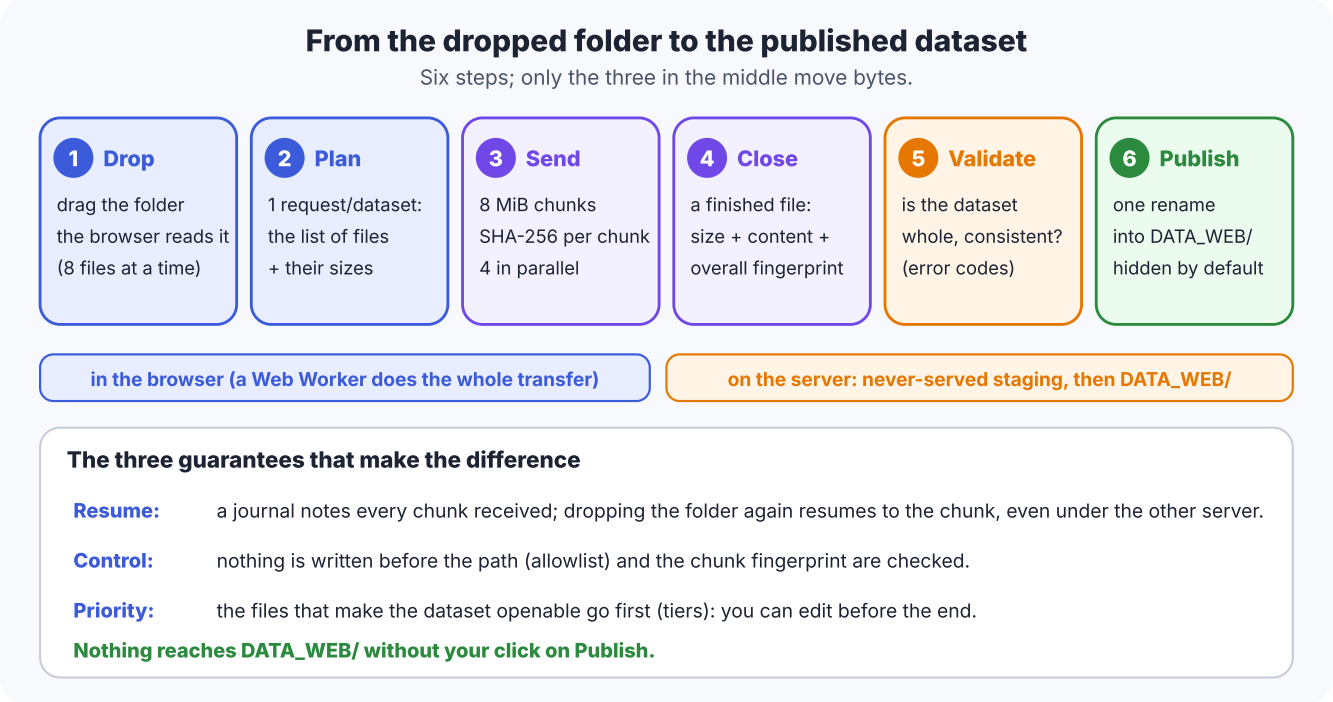
| CODE | MESSAGE ON SCREEN |
|---|---|
| `source_changed` | the dataset was re-processed since the update started |
| `brick_undecodable` | a brick could not be decoded |
| `incomplete` | some units are not done yet |
| `no_progress` | the server made no progress |
| `request_failed` | a request keeps failing |
| `prepare_failed` | the dataset could not be read |
| `unit_blocked` | a lower level of the pyramid is not complete yet |
| `bad_webp` | the server refused a converted brick |
| `insufficient_disk` | not enough disk space on the server (needed / free) |
| `no_executor` | neither this browser nor the server can run this update |
| `unit_timeout` | the server cannot convert one unit within its time limit |

📌 **REMEMBER**

- A migration = **units** ticked in a **journal**; two executors, a browser or a server, write into it.
- The **last move** is the version change, **under the lock** of `metadata.json`; until it has happened, nothing is lost.
- The **governor** protects the operator's address; **three requests per unit** have been enough since 1.59.3.

## 17.13 The course of an import

Chapter 14 (section 14.3) shows the **Import** tab. Here is what happens **underneath** each gesture.




▲ From the dropped folder to the published dataset: six steps, three of which carry bytes.

## 1. Drop: reading the folder

The browser reads the dragged folder with the `webkitGetAsEntry` interface, the only one that works for a **folder** in all current browsers. It returns only about a hundred entries per call: each folder is **drained** in a loop, otherwise a dataset would be cut off at its first hundred packs. Eight files are resolved at a time (opening them one by one would take minutes for 100,000 packs) and a limit of 400,000 entries protects memory; whatever could not be read is **counted and reported**.

A dataset is **recognised** by its `metadata.json`. Its **type** comes from the `type` field of that file; failing that, from the name of the parent folder if it is called `3d`, `2d` or `live`. The **name of the dropped folder is the dataset's identifier**: hence the message "Drop the dataset FOLDER, not its contents".

> ⚠️ **WARNING**
>
> The import requires a **secure connection** (HTTPS, or `localhost`): the SHA-256 fingerprints rely on `crypto.subtle`, which the browser does not offer over plain HTTP. The panel says so once, up front, rather than failing file by file.

## 2. Plan: one request per dataset

For each dataset, the browser sends **the list of its files with their sizes** (never the bytes). The server **re-derives** everything: the shape of the type and folder, each path against the allowlist, and the state of what is already stored or published. For each file it answers: accepted or refused, tier, file number (`fileId`), and — for a partially received file — the **missing blocks**.

A few safeguards of the planning step:

| LIMIT | VALUE |
|---|---|
| files per dataset | 200,000 |
| datasets per drop | 200 |

| LIMIT | VALUE |
|---|---|
| **size of one file** | 1 TiB |
| **space to leave free on the disk** | 512 MiB |

If the disk cannot hold what remains to be sent, the answer is `insufficient_disk` **before a single byte** (with `neededBytes` and `freeBytes`): a disk that fills up along the way would block every other writer on the host. A list too large for the host's request size (`post_max_size`) is reported too.

### 3 and 4. Send and close

The rest is detailed in sections 17.16 (blocks and journal) and 17.18 (verify, publish). The whole transfer happens in a **Web Worker**: reading an 8 MiB slice and hashing it costs tens of milliseconds of pure computation; on the main thread, that would be a lost frame for every block.

### The import API actions

| ACTION | ROLE |
|---|---|
| `limits` | block size, ceiling, parallelism, grace period, server type |
| `ping` | one authenticated round trip: tells "network cut" from "session expired" |
| `list`, `state` | the datasets being imported; the detail of one dataset; **the list also triggers the clean-up** |
| `plan` | plans a dataset (above) |
| `chunk` | receives a block |
| `file_done` | closes a file |
| `validate` | overall check of the dataset |
| `blob` | reads a file being imported (preview) — under the admin session |
| `metadata`, `save_metadata`, `save_thumbnail` | editing during the transfer |
| `publish`, `discard`, `gc` | publish, delete, clean up |

## 17.14 The allowlist

Only what the **pipeline produces** is accepted. Everything else is refused **before the first byte**, by two successive barriers.

▲ Two barriers: the shape of the path, then an explicit rule per dataset type.

> ❓ **WHY THIS CHOICE?**
>
> **Why a closed list rather than a list of prohibitions?** A blocklist always forgets something (a rare extension, a Windows device name). A list of what is **allowed** cannot forget: whatever was not anticipated is refused. The landing area is also **never served** (chapter 19), and publication happens only after validation.

The details of the rules, as the engine applies them:

- packs: `lod<N>/c<M>/pack_<n>.bin` (formats 1 to 3), `l<k>/c<c>/p<NNNNN>.bin` (format 4) or `l<k>/<column>/c<c>/p<NNNNN>.bin` (format 5), under `bricks/` or under `bricks/tNNN/` for a series (type `live` only);
- format 5: the indexes `index/<column>.bin` (tier 0), the cut copies `xy/`, `xz/`, `yz/` and `cuts/` (last tier, like `planes/`);
- `planes/` and `mips/`: `manifest.json` and the named packs (`zNNNNN.bin`, `lNNNNN.bin`), with a `tNNN/` sub-folder for a series;
- `download/`: a plain name (no sub-folder), up to 181 characters, a **data** extension (list in section 17.6), **judged on its last extension**; Windows reserved names (`CON`, `NUL`…) and names ending in a dot or a space are refused;
- never accepted: `.php`, `.js` and `.htaccess` files, any hidden file, any path going up (`..`), and the formats a browser **executes** when opened (HTML, SVG, XML).

ℹ️ **GOOD TO KNOW**

A file named `a.php.png` **passes** (its last extension is `png`): this is not a hole, because `download/` is never executed and always served as an attachment. What matters is that the file lands in a folder where **nothing executes**.

## 17.15 Tiers: why a dataset is editable before the end

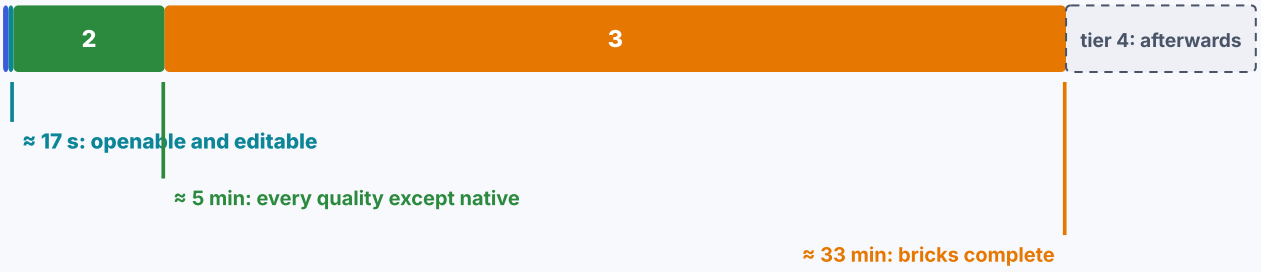
A complete dataset weighs tens of gigabytes, and the upload can last hours. Files are therefore sent **by priority**: the most useful first.

### Tiers: openable in seconds, complete in hours

Top: the real bytes of the demo dataset. Bottom: the same split applied to 10 GB of bricks, at 5 MB/s.

| tier | what it holds | files | bytes |
|---|---|---|---|
| 0 · core | metadata.json, manifest, index.bin, thumbnail | 4 | 47.0 kB |
| 1 · preview | the coarsest level, every channel | 3 | 31.9 kB |
| 2 · middle levels | the levels in between, coarse to fine | 6 | 1.3 MB |
| 3 · native level | level 0; frames 1, 2, 3… of a series | 6 | 7.9 MB |
| 4 · planes, mips, originals | planes/, mips/, download/ (not counting the originals: 530 MB) | 117 | 9.1 MB |

**Scenario: 10 GB of bricks (same proportions) at 5 MB/s**




From tier 1, the dataset opens at low resolution; once edited, metadata.json is locked: the rest of the transfer does not replace your settings.

▲ The five tiers, with the real bytes of the demonstration dataset and a scenario of 10 GB at 5 MB/s.

| TIER | CONTENT | WHY IN THIS ORDER |
|---|---|---|
| 0 · core | `metadata.json`, manifest, `index.bin`, thumbnail | enough to **mount** the dataset |
| 1 · preview | the **coarsest** level, all channels (and `tracks.json` for a series) | enough to display something |
| 2 · intermediate | the levels between the coarsest and the native, from coarsest to finest | quality rises progressively |
| 3 · native | level 0, and frames 1, 2, 3… of a series | full-quality work |
| 4 · the rest | `planes/`, `mips/`, `download/` | never needed to **open** |

A pack's tier depends on the **whole** dataset (which level is the coarsest?): the server therefore recomputes it once the full list is known. A series opens on its **first frame**; the following ones are tier 3.

> 🔢 **WORKED EXAMPLE**
>
> **The figure's scenario.** The demonstration dataset has 47 kB of tier 0 and 32 kB of tier 1: scaled to 10 GB of bricks (same proportions) at 5 MB/s, the dataset is **openable and editable in about 17 seconds**, every quality except native is there in 5 minutes, the complete bricks in 33 minutes. Tier 4 may take longer than everything else.

### When the dataset becomes "editable"

The dataset moves to the **editable** state when **all** the tier 0 and 1 files are finished and `metadata.json` and a mounting file (manifest or 2D preview) have arrived. You can then open it at low resolution and rename it, adjust the channels, orient it.

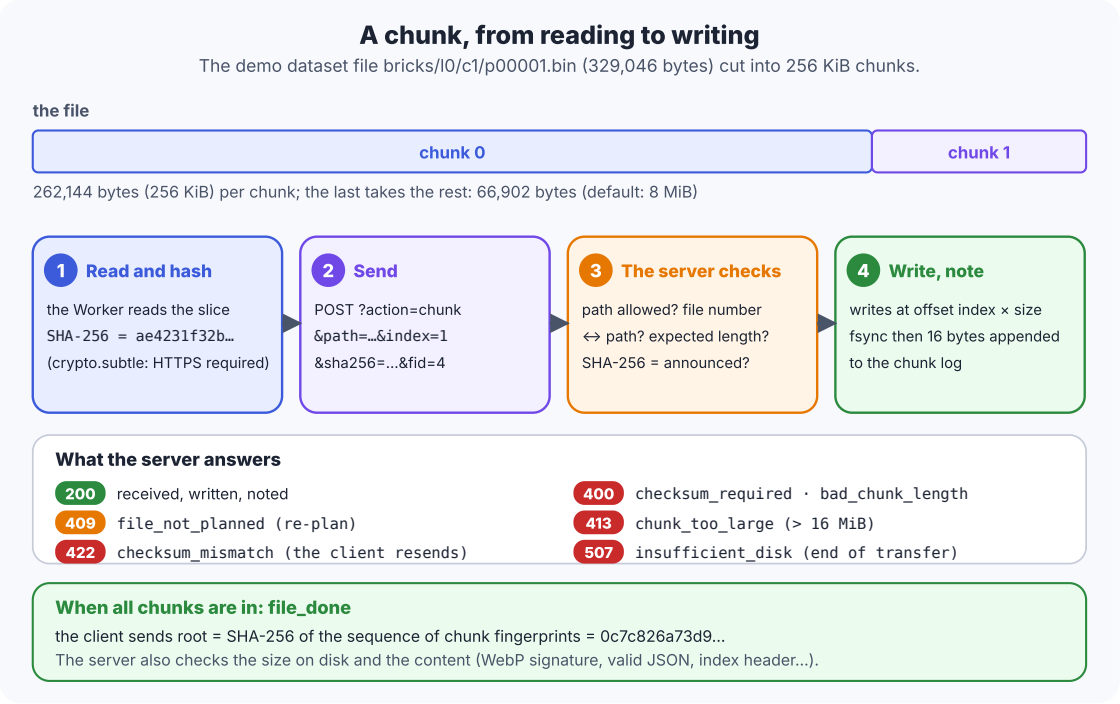
### The lock on `metadata.json`

As soon as you **save** an edit on a dataset being uploaded, the server writes your `metadata.json` and sets a lock ( `metaLocked` ). The transfer **no longer rewrites it**: receiving a new block of `metadata.json` , or a new planning pass, skips the file. Without this, the end of the transfer would silently have **erased your work** by sending the original file back.

## 17.16 Blocks and the import journal

### A block

Each file is cut into **8 MiB blocks** by default. The server may impose a smaller size (never larger): a PHP host limits the size of a request.




▲ A block, from reading to writing: hashing, check, writing at its exact offset, note in the journal.

| BLOCK LIMIT | VALUE |
|---|---|
| **default size** | 8 MiB |
| **ceiling (Python)** | 16 MiB |
| **floor** | 256 KiB |
| **PHP host** | 80 % of `post_max_size`, and at most a quarter of `memory_limit` |
| **blocks in parallel** | 4 |

If a host refuses the chosen size (`body_truncated`), the interface **reduces** the size, re-plans and says “Block size reduced to … — drop the folder again to resume”. Blocks already received remain valid: they are counted from the size **recorded** for each file.

🔢 **WORKED EXAMPLE**

**The example’s figure.** A 22 GiB original represents 2,816 blocks of 8 MiB; the list of what has been received fits in 2,816 bits, i.e. **352 bytes**. Blocks may arrive **out of order and in parallel**: each is written at its exact offset (`index × block size`).

## Three levels of checking

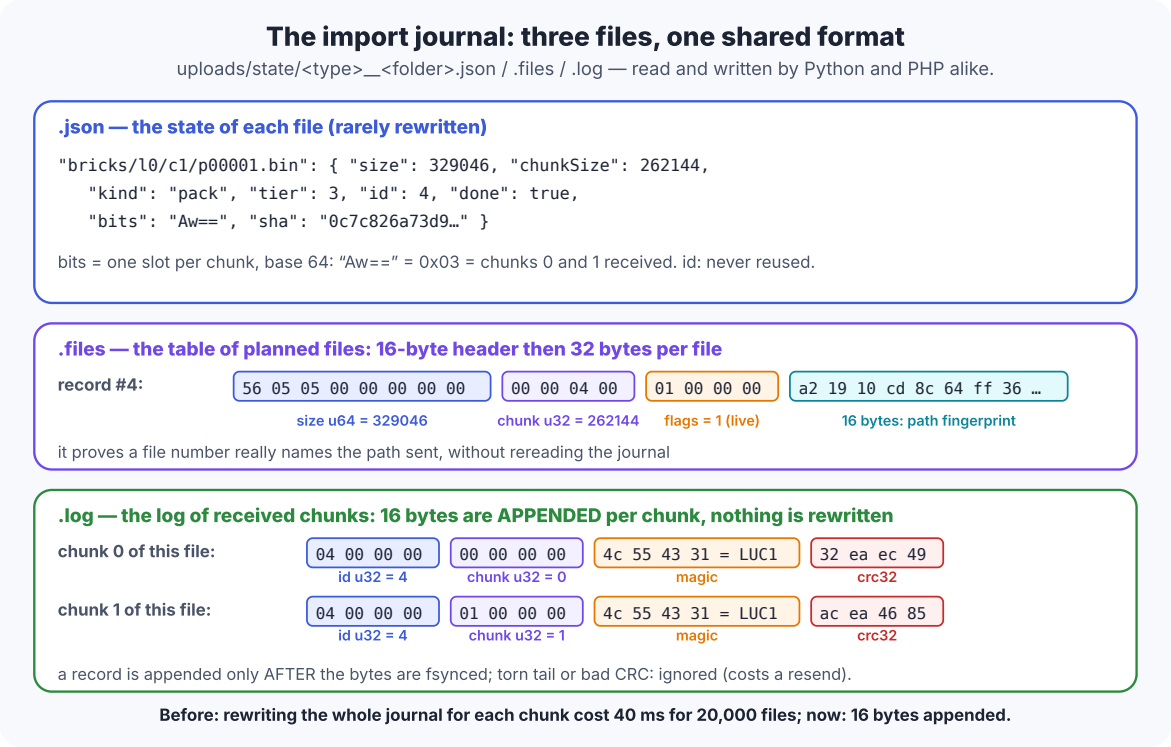
| WHEN | WHAT |
|---|---|
| **at every block** | SHA-256 recomputed **before** writing; expected length; does the file number match the path? |
| **when a file is closed** | all blocks present, exact size on disk, **global fingerprint** = SHA-256 of the sequence of block fingerprints, plausible content (WebP signature, valid JSON, index header…) |
| **at dataset validation** | overall consistency (section 17.18) |

❓ **WHY THIS CHOICE?**

**Why a “fingerprint of fingerprints”?** A browser cannot hash a 22 GiB file as a stream, and re-reading it a second time would double the work. The fingerprint of the **sequence of block fingerprints** proves the same thing — every block verified, in the right order, none missing — **without** re-reading anything.

When the upload fails, two families of failures are handled differently: a **server** failure (5xx error, saturation, wrong fingerprint) uses up one of the **6 attempts**, with a growing wait of 1 to 30 seconds (± 25 %); a **network** failure (no answer) uses up **nothing**: the whole transfer goes on hold until the connection returns. An outage of several minutes costs time, never files.

## The journal: three files, one shared format




▲ The import journal: the state of each file, the table of planned files, the log of received blocks.

They live in `uploads/state/` and are named `<type>__<folder>`. Python and PHP both read **and** write them: **an import started under one server resumes under the other**.

| FILE | ROLE | REWRITTEN |
|---|---|---|
| `.json` | the state of each file: size, block size, tier, kind, number, one cell per block (base 64), `done` | at planning and at every finished file |
| `.files` | the table of planned files: 16 header bytes ( `LUFT` ) then 32 bytes per file | when a number changes |
| `.log` | one **16-byte record per received block**: file number, block index, `LUC1` , CRC32 | **never**: append only |

🔧 FOR THE CURIOUS

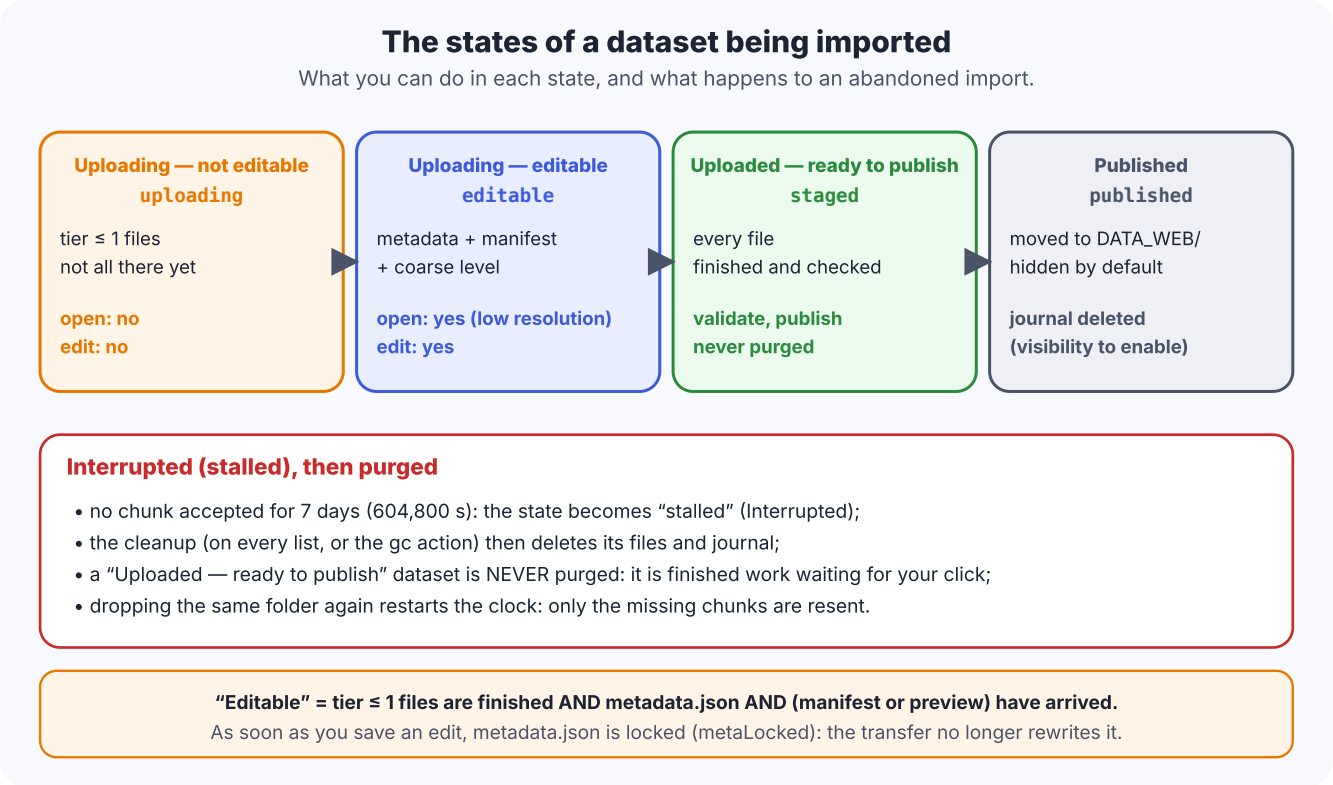
**Why an "append-only" journal?** Rewriting the whole JSON file at every 8 MiB block cost 40 ms for 20,000 files and half a second for 200,000, **under the dataset's lock**: the four parallel streams queued behind each other. Appending 16 bytes costs almost nothing. The journal is **compacted** (folded into the `.json` , then the `.log` is emptied) at planning, at the end of a file and at every journal write.

What keeps the journal safe even after a crash:

- a record is appended only **after** the `fsync` of the bytes: a record that survives always describes bytes really on disk;

- a truncated tail, or a record whose signature or CRC does not match, is **ignored**: at worst this costs one resend;
- the file number is **never reused**: when a file is re-planned (size changed) or reset, it receives a new number, and a record written for the old one can no longer land on the new one;
- the file's path is checked by a **16-byte fingerprint** stored in the table: this proves that a number designates the path that was sent without re-reading the journal.

## 17.17 Import states and the purge




▲ Four states; an abandoned dataset is purged after seven days, a finished one never.

| STATE (CODE) | LABEL ON SCREEN | CONDITION |
|---|---|---|
| `uploading` | Uploading — not editable | the tier 0 and 1 files are not all there |
| `editable` | Uploading — editable | tiers 0 and 1 finished, `metadata.json` and a mounting file arrived |
| `staged` | Uploaded — ready to publish | **all** planned files are finished |
| `stalled` | Interrupted | no block accepted for 7 days |

### The seven-day purge

A dataset that nobody finishes occupies real disk in the landing area. The operator has **seven days** (604,800 seconds) to drop the folder again and resume; after that, the clean-up frees its files and its journal, and the upload starts again from scratch.

- the clean-up runs **every time the list is displayed**, or on demand (action `gc`);

- the countdown is measured from the **last activity** (a file uploading for hours counts as activity);
- a dataset in the `Uploaded — ready to publish` state is **never** purged: it is complete and validated, and it is only waiting for your click. Deleting it after a week would destroy tens of gigabytes of finished work to spare a resend nobody asked for.

### What stays readable during the upload?

The preview of an unpublished dataset goes through the `blob` action: a file from the landing area is served, **under the administration session**, as opaque bytes (with `nosniff`), never as a document. The landing area is reachable by **no address**; it is blocked in the Python server, in the root `.htaccess`, in `router.php` and by a deny file of its own.

## 17.18 Validate, publish, replace

### Validate

The validation action re-reads the **entire** dataset and checks that it forms a whole. It returns a list of codes; there are a few dozen, and here are their families.

| CODE | WHAT IT MEANS |
|---|---|
| `missing_metadata`, `metadata_not_json` | no `metadata.json`, or unreadable |
| `metadata_type_mismatch` | the declared type is not that of the drop folder |
| `metadata_no_dimensions`, `metadata_bad_dimensions`, `metadata_no_channels` | invalid dimensions or channels |
| `metadata_no_image`, `metadata_bad_image` | (2D) `image` block absent or file names not conforming |
| `missing_manifest`, `manifest_not_json` | no `bricks/manifest.json`, or unreadable |
| `manifest_no_levels`, `manifest_level_N_…`, `manifest_bad_brick_size`, `manifest_bad_encoding` … | inconsistent manifest (levels, 64 voxels, encoding) |
| `missing_index:…`, `index_hash_mismatch:…` | `index.bin` absent, or different from the manifest's fingerprint |
| `index_bad_magic`, `index_bad_length`, `index_shape_mismatch:…`, `index_grid_mismatch:…` | the index has the wrong shape or does not describe the manifest's levels |
| `missing_pack:<path>` | a pack that the index cites has not arrived |
| `manifest_v5_…`, `missing_store:<path>` | (format 5) inconsistent v5 manifest, or a cut copy (`xy/`, `xz/`, `yz/`) is missing |
| `truncated_pack:<path>` | a pack is shorter than what the index requires |
| `manifest_pack_escapes` | a pack points outside the dataset's folder |
| `incomplete_files` | planned files are not finished |
| `stray_files` | a file present in the landing area was never accepted by a plan |

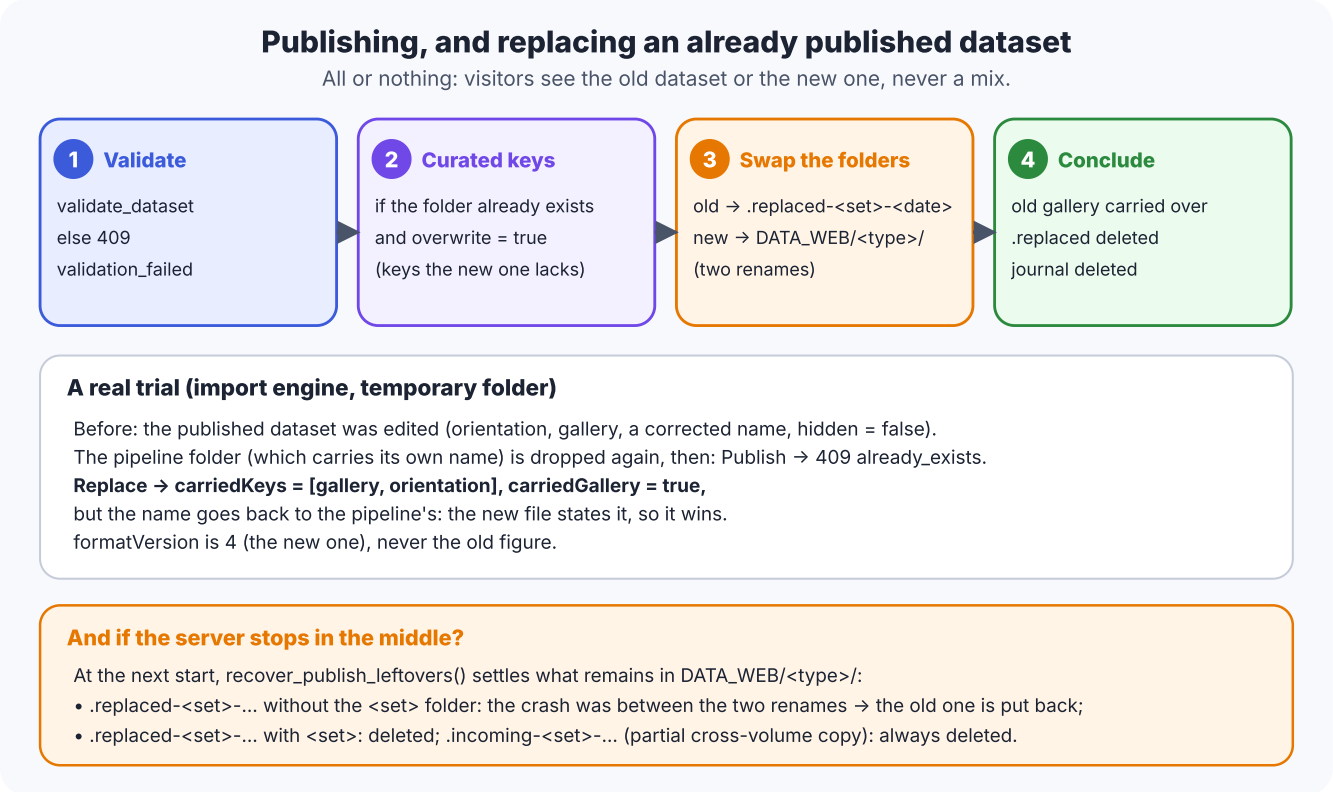
| CODE | WHAT IT MEANS |
| --- | --- |
| `missing_preview`, `missing_image` | (2D) one of the two copies is missing or empty |

🔢 **WORKED EXAMPLE**

**Real trials** on the demonstration dataset, in a temporary folder. Everything arrived: `{ok: true}`. The pack `l0/c1/p00001.bin` is **truncated** by 1,000 bytes: `truncated_pack:l0/c1/p00001.bin`. It is **deleted**: `missing_pack:l0/c1/p00001.bin`. `index.bin` is **altered**: `index_hash_mismatch:bricks`. A file `stray.txt` is dropped in: `stray_files`. A block with a wrong fingerprint gets `422 checksum_mismatch`; a block with no fingerprint, `400 checksum_required`.

A truncated pack passes its own block fingerprints if the client simply never sent the end: only the overall validation catches it, **before** a visitor sees a black screen.

## Publish




▲ Validate, carry over the operator's work, swap the folders, conclude.

Publication moves the validated dataset from the landing area to `DATA_WEB/<type>/<folder>/`.

- it is **hidden by default**: the operator decides when the public sees the dataset;
- it is a **rename** (almost atomic) when the landing area and `DATA_WEB/` are on the same disk; otherwise, a copy into a neighbouring `.incoming-…` folder followed by a rename, so that a half-copied dataset is never visible under its real name;
- before moving, temporary write files are swept away: they never travel to the site.

### Replacing an already published dataset

If the destination folder exists, publication answers `409 already_exists`, and the interface asks for confirmation of the replacement. With `overwrite`, in order:

1. the curated keys of the old file that the new one **does not contain** are carried over (section 17.2);
2. the old folder is renamed `.replaced-<dataset>-<date>`;
3. the new one is renamed into its place;
4. the **gallery** of the old folder (images and thumbnails) is moved into the new one, then the `gallery` list keeps only the files actually present;
5. the old folder is deleted, together with the import journal.

If the server stops **between the two renames**, at the next start-up `recover_publish_leftovers()` puts the old dataset back in place; a `.replaced-…` whose dataset exists is deleted, and an `.incoming-…` (partial copy) always is.

> ℹ️ **GOOD TO KNOW**
>
> The interface does **not resend** a folder whose name already exists in `DATA_WEB/`: it treats it as "already published" rather than resending gigabytes. Replacement is offered at the moment of **Publish**, when the server reports that the name exists.

### What to take away

> 📌 **REMEMBER**
>
> - A **closed allowlist**: whatever is not anticipated is refused before the first byte; the landing area is **never served**.
> - **Tiers**: the files that make the dataset openable go first, and an edited dataset keeps its `metadata.json`.
> - **Hashed blocks** and an **append-only journal**: a cut costs at worst one block, and the import resumes under either server.
> - An **"all or nothing" publication**, hidden by default, which carries over the operator's work and the gallery.

> 📖 **GOING FURTHER**
>
> The screens, buttons and floating dock of the import are in chapter 14; the security of the landing area is in chapter 19; the exact contract of the migrations is in `DOCS/dataset-migrations/SPEC.md`.

# 18. Hosting, updating and releasing a version

- Lumen3D can be served by **three kinds of server** (Python, PHP, or plain files). This chapter says what each can do, how it answers the browser and why the pages stay fast.
- An **update** is a **signed** package: the platform refuses anything it cannot prove authentic, then switches over while keeping a way back (on Python).
- Every release is **fully tested** (201 test files), signed, then published in one go. The platform works **without Internet**.

This chapter is for the person who **installs and maintains** the site: the administrator, or the IT person who helps them. Chapters 9 to 12 explain what the visitor sees; this one explains **the machine behind it**.

Nothing here requires writing code. But understanding these mechanisms lets you **diagnose** a slow site, a refused update or a temperamental host.

ℹ️ **GOOD TO KNOW**

The version numbers quoted are those of this document: web platform **1.65.0**, pipeline **0.22.0**, data format **5**. The administration panel in brief is in chapter 14; security in general is in chapter 19.

## 18.1 Three ways to serve Lumen3D

The browser always speaks the same language (HTTP). What changes is **the program that answers** on the other side. There are three families.

**Who can do what: the ways to serve Lumen3D**

| | dev_server.py<br>Python | fast_server.py<br>Python, read-only | api/*.php<br>PHP + Apache | Static host<br>files only |
|---|---|---|---|---|
| Serves the files | yes | yes | yes | yes |
| Nonce + CSP on HTML pages | yes | yes | yes | no |
| Dataset catalogue (on demand) | yes | yes | yes | no |
| Administration API | yes | no | yes | no |
| Browser import | yes | no | yes | no |
| Platform update | with rollback | no | no rollback | no |
| /api/health probe | yes | yes | no | no |

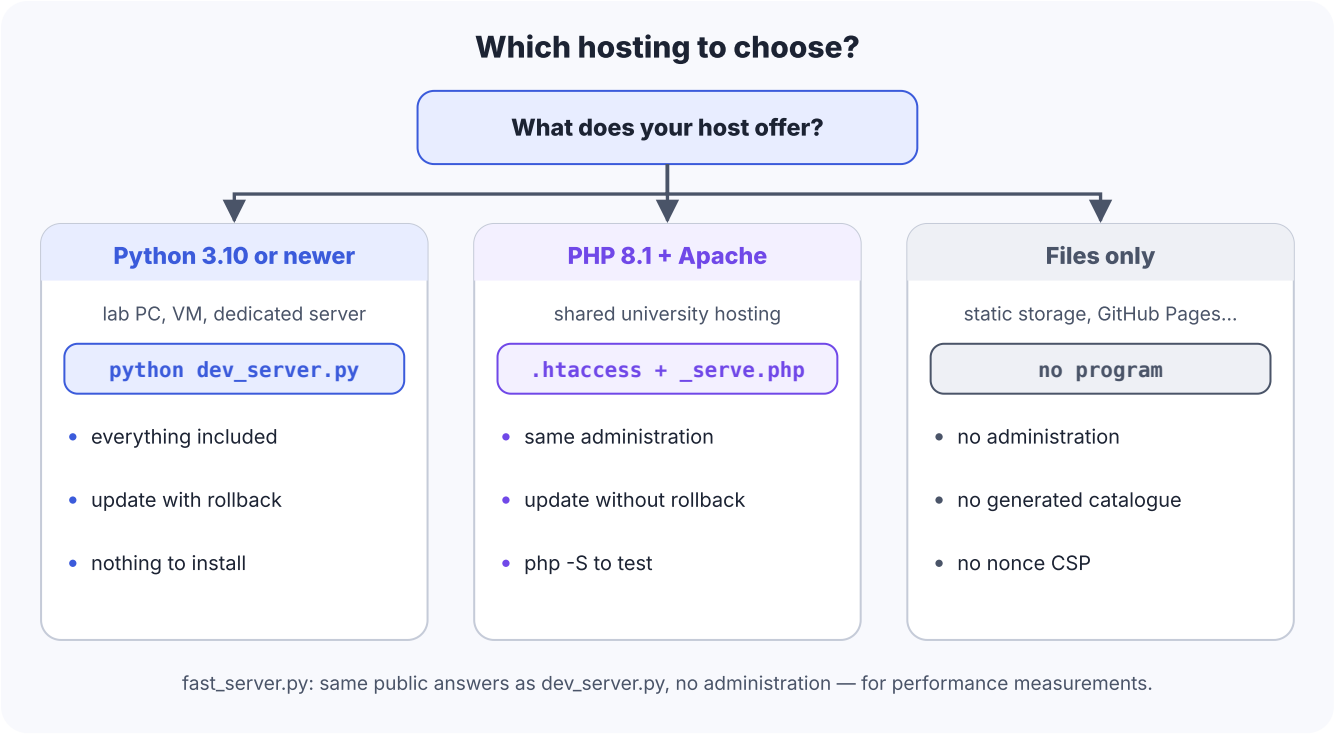
▲ What each way of serving can do: from the most complete to the simplest.

💡 **A PICTURE TO UNDERSTAND**

**A library desk.** The full desk (Python or PHP) checks your card, finds the book and keeps the loan register. A simple self-service shelf (static host) lets you take a book, but keeps no register and checks nobody.

- `dev_server.py`: the Python server. It does **everything** (pages, catalogue, administration, import, update). It is the recommended server.
- `fast_server.py`: the same server, **read-only**. It answers a visitor exactly like `dev_server.py`, but **every administration route is refused** (404). It is used to measure performance.
- **PHP** (folder `api/`, `router.php`, `_serve.php`, `.htaccess`): for university hosts that offer PHP but not Python. Same functions, same files, same password.
- **"Files only" host**: fine for viewing, but with no administration, no generated catalogue and none of the nonce protection of chapter 19.

### Which one to choose?




▲ The decision tree for a hosting choice.

🔢 **WORKED EXAMPLE**

**A laboratory with a dedicated computer.** It runs `python dev_server.py --dev-trust-local`: a single command, nothing to install, updates with rollback. **A university that offers only PHP**: it drops in `install.php` (§18.14); Apache reads the supplied `.htaccess` file, which routes the pages to `_serve.php`.

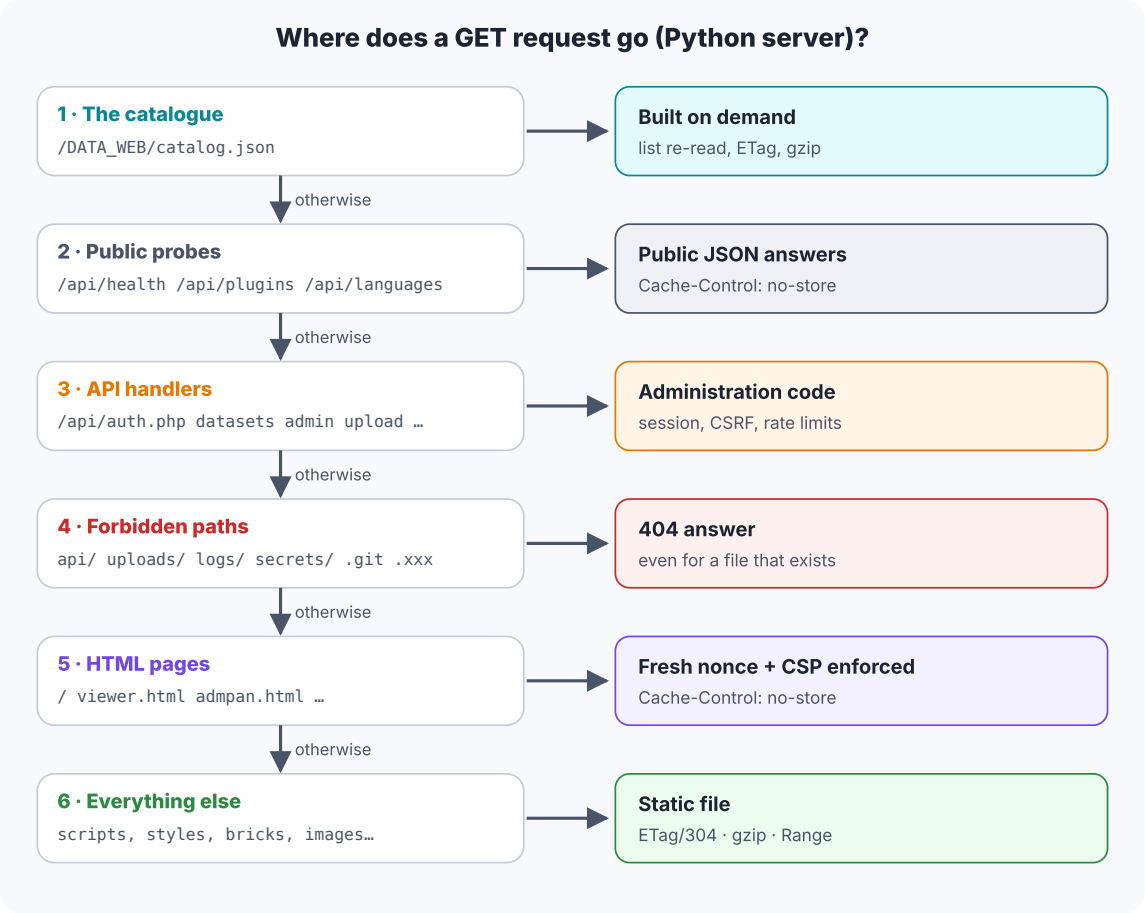
🔧 **FOR THE CURIOUS**

`dev_server.py` **options**: `--host` (default `localhost`), `--port` (default `8080`), `--set-password` (change the administrator password from the command line), `--check` (validate an installation without opening a port, §18.10), `--root` (the installation to check), `--dev-trust-local` (trust local plugins; **reserved for the development machine**), `--trusted-proxy` (§18.6), `--migrate-types`, `--verbose` (log every request). By default no request is written to the console: a synchronous write per request would slow the answers.

**Minimum versions**: Python 3.10 (the README says so); PHP 8.1, the version tested by continuous integration, with the `zip`, `mbstring`, `sodium` and `gd` extensions. The `install.php` installer itself needs only PHP 7.4 to start.

## 18.2 The path of a request

A visitor asks for an address. The Python server sorts the request into **one of the six categories** below, in this order. As soon as one fits, it stops.




▲ The six destinations of a GET request on the Python server.

- The **catalogue** and the **probes** are fabricated answers, never files.
- **Forbidden** paths get a 404, **even if the file exists**: the server does not confirm it is there.
- `.html` pages go through a separate treatment: a fresh secret code (the “nonce”) is inserted into every page at every load (chapter 19).
- Everything else is a **static file**: bricks, scripts, images.

## The catalogue is not a file

The address `DATA_WEB/catalog.json` **does not exist on disk**. The server fabricates it on demand by reading the `metadata.json` cards of each folder of `DATA_WEB/3d`, `2d` and `live`.

- Datasets that are **configured or have a thumbnail**, and are **not hidden**, are listed.
- Sorted by descending date; a dataset with no date goes **last**.
- The result is cached, with a fingerprint (ETag); the list is recomputed only if a card has changed (the disk is consulted at most once every 2 seconds).

🔢 WORKED EXAMPLE

**Adding a dataset by hand.** You copy a finished folder into `DATA_WEB/3d/` : on the next visit it appears in the Explorer. Nothing to regenerate. On a PHP host, `api/catalog.php` does the same job, called by a rewrite rule in `.htaccess` .

🔧 FOR THE CURIOUS

**On Apache, rewriting is essential**: without the `mod_rewrite` module, the catalogue address answers 404 and HTML pages come out without nonce protection. The supplied `.htaccess` file wraps each directive in `<IfModule …>`, because a directive from a module that is not loaded would bring **the whole site** down with a 500 error. It never uses `Options` (which requires a right many shared hosts do not grant): folder listings are refused by a rewrite rule.

### The health probe

The address `/api/health` answers one minimal line of JSON: `{ok, web, server, trustEpoch}` (and `lastUpdate` just after an update). It is **public** and says only what is already public (the version on GitHub).

It serves two purposes: the update supervisor (§18.11) queries it to know whether the new version is alive, and you can hand it to a monitoring tool. **PHP hosts do not have this probe.**

## 18.3 HTTP caching: never re-download for nothing

A dataset weighs gigabytes; the platform, about thirty scripts. Without a cache, every visit would start from zero. The server therefore tells the browser **how long to keep** each file.

💡 A PICTURE TO UNDERSTAND

**The fridge and the label.** A yoghurt carries a date: until that date, no need to ask the shop again. A morning newspaper, though, must be **rechecked**: “is this still today’s?”. The HTTP cache is that system of labels.

**The HTTP cache decision table**

| Which file? | URL with ?v=… | URL without ?v= |
|---|---|---|
| **HTML page** (fresh nonce every time) | no-store | no-store |
| **Brick pack** `.bin .rgba .gz` | immutable, 1 year | no-cache + ETag |
| **Script, style** `.js .mjs .css` | immutable, 7 days | no-cache + ETag |
| **JSON, manifest, catalogue** ?v= changes nothing | no-cache + ETag | no-cache + ETag |
| **API answers** `/api/…` | no-store | no-store |

no-cache = the browser keeps the file but asks again: the server answers “304, unchanged” without resending the bytes.

Images, fonts: no-cache + ETag on the Python server; 7 days on Apache (.htaccess).

▲ The decision table: one rule per file type, depending on whether a ?v= is present.

## Three rules to remember

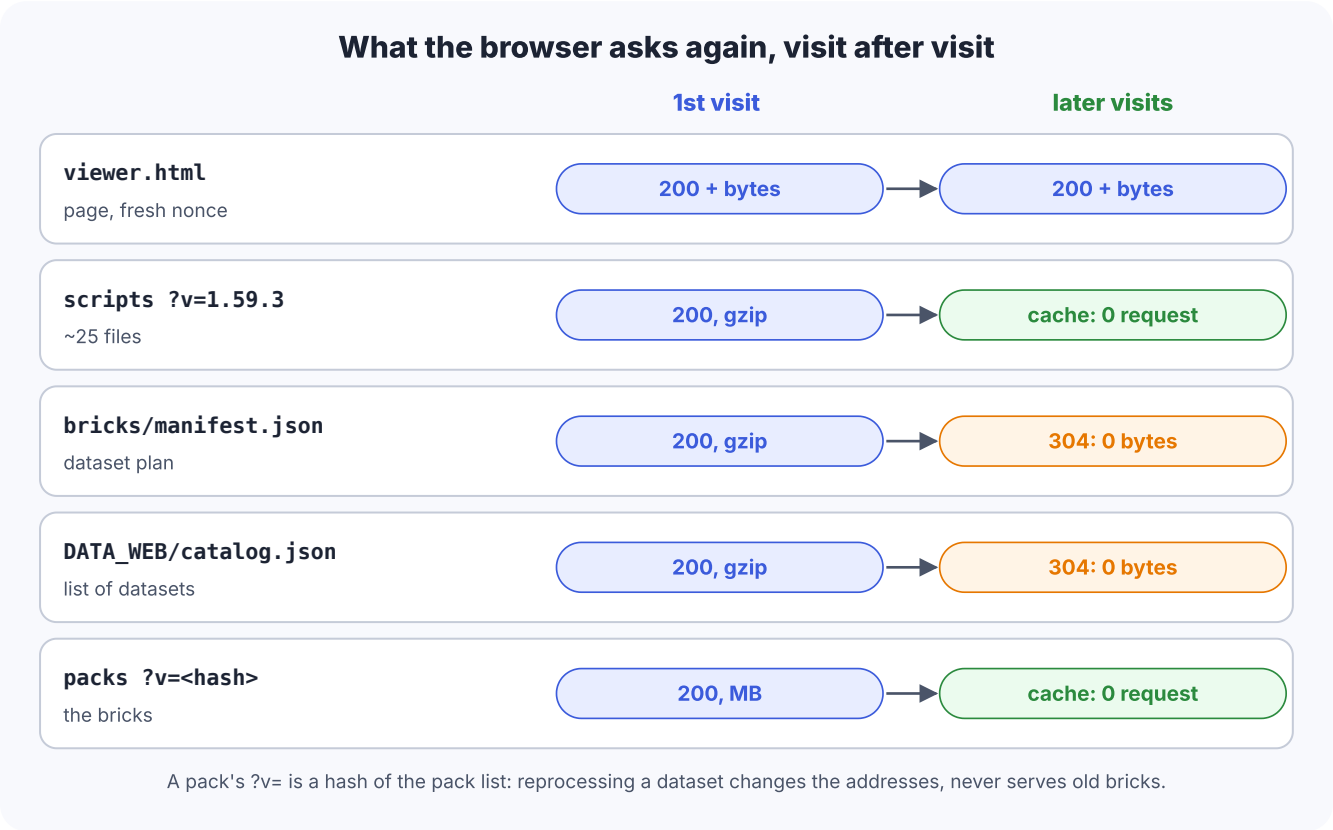
1. **A “versioned” address never changes content.** If it carries `?v=…`, the browser keeps it without asking again: **1 year** for a brick pack, **7 days** for a script or a style.
2. **Everything else is “no-cache + ETag”.** The browser keeps the file but **asks again** with the fingerprint it holds; the server answers “304, unchanged” **without resending the bytes**.
3. **An HTML page is never kept** ( `no-store` ): its secret code changes at every load.

## Where does `?v=` come from?

- **Brick packs**: the loader (chapter 10) adds `?v=` followed by a fingerprint of the manifest’s **list of packs**. Reprocessing a dataset under the same name changes the packs, hence **the addresses**: old voxels can never be re-read by mistake.
- **Scripts and styles**: when a release is built, the build tool adds `?v=<version number>` to every local link, and gathers a page’s scripts into **a single file** ( `js/bundle/<page>.js?v=<fingerprint>` ).
- **The theme** ( `config/theme.css` ): its `?v=` is the file’s last modification date, so that a colour change appears **immediately** despite the cache.

> ❓ **WHY THIS CHOICE?**
>
> **Why such care for a script?** Some hosts limit the number of requests per address: thirty scripts re-requested on every page can trigger a 429 error (“too many requests”) and stop the viewer from starting. Versioned addresses avoid those re-requests.

## What a browser sees, from one visit to the next




▲ First visit, then later visits: what is asked again.

Here are answers **actually measured** on the demonstration Python server (synthetic dataset):

| REQUEST | CACHE-CONTROL | REMARK |
|---|---|---|
| `viewer.html` | `no-store` | page + fresh nonce |
| `js/core/utils.js` | `no-cache` | ETag `…-gz` (compressed version) |
| `js/core/utils.js?v=1.59.3` | `public, max-age=604800, immutable` | 7 days |
| `…/bricks/manifest.json` | `no-cache` | gzip: 1,151 bytes |
| `…/p00000.bin` | `no-cache` | 1,383,748 bytes |
| `…/p00000.bin?v=abc` | `public, max-age=31536000, immutable` | 1 year |
| `/api/health` | `no-store, no-cache, must-revalidate` | 66 bytes |

> ⚠️ **WARNING**
>
> **Without `?v=`, a pack is revalidated, not kept for a year.** A long lifetime on an address that nothing changes would have served the **old voxels** after a reprocessing: a silent scientific error. That is why the "1 year" rule demands proof ( `?v=` ) that the address is new.

> 🔧 **FOR THE CURIOUS**
>
> **Both backends apply the same table**, in `dev_server.py` ( `_static_cache_policy` ) and in the root `.htaccess` (environment variables set by `mod_rewrite` when the request contains `v=` ). A test compares the two ( `tests/test_backend2_twins_parity.php` ).
>
> **One difference to know for images and fonts**: the Python server answers them `no-cache` + ETag; Apache, through the `.htaccess` , gives them 7 days. The Python ETag is made of the modification date and the size ( `"18dc4a43bc302352-4a17-gz"` : the `-gz` suffix marks the compressed version).

## 18.4 Compression and byte ranges

### gzip: lighter text files

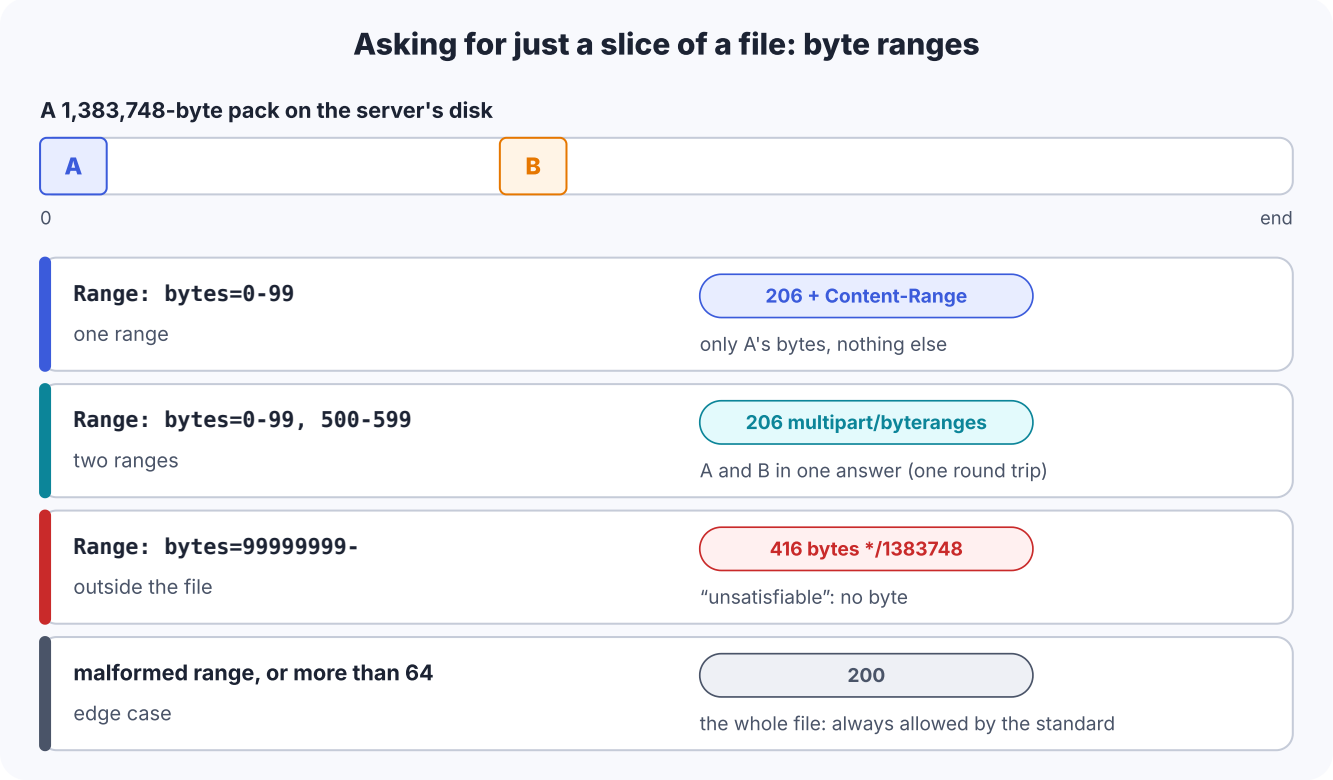
A brick manifest can weigh tens of MB of JSON: compressed, it weighs seven times less. The server therefore compresses **text** files on the fly: `.json` , `.js` , `.mjs` , `.css` , `.svg` , `.txt` , `.csv` , `.md` .

- Threshold: at least **1 KiB** (and at most 256 MiB); level 6.
- The result is **cached** (96 MiB budget) and redone only if the file changes.
- No compression when the browser asks for a byte **range**, nor for `download/` files (which always go out as attachments).
- **Brick packs are never compressed**: they are already WebP images.

On Apache, compression is handled by the `mod_deflate` module (JSON, HTML, CSS, JS, SVG), still not for packs.

## Byte ranges: asking for just a piece

To display a **cut** of the volume, the viewer does not need a whole 16 MiB pack: it asks for just a few stretches (chapter 10). The browser then writes a `Range: bytes=…` header.




▲ One range, several ranges, an absurd range: the server’s answers.

💡 **A PICTURE TO UNDERSTAND**

**Photocopying a few pages of a book.** Rather than carrying off the whole volume, you ask for “pages 1 to 3 and 200 to 210”. The server answers with exactly those pages, each labelled with its position in the book.

The Python server handles the **three cases** of the HTTP standard (RFC 9110):

- **One range**: a `206` answer with `Content-Range: bytes 0-99/1383748`. A “last N bytes” range (`bytes=-N`) is understood.
- **Several ranges** (`bytes=0-99, 500-599`): a single `206` answer of type `multipart/byteranges`, **one round trip** instead of two. Ranges that touch are merged; beyond **64 distinct ranges** (or if the header is malformed), the server returns the **whole file** (`200`), which the standard allows.
- **A range outside the file**: `416` with `Content-Range: bytes */<size>`.

🔢 **WORKED EXAMPLE**

**Checked on the demonstration dataset.** On a pack of 1,383,748 bytes: `bytes=0-99` answers `206` with 100 bytes; `bytes=0-99, 500-599` answers `206 multipart/byteranges` with 473 bytes in total (two times 100 bytes, plus the part headers); `bytes=99999999-` answers `416`.

🔧 FOR THE CURIOUS

`If-Range` is honoured (a range is served only if the ETag or the date still match). Range requests do **not** count as a download in the statistics. On the Apache side, the HTTP server handles ranges itself, with its own limits ( `MaxRanges` ). The brick loader, for its part, has a per-host **fallback** for when multiple ranges are not understood (chapter 10).

## 18.5 Closed folders and guard files

A website serves files; but not all of them should go out. Passwords, signing seeds and unvalidated imports **must never** be downloadable.

**The site's folders: which are served, which never**

| | | |
|---|---|---|
| `api/` | *.json (password, stats...) and _*.php | closed |
| `uploads/` | unvalidated imports: 4 locks | closed |
| `secrets/ logs/ backups/` | signing seeds, logs, copies | closed |
| `.git .htaccess .xxx` | any name starting with a dot | closed |
| `*.lumen-old\|new\|backup` | files parked by an update | closed |
| `DATA_WEB/` | execution banned; download/ as attachment | no execution |
| `js/` | execution banned (plugins land there) | no execution |
| `config/` | public; protected from updates | served |

▲ Which folders are served, which are always refused.

💡 A PICTURE TO UNDERSTAND

**The doors of a public building.** The lobby is open to everyone; the offices are locked; the archive room has **several different locks**, so that forgetting one does not let anyone in.

### Four locks for `uploads/`

The imported bytes (chapter 14) are not yet validated. Four **independent** barriers protect them: the Python server's list of forbidden roots, the root `.htaccess`, the `router.php` router, and a "deny all" `.htaccess` written inside the folder itself. The preview of a dataset being imported goes through an API address that requires the administrator session.

### The “lumen-guard v2” guard files

Two served folders ( `DATA_WEB/` and `js/` ) receive an `.htaccess` that **forbids script execution**: even if someone drops a malicious `.php` there, it will not run. `uploads/` and the media library have their own guard.

Each guard carries the mark `lumen-guard v2`. The server **rewrites** a guard only if that mark is missing: the two servers (Python and PHP) therefore never fight over the same file for a stray space.

❓ **WHY THIS CHOICE?**

**Why does `DATA_WEB/` need a guard?** Because this folder is at once **served** to the public and **open for writing** by the operator (by FTP or by the import). A booby-trapped file there would be the ideal way in. The guard is rewritten at run time: an update never delivers it ( `DATA_WEB` escapes updates, §18.13).

🔧 **FOR THE CURIOUS**

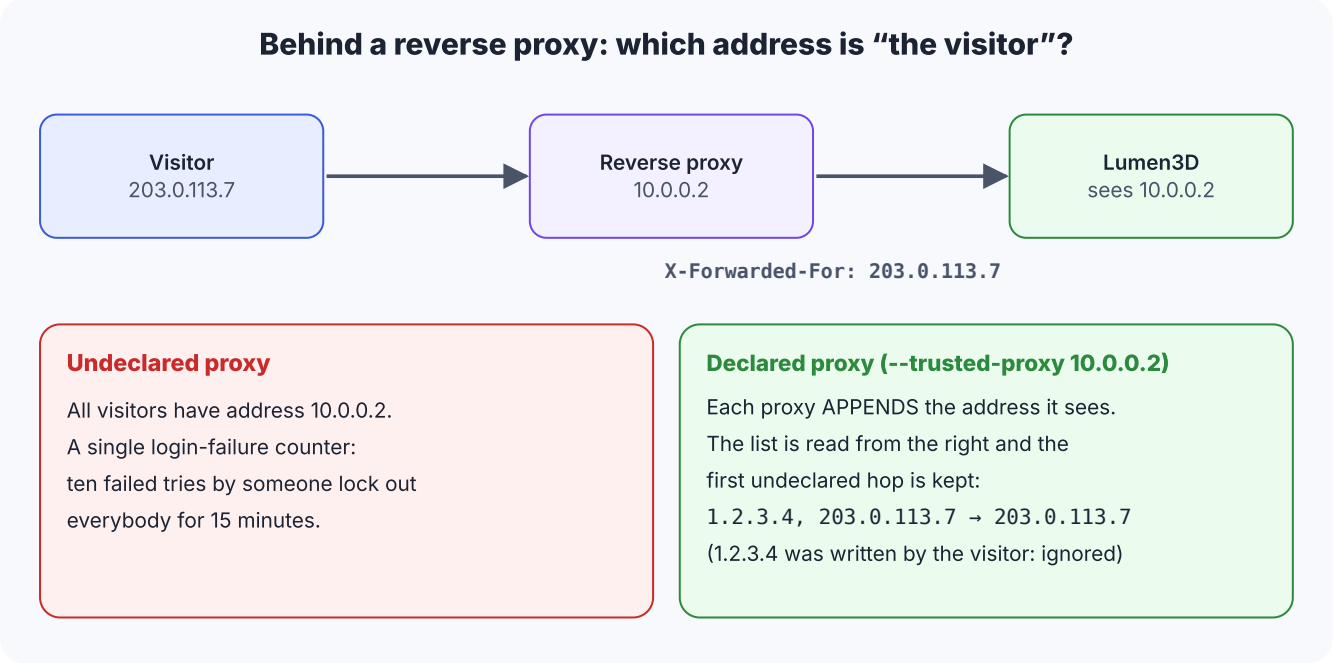
**PHP router details.** `router.php` (for `php -S` ) refuses every `api/*.json` and `api/_*.php` , the folders `secrets/ logs/ backups/ uploads/ .git` , any name starting with a dot (except `.well-known` ) and `*.lumen-old|new|backup` . It computes the path **without** `parse_url()` : an address starting with two slashes ( `//api/admin_credential.json` ) makes `parse_url()` read “api” as a host name, and the password file used to come out with a `200` . The path is therefore split by hand, decoded, normalised, then compared with the rules. Files parked by an update ( `*.lumen-old` ) would be served as text and would publish the old code: hence their prohibition.

**File permissions.** Files created by the server inherit the **rights of the root folder**: `0770/0660` on a shared host where the web server and your FTP account share a group, `0755/0644` elsewhere. If the server runs under a different user from the owner, group write rights are added so that the owner keeps control (FTP, deletion). Override: `LUMEN_DIR_MODE` and `LUMEN_FILE_MODE` (octal values). The secrets in `api/` keep restrictive rights.

## 18.6 Behind a proxy, rate limits and the session lock

### The reverse proxy and the visitor’s address

Many institutes put a **reverse proxy** (Nginx, Apache…) in front of the server: it handles HTTPS and forwards requests. The problem: Lumen3D now sees only **the proxy’s address**.




▲ Without a declaration, all visitors share the proxy’s address and a single login counter.

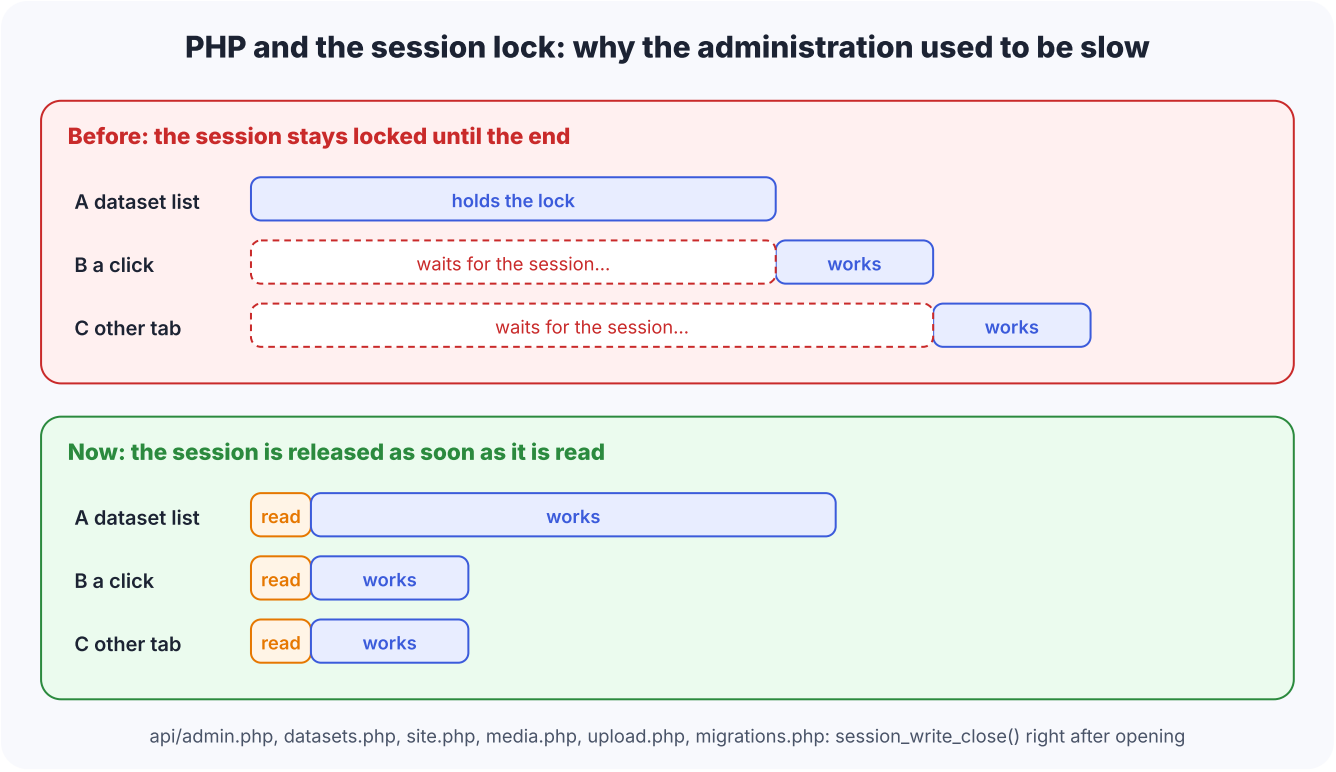
- **Declare the proxy**: `--trusted-proxy 10.0.0.2` (repeatable option, addresses or networks), the variable `LUMEN_TRUSTED_PROXIES="ip,network"`, or the file `api/trusted-proxies.json` (`{"proxies":[…]}`), which **both servers** read.
- **Why read the list from the right?** Each proxy **appends** the address it sees. Everything on the left may have been written by the visitor: it is not trusted.
- **Secure cookie**: the session gets the `Secure` attribute if a declared proxy announces HTTPS (`X-Forwarded-Proto`), or if `LUMEN_COOKIE_SECURE=1` forces it.

## Rate limits

| WHERE | LIMIT | WHAT FOR |
|---|---|---|
| **Admin login** | 10 failures / 15 min / address, global ceiling of 200 | counter mass guessing |
| **Public counters (`telemetry`)** | token bucket: 60 in reserve then 1/s per address; 600 then 20/s for everyone; GET refused | prevent inflating the statistics |
| **Import, data updates** | the browser’s governor (chapter 17) | avoid getting the operator’s address banned |
| **GitHub (admin)** | answer cached for 5 minutes | 60 requests/hour per address without an account |

## The PHP session lock

On PHP, **a session is a file that is locked** as long as a request is using it. Two requests from the same administrator **queue up**. Now the *Updates* tab fires three checks towards GitHub: the *Datasets* tab used to wait behind them.




▲ Before: requests queue up. Now: the session is read then released.

📌 **REMEMBER**

**Rule of the code**: every entry point that only **reads** the session releases it at once ( `session_write_close()` ). Only the login keeps it, because it writes it. A new PHP entry point must follow the same rule.

## 18.7 Running a server day to day

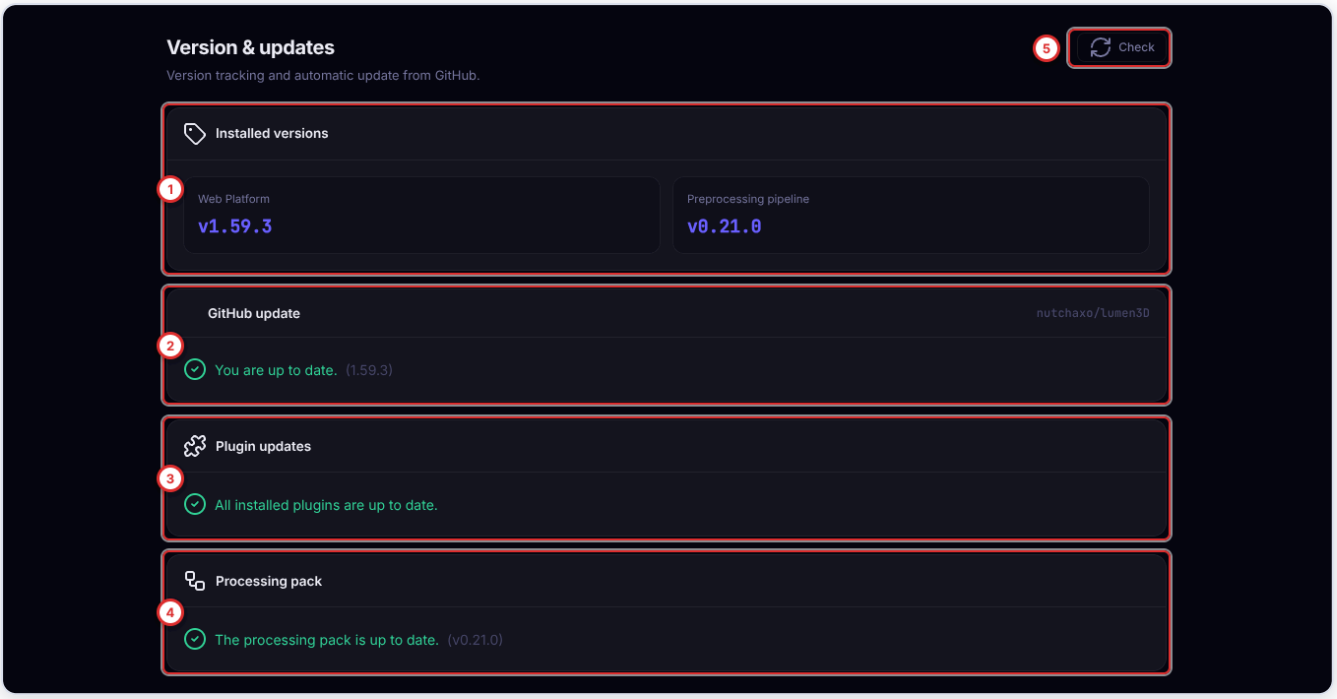
| NEED | WHERE TO LOOK |
|---|---|
| Is the site answering? | `/api/health` (Python) |
| Server logs | `logs/` folder (the server restarted by an update writes `dev-server-<timestamp>.log` ) |
| An update failed | `logs/update-pivot-*.log` , `backups/pivot-journal.json` , `backups/last-update.json` |
| Backups before an update | `backups/backup-<version>-<date>.zip` (the 3 most recent) |
| Change the password | the *Security* tab, or `python dev_server.py --set-password` |
| Validate an installation | `python dev_server.py --check` |

👉 TIP

**Before tinkering, run `--check`.** This command checks, without opening a port, that the essential files are there, that `dev_server.py` compiles, that the language files are readable and that the version is consistent. Exit code 0 = all is well; 1 = a problem (a JSON report says which).

## 18.8 Updating the platform: the principle

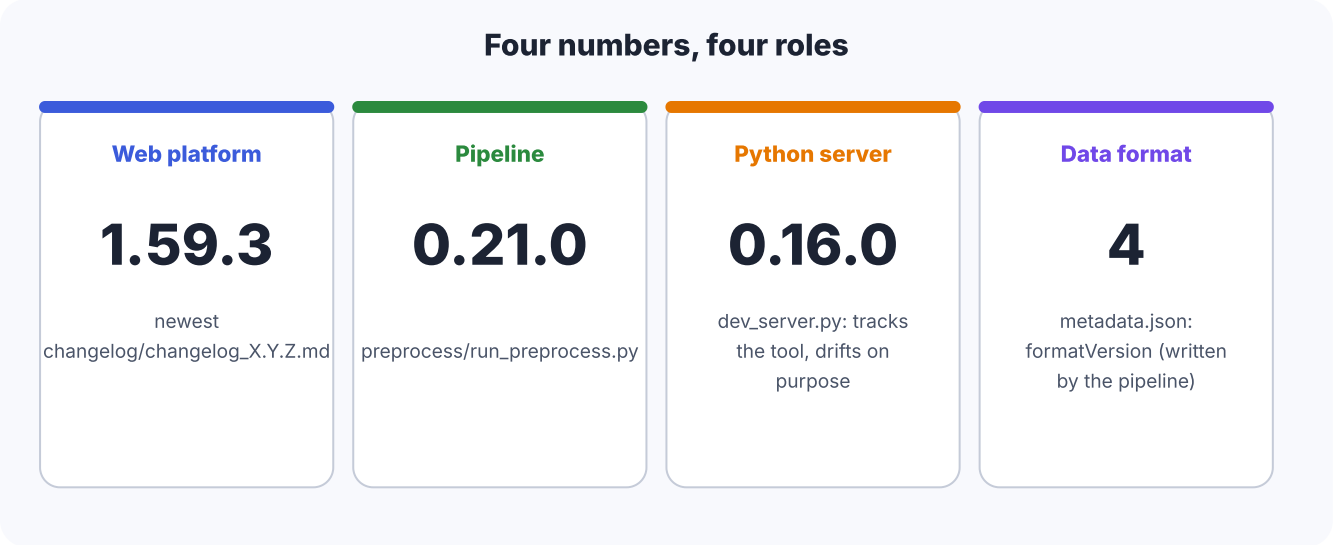
An update is started from **System › Updates**. The administrator sees the state, clicks, and the platform does the rest.




▲ The Updates tab of the administration panel (demonstration dataset, up to date).

| N | WHAT IT IS |
|---|---|
| 1 | **Installed versions**: web platform and preprocessing pipeline |
| 2 | **GitHub update**: “You are up to date”, or the available update |
| 3 | **Plugin updates** |
| 4 | **Processing pack**: the data-preparation software |
| 5 | **Check**: asks GitHub for the state again |

## Four numbers, four roles




▲ The version numbers you will come across.

- The **platform** has **no** version constant: it is the **name of the most recent changelog file** (`changelog_1.65.0.md`). Releasing = creating that file.
- The **Python server** has its own version (`0.16.0`), which **deliberately drifts**: it tracks the server tool, not the platform.
- The **pipeline** (0.22.0) is another piece of software, installed on the processing computer.

## How the platform learns that a version exists

1. It queries GitHub: `releases/latest` of the platform's repository (hard-coded, never typed in by the user). The answer is **kept for 5 minutes**.
2. It compares with its own version. If the most recent is strictly greater, an update is **available**.
3. It looks for **exactly** the elements it needs in the GitHub release.

**⏱ IN 30 SECONDS**

Elements of a release: `lumen3d-web-<version>.zip` (the platform), `SHA256SUMS` (the list of fingerprints), `SHA256SUMS.sig` (the signature of that list), `lumen3d-release-notes.json` (all the release notes), and the **processing packs** named after the **pipeline's** version.

**🔧 FOR THE CURIOUS**

**The pack name carries the pipeline version**, which lets the *Pipeline* tab compare the installed pack with the published one **without downloading anything**, and so offer a pipeline update without a platform update.

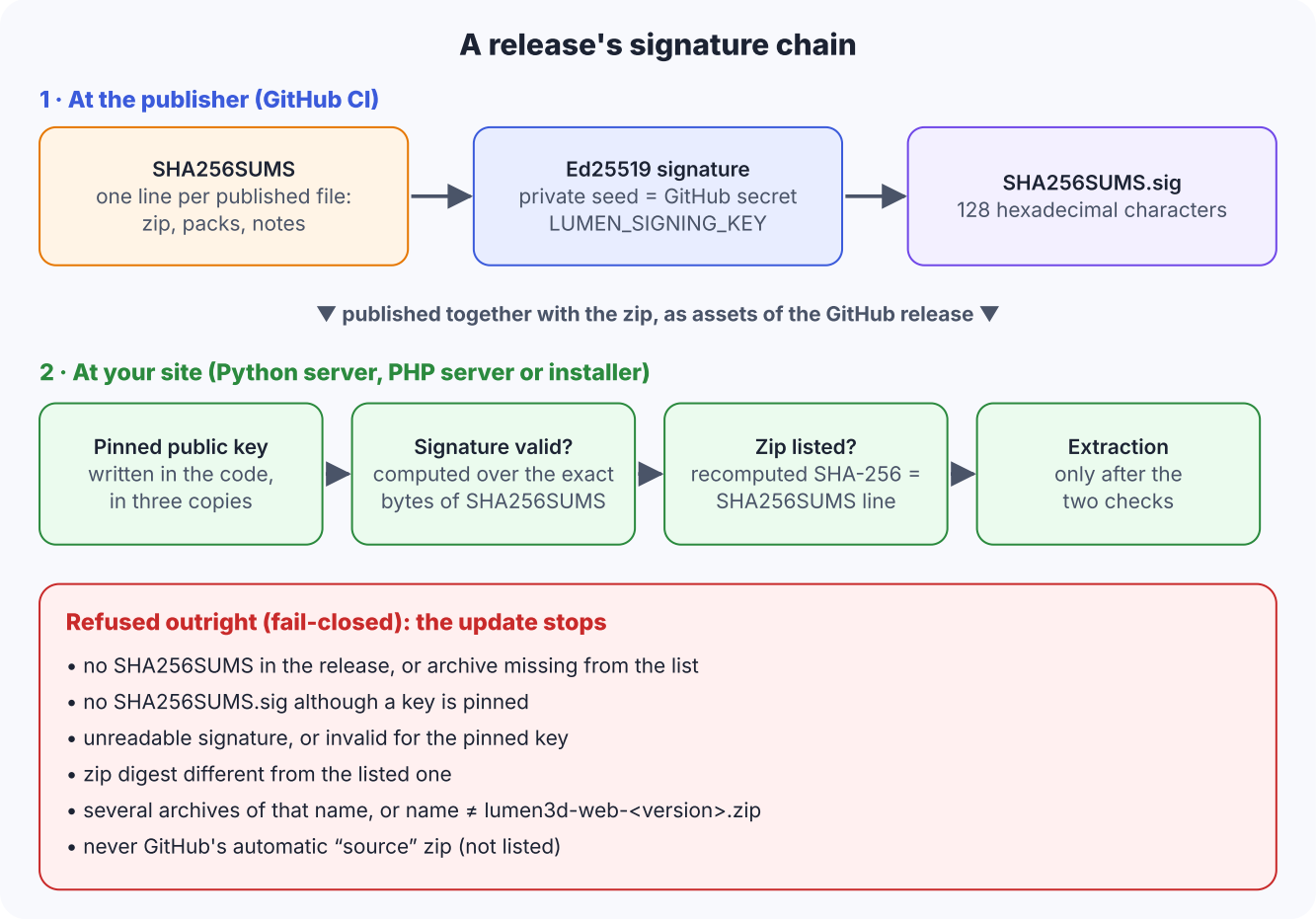
**Release notes**: each release embeds the changelogs of **all** versions (the flat level of `changelog/`, not the archive). A site several versions behind thus reads the notes of each version it is about to absorb, without calling GitHub once per version. Rendering: a foldable tree per version (bold title of each entry), also available as a stand-alone page (`admpan.html?changelog=1`).

**GitHub's limit**: without an account, 60 requests per hour per address. Behind a university NAT it runs out quickly: the panel then shows "too many requests" with the delay, instead of a misleading "403".

## 18.9 The signature: proving a package is authentic

A package that arrives over the Internet could have been **replaced on the way**. Lumen3D therefore installs nothing it cannot prove: the archive must match a list of fingerprints, and that list must carry the publisher's signature.

> 💡 **A PICTURE TO UNDERSTAND**
>
> **A wax seal on a letter.** Only the publisher owns the seal (the private key). Anyone can **check** that a letter carries it (the public key, known to your server), but nobody can make a fake one.




▲ The chain: list of fingerprints, Ed25519 signature, pinned public key, then extraction.

### Two checks, in this order

1. **The signature** (Ed25519 algorithm) is verified on the **exact bytes** of `SHA256SUMS`, with the public key your server already knows.
2. **The archive's SHA-256 fingerprint**, recomputed, must equal **its line** in `SHA256SUMS`. An archive missing from the list is refused.

The word "**fail-closed**" sums up the rule: at the slightest doubt, the door stays shut. The update **stops** and shows the reason. Without a valid signature, nothing is installed.

### Where does the public key live?

In **three constants**, which must be identical: the Python server ( `_RELEASE_PUBKEY_HEX` ), the installer ( `install.php` ) and the PHP administration code ( `LUMEN_RELEASE_PUBKEY` ). The **private** key exists only in GitHub's `LUMEN_SIGNING_KEY` secret.

> ⚠️ **WARNING**
>
> **Why the key is "in the code" and not "in the package".** A key delivered with the package would be the forger's. The pinned key is therefore placed in the repository's **source code**, and travels in every release. A server accepts only the key it knows.

> 🔧 **FOR THE CURIOUS**
>
> **Changing the key (rotation).** The release that **introduces** the new key must still be signed with the **old** one: the server verifies it with the key it knows, then installs the code that pins the new one. Only the following releases are signed with the new key. The tool `tools/gen_signing_key.py` prints a key pair (it writes nothing to disk). **The plugin catalogue has its own key pair** (chapter 15): never the same seed for both.
>
> **On PHP**, the signature is verified by the **libsodium** library ( `sodium_crypto_sign_verify_detached` ); on Python, by an Ed25519 verifier written in pure Python ( `ed25519_pure.py` , RFC 8032), which needs nothing beyond the standard library. A key left empty would downgrade verification to "SHA-256 only" with a warning; the key is set today.

## 18.10 Updating on a Python server: eight steps

As long as the server has not switched over, **the running installation is not touched**. The update runs in a thread of the **live** server, which keeps answering visitors.

▲ Eight steps; only the last one modifies the site.

| STEP | WHAT IS DONE | WHAT STOPS IT |
|---|---|---|
| **1 Checks** | same volume, free space ≥ max(3 × the archive, 300 MB), `backups/` writable | a previous switch-over not reconciled |
| **2 Backup** | zip of everything the release manages (protected paths, §18.13, are excluded); re-read after writing | an unreadable file: the backup **aborts** |
| **3 Download** | size compared with the announced one | truncated archive |
| **4 Authenticity** | signature then SHA-256 (§18.9); archive intact ( `testzip` ) | any doubt |
| **5 Preparation** | safe extraction into a separate folder | suspicious entry (see below) |
| **6 Dry start** | `dev_server.py --check` **on the new tree** | an essential file is missing |
| **7 Plan + journal** | list of files to put in place and to remove, written to disk | — |
| **8 Switch-over** | a detached supervisor takes over (§18.11) | — |

⚠️ **WARNING**

**A release that contains `api/plugin-trust.json` is refused.** This file lists the plugins the operator has approved: a malicious release could otherwise **pre-approve** a booby-trapped plugin. Likewise, the staged release must carry **exactly** the announced version, and every file must match the fingerprint recorded in its `version.json` .

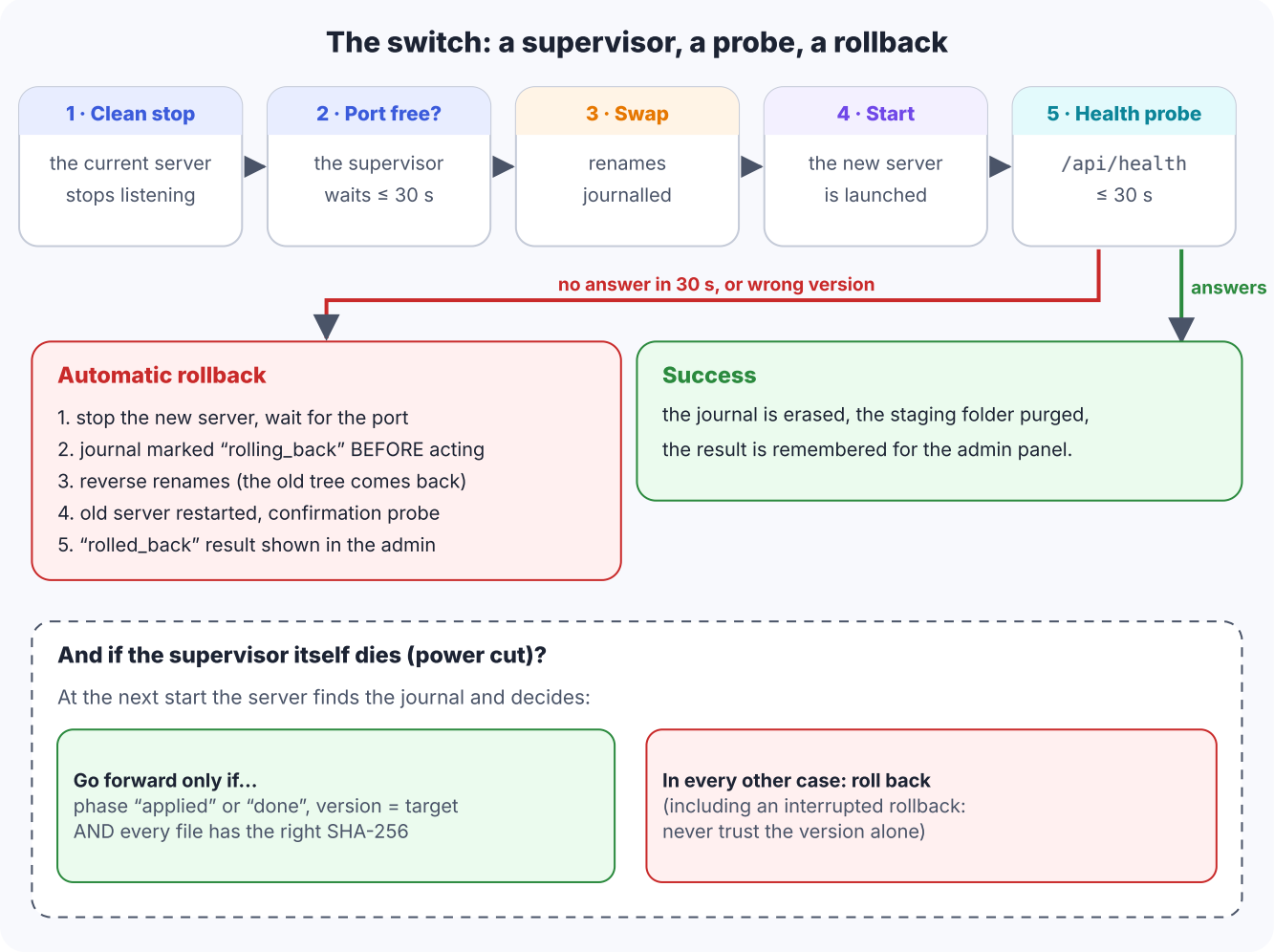
🔧 **FOR THE CURIOUS**

**Safe extraction.** Every entry of the archive is checked before extraction: an absolute path, a drive letter, a backslash or `..` are refused (an archive received from the Internet is untrusted input). A clean-up keeps the **3** most recent backups and deletes stale temporary folders.

**Four approaches were compared** at design time (in-place journal, git checkout, permanent supervisor, swap by preparation): the last was chosen because **the site is touched only by renames**, never by an in-place copy. An interruption half-way would otherwise leave a half-old, half-new tree that could not start.

## 18.11 The switch-over and the way back

The server that prepared the update cannot replace itself: its files are **in use**. It therefore launches a **supervisor**: a **copy** of `dev_server.py` , placed in a private temporary folder, which holds no file of the installation.




▲ The switch-over, step by step, with its rollback branch.

💡 A PICTURE TO UNDERSTAND

**The blue-green move.** You set up the new flat next to the old one, check that the water and electricity work, and only then move in. If something is wrong, you go back to the old one, which stayed intact.

## The five beats

1. The current server **stops listening** (one second after letting the answers in progress go out).
2. The supervisor waits for **the port to be freed** (30 s at most).
3. It **swaps** the files by **renames**, each noted in the journal. No byte is copied: everything is almost instantaneous.
4. It **starts the new server**.
5. It **queries** `/api/health` every half second for 30 s. Success = the probe answers with **the target version**.

## If the probe fails

The supervisor stops the new server, marks the journal “rolling_back” **before** acting, applies the renames **in reverse**, restarts the old server and probes it in turn. The panel then shows: “The new version did not start — automatic restoration done. The old version is working.”

🔢 WORKED EXAMPLE

**A two-step rename.** To replace `viewer.html`: the old one goes into a mirror folder (`old/`), then the new one takes its place. At each step the supervisor **looks at the disk** (not at its memory): if it is interrupted and restarted, it resumes **exactly** where it stopped. The rollback is safe from **any** intermediate state.

### And if the supervisor dies too?

At the next start-up, the server finds the **journal**. It moves forward only if **everything** is proven: phase "applied" or "done", version equal to the target **and** every file with the right fingerprint. In **every** other case, it goes back. The version alone is not enough to conclude: it is only one file among thousands.

🔧 FOR THE CURIOUS

**Windows details** that shaped the design: a server process locks its working folder (the supervisor runs from the temporary folder); antivirus programs place transient locks on fresh files (the rename is retried up to 10 times); the server is restarted as a **detached** process. A listening address of `0.0.0.0` cannot be reached: the probe uses `127.0.0.1`. Otherwise every update on `--host 0.0.0.0` would have been wrongly cancelled.

**Tracking files**: `backups/pivot-journal.json` (phases `planned` → `applying` → `applied` → `done`; or `rolling_back`), `logs/update-pivot-<timestamp>.log`, and `backups/last-update.json`, the "last result" the panel reads after the restart (`done` or `rolled_back`). The tests `tests/test_update_pivot.py` and `tests/test_update_reconcile.py` cover full application, restoration, replay after a partial crash and the double rollback.

## 18.12 Updating on PHP hosting

A PHP host **also updates itself**, but differently: PHP has no continuously running server, so there is nothing to restart. Everything happens in **a single request**, from click to answer.

### Two ways to update, depending on the server

| | Python — dev_server.py | PHP — api/_admin_lib.php |
|---|---|---|
| **Flow** | background task + supervisor | a single synchronous request |
| **Zip backup** | yes (3 kept) | no |
| **Dry run** | --check | no |
| **File swap** | journalled renames | old file parked, then new one placed |
| **Restart + probe** | yes, /api/health | n/a: PHP starts afresh every request |
| **Rollback** | automatic | none: just run again (idempotent) |
| **Removed files** | diff of the version.json files | fixed list (api/config.php) |
| **Signature + SHA-256** | identical, fail-closed | identical, fail-closed |

▲ The two ways of updating, side by side.

⚠️ **WARNING**

**No automatic rollback on PHP.** A PHP update that fails half-way does not “undo” itself. It is designed so that **you can simply run it again**: the copy is idempotent, and the release notes ( `changelog/` ) are copied **last**, so that a failure leaves the **old** version declared and a new attempt is allowed.

## How it runs

1. **Lock**: one update at a time (a second click gets “update in progress”).
2. **Target version**: same search as on Python; only the archive named `lumen3d-web-<version>.zip` is accepted.
3. **Authenticity**: same double check (libsodium signature, SHA-256).
4. **Extraction** into a working folder **under the site root** (same disk: a rename there is instantaneous; outside the site, it failed on some shared hosts).
5. **Archive checks**: at most 5,000 entries, 128 MiB uncompressed, no suspicious path ( `..` , `.` , empty, backslash, drive).
6. **Putting files in place**: each old file is **parked** ( `.lumen-old` ) then replaced; protected paths are skipped (§18.13).
7. **Tidying**: old files removed, PHP code cache (opcache) invalidated so that the **new** version answers from the next request.

## Special cases

- **Busy file** (Windows): the new file is parked as `.lumen-new` and finished in a later request.

- **Root** `.htaccess` : the `LUMEN3D` block is **rewritten**, but **your lines** ( `AddHandler` , `RewriteBase` , `php_value` ...) are kept, and a `.lumen-backup` copy is preserved. Without this, an update could make PHP be served **as text**.
- **Removed files**: on PHP, only a short list is deleted ( `api/config.php` , an old file that held a password hash). There is no comparison of the `version.json` files.

> 🔧 **FOR THE CURIOUS**
>
> The execution time is extended to 600 s. The Python-only files ( `dev_server.py` , `fast_server.py` , `ed25519_pure.py` , `start.bat` ) are **skipped** on a PHP host. The `js/modules/` folder is never overwritten (an installed plugin is not part of the release).

## 18.13 What an update never touches

A release **replaces the code**, never **your data or your work**.

**What an update touches, and what it never touches**

**Never touched**

**`DATA_WEB/`**
your datasets

**`uploads/`**
imports in progress

**`logs/ backups/ secrets/`**
logs, copies, seeds

**`api/admin_credential.json`**
your password (hash)

**`api/stats.json`**
the usage counters

**`api/plugin-trust.json …`**
approvals, disabled plugins

**`api/marketplace-state.json`**
anti-rollback serial

**`config/instance.json theme legal pages/`**
your customisation

**`api/page-drafts/`**
unpublished drafts

**`install.php`**
the installer stays locked

**Replaced by the new version**

**HTML pages**
index, viewer, compare, 2d, admpan ...

**`js/ css/ lang/`**
code, styles, translations

**`api/*.php`**
the server's PHP code

**`dev_server.py …`**
Python server and engines

**`changelog/`**
release notes (the version = the newest)

**`.htaccess`**
LUMEN3D block rewritten, your lines kept

**`config/defaults/`**
neutral templates

An installed plugin's code (js/modules/) is not in the release: it is therefore never overwritten.

▲ On the left what is untouchable, on the right what the update replaces.

- The list of protected paths is in `_UPDATE_PROTECT` (Python) and `admin_update_protected()` (PHP). The two are **twins**.
- In `config/` , only the files the operator edits are protected; `config/defaults/` is part of the release and **stays** updated.
- **Deletions** are computed on Python by the difference between the two `version.json` files (old and new): a file shipped before but not any more is deleted; **a file unknown to both** (that you added) is never touched.

### Plugins during an update

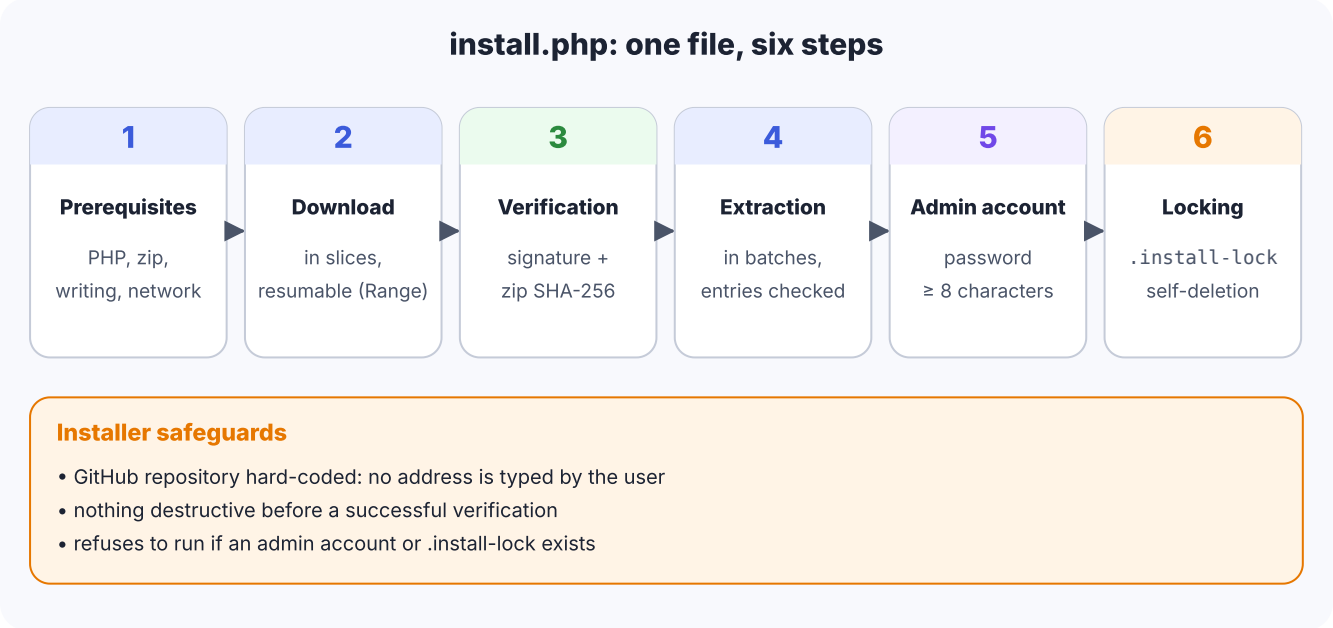
Before switching over, the *Updates* tab runs a **pre-flight**: it asks which installed plugins would be **incompatible** with the target version (each plugin's `platformCompat` field, chapter 15).

- An incompatible plugin is **set aside** ("quarantine") after the switch-over, not deleted: its folder stays in place.
- As soon as an update of the plugin, or of the platform, makes it compatible, it is **re-enabled automatically** at the next discovery.
- Only one case **blocks** the update: if **no render mode at all** would remain available (the viewer could no longer display anything).

> 📌 **REMEMBER**
>
> **A crashing plugin never brings the viewer down.** The registry isolates each plugin (a try/catch per plugin, plus a global barrier): the 3D canvas always starts, even if the whole plugin subsystem fails.

## 18.14 First installation: `install.php`

For an empty host, **a single file** is enough: you drop `install.php` into an empty web folder and open it in a browser.




▲ The six steps of the installer.

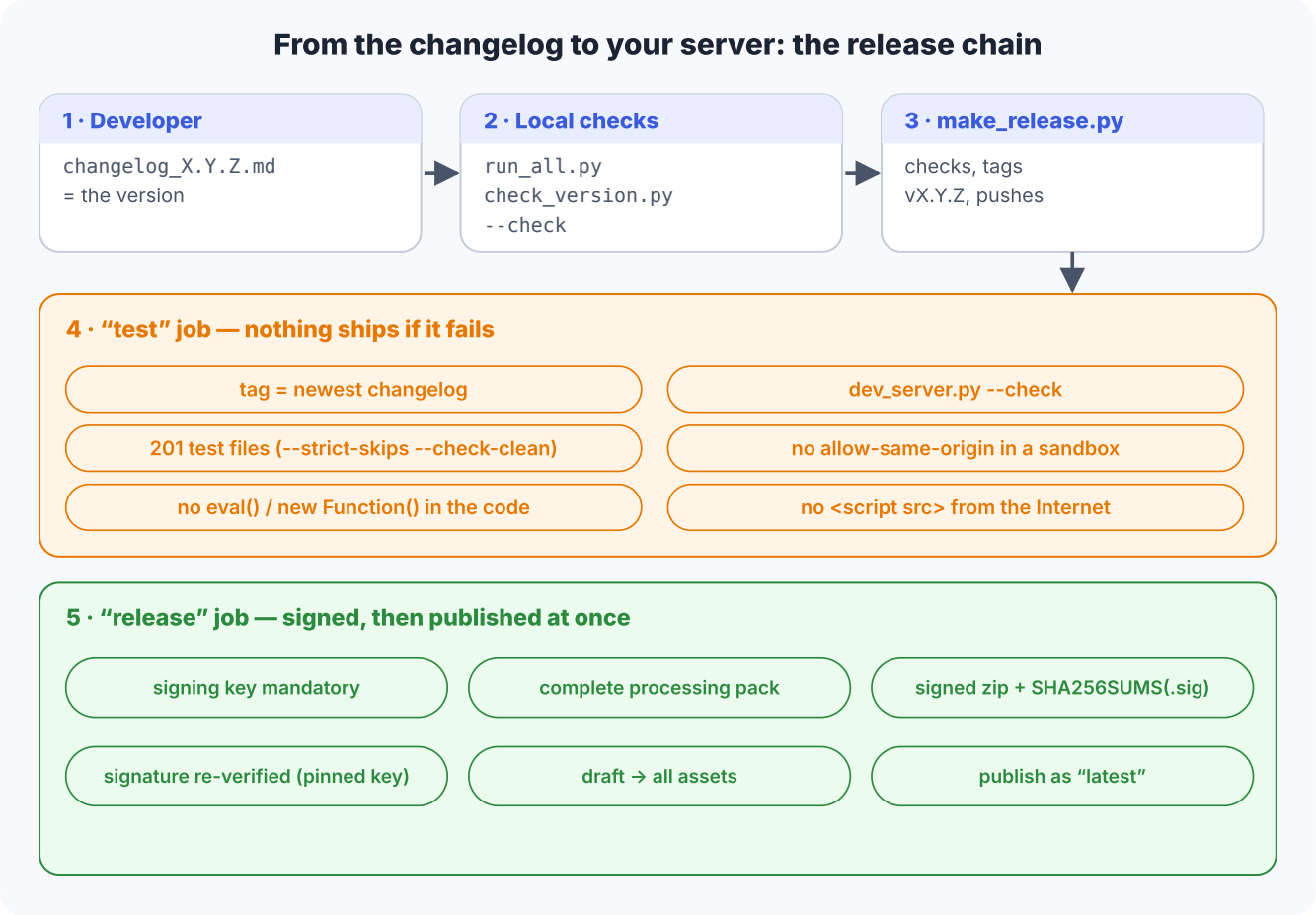
- It downloads **the latest release** of the platform's repository, **verifies** the signature and the fingerprint (§18.9), extracts, then asks you to create the **administrator account** (8 characters minimum).
- The download is done **in slices** (8 MiB or 10 s) with resume, to survive the time limits of shared hosts. Extraction is done in batches of 200 entries.
- At the end, it writes `.install-lock` and **offers to delete itself**. If it finds an administrator account or this lock, it **refuses to run**: it cannot be replayed to overwrite a live site.

🔧 **FOR THE CURIOUS**

Extra safeguards: GitHub repository **hard-coded** (no address is typed in); every archive entry checked (no traversal, symbolic link or absolute path); **zip bombs** bounded (20,000 entries, 500 MB uncompressed, archive ≤ 512 MB); `install.php` never overwrites itself; anti-forgery token on every request; state in `.install-state.json` (no secret, deleted at the end). The account file is created with **exclusive creation** and in the same format as that of the two servers.

## 18.15 Releasing a version (for the publisher)

This section describes the chain on the **publisher's** side: the person who evolves the platform. It shows why the version that reaches you has a good chance of being correct.




▲ From idea to your server: the whole path of a release.

1. **Write the changelog.** Create `changelog/changelog_X.Y.Z.md` : **it is the version number**. Sections `[ADDED]` , `[OPTIMIZED]` , `[FIXED]` , `[CHANGED]` ; one entry per bullet, `- **Short title**: what changes and why` . The administration panel shows only the bold title in folded mode.
2. **Check locally**: `python tests/run_all.py -j 4` , `python tools/check_version.py --tag vX.Y.Z` , `python dev_server.py --check` .
3. **Tag**: `python tools/make_release.py` (or `--dry-run` to simulate). It checks the changelog, refuses a dirty tree or any branch other than `main` , sets the `vX.Y.Z` tag and **pushes the tag**.
4. **CI takes over**: it publishes only if **everything** passes (§18.16).

### What the “release” job does

- **Refusal without a key**: without the `LUMEN_SIGNING_KEY` secret, it stops: never an unsigned release.
- **Processing packs**: the full pack is built **before** the zip, so that its fingerprint appears in the signed list. The light pack is copied from the tree.
- **“Curated” zip**: built from a **whitelist** of files (anything not on it is excluded, so documentation, tools or data never leak by accident). Fixed timestamp: two builds give the same bytes.
- **Signature** of `SHA256SUMS`, then **re-verification** against the key pinned in the code: a release that servers would refuse is itself refused.
- **Publication in three stages**: the release is created as a **draft**, receives **all** its files, **then** is published. A site that asks for “the latest release” therefore never sees a release without its zip.

> ❓ **WHY THIS CHOICE?**
>
> **Why does CI create the release, and not `make_release.py` ?** A release created by hand would first appear **without files**: the updater would then fall back on GitHub’s automatic “source” zip, which **cannot be verified**. Creation is therefore left to CI.

### Two other forms of publication

| | PLATFORM | PLATFORM PLUGIN | THIRD-PARTY PLUGIN |
|---|---|---|---|
| **Version** | new `changelog_X.Y.Z.md` | `plugin.json` → `version` | same (informative) |
| **Publish** | `make_release.py` → CI | `publish_plugin.py <folder> --push` | drop the folder in |
| **Apply** | admin → Updates | admin → Catalog | approve (fingerprint) |
| **Signature** | Ed25519 of the release | Ed25519 of the catalogue (another key) | out of scope |

Platform plugins do **not** ship with the release: they are published in the **signed catalogue** and installed on demand (chapter 15). Their publication takes effect when `dev` is merged into `main`, because the catalogue is read from `main`.

## 18.16 The test suite: what guards the door

A release is signed **only if** the test suite is entirely green. This suite is launched by **a single command**.

▲ 201 test files, three languages.

| 201 | 124 | 55 | 22 |
|---|---|---|---|
| test files | Node (viewer, workers, protocols) | Python (server, pipeline, import) | PHP (twin server) |

- `python tests/run_all.py` discovers every `tests/test_*.py|js|php` and `tests/js/test_*.mjs` file, and runs **each in its own process**.
- `--strict-skips`: a test that cannot run (a missing PHP extension) must say so, and this mode counts it as a **failure**. A test therefore cannot stop running without anybody noticing.
- `--check-clean`: a test that **dirties the repository** (a file created or modified) fails. Everything must happen in temporary folders.

❓ **WHY THIS CHOICE?**

**Why test the “twins” with the same vectors?** Several rules exist in **two or three languages** (version compatibility, trust fingerprint, import, release notes). The same file of cases is played by Python, PHP and JavaScript: if they diverge, a test fails. For example, `tests/compat-vector.json` contains 42 cases.

## Continuous integration, at every change

Two GitHub workflows: `ci.yml` (at every code push to `main` or `dev`, and for any merge request) and `release.yml` (at every `v*` tag). They chain:

| CHECK | WHAT IT GUARANTEES |
|---|---|
| `check_version.py` | the tag equals the most recent changelog; no duplicate or badly named file |

| CHECK | WHAT IT GUARANTEES |
|---|---|
| `dev_server.py --check` | a fresh installation starts dry |
| **full suite** | no behaviour has degraded |
| no `allow-same-origin` | a plugin sandbox never loses its isolation |
| no `eval()` / `new Function()` | the code stays compatible with the strict security policy |
| no `<script src="http…">` | no library comes from the Internet |

## 18.17 Working without Internet

Lumen3D was designed for a **closed network**. Everything the **visitor** needs is in the site's folder.

### What works without Internet, and what needs it

**No Internet (closed institute network)**

- ✔ all pages and their scripts
- ✔ Three.js, Lucide, Plotly: site files
- ✔ your datasets and the catalogue
- ✔ the administration panel
- ✔ import and data updates
- ✔ 3D display (local graphics card)

**Needs Internet — the administrator only**

**`api.github.com`**
look for a new version

**`github.com`**
download the release and packs

**`raw.githubusercontent.com`**
read the signed plugin catalogue

**A visitor never contacts GitHub.**

**The one exception on the visitor's side**

`fonts.googleapis.com fonts.gstatic.com`

Fonts do not hold the page back:
without a network the page shows with the system fonts.

▲ What works offline, and what needs Internet.

| LIBRARY | VERSION | FILE |
|---|---|---|
| **Three.js** | 0.147.0 | `js/vendor/three.min.js` (+ GLTF and OrbitControls loaders) |
| **Lucide** | 0.344.0 | `js/vendor/lucide.min.js` |
| **Plotly** | 2.27.0 | `js/vendor/plotly.min.js` (loaded only when a chart is opened) |

- Each library is checked by an **integrity fingerprint (SRI)**: if a single byte changes, the browser refuses it. The `js/vendor/` folder is protected from line-ending conversions so that the fingerprint stays valid.

- The security policy allows **no foreign script**: adding a `<script src="https://…">` tag would break the page, and continuous integration refuses it.
- **The only visitor-side exception**: fonts (Google Fonts). They are declared to load **in the background**: on a network that filters them, the page displays at once with the system fonts.
- **On the administrator side**: GitHub is contacted only to look for a release, download it and read the plugin catalogue. **A visitor never contacts GitHub.**

🔢 **WORKED EXAMPLE**

**An institute network with no way out.** The viewer, the Explorer, the import and the **data** updates work. Only the administration's *Updates* tab will show "unreachable"; to update, drop the release in by hand (or do it from a connected computer).

## 18.18 What to do if… ?

| SYMPTOM | PROBABLE CAUSE | WHAT TO DO |
|---|---|---|
| **Update refused: "This version cannot be verified"** | archive missing from `SHA256SUMS`, or invalid signature | force nothing; tell the publisher |
| **Update "automatic restoration done"** | the new version did not answer `/api/health` within 30 s | read `logs/update-pivot-*.log` |
| **"The server is not answering" after an update** | long switch-over or crash | check `logs/`, then reload; the server reconciles by itself at start-up |
| **After a PHP update, a page is broken** | interrupted copy | **run** the update **again** (idempotent) |
| **A plugin disappeared after an update** | its `platformCompat` no longer covers the version | *Plugins* tab: update the plugin |
| **429 error on the viewer** | the host limits requests per address | check that the `?v=` addresses are being served (cache) |
| **Everyone is blocked at login** | undeclared proxy (only one address seen) | `--trusted-proxy` (§18.6) |
| **PHP administration is slow** | session lock | check that the code is up to date (§18.6) |
| **Without HTTPS, the import refuses to start** | the browser requires a secure context to compute fingerprints | HTTPS, or `localhost` |
| **A new dataset does not appear** | card hidden, or neither configured nor thumbnailed | edit the card (chapter 14) |

📌 **REMEMBER**

- Three servers: **Python** (everything), **PHP** (everything, no automatic rollback), **files** (read-only).
- A `?v=` tells the browser “this address never changes”: packs 1 year, scripts 7 days; everything else is **revalidated** (304).
- A release is **proven** (pinned Ed25519 signature + SHA-256), **tested** (201 files) and **switched** with a safety net.
- Your **data, passwords and customisations** are never touched by an update.

📖 **GOING FURTHER**

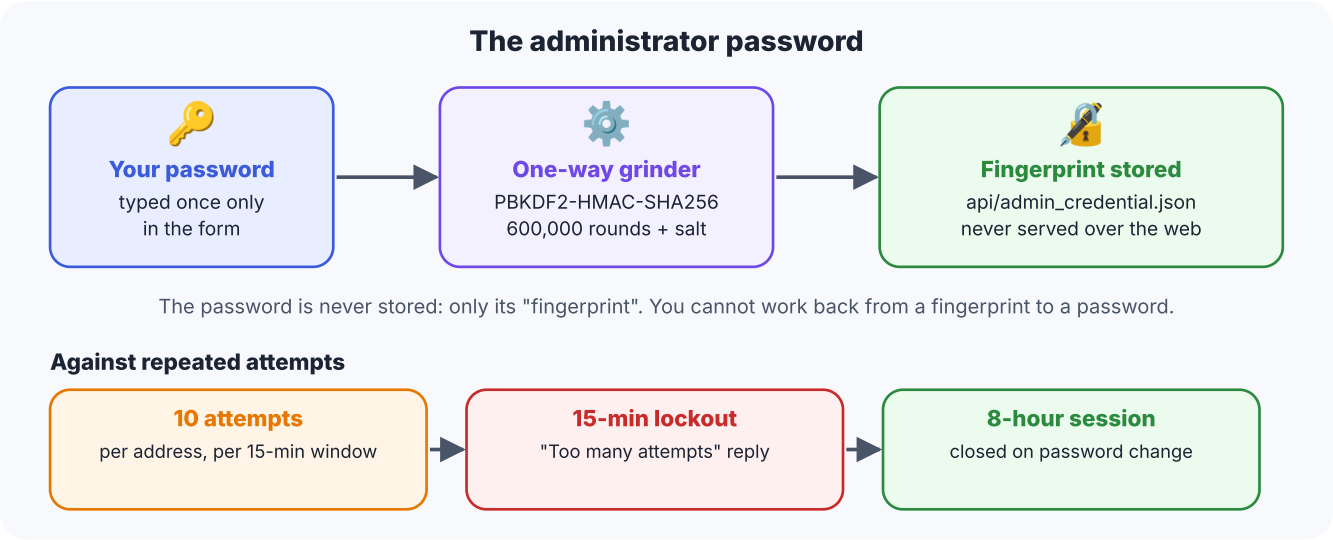
- Chapter 10: the brick loader, byte ranges in practice and the `?v=` of packs.
- Chapter 14: the *Import* tab, editing a dataset and the administration screens.
- Chapter 15: plugins, trust, sandbox and the signed catalogue.
- Chapter 19: password, security policy (nonce) and reliability.
- Design details: `DOCS/update-system/` (technical, publication, walkthrough).

# 19. Security and reliability

- Lumen3D protects the administration panel, the code that runs in the pages and the files that are uploaded. Each protection has a **simple reason**.
- No study data is ever sent to a third party, and the platform works **offline**.
- Reliability rests on **all-or-nothing** publishing and on a suite of about **200 test files** that blocks any faulty release.
- A single table, "**What happens if…?**" (sections 19.8 to 19.12), lists every foreseen failure and the fallback waiting for it.

## 19.1 The administrator password

There is **no default password**: the first visitor to the panel creates the account (8 characters minimum), and this creation can never overwrite an existing account.




▲ From password to fingerprint, and the protections against repeated attempts.

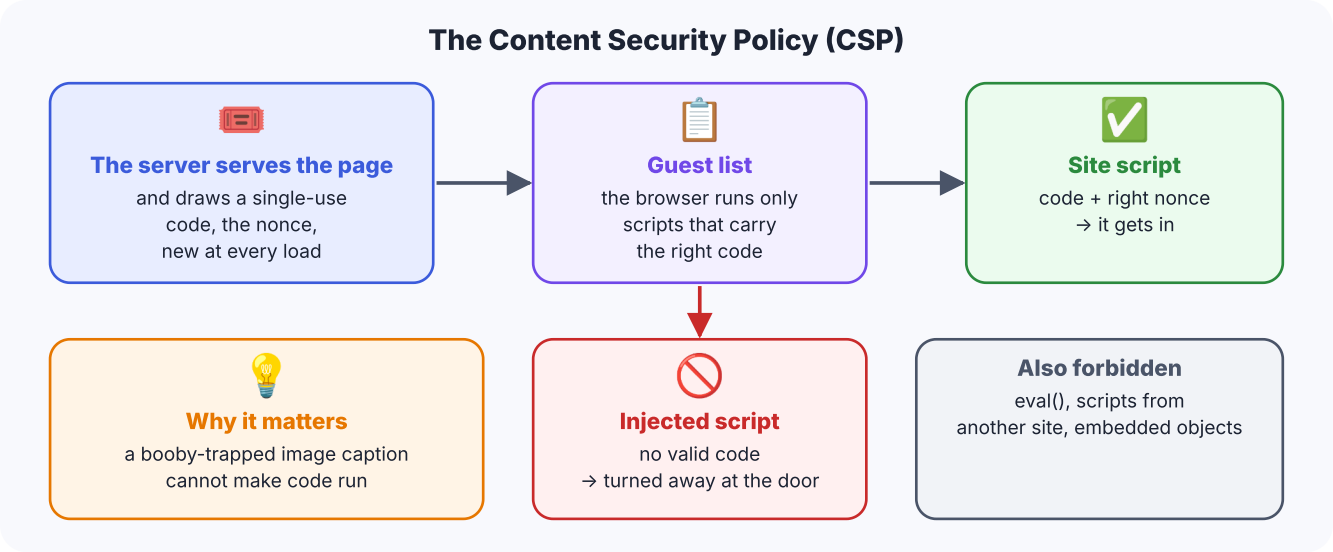
💡 **A PICTURE TO UNDERSTAND**

**A fingerprint.** The server keeps the fingerprint of your password, not the password. With a fingerprint you can recognise the right person, but you cannot rebuild the finger.

- The fingerprint is computed with **PBKDF2-HMAC-SHA256**, with a salt unique to the account and 600,000 rounds: deliberately slow, to discourage mass guessing.
- After **10 failures** in 15 minutes from the same address, access is blocked for 15 minutes. Failures are counted **before** the check, and a wrong user name costs the same time as a wrong password.
- A session lasts **8 hours**. Changing the password closes all other sessions.
- Every change also goes through an anti-forgery token (CSRF).

## 19.2 The Content Security Policy (CSP)

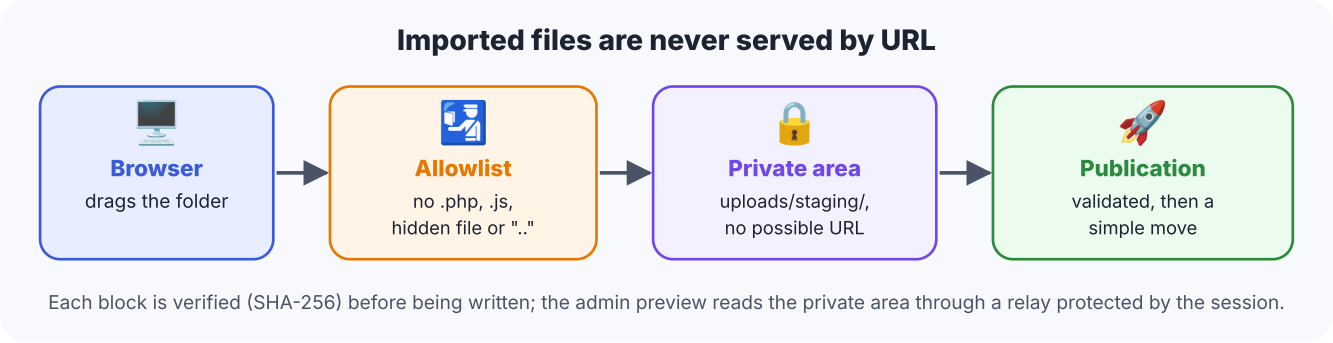
> 💡 **A PICTURE TO UNDERSTAND**
>
> **A guest list at the door.** On each page load, the server draws a secret code (the “nonce”) and hands it to the site’s scripts. The browser lets in only those that present it.




▲ The nonce, a guest list renewed at every load.

The policy is **enforced** (not merely observed). The libraries (Three.js, Lucide, Plotly) are hosted by the site itself: no script comes from elsewhere.

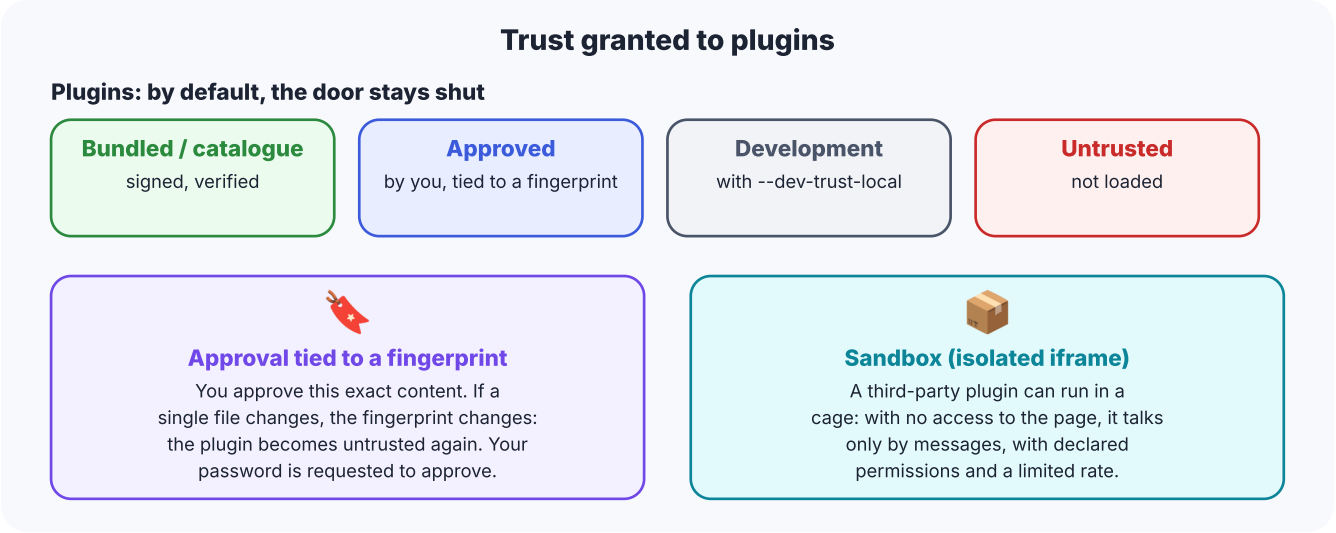
## 19.3 Uploaded files are never served




▲ The path of an uploaded file up to its publication.

- Files arrive in `uploads/`, a folder that is **unreachable by URL** (four independent locks).
- An **allowlist** refuses, before anything is written, whatever the pipeline does not produce: no `.php`, no `.js`, no hidden file, no upward path (`..`).
- Only an **explicit publish**, after validation, moves the dataset to `DATA_WEB/`. That folder forbids the execution of scripts.

## 19.4 Plugins: trust and cage




▲ Trust levels, approval tied to a fingerprint, sandbox.

❓ WHY THIS CHOICE?

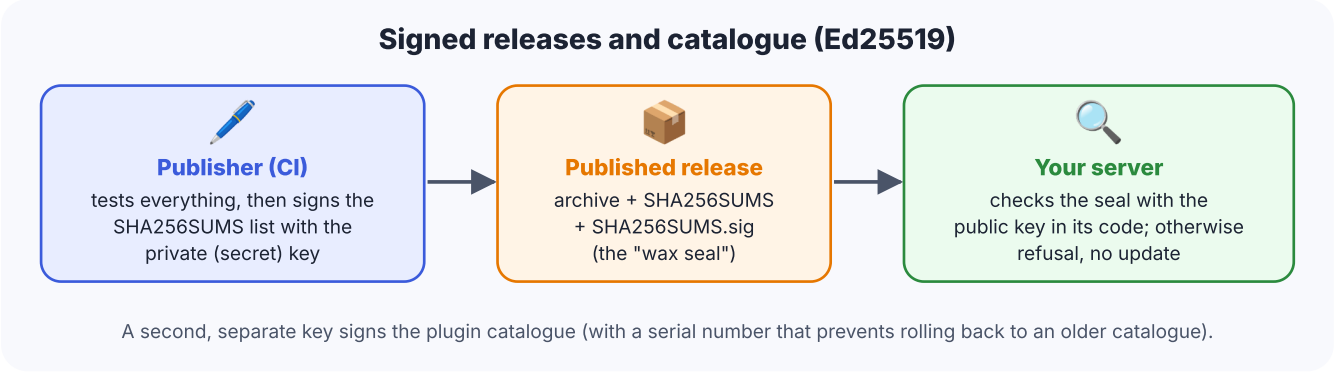
A plugin is code. Letting it in unchecked would be like giving a stranger a copy of your keys. By default, an unrecognised plugin **is not loaded**.

- Your approval is tied to the plugin's **exact content** (fingerprint): if it is modified, it loses its trust.
- A plugin also declares which platform versions it is compatible with: when in doubt, it is set aside.

## 19.5 Signed versions and catalogue

💡 A PICTURE TO UNDERSTAND

**A wax seal.** Only the publisher holds the seal (the private key). Anyone can check that a letter bears it (public key), but nobody can make a fake one.




▲ The chain of trust of an update: signature, verification, refusal.

- The signature is an **Ed25519** one. The public key is **built into the server's code**: a key supplied with the download could not be trusted.
- A version without a valid signature is **refused**: the updater applies only the archive named after the version and listed in the signed list.

- The plugin catalogue has **its own key** and an increasing serial number, so that nobody can reinstall an older version that has since been fixed.

## 19.6 Your data stays with you

- **No study data** is sent to a third party.
- **Offline**: all the code is hosted on the server. Only the Google Fonts are loaded remotely. GitHub is contacted only by the administrator, to check for updates.
- The statistics are simple counters (visits, views, downloads). What is **kept**: one overall total, one total per day and one total per dataset (with the date of its last view), in `api/stats.json`. Nothing in it is linked to a person.
- The visitor's address is used **in memory only**, to limit the rate: it is reduced to one of 4,096 slots of a "token" table (60 counts in a burst per address, then 1 per second; 600 in a burst for the whole site, then 20 per second). It is written nowhere.

## 19.7 Reliability

| RISK | PROTECTION |
|---|---|
| **An update breaks the site (Python server)** | switch-over checked by a health probe, **automatic rollback**, prior backup |
| **An update fails (PHP hosting)** | SHA-256 and signature checks **before** any file is touched; no automatic rollback, but the copy can be **replayed** without damage (chapter 18) |
| **A publication is interrupted** | **all or nothing**: the pipeline builds everything aside and publishes by a rename; an interruption is picked up at the next launch |
| **A network outage during an import** | resume to the nearest block |
| **A corrupt brick** | reported and ignored, never sent to the graphics card |
| **A regression in the code** | about **200 test files** must pass before each release |

> 📌 **REMEMBER**
>
> Each version is **tested in full** before being signed and published. A version that fails the tests simply cannot be released.

## 19.8 What happens if…?

- A reliable platform is not one where nothing ever fails: it is one where **every failure has a plan**.
- This part goes through them all: what Lumen3D does, what you see, what you have to do.
- Four families: **the image** (bricks, graphics card), **the data** (import, migration, metadata), **the server** (update, access) and **the browser**.

Three principles guide every fallback. They explain why the answer is almost always “the rest carries on”.

### Degrade, don’t crash

Less detail beats a frozen tab. We drop to the coarser level, set the doubtful brick aside, show a message.

### All or nothing

A publication, a conversion or an update is applied **in full** or not at all. Never half a dataset online.

### Resume, don’t restart

Long operations keep a **journal**: after an interruption, they pick up where they stopped.

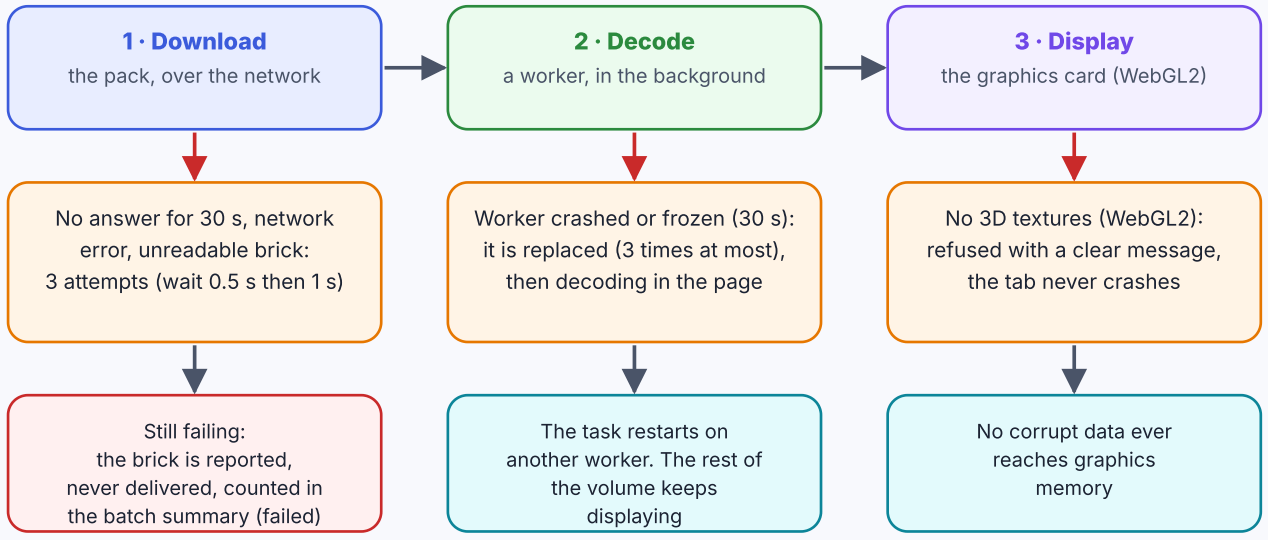
💡 **A PICTURE TO UNDERSTAND**

**A belt, braces and a waistcoat.** Every risk has its safety net, and often two: the corrupt brick is retried *then* set aside; the server is tested before *and* after the switch-over.

## 19.9 On screen: bricks, workers and metadata

A brick goes through three stages (network, decoding, graphics card). Each has its fallback.

**A brick in trouble: the ladder of fallbacks**




**Golden rule: a doubtful brick is dropped, never shown wrongly, and the tab does not crash.**

▲ The three stages of a brick and the fallback of each.

| IF... | WHAT LUMEN3D DOES | WHAT YOU SEE OR MUST DO |
|---|---|---|
| **A brick is corrupt (cannot be decoded, unexpected size)** | It is **reported** ( `onBrickError` ) and **never delivered** to the graphics card; 3 attempts, waiting 0.5 s then 1 s | The rest of the volume is displayed. Nothing to do |
| **A pack stops answering (nothing arrives for 30 s)** | The download is abandoned, then retried (3 attempts) | A progress bar that starts again. Nothing to do |
| **A brick is absent from the index file** | This is not a failure: a completely empty brick is not stored, it is a volume of zeros (chapter 7) | Nothing: it is empty space |
| **A decoding worker crashes or freezes for more than 30 s** | It is replaced (3 times at most) and its task starts again; without a worker, decoding is done in the page | A slight slowdown, never a frozen tab |
| **Another dataset is opened during a load** | The current download batch is cancelled (reason `dataset-switch` ), without an error | Nothing: the old dataset simply stops loading |
| **The dataset changes brick folder along the way** | Tasks planned on the old folder are refused ( `BRICKS_MOUNT_CHANGED` ) rather than mixed | Nothing: the load is planned again |
| **The graphics card offers no 3D textures (no WebGL2)** | Refused **with a message**; the tab does not crash | Use a recent browser or computer |
| **The metadata file is malformed** | It is **rejected as a whole**, never mounted in part: root that is not an object, invalid x/y/z dimensions, inconsistent channel count, invalid voxel sizes... | A message gives the reason; the administrator fixes the dataset |
| **An optional block is malformed (stabilisation, cell tracking, gallery...)** | That block alone is **dropped with a warning**; the dataset mounts without that feature | The volume is displayed, without the layer concerned |
| **The dataset has more than 4 channels** | The pipeline wrote them all; the viewer draws **four** (RGBA texture) and says so when mounting | The first four channels are visible |
| **The calibration is missing** | No 3D scale bar; a 3D distance measurement is **refused**; in the Studio, measurements are in pixels. No scale is invented (chapters 9 and 12) | The scale does not appear |
| **You click beside the specimen to measure** | The point falls on the bounding box with nothing visible beneath: it is **refused** with a notice | Click on the structure itself |

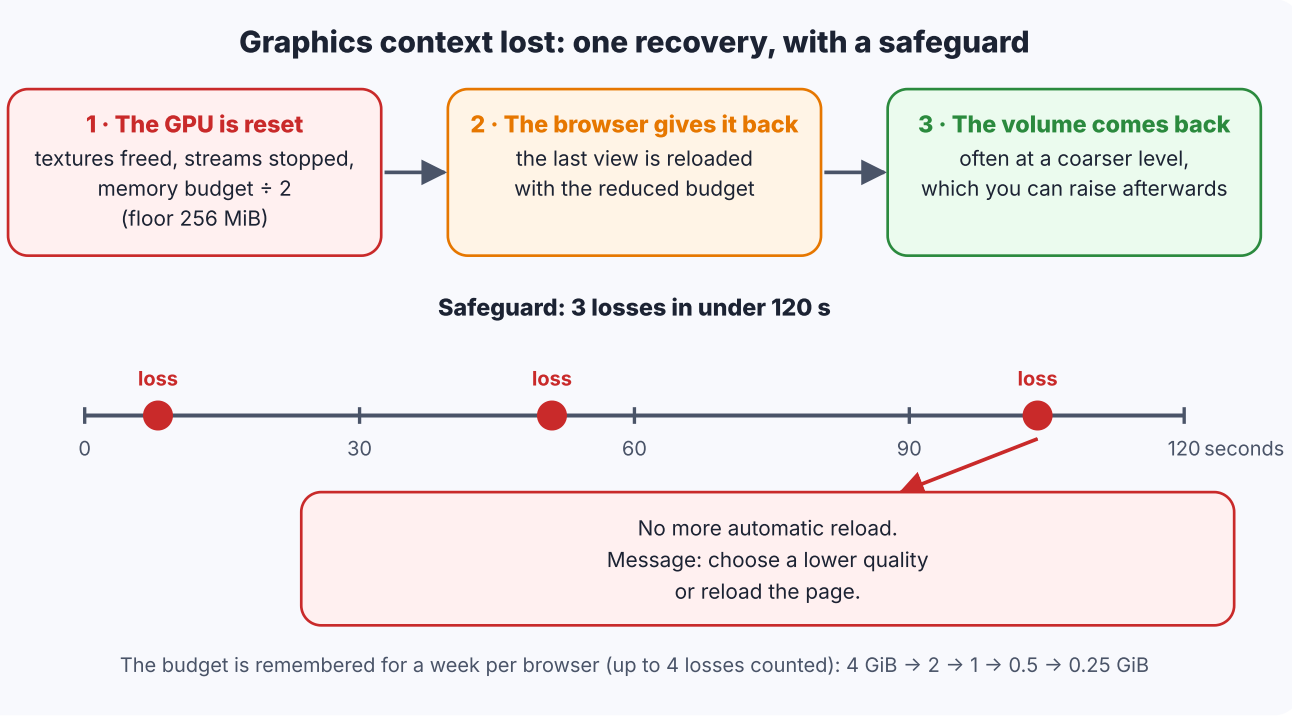
| IF... | WHAT LUMEN3D DOES | WHAT YOU SEE OR MUST DO |
|---|---|---|
| **A Compare panel fails to open** | Its error ( `PANEL_ERROR` ) is shown on that panel alone, with a **Retry** button; saving and the link do not wait for that panel | The other panels carry on |
| **A plugin is unrecognised, modified or incompatible** | It is **not loaded** (quarantined); the other plugins are untouched | The button does not appear (chapters 15 and 19.4) |
| **A `#state=` link is too big or tampered with** | It is not applied (beyond 2 MiB of text, or 16 MiB once decompressed, it is refused); the page opens on its normal view | The link restores nothing; the page works |
| **Browser storage is blocked (private browsing)** | Every read or write is protected: the page works, just without memory | Theme, language and settings are not remembered |

🔧 **FOR THE CURIOUS**

The ceilings of **2 MiB** of text and **16 MiB** decompressed, like the **64 KiB** written into the address, protect against a "decompression bomb": a small link that would expand into gigabytes. See appendix B for all the browser's memories.

## 19.10 The graphics card: full memory and lost context

Graphics-card memory is the only real hard ceiling. Lumen3D **plans it before** using it (chapter 10), but a card can still refuse, or reset in the middle of a frame.




▲ Loss of the graphics context: a single automatic reload, then a safeguard.

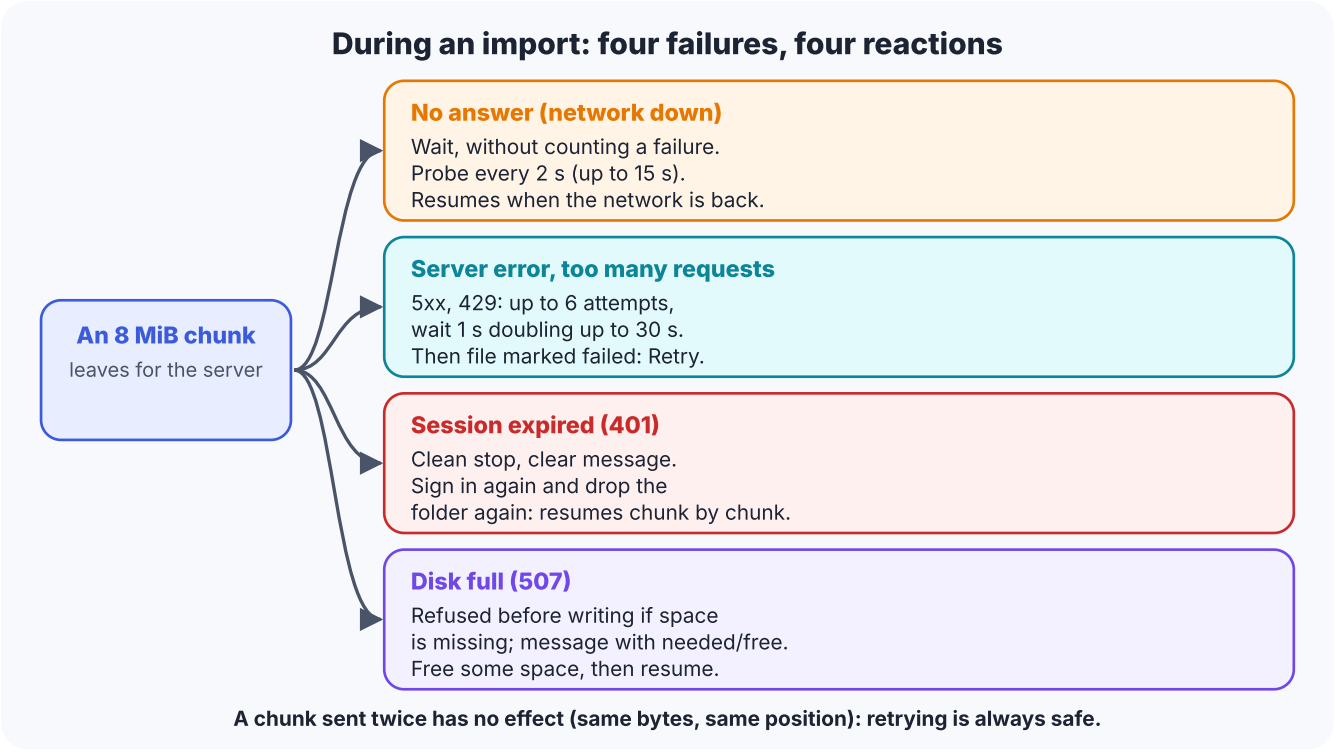
| IF... | WHAT LUMEN3D DOES | WHAT YOU SEE OR MUST DO |
|---|---|---|
| **The requested level exceeds the memory budget** | It is not allocated: refusal `SVR_OVER_BUDGET`, move on to the next **coarser** level | A status message says the level could not be allocated and the next one is being tried |
| **The card refuses an allocation despite the budget** | Refusal `SVR_ALLOC_FAILED`; the budget is capped **below the size that failed** for the whole session, then a coarser level | Same message; a lower quality is displayed |
| **No level fits** | The load is declared unavailable (insufficient memory at every level) | Close other tabs, or choose a lower quality |
| **The WebGL context is lost (driver, sleep, too much memory)** | Textures freed, streams stopped; the budget is **halved** (floor 256 MiB), remembered for a week | A status message: graphics context lost, waiting for it to return |
| **The context is restored** | The last view is reloaded with the reduced budget | A status message: context restored, reloading with a lower budget |
| **3 losses within 120 s** | The automatic reload **stops** (otherwise the loop would lock up the GPU) | Choose a lower quality or reload the page |
| **A resting frame takes too long (over 200 ms)** | The ceiling of samples per ray is reduced (chapter 9); a context loss halves it | The image is slightly less fine, the view stays smooth |
| **You want to force a budget** | Operator setting: the browser key `lumen3d.vramBudgetMB` replaces the estimate | Appendix B |

❓ **WHY THIS CHOICE?**

**Why halve the budget?** A context loss often comes from a driver that cuts off an overlong frame, or from saturated memory. Reloading with the same request would repeat the same mistake: so we ask for **less** next time.

## 19.11 Imports, conversions and metadata

An import of several gigabytes takes hours: an interruption is the rule, not the exception. Everything rests on a **journal** (chapter 17).




▲ During an import: four failures, four reactions.

| IF... | WHAT LUMEN3D DOES | WHAT YOU SEE OR MUST DO |
|---|---|---|
| **The network is cut during an import** | The transfer goes on hold (no failure counted), probes the network (2 s, then up to 15 s) and resumes by itself | The message “Connection lost. The transfer resumes by itself when the network returns.”, then the resumption |
| **The server answers 5xx, 429 or a wrong checksum** | Up to **6 attempts**, waiting 1 s doubled up to 30 s | If everything fails: file in error, **Retry** button |
| **The session expires during the import** | Clean stop, with the message “Session expired — sign in again, then drop the folder once more.” | Sign in again and drop the folder again: resumes **to the nearest chunk** |
| **The disk is full** | Before writing, the need is compared with the free space; on the way, error 507 with “needed / free” | Free some space, then resume |
| **You close the tab mid-import** | The server journal keeps the chunks received (binary map of chunks) | Drop the folder again: only the missing chunks start again |
| **A chunk is sent twice, or out of order** | Written at its exact position; sent again, it has no effect | Nothing |
| **A file is forbidden ( `.php` , `.js` , hidden file, path with `..` )** | Refused **before** anything is written, by the allowlist | The file is listed as refused |

| IF… | WHAT LUMEN3D DOES | WHAT YOU SEE OR MUST DO |
|---|---|---|
| **A folder of unknown type is dropped, or the announced type does not match the folder** | Refused (`metadata_type_mismatch`, `metadata_bad_type`); the editor says “invalid type” | Only `3d`, `2d` and `live` exist (chapter 17) |
| **A pack is truncated or missing** | The publication is **refused**: the brick index must point inside packs that are long enough (`incomplete_files`) | Finish the transfer, then publish |
| **An unexpected file lingers in the waiting area** | Publication refused (`stray_files`) rather than carrying it into the site | Delete it or redo the import |
| **`metadata.json` is missing, unreadable or invalid** | Publication refused with the list of errors (`missing_metadata`, `metadata_no_dimensions`, `metadata_bad_dimensions`…) | Fix it in the dataset editor |
| **A manifest announces a brick other than 64 or levels out of order** | Refused at import, by both servers | Run the pipeline again |
| **The source dataset changes during a conversion** | The job is invalidated (`source_changed`) rather than mixing two versions | Start the conversion again |
| **A conversion is interrupted (tab closed, server restarted)** | The journal and the tile store are shared by the browser and the server: it resumes at the same point, even with the other executor | Reopen the **Data updates** tab |
| **The finalisation of a conversion is interrupted** | It can be resumed (`complete:false` until it is done), then replaces the old content in a single move | Run it again: it finishes |
| **A unit kills the request twice (time-limited host)** | Answer `unit_timeout`: that conversion switches to the browser; the other units keep their progress | Nothing: the work carries on |
| **The host stops answering or refuses (429, 503)** | The request governor halves its pace and pauses (5 s, doubled up to 60 s) | A slower conversion, never a flood |
| **The server cannot decode WebP or the browser cannot encode it losslessly** | A capability probe detects it; the other executor is offered | A message in the tab |
| **The pipeline is interrupted in the middle of a publication** | The `.swap-in-progress` marker lets the next launch **finish or undo** the swap | Run the pipeline again |
| **A dataset is republished over an existing one** | The laboratory’s settings (hidden, orientation, gallery, corrected stage…) are **kept** | Nothing to redo |

🔢 WORKED EXAMPLE

**A 20-minute outage.** You are importing an 8 GB embryo. At 60 %, the Wi-Fi drops. The transfer goes “offline”, probes every 2 to 15 s, then resumes by itself when the network returns: **no file is lost** and no attempt is counted. Had you shut the computer, it would be enough to drop the folder again.

## 19.12 Server: updates, access and hosting limits

| IF… | WHAT LUMEN3D DOES | WHAT YOU SEE OR MUST DO |
|---|---|---|
| **The new version does not start (Python server)** | Dry-run check before the switch-over; after it, `/api/health` probe (about 30 s); on failure: **automatic rollback** to the old version | A “cancelled” state in the Updates tab |
| **The server stops during the switch-over** | A journal remains; at the next start, it is either completed (target version reached) or **entirely undone** | Restart the server |
| **The update fails on PHP hosting** | **No automatic rollback**: the copy can be replayed without damage, a busy file is set aside ( `.lumen-new` ) then finished at the next request | Run the update again |
| **Two administrators start the update together** | The second request is refused ( `update_in_progress` ) | Wait for the first to finish |
| **The download is truncated** | The announced size is checked, and the SHA-256 sum too: error and abandon | Run it again |
| **Space is short for the update** | The prior check (same volume, disk space, backup folder) refuses before touching anything | Free some space |
| **The version is not signed, or the signature is wrong** | It is **refused**; only the archive named after the version and listed in the signed list is applied | A refusal ( `unverifiable_release` ) whose reason is shown in the Updates tab |
| **The plugin catalogue is older than the last one seen** | Refused (decreasing serial number: `catalog_rollback` ) | No plugin is installed from that catalogue |
| **A plugin becomes incompatible after an update** | It is set aside and re-enabled when a compatible version exists | See the Plugins tab |
| **You type ten wrong passwords** | 15-minute block for that address; failures are counted before the check | Wait, or use the right password |
| **Your administration session expires (8 h)** | The form’s unsaved changes are **kept** on screen | Sign in again, then save |

| IF... | WHAT LUMEN3D DOES | WHAT YOU SEE OR MUST DO |
|---|---|---|
| **A visit counter is flooded with requests** | Two token buckets (per address and global) ignore the excess; a GET does not count | Nothing |
| **The host limits PHP's time or memory** | Each conversion step sets its own time limit; big units are cut into 8 octants; request bodies are capped (32 MiB per unit) | A slower conversion, never a corrupt one |
| **No Internet** | Everything works: the libraries are local; only the fonts (falling back to system ones) and the administrator's GitHub checks need it | Updates and catalogue unavailable |

> ⚠️ **WARNING**
>
> Fallbacks **protect**, they do not **repair** everything. A pack erased from the disk, a metadata file edited by hand, or hosting with no free space call for human action: the platform refuses cleanly and tells you why, that is all.

## 19.13 What to remember

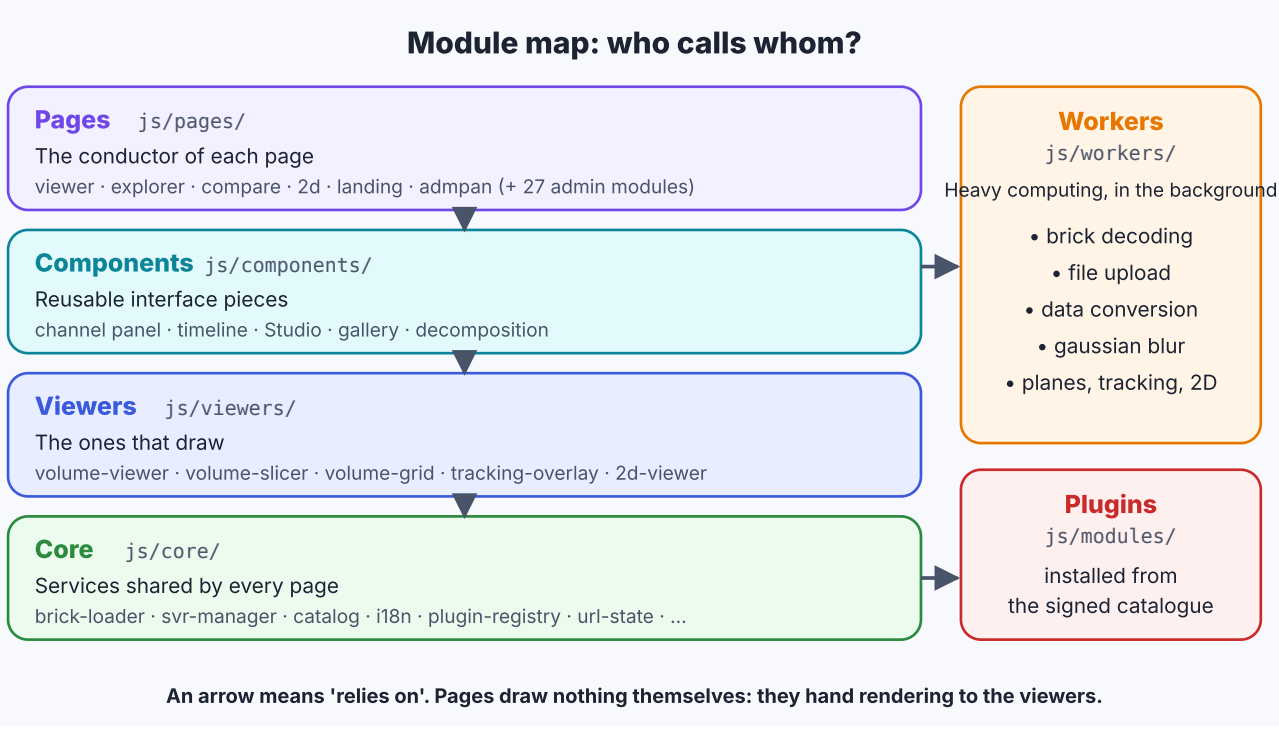
> 📌 **REMEMBER**
>
> - **Image**: a doubtful brick is set aside, never displayed wrongly; the graphics card is spared before it is called upon.
> - **Data**: we publish in full or not at all; long operations **resume**.
> - **Server**: Python returns by itself to the old version; PHP can be rerun safely, but by hand.
> - **No interruption** (network, tab, session) loses an import already begun.

# 20. Appendices

- Five **reference** appendices: to be looked things up in, not read in one go.
- **A**: what is where in the code. **B**: what your browser remembers, and the address parameters. **C**: what Lumen3D does not do.
- **D**: all the important figures in one place, with the chapter that explains each. **E**: the history of the versions.

## 20.A Module map

Lumen3D is written in “plain” JavaScript: no framework, no compilation. Each file has **one role** and a name that says it. This appendix is the index, one file per line.




▲ Five families of files. Pages orchestrate, viewers draw, the core lends a hand, workers compute.

💡 A PICTURE TO UNDERSTAND

**A restaurant.** The *pages* are the waiters who take your order. The *viewers* are the cooks who draw the dish. The *core* is the pantry, the knives and the work surface. The *workers* are the washing-up, backstage: you do not see it, but nothing would leave the kitchen without it.

## The core: `js/core/` (33 files)

| FILE | WHAT IT DOES, IN PLAIN WORDS | CH. |
|---|---|---|
| `brick-loader.js` | fetches bricks from the packs, in batches, without downloading the same thing twice | 10 |
| `brick-decode-worker.js` | worker: decodes the WebP images and cuts the mosaics back into bricks | 10 |
| `svr-manager.js` | manages the 3D atlas in graphics memory and its memory budget | 10 |
| `cut-store.js` | reads the format-5 cut copies ( `xz/` , `yz/` , `cuts/` ) for the Studio | 7, 12 |
| `data-access.js` | `ctx.data` : the exact, lossless voxels for tools that measure | 15 |
| `catalog.js` | loads the list of datasets, filters it, searches it, finds related datasets | 3 |
| `utils.js` | common toolbox: the type vocabulary (3d, 2d, live), dates, 1-2-5 scale bars | 1 |
| `tool-manager.js` | knows which tool is active (navigate, slice, measure) and handles keyboard shortcuts | 12 |
| `volume-source-manager.js` | normalises the description of a dataset's "sources" (bricks, time series) | 10 |
| `url-state.js` | writes and reads the view state in the address ( `#state=` ) | 12 |
| `workspace-state.js` | saves and restores the workspace in the browser | 12 |
| `measurement-store.js` | keeps the distances measured during the session in memory | 12 |
| `export-manager.js` | engine of the Download Center: workspace, measurements, metadata, citation | 12 |
| `display-presets.js` | backgrounds of the 3D display: dark, light, paper, transparent, custom | 9 |
| `slice-compositor.js` | recolours a slice from its raw values, for the Studio | 12 |
| `plane-loader.js` / `plane-codec.js` | read the XY planes of format 2 and define their PNG format | 7, 17 |

| FILE | WHAT IT DOES, IN PLAIN WORDS | CH. |
|---|---|---|
| `plugin-registry.js` | discovers, loads and plugs in the plugins; builds the toolbar | 15 |
| `plugin-trust.js` | checks a plugin's fingerprint and trust level | 15 |
| `plugin-sandbox.js` | the cage (isolated iframe) of approved third-party plugins | 15 |
| `compat.js` | says whether a plugin is compatible with the platform version | 15 |
| `i18n.js` | translations: English, French, Spanish, Dutch and added languages | 16 |
| `instance-config.js` | reads the site configuration (name, specimen, texts): the "white label" | 16 |
| `theme.js` / `theme-boot.js` | light/dark switch; `theme-boot` applies it before the first paint | 16 |
| `colorblind.js` | eight colour-blindness simulations, applied to the whole page | 16 |
| `page-renderer.js` | draws a page built in the editor (sections, columns, widgets) | 16 |
| `page-templates.js` | default content of the About page | 16 |
| `page-edit-frame.js` | live editing mode of the page editor | 16 |
| `page-vars.js` | variables in page texts (year, date, number of datasets…) | 16 |
| `page-background.js` | animated backgrounds of built pages | 16 |
| `dialog.js` | small choice windows (Confirm / Cancel); Esc = cancel | 16 |
| `ui-actions.js` | handles clicks on the header buttons (the security policy forbids handlers in the HTML) | 19 |
| `font-loader.js` | switches on the Google fonts without delaying the first paint | 19 |
| `perf-telemetry.js` | diagnostic timers, kept locally, never sent | 10 |

### The viewers: `js/viewers/` (5 files)

| FILE | WHAT IT DOES, IN PLAIN WORDS | CH. |
|---|---|---|
| `volume-viewer.js` | the heart: the 3D scene, ray marching, the camera, captures | 9 |
| `volume-slicer.js` | computes a plane slice (straight or oblique) in the volume | 12 |
| `volume-grid.js` | reference grid, axes gizmo and 3D scale bar | 12 |
| `tracking-overlay.js` | draws the tracked cells of a time-lapse over the volume | 12 |
| `2d-viewer.js` | draws a 2D photograph (zoom, pan, scale) | 12 |

### The components: `js/components/` (5 files)

| FILE | WHAT IT DOES, IN PLAIN WORDS | CH. |
|---|---|---|
| `channel-panel.js` | the channel panel: colour, window, gamma, opacity | 11 |
| `decomposition-panel.js` | the “Decompose by channel” interface | 12 |
| `studio-editor.js` | the Studio: annotate and export a figure | 12 |
| `timeline.js` | the time strip and the series player | 12 |
| `dataset-gallery.js` | the gallery of images attached to a dataset, with its enlargement | 12 |

### The workers: `js/workers/` (9 files)

A worker is a background employee: it works without blocking the screen (chapter 2).

| FILE | WHAT IT DOES, IN PLAIN WORDS | CH. |
|---|---|---|
| `gaussian-blur-worker.js` | Gaussian blur of a channel, plane by plane | 11 |
| `plane-decode-worker.js` | decodes the PNG tiles of the XY planes | 7 |
| `studio-plane-worker.js` / `studio-plane-ops.js` | maximum per channel over a thickness; the pure calculations are in `ops` | 12 |
| `pixel-2d-worker.js` / `pixel-ops-2d.js` | display adjustments and staining isolation of a photograph; pure calculations in `ops` | 12 |

| FILE | WHAT IT DOES, IN PLAIN WORDS | CH. |
|---|---|---|
| `tracks-load-worker.js` | prepares the cell-tracking tables (32-bit integers) | 12 |
| `upload-worker.js` | sends the files of an import: reading, fingerprint, sending, retries | 17 |
| `migration-worker.js` | runs the units of a format conversion on the browser side | 17 |

## The pages: `js/pages/` (10 files)

| FILE | WHAT IT DOES, IN PLAIN WORDS | CH. |
|---|---|---|
| `viewer.js` | the main page: assembles everything for a 3D volume or a series | 3 |
| `2d.js` | the page for 2D photographs | 12 |
| `explorer.js` | the grid of datasets, its filters and the search | 3 |
| `compare.js` | the Compare page: up to four synchronised panels | 12 |
| `compare-policy.js` | chooses each panel's quality so as to fit a shared budget | 12 |
| `landing.js` | the home page | 3 |
| `about.js` / `legal.js` / `page-view.js` | About, Legal, custom pages | 3, 16 |
| `admpan.js` | the administration page (starts the `shell` module) | 14 |

## The administration: `js/pages/admin/` (27 files)

| FILE | WHAT IT DOES, IN PLAIN WORDS | CH. |
|---|---|---|
| `shell.js` / `shared.js` / `bus.js` | the frame (menu, theme, login), the common tools (requests, messages) and the messaging between tabs | 14 |
| `tab-datasets.js` | the **Datasets** tab: list, editor, orientation, gallery | 14 |
| `tab-upload.js`, `upload-manager.js`, `upload-dock.js` | the **Import** tab, its orchestrator and the floating badge | 17 |
| `tab-dataset-updates.js`, `migration-runner.js` | **Data updates** and its engine (including the request governor) | 17 |

| FILE | WHAT IT DOES, IN PLAIN WORDS | CH. |
|---|---|---|
| `tab-dataset-types.js` | **Data types**: rename 3d, 2d, live | 16 |
| `tab-branding.js`, `tab-appearance.js`, `tab-legal.js` | **Identity**, **Appearance**, **Legal** | 16 |
| `tab-pages.js` + `pages-controls.js`, `pages-translate.js`, `pages-variables.js` | the page editor and its panels | 16 |
| `tab-plugins.js`, `tab-marketplace.js`, `plugin-update.js` | **Plugins**, **Catalog** and the common plugin update rule | 15 |
| `tab-updates.js`, `tab-changelog.js`, `markdown.js` | **Updates**, the release-notes page, the Markdown reader | 18 |
| `tab-pipeline.js`, `tab-docs.js`, `tab-security.js`, `tab-stats.js` | **Pipeline**, **Documentation**, **Security**, **Statistics** | 14 |

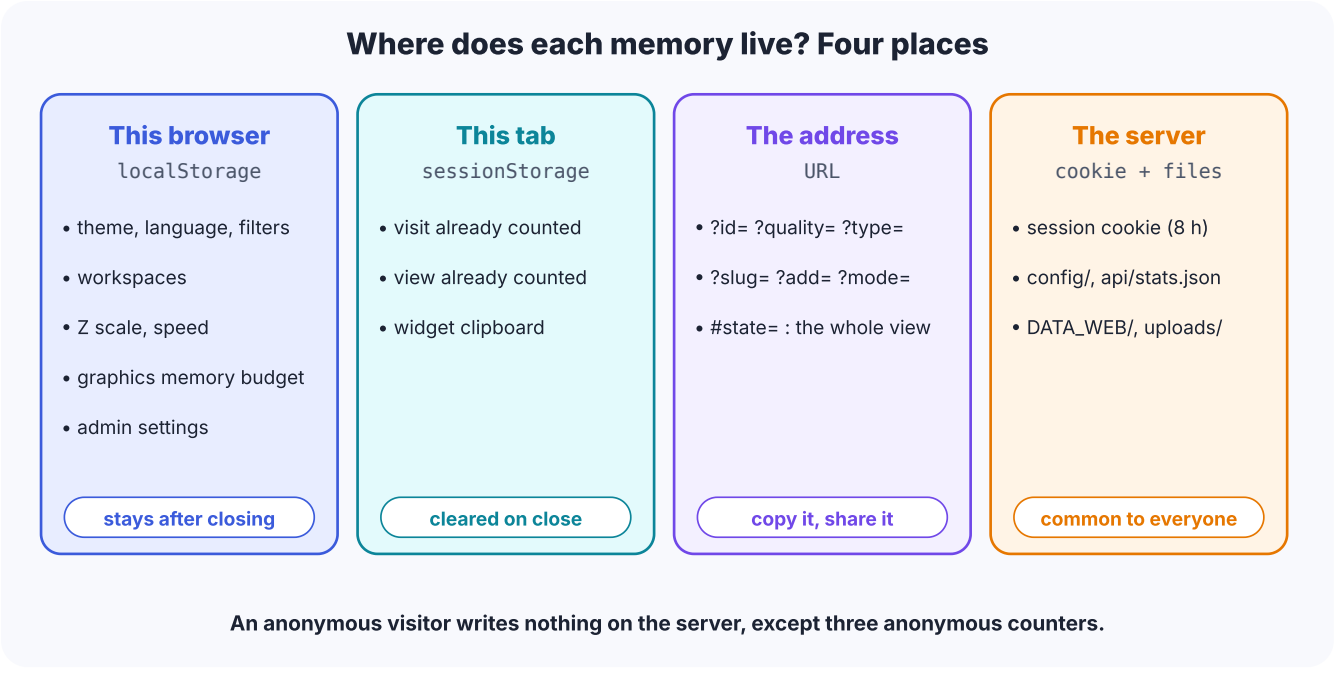
## Outside `js/`

| FILE | WHAT IT DOES, IN PLAIN WORDS | CH. |
|---|---|---|
| `dev_server.py` | the Python server: serves the pages and also does the administration work | 2, 18 |
| `api/*.php` and `router.php` | the twin server in PHP, for hosting without Python | 2, 18 |
| `fast_server.py` | a fast static server, with no administration (performance tests) | 2 |
| `upload_staging.py` | the import engine (allowlist, journal, validation, publication) | 17 |
| `dataset_migrations.py` | the data-format conversions, Python side | 17 |
| `format5.py` | the shared format-5 engine (levels, compressions, mosaics, packs, cut copies), used by the migration and by the pipeline | 7, 17 |
| `tools/convert_dataset.py` | convert a dataset to the latest format on your own computer | 17 |
| `ed25519_pure.py` | Ed25519 signature verifier, in pure Python | 19 |
| `js/migrations/` | the three conversions (`m002`, `m003`, `m004`) on the browser side, and a speed-test block | 17 |

| FILE | WHAT IT DOES, IN PLAIN WORDS | CH. |
|---|---|---|
| `js/modules/` | the plugins: `tools/` (toolbar), `channels/` (channel panel), `shaders/` (render modes) | 12, 15 |
| `js/vendor/` | the third-party libraries, hosted locally (Three.js, Lucide, Plotly) | 2 |
| `preprocess/` | the preparation pipeline, in Python | 4 to 8 |
| `tests/`, `tools/` | the test suite (201 files) and the publishing tools | 18 |

# 20.B Browser memories and address parameters

A browser offers several “drawers” where a site can put small things. Lumen3D uses **four** and never stores study data in them.




▲ Four places where a piece of information lives.

> ℹ️ **GOOD TO KNOW**
>
> Every access to these drawers is protected: in private browsing, or if storage is blocked, the page works, just without memory (chapter 19). Clearing the site data erases everything below without breaking anything.

## B.1 What this browser remembers ( `localStorage` )

| KEY | WHAT IT KEEPS |
|---|---|
| `iribhm-theme` | the chosen theme, light or dark |
| `iribhm-lang` | the interface language |
| `iribhm-colorblind` | the colour-blindness simulation filter |
| `iribhm.workspace.<scope>.<id>` | the saved workspace of a dataset (camera, channels, tools), as JSON; `<id>` looks like `3d/Dataset-name` |
| `iribhm.viewer.zScale.<id>` | the display stretch in Z, dataset by dataset |
| `iribhm.viewer.playbackFps` | the preferred playback speed for time series (one for the whole viewer) |
| `iribhm-gallery-dock` | the state of the gallery: open, wide or reduced to a badge |
| `lumen3d.gpuContextLosses` | the dates of the last graphics-context losses (8 at most): each loss in the week halves the budget |

| KEY | WHAT IT KEEPS |
|---|---|
| `lumen3d.vramBudgetMB` | **operator setting**: imposes a graphics-memory budget, in MB. Never written by the site (chapter 10) |
| `adm-theme` , `adm-sidebar-collapsed` | theme and collapsed menu of the administration panel |
| `adm-upload-dock-size` | size of the import badge |
| `lumen-dupd-ds-executor` , `lumen-dupd-speedtest` , `lumen-dupd-log` | data conversions: executor chosen per dataset, last speed test, log of the last 20 operations |

> 👉 **TIP**
>
> To bring an over-cautious graphics budget (after several context losses) back to normal, delete `lumen3d.gpuContextLosses` in the browser's tools, or wait a week.

## B.2 What this tab remembers ( `sessionStorage` ) and the cookie

| NAME | ROLE |
|---|---|
| `lumen_visit` | "the visit has already been counted": one visit counter per session, not per page |
| `lumen_view_<id>` | "this dataset's view has already been counted" (administration previews do not count) |
| `lumenWidgetClip` | clipboard of a widget copied in the page editor |
| cookie `admpan_token` | the administration session: 8 hours, invisible to scripts ( `HttpOnly` ). Visitors do not receive one |

## B.3 Address parameters

They go after `?` (for example `viewer.html?id=3d/Embryo-E95-Em2-Pecam1-Sox2&quality=high` ).

| PAGE | PARAMETER | MEANING |
|---|---|---|
| `viewer.html` , `2d.html` | `id=<type>/<folder>` | the dataset to open |
| `viewer.html` | `quality=` | `preview` (256), `balanced` (512), `high` (1024), `2048x2048` , `4096x4096` or `native` ; the forms `512x512` , `1024x1024` , `low` , `medium` are accepted |
| `viewer.html` , `2d.html` | `hideHeader=true` | "no banner" page: this is how a Compare panel embeds it |
| same | `panelIndex=<n>` | number of the panel in Compare |
| same | `mode=admin` | administration preview: the view is **not** counted in the statistics |
| same | `path=<type>/<folder>` or `staging:<type>/<folder>` | with `mode=admin` : open a dataset not yet published |

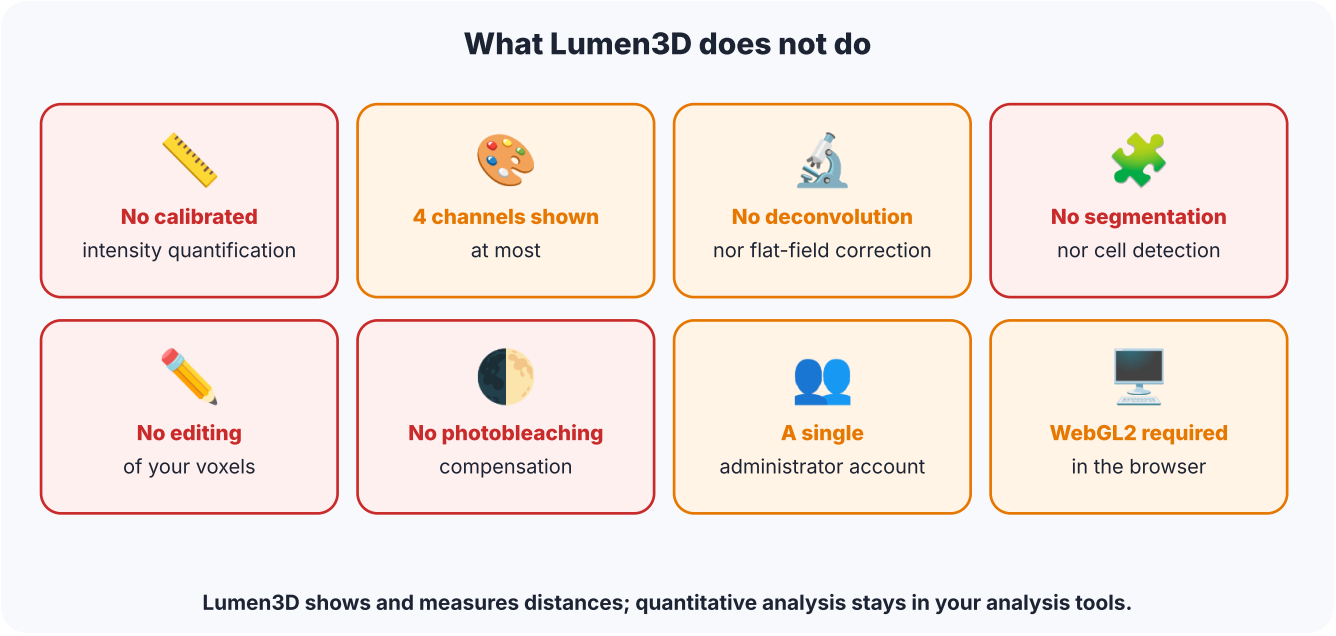
| PAGE | PARAMETER | MEANING |
| --- | --- | --- |
| `explorer.html` | `type=3d`, `2d` or `live` | pre-ticks the type filter |
| `compare.html` | `add=<id>` (repeatable) | datasets to put in the panels (four at most) |
| `page.html` | `slug=<name>` | the custom page to display |
| `page.html`, `about.html`, `index.html` | `preview=draft` | shows the draft rather than the published version |
| `page.html` | `edit=1` | editing mode (used inside the page editor) |
| `admpan.html` | `editor=<slug>` | opens the page editor alone on that page |
| `admpan.html` | `changelog=1` | the release-notes page, without the panel |
| all view pages | `#state=…` | **the complete view**, compressed; written by itself when you change something, at most once a second |

🔧 **FOR THE CURIOUS**

`#state=` is a JSON compressed (deflate) then written in base64. It can contain camera, channels, tools, measurements and plugin state. Ceilings: 64 KiB written into the address, 2 MiB of text read, 16 MiB once decompressed (protection against “bombs”). A link that is too big or tampered with is ignored. The parameters of a page embedded in Compare are set by Compare itself.

## 20.C What Lumen3D does not do

A good tool says what it cannot do. Here are the **deliberate** limits, checked in the code.




▲ Eight things Lumen3D does not do.

| LIMIT | WHY, AND WHAT TO DO |
|---|---|
| **No calibrated intensity quantification** | The pipeline brings 12 or 16 bits down to 0–255 between an estimated background and signal; the grey levels shown are **relative** (chapters 5 and 11). To compare intensities between samples, go back to the original Imaris file |
| **No objects thinner than about 3 voxels** | The cleaning removes isolated points and then smooths the background with a median: a point, a thread or a sheet one or two voxels thick is erased, even when bright (chapter 5, § 5.10). Check in the original Imaris file |
| **4 channels displayed at most** | Volumes are loaded into a four-component texture (red, green, blue, alpha). The pipeline writes all the channels; the viewer draws the first four (chapter 11) |
| **No deconvolution, no flat-field correction** | The pipeline subtracts a background and sets a window; it corrects neither the lens blur (point spread function) nor uneven illumination |
| **No segmentation, no cell detection** | Cell tracking is **read** from the analysis done in Imaris, never recomputed (chapter 8) |
| **No editing of your voxels** | The viewer reads the data and does not modify it; blur, window or gamma are **display** settings. The only change of value is the pipeline's 16 → 8-bit reduction (chapter 6) |

| LIMIT | WHY, AND WHAT TO DO |
|---|---|
| **No photobleaching compensation** | The pipeline **measures** the level per image, but no viewer setting uses it: a fading series is displayed darker (chapter 5) |
| **Measurements not saved on the server** | They live in memory during the session and in the link or workspace you save (chapter 12) |
| **No region statistics** | Distances, slices, figures: yes. Volume of a structure or mean intensity of a zone: no |
| **A single administration account** | The public has no account; the administrator is unique, protected by a password (chapter 19) |
| **WebGL2 required** | Without 3D textures, bricks cannot be displayed: a message explains it (chapter 19). On a phone, graphics memory limits quality |
| **Inputs: Imaris `.ims` files and TIFF photographs** | Other microscope formats must be converted first; photographs expect the TIFF exported by ImageJ/Fiji from a Leica file (chapter 4) |
| **No 2D image pyramid** | A photograph is a single native WebP file with a preview; the old tiled viewer was removed |

⚠️ **WARNING**

“Does not do it” does not mean “does it badly”. If your scientific question needs one of these functions, do it in the appropriate analysis tool, then use Lumen3D to **show** the result.

# 20.D Key figures of the platform

All the important constants, gathered. The last column says where the idea is explained.

## D.1 The data

| CONSTANT | VALUE | CH. |
|---|---|---|
| **Brick** | $64^3$ useful voxels; $66^3$ stored with a 1-voxel border (formats 4 and 5) | 7 |
| **Mosaic of a brick** | 9 × 8 slices of $66^2$: a lossless WebP image of 594 × 528 px | 7 |
| **Pack** | at most 64 bricks or 16 MiB, in super-blocks of 4 × 4 × 4 bricks | 7 |
| **Levels of detail** | halved in X and Y; in Z only while vz ≤ 1.5 × vxy | 6, 7 |
| **Current data format** | 4 (formats 1 to 3 are converted in place) | 7, 17 |
| **XY plane (format 2)** | one file per plane; 512 × 512 PNG tiles | 7, 17 |
| **Layer projection (format 3)** | one maximum-intensity projection per layer of 64 planes | 7 |
| **Maximum size accepted** | $2^{20}$ voxels per axis; 64 channels at most accepted, **4 displayed** | 19 |
| **Depth reduction** | 16 or 12 bits → 8 bits (the pipeline's only loss) | 5, 6 |

## D.2 Background cleaning (pipeline)

| CONSTANT | VALUE | CH. |
|---|---|---|
| **Background floor** `bg_floor` | 99th percentile of the 8 corner cubes | 5 |
| **Saturation** `sig_max` | 99.9th percentile (of one voxel in 4) | 5 |
| **Mask threshold** | above 1.1 × `bg_floor` | 5 |
| **Mask cleaning** | opening, then dilation × 3 | 5 |
| **Time series** | `sig_max` computed only on the voxels above the background, if there are at least 1,000 | 5 |

## D.3 Streaming and graphics memory

| CONSTANT | VALUE | CH. |
|---|---|---|
| **Simultaneous brick loads** | 24 | 10 |

| CONSTANT | VALUE | CH. |
|---|---|---|
| **Simultaneous pack bodies** | 4 | 10 |
| **Compressed pack memory kept** | 192 MiB (packs) · 64 MiB (ranges) · 32 MiB (prefetch) | 10 |
| **Look-ahead** | 64 MiB ahead of the decoded brick | 10 |
| **Time-outs** | 30 s without an answer (download) · 30 s (decoding) | 10, 19 |
| **Attempts per brick** | 3, waiting 0.5 s then 1 s | 19 |
| **Worker replacements** | 3 at most, then decoding in the page | 19 |
| **Quality 512** | finest level whose longest XY side is ≤ 768 voxels | 10 |
| **Quality 1024** | same, ≤ 1,536 voxels | 10 |
| **Atlas** | up to 8 pages | 10 |
| **Graphics budget: software rendering** | 256 MiB | 10 |
| **Graphics budget: integrated card** | 0.5 · 1 · 2 GiB depending on the device's memory | 10 |
| **Graphics budget: discrete card** | 3 or 4 GiB | 10 |
| **Graphics budget: unknown** | 1 or 2 GiB | 10 |
| **Graphics context loss** | budget ÷ 2 per loss in the week (floor 256 MiB) | 19 |
| **Safeguard** | 3 losses within 120 s: no more automatic reload | 19 |
| **Compare** | 4 panels at most; shared budget 1.5 GiB; 180 s for a panel to settle | 12 |

## D.4 Rendering

| CONSTANT | VALUE | CH. |
|---|---|---|
| **Samples per ray (at rest)** | `clamp(6·10⁹ / pixels, 256, 4096)` | 9 |
| **Target frame** | 16.7 ms (60 frames/s); we degrade beyond 1.3×, and climb back below 1.1× | 9 |
| **Resting frame too slow** | beyond 200 ms, the sample ceiling is reduced | 9 |

| CONSTANT | VALUE | CH. |
|---|---|---|
| **Windows safeguard (TDR)** | a graphics computation of more than about 2 s resets the card | 9 |

## D.5 Security and access

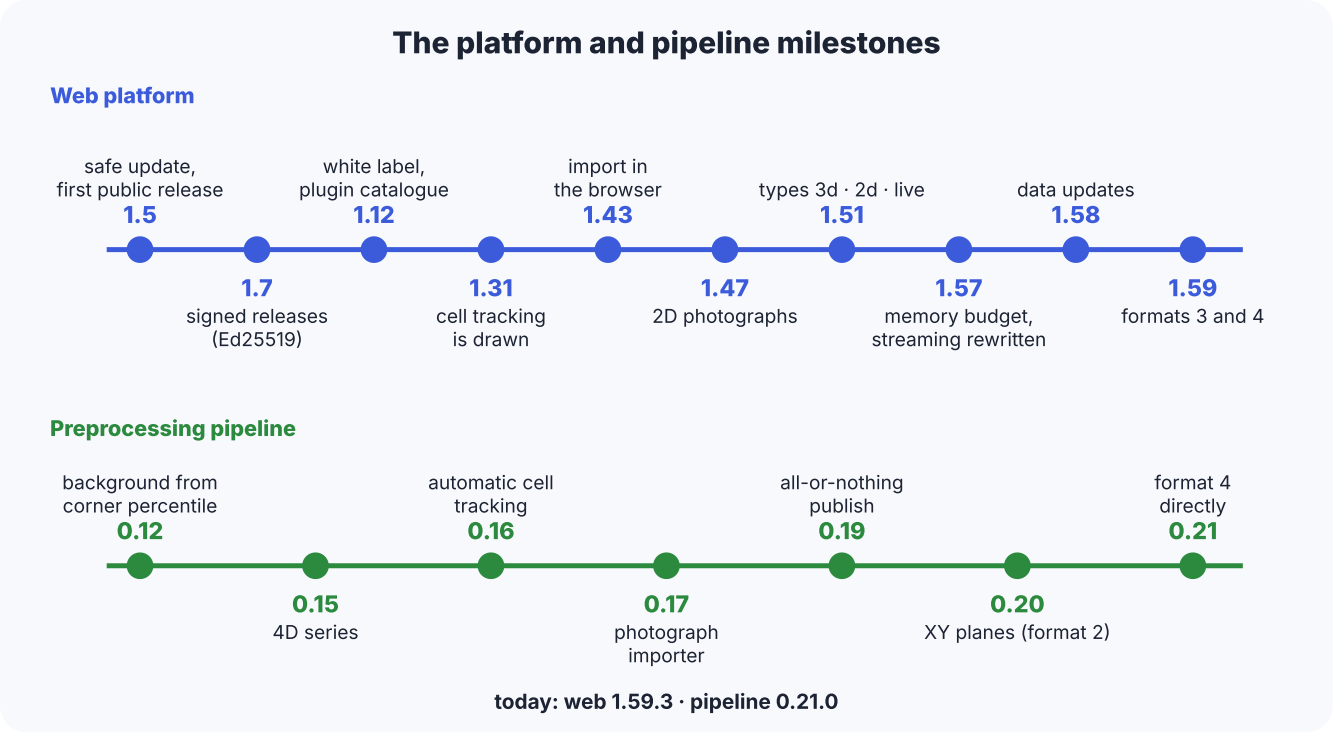
| CONSTANT | VALUE | CH. |
|---|---|---|
| **Password** | at least 8 characters; PBKDF2-HMAC-SHA256, 600,000 rounds | 19 |
| **Login failures** | 10 in 15 minutes from one address → 15-minute block; global ceiling of 200 per window | 19 |
| **Administration session** | 8 hours; closed at every password change | 19 |
| `#state=` **link** | 64 KiB written · 2 MiB read · 16 MiB decompressed | 12, 20.B |
| **A dataset's gallery** | 40 images, 8 MiB each | 14 |
| **Visit counters** | per address: 60 in a burst then 1 per second; global: 600 then 20 per second; 4,096 slots | 19 |

## D.6 Import, conversions and updates

| CONSTANT | VALUE | CH. |
|---|---|---|
| **Import chunk** | 8 MiB (between 256 KiB and 16 MiB); 4 sends in parallel | 17 |
| **Attempts per chunk** | 6, wait of 1 s doubled up to 30 s; network probe 2 s to 15 s | 19 |
| **Import request time-out** | at least 60 s (down to 50 KB/s at the floor) | 17 |
| **Request governor (conversions)** | 6 in flight, 0.5 to 6 per second (starting at 4), pause of 5 s to 60 s | 17 |
| **Bricks read per request** | 128 | 17 |
| **Body of a conversion unit** | 32 MiB at most (PHP hosts) | 17 |
| **Conversion unit** | a layer of 64 planes × one channel × one 512² tile (8 × 8 bricks) | 17 |
| **Health probe after update (Python)** | about 30 s | 18 |
| **Plugins in the signed catalogue** | 28 | 15 |
| **Test files** | 201 (one process per file) | 18 |

# 20.E Versions and history

The web platform and the preparation pipeline are two pieces of software, with **two independent numbers**. Each version has a note, `changelog/changelog_X.Y.Z.md` for the platform and `preprocess/changelog/` for the pipeline.




▲ The main milestones of the platform and the pipeline.

> ℹ️ **GOOD TO KNOW**
>
> The platform version exists only in the **name of the latest notes file**: to move to a new version, a note is added. The Python server's version ( `dev_server.py` ) is a third number, separate and older: it tracks the server tool, not the platform.

## E.1 Web platform

| VERSION | WHAT IT BRINGS |
|---|---|
| 1.3 – 1.4 | Download Center as a file explorer; complete redesign of the administration panel |
| 1.5 | **First public release on GitHub**: safe update by folder swap with rollback, health probe, one-file installer, plugin ↔ platform compatibility |
| 1.6 | Isolation of third-party plugins: trust, sandbox, strict security policy, libraries hosted locally |
| 1.7 | Signed releases (Ed25519), security policy on PHP hosts too, sandbox hardening |

| VERSION | WHAT IT BRINGS |
|---|---|
| **1.8 – 1.13** | White label: site configuration, first-run wizard, signed plugin catalogue (1.12), sections → columns → widgets page editor (1.13) |
| **1.14** | Update of PHP hosts (1.14.2) |
| **1.25** | The About page becomes an editable page |
| **1.31** | Cell tracking is finally drawn in the viewer |
| **1.43 – 1.44** | Import of a dataset from the browser, in tiers and resumable; Pipeline pack downloadable like a release |
| **1.45 – 1.49** | Image gallery per dataset; 2D photograph viewer (1.47) |
| **1.51** | A single type vocabulary: `3d` , `2d` , `live` |
| **1.53 – 1.54** | Cell tracking becomes a layer of a series (1.53); Compare drives the real pages (1.54) |
| **1.55 – 1.56** | Camera poses and Z-stack browser; 3D view export, annotation rotation, channel recolouring in the Studio |
| **1.57** | "Audit wave": streaming rewritten in batches, graphics-memory budget, signed release and mandatory test suite at every publication |
| **1.58** | Data updates: in-place conversion of a dataset to format 2 |
| **1.59** | Formats 3 and 4 (v3 bricks with border, `index.bin` , detail on zoom); executor chosen per dataset and speed test (1.59.1); request governor (1.59.2); eight times fewer requests (1.59.3) |
| **1.60 – 1.62** | Graphics compression of timelapses (1.60); 60-day trash for deleted datasets (1.62); import / export of pages (1.61) |
| **1.63 – 1.65** | **Format 5**: the resolution × compression matrix, the cut copies, lossless access for tools ( `ctx.data` ), a "lossless" choice on every export (1.64); conversion in the browser on any host, import of a v5 tree, local conversion tool (1.65) |

## E.2 Preparation pipeline

| VERSION | WHAT IT BRINGS |
|---|---|
| **0.12 – 0.13** | Otsu threshold abandoned (0.12.0); background estimated by corner percentile and mask-based denoising (0.12.13 to 0.13.0) |
| **0.14** | Standalone one-file launcher |

| VERSION | WHAT IT BRINGS |
| --- | --- |
| **0.15 – 0.16** | True 4D series; cell tracking attached automatically |
| **0.17 – 0.18** | Photograph importer; new type vocabulary |
| **0.19** | “All or nothing” publication in a staging area, with recovery after interruption |
| **0.20** | XY planes written directly (format 2) |
| **0.21** | Format 4 written directly, byte-identical to the conversion; exact skipping of empty space |
| **0.22** | Format 5 on request ( `--format 5` ), byte-identical to the m005 conversion |

📌 **REMEMBER**

Current version of this document: platform **1.65.0**, pipeline **0.22.0**, data format **5**.

# 21. Glossary

- Terms are listed in **alphabetical order**. Each definition fits in one or two sentences.
- "→ chapter N" sends you to where the word is explained in detail.
- Names in square brackets, like **Studio**, are the ones you see on screen.

### Apron (border)

An extra voxel around a brick, copied from its neighbours, so that filtering leaves no seam between two bricks. Format 4 and 5 bricks are $66^3$ for $64^3$ of useful data. → chapter 7

### Atlas

A large 3D texture in graphics memory where the bricks to display are stored, like the pigeonholes of a locker. → chapter 10

### Atomic

Said of an operation that succeeds in full or not at all, with no intermediate state visible. Publishing a dataset is atomic. → chapters 8 and 17

### Background noise

Stray signal present even where there is nothing to see (scattered light, sensor electronics). → chapter 5

### Batch (of bricks)

A set of bricks requested together from the loader, which handles its cancellation and returns a summary (delivered, failed, skipped). → chapter 10

### Bit depth

The number of bits per value: 8 bits give 256 levels, 16 bits give 65,536. → chapter 4

### Blue-green (deployment)

An update method: the new version is prepared next to the old one, tested, then swapped in; the old one stays available for a rollback. → chapter 18

### Bounding box

The parallelepiped that surrounds the whole volume. A click that reaches only this box, with no visible structure beneath, is not a valid measurement. → chapter 12

### Brick

A small cube of 64×64×64 voxels: the unit in which a volume is cut up and loaded. → chapter 7

### Cache

Memory where what has already been downloaded is kept so that it is not requested again. → chapter 10

### Calibration (physical)

The real size of a voxel, in micrometres, in each direction. Without it, the scale bar and measurements do not exist. → chapters 4 and 9

### Catalogue

Two meanings to tell apart: the list of the site's datasets, generated on demand from the files, and the signed plugin catalogue, from which tools are installed. → chapters 3 and 15

### Cell tracking

Following each cell from frame to frame in a time-lapse (trajectories, lineages, divisions). → chapter 8

### Channel

One image of the same object for a given fluorescence colour (for example DAPI, Pecam1, Sox2). → chapter 4

### Column (compression)

In format 5, one of the five ways to store a level: lossless, ±2, ±4, light (WebP 80), very light (WebP 50). Chosen in the quality matrix. → chapters 7 and 10

### Confocal (microscope)

A microscope that records only the light coming from a very thin plane of the specimen, plane after plane; the stack of planes forms the volume. → chapter 4

### Context (page or panel)

The situation a page runs in: alone, or embedded as a panel (Compare, split view). A plugin loads in a panel only if it has declared it can. → chapter 15

### CSP (Content Security Policy)

A rule given to the browser: run only scripts that carry a single-use code (nonce) and come from the site itself. → chapter 19

### CSRF (anti-forgery token)

An attack in which a third-party site makes a request without your knowledge, from your session. A secret token, required for every change, prevents it. → chapter 19

### Cut copies

In format 5, three re-cut copies of the volume (XY planes, XZ / YZ rows, small oblique bricks) that make the Studio's native cuts fast. → chapter 7

### DAPI, Pecam1, Sox2

The three channels of the demonstration dataset: DAPI stains DNA (the nuclei), Pecam1 marks the vessels, Sox2 the neural tissue. → chapter 4

### Data format

The way a dataset is laid out on disk; numbered 1 to 5. The current format is 5. → chapters 7 and 17

### Dead zone

In format 5, a threshold (0 to 12) below which a lossy-compressed brick puts small values back to 0, so the background stays black; written in one byte at the end of the brick. → chapter 7

### Deconvolution

A calculation that tries to undo the blur of the optics to sharpen the image. Lumen3D does not do it. → chapter 20

### Dilation

An operation that thickens the areas of a mask by one or more pixels. → chapter 5

### Ed25519 (signature)

A digital signature method: a "wax seal" that only the publisher can apply and that anyone can verify. → chapter 19

### Embryonic day (E8.5)

The age of a mouse embryo: E8.5 means eight and a half days after fertilisation. The platform reads the stage from the file name (E8-5 means E8.5). → chapter 8

### ESS (Empty Space Skipping)

Bricks whose voxels are all 0 are neither stored nor traversed, which saves bytes and computation. In formats 4 and 5 the rule is exact (a single non-zero voxel is enough to keep the brick); before, an occupancy threshold of 0.05 % also discarded nearly empty bricks. → chapters 7 and 9

### Executor

Whoever does the work of a data conversion: the browser or the server. It can change along the way, the journal is shared. → chapter 17

### Exposure

A global brightness setting applied to the display; it does not alter the data. → chapter 11

### Finalisation

The last step of a data conversion: the new files replace the old ones in a single move, then the format number is updated. → chapter 17

### Fingerprint (hash, SHA-256)

A short code computed from a file's content: if a single byte changes, the fingerprint changes. It is used to verify a download, a plugin, an import chunk. → chapters 15 and 19

### Fluorophore

A molecule that absorbs light and re-emits it in another colour; it is what labels a structure in the sample. One channel corresponds to one fluorophore. → chapter 4

### Gamma

A setting that brightens or darkens the mid-tones without touching the extremes. → chapter 11

### GPU

The graphics card's processor, specialised in parallel computing; it runs the ray marching. → chapter 9

### HDF5

A scientific file format that stores large arrays of numbers, like a filing cabinet with drawers. Imaris files are one. → chapter 4

### Health probe

A small request ( `/api/health` ) that checks that the server answers with the right version after an update. → chapter 18

### Histogram

A graph that counts how many voxels have each intensity value; it helps set min and max. → chapter 11

### i18n (internationalisation)

The mechanism that lets the interface be shown in several languages (English, French, Spanish, Dutch…) and lets more be added. → chapter 16

### Idempotent

Said of an operation that can be repeated without harm: the result is the same as doing it once. This is what makes resumptions safe. → chapter 17

### .ims file

An Imaris file: an HDF5 container that holds the channels, the resolution levels and the metadata of an acquisition. → chapter 4

### index.bin

A binary file of formats 4 and 5 (one per column in format 5) that says which pack holds each brick; it replaces the large JSON manifest. → chapter 7

### Intensity

The value of a voxel: how much light was measured at that spot. In the viewer it is relative (0 to 255), not calibrated. → chapters 4 and 11

### Journal

A file kept up to date during a long operation (import, conversion): it notes what is done, which makes it possible to resume after an interruption. → chapter 17

### Kabsch (algorithm)

A method that finds the rotation and shift that best align two clouds of points; used to stabilise a time-lapse. → chapter 8

### Lineage, mitosis, fusion

In cell tracking: the lineage is the sequence of a cell and its descendants; mitosis is a division into two cells; fusion joins two cells into one. → chapter 12

### LOD (level of detail) / level

A version of the volume at a given resolution; level 0 is the finest. → chapter 6

### LOD0

The finest level of detail: the native resolution of the acquisition. → chapter 6

### Lossless WebP

A compressed image format that restores the original values exactly; it stores the brick mosaics. → chapter 6

### Lossy, lossless compression

With loss, the file is smaller but the values are slightly modified; lossless, the original values are recovered exactly. Bricks are stored losslessly; format 5 adds, for display only, ±2, ±4 and lossy columns. → chapters 6 and 7

### Manifest

A file that describes the content of a dataset (levels, bricks, packs). → chapter 7

**Mask**

> A black-and-white image that says which pixels count (white) and which are discarded (black). → chapter 5

**Matrix (quality)**

> The "resolution × compression" table of the quality panel of a format-5 dataset: one row per level, one column per compression; each cell shows its size and graphics memory. → chapter 10

**Median filter**

> Replaces each value with the median of its neighbours: it removes isolated spots without blurring the edges. → chapter 5

**Migration**

> Updating an already published dataset to a more recent format, without going through the pipeline again. → chapter 17

**MIP (maximum intensity projection)**

> An image where each pixel takes the strongest value met along an axis. → chapters 7 and 12

**Morphological opening**

> Erosion followed by a dilation: it erases small isolated dots of a mask and keeps the large shapes. → chapter 5

**Mosaic**

> Several small images pasted into one large one; a brick is stored as a mosaic of its slices. → chapter 7

**Nonce**

> A random single-use code, drawn at each page load, which authorises the site's scripts. → chapter 19

**Opacity**

> A factor applied to a channel's intensity: 0 makes it invisible, 1 leaves it at full intensity. It is not the tissue's physical absorption, which exists (as a constant) only in Natural fluorescence mode. → chapter 11

**Optical section, focal plane**

> The image of a thin slice of the specimen, sharp at a given depth. The section thickness is the distance between two planes (3.0 µm in the demonstration dataset). → chapter 4

**Pack**

> A `.bin` file that groups many bricks to limit the number of downloads. → chapter 7

**Page table**

> A table that says where each brick is in the atlas, like a plan of the lockers. → chapter 10

**PBKDF2**

> A method that turns a password into a fingerprint that is deliberately slow to compute (600,000 rounds), to discourage mass guessing. → chapter 19

**Percentile**

> The value below which a given percentage of the measurements lie: the 99th percentile is exceeded by 1 % of the values. → chapter 5

**Photobleaching**

> Progressive loss of fluorescence of a sample lit for a long time. In a time series, the signal then drops without the structure changing. → chapter 5

**Pivot**

> The moment of the update when a supervisor replaces the old site folder with the new one, restarts the server and checks that it answers; otherwise it goes back. → chapter 18

**Pixel**

> The smallest element of a 2D image. → chapter 4

**Planes**

> The `planes/` folder of format 2: one file per z plane of the native level, to read an XY slice without loading 64 planes. → chapter 7 (and 17)

**Plugin**

> An optional tool added to the viewer (measurement, capture, filter...); it is installed from the signed catalogue. → chapters 12 and 15

### PNG

A lossless image format; the planes of format 2 are 512×512 PNG tiles. → chapter 7 (and 17)

### Poisson noise

Random variation of a weak signal, due to the limited number of photons received: the weaker the signal, the grainier it looks. Camera noise is added to it. → chapter 5

### PSF (point spread function)

The blurred shape that optics give to a point of light. Deconvolution tries to undo it; Lumen3D does not correct it. → chapter 20

### Pyramid

A set of versions of the volume at increasingly coarse resolutions, stacked like the floors of a pyramid. → chapter 6

### Quaternion

Four numbers that describe a 3D rotation without gimbal lock; they are used to store an embryo's orientation. → chapter 12

### Ray marching

For each pixel of the screen, a ray is followed through the volume, accumulating colour and opacity. → chapter 9

### Registration

Aligning the successive frames of a time-lapse to compensate for the embryo's movement (Kabsch algorithm). → chapter 8

### Rollback

Automatic return to the old version when the new one does not start. The Python server does it; PHP hosting does not. → chapters 18 and 19

### Sample (rendering)

A value read at one point of the volume during ray marching. A ray takes hundreds of them. → chapter 9

### Sandbox

A cage in which a third-party plugin runs: it sees neither the page nor your cookies, and can ask only for what it has explicitly been allowed. → chapter 15

### Saturation

A value that exceeds the maximum representable and is clipped. The pipeline deliberately saturates the brightest 0.1 % of voxels ( `sig_max` ). → chapter 5

### Shader

A small program run by the graphics card; the viewer's does the ray marching. → chapter 9

### Signature

See Ed25519. → chapter 19

### Specimen

The name the institution gives to what it observes (embryo, organ…); it changes throughout the interfaces. → chapter 16

### SRI (Subresource Integrity)

The expected fingerprint of a script or style sheet, written into the page: if the loaded file differs, the browser refuses it. → chapter 19

### Staging

A private area where imported files arrive before publication; it is never reachable by URL. → chapters 17 and 19

### Stereomicroscope

A low-magnification microscope that gives a colour image of the whole embryo; it produces the 2D-type photographs. → chapter 12

### Streaming

Progressive loading: what has arrived is displayed and completed continuously, without waiting for the whole file. → chapter 10

### Studio

The figure tool: cropping, annotations, scale bars and image export for publication. → chapter 12

### Super-block

A group of 4×4×4 neighbouring bricks that a pack stores together so that a region is read in one piece. → chapter 7

**Three.js**

> A JavaScript library that simplifies 3D display in the browser. → chapter 2

**Tier**

> A priority group of an import: the files that make a dataset openable (metadata, coarsest level) go first. → chapter 17

**Time-lapse**

> A series of acquisitions of the same specimen over time, each frame being a 3D volume. → chapter 4

**Token**

> A small code that proves a right: the anti-forgery (CSRF) token of a change, or a “token bucket” that limits the number of requests admitted per second. → chapter 19

**Transfer function**

> A rule that converts a volume value into colour and opacity. In Lumen3D, it is the chain of window, gamma, opacity and colour in the channel panel; chapter 11 describes it without using this term. → chapter 11

**Trilinear interpolation**

> Estimating a value between voxels by blending the eight neighbours, which gives a smooth image. → chapter 9

**Unit (of conversion)**

> A piece of work in a data conversion: a layer of 64 planes, of one channel, for one 512×512 tile. → chapter 17

**Voxel**

> A pixel in 3D: a small cube with an intensity value. Its real size (in µm) is the calibration. → chapter 4

**VRAM**

> Graphics-card memory. Its budget limits the atlas size the viewer can use. → chapter 10

**WebGL context (loss of)**

> A reset of the graphics card by the browser or the driver: the textures disappear. The viewer reloads the view with a reduced memory budget. → chapter 19

**WebGL2**

> A browser technology that gives access to the graphics card, notably to 3D textures. → chapter 9

**White label**

> The platform’s ability to take on the name, colours, texts and data types of the institution that hosts it, without writing code. → chapter 16

**Widget**

> A block of content on a page built in the editor (text, image, button…), placed in a column of a section. → chapter 16

**Windowing**

> Choosing the displayed min and max values: below, black; above, full intensity. → chapter 11

**Worker (Web Worker)**

> A background process of the browser: it decodes and computes without freezing the screen. → chapter 10

**X-gal**

> A stain that turns cells expressing a reporter gene blue; the 2D viewer can isolate this stain. → chapter 12

**Z-stack**

> The set of successive planes of a volume, stacked along the Z axis. → chapter 12